MIT 6.031《Software Construction》软件构造 学习笔记

> **课程**：MIT 6.031 — Software Construction（软件构造），Spring 2022 **课程主页**：https://web.mit.edu/6.031/www/sp22/ **授课形式**：每周一、三、五，各 90 分钟（MWF 11:00–12:30） **先修课程**：6.009（Fundamentals of Programming） **本笔记基础**：对课程公开站点 sp22（Spring 2022，主讲站点）与 sp21（Spring 2021，Java 版同一课程）的全部公开页面进行系统抓取后撰写，覆盖 Reading 1–29 全部章节。

关于本笔记的语言说明（请先读这段）

MIT 6.031 在 **Spring 2022（sp22）** 学期把授课语言从 **Java** 切换为 **TypeScript**。因此：

- 你指定的 sp22 站点上，所有代码示例都是 TypeScript。
- 但本笔记按你的要求，**全部代码示例提供 Java 版本**。
- 这些 Java 示例并非杜撰：它们来自 MIT 6.031 的 **sp21（Spring 2021）Java 版**同一门课、同一套讲义。sp22 与 sp21 的阅读材料在**讲解文字上几乎逐段对应**（例如 Reading 11 的正文两版高度一致，只有代码从 Java 换成了 TypeScript），所以 Java 示例与 sp22 的讲解是精确对应的，只是换了语言载体。
- 少数 sp22 特有的机制（`Promise` / `async` / `await`、结构化类型系统、`readonly`）在 Java 中没有一一对应的孪生章节，笔记中会明确标注为「Java 类比 / 补充说明」。
- 每篇笔记开头的引用块都会说明该讲的语言对应情况。

结论：你可以把本笔记当作「用 Java 讲的 6.031」来学——概念、术语、章节结构完全忠实于 sp22 原版，代码可以在 JDK 17+ 上编译运行。

课程概览

一、这门课到底在教什么

6.031 的目标可以用一句话概括：**学会写出好的代码**。而"好"被精确定义为三个可检验的属性，它们贯穿全课程、出现在每一次作业评分、每一次代码审查、每一道测验题中：

属性	英文	含义
免于 bug	Safe from bugs (SFB)	今天正确，并且在未知的未来依然正确
易于理解	Easy to understand (ETU)	与未来的程序员（包括未来的你自己）清晰沟通

属性	英文	含义
易于修改	Ready for change (RFC)	设计上能够适应变化，而不需要推倒重写

这三个目标（6.031 内部常缩写为 **SFB / ETU / RFC**，或"the big three"）是本课程最重要的单一思想。**每一个设计原则、每一种语言特性、每一条编码规范，最终都要回答同一个问题：它如何让代码更 SFB、更 ETU、或更 RFC?** 在本笔记中，你会反复看到这个三元组被用来评估一个做法是好是坏。

值得强调的是：**这三个目标经常互相冲突**。过度抽象会让代码难以理解；为了让代码容易修改而引入的间接层可能引入 bug；为了安全性做的防御性拷贝可能损害性能。软件构造的核心技能，正是在具体语境下做出有意识的权衡，而不是机械套用规则。

二、课程范围：一张知识地图

6.031 的知识体系可以分成四个层次，这也是本笔记 29 讲的推进顺序：

第一层：代码层面的正确性与表达力（Reading 1–9）

- **静态检查**（R1）：让编译器在运行前抓住错误。类型系统是最廉价的测试。
- **基础语言机制**（R2）：原始类型与引用类型、`final`、快照图（snapshot diagram）。
- **测试**（R3）：测试优先编程（test-first programming）、输入空间划分、边界值、代码覆盖率——**测试不是事后验证，而是设计活动**。
- **代码审查**（R4）：DRY、fail fast、魔法数字、命名、作用域。
- **版本控制**（R5）：Git 的对象图模型，为什么版本控制同时服务于三大目标。
- **规格说明**（R6–R7）：**本课程的第一个理论高峰**。规格说明是方法与其调用者之间的契约，由前置条件与后置条件构成；规格说明的强弱可以比较；应当使用声明式而非操作式规格；应当允许实现自由度（underdetermined）。
- **可变性与不可变性**（R8）：别名（aliasing）是可变对象危险的根源；防御性拷贝；不可变对象天然免于 bug。
- **避免调试**（R9）：把 bug 消灭在设计阶段——让 bug 不可能、让 bug 显而易见、让 bug 尽早失败。

第二层：数据抽象——本课程的核心（Reading 10–17）

- **抽象数据类型（ADT）**（R10）：用操作集合而非内部表示来定义类型；creators / producers / observers / mutators 四类操作的划分；表示独立性（representation independence）。
- **抽象函数与表示不变量**（R11）：**全课程最重要的两个概念**。AF 回答"内部表示代表什么抽象值"，RI 回答"内部表示的哪些状态是合法的"。二者必须写在代码注释里，由 `checkRep()` 检查。
- **接口、泛型与枚举**（R12）：接口与实现分离、子类型、Liskov 替换原则。
- **调试**（R13）：科学方法式调试——观察、假设、实验、重复。
- **递归**（R14）：递归分解、基础情形与递归步骤、辅助函数。
- **相等性**（R15）：等价关系三性质；引用相等 vs 值相等；`equals()` / `hashCode()` 的契约。
- **Map / Filter / Reduce**（R16）：一等函数、高阶函数、用函数抽象控制流。
- **递归数据类型**（R17）：不可变列表 `ImList`、递归类型的 AF/RI。

第三层：从数据到系统（Reading 18–27）

- **正则表达式与文法**（R18）、**解析器**（R19）：用文法描述输入语言，用解析器生成器把字符序列变成解析树，再转成抽象语法树。
- **回调与 GUI**（R20）：控制反转、事件循环、观察者模式——并在此引入并发的动机。
- **并发**（R21–R24）：**本课程的第二个理论高峰**。
- 共享内存 vs 消息传递两大模型（R21）；
- 竞态条件与交错（R21、R23）；
- 异步计算与 Promise（R22）；
- **互斥**：锁、`synchronized`、临界区、死锁与锁顺序（R23）；
- **消息传递**：阻塞队列、生产者–消费者模式（R24）。
- **网络**（R25）：客户端/服务器架构、套接字、字节流协议。
- **小语言**（R26–R27）：把文法、递归数据类型、解释器、访问者模式串成一条完整链路——这是全课程的"毕业设计级"综合应用。

第四层：工程与社会（Reading 28–29）

- **软件工程伦理**（R28）：四种道德透镜（后果主义、义务论、美德伦理、社会契约）、ACM 道德准则、伦理案例分析。
- **团队版本控制**（R29）：功能分支、拉取请求、合并与冲突解决、团队工作流。

三、课程哲学：为什么 6.031 这样设计

1. 软件构造的本质是"管理复杂性"。 课程反复强调：代码是写给人读的，只是顺便让机器执行。随着系统变大，唯一能限制复杂度的武器是**抽象**（abstraction）与**模块化**（modularity）。ADT、规格说明、不变量的共同目的，都是把"需要同时考虑的东西"变少。

2. 规格说明优先于实现。 6.031 的观点是：在写实现之前，你必须先能精确说出这个函数"做什么"（而不是"怎么做"）。规格说明让调用者与实现者解耦（decoupling）——调用者只依赖契约，实现者可以自由更换算法与数据结构。这就是 RFC 的技术基础。

3. 正确的做法是让错误不可能发生，而不是事后检查。 这条原则贯穿全课程：静态类型检查优于运行时断言，不变量优于临时检查，不可变对象优于防御性拷贝，编译期错误优于运行期错误。"Fail fast"（尽早失败）是它的一种表现：错误越早暴露，定位成本越低。

4. 测试是设计活动，不是收尾工作。 测试优先编程（test-first programming）要求先写测试再写实现。这不仅提高测试质量，更重要的是强迫你在写代码前想清楚规格说明和边界情况。

5. 迭代与反馈是学习的核心机制。 课程刻意压缩讲授时间、增加练习时间。每周 3 次课、每次课先来一个 3 分钟 nanoquiz；每个 Problem Set 走 **alpha → 同伴代码审查 → beta 迭代** 的流程；最后还有三人团队项目（Star Battle）。课程引用了 Wieman 等人的 *Course Transformation Guide* 和 Deslauriers 等人的物理课堂研究来论证这一设计。

6. 工程决策包含伦理维度。 最后一讲（R28）提醒：写出"技术上正确"的软件还不够，工程师要对软件的社会后果负责。这在三大目标之外增加了一个维度：**软件是否对使用者与社会负责任**。

四、6.031 的核心方法论：一句话总结每一层

层次	核心问题	主要武器
写单个函数	它"应该做什么"?	规格说明（前置/后置条件）、测试优先、静态检查
写一个数据类型	内部表示代表什么？何时合法?	AF、RI、表示独立性、`checkRep()`
写一个模块	模块之间如何解耦?	接口、封装、信息隐藏、设计模式
写一个并发程序	交错执行下如何保持正确?	不可变性、锁、消息传递、避免死锁
写一个系统	客户端与服务器如何通信?	协议、套接字、字节流、抽象边界
做一个工程决策	这样做对谁有好处、对谁有风险?	四大道德透镜、ACM 道德准则

五、评测与学习方式

项目	占比	内容
Quizzes	24%	Quiz 1（12%）+ Quiz 2（12%），闭卷，可带一张双面笔记纸
Problem sets	54%	PS0（6%）+ PS1–PS4（各 12%）
Code review	5%	同伴代码审查的参与度与评论质量
Project	7%	三人团队项目 ⭐⚔️（Star Battle）
Classwork	10%	课前阅读练习的努力 + nanoquiz 成绩 + 课堂练习

给自学者（如你）的建议：6.031 的知识密度很高，但它的学习曲线设计得很友好——每一讲都建立在前面几讲之上。建议的学习路径是：

1. 按顺序读本笔记的 Reading 1–15，**把 AF/RI（R11）和规格说明（R6–R7）读三遍**，这两处是整门课的地基。
2. 每读完一讲，把笔记里的**错误代码**先自己改对，再看正确代码。
3. Reading 16–19 是"数据 → 语言"的桥梁，务必动手写一个小的解析器。
4. Reading 21–24 是并发，**必须动手跑代码**：写一个 `counter++` 的竞态程序，亲眼看它出错，比读十遍文字都有效。
5. 最后用 Reading 26–27 的小语言把前面所有知识串起来。

文档结构导航

部分	内容
第一部分	课程资料获取与结构化整理记录：抓取范围、公开性说明、35 条公告、日程映射、作业/项目/测验记录、课程政策与评分标准、29 条结构化阅读记录

部分	内容
第二部分	分讲学习笔记 Reading 1–29（每讲统一七节结构）
第三部分	软件构造核心原则速查表（按类别汇总规则与代码模板）

第一部分：课程资料获取与结构化整理记录

1.1 数据获取范围与方法

本次整理对 MIT 6.031 的两个完全公开的课程站点进行了系统性抓取与文本抽取：

站点	角色	语言	抓取页面数
`https://web.mit.edu/6.031/www/sp22/`	**主站点**（用户指定学期，Spring 2022）	TypeScript	53
`https://web.mit.edu/6.031/www/sp21/`	**Java 孪生版本**（同一门课，Spring 2021）	Java	54

为什么要抓两个站点？ 用户要求提供 **Java** 代码示例，而 sp22 学期的 6.031 已把授课语言从 Java 切换为 TypeScript。所幸 sp21（Spring 2021）是同一门课、同一套讲义的 **Java 版本**， 其讲解文字与 sp22 几乎逐段一一对应（例如 Reading 11 "Abstraction Functions & Rep Invariants" 两版正文高度一致，只有代码从 Java 换成了 TypeScript）。 因此本笔记采用 **sp22 的章节结构、主题顺序与术语体系**，同时以 **sp21 的 Java 原文** 作为全部代码示例的权威来源，从而既满足"以 sp22 为准"的要求，又满足"提供可运行 Java 代码"的要求。

抓取到的原始材料保存在 `/home1/runguoli/learning/mit6031_data/`：

```
mit6031_data/
├── sp22/          # 53 个页面：29 篇 Reading + 5 个 Problem Set + Project + Quiz Archive
│                  # + General/FAQ/协作政策/代码审查政策/评分政策/公告存档
├── sp21/          # 54 个页面：30 篇 Reading（Java 版）+ 作业与政策页
├── readings_index.json   # 29 条结构化阅读记录（见 1.6）
├── build_index.py        # 结构化提取脚本
└── crawl.py              # 抓取与 HTML→文本 抽取脚本
```

抽取出的纯文本（`.txt`）保留了原文的正文、代码块、表格与列表结构，用于后续笔记撰写。

1.2 公开性说明（哪些能拿到、哪些受限）

资源	是否公开	说明
29 篇 Reading 正文（sp22）	✅ 完全公开	含完整讲解、代码示例、图片说明、在线练习题
30 篇 Reading 正文（sp21，Java）	✅ 完全公开	Java 代码示例的权威来源
Course Schedule / 阅读材料清单	✅ 公开	以主页面 "Readings" 索引 + 各 Reading 页形式提供
Problem Set 0–4 Handout（描述文档）	✅ 公开	含作业目标、截止日期、设计要求、评分说明
Project Handout（Star Battle 团队项目）	✅ 公开	含迭代计划、里程碑日期、团队要求
Quiz Archive（历年测验与解答 PDF）	✅ 公开	Fall 2021 / Spring 2021 等学期
General Info & FAQ、协作政策、代码审查政策、评分细则	✅ 公开	见 1.7
Announcements（35 条公告）	✅ 公开	见 1.3
本学期的 Quiz 1 / Quiz 2 题目与解答	❌ 受限	托管在 `quiz.mit.edu`，仅 MIT 学生可访问；仅题目名称公开
Problem Set 起始代码仓库（starter code）	❌ 受限	通过 `github.mit.edu` 发放，仅选课学生可访问
Didit / Omnivore / Caesar / Piazza / Gradescope	❌ 受限	自动评分、成绩、同伴代码审查、讨论区，均需 MIT 账号
课堂 nanoquiz、课堂练习、clicker 题目	❌ 受限	仅课堂内进行，不公开
TypeScript Tutor 在线练习	⚠️ 部分	练习正文与题目出现在 Reading 页中（公开），交互式作答需登录

> 说明：受限资源的处理方式遵照任务要求——**仅记录名称并注明"未公开"**，不尝试绕过任何访问限制。

1.3 公告（Announcements）记录

sp22 学期共 **35 条公告**，按时间倒序排列如下（完整正文已保存于 `mit6031_data/sp22/announcements.txt`）：

日期	公告主题
Thu May 19	Project, Quiz 2, and final grades
Fri May 13	Quiz 2 today
Fri May 6	Project, reflection, and last class
Tue May 3	Quiz 2 during final exam period

日期	公告主题
Fri Apr 29	Problem Set 4 grades
Wed Apr 27	Problem Set 4 reflection
Sun Apr 24	Project groups and handout
Fri Apr 22	Problem Set 4 alpha reports
Wed Apr 20	Problem Set 4 code review open, due Friday 11am
Fri Apr 15	Project group signup
Fri Apr 15	Problem Set 3 grades
Mon Apr 11	Problem Set 4
Fri Apr 8	Problem Set 3 alpha reports
Wed Apr 6	Problem Set 3 code review open, due Friday 11am
Mon Mar 28	Quiz 1 grades
Fri Mar 18	Problem Set 2 grades
Fri Mar 18	Problem Set 3
Thu Mar 17	Quiz 1 tomorrow
Tue Mar 15	Problem Set 2 reflection
Fri Mar 11	Problem Set 2 alpha reports
Wed Mar 9	Problem Set 2 code review open, due Friday 11am
Tue Mar 8	Quiz 1 next week
Fri Mar 4	Problem Set 1 grades
Mon Feb 28	Problem Set 2
Fri Feb 25	Problem Set 1 alpha reports
Thu Feb 24	no class on Friday because of snow
Wed Feb 23	Problem Set 1 code review open, due Friday 11am
Fri Feb 18	Problem Set 0 grades
Mon Feb 14	Problem Set 1
Fri Feb 11	Problem Set 0 alpha reports
Wed Feb 9	Problem Set 0 code review open, due Friday 11am
Mon Jan 31	Problem Set 0 and Getting Started

日期	公告主题
Mon Jan 31	Reading exercises, nanoquizzes, and other classwork
Sun Jan 30	Getting started in 6.031
Wed Jan 19	Welcome to 6.031!

公告中体现的关键学期节点：

- **Wed Jan 19**：欢迎来到 6.031（课程开始）
- **Sun Jan 30 / Mon Jan 31**：Getting started、Problem Set 0 发布、阅读练习与 nanoquiz 机制说明
- **Wed Feb 9 / Fri Feb 11**：PS0 代码审查开放（周五 11am 截止）→ PS0 alpha 报告发布
- **Mon Feb 14**：Problem Set 1 发布
- **Thu Feb 24**：因暴雪周五停课（说明课程会因天气调整）
- **Thu Mar 17 / Fri Mar 18**：**Quiz 1**
- **Mon Mar 28**：Quiz 1 成绩发布
- **Fri Apr 15**：项目分组报名（Project group signup）
- **Sun Apr 24**：项目分组与项目 Handout 发布
- **Mon Apr 25**：团队契约（team contract）截止
- **Fri May 6**：**最后一节课** + 项目截止（22:00）+ 个人反思截止（22:00）
- **Fri May 13**：**Quiz 2**（1:35–2:25pm，期末考试时段）
- **Thu May 19**：项目成绩、Quiz 2 成绩、最终成绩归档

> 公告中还明确了考试纪律：Quiz 闭卷、闭笔记，允许携带 **一张 8.5×11 英寸双面手写/打印笔记纸**；Quiz 2 覆盖 Reading 1–29，重点为 Reading 17–29。

1.4 课程日程与阅读材料映射

6.031 每周三次课（周一/周三/周五，各 90 分钟）。大部分课程要求**课前完成阅读 + 在线练习（课前晚 10pm 截止）**，每次课以 **3 分钟 nanoquiz** 开始。 下表给出 sp22 的 29 篇阅读材料、主题、对应作业，以及本笔记使用的 Java 源章节：

Reading	主题（sp22 原版）	本笔记 Java 代码来源	相关作业	公开性
1	Static Checking	—	Problem Set 0	✅ 公开
2	Basic TypeScript	sp21 Reading 02: Basic Java	Problem Set 0	✅ 公开

Reading	主题（sp22 原版）	本笔记 Java 代码来源	相关作业	公开性
3	Testing	—	Problem Set 1	✅ 公开
4	Code Review	—	—	✅ 公开
5	Version Control	—	—	✅ 公开
6	Specifications	—	Problem Set 1	✅ 公开
7	Designing Specifications	—	Problem Set 1	✅ 公开
8	Mutability & Immutability	—	—	✅ 公开
9	Avoiding Debugging	—	Problem Set 2	✅ 公开
10	Abstract Data Types	—	Problem Set 2	✅ 公开
11	Abstraction Functions & Rep Invariants	—	Problem Set 2	✅ 公开
12	Defining ADTs with Interfaces, Generics, Enums, and Functions	—	Problem Set 2	✅ 公开
13	Debugging	—	—	✅ 公开
14	Recursion	—	—	✅ 公开
15	Equality	—	Problem Set 3	✅ 公开
16	Map, Filter, Reduce	—	Problem Set 3	✅ 公开
17	Recursive Data Types	—	Problem Set 3	✅ 公开
18	Regular Expressions & Grammars	—	Problem Set 3	✅ 公开

Reading	主题（sp22 原版）	本笔记 Java 代码来源	相关作业	公开性
19	Parsers	—	Problem Set 3	✅ 公开
20	Callbacks and Graphical User Interfaces	—	—	✅ 公开
21	Concurrency	—	Problem Set 4	✅ 公开
22	Promises	无直接对应（用 `CompletableFuture` 类比）	Problem Set 4	✅ 公开
23	Mutual Exclusion	—	Problem Set 4	✅ 公开
24	Message-Passing	—	Problem Set 4	✅ 公开
25	Networking	—	Problem Set 4	✅ 公开
26	Little Languages I	—	—	✅ 公开
27	Little Languages II	—	—	✅ 公开
28	Ethical Software Engineering	—	—	✅ 公开
29	Team Version Control	—	Project ⭐⚔️ (Star Battle)	✅ 公开

关于 Quiz 覆盖范围：Quiz 1 覆盖 Reading 1–16；Quiz 2 覆盖 Reading 1–29（重点 17–29）。

1.5 作业、项目与测验记录

Problem Sets（共 5 个，占总成绩 54%）

作业	主题	训练目标（摘自 Handout）	Alpha	Code review	Beta
PS0	Turtle Graphics（海龟绘图）	课程入门、TypeScript/Git/VS Code 工具链、快照图、静态检查	Feb 7	Feb 11	Feb 14
PS1	Flashcards（抽认卡）	**测试优先编程与规格说明**：给定规格写单测，再实现；并强化一个规格	Feb 22	Feb 25	Feb 28

作业	主题	训练目标（摘自 Handout）	Alpha	Code review	Beta
PS2	Cityscape（城市天际线）	**设计与实现可变 ADT**：规格由课程给定，练习 AF/RI 与 `checkRep`	Mar 7	Mar 11	Mar 14
PS3	Memely（表情包）	**解析器、递归数据类型、不可变类型的相等性**；规格可自行加强	Apr 4	Apr 8	Apr 11
PS4	Memory Scramble（记忆翻牌）	**并发共享可变数据类型 + 客户端/服务器系统**（HTTP 协议）	Apr 19	Apr 22	Apr 25

每个 PS 采用 **alpha → 同伴代码审查 → beta 迭代** 的流程：alpha 提交后接受评分与同学代码审查，beta 需根据审查意见与失败测试修订代码。总评成绩约各占一半。

Project（占总成绩 7%）：名称为 ⭐⚔ **（Star Battle）**，三人一组，要求设计、实现、测试、文档全面参与，团队统一评分。 里程碑：团队契约（Apr 25）→ Iteration #0（Apr 28）→ Iteration #1（May 3）→ 项目截止与个人反思（May 6, 10pm）。

Quizzes（占总成绩 24%）：Quiz 1（12%）与 Quiz 2（12%，期末考试时段）。春季学期 Quiz 1 在 Mar 17 左右，Quiz 2 在 May 13。 Quiz 通过 `quiz.mit.edu` 在线进行，闭卷，允许一张双面笔记纸。**Quiz Archive 公开**（Fall 2021、Spring 2021 的 Quiz 1/Quiz 2 及解答 PDF，以及 Fall 2020 及更早的 Java 版测验）。

1.6 结构化阅读记录（readings_index.json）

按照任务要求，每篇阅读材料都建立了一条结构化记录。完整 JSON 保存于 `/home1/runguoli/learning/mit6031_data/readings_index.json`，字段格式如下：


```
{
  "reading_number": 11,
  "topic": "Abstraction Functions & Rep Invariants",
  "sp22_url": "https://web.mit.edu/6.031/www/sp22/classes/11-abstraction-functions-rep-inva
  "key_concepts_raw": ["invariants", "representation exposure", "abstraction functions", "r
  "related_problem_set": "Problem Set 2",
  "primary_language_sp22": "TypeScript",
  "java_source_url": "https://web.mit.edu/6.031/www/sp21/classes/11-abstraction-functions-r
  "available_public_info": "阅读材料页面完全公开，含完整讲解文本、代码示例、练习题与解答。"
}
```


全部 29 条记录的关键概念（`key_concepts_raw`，从各 Reading 的 "Objectives" 小节自动提取）：

Reading	关键概念（原文提取）
1	static typing；the big three properties of good software；Hailstone sequence
2	Learn basic JavaScript and TypeScript syntax and semantics；Transition from writing Python to writing TypeScript；Getting started with TypeScript

Reading	关键概念（原文提取）
3	understand the value of testing, and know the process of test-first programming;；be able to judge a test suite for correctness, thoroughness, and size;；be able to design a test suite for a function by partitioning its input space and choosing good test cases;；be able to judge a test suite by measuring its code coverage; and；Validation
4	code review: reading and discussing code written by somebody else；Code review
5	Know what version control is and why we use it；Understand how Git stores version history as a graph；Practice reading, creating, and using version history；Introduction
6	Understand preconditions and postconditions in function specifications, and be able to write correct specifications；Be able to write tests against a specification；Understand how to use exceptions；Introduction
7	Understand underdetermined specs, and be able to identify and assess specs that are not deterministic；Understand declarative vs. operational specs, and be able to write declarative specs；Understand strength of preconditions, postconditions, and specs, and be able to compare spec strength；Be able to write coherent, useful specifications of appropriate strength；Introduction
8	Understand mutability and mutable objects；Identify aliasing and understand the dangers of mutability；Use immutability to improve correctness, clarity, & changeability；Creating and using objects；reading exercises；Classes and objects
9	First defense: make bugs impossible；The best defense against bugs is to make them impossible by design.
10	Abstract data types；Representation independence；Introduction
11	invariants；representation exposure；abstraction functions；representation invariants
12	interfaces: separating the interface of an ADT from its implementation;；generic types: defining a family of ADTs using generic type parameters;；enumerations: defining an ADT with a small finite set of values;；global functions operating on an opaque type: rare in TypeScript but common in non-object-oriented languages.；define ADTs using classes, interfaces, generics, and enumerations；determine whether one type is a subtype of another
13	The topic of today's class is systematic debugging.
14	be able to decompose a recursive problem into recursive steps and base cases；know when and how to use helper functions in recursion；understand the advantages and disadvantages of recursion vs. iteration；Recursion
15	Understand the properties of an equivalence relation.；Understand equality for immutable types defined in terms of the abstraction function and in terms of observation.；Differentiate between reference equality and object equality.；Differentiate between observational and behavioral equality for mutable types.；Introduction
16	（本讲 Objectives 为段落式描述，见正文）

Reading	关键概念（原文提取）
17	Understand recursive data types；Read and write data type definitions；Understand and implement functions over recursive data types；Understand immutable lists and know the standard operations on immutable lists；Know and follow a recipe for writing programs with ADTs；Introduction
18	Understand the ideas of grammar productions and regular expression operators；Be able to read a grammar or regular expression and determine whether it matches a sequence of characters；Introduction；grammars, with productions, nonterminals, terminals, and operators；regular expressions
19	Be able to use a grammar in combination with a parser generator, to parse a character sequence into a parse tree；Be able to convert a parse tree into a useful data type；Parser generators
20	（本讲 Objectives 为段落式描述，见正文）
21	The message passing and shared memory models of concurrency；Concurrent processes and threads, and time slicing；The danger of race conditions；Concurrency；Multiple computers in a network；Multiple applications running on one computer
22	This reading discusses concurrent computation using promises.；Then we will dig below the covers to understand more about what is really happening with Promise, await, and async.；Promises
23	Library example；`；// Book represents a physical copy of a book.；// Equality operation is ===, safe for use in sets and maps.；class Book { ... }；// User represents a human patron of the library. | | 24 | Two models for concurrency；In our introduction to concurrency, we saw two models for concurrent programming: shared memory and message passing.；[图: shared memory] | | 25 | （本讲 Objectives 为段落式描述，见正文） | | 26 | （本讲 Objectives 为段落式描述，见正文） | | 27 | Visitors are a common feature of little languages implemented as recursive data types.；Functions on recursive types；declare the operation as an instance method in the interface that defines the data type, and；implement the operation in each concrete variant.；`；Formula = Variable(name:string)
28	Explain the ethical principles to consider when you design, build, and maintain software.；Apply four different moral lenses as you examine the ethical ramifications of a particular system.；SFB, ETU, and RFC
29	Review Git basics and the commit graph；Practice multi-user Git scenarios；Git workflow

1.7 课程政策与评分标准记录

评分构成（摘自 General Information 的 Grading 小节）

项目	占比	细则
Quizzes	**24%**	Quiz 1 占 12%，Quiz 2 占 12%
Problem sets	**54%**	PS0 占 6%，PS1–PS4 各占 12%
Code review	**5%**	由是否持续参与、评论是否实质有用评判

项目	占比	细则
Project	**7%**	三人团队项目，团队统一评分
Classwork	**10%**	课前阅读练习的努力程度 + nanoquiz 成绩 + 课堂练习努力程度

字母等级默认分界：**≥90 为 A，≥80 为 B，≥70 为 C**；分界线只可能下调，绝不上升，且无名额限制、不按比例给分。

课堂与考勤政策

- 每周三次 90 分钟课程，必须出席并积极参与；需自带笔记本电脑。
- 每次课以 **3 分钟闭卷 nanoquiz** 开始，考查本次阅读与近期课堂内容。
- **自动丢弃最低 5 次课堂成绩**（相当于自动豁免最多 5 次课），也可按流程补课获得部分学分。
- 课程刻意减少讲授时间、增加练习时间，因为「练习与反馈是学习的关键」（引用 Wieman 等、Deslauriers 等的研究）。

协作与公开分享政策（Collaboration and Sharing）

行为	是否允许	说明
向本学期 6.031 教职员求助	✅ 鼓励	一对一、Piazza、课程网站材料均可
与当前 6.031 同学协作	⚠️ 有限	仅鼓励高层次讨论；**禁止代码级交流，包括伪代码**
通过 MIT 项目（如 HKN）的辅导员求助	⚠️ 有限	适用与"同学协作"相同的规则
向非 6.031 教职员/学生求助	❌ 禁止	需先获教师许可
查阅非自己编写的代码	⚠️ 有限	6.031 语境下产出的代码禁止查阅（本学期教职员发放的除外）；外部代码必须正确署名
分享自己写的代码	❌ 禁止	任何形式均禁止，包括公开 GitHub/GitLab 仓库

> **核心精神**：讨论思路可以，逐步算法说明（含伪代码）不可以；帮助别人时，自己的代码不应可见（"帮助同学时先关掉自己的代码"）。 代码审查中会看到同学的解答，可以受启发，但不能抄袭。

代码审查政策（Code Reviewing）：课程鼓励 "review-before-commit"（Google 源码仓库要求任何一行代码都必须被另一位工程师阅读、反馈并签署）。 审查的目标是 **safe from bugs / easy to understand / ready for change**。审查清单包括： bug 或潜在 bug、重复代码（DRY）、代码与规格不一致、off-by-one、全局变量与过大的变量作用域、魔法数字、可以更防御性或更快失败的代码、 不清晰的代码、糟糕的命名、不一致的缩进、可简化的控制流、单行/单方法塞入过多内容、晦涩代码缺注释、与代码冗余的琐碎注

释、一变量多用、 语言误用（如 `==` vs `===`、`var` vs `let`、`for...in` vs `for...of`），以及对课程已讲授设计概念（ADT、规格说明、不变量等）的误用或未使用。

求助渠道政策：一般问题走 Piazza（**不要把代码贴到 Piazza**）；成绩问题不走 Piazza，也不直接联系助教，而是通过专门流程； TS Tutor 的 bug 用练习内的 "Report a Problem" 上报；提问时须给出可点击的精确 URL（课程网站每个段落都可链接）。

第二部分：分讲学习笔记（Reading 1–29）

> 以下按阅读材料编号顺序呈现全部 29 讲。每讲严格遵循统一结构： **概述 → 核心概念与设计原则详解 → 代码示例与对比分析 → 与其他设计原则的关联 → 关键要点 → 常见陷阱与注意事项 → 思考题（带答案）**。
>
> 每讲开头的说明块会指出该讲在 sp22（TypeScript 原版）与 sp21（Java 版）之间的语言对应关系。

Reading 1: 静态检查（Static Checking）

> 说明：本讲 sp22 原版使用 TypeScript，本笔记按用户要求提供 Java 代码示例；类型/API 与 sp21（6.031 Java 版）原文保持一致。

概述

本讲包含两个主题：**静态类型（static typing）** 与 **好软件的三大性质（the big three properties of good software）**，并以**冰雹序列（hailstone sequence）** 作为贯穿全讲的运行示例。核心问题是：怎样让错误尽可能早地、尽可能自动地被发现？答案是把手动检查变成**编译期（compile time）** 的机器检查，也就是静态检查。这一思想直接服务于课程的第一个目标「Safe from bugs」（在程序运行之前就拦住一整类错误），同时通过把类型、`final`、前置条件等假设显式写进代码而服务于「Easy to understand」，并通过让编译器指出所有需要同步修改的位置而服务于「Ready for change」。

核心概念与设计原则详解

冰雹序列（Hailstone Sequence） – **定义与目的**：从正整数 n 出发，若 n 为偶数则下一项为 n/2，若 n 为奇数则下一项为 3n+1，序列在到达 1 时结束。例：3, 10, 5, 16, 8, 4, 2, 1。它是本讲的「小白鼠」：一个足够简单、可以完整读懂的循环，却同时暴露出类型声明、整数运算、数组越界、参数前置条件等一连串软件构造问题。 – **直观解释（"它是什么？"）**：因为奇数规则 3n+1 会把数「弹上去」，序列会先上下跳动，最后落回地面——就像云中的冰雹反复上下翻滚，直到足够重才落地。数学上至今没有人能证明**所有**起点最终都会落到 1（Collatz 猜想仍未解决），这个「连数学家都证明不了它一定停机」的性质，正好说明为什么我们不能靠推理替代测试（见 Reading 03）。 – **关键规则与最佳实践**： – 用整数运算写循环时，先想清楚「这个表达式是在 int 域还是 double 域里求值」，`n / 2` 与 `n / 2.0` 完全不同。 – 序列长度不可预测（n=27 需要 112 项才能降到 1），因此绝不能用固定长度的容器去装它。 – 规格说明里必须写清前置条件，冰雹序列要求 `n >`

`0`；`n = 0` 或 `n < 0` 会造成无限循环（0 永远是偶数，−5 → −14 → −7 → −20 → −10 → −5 形成死循环）。 – 循环体内修改参数会让规格说明中「n 是起始值」的假设失效，应使用独立的循环变量。

类型（Type） – **定义与目的**：**类型是一组值的集合，以及可以在这些值上执行的操作**。类型的作用是让编译器知道哪些操作是合法的，从而把「把操作施加到错误类型的参数上」这一类 bug 彻底挡在编译期之外。它服务的首要目标是 Safe from bugs，其次是 Easy to understand（类型就是写在代码里的假设）。 – **直观解释（"它是什么？"）**：把类型想成「容器上的标签」。标着 `int` 的盒子里只装 32 位整数，标着 `String` 的盒子里装字符序列。你在 `String` 盒子上按「乘法」按钮时，编译器会在你按下运行按钮之前就告诉你：「这个按钮在这个盒子上不存在」。 – **关键规则与最佳实践**： – Java 的**原始类型（primitive types）**：`int`（约 $\pm 2^{31}$）、`long`（约 $\pm 2^{63}$）、`boolean`、`double`、`char`，一律小写。 – Java 的**对象类型（object types）**：`String`（字符序列）、`BigInteger`（任意精度整数）、以及你自定义的类，按约定首字母大写。 – 每个原始类型都有对应的**包装类型（wrapper type）**：`int` / `Integer`、`long` / `Long`、`double` / `Double`，泛型参数必须用包装类型。 – 变量声明要「能写就写」：把类型写下来既是给编译器的承诺，也是给未来读者的文档。

操作的三种书写形式与重载（Overloading） – **定义与目的**：操作（operation）是「吃进输入、吐出输出、有时还会改变自身值」的函数，无论语法怎么变，我们都把它当函数看。识别这三种形式有助于读懂 API 文档，也是理解重载的前提。 – **直观解释（"它是什么？"）**：同一个「加法」概念，可以写成中缀运算符 `a + b`、对象方法 `bigint1.add(bigint2)`、或者类里的静态函数 `Math.sin(theta)`。它们在语法上长得完全不同，在语义上都是「函数」。 – **关键规则与最佳实践**： – 运算符形式：`a + b` 调用的是 `+ : int × int → int`，箭头前是输入类型，箭头后是输出类型。 – 对象方法形式：`bigint1.add(bigint2)` 调用 `add : BigInteger × BigInteger → BigInteger`，接收者对象是隐含的第一个参数（Java 中叫 `this`，Python 中叫 `self`）。 – 静态函数形式：`Math.sin(theta)` 调用 `sin : double → double`；此时 `Math` **不是对象**，而是包含该函数的类。 – 属性/实例变量形式：数组用 `a.length`（无括号，因为是实例变量），字符串用 `str.length()`（有括号，因为是方法调用），二者语法不同，别记混。 – **重载（overloading）**：同一个名字对应不同参数类型的多个函数。Java 中 `+ - * /` 对数值原始类型重载，方法的参数列表不同也可以重载。重载让代码更易读（`+` 对字符串就是拼接），但也是 `==` 在对象上语义突变的根源（见 Reading 02）。

静态类型（Static Typing） – **定义与目的**：静态类型语言在**程序运行之前**就知道所有变量的类型，因而编译器能推断出所有表达式的类型。它消灭的是一大类 bug：**把操作施加到错误类型的参数上**。这直接对应 Safe from bugs。 – **直观解释（"它是什么？"）**：想象一位在你身边盯着你打字的助教。你刚写完 `"5" * "6"`，他就说：「字符串不能相乘。」你还没运行程序，错误就已经被拦住了。Eclipse / VS Code 在你打字时就在做这件事。 – **关键规则与最佳实践**： – 静态检查能抓：语法错误（多余的标点、错字）、拼错的名字（`Math.sine(2)`）、参数个数错误（`Math.sin(30, 20)`）、参数类型错误（`Math.sin("30")`）、返回值类型错误（声明返回 `int` 却 `return "30";`）。 – 静态检查**不能**抓：与**具体值**有关的错误，比如除零、下标越界、非法转换（`Integer.valueOf("hello")`）。因为编译器只知道「y 是 int」，不知道「y 恰好是 0」。 – 静态类型保证「变量一定持有该类型中的某个值」，但不保证是哪一个值；值相关的检查属于动态检查。 – 类型越精确（如用 `List<Integer>` 而不是裸 `List`），编译器能替你抓的错误就越多。

静态检查、动态检查、不检查（Static Checking, Dynamic Checking, No Checking） – **定义与目的**：语言可以提供的三种自动检查档次——静态检查（运行前发现）、动态检查（运行时发现）、不检查（语言完全不管，你自己盯着，否则就得到错误答案）。三者优先级为：静态优于动态，动态优于不检查。这三档构成了本讲最重要的判断框架。 – **直观解释（"它是什么？"）**：把它想成机场安检的三道关：静态检查是登机前的行李检查（还没上飞机就被拦下），动态检查是飞行中的警报（出问题立刻报警并迫降），不检查是根本没有报警器（飞机安静地飞向错误的方向，直到撞山）。 – **关键规则与最佳实践**： – 写代码时反复问自己：「这个错误会在编译期被抓住、运行期被抓住、还是完全抓不住？」抓不住的错误要有额外的防御（断言、显式检查、更安全的类型）。 – 静态检查面向**类型**，动态检查面向**具体值**；这是一条极为有用的心智分界线。 – Java 做了不少动态检查（数组越界抛 `ArrayIndexOutOfBoundsException`、除零抛 `ArithmeticException`、`null` 上调用方法抛 `NullPointerException`），这比 C/C++ 安全得多——C/C++ 的数组越界是**不检查**的，正是缓冲区溢出漏洞与网络蠕虫的温床。 – TypeScript 在运行期的行为完全由 JavaScript 提供，而 JavaScript 在很多场合选择「不检查」：越界返回 `undefined`，除零返回 `Infinity`。错误值会一路传播，直到离原始错误很远的地方才爆发——这让调试变难。

原始类型不是真正的数（Primitive Types Are Not True Numbers） – **定义与目的**：Java 的数值原始类型有一些不符合直觉的角落，导致某些**本该**被动态检查的错误变成了「静默的错误答案」。理解这些角落是避免「数值 bug」的关键。 – **直观解释（"它是什么？"）**：`int` 是一条首尾相接的环形跑道，跑到最右端会突然跳回最左端；`double` 是一个只有 53 位有效二进制数字的标尺，太长的小数只能被截断。 – **关键规则与最佳实践**： – **整数除法**：`5/2` 得到 `2` 而不是 `2.5`，小数部分被直接丢掉（截断），不报错。 – **整数溢出（overflow）**：`int` / `long` 是有限集合，超出范围会**静默回绕**，得到一个合法范围内的错误数字。 – **浮点特殊值**：`double` 有 `NaN`（Not a Number）、`POSITIVE_INFINITY`、`NEGATIVE_INFINITY`。除零或对负数开平方不会抛异常，而是得到这些特殊值，继续算下去会得到错误的最终结果。 – **精度限制**：`double` 只能精确表示 2^{53} 以内的整数，超过之后相邻可表示整数之间的间隔大于 1，`x + 1` 可能等于 `x`（这正是 sp22 中 TypeScript 的 `number` 陷阱）。 – Java 补充说明：`Math.abs(Integer.MIN_VALUE)` 返回的仍是 `Integer.MIN_VALUE`（一个负数！），因为 -2^{31} 的相反数超出了 `int` 的范围。

数组与集合（Arrays and Collections） – **定义与目的**：把冰雹序列存起来而不是打印出来，需要数据结构。Java 提供**定长数组**和**可变长 List** 两类「列表状」容器；选择哪一个直接影响安全性。 – **直观解释（"它是什么？"）**：数组是一排固定编号的储物柜，造好之后柜子数量就定死了；`List` 是一根可以随时加长缩短的伸缩杆。 – **关键规则与最佳实践**： – 数组声明与构造：`int[] a = new int[100];`，一旦创建长度不可变；操作有下标 `a[2]`、赋值 `a[2] = 0`、长度 `a.length`。 – List 声明与构造：`List<Integer> list = new ArrayList<Integer>();`，操作有 `list.get(2)`、`list.set(2, 0)`、`list.size()`、`list.add(n)`。 – **为什么左边写 `List` 右边写 `ArrayList`**：`List` 是**接口（interface）**，只规定必须提供哪些操作；`ArrayList` 是**具体类**，提供实现。声明变量与返回类型时优先用接口，代码更通用、更灵活。 – **为什么不能写 `List<int>`**：泛型参数必须是对象类型，所以要用包装类型 `Integer`。Java 会在 `int` 与 `Integer` 之间自动装箱/拆箱，所以 `Integer i = 5;` 合法。 – 定长数组 + 魔法数字是经典的 bug 来源：冰雹序列长度不可预测，`new int[100]` 对 n=27（需要 112 项）就会越界。这种错误叫**缓冲区溢出（buffer overflow）**，在 C/C++ 中是安全灾难，在 Java 中会被动态检查为异常。

迭代（Iterating） – **定义与目的**：for-each 循环让「遍历容器中的每个元素」这一最常见任务变得简洁、不易出错。 – **直观解释（"它是什么？"）**：`for (int x : list)` 读作「对 list 中的每一个 x」，你不需要自己维护下标计数器，也就没有机会把 `<=` 写成 `<`、把起始值写成 1。 – **关键规则与最佳实践**： – Java 写法：`for (int x : list) { max = Math.max(x, max); }`，数组和 `List` 通用。 – 循环变量优先用原始类型 `int` 而不是 `Integer`，因为原始类型的 `==` 更简单、更不易出错（见 Reading 02）。 – 只有在**确实需要下标**时才写 `for (int i = 0; i < list.size(); i++)`，这种写法冗长且藏 bug 的地方更多。 – 绝不要在迭代过程中修改正在遍历的集合（增、删、替换），这会破坏迭代器，甚至让程序崩溃。

方法与规格说明（Methods and Specifications） – **定义与目的**：Java 中语句必须放在方法里，方法必须放在类里。方法上方的 `/** ... */` 注释是**规格说明（specification）**，它描述操作的输入与输出，是模块与客户之间的契约。 – **直观解释（"它是什么？"）**：规格说明是「说明书」，不是「实现日记」。说明书只写调用者需要知道的事，不写内部怎么实现。 – **关键规则与最佳实践**： – `public` 表示任何代码都可以引用；`private` 等访问修饰符用来获得更强的安全性，并保证不可变类型的不可变性（Reading 08 会展开）。 – `static` 表示这个方法不带隐含的 `this` 参数，用类名调用：`Hailstone.hailstoneSequence(83)`。 – 规格说明要**简洁、清晰、精确**，只写类型声明没说的事：不必写「返回一个整数列表」（`List<Integer>` 已经说了），但必须写「序列以 n 开始、以 1 结束」以及「要求 n > 0」。 – 有前置条件（precondition）时一定要写出来，并用防御性检查兜底；注释是给人看的，编译器不看注释。

改变值 vs. 重新赋值变量（Mutating Values vs. Reassigning Variables） – **定义与目的**：这是两个必须严格区分的概念：改变变量指向哪里（重新赋值）与改变值本身的内容（修改）。**不可变性（immutability）** —— 有意禁止某些东西在运行时改变 —— 是本课程的核心设计原则。 – **直观解释（"它是什么？"）**：在快照图（snapshot diagram，Reading 02 详述）里，变量是一支箭头。**赋值**是让箭头指向别处；**修改**是让箭头指向的那个气泡内部发生变化。 – **关键规则与最佳实践**： – **不可变类型（immutable type）**：值一旦创建就不能改变。`String` 在 Java 和 Python 中都是不可变的。 – **不可重新赋值的引用（unreassignable reference）**：用 `final` 声明，变量只被赋值一次。若编译器无法确信它只被赋值一次，就报编译错误——所以 `final` 提供的是**静态检查**。 – 好习惯是对**方法参数**和尽可能多的局部变量加 `final`：这些声明既是文档，又被编译器静态检查。 – 注意两个容易混淆的组合：`final` 引用可以指向**可变对象**（`final StringBuilder sb = new StringBuilder("a"); sb.append("b");` 合法）；非 `final` 引用也可以指向**不可变对象**（`String s = "a"; s = "ab";` 合法，只是箭头改指了）。

记录假设（Documenting Assumptions） – **定义与目的**：写下一个变量的类型，就是在记录一条假设：「这个变量永远指向一个整数」。Java 在编译期检查这条假设。写下 `final` 是另一条假设：「这个变量初始化后不会再被赋值」，Java 同样静态检查。 – **直观解释（"它是什么？"）**：程序里满是假设，而人的记忆不可靠。假设不写下来，三个月后的你（以及接手你代码的同事）只能靠猜。 – **关键规则与最佳实践**： – 能自动检查的假设优先用语言机制表达（类型、`final`），因为机器检查永不疲倦。 – 不能自动检查的假设（例如 `n` 必须为正）必须写进规格说明，必要时用运行期检查兜底。 – 程序要同时服务两个目标：**与计算机沟通**（先让

编译器相信程序语法与类型正确，再让逻辑在运行时给出正确结果）和**与人沟通**（让程序易于理解，以便将来有人能修好它、改进它、改造它）。

黑客式编程 vs. 工程化编程（Hacking vs. Engineering） – **定义与目的**：本讲写的冰雹代码其实相当「黑」。区分「黑」与「工程」的判断标准，是乐观主义与悲观主义的分野。 – **直观解释（"它是什么？"）**：黑客相信「一次就能写对、bug 一眼就能找到」；工程师相信「一定会错，所以要提前布防」。 – **关键规则与最佳实践**： – 坏：写一大堆代码才第一次测试；把所有细节记在脑子里；假设 bug 不存在或很容易修。 – 好：一次写一点、边写边测（Reading 03 的**测试优先编程**）；把代码依赖的假设写下来；用静态检查替自己防御「愚蠢的错误」——尤其是自己的。

好软件的三大目标（The Big Three） – **定义与目的**：本课程的全部内容都围绕三个性质展开，每个语言特性、每种编程实践、每个设计模式都要问「它如何服务于这三点」。 – **直观解释（"它是什么？"）**：把三大目标当成三把尺子，任何设计决策都拿它们量一量。 – **关键规则与最佳实践**： – **Safe from bugs（免受 bug 之害）**：正确性（现在行为正确）与防御性（将来行为也正确）。静态检查、`final`、动态检查都有贡献。 – **Easy to understand（易于理解）**：代码要与未来的程序员沟通，那个人很可能是几个月后的你自己。显式类型、写明假设的注释都有贡献。 – **Ready for change（为变化做好准备）**：软件永远在变。静态检查在你改名字或改类型时，会立刻在所有使用处显示错误，提醒你同步修改；函数与接口把可变的部分隔离起来。 – 还有其他重要性质（性能、可用性、安全性），它们可能与三大目标冲突，但三大目标是 6.031 最优先考虑的。

为什么这门课用静态类型语言（以及 TypeScript 与 Java 的取舍） – **定义与目的**：sp22 选择 TypeScript、sp21 选择 Java，理由高度一致：**安全性**与**普遍性**。 – **直观解释（"它是什么？"）**：静态检查把安全性「调高」，让你先在一个安全的、被静态检查的语言里学会好的工程习惯，再迁移到动态语言。 – **关键规则与最佳实践**： – 动态类型语言也在向静态化靠拢：Python 3.5+ 的类型注解 + Mypy，JavaScript + 类型声明 = TypeScript。这反映了工程界的普遍信念：**静态类型是构建与维护大型系统的必需品**。 – **渐进类型（gradual typing）**：同一份代码中，一部分有静态类型声明，另一部分没有——这让小原型可以平滑地长成大型可维护系统。 – sp22 认为 TypeScript 相比 Java 类型系统更丰富、样板代码更少、更适合现代 Web 界面；Java 的优点则在于生态成熟、`List` / `Set` / `Map` 等 API 稳定、工具链完善。 – 双方共同的缺点：语言都很大、背负历史包袱（`switch` 语句、原始类型），且都有内部不一致之处（如 Java 的 `final` 在不同上下文中含义不同，`static` 关键字与静态检查毫无关系）。 – **最重要的一点**：本课程教的不是语言特性，而是可迁移的能力——安全、清晰、抽象、工程直觉。

代码示例与对比分析

场景 1：把「值相关的错误」误当成「类型相关的错误」——大整数溢出

❌ *错误代码*

```
// 错误：以为把两个 int 相乘就能得到 40 亿
public class Overflow {
    public static void main(String[] args) {
        int big = 200000;          // 200,000
        big = big * big;           // 期望 40,000,000,000
        System.out.println(big);   // 实际打印 1345294336 —— 完全错误的答案
    }
}
```

【错误代码的问题】 1. 编译期没有任何错误：`int * int` 类型完全正确，静态检查帮不上忙。 2. 运行期也没有任何错误：Java 对整数溢出**不做**动态检查，结果静默回绕成一个仍落在 `int` 范围内的错误数字。 3. 错误值会继续参与后续计算，直到某个完全无关的地方才暴露成怪现象，调试成本极高。 4. 如果在真实系统中这段代码用来算金额、速度、坐标，错误答案可能造成难以挽回的后果（见下文 Ariane 5 的故事）。

✅ *正确代码*

```
// 正确：先想清楚取值范围，再选足够宽的类型
import java.math.BigInteger;

public class OverflowFixed {
    public static void main(String[] args) {
        long big = 200000L;              // 用 long 容纳 4e10
        big = big * big;
        System.out.println(big);         // 40000000000

        // 需要任意精度时用 BigInteger（对应 Python 的 int）
        BigInteger b = BigInteger.valueOf(200000);
        System.out.println(b.multiply(b));   // 40000000000
    }
}
```

【为什么这样更好】 这里的关键变化不是「写对了一个运算符」，而是**先把假设写下来**：$200000^2 \approx 4\times10^{10}$，远超 `int` 的 2.1×10^9。`long` 把可表示范围扩大到这个假设成立；`BigInteger` 则让假设「永不溢出」彻底成立。类型选择本身就是一条可被读者检验的文档。

【代码对比解说】 注意 `200000L` 里的 `L`：如果写成 `long big = 200000; big = big * big;`，Java 会先把 `big` 提升为 `long` 再做乘法，结果是正确的；但若写成 `long big = 200000 * 200000;`，右边两个 `int` 会**先按 int 相乘溢出**，再把已经错误的 `1345294336` 提升为 `long`——这是极其经典的陷阱。规则是：**溢出发生在表达式求值的那一步，而不是赋值的那一步**。

【设计原则透视】 对应 sp22 中的 `Number.MAX_SAFE_INTEGER` 陷阱：类型只界定「值的集合」，该集合的**边界**是表示不变量（representation invariant, RI）的一部分。静态类型系统保证「值属于某个类型」，但不保证「值落在有意义的范围内」。这正是「静态检查面向类型、动态检查面向具体值」这条分界线的具体体现——而当语言连动态检查都不做时，责任就落回程序员的设计上。在 Reading 11（抽象函数与表示不变量）中，我们会把「取值范围」正式写成 RI，并用测试去覆盖边界（Reading 03）。

场景 2：整数除法——最容易被静态检查放过的一类错误

❌ *错误代码*

```
// 错误：5/9 在 int 域里求值，结果是 0
public class Celsius {
    public static double fahrenheitToCelsius(double f) {
        return (f - 32) * (5 / 9);   // 括号让它看起来像「先算比例」，实际先算出了 0
    }

    public static void main(String[] args) {
        System.out.println(fahrenheitToCelsius(212.0));   // 打印 0.0，而不是 100.0
    }
}
```

【错误代码的问题】 1. 编译期无错误：`(double) * (int/int)` 类型合法。 2. 运行期无异常：整数除法 `5/9 == 0` 是语言的正常行为，不抛错。 3. 结果为「看起来很像正确答案」的 0.0——比崩溃更危险，因为它可能被当成「温度本来就是 0 度」而流向生产环境。 4. 这个 bug 只会被某些输入触发；任何只用冻结温度（f = 32）的测试都无法发现它。

✅ *正确代码*

```
// 正确：把常量写成 double，强制在浮点域求值
public class CelsiusFixed {
    public static double fahrenheitToCelsius(double f) {
        return (f - 32) * (5.0 / 9);   // 5.0/9 == 0.5555...
    }

    public static void main(String[] args) {
        System.out.println(fahrenheitToCelsius(212.0));   // 100.0
    }
}
```

【为什么这样更好】 `5.0` 是一个 `double` 字面量，因此 `5.0 / 9` 触发二元数值提升，在浮点域求值得到 0.555...，整条表达式随后都在 `double` 域中进行。更稳妥的写法还有 `(f - 32) * 5 / 9`（乘法先行，天然保持 double），但显式写 `5.0 / 9` 的可读性最好，因为它把「这是一个比例」这一意图直接写在了代码里。

【代码对比解说】 两种写法在字符上的差别只有两个字符（`5` 改成 `5.0`），语义差别却是「100 度」与「0 度」。这类 bug 的可怕之处在于它**同时躲过了静态检查与动态检查**：类型没错，值也没「非法」。唯一的防线是**测试**（Reading 03）：只要针对「非冻结温度」这一输入子域取一个测试用例，`212.0 → 100.0` 立刻失败。

【设计原则透视】 这是「原始类型不是真正的数」的教科书案例：`int` 上的 `/` 与数学上的除法不是同一个函数。从设计原则看，它同时违反了三目标中的两个——不安全（错误答案）且不易理解（`5/9` 的读者必须自己推断求值域）。修复它的手段（写 `5.0`）本质上是在代码里**显式记录求值域的假设**，这正是本讲「记录假设」的实践。

场景 3：固定长度数组造成缓冲区溢出——用 List 替代

❌ *错误代码*

```
import java.util.List;
import java.util.ArrayList;

// 错误：用魔法数字 100 装一个长度不可预测的序列
public class HailstoneBuggy {
    public static int[] hailstoneSequence(int n) {
        int[] a = new int[100];     // <==== DANGER, WILL ROBINSON!
        int i = 0;
        while (n != 1) {
            a[i] = n;
            i++;
            if (n % 2 == 0) {
                n = n / 2;
            } else {
                n = 3 * n + 1;
            }
        }
        a[i] = n;
        i++;
        return a;                   // 返回长度恒为 100 的数组，有效元素只有 i 个
    }
}
```

【错误代码的问题】 1. `100` 是魔法数字：`hailstoneSequence(27)` 需要 112 项，立刻抛出 `ArrayIndexOutOfBoundsException`——一个只在特定输入下才发生的运行期错误。 2. 返回值的**契约模糊**：数组长度恒为 100，调用者无法判断哪些元素有效；一旦调用者用 `a.length` 去遍历，就会读到一堆无意义的 0。 3. 在 C/C++ 里这不再是异常而是**缓冲区溢出**：越界写入会破坏相邻内存，历史上造成过大量网络安全事件与网络蠕虫。 4. 代码把「序列有多长」这一假设硬编码进来，任何关于序列长度的新事实（例如数学上的新结果）都需要改动多处。

✅ *正确代码*

```
import java.util.List;
import java.util.ArrayList;

// 正确：用自动扩容的 List，长度由数据自己决定
public class Hailstone {
    /**
     * Compute a hailstone sequence.
     * @param n starting number for sequence; requires n > 0.
     * @return hailstone sequence starting with n and ending with 1.
     */
    public static List<Integer> hailstoneSequence(final int n) {
        final List<Integer> list = new ArrayList<Integer>();
        int current = n;
        while (current != 1) {
            list.add(current);
            if (current % 2 == 0) {
                current = current / 2;
            } else {
                current = 3 * current + 1;
            }
        }
        list.add(current);
        return list;
    }
}
```

【为什么这样更好】 `List` 会自动随元素增加而扩容（直到内存耗尽），因此「序列长度」这一假设彻底从代码中消失，也就没有了 `ArrayIndexOutOfBoundsException`。返回类型 `List<Integer>` 的 `size()` 精确等于有效元素个数，调用者不再需要猜。同时规格说明明确写出了前置条件（`requires n > 0`）与后置条件（以 n 开始、以 1 结束），把类型表达不了的假设写了下来。

【代码对比解说】 这里有三处修改值得逐条体会：第一，容器从定长数组改为可变长 `List`，把「长度」这个不可控因素交给容器管理；第二，声明类型用接口 `List` 而不是实现类 `ArrayList`，让实现可以随时替换而不影响客户代码（对应 Reading 12 的接口与实现分离）；第三，在规格说明中显式写出 `n > 0`。第三点尤其重要：Java 编译器**不会**检查 `n > 0`，它是一条纯文档假设，但是它的缺失会导致 `n = 0` 时无限循环（0 是偶数，永远除以 2 还是 0），`n = -5` 时陷入 −5 → −14 → −7 → −20 → −10 → −5 的死循环。

【设计原则透视】 数组版本暴露的是**表示不变量被破坏**：`0 <= i < a.length` 是这段代码必须始终成立的 RI，而代码没有任何机制保证它（见 Reading 11）。`List` 版本把这个不变量交给容器自身承担，从而在抽象边界上把「不可能出错」变成结构性事实。此外，接口类型 `List` 让客户不依赖具体表示，这是「为变化做好准备」的直接体现：把 `ArrayList` 换成 `LinkedList` 时，客户代码无需改动。

场景 4：`final` 把「不重新赋值」的假设交给编译器

❌ *错误代码*

```
import java.util.List;
import java.util.ArrayList;

// 错误：循环体反复重写参数，规格说明中的假设在循环里悄悄失效
public class HailstoneMutating {
    /**
     * @param n starting number for sequence; requires n > 0.
     * @return hailstone sequence starting with n and ending with 1.
     */
    public static List<Integer> hailstoneSequence(int n) {
        List<Integer> list = new ArrayList<Integer>();   // 从未被重新赋值，却不是 final
        while (n != 1) {
            list.add(n);
            if (n % 2 == 0) {
                n = n / 2;          // 参数被重新赋值
            } else {
                n = 3 * n + 1;      // 参数被重新赋值
            }
        }
        list.add(n);
        return list;
    }
}
```

【错误代码的问题】 1. 参数 `n` 在循环中被反复改写，规格说明里「n 是序列的起始值」这一说法在循环体内部不再成立，读者必须时刻跟踪它的当前含义。 2. `list` 明明从未被重新赋值，却声明为可变引用：读者要通读整个方法体才能确认这一点，编译器也无法替他确认。 3. 这类「顺手复用参数」的习惯在方法变长以后极易演变成真正的 bug，例如某次改动把循环后的 `n` 当成起始值使用。 4. 假设没有被写下来，也就没有被检查——一旦有人后来往循环里加了一行改写 `n` 的代码，没有任何提示。

✅ *正确代码*

```
import java.util.List;
import java.util.ArrayList;

// 正确：参数只读，循环用独立的局部变量；能加 final 的都加上
public class HailstoneFinal {
    /**
     * @param start starting number for sequence; requires start > 0.
     * @return hailstone sequence starting with start and ending with 1.
     */
    public static List<Integer> hailstoneSequence(final int start) {
        final List<Integer> list = new ArrayList<Integer>();
        int current = start;                    // 唯一的可变变量，职责清晰
        while (current != 1) {
            list.add(current);
            if (current % 2 == 0) {
                current = current / 2;
            } else {
                current = 3 * current + 1;
            }
        }
        list.add(current);
        return list;
    }
}
```

【为什么这样更好】 参数 `start` 与容器 `list` 都加了 `final`：编译器会**静态检查**它们从不被重新赋值，这条「不会变」的假设从此不需要读者去验证。方法里只剩一个可变量 `current`，它的含义在整个方法内保持稳定（「当前正在处理的冰雹数」），读者可以放心地只看它的名字。若将来有人误在循环里写 `start = ...`，编译器会立刻报错，把 bug 挡在编译期。

【代码对比解说】 变量数量没有变（仍然是三个），但**职责边界**变清晰了：`start` 是只读输入，`list` 是只读引用（内容可变），`current` 是唯一的状态载体。`final` 的代价几乎为零（多打六个字母），收益是编译器替你守护一条不变量。注意 `final List<Integer> list` 只保证**引用**不被重赋，`list.add(...)` 完全合法——这正是「不可重新赋值的引用」与「不可变的值」之间的区别，Reading 02 会用快照图把这一点画清楚，Reading 08 会讨论如何用 `Collections.unmodifiableList` 让值本身也不可变。

【设计原则透视】 `final` 是本讲「记录假设」原则的语言化表达：类型记录「值属于哪个集合」，`final` 记录「引用不再改变」，规格说明记录「值必须满足什么条件」。三者构成一个从强到弱的假设梯度：类型由编译器完全检查，`final` 由编译器完全检查，前置条件必须靠文档加运行期检查。一个成熟的工程师会把尽可能多的假设推到「编译器能检查」的那一端。

场景 5：把「值相关的错误」写进规格说明并用防御性检查兜底

❌ *错误代码*

```
// 错误：规格说明里什么都不说，调用者只能靠猜
public class Average {
    public static double average(int sum, int n) {
        return sum / n;          // n == 0 时抛 ArithmeticException
    }
}
```

【错误代码的问题】 1. 方法契约完全没有说明 `n` 不能为 0，调用者无法从签名或注释中得知这个约束。 2. 一旦传入 0，运行期抛出 `ArithmeticException`；如果传入的是 `double` 类型的 0，则连异常都没有，直接得到 `NaN` 或 `Infinity` 并继续传播。 3. 错误在离调用点很远的地方才暴露，定位成本高。 4. 规格说明缺失会让测试无从下手：不知道哪些输入是「合法的」，也就无法写出正确的测试套件（Reading 03 强调「测试套件必须是规格说明的合法客户」）。

✅ *正确代码*

```
// 正确：把前置条件写进规格说明，并在运行期显式检查
public class Average {
    /**
     * @param sum the sum of the values being averaged
     * @param n how many values were summed; requires n > 0
     * @return the arithmetic mean sum / n
     * @throws IllegalArgumentException if n <= 0
     */
    public static double average(int sum, int n) {
        if (n <= 0) {
            throw new IllegalArgumentException("n must be positive, but was " + n);
        }
        return (double) sum / n;
    }
}
```

【为什么这样更好】 规格说明把「`n > 0`」这条编译器无法检查的假设写了下来，调用者读一眼就知道自己的责任。运行期检查把「不检查」变成「动态检查」：错误在**出错的那一行**立刻以清晰的异常信息暴露，而不是变成一个 `NaN` 传染给整个程序。注意 `(double) sum / n`：显式转换避免了整数除法带来的截断，这是场景 2 的教训在这里的复现。

【代码对比解说】 关键转变是把一条隐式假设变成三层显式声明：注释（给人看）、异常声明（给调用者看）、运行期检查（给运行时看）。有人会认为运行期检查「浪费性能」，但对于一个 `n <= 0` 就完全无意义的函数，这正是「工程师是悲观主义者」的具体体现——防御性检查的成本是几纳秒，收益是不必在半夜排查一个 `NaN`。

【设计原则透视】 这段代码展示了本讲全部三条主线的交汇：类型（`int sum, int n`）由编译器静态检查，前置条件（`n > 0`）只能靠文档与运行期检查，而「把假设写下来」直接服务于 Easy to understand。到了 Reading 06（规格说明）我们会把前置条件、后置条件、异常行为形式化成更严格的契约；到了 Reading 07（设计规格说明）我们会讨论「前置条件该由谁负责检查」这一经典权衡。

（sp22 原版 TypeScript 写法对照）

```
// sp22 原版 TypeScript 写法：const 提供「不可重新赋值」的静态检查
const n: number = 5;          // 终生指向 5
let m: number = 5;            // 可重新赋值
m = m + 1;                    // 合法
// n = n + 1;                 // 静态错误：Cannot assign to 'n' because it is a constant.

// const 只固定引用，不冻结值：数组内容仍可变
const array: Array<number> = [];
array.push(5);                // 合法：引用没变，值变了

// TypeScript 的静态类型在编译后会被丢弃，生成的是纯 JavaScript：
//   function hello(name: string): string { return 'Hi, ' + name; }
// 编译为
//   function hello(name) { return 'Hi, ' + name; }
// 所以运行期的一切行为仍由 JavaScript 的（常常不做检查的）语义决定。
```

这组 TypeScript 写法与 Java 的 `final` 形成精确对应：`const` ↔ `final`，都只约束**引用**而不冻结**值**；而 TypeScript 编译后丢弃类型信息这一点，提醒我们「静态检查只存在于编译期」，运行期的安全仍需靠设计与测试（Reading 03）来保证。

与其他设计原则的关联

- 与 **Reading 02（基本 Java / Basic Java）**：本讲只把类型当作「值的集合 + 操作」来介绍，Reading 02 会把它落到具体语法上——原始类型与对象类型的区别、`==` 与 `.equals()`、`List` / `Set` / `Map`、`final` 与可变对象的组合，并用**快照图**把「重新赋值 vs. 改变值」可视化。本讲场景 3 中的 `List<Integer>` 在 Reading 02 会展开为完整的集合 API。
- 与 **Reading 03（测试 / Testing）**：本讲反复强调「静态检查抓不到与具体值相关的错误」。如何抓住它们？答案是系统化测试。Reading 03 的**输入空间划分**与**边界值**正对应本讲中的整数溢出、`Integer.MIN_VALUE`、空字符串、空集合——本讲场景 2 的整数除法 bug 只有靠一个「非边界温度」的测试用例才能发现。
- 与 **Reading 04（代码审查 / Code Review）**：本讲说「写一点点、测一点点」「把假设写下来」，代码审查是把这两条落到团队流程上的实践：让另一个人用本讲的三把尺子（安全、易理解、可修改）来读你的代码。
- 与 **Reading 06（规格说明 / Specifications）** 和 **Reading 07（设计规格说明 / Designing Specifications）**：本讲第一次出现了「规格说明」与「前置条件」，正式的契约式设计在那里展开。
- 与 **Reading 08（不可变性 / Immutability）**：本讲区分了「不可重新赋值的引用」与「不可变的值」，Reading 08 会系统化不可变性的设计原则，并说明它为什么是「Ready for change」的核心武器。
- 与 **Reading 09（避免调试 / Avoiding Debugging）**：本讲提出的「工程师是悲观主义者」在 Reading 09 中被系统化成一系列构造性建议（如 `assert`、快速失败、把假设变成检查）。
- 与 **Reading 11（抽象函数与表示不变量 / Abstraction Functions & Rep Invariants）**：本讲场景 3 中的 `0 <= i < a.length` 是一条尚未命名的表示不变量，Reading 11 会给出 RI/AF 的正式框架。
- 与 **Reading 21–23（并发 / Concurrency、线程安全 / Thread Safety、锁 / Locks）**：本讲「避免改变」的直觉在并发中变成硬性要求——共享可变状态是线程安全问题的根源。

关键要点

- **把检查尽量往左移**：静态检查优于动态检查，动态检查优于不检查。写每一行代码时都问自己「这个错误会在哪一档被抓住」，抓不住的地方必须额外布防。
- **区分「面向类型」与「面向值」的错误**：编译器能保证变量持有某个合法值，但永远不知道是哪一值。除零、越界、非法转换、溢出、精度丢失都只能靠测试与设计来防。
- **假设必须写下来**：类型和 `final` 交给编译器检查；前置条件写进 Javadoc，并用运行期检查兜底。没写下来的假设，未来一定会被忘记。
- **优先选择「不变量由结构保证」的设计**：用自动扩容的 `List` 代替 `int[100]`，永远不要让魔法数字承担正确性责任。
- **用三大目标检验每一个设计决策**：Safe from bugs / Easy to understand / Ready for change。

常见陷阱与注意事项

1. **以为「编译通过」就等于「正确」**：`big = big * big;` 与 `(f - 32) * (5 / 9)` 都能顺利编译。→ 后果：静默的错误答案，且会继续向程序下游传播。
2. **把 `int` 当成数学上的整数**：忘记 `5/2 == 2`、超出 $\pm 2^{31}$ 会回绕、`double` 超过 2^{53} 后 `x + 1 == x`。→ 后果：数值结果错误，且错误只在特定数值范围内出现，测试容易被「恰好选中安全值」而漏过。
3. **用 `==` 比较对象**（本讲铺垫，Reading 02 详述）：`==` 在 Java 中对对象重载为「是否指向同一对象」。→ 后果：字符串内容相同却返回 `false`，或因为字符串驻留（interning）而「偶尔正确」，成为最难查的一类 bug。
4. **把参数当作可随意改写的临时变量**：在循环里反复修改参数。→ 后果：规格说明中「参数是起始值」的假设失效，后续维护者读到循环后的参数值时会得到错误结论；也无法对它加 `final`。
5. **忽略前置条件**：`hailstoneSequence(0)` 或 `hailstoneSequence(-5)` 会无限循环；`average(sum, 0)` 会抛异常。→ 后果：程序挂死或崩溃，而且发生在与错误源头无关的位置。
6. **依赖 C/C++ 式的「数组越界无所谓」直觉**：Java 会抛 `ArrayIndexOutOfBoundsException`，但不代表可以靠它兜底。→ 后果：把本该在编译期或设计期解决的问题留到生产环境的运行期。

思考题（带答案）

问题 1：下面这段 Java 代码在编译期、运行期分别会出什么问题？如果没有问题，结果是否等于 8.0？

```
double x = 16.0;
double y = 2;
double z = x / y;       // 期望 8.0
int w = 5 / 2;          // 期望 2.5
```

答案：`x / y` 没有任何问题：`x` 是 `double`，二元数值提升让 `y` 也变成 `double`，结果是 8.0。`int w = 5 / 2;` **编译通过、运行通过，但结果是 2 而不是 2.5**——两个 `int` 之间的 `/` 是整数除法，小数部分被截断，这是「不检查」的典型例子：既不是静态错误，也不是动态错误，只是错误答案。这正对应本讲判断框架中的第三档。

问题 2：为什么 `Math.sin("30")` 是静态错误，而 `Integer.valueOf("8000000000")` 是动态错误？请用「类型 vs. 具体值」这条分界线解释。

答案：`Math.sin` 的签名是 `sin(double)`，参数类型是 `double`，而 `"30"` 的类型是 `String`。这种不匹配在编译期就完全可见，编译器不需要知道任何运行期的值就能判定它非法，因此是**静态错误**。相反，`Integer.valueOf(String)` 的参数类型就是 `String`，类型完全正确；错误只发生在**这个具体字符串**无法被解析成 `int` 范围内的十进制整数时——`"8000000000"` 是 80 亿，超出了 `int` 的合法范围。编译器无法预知运行时收到哪个字符串，只能由运行期抛出 `NumberFormatException`。这条分界线可以概括为：**静态检查面向类型（值的集合），动态检查面向具体值。**

问题 3：本讲的 `hailstoneSequence` 规格说明写了 `requires n > 0`。为什么不能像类型那样让编译器检查它？如果不写这条前置条件，可能出现什么后果？你能想出两种让这条假设被强制检查的手段吗？

答案：`n > 0` 是一个关于**值**的约束，而编译器只知道 `n` 是 `int`；`int` 的合法值集合包含负数和 0，因此「正整数」不是一个可以用 Java 类型表达的集合（除非定义一个 `PositiveInt` 类，把约束编码进构造过程——这是 Reading 11 中表示不变量的思路）。不写这条前置条件，调用者可能传入 `0`（0 是偶数，`0/2 == 0` 永远循环）或 `-5`（进入 −5 → −14 → −7 → −20 → −10 → −5 的环），程序静默挂死。两种强制检查手段：其一，在方法开头写运行期检查 `if (n <= 0) throw new IllegalArgumentException(...)`，把「不检查」升级为「动态检查」；其二，在规格说明中把它变成调用者的义务，并由测试套件保证只使用合法输入（这个「让违反前置条件的调用成为调用者的 bug」的立场将在 Reading 06/07 中正式讨论）。

Reading 2: Java 基础（sp22 原版为 Basic TypeScript）

> 说明：本讲 sp22 原版使用 TypeScript，本笔记按用户要求提供 Java 代码示例；类型/API 与 sp21（6.031 Java 版）原文保持一致。文中凡涉及本讲之外、属于 Java 生态的额外知识，均以「补充说明」标注。

概述

本讲的目标是从 Python 平滑过渡到静态类型语言的日常写法：学会基本语法与语义，掌握**快照图（snapshot diagram）** 这一描述运行时状态的通用工具，并建立「原始类型 vs. 对象类型」「重新赋值 vs. 改变值」「可变 vs. 不可变」三组贯穿全课程的核心区分。它同时服务于三大目标：泛型与静态类型让集合操作**免受 bug 之害**（Safe from bugs），快照图与 `final` 让「谁在什么时候改了什么东西」变得**易于理解**（Easy to understand），而「声明接口类型、选择实现类」的习惯让代码**为变化做好准备**（Ready for change）。

核心概念与设计原则详解

快照图（Snapshot Diagram） – **定义与目的**：快照图是程序在某一时刻的运行时内部状态的图示，包含**栈（stack）**（正在进行的方法及其局部变量）与**堆（heap）**（当前存在的对象）。它让我们能用图而不是用嘴来讨论「不可变的值 vs. 不可重新赋值的引用」「指针别名（aliasing）」「栈 vs. 堆」「抽象 vs. 具体表示」这些微妙问题。 – **直观解释（"它是什么？"）**：把变量画成一支持有箭头的标签，箭头指向一个值；原始类型的值直接写在箭头旁边，对象类型的值是堆上的一个圆圈（气泡），圆圈里可以写字段名与字段值。给引用加双箭

头表示「不可重新赋值」（`final`），给对象圆圈加双边框表示「不可变值」（如 `String`）。 – **关键规则与最佳实践**： – 语法是**灵活的**，不必画出全部细节：只画与当前讨论相关的部分。讨论字符串内容时画字符，讨论对象身份时只画一个圆圈。 – **赋值**（`x = ...`）改变箭头的指向；**修改**（`list.add(...)`、`sb.append(...)`）改变箭头所指气泡的内部。 – 双线（双箭头 / 双边框）只在需要强调「不可变」时使用；当可变性显而易见或不相关时，保持单线，避免图画得太吵。 – 快照图的记法适用于任何现代语言（Python、Java、C++、Ruby），并在后续课程中泛化为对象模型。 – 在 6.031 中，快照图是**团队沟通工具**：课堂讨论、结对编程、代码评审时都可以画。

原始类型与对象类型（Primitive Types vs. Object Types） – **定义与目的**：Java 把值分成两类，理解这个区分是理解 `==`、参数传递、内存布局的前提。它首先服务于 Safe from bugs（选错类型会导致错误的相等性判断）。 – **直观解释（"它是什么？"）**：原始类型的值就是值本身（一个小方块里写着 5）；对象类型的值是一个「指针」，指向堆上的一个气泡。因此**原始类型没有内部结构**，而对象类型可以有内部结构（引用其他原始值或对象）。 – **关键规则与最佳实践**： – 原始类型：`int`、`long`、`boolean`、`double`、`float`、`char` 等，约定**全部小写**，占用固定的小块内存。 – 对象类型：`String`、`BigInteger`、`List`、`Turtle` 以及自定义类，约定**首字母大写**，占用可变且可能很大的内存。 – 判别规则（来自本讲练习）：原始类型并不只是「数值」，`char` 是原始类型而 `String` 不是；判别的可靠依据是**约定的大小写**、**是否有内部结构**、**是否占用固定小内存**。 – 每个原始类型都有**包装类型（wrapper type）**：`int` / `Integer`、`long` / `Long`、`double` / `Double`。Java 会在两者之间自动装箱（boxing）与拆箱（unboxing），所以 `Integer i = 5;` 合法。 – 泛型参数只能是对象类型，因此写 `List<Integer>` 而不是 `List<int>`；遍历时优先用原始类型 `int num`，因为原始类型的 `==` 更简单可靠。

改变值 vs. 重新赋值变量（Mutating Values vs. Reassigning Variables） – **定义与目的**：这是本讲最重要的一组区分。混淆它们会直接导致共享可变状态引发的 bug，也会让人误以为 `final` 就意味着「不可变」。 – **直观解释（"它是什么？"）**：**重新赋值**是把变量的箭头指向另一个气泡；**改变值**是让同一个气泡里的内容发生变化。前者改变「谁」，后者改变「什么」。 – **关键规则与最佳实践**： – `String s = "a"; s = s + "b";` 是**重新赋值**：`s` 从指向 `"a"` 改为指向新对象 `"ab"`（`String` 不可变，原对象没变）。 – `StringBuilder sb = new StringBuilder("a"); sb.append("b");` 是**改变值**：箭头没动，气泡内部从 `"a"` 变成 `"ab"`。 – 参数传递永远是「按值传递引用」：方法内对参数**重新赋值**不影响调用者；对参数所指对象**修改**会影响调用者。因此 `void f(String s, StringBuilder sb) { s.concat("b"); s += "c"; sb.append("d"); }` 在调用 `f(t, tb)` 之后，`t` 仍是 `"a"`（`s += "c"` 只是让局部变量 `s` 改指新对象，`s.concat("b")` 的返回值被丢弃），而 `tb` 变成 `"ad"`。 – 只读方法（如 `concat`）返回新对象而非修改原对象；修改方法（如 `append`）返回 `this` 并改变内部状态。API 文档会写明是哪一种。

不可变值与不可重新赋值的引用（Immutable Values vs. Unreassignable References） – **定义与目的**：Java 提供两种「不变」的保证，分别作用于**值**和**引用**。理解它们正交组合出的四种情况，是安全设计的基础。 – **直观解释（"它是什么？"）**：**不可变类型（immutable type）**：值一旦创建就永远不变（`String`）。**不可重新赋值的引用**：变量只被赋值一次（`final`）。二者相互独立，可以任意组合。 – **关键规则与最佳实践**： – `final int n = 5;`：`n` 终生指向 5，重新赋值会产生**编译错误**——`final` 提供的是静态检查。 –

`final` 引用可以指向**可变对象**：`final StringBuilder sb = new StringBuilder("a"); sb.append("b");` 完全合法。 – 非 `final` 引用可以指向**不可变值**：`String s = "a"; s = "ab";` 合法，只是箭头改指了。 – 优先用 `final` 声明方法参数与尽可能多的局部变量：它既是给读者的文档，也被编译器检查。 – 不可变性是 Safe from bugs 的基石：不可变对象可以被自由共享（别名不再是威胁），也天然线程安全（见 Reading 21）。

`==` 与 `.equals()`：两种相等性 – **定义与目的**：Java 对原始类型与对象类型使用**不同的相等性测试**，用错会得到「内容相同却不相等」的结果。这是本讲最直接对应 Safe from bugs 的一条规则，也是 Python 背景学生最容易踩的坑。 – **直观解释（"它是什么？"）**：`==` 对原始类型比较**值**（`5 == 5` 为 `true`，`'a' == 'a'` 为 `true`）；对对象类型比较**身份**（两个箭头是否指向同一个气泡），在 Python 中对应的是 `is`。`.equals()` 对对象比较**内容**（`"abc".equals("abc")` 为 `true`）。 – **关键规则与最佳实践**： – 比较原始类型（`int`、`char`、`double`）用 `==`；比较对象（`List`、数组、`String`、其他对象）用 `.equals()`。 – 在原始类型上调用方法（如 `5.equals(5)`）是**静态错误**，因为 Java 不允许在原始类型上调用方法——这算是幸运的，错误很容易被发现。 – 反过来，在对象上用 `==` **不会报错**，只会安静地给出错误答案，这才是真正危险的方向。 – 危险点还在于「优化会掩盖 bug」：`String s = "foobar"; String t = "foobar";` 时 `s == t` 很可能返回 `true`，因为 Java 复用了同一个字符串对象（字符串驻留）。这会让写错的代码在测试中「碰巧通过」。 – `char` 是原始类型，字面量用**单引号**（`'a'`）；`String` 是对象，字面量用**双引号**（`"abc"`、`""`），二者不可混用。

集合：List、Set 与 Map（Java Collections） – **定义与目的**：三门「日常数据结构」，分别对应有序可重复序列、无序不重复集合、键值映射。它们的共同价值是：把「容器大小管理」这件容易出错的事交给库去承担（回忆 Reading 01 的 `int[100]` 缓冲区溢出）。 – **直观解释（"它是什么？"）**：`List` 像 Python 的 list（有顺序、可重复）；`Map` 像 Python 的 dict（键必须可哈希）；`Set` 像数学集合（要不就在里面，要不就不在）。 – **关键规则与最佳实践**： – List：`lst.size()`、`lst.add(e)`、`lst.isEmpty()`、`lst.contains(e)`、`lst.get(i)`、`lst.set(i, e)`。 – Map：`map.put(k, v)`、`map.get(k)`、`map.containsKey(k)`、`map.remove(k)`。 – Set：`s1.contains(e)`、`s1.containsAll(s2)`（判断 s1 ⊇ s2）、`s1.removeAll(s2)`。 – 快照图记法：`List` 画成带下标字段的对象，`Map` 画成含键/值对的对象，`Set` 画成一组**无名字段**的对象。 – 键必须可哈希：`Map` 的键与 `Set` 的元素必须是「适合做键的类型」；在 Java 中这意味着它们必须正确实现 `hashCode` 与 `equals`（Reading 15 会详细展开）。

字面量（Literals）与不可变集合 – **定义与目的**：快速构造集合的语法糖，同时引入「不可变集合」这一安全默认值。 – **直观解释（"它是什么？"）**：Java **没有** Python 那样的 list/dict 字面量，但有数组字面量，以及 `List.of` / `Set.of` / `Map.of` 这些静态工厂方法。 – **关键规则与最佳实践**： – 数组字面量：`String[] arr = { "a", "b", "c" };`——注意这创建的是**数组**，不是 `List`。 – `List.of("a", "b", "c")` 创建**不可变** List：不能添加、删除或替换元素。 – `Set.of("a", "b", "c")`、`Map.of("apple", 5, "banana", 7)` 同理，创建不可变集合。 – 需要「已初始化但可变」的 List 时，用拷贝构造：`new ArrayList<>(List.of("Huey", "Dewey", "Louie"))`。这比连续调用多次 `add()` 简洁得多。 – 在测试中优先使用不可变字面量，可以防止测试用例之间通过共享集合互相污染。

泛型（Generics）：声明集合变量 – **定义与目的**：泛型让集合**限制元素的类型**，从而让编译器静态检查「只加入正确类型的元素」，并在取出元素时保证类型正是所期望的。这是 Reading 01「静态检查」在容器上的直接应用。 – **直观解释（"它是什么？"）**：`List<String>` 读作「装着 String 的 List」。它是给容器贴标签，而不是给元素贴标签。 – **关键规则与最佳实践**： – 声明形式：`List<String> cities;`、`Set<Integer> numbers;`、`Map<String,Turtle> turtles;`。 – 不能创建原始类型的集合：`Set<int>` 非法，必须写 `Set<Integer>`。 – 自动装箱/拆箱让包装类型几乎无感：`sequence.add(5);` 把 5 包装成 `Integer`；`int second = sequence.get(1);` 自动拆箱为 `int`。 – 构造时的菱形语法省略右侧参数：`List<String> names = new ArrayList<>();`。 – **补充说明**：raw type（裸类型，如 `List numbers`）会关闭泛型检查，只在维护遗留代码时才会遇到；新代码一律使用泛型。

接口与实现类：ArrayList、LinkedList、HashSet、HashMap – **定义与目的**：Java 帮我们区分**规格说明**（这个类型做什么）与**实现**（代码长什么样）。`List`、`Set`、`Map` 都是**接口（interface）**，只规定行为，不提供实现；使用者可以在不同场景选择不同实现。 – **直观解释（"它是什么？"）**：接口是「插座标准」，实现类是「具体的插座」。你的电器（客户代码）只依赖标准，因此可以随时换插座。 – **关键规则与最佳实践**： – 创建：`List<String> firstNames = new ArrayList<>();`、`List<String> lastNames = new LinkedList<>();`。 – `ArrayList` 与 `LinkedList` 都完整提供 `List` 的全部操作，行为完全一致，只是**性能不同**；互换它们不会破坏代码。 – 在 6.031 中：**拿不准就用 `ArrayList`**；`Set` 默认用 `HashSet`（需要有序时用 `TreeSet`）；`Map` 默认用 `HashMap`。 – 声明变量与返回类型时**永远写接口类型**（`List` 而非 `ArrayList`），这样实现可以随需求变化而替换。 – 用另一个集合初始化：`new ArrayList<>(Set.of("Duck"))`。

迭代（Iteration） – **定义与目的**：遍历集合是最常见的任务，for-each 语法让这件事简洁且不易出错。 – **直观解释（"它是什么？"）**：`for (String city : cities)` 读作「对 cities 中的每个 city」。底层使用 Iterator 设计模式（本课程后段会展开）。 – **关键规则与最佳实践**： – List 与 Set 都可以直接 for-each；`Map` 不能直接 for-each，需要遍历 `turtles.keySet()`（或 `values()`、`entrySet()`）。 – 循环变量写原始类型：`for (int num : numbers)` 而不是 `Integer num`，因为原始类型的相等性判断更简单、更不易出错。 – **绝不要在迭代过程中修改正在遍历的集合**：添加、删除或替换元素会破坏迭代，甚至让程序崩溃（`ConcurrentModificationException`）。这条警告对 Python 同样成立。 – 需要下标时才写 `for (int ii = 0; ii < cities.size(); ii++)`；这种写法冗长，且藏 bug 的地方更多（起始值写成 1、用 `<=` 而不是 `<`、某处变量名写错等）。

变量声明与作用域（Variable Declarations and Scope） – **定义与目的**：Java 的局部变量作用域是**它所在的那一对花括号**，这与 Python「函数级作用域」不同，会导致一类「在分支里声明、在分支外使用」的编译错误。 – **直观解释（"它是什么？"）**：花括号是一道墙，墙内声明的变量出了墙就不存在。 – **关键规则与最佳实践**： – `int b = 2;` 写在 `if` 块里，块外的 `b *= 3;` 就是静态错误（`cannot find symbol`）。 – 正确的修法是**在分支之前声明并初始化**：`int b = 0; if (...) { b = 2; } else { b = 4; } b *= 3;`。注意 Java 要求局部变量在使用前**确定被赋值**，所以如果注释掉 `else` 分支，编译器会在最后一行报「变量可能未

初始化」——这是**编译期**发现的错误，而不是运行期。 – **TypeScript 对照（sp22 原版写法）**：TypeScript 用 `let`（可重新赋值）与 `const`（不可重新赋值）声明局部变量，`const` 提供不可重新赋值的**静态检查**；`let` / `const` 的作用域同样是所在花括号，而老式的 `var` 是函数级作用域，已被强烈不推荐。Java 中最接近 `const` 的机制是 **`final`**，其作用域规则本质上与 `let` 相同（都在花括号内），没有 `var` 那样的函数级作用域问题。 – **补充说明**：Java 10+ 提供 `var` 关键字做局部变量类型推断（`var x = 5;`），但它仍是**静态类型**——类型在编译期被推断出来，与 JavaScript 的 `var` 完全无关，不要因同名而混淆。

枚举（Enumerations） – **定义与目的**：当一个类型只有**小的、有限的**一组不可变取值时（月份、星期、罗盘方位、可选颜色），枚举让这些取值成为命名常量，并且**是一个全新的类型**，因而可以被静态检查。 – **直观解释（"它是什么？"）**：枚举把「一堆散落的常量」升级为一个类型。以往用 `int` 或字符串常量表示时，任何整数或任何字符串都能被塞进去；枚举则只接受它自己列出的那几个值。 – **关键规则与最佳实践**： – 声明：`public enum PenColor { BLACK, GRAY, RED, PINK, ORANGE, YELLOW, GREEN, CYAN, BLUE, MAGENTA; }` – 使用：`PenColor drawingColor; drawingColor = PenColor.RED;`——像命名静态常量一样引用。 – 枚举比数值常量**更类型安全**：`int month = TUESDAY;` 不会报错（如果用的是整数常量），而 `Month month = DayOfWeek.TUESDAY;` 是**静态错误**。 – 枚举能抓住拼写错误：`String color = "REd";` 不会报错，而 `PenColor drawingColor = PenColor.REd;` 是**静态错误**。 – Python 3 也有枚举，但**不做静态类型检查**——这正是 Java 枚举的优势。

API 文档与规格说明（API Documentation and Specifications） – **定义与目的**：API（application programming interface）是别人提供的、你可以编程调用的方法与类。API 文档中的详细描述就是**规格说明（specification）**，它让你无需阅读实现代码就能正确使用 `String`、`Map`、`BufferedReader`。 – **直观解释（"它是什么？"）**：文档是「黑盒说明书」，实现是「白盒内部」。规格说明的存在，正是抽象边界得以成立的前提。 – **关键规则与最佳实践**： – 读文档的顺序：类描述 → 构造器摘要 → 方法摘要 → 点击具体方法看**方法签名**（返回类型、方法名、参数、可能抛出的异常）、完整描述、参数说明、返回值说明。 – 类层次与方法摘要能帮你发现「`HashMap` 是 `Map` 的实现」这类关系。 – 用搜索框直接跳到类、接口或方法，是查阅 API 的最快路径。 – **TypeScript 对照（sp22 原版写法）**：TypeScript 的三处文档来源是 MDN JavaScript 参考（通用特性与浏览器 API）、Node.js 参考（非浏览器 API，如 `fs.readFileSync`、`os.homedir`）以及 TypeScript Handbook（TypeScript 特有特性）。Java 的对应物是 Java API 文档中的 `java.lang.String`、`java.util.List`、`java.util.Map`、`java.io.BufferedReader` 等。

TypeScript 与 Java 语法对照速查（本讲 sp22 内容的 Java 对应写法）

sp22 原版写法（TypeScript）	本笔记的 Java 写法	说明
`let n: number = 3;`	`int n = 3;`	TypeScript 的 `number` 同时表示整数与浮点；Java 区分 `int` / `long` / `double`
`const n: number = 5;`	`final int n = 5;`	二者都提供「不可重新赋值」的静态检查

sp22 原版写法（TypeScript）	本笔记的 Java 写法	说明
`Array<string>` 或 `string[]`	`List<String>` 或 `String[]`	Java 的 `List` 可变长，数组定长
`let cities = new Array<string>();`	`List<String> cities = new ArrayList<>();`	Java 用接口类型声明、实现类构造
`new Map([["apple", 5]])`	`Map.of("apple", 5)` / `new HashMap<>()`	`Map.of` 返回**不可变** Map
`new Set<number>()`	`new HashSet<>()`	元素必须可哈希
`for (const city of cities)`	`for (String city : cities)`	都用 for-each；TypeScript 慎用 `for...in`（遍历的是下标）
`s.has(e)` / `s.add(e)` / `s.size`	`s.contains(e)` / `s.add(e)` / `s.size()`	Java 的方法调用一律带括号
`arr.push(x)` / `arr.length`	`list.add(x)` / `list.size()`	Java 数组用 `arr.length`（无括号），List 用 `size()`
`enum PenColor { RED, GREEN }`	`public enum PenColor { RED, GREEN; }`	语义几乎一致
`{ x: 5, y: -2 }`（record type）	没有直接对应，需定义类	**补充说明**：Java 16+ 的 `record Point(int x, int y) {}` 最接近 TypeScript 的 record type
`const { quotient, remainder } = f(23, 7);`	无析构语法	**补充说明**：Java 需逐个 `result.quotient()` 取值

代码示例与对比分析

（sp22 原版 TypeScript 写法对照：变量声明、作用域与 record type）

```
// sp22 原版 TypeScript 写法
let a: number = 5;              // 可重新赋值      ↔ Java: int a = 5;
const b: number = 5;            // 不可重新赋值    ↔ Java: final int b = 5;

if (a > 10) {
  let c: number = 2;            // 作用域仅限这一对花括号    ↔ Java: int c = 2;
} else {
  // c 在这里不可见：TypeScript 与 Java 一样是块级作用域
  // c = 4;                     // 静态错误：Cannot find name 'c'
}

// 对象字面量与 record type
let point: { x: number, y: number } = { x: 5, y: 2 };
// Java 没有对象字面量；最接近的是 Java 16+ 的 record：
//   record Point(int x, int y) { }
//   Point point = new Point(5, 2);

// 解构赋值（Java 没有对应语法）
// const { quotient, remainder } = integerDivision(23, 7);
```

两者的**块级作用域**规则是一致的，所以「在 `if` 分支里声明、在分支外使用」在两种语言里都是编译错误；差别在于 Java 没有对象字面量、record type 与解构赋值，需要写一个显式的类（或使用 record）来打包多个返回值。

场景 1：用 `==` 比较对象内容——最典型的「不报错的 bug」

❌ *错误代码*

```
// 错误：用 == 比较字符串内容
public class NameCheckBuggy {
    public static void main(String[] args) {
        String s = "foobar";
        String t = "foobar";
        System.out.println(s == t);            // 打印 true —— 靠的是字符串驻留，纯属运气

        String u = new String("foobar");
        System.out.println(s == u);            // 打印 false —— 内容一样却「不相等」
    }
}
```

【错误代码的问题】 1. `s == t` 返回 `true` 不是因为内容相同，而是因为 Java 复用了同一个字符串对象（字符串驻留）。这个「优化」会让写错的代码在测试中碰巧通过。 2. `s == u` 返回 `false`：内容完全相同却判定不相等，逻辑随之走错分支。 3. 正确性依赖于「字符串从哪里来」这一与语义无关的实现细节，属于典型的**不可靠代码**：把字面量换成从文件或网络读入的字符串，行为立刻改变。 4. 编译期毫无提示，运行期也不抛异常，错误会静默传播到很远的地方。

✅ *正确代码*

```
// 正确：对象比内容用 equals()，原始类型比值用 ==
public class NameCheck {
    public static void main(String[] args) {
        String s = "foobar";
        String t = "foobar";
        String u = new String("foobar");

        System.out.println(s.equals(t));       // true —— 内容相同
        System.out.println(s.equals(u));       // true —— 内容相同，与对象身份无关

        int i1 = 5;
        int i2 = 5;
        System.out.println(i1 == i2);          // true —— 原始类型比较值

        char c1 = 'a';
        char c2 = 'a';
        System.out.println(c1 == c2);          // true —— char 是原始类型
    }
}
```

【为什么这样更好】 `equals()` 问的是「内容是否相同」，这正是几乎所有业务逻辑真正关心的问题；它给出的答案不依赖对象从哪来、是否被驻留、是否被缓存，因此行为稳定、可预测。而 `==` 只在原始类型上表达「值相同」，语义清晰无歧义。

【代码对比解说】 这段代码的教学价值在于它把一条规则拆成了两个方向：`==` 用于原始类型（`int`、`char`、`double`），`equals()` 用于对象（`String`、`List`、数组、自定义对象）。注意两个方向的「可发现性」完全不同：在原始类型上调 `.equals()` 是**静态错误**（`5.equals(5)` 无法编译），错误立刻暴露；而在对象上用 `==` 完全合法，只是答案错——所以真正需要肌肉记忆的是后者。此外还要注意 `s1 == i1`（`String` 与 `int`）是静态错误，因为二者类型不可比较。

【设计原则透视】 这条规则直接对应「Safe from bugs」：`==` 在 Java 中是**重载**的（对原始类型比值、对对象比身份），重载带来的歧义正是 bug 的温床。更深的层面，`equals()` 的正确语义依赖对象类型对 `equals` / `hashCode` 的正确实现——这就是 Reading 15（相等性）的主题，而 `Map` / `Set` 的键必须可哈希正是本讲已经埋下的伏笔。在 Reading 11（抽象函数）中我们会看到，`equals` 的语义本质上由抽象函数决定。

场景 2：在迭代过程中修改集合

❌ *错误代码*

```
import java.util.List;
import java.util.ArrayList;

// 错误：一边 for-each 遍历，一边删除元素
public class WordFilterBuggy {
    public static List<String> removeShortWords(List<String> words) {
        for (String w : words) {
            if (w.length() < 3) {
                words.remove(w);        // 破坏迭代器 → ConcurrentModificationException
            }
        }
        return words;
    }
}
```

【错误代码的问题】 1. 运行期抛出 `ConcurrentModificationException`（若恰好删到最后一个元素附近，也可能不抛异常而是**静默漏删**，更难查）。 2. 编译期无任何错误：语法与类型完全正确。 3. 这个 bug 的行为依赖集合的具体实现与元素分布，因此「本地测不出来、线上偶发」。 4. 同样的错误在 Python 中也存在（`for num in numbers: numbers.remove(num)`），从 Python 迁过来时极容易原样照搬。

✅ *正确代码*

```
import java.util.List;
import java.util.ArrayList;
import java.util.Iterator;

// 正确一：用集合自己的批量操作，由库保证迭代安全
public class WordFilter {
    public static List<String> removeShortWords(List<String> words) {
        words.removeIf(w -> w.length() < 3);
        return words;
    }

    // 正确二：需要更复杂的条件时，显式使用 Iterator 的 remove()
    public static List<String> removeShortWordsWithIterator(List<String> words) {
        Iterator<String> iter = words.iterator();
        while (iter.hasNext()) {
            String w = iter.next();
            if (w.length() < 3) {
                iter.remove();          // 通过迭代器删除，迭代状态保持一致
            }
        }
        return words;
    }
}
```

【为什么这样更好】 `removeIf` 把「遍历 + 条件删除」交给集合自己实现，库内部使用的正是 `Iterator.remove()`，因此迭代状态始终一致。当条件复杂到 `removeIf` 表达不了时，显式使用 `Iterator` 是标准做法：`iter.next()` 与 `iter.remove()` 配合，保证「读一个、判一个、删一个」的节奏

不会被打乱。若删除逻辑很复杂，还有一种更朴素的策略：先把要删的元素收集到另一个列表，遍历结束后再统一删除。

【代码对比解说】 关键区别在于**谁掌握迭代状态**。for-each 把迭代状态藏起来了（这正是它简洁的原因），同时也把「不允许中途修改」变成了一条隐性契约；一旦违反，库只能通过 `modCount` 检查在下次 `next()` 时抛异常来「报警」。显式 `Iterator` 把状态交回你手上，于是你有了合法修改的入口。这三段代码在功能上等价，在安全性上差别巨大——这正是「用库提供的正确抽象，而不是自己拼装」这一工程直觉的体现。

【设计原则透视】 这组对比体现了本讲「可变 vs. 不可变」的实用推论：**共享可变状态是 bug 之源**。迭代器与集合之间共享着一个可变的状态（当前位置），外部修改破坏了迭代器依赖的不变量（类似于 Reading 11 中的表示不变量被外部代码破坏）。最彻底的解法是避免可变性——例如让方法返回一个**新** List 而不是原地修改（`filter` 式的函数式风格），这也预告了 Reading 16（map/filter/reduce）的主题。

场景 3：用字符串常量表示有限取值 vs. 用枚举

❌ *错误代码*

```
// 错误：用字符串常量表示有限的颜色集合
public class PenBuggy {
    public static final String RED = "RED";
    public static final String GREEN = "GREEN";

    private String drawingColor = "REd";      // 拼写错误，编译器不报

    public void setDrawingColor(String color) {
        this.drawingColor = color;            // 可以传入 "purple"、""、null……
    }

    public static void main(String[] args) {
        PenBuggy pen = new PenBuggy();
        pen.setDrawingColor("bleu");          // 静态检查完全帮不上忙
        System.out.println(pen.drawingColor);
    }
}
```

【错误代码的问题】 1. 拼写错误无法被发现：`"REd"` 与 `"RED"` 都是合法字符串，错误要等到运行期比较失败时才暴露。 2. 类型太宽：任何字符串都能通过 `setDrawingColor`，包括拼错、空串甚至 `null`。 3. 语义不可读：`setDrawingColor("3")` 这样的调用看不出意图，读者需要去查常量表。 4. 「有限集合」这一重要的设计约束完全没有被代码表达出来，也就无法被编译器守护。

✅ *正确代码*

```
// PenColor.java —— 枚举是一个独立的新类型
public enum PenColor {
    BLACK, GRAY, RED, PINK, ORANGE,
    YELLOW, GREEN, CYAN, BLUE, MAGENTA;
}
```

```
// Pen.java
public class Pen {
    private PenColor drawingColor = PenColor.RED;

    public void setDrawingColor(PenColor color) {
        this.drawingColor = color;
    }

    public PenColor getDrawingColor() {
        return drawingColor;
    }

    public static void main(String[] args) {
        Pen pen = new Pen();
        pen.setDrawingColor(PenColor.RED);
        // pen.setDrawingColor("bleu");    // 静态错误：String 不能转换为 PenColor
        // pen.setDrawingColor(PenColor.REd); // 静态错误：找不到符号 REd
    }
}
```

【为什么这样更好】 枚举把「这个类型只有这十种合法取值」这条设计约束编码进了类型系统，于是拼写错误、类型不匹配、传入域外值全部变成**编译错误**。这正是 Reading 01 中「把假设交给编译器检查」的最佳实践：能静态检查的假设，绝不留到运行期。

【代码对比解说】 三种表示方式的安全性依次递增：`int` 常量（`int month = TUESDAY;` 不报错，且数值含义不可读）→ 字符串常量（能读懂，但拼写不受保护、取值域不受限制）→ 枚举（拼写受保护、取值域受保护、语义可读）。选择哪种取决于约束的强度：如果确实会出现「域外值」，就应当重新审视设计，而不是放宽类型。

【设计原则透视】 枚举是「用类型表达约束」的典范，属于三大目标中的两项之和：Safe from bugs（编译期拦截非法取值）与 Easy to understand（`PenColor.RED` 比 `2` 或 `"red"` 可读得多）。它也为 Reading 12（接口、泛型与枚举）打下基础——在那里我们会看到枚举可以有字段与方法。此外，枚举天然是**不可变**的有限值集合，因此可以作为 `Map` 的键与 `Set` 的元素安全使用。

场景 4：放弃泛型——用裸类型装不同类型

❌ *错误代码*

```
import java.util.List;
import java.util.ArrayList;

// 错误：使用裸类型（raw type），等于主动放弃静态检查
public class RawListBuggy {
    public static void main(String[] args) {
        List numbers = new ArrayList();       // 没有类型参数
        numbers.add(5);
        numbers.add("six");                   // 编译通过 —— 灾难
        int first = (Integer) numbers.get(0); // 需要显式强转
        int second = (Integer) numbers.get(1);// 运行期 ClassCastException
    }
}
```

【错误代码的问题】 1. 编译器不再检查元素类型，`"six"` 能混进一个「数字列表」，错误被推迟到运行期。2. 取出元素必须显式强转 `(Integer)`，每处强转都是一个潜在崩溃点。 3. 强转错误抛出的 `ClassCastException` 发生在**离错误源头很远**的地方（写入点在第 7 行，崩溃点在第 10 行），排查成本高。4. 代码读起来无法判断这个列表装的是什么，读者必须通读所有 `add` 调用。

✅ *正确代码*

```
import java.util.List;
import java.util.ArrayList;

// 正确：用泛型把元素类型写进类型，交给编译器检查
public class TypedList {
    public static void main(String[] args) {
        List<Integer> numbers = new ArrayList<>();
        numbers.add(5);                       // 自动装箱：int → Integer
        numbers.add(6);
        // numbers.add("six");                // 静态错误：不兼容的类型

        int second = numbers.get(1);          // 自动拆箱：Integer → int，无需强转
        System.out.println(second);           // 6

        // 不允许 List<int>：泛型参数必须是对象类型
        // List<int> bad = new ArrayList<>();  // 静态错误
    }
}
```

【为什么这样更好】 泛型把「这个容器里装的是什么」写进类型，于是错误在**写入的那一行**就被编译器拦下，而不是等到读取时崩溃。取出元素时类型已知，不再需要强转，代码既更安全也更简洁。

【代码对比解说】 注意 Java 的自动装箱/拆箱：`numbers.add(5)` 会自动把 `int` 包装成 `Integer`，`int second = numbers.get(1)` 会自动拆箱，因此泛型带来的语法负担几乎为零。但要注意拆箱的风险：**补充说明**，如果 `List<Integer>` 中某个元素是 `null`，拆箱会抛 `NullPointerException`。另外要区分「编译期类型」与「运行期类型」：泛型参数在运行期被擦除（type erasure），所以 `List<Integer>` 与 `List<String>` 在运行期是同一个类——这正是为什么不能写 `new T[]`、也为什么不能重载 `f(List<Integer>)` 与 `f(List<String>)`（后者是静态错误）。

【设计原则透视】 泛型是本讲所有内容中最纯粹地体现「静态检查」价值的一处：它把一条本来只能靠文档与约定维持的假设（「这个列表里都是整数」）变成了机器可验证的类型声明。它与 Reading 01 的 `Array<number>`、`List<int>` 讨论一脉相承，也直接连接到 Reading 12（泛型）与 Reading 10（抽象数据类型）——在那里我们关心的不再是「容器里装什么」，而是「这个数据类型的抽象行为是什么」。

场景 5：`final` 引用不等于不可变值——泄漏内部表示

❌ *错误代码*

```
import java.util.List;
import java.util.ArrayList;

// 错误：以为 final 就万事大吉，把内部可变列表直接交出去
public class PlaylistBuggy {
    private final List<String> songs = new ArrayList<>();

    public void add(String song) {
        songs.add(song);
    }

    public List<String> getSongs() {
        return songs;                  // 泄漏内部表示：调用者拿到的是同一个可变对象
    }

    public static void main(String[] args) {
        PlaylistBuggy p = new PlaylistBuggy();
        p.add("Yesterday");
        List<String> leaked = p.getSongs();
        leaked.clear();                // 从外部把「封装」好的状态清空了
        System.out.println(p.getSongs().size());   // 0 —— 内部状态被悄悄破坏
    }
}
```

【错误代码的问题】 1. `final` 只保证**引用**不被重新赋值，`songs` 指向的列表内容仍可被任何人修改。 2. `getSongs()` 把内部表示直接暴露出去（**表示泄漏，representation exposure**），封装形同虚设。 3. 外部代码可以在毫无提示的情况下破坏对象的不变量（例如「歌单里至少有一首歌」），这类 bug 极难定位。 4. 若将来把内部表示从 `List` 换成别的结构，所有依赖「拿到的是可变列表」的客户代码都会失效——违反 Ready for change。

✅ *正确代码*

```
import java.util.List;
import java.util.ArrayList;
import java.util.Collections;

// 正确：引用不可重新赋值 + 不把可变内部表示交出去
public class Playlist {
    private final List<String> songs = new ArrayList<>();

    public void add(String song) {
        songs.add(song);
    }

    /** @return an unmodifiable view of the songs in this playlist */
    public List<String> getSongs() {
        return Collections.unmodifiableList(songs);   // 补充说明：Java 标准库 API
    }

    /** @return a snapshot copy of the songs in this playlist */
    public List<String> snapshot() {
        return List.copyOf(songs);                    // 补充说明：Java 10+ API
    }

    public static void main(String[] args) {
        Playlist p = new Playlist();
        p.add("Yesterday");
        p.getSongs().clear();          // 运行期抛 UnsupportedOperationException
    }
}
```

【为什么这样更好】 返回 `Collections.unmodifiableList(songs)` 让调用者拿到的引用**不能修改**：任何写操作（`add`、`clear`、`set`）都会抛出 `UnsupportedOperationException`，从而把「不得从外部修改」这条规则变运行期为显式错误，而不是静默破坏。若调用者需要一份可以自由改动的数据，就给他 `List.copyOf(songs)`（一份独立的快照），这样他怎么改都不会影响原始对象。

【代码对比解说】 两种保护手段的差别值得记牢：`Collections.unmodifiableList` 返回的是一个**视图（view）**——它不复制数据，因此开销小，但它会**随原列表的变化而变化**（`p.add(...)` 之后视图里也能看到新元素）；`List.copyOf` 返回的是一个**副本**，与原列表彻底解耦。选择哪一个取决于语义：想表达「这是当前状态的只读视图」就用视图，想表达「这是此刻的快照」就用副本。不要为了「不可变」而盲目复制大集合，那是无谓的性能损失。

【设计原则透视】 这组对比精准地示范了本讲的核心区分：`final` 管的是「箭头不动」，不可变性管的是「气泡不变」，而**封装**管的是「气泡不许别人碰」。真正的安全来自三者配合：`private` 限制访问，`final` 固定引用，不可变返回类型切断外部修改路径。这正是 Reading 08（不可变性）的主题，而「内部表示不得泄漏」在 Reading 11（抽象函数与表示不变量）中会作为 RI 的一部分被正式定义：只要内部表示可能被外部代码随意改动，我们就根本无法为它写出任何有意义的 RI。

与其他设计原则的关联

- 与 **Reading 01（静态检查 / Static Checking）**：本讲是 Reading 01 的语法落地。`final` 对应 `const`，`List<Integer>` 对应 `Array<number>`，泛型把 Reading 01 中「静态检查面向类型」的结论变成日常写法；`int[100]` 的缓冲区溢出教训则直接解释了为什么要用 `List`。
- 与 **Reading 03（测试 / Testing）**：本讲场景 1 的 `==` 陷阱与场景 2 的迭代修改陷阱，都是「单元测试必须覆盖的划分」。互不相等的字符串（字面量 vs. `new String`）正是「字符串相等性」这一输入子域的边界情形。
- 与 **Reading 06（规格说明 / Specifications）** 和 **Reading 07（设计规格说明 / Designing Specifications）**：`Collections.unmodifiableList` 的注释其实就是一条后置条件；「前置条件该不该检查」「返回类型该不该用接口」都是规格设计问题。
- 与 **Reading 08（不可变性 / Immutability）**：本讲场景 5 提出的「表示泄漏」在那里得到系统解决；不可变类型如何带来安全性与线程安全性，是 Reading 08 的核心。
- 与 **Reading 10（抽象数据类型 / Abstract Data Types）**：本讲的 `List` / `Set` / `Map` 就是 Java 库中的 ADT；Reading 10 会解释「为什么用接口声明、用实现类构造」这一习惯背后的抽象思想。
- 与 **Reading 11（抽象函数与表示不变量 / Abstraction Functions & Rep Invariants）**：本讲的快照图是 RI/AF 的图示语言；`private final` 字段与「不泄漏表示」是维持 RI 的必要条件。
- 与 **Reading 12（接口、泛型与枚举 / Interfaces, Generics, Enums）**：本讲只介绍了这三者的基本用法，Reading 12 会把它们整合成完整的抽象与参数化设计工具。
- 与 **Reading 15（相等性 / Equality）**：本讲 `.equals()` 的正确语义、`Map` 键必须可哈希，都在 Reading 15 中展开为 `equals` / `hashCode` 契约。
- 与 **Reading 21–23（并发 / Concurrency、线程安全 / Thread Safety、锁 / Locks）**：本讲「共享可变状态是 bug 之源」的直觉，在并发中成为硬约束——不可变对象天生线程安全，可变对象必须加锁。

关键要点

- **对象比内容用 `equals()`，原始类型比值用 `==`**；`==` 用在对象上不会报错，只会安静地给出错误答案，因此必须靠习惯而不是靠编译器来防。
- **区分四个正交概念**：引用是否可重新赋值（`final`）、值是否可变（`String` vs `StringBuilder`）、变量作用域（花括号）、参数传递（按值传递引用）。
- **声明用接口，构造用实现**：`List<String> x = new ArrayList<>();`，让实现可替换。
- **把类型信息完整写出来**：用 `List<String>` 而非裸 `List`，用 `enum` 而非字符串常量，让编译器替你守护尽可能多的假设。
- **不要泄漏可变内部表示**：返回只读视图或快照副本；`final` 不解决封装问题。

常见陷阱与注意事项

1. **`==` 与 `.equals()` 混用**：字符串字面量的驻留让你「测出来是对的」。→ 后果：一旦换成从文件、网络或 `substring` 得到的字符串，逻辑立刻走错分支。

2. **以为 `final` 就等于不可变**：`final List<String> l = new ArrayList<>(); l.add("x");` 是合法的。→ 后果：误以为对象受到保护，实际内部状态仍可被任意修改。
3. **在 for-each 中增删元素**：`ConcurrentModificationException`，或者更糟——静默漏删。→ 后果：行为随集合实现与数据分布而变，难以复现。
4. **忘记原始类型集合的限制**：写 `List<int>` 或 `Set<int>` 会编译失败；用 `List<Integer>` 时若元素为 `null`，拆箱抛 `NullPointerException`。→ 后果：编译期报错（尚可接受）或运行期空指针（危险）。
5. **混淆 TypeScript 的 `let` 与 Java 的 `var`**：sp22 中的 `let` 对应 Java 的「声明 + 可重新赋值」，而 Java 10+ 的 `var` 只是类型推断，是静态类型。→ 后果：把 `var` 当成动态类型使用，写出难以理解的代码。
6. **把 API 文档当摆设**：不查 `assertEquals` 的参数顺序、不查 `Map.put` 返回的是旧值还是新值、不查 `replace` 返回 `boolean` 还是旧值。→ 后果：写出语义相反的测试或逻辑，且自己看不出来（详见 Reading 03 与 Reading 06）。

思考题（带答案）

问题 1：运行下面的代码，`s` 最终指向的内容是什么？`sb` 最终指向的内容是什么？请分别说明是「重新赋值」还是「改变值」。

```
public class Params {
    static void f(String s, StringBuilder sb) {
        s.concat("b");
        s += "c";
        sb.append("d");
    }

    public static void main(String[] args) {
        String t = "a";
        StringBuilder tb = new StringBuilder("a");
        f(t, tb);
        System.out.println(t + " / " + tb);
    }
}
```

答案：输出 `a / ad`。`t` 仍是 `"a"`：方法内 `s.concat("b")` 返回新字符串但返回值被丢弃（`String` 不可变，没有任何东西被改变）；`s += "c"` 让**局部变量** `s` 重新指向新对象 `"ac"`，这属于「重新赋值」，只影响方法内的那个箭头，方法返回后 `t` 的箭头毫无变化。`tb` 变成 `"ad"`：`sb.append("d")` 是「改变值」——局部变量 `sb` 与调用者的 `tb` 指向同一个对象，因此对该对象内部状态的修改对外可见。这组对比说明：**Java 的参数传递总是按值传递引用**；能影响调用者的只有「对共享对象的修改」，而绝不是「对参数重新赋值」。

问题 2：下面三个声明中，哪些在编译期就会失败？为什么？

```
List<int> a = new ArrayList<>();
List<Integer> b = new ArrayList<>();
Integer c = 5;
```

答案：只有第一个失败。`List<int>` 是静态错误，因为泛型类型参数必须是**对象类型（引用类型）**，不能是原始类型；必须写作 `List<Integer>`。第二个合法（菱形语法推断出 `ArrayList<Integer>`）。第三个也合法：Java 会自动装箱，把 `int` 字面量 5 包装成 `Integer` 对象。**补充说明**：`Integer c = null; int d = c;` 会在运行期抛 `NullPointerException`——自动拆箱不检查 `null`，这是包装类型相对于原始类型多出的风险点。

问题 3：一位同学写了 `private final List<String> names = new ArrayList<>();`，并认为「加了 `final`，所以 `names` 是不可变的，外部无法修改」。请指出这句话的两处错误，并给出修正方案。

答案：第一处错误：`final` 约束的是**引用**，不是**值**。`names` 这个箭头不能再指向别的列表，但 `names.add(...)`、`names.clear()` 完全合法，列表内容随时可变。第二处错误：即使字段是 `private`，只要把内部列表本身作为返回值交出去（`return names;`），外部代码就能通过这个引用修改它——封装被绕过，表示被泄漏。修正方案分两点：(1) 对外返回只读视图 `Collections.unmodifiableList(names)` 或快照副本 `List.copyOf(names)`；(2) 如果语义上确实要求「集合内容也不可变」，就改用 `List.of(...)` 构造，或把所有修改方法收拢到类自己的接口内。**补充说明**：`Collections.unmodifiableList` 返回的是视图，原列表变化时视图也会变化；`List.copyOf` 返回的是独立副本。选视图还是副本，取决于你想表达「只读视图」还是「当下快照」。

Reading 3: 测试（Testing）

> 说明：本讲 sp22 原版使用 TypeScript 与 Mocha，本笔记按用户要求提供 Java 代码示例与 JUnit 写法；方法与 API 与 sp21（6.031 Java 版）原文保持一致。

概述

本讲回答一个问题：**怎样选择测试用例，才能用最少的测试抓到最多的 bug？** 答案是**系统性测试（systematic testing）**：先对规格说明做**输入空间划分（partitioning）**，取每个子域中的**边界值（boundary values）**，再把测试写成**自动化（automated）** 的单元测试，用**代码覆盖率（code coverage）** 检查遗漏，用**回归测试（regression testing）** 锁住已修复的 bug。本讲的全部内容都服务于三大目标：测试直接对应 **Safe from bugs**；把测试策略写下来对应 **Easy to understand**；而「测试只依赖规格说明、不依赖实现」正是 **Ready for change** 的保证——因为实现可以在规格允许的范围内自由变化。

核心概念与设计原则详解

验证（Validation）的三种手段 – **定义与目的**：验证是「发现程序中的问题、从而提高对正确性的信心」这一更一般的过程。测试只是其中一种。它服务的质量目标是 Safe from bugs。 – **直观解释（"它是什么？"）**：验证就像确认一份文稿没问题：你可以**形式化证明**它正确（verification，构造正确性的形式化证明）、请别人**仔细通读**（code review，非形式化推理）、或者**实际跑一遍看结果**（testing）。 – **关键规则与最佳实践**： – **形式化验证（verification）** 手工做很繁琐，自动化工具支持仍是活跃研究领域；只有少数关键部件会被形式化验证（操作系统的调度器、虚拟机的字节码解释器、操作系统的文件系统）。 – **代码审查（code review）** 让另一个人仔细读代码并做非形式化推理，成本低、收益高（详见 Reading 04）。 – **测试（testing）** 在精心选择的输入上运行程序并检查结果。 – 即使做了最好的验证，软件仍然很难达到完美质量。典型**残余缺陷率**

（residual defect rate）：工业软件 1–10 缺陷/千行，高质量验证 0.1–1 缺陷/千行（成熟的浏览器 JS 库可达此水平），最顶级的安全关键系统 0.01–0.1 缺陷/千行（NASA 与 Praxis 这类公司可达）。 – 这个数字对大系统很打击人：100 万行工业代码按 1 缺陷/千行算，等于漏掉了 1000 个 bug。

为什么软件测试很难（Why Software Testing Is Hard） – **定义与目的**：理解三条「行不通的路」，才能理解为什么必须系统化地选测试用例。 – **直观解释（"它是什么？"）**：软件不像硬件——硬件可以抽样（测 1% 硬盘推断整批），可以用「加速老化」推断寿命（24 小时开关冰箱 1000 次）；软件的失败点分布**不连续、不离散**，抽样统计完全失效。 – **关键规则与最佳实践**： – **穷举测试（exhaustive testing）不可行**：测试空间通常太大。穷举测试一个 32 位浮点乘法 *a*b 需要 2^{64} 个用例。 – *随意测试（haphazard testing）（「随便跑一下看看行不行」）找到 bug 的概率低，也不增加我们对正确性的信心——它没有告诉我们任何关于未测部分的信息。 – 随机/统计测试（random or statistical testing）不适用于软件：物理系统的缺陷在空间上连续或均匀分布，所以抽样有意义；而软件行为在输入空间上是不连续、离散的。「系统在很大范围内的输入上都工作正常，然后在某一个边界点上突然失败」是常态。著名的 Pentium 除法 bug 大约每 90 亿次除法才出现一次；栈溢出、内存耗尽、数值溢出都是突然发生且每次都以同样方式发生，没有概率性变化。 – 因此：测试用例必须被仔细地、系统地选择*——这正是本讲的主线。

测试优先编程（Test-First Programming） – **定义与目的**：先写规格说明与测试，再写实现。它把「发现 bug」提前到「刚引入 bug 的那一刻」，是本讲最重要的工程实践。 – **直观解释（"它是什么？"）**：先想清楚「这个函数该做什么、怎么算做对了」，再动手写实现。就像建筑先出图纸再施工。 – **关键规则与最佳实践**： – **模块（module）**：软件中可以独立设计、实现、测试、推理的一部分。本讲聚焦以方法（函数）为单位的模块，后续 Reading 会扩展到类。 – **规格说明（specification / spec）**：描述模块的行为——参数类型与附加约束（例如 `sqrt` 的参数必须非负）、返回类型以及返回值与输入的关系。在 Java 中，规格说明由**方法签名 + 上方注释**组成。 – **实现（implementation）** 提供行为，**客户（client）** 使用模块；规格说明同时约束这两方。 – **测试用例（test case）**：一组具体输入 + 规格说明要求的预期输出行为。**测试套件（test suite）**：一组测试用例。 – 开发顺序：**Spec（写规格说明）→ Test（写测试）→ Implement（写实现）**。实现通过了测试，就完成了。 – 最大收益是 Safe from bugs：不要把所有测试留到最后，那时你有一大堆未经检验的代码，bug 可能在任何地方。

系统性测试的三个性质：正确、彻底、小（Correct, Thorough, Small） – **定义与目的**：系统性测试意味着「有原则地」选择用例，目标是设计出同时满足三个性质的测试套件。 – **直观解释（"它是什么？"）**：像选代表团的成员——每个族群（子域）都要有代表（彻底），但代表总数要少（小），而且不能把不该来的人算进来（正确）。 – **关键规则与最佳实践**： – **正确（Correct）**：测试套件是规格说明的**合法客户**，接受该规格的所有合法实现而不抱怨。这让你可以自由改动内部实现而无需改测试。 – **彻底（Thorough）**：能找出实现中真实存在的、程序员**很可能犯**的错误。 – **小（Small）**：用例少 → 写起来快、规格变化时好改、运行快 → 你会跑得更频繁。 – 对照三性质：穷举测试彻底但大得不可行；随意测试小但不彻底；随机测试要达到彻底必须付出巨大规模的代价。 – **心态转变**：写代码时你的目标是「让它工作」；设计测试套件时你的目标是「让它失败」。好的测试者会故意去戳程序最脆弱的地方。测试优先编程有助于获得这种「残酷的视角」，因为你在还没写出代码、不会把代码当作脆弱蛋壳的时候就已经戴上了测试的帽子。

输入空间划分（Partitioning the Input Space） – **定义与目的**：把输入空间拆成若干**子域（subdomain）**，使每个子域内部的输入行为相似，然后从每个子域取一个代表作为测试用例。这是「用少量用例覆盖大量行为」的核心技术。 – **直观解释（"它是什么？"）**：不去检查沙滩上的每一粒沙，而是按「沙粒大小」分成几类，每类抓一把看看。子域的名称来自数学：它是函数定义域（domain）的子集。 – **关键规则与最佳实践**： – **划分（partition）** 必须满足两点：子域两两**不相交（disjoint）**，且**完全覆盖（complete）** 输入空间——每个输入恰好落在一个子域中。 – 紧凑写法就是列出一串谓词：`// partition: a >= 0; a < 0`。 – **划分必须「正确」**：每个子域都要能被**合法**的测试用例覆盖。例如对规格要求 `x` 非负的 `sqrt`，划分成 `x < 0; x >= 0` 虽不相交且完整，但 `x < 0` 子域无法用合法输入覆盖，因此不是一个好划分。 – 划分应当围绕**规格说明中行为发生变化的地方**：`Math.max(a, b)` 的规格对不同子域有不同要求，所以取 `a < b; a > b; a = b`（注意必须有 `a = b`，否则不完整）。 – 有时按**输出**表达划分更方便：`// partition on a.multiply(b): 0, positive, negative` 是 `{(a,b) | a*b = 0}`、`> 0`、`< 0` 三个子域的简写。 – 对于多参数函数，把每个参数的关注点写成**多个独立划分**，让一个测试用例同时覆盖来自不同划分的多个子域，可以避免笛卡尔积爆炸（6×6 = 36 个用例降到 6 个）。但要注意：独立划分会漏掉参数间的**交互**，所以要**额外补一个捕捉交互的划分**（如「a 与 b 同号 / 异号 / 有一个为 0」）。

边界值（Boundary Values） – **定义与目的**：bug 常常出现在子域之间的边界上，因此边界值必须被写成**单元素子域**，从而保证测试套件必然包含它们。 – **直观解释（"它是什么？"）**：边界是行为「换挡」的地方，换挡处最容易出错。 – **关键规则与最佳实践**： – 常见边界：**0**（正负数之间）、数值类型的**最大/最小值**（`Integer.MAX_VALUE`、`Integer.MIN_VALUE`、`Long.MAX_VALUE`）、集合类型的**空值**（空字符串、空列表、空集合）、序列的**第一个与最后一个元素**。 – 为什么边界容易出 bug：程序员常犯 off-by-one（写 `<=` 而不是 `<`、计数器初值写 0 而不是 1）；某些边界需要被当作特例处理；边界是行为的不连续点。 – 加入边界后，原来的子域要**收缩以排除边界**。以 `Math.abs` 为例，完整划分是：`a = Integer.MIN_VALUE`；`Integer.MIN_VALUE < a < 0`；`a = 0`；`0 < a < Integer.MAX_VALUE`；`a = Integer.MAX_VALUE`，五个子域不相交且完整覆盖。 – Java 的 `Math.abs` 在 `Integer.MIN_VALUE` 上有一个**反直觉行为**：规格说明明确写「若参数等于 `Integer.MIN_VALUE`，结果是同一个（负）值」。原因是 -2^{31} 的相反数 2^{31} 超出了 `int` 的范围。这类「规格里写明的怪行为」正是最该测的边界。 – 对「字符串长度至多 5 且只含 'W'/'L'」的 `winLossRatio`，合适的边界包括 `""`（空串）、`"WWWWW"`（全胜）、`"LLLLL"`（全负）。

自动化单元测试与 JUnit（Automated Unit Testing） – **定义与目的**：**单元测试（unit test）** 测试单个模块，尽可能隔离。自动化让测试真正被执行——没人愿意手动跑一百次。 – **直观解释（"它是什么？"）**：把测试写成一个可执行的方法，断言（assert）期望结果；运行测试类，得到「全部通过」或「这些用例失败了：……」。 – **关键规则与最佳实践**： – Java 使用 **JUnit**：测试方法用 `@Test` 注解标注，方法体内调用被测量模块，然后用 `assertEquals`、`assertTrue`、`assertFalse` 等断言方法检查结果。 – **参数顺序至关重要**：`assertEquals` 的**第一个参数是期望值（expected）**，通常是个常量；**第二个参数是实际值（actual）**，即代码真正算出来的东西。所有比较值的 JUnit 断言都遵循「expected 在前，actual 在后」。写反了会导致失败信息令人困惑。 – **TypeScript 对照（sp22 原版写法）**：Mocha 的 `assert.strictEqual` 顺序**正好相反**——第一个参数是 **actual**，第二个是 **expected**。这是从 TypeScript 切到 Java 时最容易搞错的一点。 – 断言可以带可选的**消息字符串**作为最后一个参数，让失败信息更有用；一个测试方法里的断言失败

后该方法立即返回，但**其他 `@Test` 方法仍会运行**。 – 对返回结构（集合、数组）的比较，需要注意 `assertEquals` 只对内置类型可靠；**补充说明**：JUnit 5 的 `assertIterableEquals` 可以比较可迭代对象的内容，而手写断言应当检查**结构的关键性质**（如「集合大小为 1 且包含 hello」）而不是逐元素比对。

记录测试策略（Documenting Your Testing Strategy） – **定义与目的**：把用到的划分、子域，以及每个测试用例覆盖了哪些子域**写下来**。这让测试套件的彻底性对读者可见，直接服务于 Easy to understand。 – **直观解释（"它是什么？"）**：测试类顶部的注释就是这份测试套件的「说明书」，读它的人不必逐个反推用例意图。 – **关键规则与最佳实践**： – 在**测试类顶部**的注释里写下划分与子域：

```
public class MaxTest {
    /*
     * Testing strategy
     *
     * partition:
     * a < b
     * a > b
     * a = b
     */
}
```

- 每个测试方法上方写注释，说明它覆盖哪些子域：`// covers a < b`。
- 大多数测试套件会有多个划分，大多数测试用例会覆盖多个子域，例如 `// covers a = 1, b != 1, a and b have same sign`。
- 策略注释写在**测试类**里，规格说明（`@return ...`）写在**被测类**里，两者不要混。

黑盒测试与白盒测试（Black Box and Glass Box Testing） – **定义与目的**：**黑盒测试**只依据规格说明选择用例，不看实现；**白盒测试（glass box testing）** 借助对实现的了解来选择用例。前者保证测试正确性（不依赖实现细节），后者提高彻底性（覆盖实现特有的分支）。 – **直观解释（"它是什么？"）**：黑盒测试是「只看说明书办事」；白盒测试是「知道内部构造后专门去戳容易坏的地方」。 – **关键规则与最佳实践**： – 黑盒测试是测试优先编程的默认方式——你还没写实现，自然只能看规格。本讲的 `abs`、`max`、`BigInteger.multiply` 例子全是黑盒测试。 – 白盒测试的常见触发点：实现会**按输入选择不同算法**（例如 `sort` 根据 `values.size()` 在 `radixSort` / `quickSort` / `mergeSort` 之间切换，于是 0、9、10、1e9 这些阈值附近成为新边界）；实现有**内部缓存**（于是要测试重复输入）；内部表示有结构（例如 `BigInteger` 用十进制数字数组表示，则「一位数相乘」与「多位数相乘」是不同子域）。 – **白盒测试的纪律**：绝不能要求规格说明没有承诺的实现行为。若规格说明写「输入格式错误时抛出异常」，测试就**不应该**具体断言 `NullPointerException`，因为规格允许任何异常——否则你就限制了实现者的自由，测试也不再「正确」。

代码覆盖率（Coverage） – **定义与目的**：覆盖率用来判断测试套件**执行程序的充分程度**，是白盒测试的量化工具。它服务于 Safe from bugs（找出没测到的代码）与 Easy to understand（让测试缺口可视化）。 – **直观解释（"它是什么？"）**：覆盖率工具把每一行标成绿色（执行过）、红色（没执行过）、黄色（分支只走了一

个方向）。 – **关键规则与最佳实践**： – **语句覆盖（statement coverage）**：每个语句都被某个用例执行过吗？ – **分支覆盖（branch coverage）**：每个 `if` / `while` 的真假两个方向都被走过吗？ – **路径覆盖（path coverage）**：每条可能的路径（所有分支组合）都被走过吗？ – 强度关系：分支覆盖强于语句覆盖，路径覆盖强于分支覆盖。工业界常见目标是 100% 语句覆盖，但由于「不该到这里」这类**不可达的防御性代码**，实际上很少达到；100% 分支覆盖非常可取；安全关键行业有更严苛的标准（如 MC/DC，修正条件/判定覆盖）；**100% 路径覆盖不可行**，需要指数规模的测试套件。 – 标准做法是：先写黑盒测试，再测量覆盖率，然后针对**红色与黄色**的代码补测试用例。Java 生态常用工具是 Eclipse 的 **EclEmma**（Run → Coverage As），TypeScript 生态是 Istanbul/nyc。 – 以 `Hailstone` 为例：初始 `n = 3` 时序列为 3, 10, 5, 16, 8, 4, 2, 1，`n = n / 2` 这行会被执行（绿色）；初始 `n = 16` 时序列是 16, 8, 4, 2, 1，全是偶数，`n = 3 * n + 1` 这行**永远不会执行**（红色）；初始 `n = 1` 时循环体一次都不执行，整个循环体都是红色，而 `while (n != 1)` 这一行是**黄色**（条件永远为真、从未取过假方向）。

单元测试与集成测试（Unit and Integration Testing） – **定义与目的**：单元测试隔离地测单个模块；**集成测试（integration test）** 测多个模块的组合甚至整个程序。二者互补：前者定位快，后者能发现模块之间的连接问题。 – **直观解释（"它是什么？"）**：单元测试是「单独尝一尝每道工序的成品」；集成测试是「把整道菜端上桌尝一尝」。做披萨时，单独烤一张饼皮看它够不够脆 —— 单元测试；尝一口用现成香料做的酱 —— 单元测试；把酱和配料铺在饼皮上一起烤，看饼皮在潮湿配料下还能不能烤好 —— 集成测试。 – **关键规则与最佳实践**： – 隔离测试让调试容易得多：单元测试失败时，你可以更有信心 bug 就在这个模块里。 – 只有集成测试时，测试一旦失败，bug 可能在任何地方，必须到处排查。 – **典型错误**：测试 `extract()` 时先用 `load()` 把文件读进来，再把结果传给 `extract()`。这**不是** `extract` 的单元测试——失败时你无法判断是 `load` 还是 `extract` 有 bug。 – 正确做法：把文件内容写成**字面量字符串**直接传给 `extract()`。使用「贴近真实文件内容」的划分是合理的（因为这才是 `extract` 的实际用法），但不要真的调用 `load`。 – 对 `index()` 这类高层模块，隔离很难：调用 `index` 就同时测试了它内部调用的所有函数。这就是为什么必须先有 `load` 和 `extract` 的独立单元测试——它们让你能心安理得地把 bug 定位在 `index` 的连接逻辑里。 – 真要隔离高层模块，可以为它调用的模块写**桩（stub）**：例如 `load` 的桩完全不访问文件系统，无论传入什么 `File` 都返回固定的模拟内容。类的桩常被称为 **mock object**。桩是构建大型系统的重要技术，但 6.031 一般不用它。

自动化回归测试（Automated Regression Testing） – **定义与目的**：**自动化测试**指自动运行测试并检查结果；**回归测试**指每次改动代码后重跑全部测试。二者结合是现代软件工程的最佳实践。 – **直观解释（"它是什么？"）**：自动化测试是「一键跑完所有检查」；回归测试是「每次动完代码就按一下那个按钮」，防止「修好一个 bug、引入两个新 bug」。 – **关键规则与最佳实践**： – **测试驱动（test driver / test harness / test runner）** 不应是交互式程序（不能提示你输入、打印结果让你肉眼判断），而应当在**固定**的测试用例上调用模块并**自动**检查结果，输出「all tests OK」或「these tests failed: …」。JUnit 让你构建这样的驱动器。 – 自动化框架让「运行」变容易，但**测试用例仍要你自己想**——自动生成测试用例仍是活跃的研究课题。 – 每当你发现并修复一个 bug，就把引发它的输入**加入**测试套件，这叫作**回归测试用例（regression test）**。一个好测试的标准就是「它能引出一个 bug」——而每个回归测试都确实曾在某个版本上引出过 bug。 – 回归测试还能防止**回退（reversion）**：这个 bug 既然犯过一次，就很可能再犯。 – 这引出了**测试优先调试（test-first**

debugging)：bug 一出现，立刻写一个能引出它的测试用例并加入套件；修好之后，所有测试通过，调试结束，同时你永久得到了一个回归测试。 – 值得重跑全部测试的时机：`git add/commit/push` 之前、为了提高性能重写函数之后、使用覆盖率工具时、以及你认为自己修好了一个 bug 之后。

迭代式测试优先编程（Iterative Test-First Programming） – **定义与目的**：有效的软件工程不是线性流程。要在每一步都准备好**回头修改**前面的成果。 – **直观解释（"它是什么？"）**：规格说明、测试、实现三者互相校验，像三角架一样彼此支撑；任何一条腿发现问题，都要回去调整另外两条。 – **关键规则与最佳实践**： – 三步：写规格 → 写测试（发现问题就回头改规格与测试）→ 写实现（发现问题就回头改规格、测试与实现）。 – 写测试是**理解规格**的好方法：规格可能不正确、不完整、有歧义、漏掉边界情况；写测试能在你为错误规格浪费时间写实现之前发现这些问题。 – 反过来，写实现也可能让你发现测试缺失或错误，或促使你回过头修改规格。 – 因此不要在某一步追求完美再前进：**大规格**先写其中一部分，测它、实现它，再扩展；**复杂测试套件**先选几个重要划分做小套件，实现一个简单版本通过它，再逐步增加划分；**棘手实现**先写一个简单粗暴的版本（如用线性查找代替二分查找）来验证规格与测试，然后再写难的那个。 – 迭代需要不同于「一次做对」的心态：尽快得到一个粗略可用的解，然后持续精化，以便必要时推翻重做。这在问题困难、解空间未知时最省时间。

代码示例与对比分析

场景 1：交互式手动检查 vs. 自动化 JUnit 测试

❌ *错误代码*

```
import java.util.List;
import java.util.ArrayList;

// 错误：靠打印结果、用肉眼判断的「测试」
public class HailstoneMain {
    public static List<Integer> hailstoneSequence(int n) {
        List<Integer> list = new ArrayList<>();
        while (n != 1) {
            list.add(n);
            if (n % 2 == 0) {
                n = n / 2;
            } else {
                n = 3 * n + 1;
            }
        }
        list.add(n);
        return list;
    }

    public static void main(String[] args) {
        System.out.println(hailstoneSequence(3));    // 人工看一眼：好像是对的
    }
}
```

【错误代码的问题】 1. 结果需要**人眼判断**：没有断言，没有「通过/失败」，无法在 CI 中自动运行。 2. 不可重复、不可回归：改代码后必须有人记得再跑一次并再看一眼，而人会忘记。 3. 一旦输出变长（如 n=27 的 112 项），肉眼比对事实上不可行，人会「看上去差不多就放过」。 4. 无法表达「预期输出行为」：规格说明中「以 n 开始、以 1 结束」这类约束完全没有被检查，只检查了这一组输入恰好产生了这一串数字。

✅ *正确代码*

```
import java.util.List;
import java.util.ArrayList;
import java.util.Arrays;
import org.junit.Test;
import static org.junit.Assert.assertEquals;

public class HailstoneTest {
    /*
     * Testing strategy
     *
     * partition on n:
     *   n = 1
     *   n = 2
     *   n is an odd number > 1 (so the 3n+1 rule is used at least once)
     *   n is an even number > 2
     *   n is a power of 2 (the sequence decreases monotonically)
     */

    @Test
    public void testStartsAtOne() {
        // covers n = 1
        assertEquals(Arrays.asList(1), Hailstone.hailstoneSequence(1));
    }

    @Test
    public void testStartsAtTwo() {
        // covers n = 2
        assertEquals(Arrays.asList(2, 1), Hailstone.hailstoneSequence(2));
    }

    @Test
    public void testOddStart() {
        // covers n is an odd number > 1
        assertEquals(Arrays.asList(3, 10, 5, 16, 8, 4, 2, 1),
                     Hailstone.hailstoneSequence(3));
    }

    @Test
    public void testPowerOfTwo() {
        // covers n is a power of 2
        assertEquals(Arrays.asList(16, 8, 4, 2, 1),
                     Hailstone.hailstoneSequence(16));
    }
}
```

【为什么这样更好】 每个测试方法都把自己的预期**写成可执行的断言**，运行结果是二值的（通过/失败），可以一键重复执行、可以进入持续集成、可以在每次改动后作为回归测试。测试类顶部的策略注释让读者立刻看到划分与覆盖范围，无需反推每个用例的意图。而且一旦实现被改坏，失败信息会精确告诉你哪一条划分出了问题。

【代码对比解说】 `main` 加 `println` 的模式并非毫无价值——它适合探索性的临时试验。但它不是测试：它没有**预期值**，因此无法自动判断对错。这里的关键跃迁是把「我知道正确答案是什么」这一人类知识，固化成机器可检查的断言。注意 `assertEquals(expected, actual)` 的**参数顺序**：`Arrays.asList(2, 1)` 是期望值，`Hailstone.hailstoneSequence(2)` 是实际值，写反会让失败消息变成误导人的形式。

【设计原则透视】 测试驱动是**自动化**的具体形态，对应「Nothing makes tests easier to run, and more likely to be run, than complete automation」。从设计上看，这段测试同时展示了「正确」（只断言规格说明承诺的行为：以 n 开始、以 1 结束、中间项符合递推）与「彻底」（覆盖了偶数、奇数、幂次、`n = 1` 的边界）。至于 `assertEquals` 对 `List` 的比较：**补充说明**，JUnit 的 `assertEquals` 对 `List` 依赖 `AbstractList.equals` 的逐元素比较，因此上面的写法可用；对数组则必须改用 `assertArrayEquals`，因为数组的 `equals` 就是引用相等——这正是 Reading 15（相等性）要讲的内容。

场景 2：`assertEquals` 参数顺序写反

❌ *错误代码*

```
import org.junit.Test;
import static org.junit.Assert.assertEquals;

// 错误：把实际值写在第一个参数，把期望值写在第二个
public class AbsTestBuggy {

    @Test
    public void testNegative() {
        int result = Math.abs(-3);
        assertEquals(result, 3);          // actual 在前，expected 在后
    }

    @Test
    public void testNegativeIntMin() {
        int result = Math.abs(Integer.MIN_VALUE);
        assertEquals(result, Integer.MIN_VALUE);   // 边界：abs 可以返回负数！
    }
}
```

【错误代码的问题】 1. JUnit 的失败信息格式是「expected: but was: 」，参数写反后 X 与 Y 互换，**失败信息会误导**你去找相反的方向。 2. 对于「期望值和实际值恰好类型相同但语义相反」的场景（如 `assertEquals(result, 3)` 中 `result` 与 `3` 都是 `int`），**编译器完全无法提醒**，错误只能靠纪律避免。 3. 违反「所有比较值的 JUnit 断言都遵循 expected 在前」这一统一约定，让测试代码难以被快速扫读。 4. 第二个用例把 `Integer.MIN_VALUE` 当成期望值写在后面，掩盖了它其实是**边界测试**这一重要意图。

✅ *正确代码*

```
import org.junit.Test;
import static org.junit.Assert.assertEquals;

public class AbsTest {

    @Test
    public void testPositive() {
        // covers 0 < a < Integer.MAX_VALUE
        assertEquals(17, Math.abs(17));
    }

    @Test
    public void testNegative() {
        // covers Integer.MIN_VALUE < a < 0
        assertEquals(3, Math.abs(-3));
    }

    @Test
    public void testZero() {
        // covers a = 0
        assertEquals(0, Math.abs(0));
    }

    @Test
    public void testIntMinValue() {
        // covers a = Integer.MIN_VALUE
        // the spec permits a negative result at this boundary
        assertEquals(Integer.MIN_VALUE, Math.abs(Integer.MIN_VALUE));
    }

    @Test
    public void testIntMaxValue() {
        // covers a = Integer.MAX_VALUE
        assertEquals(Integer.MAX_VALUE, Math.abs(Integer.MAX_VALUE));
    }
}
```

【为什么这样更好】 期望值在前、实际值在后，是 JUnit 所有比较断言的统一约定；失败信息因此永远读作「我期待 X，但实际得到 Y」，方向正确、可读性强。此外，这组测试把 `Math.abs` 的**五子域划分**完整地写了出来（`Integer.MIN_VALUE`、负区间、0、正区间、`Integer.MAX_VALUE`），每个方法上方注明它覆盖哪个子域，彻底性一目了然。

【代码对比解说】 参数顺序问题看似琐碎，实则是「可诊断性」问题：测试失败时你需要的信息不是「程序算错了」，而是「算成了什么、应该是什么」。当测试套件有几百个用例时，一个方向被反转的失败消息可能让你浪费半小时。另外注意 `testIntMinValue` 这个用例：`Math.abs(Integer.MIN_VALUE)` **返回一个负数**，这是 Java 规格说明明确写出的行为（两补码表示导致 -2^{31} 的相反数超出 `int` 范围），一个凭直觉写出的「abs 永远非负」的测试会在这里失败——而这正是**边界值测试的价值**：它把「直觉」和「规格」的差异逼出来。

【设计原则透视】 这组对比与 Reading 01 的分界线呼应：`assertEquals` 的参数顺序属于「类型系统无法检查的约定」，只能靠纪律与代码审查（Reading 04）守护。同时，五个子域构成的划分展示了「正确」与「彻底」的平衡——它既没有写成穷举（不可能穷举 2^{32} 个 int），也不是随意挑几个值，而是围绕规格说明中**行为发生变化的地方**取样，并显式包含了两个极端边界。

（sp22 原版 TypeScript 写法对照：Mocha 的断言顺序正好相反）

```
// sp22 原版 TypeScript 写法（Mocha）
import assert from 'assert';

describe("Math.abs", function() {
  /*
   * Testing strategy
   * partition: a < 0; a = 0; a > 0
   */

  it("covers a < 0", function() {
    const result: number = Math.abs(-3);
    assert.strictEqual(result, 3);      // 注意：第一个参数是 actual，第二个才是 expected
  });

  it("covers a = 0", function() {
    assert.strictEqual(Math.abs(0), 0);
  });
});
```

把这两种框架并排看：**JUnit 是 `assertEquals(expected, actual)`，Mocha 是 `assert.strictEqual(actual, expected)`**——参数顺序完全相反。这是本讲跨语言时最容易出错的一个细节。另外注意 TypeScript 中的 `number` 无法表示 `Integer.MIN_VALUE` 那种边界（它是 64 位浮点），所以在 sp22 中 `Math.abs` 的边界讨论以 `Number.MAX_SAFE_INTEGER` 等常量呈现，而 Java 中则有 `Integer.MIN_VALUE` 这个「`abs` 会返回负数」的著名特例。

场景 3：断言信息不足——随机结果如何正确断言

❌ *错误代码*

```
import java.util.Set;
import org.junit.Test;
import static org.junit.Assert.assertEquals;
import static org.junit.Assert.assertTrue;

// 错误：对「有多个正确答案」的函数写出过强或过弱的断言
public class PickRandomlyTestBuggy {

    @Test
    public void testDrawFromSet() {
        Set<Integer> set = Set.of(293, 384, 10, 5, -3, 99);
        int result = pickRandomly(set);
        assertEquals(-3, result);        // 断言了一个实现未承诺的具体结果
    }

    @Test
    public void testDrawFromSetNoMessage() {
        Set<Integer> set = Set.of(293, 384, 10, 5, -3, 99);
        int result = pickRandomly(set);
        assertTrue(set.contains(result)); // 正确但失败时毫无线索
    }
}
```

【错误代码的问题】 1. `assertEquals(-3, result)` 断言了规格说明**没有**承诺的行为：`pickRandomly` 只承诺「返回集合中的某个元素」，任何合法实现都可能在这次调用中返回 293。这让测试**不正确**——它会拒绝合法实现。 2. 更糟的是，这种测试有时会「碰巧通过」（当实现恰好返回 −3 时），于是它既不可靠又给人虚假的安全感。 3. `assertTrue(set.contains(result))` 本身是正确的断言，但失败时只告诉你「断言失败」，不告诉你实际收到了什么值、期望来自哪个集合——调试时毫无线索。 4. 缺少对「返回值来自给定集合」这一关键性质的清晰表达，读者需要自己去推断测试意图。

✅ *正确代码*

```
import java.util.Set;
import org.junit.Test;
import static org.junit.Assert.assertEquals;
import static org.junit.Assert.assertTrue;

public class PickRandomlyTest {

    /*
     * Testing strategy
     *
     * partition on set:
     *   the set has exactly one element (only one legal answer)
     *   the set has more than one element (multiple legal answers)
     */

    @Test
    public void testSingleElementSet() {
        // covers the set has exactly one element
        Set<Integer> set = Set.of(42);
        assertEquals(42, pickRandomly(set));    // 唯一合法答案，可以精确断言
    }

    @Test
    public void testDrawFromSet() {
        // covers the set has more than one element
        Set<Integer> set = Set.of(293, 384, 10, 5, -3, 99);
        int result = pickRandomly(set);
        assertTrue("expected result to be from " + set + " but actually was " + result,
                   set.contains(result));
    }
}
```

【为什么这样更好】 对「有唯一合法答案」的子域用精确断言，对「有多个合法答案」的子域用**性质断言**（assertion on a property），并在消息里同时打印期望的来源集合与实际得到的值。这样测试既**正确**（不接受任何非法实现，也不拒绝任何合法实现），又在失败时**可诊断**。

【代码对比解说】 这里体现的是一条重要原则：**断言的强度必须恰好等于规格说明的承诺强度**。断言过强（`assertEquals(-3, result)`）会拒绝合法实现，让测试变成规格的额外约束；断言过弱（什么都不查）则抓不到 bug。带消息的 `assertTrue` 是恰当的中间地带：它只表达规格真正承诺的性质，同时提供足够的调试信息。**补充说明**：JUnit 5 提供 `assertAll` 可以把多条性质断言聚合成一次报告，避免「修好第一条才发现第二条也失败」的往返。

【设计原则透视】 这条原则直接对应「正确（Correct）」的定义——**测试套件是规格说明的合法客户**，它必须接受所有合法实现。这也解释了为什么**测试只依赖规格说明**是如此重要：一旦测试依赖实现细节，实现就无法自由变化，「Ready for change」随之丧失。这条线索将在 Reading 06（规格说明）中被形式化为「规格说明同时约束客户与实现者」的契约思想。

场景 4：不是真正的单元测试——测试之间发生耦合

❌ *错误代码*

```
import java.io.File;
import java.util.List;
import org.junit.Test;
import static org.junit.Assert.assertEquals;
import static org.junit.Assert.assertTrue;

// 错误：测试 extract() 时先调用 load()，两个模块被绑在一起
public class ExtractorTestBuggy {

    @Test
    public void testExtractWords() {
        File file = new File("testdata/document.txt");
        String doc = load(file);              // 依赖另一个模块！
        List<String> words = extract(doc);
        assertTrue(words.contains("hello"));
        assertEquals(3, words.size());
    }
}
```

【错误代码的问题】 1. 失败时**无法定位**：bug 可能在 `load`（文件缺失、编码错误、路径拼接错），也可能在 `extract`。 2. 测试依赖文件系统状态：换一台机器、换个工作目录、文件被改动，测试就失败——脆弱且不可移植。 3. 它不是 `extract` 的单元测试，而是 `load` + `extract` 的**集成测试**，只是被误当成了单元测试。 4. 覆盖率的意义被削弱：`extract` 的分支覆盖情况被 `load` 的行为遮蔽（例如文件读不到时 `extract` 根本没被调用）。

✅ *正确代码*

```
import java.io.File;
import java.nio.file.Files;
import java.nio.file.Path;
import org.junit.Test;
import static org.junit.Assert.assertEquals;

// load 的测试：单独处理文件输入，不牵涉 extract
public class LoadTest {
    /*
     * Testing strategy
     *
     * partition on file:
     *   the file is empty
     *   the file has exactly one line without a trailing newline
     *   the file has multiple lines, with a trailing newline
     */

    @Test
    public void testEmptyFile() throws Exception {
        // covers the file is empty
        Path p = Files.createTempFile("load-test", ".txt");
        assertEquals("", load(p.toFile()));
    }

    @Test
    public void testOneLine() throws Exception {
        // covers the file has exactly one line without a trailing newline
        Path p = Files.createTempFile("load-test", ".txt");
        Files.writeString(p, "hello");
        assertEquals("hello", load(p.toFile()));
    }
}
```

```
import java.util.List;
import java.util.Arrays;
import org.junit.Test;
import static org.junit.Assert.assertEquals;

// extract 的测试：输入是字面量字符串，完全不碰文件系统
public class ExtractTest {
    /*
     * Testing strategy
     *
     * partition on s:
     *   s is empty
     *   s has no words (only whitespace and punctuation)
     *   s has one word
     *   s has several words separated by whitespace and punctuation
     *   s starts or ends with punctuation
     */

    @Test
    public void testEmptyString() {
        // covers s is empty
        assertEquals(List.of(), extract(""));
    }

    @Test
    public void testNoWords() {
        // covers s has no words
        assertEquals(List.of(), extract("  ,;. "));
    }

    @Test
    public void testSeveralWords() {
        // covers s has several words separated by whitespace and punctuation
        assertEquals(List.of("hello", "world", "again"),
                     extract("hello, world!  again"));
    }
}
```

【为什么这样更好】 `extract` 的测试把输入写成**字面量字符串**，与文件系统彻底解耦：测试可移植、运行快、失败时你几乎可以确定 bug 就在 `extract` 里。`load` 有它自己的单元测试，专门覆盖空文件、单行、多行等划分。而对 `index`，我们再写一组（不可隔离的）测试来验证「连接逻辑」——因为前置的两个模块已经被各自验证过了，`index` 测试失败时我们可以把注意力集中在它调用各模块的方式上。

【代码对比解说】 关键区别在于**测试用例的输入从哪里来**。使用「贴近真实文件内容」的划分是完全合理的——因为那才是 `extract` 的真实用法；但不合理的是**真的去调用 `load`**。把自己的模块从依赖中解放出来，是单元测试的核心纪律。**补充说明**：当确实无法避免依赖（如网络、时钟、数据库）时，可以为它们写**桩（stub）**或 **mock object**：一个不访问真实外部资源、总是返回固定内容的替身。桩在大型系统中很重要，但 6.031 一般不用它。

【设计原则透视】 这一组对比把「单元测试 vs. 集成测试」的分工讲清楚了：单元测试负责把 bug 局部化到模块内部，集成测试负责发现**模块之间连接处**的 bug（例如 `index` 期待 `extract` 返回 `List<String>` 却拿到别的东西）。两者缺一不可，但顺序很重要——先有可靠的单元测试，集成测试失败时你才有信心把 bug 定位在连接逻辑上。这正对应 Reading 06/07 的抽象边界思想：模块之间的契约（规格说明）是连接处正确性的依据。

场景 5：遗漏边界值——`gcd` 的划分不完整

❌ *错误代码*

```
import org.junit.Test;
import static org.junit.Assert.assertEquals;

// 错误：只测「正常」输入，忽略了边界与负值
public class GcdTestBuggy {

    @Test
    public void testBothPositive() {
        assertEquals(6, gcd(54, 24));
    }

    @Test
    public void testAnotherPair() {
        assertEquals(1, gcd(17, 5));
    }
}
```

【错误代码的问题】 1. 划分不完整：只覆盖了「两个正整数且互质/不互质」这一小块，完全没有覆盖 `x = 0`、`y = 0`、负数、`x = y`、一个数整除另一个数等子域。 2. 规格说明写的是「x 与 y 不同时为 0」，也就是说**负数是被允许的**，而负数正是实现最容易写错的地方（取模的符号语义）。 3. 这个测试套件**小但不彻底**——它写起来很快，但几乎抓不到任何真实 bug，给人一种虚假的安全感。 4. 没有测试策略注释，读者无从判断覆盖面，也无从判断该补哪些用例。

✅ *正确代码*

```
import org.junit.Test;
import static org.junit.Assert.assertEquals;

public class GcdTest {
    /*
     * Testing strategy
     *
     * partition on (x, y):
     *   x and y are both positive
     *   x and y are both negative
     *   x and y have opposite signs
     *   x = 0 or y = 0 (but not both)
     *   x is divisible by y
     *   y is divisible by x
     *   x and y are relatively prime
     *   x = y
     * boundary values: x = 0, y = 0, Integer.MIN_VALUE, Integer.MAX_VALUE
     */

    @Test
    public void testBothPositive() {
        // covers both positive, x divisible by y
        assertEquals(6, gcd(54, 24));
    }

    @Test
    public void testRelativelyPrime() {
        // covers relatively prime
        assertEquals(1, gcd(17, 5));
    }

    @Test
    public void testOneIsZero() {
        // covers y = 0 (boundary)
        assertEquals(7, gcd(7, 0));
    }

    @Test
    public void testBothNegative() {
        // covers both negative
        assertEquals(6, gcd(-54, -24));
    }

    @Test
    public void testOppositeSigns() {
        // covers opposite signs
        assertEquals(6, gcd(54, -24));
    }

    @Test
    public void testEqualValues() {
        // covers x = y, and y divisible by x
        assertEquals(9, gcd(9, 9));
```

```
    }

    @Test
    public void testBothAtIntMinValue() {
        // covers the boundary Integer.MIN_VALUE
        assertEquals(Integer.MIN_VALUE, gcd(Integer.MIN_VALUE, Integer.MIN_VALUE));
    }
}
```

【为什么这样更好】 划分被完整地写出来并逐条覆盖，其中显式包含了 0、负号组合、相等、整除关系与 `Integer.MIN_VALUE` 这些最容易出 bug 的地方。测试策略注释让「彻底性」成为可见的事实：把划分与用例一一对照，就能看出有没有遗漏。注意最后一个用例：`gcd(Integer.MIN_VALUE, Integer.MIN_VALUE)` 的值是 -2^{31}，而它的绝对值超出 `int` 范围——如果实现内部用了 `Math.abs`，这个用例会立刻暴露 bug。

【代码对比解说】 从两个用例到七个用例，覆盖的是**行为差异**而不是「更多数字」。第二组的价值不在于「测得更多」，而在于每一个新用例都对应一个**实现可能采取不同代码路径**的子域：符号处理、零值处理、整除分支。这正是「划分 + 边界」相对「多写几个用例」的本质优势。注意 `gcd(7, 0) = 7` 来自规格说明中「x 与 y 不同时为 0」的约定，而 `gcd(0, 0)` 是**非法输入**（无法用合法的测试用例覆盖），因此不应写成一个测试用例——那会违反「测试套件必须是规格说明的合法客户」这一正确性要求。

【设计原则透视】 这组对比完整演示了「正确、彻底、小」三性质的权衡：把 `gcd(0, 0)` 加进来会让套件**不正确**（它不是合法客户）；只写两个正数用例会让套件**不彻底**；而七个用例是在彻底与「小」之间取得的平衡。边界值的存在还体现了一条设计直觉：**规格说明中特别提到的行为，一定是最值得测试的行为**（`Math.abs(Integer.MIN_VALUE)`、`gcd(x, 0)` 都是规格说明专门交代过的特例）。

场景 6：白盒测试越界——断言规格没有承诺的实现细节

❌ *错误代码*

```
import org.junit.Test;
import static org.junit.Assert.fail;

// 错误：断言了一个具体的异常类型，而规格说明只承诺「抛出异常」
public class ParserTestBuggy {

    @Test
    public void testBadlyFormattedInput() {
        try {
            parse("(((");
            fail("expected an exception");
        } catch (NullPointerException e) {     // 只接受 NPE
            // ok
        }
    }
}
```

【错误代码的问题】 1. 规格说明若只写「输入格式错误时抛出异常」，那么任何异常都是合法的；这个测试却**只接受** `NullPointerException`，等于给实现者加了一条规格里没有的约束。 2. 这让测试**不正确**：一个完全合法的实现（例如抛 `IllegalArgumentException`）会被判定为失败。 3. 更隐蔽的是：如果实现先把输入解析成内部结构再检查，抛出的异常类型可能随内部重构而变化，测试随之「无故失败」，逼迫开发者去改测试而不是改实现。 4. 它把白盒测试的信息（实现细节）**固化**进测试，直接损害 Ready for change。

✅ *正确代码*

```
import org.junit.Test;
import static org.junit.Assert.fail;

public class ParserTest {

    /*
     * Testing strategy
     *
     * partition on input text:
     *   text is well-formed
     *   text is empty
     *   text has unbalanced parentheses
     *   text has illegal characters
     */

    @Test
    public void testWellFormed() {
        // covers text is well-formed
        // ... assert the parsed structure
    }

    @Test
    public void testUnbalancedParentheses() {
        // covers text has unbalanced parentheses
        try {
            parse("(((");
            fail("expected an exception for unbalanced parentheses");
        } catch (Exception e) {
            // The spec permits any exception here, so we accept any.
            // (glass box knowledge: the current implementation happens to
            //  throw IllegalArgumentException, but the spec does not promise it.)
        }
    }
}
```

【为什么这样更好】 测试只断言规格说明承诺的性质（「会抛异常」），因此接受任何合法实现，实现者保有选择异常类型的自由。白盒知识仍然有用，但它被写在注释里作为**提示**，而不是写成断言——这样当实现重构、异常类型改变时，测试依然正确。

【代码对比解说】 这是本讲第一处提醒：**做白盒测试时必须小心，测试用例不能要求规格说明没有明确承诺的实现行为**。白盒测试的合法用法是「用实现知识去**选择**用例」（例如知道 `sort` 在 `size < 10` 时用

`radixSort`，于是专门测试 9、10、0 这些阈值），而不是「用实现知识去**写断言**」。区分这两件事，是白盒测试不越界的关键。

【设计原则透视】 这组对比把「正确」这一性质的深层含义揭示出来：**测试套件是规格说明的合法客户**，而不是实现者的助手。这条原则与 Reading 06（规格说明）中的「规格说明同时约束客户与实现者」完全同构，也是「Ready for change」的技术基础——只要测试只依赖规格，实现就可以在规格允许的范围内任意演进。

与其他设计原则的关联

- 与 **Reading 01（静态检查 / Static Checking）**：Reading 01 反复强调「静态检查抓不到与具体值相关的错误」，本讲正是填补这个空缺的系统方法——输入空间划分与边界值所覆盖的，恰恰是静态类型无法区分的那些具体值（除零、越界、`Integer.MIN_VALUE`、空集合）。
- 与 **Reading 02（基本 Java / Basic Java）**：本讲的测试代码大量使用集合与相等性；`assertEquals` 对 `List` 依赖 `equals`，对数组则必须用 `assertArrayEquals`，`Set` / `Map` 作为键的元素必须可哈希——这些都指向 Reading 15（相等性）。
- 与 **Reading 04（代码审查 / Code Review）**：本讲的「验证」包含代码审查这一手段；审查时检验「测试策略是否完整、断言强度是否恰当」是标准流程之一。
- 与 **Reading 05（版本控制 / Version Control）**：自动化回归测试是版本控制工作流的一部分——提交前重跑全部测试，是本讲明确列出的最佳实践。
- 与 **Reading 06（规格说明 / Specifications）** 和 **Reading 07（设计规格说明 / Designing Specifications）**：本讲的「正确测试套件 = 规格说明的合法客户」「白盒测试不得要求规格未承诺的行为」「前置条件与非法输入如何处理」都在那两讲被形式化。测试优先编程的「先写规格」一步也需要那两讲的写作技巧。
- 与 **Reading 08（不可变性 / Immutability）**：本讲场景 3 的 `Set.of(...)` 是不可变集合，能防止测试用例之间通过共享状态互相污染；不可变数据结构也天然让测试更容易推理。
- 与 **Reading 09（避免调试 / Avoiding Debugging）**：**测试优先调试**（bug 一出现就写一个引出它的测试）是那一讲的核心建议在本讲中的具体形态。
- 与 **Reading 10（抽象数据类型 / Abstract Data Types）** 与 **Reading 11（抽象函数与表示不变量 / Abstraction Functions & Rep Invariants）**：对 ADT 的测试不能只看单个方法，还要检验操作之间的交互与 RI 是否被维持——那是「更大模块」的测试策略。
- 与 **Reading 13（调试 / Debugging）**：当单元测试失败时，如何系统地缩小范围、形成假设、验证假设，是调试那一讲的主题。
- 与 **Reading 29（团队版本控制 / Team Version Control）**：持续集成（CI）在每次提交时自动运行回归测试，正是本讲「自动化回归测试」的工程化落地。

关键要点

- **先写规格与测试，再写实现**：测试优先让你在还没「爱上」自己的代码时就戴上残酷的测试帽子，也让规格说明中的歧义与遗漏更早暴露。

- **用划分与边界代替穷举与随机**：把输入空间切成不相交且完整的子域，从每个子域取一个代表，并把边界值写成单元素子域；多参数时优先用多个独立划分，再补一个捕捉交互的划分。
- **测试套件要满足正确、彻底、小**：正确的核心是「只依赖规格说明、接受所有合法实现」；彻底意味着覆盖程序员可能犯的错；小意味着写得快、改得快、跑得快。
- **断言强度必须匹配规格承诺强度**：有唯一答案时精确断言，有多个合法答案时断言性质并附上可诊断的消息；`assertEquals` 在 JUnit 中永远是「expected 在前，actual 在后」。
- **把测试策略写下来，让覆盖率指路，并让测试自动化地反复运行**：划分注释让彻底性可见；覆盖率（语句/分支/路径）用来发现漏测，路径覆盖不可行而语句与分支覆盖是实用目标；单元测试隔离地测模块、集成测试测连接，每修好一个 bug 就把引发它的输入加入回归测试。

常见陷阱与注意事项

1. **断言了规格没有承诺的实现行为**（如只接受 `NullPointerException`、断言随机函数返回某个具体值）。→ 后果：测试拒绝合法实现，实现者被迫为了通过测试而写「迎合测试」的代码，Ready for change 彻底丧失。
2. **把集成测试当单元测试用**（测试 `extract` 时先调用 `load`）。→ 后果：测试失败时无法定位 bug，且测试依赖文件系统等外部状态，脆弱且不可移植。
3. **边界值漏测或划分不完整**：只测「正常」输入，漏掉 0、空集合、`Integer.MIN_VALUE`/`MAX_VALUE`、字符串首尾元素；或写出 `Math.max` 只有 `a < b; a > b` 而丢掉 `a = b`、以及要求非负参数却划出 `x < 0` 子域这类不合格划分。→ 后果：off-by-one、符号处理、溢出这些最常见的 bug 全部漏网（`Math.abs(Integer.MIN_VALUE)` 就是典型），或者写出无法用合法输入覆盖的用例。
4. **`assertEquals` 参数顺序写反**。→ 后果：失败信息中的 expected/actual 互换，调试方向被误导，且编译期毫无提示。
5. **在迭代中修改集合来「构造」测试输入**，或让多个测试用例共享同一个可变集合。→ 后果：测试之间互相污染，出现「单独跑通过、一起跑失败」的诡异现象（对应 Reading 02 的迭代陷阱，建议使用 `Set.of`/`List.of` 等不可变字面量）。
6. **把「100% 语句覆盖率」当成质量保证**。→ 后果：覆盖率高但断言贫弱（「执行了但没检查」）的测试套件会给出虚假的安全感；覆盖率是发现遗漏的工具，不是正确性的证明。

思考题（带答案）

问题 1：下面这个函数的规格说明是「返回数组所有元素的布尔与」，实现如下。请指出它的 bug，说明为什么「随意挑选的测试用例」很可能发现不了它，并给出一个能发现它的、基于划分的测试套件。

```
/**
 * @param bits an array of 32 true/false values
 * @return the Boolean AND of all values in the array
 */
boolean andAll(boolean[] bits) {
    boolean result = bits[0];
    for (int i = 1; i < 31; i++) {
        result = result && bits[i];
    }
    return result;
}
```

答案：bug 在循环条件 `i < 31` ——它漏掉了下标 31 的元素（off-by-one），正确应为 `i < bits.length`（或 `i < 32`）。这个 bug 属于「只在特定输入上出现」的类型：如果数组的第 32 个元素恰好与前面所有元素的与相同（例如全是 `true`，或最后一个是 `true`），结果就是正确的；只有在第 32 个元素为 `false` 而前面全为 `true` 时才会出错。所以「随意挑几个用例」（如全 `true`、交替真假）都可能碰巧给出正确结果。穷举测试需要 2^{32} 个用例，随机测试发现的概率约为 $1/2^{32}$（只有「前 31 个全真且第 32 个为假」这一个输入会失败），几乎为零。基于划分的测试套件应包含：`bits` 全为 `true`（覆盖「与为真」的子域）、`bits` 全为 `false`、前 31 个为 `true` 而第 32 个为 `false`（**关键边界：决定结果的是最后一个元素**）、第 1 个为 `false` 其余为 `true`（覆盖 `bits[0]` 就决定结果的路径）、以及交替真假。其中「最后一个元素为 `false`、其余全 `true`」这一条来自 **边界值** 分析（序列的第一个与最后一个元素是经典边界），也是唯一能揪出该 bug 的用例。这个例子同时说明：**测试的价值来自划分与边界，而不来自用例数量**。

问题 2：某同学为 `sort(List<Integer> values)` 写了测试策略注释：「partition on values: 长度为 0、长度为 1、长度小于 10、长度在 10 到 10^9 之间、长度大于 10^9」。请评估这个划分，并说明它属于黑盒测试还是白盒测试。如果他是先写的测试（测试优先编程），这个划分可能吗?

答案：这个划分**不相交且完整**（任何列表长度恰好落入其中一个子域），但它显然来自**白盒测试**——因为 10 与 10^9 这两个阈值只在实现里才存在：实现根据 `values.size()` 在 `radixSort`（< 10）、`quickSort`（< 10^9）、`mergeSort`（否则）之间切换。单纯看规格说明（「把列表按非递减顺序原地排序」），没有任何理由认为 9 和 10、10^9-1 和 10^9 是行为分界点。因此后两个子域在长度 > 10^9 时**无法用合法测试用例覆盖**（现实中造不出这么大的列表），按「划分必须正确」的标准，它们是不合格的子域——至少在实践中不合格。如果是先写测试（测试优先编程），这个划分**不可能**产生：那时还没有实现，无从知道算法切换点，只能依据规格写出「空列表 / 单元素 / 多个元素 / 已排序 / 逆序 / 含重复元素」这类黑盒子域。这正说明两种测试的恰当分工：**测试优先阶段做黑盒测试**（只依赖规格、保证正确性），**实现完成后再用白盒知识补充用例**（借助覆盖率找出实现特有的分支），但补充时必须确保这些用例仍然只用合法输入，且不断言规格未承诺的行为。

问题 3：某项目中所有测试都是集成测试（整个程序端到端跑一遍），而且都通过了。为什么这门课仍然坚持要为每个模块写单元测试? 请结合「代码覆盖率」与「回归测试」各给出一条理由。

答案：第一，**覆盖率与彻底性**：端到端测试往往只走到每段代码的一条主要路径，`load` 的文件缺失分支、`extract` 的空字符串分支、`index` 的空文件集合分支几乎必然处于红色状态；而单元测试可以对每个模块独立地做划分与边界覆盖，把覆盖率补起来——模块内部的 bug 只有在直接以各种输入照射它时才会暴露。第二，**定位与回归**：集成测试失败时，bug 可能在程序任何地方，排查成本随模块数线性增长；而单元测试失败时你可以确信 bug 在该模块内，定位几乎不需要搜索。这两点叠加起来对「回归测试」尤其重要：本讲要

求「每修好一个 bug，就把引发它的输入加入测试套件」，如果只有集成测试，那么这些回归测试运行慢、失败信息模糊，很难在每次提交前跑完，最终就没人跑了——自动化回归测试的前提是**测试又小又快**，而这正是单元测试的优势。当然，集成测试也不可或缺：模块之间的**连接处**（如 `index` 期待 `extract` 返回的格式与实际不符）只有集成测试能发现。二者互补：单元测试把 bug 局部化，集成测试守住边界。

Reading 4: 代码审查（Code Review）

> 说明：本讲 sp22 原版使用 TypeScript，本笔记按用户要求提供 Java 代码示例；类型/API 与 sp21（6.031 Java 版）原文保持一致。凡属笔记额外补充的 Java 生态知识，均以「补充说明」标注。

概述

代码审查（code review）是由非作者本人对源代码进行的仔细而系统的研究，它和校对论文是同一种活动：目的不是证明作者愚蠢，而是让缺陷在最早、最便宜的时刻暴露出来。它有两个并列的目的——改进代码（找 bug、预判 bug、检查清晰度、检查是否符合项目风格标准）和改进程序员（彼此学习新的语言特性、设计变更与编码标准）。6.031 把代码审查当作贯穿全学期的工程实践，因为这套方法有扎实的实证支持：研究表明代码审查可以发现 70%–90% 的软件缺陷，Google 的流程甚至规定没有第二位工程师签字就不能把代码推入主仓库。

本讲的重点不是"审查的社交礼仪"，而是一份可操作的好代码清单：不要重复自己（DRY）、在需要的地方写注释、快速失败（fail fast）、避免魔法数字、每个变量只有一个用途、使用好名字、用空白帮助读者、不要用全局变量、函数应当返回结果而不是打印它们、避免特例代码。这十条规则全都直接服务于 6.031 的三大目标：**Safe from bugs（免于 bug）**——DRY 让一个 bug 只需修一处，快速失败让缺陷靠近源头被发现，避免全局变量让 bug 的影响范围被限制；**Easy to understand（易于理解）**——注释、命名、避免魔法数字、空白排版让读者不必反向工程作者的意图；**Ready for change（为变更而设计）**——DRY 与"返回结果而非打印"让代码能被用于作者当初没想到的新场景。一句话总结本讲的立场：代码审查是唯一能发现"晦涩难懂代码"的手段，因为只有另一个人真的去读它、试图理解它，晦涩才会被暴露。

核心概念与设计原则详解

代码审查（Code Review） – **定义与目的**：由代码作者之外的人对源代码做仔细、系统的检查。它同时服务于安全性（发现与预判 bug）、易理解性（发现晦涩代码）与可修改性（由有经验的开发者预判未来变化并建议防护措施）。 – **直观解释（"它是什么？"）**：把代码当成一篇要投稿的论文，审查者就是审稿人。审稿人不会替你重写，而是指出"这里读者会误解"、"这个假设没有写下来"、"这两段逻辑重复了，将来改一处忘另一处"。开源项目（Apache、Mozilla）与工业界（Google）都广泛采用它，Google 的规则是：没有另一位工程师在审查中签字，任何代码都不能进入主仓库。 – **关键规则与最佳实践**： – 不要替作者重排格式：审查是针对语义与可维护性，不是把你的个人风格强加给别人；擅自把每个模块都重新格式化，队友"会恨你，而且恨得有理"。 – 保持自洽并遵守项目约定——风格是个人选择，但项目一致性是团队义务。 – 区分"两类问题"：一类是风格（大括号放哪里，属于圣战级的口味问题），一类是能强化三大目标的实质规则（DRY、fail fast、命名等），后者才是本讲的重点。 – 审查是双通道学习：作者学到新技术，审查者也从别人的代码里学

到新写法，因此不要把它当作单向的挑错。 – 审查清单不是穷尽的；随着课程推进，规格说明、表示不变量、并发与线程安全都会成为新的审查素材。

快速失败（Fail Fast） – **定义与目的**：代码应尽可能早地暴露自己的缺陷。问题被观察到的时间越早、离成因越近，定位和修复就越容易。它主要服务 Safe from bugs。 – **直观解释（"它是什么？"）**：静态检查比动态检查失败得更快，动态检查又比"算出一个错误答案并污染后续计算"失败得更快。`dayOfYear` 就是个反面典型：如果按"月/日/年"以外的顺序传参，它不会报错，只会安静地返回一个错误答案。 – **关键规则与最佳实践**： – 优先用静态检查（类型、枚举、`final`）而不是运行时检查；能用类型表达的前置条件就不要留给注释。 – 不能静态表达时，用动态检查，并且**尽早**检查——进入业务逻辑之前就检查。 – 抛异常属于"动态失败"，返回 `-1`、`null` 或 `false` 属于"静默失败"，后者最慢，因为它把错误推迟到很远的地方才爆发。 – 用更强的类型承载语义：把 `int month` 换成 `enum Month { JANUARY, ..., DECEMBER }`，把参数的顺序错误变成静态错误；把 `int month` 换成 `String month` 也能让"顺序搞错 + 类型不符"变成静态错误。 – 在分支结构的 `else` 兜底处抛异常（而不是漏掉），可以把"漏掉一种情况"变成运行时的即时失败。

防御性编程 vs 快速失败（Defensive Programming vs Fail Fast） – **定义与目的**：防御性编程指代码主动处理"理论上不该发生"的输入或状态，不假设调用者守规矩；快速失败指代码在假设被破坏时立刻、响亮地失败。两者不是对立的口号，而是**边界之分**。 – **直观解释（"它是什么？"）**：对来自程序外部（用户输入、文件、网络、第三方库）的数据，你必须防御，因为那里没有契约可依赖；对程序**内部**的调用，契约由规格说明（Reading 6）保证，此时"防御"反而有害——它会把你本该知道的 bug 悄悄吞掉，让错误在下游以更难理解的形式出现。 – **关键规则与最佳实践**： – 内部代码：相信前置条件，违反前置条件时抛异常或断言失败（fail fast）。 – 外部边界：解析、校验、拒绝非法输入，并把失败转译成上层可以理解的异常（异常转译）。 – 绝不"静默修正"内部错误：不要 `if (month < 1) month = 1;` 这类"容错"，那会把 bug 变成错误答案。 – 断言（assertions）属于"开发期快速失败"：课程在 Reading 9（Avoiding Debugging）与 Reading 11（Abstraction Functions & Rep Invariants）中会把它作为检查前置条件与表示不变量的工具深入展开。 – 记住判据：**这个检查是在保护"我不信任的人"，还是在掩盖"我自己的 bug"？** 前者是防御，后者是自欺。

不要重复自己（Don't Repeat Yourself, DRY） – **定义与目的**：重复代码是安全风险——如果两份相同或相似的代码里藏着一个 bug，维护者很可能只修了其中一份。它同时服务三大目标。 – **直观解释（"它是什么？"）**：把复制粘贴想象成不看来车就横穿马路。复制的代码块越长，风险越高。`dayOfYear` 里"4 月有多少天"这个问题被写了多次，一旦历法变了，你得改很多处，而且总会漏掉一处。 – **关键规则与最佳实践**： – 数值重复、逻辑重复、结构重复都算重复；三种都要消除。 – 消除数值重复的办法是命名常量或数据表（如 `monthLength[month]`）。 – 消除逻辑重复的办法是循环或辅助函数（如把 `dayOfMonth += ...` 收敛为一次累加）。 – 不要硬编码你手工算出来的数（例如 59、90 这种"几个月天数之和"）；用可见地由其他命名常量计算出来的表达式。 – 重复出现三次以上的字面量就该被提取，"三振出局"是常见经验法则。

在需要的地方写注释（Comments Where Needed） – **定义与目的**：好注释让代码更易理解、更安全（重要假设被记录下来）、更可修改。注释分两类：规格说明（specification）与出处说明（provenance）。 – **直观**

解释（"它是什么？"）：注释是对"代码看不出来的信息"的补充。代码本身能表达"做了什么"，注释要表达"为什么这样做、假设了什么、来源是哪里"。把代码逐行翻译成英文的注释毫无价值，因为你应当假设读者至少懂 Java。 – **关键规则与最佳实践**： – 每个方法/类上方写 Javadoc 规格说明：`/** ... @param ... @return ... */`，把前置条件写进 `@param`，后置条件写进 `@return`（详见 Reading 6）。 – 抄来或改编的代码必须注明出处（这也是 6.031 协作政策的要求），既避免版权问题，也提醒后来者"这段代码可能过时或有已知缺陷"。 – 不要写"把代码翻译成英文"的注释：`++i; // increment i` 是噪声。 – 晦涩但正确的代码需要注释解释其数学/物理依据，例如 `// Gauss's formula for the sum of 1...n`。 – 更好的做法常常是改名字/改结构来消除对注释的需求：`const tmp = 86400; // number of seconds in a day` 应改成 `secondsPerDay`。

避免魔法数字（Avoid Magic Numbers） – **定义与目的**：除 0、1（有时 2）之外凭空出现的常量都叫魔法数字，因为它们"像从稀薄空气里冒出来"，没有任何解释。消除它们同时改善可读性与可修改性。 – **直观解释（"它是什么？"）**：读到 `if (month == 2)` 时，你无法确定 2 指一月、二月、三月还是公元 2 年。魔法数字让读者必须去猜，而猜错是 bug 的温床。`turtle.rotate(3)` 里的 3 到底是 3 度、3 弧度还是 3 整圈？光看调用点无法判断。 – **关键规则与最佳实践**： – 用命名常量替代字面量：`FEBRUARY` 显然比 `2` 可读。 – 常量之间常有依赖关系：`59` 与 `90` 是手算出来的和，应当写成由其他命名常量计算出的表达式。 – 即使是 π、G 这类"永恒常量"也值得命名：一是避免抄错数字（`3.14159265358979323846` 与 `3.1415926538979323846` 谁对？），二是精度、单位这类设计决策未来会变，命名常量更易改。 – 当魔法数字大量出现时，考虑把它们当成**数据**而不是常量，放进数组/映射等数据结构，让代码退化成一次查表（`monthLength[month]`）。 – 判断"0 是否是魔法数字"的准则：它是否承载了领域含义？`for (int i = 0; ...)` 与 `if (list.size() == 0)` 里的 0 是普适常识；`if (date.getMonth() == 0)` 里的 0 是"一月"，是魔法数字。

每个变量只有一个用途（One Purpose for Each Variable） – **定义与目的**：变量不是稀缺资源——随手引入、起好名字、不再需要就停用。复用参数或变量会让读者困惑，也埋下 bug。 – **直观解释（"它是什么？"）**：`dayOfYear` 把参数 `dayOfMonth` 复用成了"一年中的第几天"，两个含义完全不同的值共用一个名字。如果几行之后有人还需要"这个月几号"，信息已经丢失了。 – **关键规则与最佳实践**： – 参数默认不应被修改：未来的改动可能还需要知道传入时的原值。 – 用 `final` 修饰参数与尽可能多的局部变量，让编译器做静态检查（Java 支持 `final` 参数，这是 Java 相对 TypeScript 的一个便利之处）。 – 需要新含义时引入新变量，而不是覆盖旧变量。 – 循环变量只在循环内使用，不要借它传递跨循环的信息。 – 如果一个变量在方法中途"换了身份"，这是明确的重构信号。

使用好名字（Use Good Names） – **定义与目的**：好的方法名与变量名又长又自解释，往往能完全替代注释。它主要服务易理解性，也间接改善安全性与可修改性。 – **直观解释（"它是什么？"）**：`tmp`、`temp`、`data` 是糟糕的名字——每个局部变量都是临时的，每个变量都是数据，这类名字因此毫无信息量。名字应让代码"自己会说话"。 – **关键规则与最佳实践**： – 遵循语言的词法约定：Java 中类名首字母大写，变量名与方法名首字母小写，多词用 camelCase（`startsWith`、`getFirstName`），全局常量用 `ALL_CAPS_WITH_UNDERSCORES`。 – 方法名通常是动词短语（`getDate`、`isUpperCase`），变量名与类名通

常是名词短语。 – 避免缩写：`message` 比 `msg` 清楚，`word` 远胜 `wd`；很多队友的母语不是英语，缩写对他们更难。 – 避免单字符变量名，除非是公认惯例：笛卡尔坐标的 `x`、`y`，循环里的 `i`、`j`。 – 名字可以暗示类型或单位：`widthInPixels`、`secondsPerDay`、`bookTitle` 让读者不必去查声明。

用空白帮助读者（Use Whitespace to Help the Reader） – **定义与目的**：一致的缩进、行内空格、多行对齐能让代码结构对眼睛"显形"，主要服务易理解性。 – **直观解释（"它是什么？"）**：`dayOfYear` 把相加的数排成整齐的列，读者一眼就能比较相邻两行差在哪；`leap` 把整行条件挤在一起，读者需要逐字符解析。同一份逻辑，排版决定了它的可读性。 – **关键规则与最佳实践**： – 缩进保持一致；不要用制表符，只用空格字符（不同工具把 tab 展开成 2/4/8 个空格，`git diff` 或换编辑器看代码时缩进会全乱）。 – 二元运算符两侧加空格（`===`、`||`），让运算符"跳出来"。 – 多行条件对齐，使相似与差异一目了然。 – 用空行分隔逻辑段落，让方法结构（准备—计算—返回）自解释。 – 编辑器应配置为按 Tab 键时插入空格（新版 6.031 环境已默认如此）。

不要使用全局变量（Don't Use Global Variables） – **定义与目的**：全局变量既能从程序任何位置读取、也能从任何位置修改，因此 bug 的影响范围无法收敛。它主要损害 Safe from bugs 与 Ready for change。 – **直观解释（"它是什么？"）**：Java 中全局变量由 `public static` 声明：`public` 让它处处可见，`static` 让它只有一份实例。任何一段代码都能改它，任何一次调用都可能被上一次调用污染。 – **关键规则与最佳实践**： – 判据是两条同时成立：它是变量（值可变），且它是全局的（处处可读可写）。 – 加上 `public static final` 且类型不可变，它就变成全局常量——处处可读、永不重赋值或变异，风险消失。全局常量常见且有用。 – 把全局变量改写为参数与返回值，或放进对象里、通过对象的方法访问。 – 单例式"配置对象"也应尽量以参数形式显式传递，让依赖关系可见。 – 快照图（snapshot diagram）中要区分局部变量、实例变量、全局变量，因为它们的生命周期完全不同。

函数应当返回结果，而不是打印它们（Functions Should Return Results, Not Print Them） – **定义与目的**：把结果打印到控制台会锁死函数的使用场景——当结果要用于计算而非给人看时，函数必须重写。它主要损害 Ready for change。 – **直观解释（"它是什么？"）**：只有程序的最高层才应该与用户/控制台交互；低层组件应把输入作为参数接收、把输出作为返回值交付。调试输出是唯一例外，但它是调试手段而不是设计的一部分。 – **关键规则与最佳实践**： – 方法签名要能表达全部结果：`List<Integer>` 不够表达"计数 + 最长单词"两个结果，应返回一个不可变的小对象。 – 不要让方法既修改全局状态又打印——两个副作用都让测试困难。 – 单个结果就返回它，不要返回"是否成功"的布尔值而把结果留在输出参数里。 – 需要多个结果时，返回一个小类（Java 16+ 可用 `record`，见补充说明）。 – 打印只发生在程序的边界层（`main`、命令行前端、UI 层）。

避免特例代码（Avoid Special-Case Code） – **定义与目的**：为"看起来特殊"的输入（0、空数组、空字符串）单独写分支，会让代码更长、更难理解、更容易出现不一致。它损害全部三大目标。 – **直观解释（"它是什么？"）**：`countLongWords` 开头的 `if (words.length == 0)` 分支是多余的——省略它，空数组会让 `for` 循环什么都不做，最终照样打印 0。更糟的是：这个特例分支漏掉了通用代码里对 `longestWord` 的初

始化，于是"空数组"与"非空但没有长单词"这两种都应得到相同结果的输入，行为却不一致。 – **关键规则与最佳实践**： – 写下特例 `if` 之前先停下来：通用代码是不是本来就能处理它？往往是。 – 先写通用情形，再考虑特例；不要"先易后难"。 – 性能理由不成立时不成立：只有在有证据表明特例真的影响性能时才优化，否则只是增加复杂度和 bug 藏身处。 – 通用代码更短、假设更少、修改点更少，因此同时更安全、更好懂、更好改。 – 特例分支一旦存在，就必须与通用路径的结果保持一致——这正是 bug 最常出现的地方。

代码示例与对比分析

场景 1：计算一年中的第几天——魔法数字、重复、参数复用三宗罪

❌ *错误代码*

```
// 错误：魔法数字 + 复制粘贴 + 复用参数 dayOfMonth
public static int dayOfYear(int month, int dayOfMonth, int year) {
    if (month == 2) {
        dayOfMonth += 31;
    } else if (month == 3) {
        dayOfMonth += 59;                    // 59 是手工算出来的 31+28
    } else if (month == 4) {
        dayOfMonth += 90;                    // 90 是手工算出来的 31+28+31
    } else if (month == 5) {
        dayOfMonth += 31 + 28 + 31 + 30;
    } else if (month == 6) {
        dayOfMonth += 31 + 28 + 31 + 30 + 31;
    } else if (month == 7) {
        dayOfMonth += 31 + 28 + 31 + 30 + 31 + 30;
    } else if (month == 8) {
        dayOfMonth += 31 + 28 + 31 + 30 + 31 + 30 + 31;
    } else if (month == 9) {
        dayOfMonth += 31 + 28 + 31 + 30 + 31 + 30 + 31 + 31;
    } else if (month == 10) {
        dayOfMonth += 31 + 28 + 31 + 30 + 31 + 30 + 31 + 31 + 30;
    } else if (month == 11) {
        dayOfMonth += 31 + 28 + 31 + 30 + 31 + 30 + 31 + 31 + 30 + 31;
    } else if (month == 12) {
        dayOfMonth += 31 + 28 + 31 + 30 + 31 + 30 + 31 + 31 + 30 + 31 + 31;
    }
    return dayOfMonth;
}
```

【错误代码的问题】 1. 复用参数：`dayOfMonth` 先表示"这个月几号"，随后被改写为"一年中第几天"，两个含义不同的值共用一个名字，读者极易误解，任何后续需要原值的改动都做不到。 2. 大量魔法数字与手工求和：`59`、`90` 是作者心算的结果，读者无法验证；一旦历法变化（例如二月改成 30 天），需要改动的数字分散在多处，极易漏改。 3. 严重重复：`dayOfMonth += ...` 出现 11 次，"4 月有多少天"这个知识在多行里重复表达。 4. 不快速失败：参数顺序传错（例如非美式日期习惯的 `dayOfYear(9, 2, 2019)`）不会报任何错，只是安静地返回错误答案；没有任何注释说明 `month` 是 1–12 还是 0–11。

✅ *正确代码*

```
// 正确：命名常量 + 数据表 + 单一用途变量 + final 参数 + fail fast + Javadoc
private static final int[] MONTH_LENGTH =
    { 0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };  // 索引 1 = 一月

/**
 * Compute the day of the year.
 * @param month month of the year, where January=1 and December=12
 * @param dayOfMonth day of the month, where 1 &lt;= dayOfMonth &lt;= days in that month
 * @param year the year, in the Gregorian calendar
 * @return the day of the year, counting from 1; for example
 *         dayOfYear(2, 9, 2019) = 40
 * @throws IllegalArgumentException if month or dayOfMonth is out of range
 */
public static int dayOfYear(final int month, final int dayOfMonth, final int year) {
    if (month < 1 || month > 12) {
        throw new IllegalArgumentException("month out of range: " + month);
    }
    if (dayOfMonth < 1 || dayOfMonth > monthLength(month, year)) {
        throw new IllegalArgumentException("day out of range: " + dayOfMonth);
    }
    int dayOfYear = dayOfMonth;                 // 新变量承担新用途，参数不被污染
    for (int m = 1; m < month; ++m) {
        dayOfYear += monthLength(m, year);
    }
    return dayOfYear;
}

/** @return the number of days in month of year, where January=1 */
private static int monthLength(final int month, final int year) {
    if (month == 2 && isLeapYear(year)) {
        return 29;
    }
    return MONTH_LENGTH[month];
}
```

【为什么这样更好】 参数被声明为 `final`，编译器会阻止复用；`dayOfYear` 是一个全新的局部变量，用途单一；月份长度变成一张数据表加一次查表，重复的逻辑被 `for` 循环收敛为一次累加，重复的数字被 `MONTH_LENGTH` 收敛为一处定义；越界检查把"参数顺序传错"从"安静的错误答案"变成"立刻抛出的异常"；Javadoc 明确写了月份从 1 开始，消除了读者最大的猜测成本。

【代码对比解说】 两版代码在"正确输入下"的输出完全一致，差别全在**可维护性曲线**上：错误版本每增加一个特殊规则（闰年、历法改革）都要在多行里同步修改，且修改点之间没有结构约束；正确版本把"每个月几天"这个领域知识集中到一个地方，把"累加"这个算法收敛成一个循环。注意这里同时用到了两条本讲的原则：用数据表消除魔法数字，用循环消除逻辑重复。还要注意 `MONTH_LENGTH[0] = 0` 这个占位元素——它让 `MONTH_LENGTH[month]` 直接以 1 为下标，是"用一点空间换掉一次减法"的常见手法，但它本身必须靠注释或命名说清楚，否则下标从 0 还是 1 开始又变成了新的猜测点。

【设计原则透视】 这一组把 DRY、避免魔法数字、每个变量一个用途、快速失败四条规则叠在一起。用规格说明的语言说：错误版本隐式假设了"`month` 是 1–12"这一前置条件却从未写下，因此客户违反它时实现返回

垃圾值；正确版本把该前置条件写进 `@param`，并在实现中主动检查，把违反前置条件变成显式的动态错误——这正是 Reading 6（Specifications）中"前置条件是客户的责任"的具体落地。

场景 2：闰年判断——把数字当字符串、糟糕命名、隐晦逻辑

❌ *错误代码*

```
// 错误：靠字符串下标取十进制位、名字无意义、隐式魔法数字、逻辑难以核对
public static boolean leap(int y) {
    String tmp = String.valueOf(y);
    if (tmp.charAt(2) == '1' || tmp.charAt(2) == '3' || tmp.charAt(2) == 5
            || tmp.charAt(2) == '7' || tmp.charAt(2) == '9') {
        if (tmp.charAt(3) == '2' || tmp.charAt(3) == '6') return true;   /*R1*/
        else return false;                                               /*R2*/
    } else {
        if (tmp.charAt(2) == '0' && tmp.charAt(3) == '0') return false;  /*R3*/
        if (tmp.charAt(3) == '0' || tmp.charAt(3) == '4' || tmp.charAt(3) == '8') return tr
    }
    return false;                                                        /*R5*/
}
```

【错误代码的问题】 1. 隐式假设年份恰好是 4 位数字：`charAt(2)`、`charAt(3)` 在 `leap(916)`（3 位）或 `leap(10016)`（5 位）时会抛 `StringIndexOutOfBoundsException`，或读到完全错误的位置。 2. `tmp.charAt(2) == 5` 是真实的静默 bug：`charAt` 返回 `char`，与 `int` 字面量比较是合法的数值比较，`'5'` 的码点是 53，永远不等于 5。于是"十位是 5"这一支永远不会命中，代码不会报编译错误，只会算错。 3. 名字 `leap`、`tmp` 毫无信息量；`R1`—`R5` 这样的行号注释只告诉你"这是第几条返回"，不告诉你"为什么"。 4. 只支持公元 4 位数年份、无法表达闰年规则本身，读者必须做心算才能确认它对不对。

✅ *正确代码*

```
// 正确：命名好、规则直接可见、逻辑用算术表达、使用辅助方法消除重复
/**
 * Test whether a year is a leap year in the Gregorian calendar.
 * @param year a year in the Gregorian calendar; requires year &gt;= 1
 * @return true if and only if year is a leap year, i.e. divisible by 4
 *         but not by 100, unless it is also divisible by 400
 */
public static boolean isLeapYear(final int year) {
    return isDivisibleBy(year, 4)
        && (!isDivisibleBy(year, 100) || isDivisibleBy(year, 400));
}

/** @return true if and only if number is divisible by factor */
private static boolean isDivisibleBy(final int number, final int factor) {
    return number % factor == 0;
}
```

【为什么这样更好】 方法名 `isLeapYear` 是动词短语且自解释；规则直接写成算术表达式，与公历闰年定义一一对应，读者可以逐字核对；`isDivisibleBy` 消除了四处重复的取模逻辑；不再依赖字符串下标，因此对任意位数的年份都正确；剩下的数字 4、100、400 是**领域定义的一部分**——它们就是闰年定义本身，而不是"算出来的中间量"，因此可以接受为具名方法参数中的字面量（若想更进一步，也可声明为 `private static final int YEARS_PER_LEAP_CYCLE = 4;` 之类的常量）。

【代码对比解说】 错误版本的"聪明之处"（用十进制字符判断整除性）恰恰是它最大的问题：它把程序员的心理过程固化成代码，而把问题定义丢掉了。正确版本反过来——先写下定义，再让代码逐字对应定义。注意 `isDivisibleBy` 并没有让代码变短很多，但它把"取模等于 0"这一惯用法命名了，使 `isLeapYear` 的三行读起来像一句英文。这就是"好名字可以替代注释"的典型场景：正确版本几乎不需要行内注释，因为名字已经说明了一切。补充说明：Java 8 的 `java.time.Year.isLeap(int)` 已经实现了这个规格，实际项目中应优先复用标准库，而不是自己重写——这也是一条 DRY。

【设计原则透视】 这组对比同时体现易理解性（命名与结构）与安全性（消除静态无法发现、运行时静默出错的 `char == int` 比较）。它也与 Reading 1（Static Checking）呼应：错误版本把类型系统本来可以帮忙的地方（用 `int` 直接做算术）换成了字符串操作，从而主动放弃了静态检查的保护。此外 `isDivisibleBy` 是一个纯粹的函数——没有副作用、结果只依赖参数——这种"纯函数"是最容易测试、最容易推理的代码形态，与 Reading 6（Specifications）中"后置条件只谈参数与返回值"的理想规格一致。

场景 3：统计长单词——全局变量、打印结果、特例代码

❌ *错误代码*

```
// 错误：可变全局变量 + 打印结果 + 空列表特例分支（且特例与通用路径不一致）
public static int LONG_WORD_LENGTH = 5;      // 缺少 final：任何人都能改这个"常量"
public static String longestWord;            // 全局可变状态，处处可读写

public static void countLongWords(String text) {
    String[] words = text.split(" ");
    if (words.length == 0) {                 // 特例分支：多余，且漏了 longestWord 的初始化
        System.out.println("0");
        return;
    }
    int n = 0;
    longestWord = "";
    for (String word : words) {
        if (word.length() > LONG_WORD_LENGTH) ++n;
        if (word.length() > longestWord.length()) longestWord = word;
    }
    System.out.println(n);                   // 结果被打印，无法被程序复用
}
```

【错误代码的问题】 1. 全局可变状态：`longestWord` 在任何地方都能被读取和修改，调用一次之后结果会残留，两个线程或两次嵌套调用会互相污染，bug 的影响范围无法界定。 2. 结果被打印而不是返回：想把这个计数用于排序、写报告或聚合时就只能重写方法；单元测试也不得不捕获标准输出，测试变得脆弱（与 Reading 3 Testing 直接冲突）。 3. 特例分支与通用分支行为不一致：空数组时 `longestWord` 不会被重置为

`""`，于是"空数组"与"非空但没有长单词"这两种都该得到 0 的输入，留下了 `longestWord` 的残留值——这正是课程原文指出的隐藏 bug。 4. `LONG_WORD_LENGTH` 声明为 `public static`（而非 `public static final`），它仍然是一个全局变量。

✅ *正确代码*

```
// 正确：全局常量 + 返回结果对象 + 无特例分支 + 参数 final
public static final int LONG_WORD_LENGTH = 5;   // 全局常量：只读，风险消失

/** The two results of countLongWords, bundled into one immutable value. */
public static final class WordStats {
    private final int count;
    private final String longestWord;
    public WordStats(final int count, final String longestWord) {
        this.count = count;
        this.longestWord = longestWord;
    }
    /** @return the number of words longer than LONG_WORD_LENGTH */
    public int count() { return count; }
    /** @return the longest word in the text, or "" if there are no words */
    public String longestWord() { return longestWord; }
    @Override public String toString() {
        return count + " long words, longest = \"" + longestWord + "\"";
    }
}

/**
 * Count the words in text that are longer than LONG_WORD_LENGTH,
 * and find the longest word.
 * @param text words separated by single spaces
 * @return the count and the longest word; for empty text, count is 0 and
 *         the longest word is "" (the general case handles this correctly)
 */
public static WordStats countLongWords(final String text) {
    int count = 0;
    String longest = "";
    for (String word : text.split(" ")) {        // 空数组时循环体不执行，自然得到 (0, "")
        if (word.length() > LONG_WORD_LENGTH) ++count;
        if (word.length() > longest.length()) longest = word;
    }
    return new WordStats(count, longest);
}
```

【为什么这样更好】 没有任何全局可变状态，调用者拿到的是一个全新的不可变结果对象，两次调用互不影响；结果被返回而不是打印，因此可以被测试、聚合、格式化或在 GUI 中展示；特例分支被彻底删除，空输入自然走通用路径并得到与"非空但无长单词"完全一致的 `(0, "")`，那个隐藏的不一致 bug 随之消失；`LONG_WORD_LENGTH` 加上了 `final`，从全局变量升级为全局常量。

【代码对比解说】 关键判断是"两个结果怎么一起返回"。错误版本用"打印一个 + 全局变量存一个"的混合手法，代价是两种副作用都让复用和测试变难。正确版本用一个小类打包结果——这比返回 `Object[]` 或

`Pair<Integer, String>` 更好，因为 `WordStats` 有名字、有类型、有文档，字段含义不会在调用点丢失。补充说明：Java 16+ 可以用 `public record WordStats(int count, String longestWord) { }` 一行得到同样的不可变载体。还要注意正确版本里循环外只初始化一次计数器，而**没有任何**为 `words.length == 0` 写的分支——通用代码覆盖特例，正是"避免特例代码"的字面示范。

【设计原则透视】 这一组把"不要全局变量""返回结果而非打印""避免特例代码"三条规则串在一起，并且直接展示了它们共享的底层理由：**局部性**。状态越局部，能改变它的代码越少，bug 的影响范围就越小；结果越显式（通过返回值流动），调用者与实现者之间的契约就越清楚。这与 Reading 6（Specifications）中"后置条件要说明返回值与副作用"的要求一致：错误版本的方法签名 `void` 完全无法表达它的两个结果，规格说明根本写不出来；正确版本签名本身就承载了大部分后置条件。`WordStats` 的不可变性也预告了 Reading 8（Immutability）的核心论点：不可变对象可以被安全共享，无需防御性拷贝。

场景 4：注释与命名——冗余注释 vs 有信息量的规格说明

❌ *错误代码*

```
// 错误：把代码翻译成英文的注释、无信息量的名字、与代码重复的说明
public static int h(int n) {
    List<Integer> l = new ArrayList<>();
    int i = 0;
    while (n != 1) {          // test whether n is 1  <-- 无价值注释
        ++i;                  // increment i          <-- 无价值注释
        l.add(n);             // add n to l           <-- 无价值注释
        if (n % 2 == 0) n = n / 2; else n = 3 * n + 1;
    }
    l.add(1);
    return l.size();          // 返回值含义完全不明
}

int tmp = 86400;              // tmp is the number of seconds in a day（应当直接改名！）
```

【错误代码的问题】 1. 注释只是把代码念了一遍，读者得不到任何新信息，反而增加阅读负担；真正需要解释的地方（`n` 必须大于 0、序列定义来自 Collatz 猜想）却完全没写。 2. 名字 `h`、`l`、`i`、`n`、`tmp` 让人无法在不读完整个方法的情况下知道任何东西；`tmp` 那条注释实际上在说"这个变量应该改名叫 `secondsPerDay`"。 3. 没有规格说明，调用者不知道 `h(0)` 或 `h(-5)` 会发生什么（无限循环），也不知道返回值到底代表什么。 4. 参数 `n` 在循环中被反复修改，调用者无法从方法体内恢复原值。

✅ *正确代码*

```
// 正确：让名字承担解释职责，只在代码无法自述之处写注释，并把规格写成 Javadoc
private static final int SECONDS_PER_DAY = 86400;

/**
 * Compute the hailstone sequence.
 * See https://en.wikipedia.org/wiki/Collatz_conjecture
 * @param n starting number of the sequence; requires n &gt; 0
 * @return the hailstone sequence starting at n and ending with 1;
 *         for example, hailstoneSequence(3) = [3, 10, 5, 16, 8, 4, 2, 1]
 */
public static List<Integer> hailstoneSequence(final int n) {
    List<Integer> sequence = new ArrayList<>();
    int current = n;                        // 参数 n 保持原值不变
    while (current != 1) {
        sequence.add(current);
        current = (current % 2 == 0) ? current / 2 : 3 * current + 1;
    }
    sequence.add(1);
    return sequence;
}

// 只有在代码本身无法表达"为什么"时才写注释：
int sum = n * (n + 1) / 2;   // Gauss's formula for the sum of 1...n
// here we are using the sin x ~= x approximation, which works for very small x
double moonDiameterInMeters = moonDistanceInMeters * apparentAngleInRadians;
```

【为什么这样更好】 名字本身说明了一切：`hailstoneSequence`、`sequence`、`current`、`SECONDS_PER_DAY`，读者无需注释即可读懂；注释被用在真正需要的地方——数学公式的来源与近似条件的适用前提，这两处是代码无法自述的"为什么"；Javadoc 明确写出前置条件 `n > 0`（这正是 Reading 6 的规格说明），把"`h(0)` 会怎样"变成客户的责任而不是读者的猜测；参数被 `final` 保护，循环使用新变量 `current`。

【代码对比解说】 判断注释好坏的标准不是"有没有注释"，而是"这条注释是否提供了代码本身没有的信息"。翻译型注释的信息量为零；出处型注释（这段代码抄自哪里）、假设型注释（这里用了什么近似、在什么条件下成立）、规格型注释（前置/后置条件）的信息量很高。同样地，把 `tmp` 改名成 `secondsPerDay` 之后，那条注释就可以整条删除——这是"用更好的名字消除注释"的典型收益。注意正确版本里的 `? :` 条件表达式把 Collatz 规则放在一行，配合好名字反而比 `if/else` 更紧凑；这属于"排版服务读者"的取舍，团队一致即可。

【设计原则透视】 这一组把"注释"与"命名"两条规则捆在一起，落到 Reading 6（Specifications）的核心：注释中最重要的那一类就是规格说明，它把接口的假设变成可传递的契约。同时它体现了 Reading 4 与 Reading 3（Testing）的分工：有了 `@param n requires n > 0`，测试就知道不该去测 `h(0)`；反过来，如果没有规格，测试作者只能靠猜——这正是课程中"gcd 的规格在哪里"那组练习想说明的问题。

场景 5：防御性编程 vs 快速失败——静默容错还是立刻报错?

❌ *错误代码*

```
// 错误：对内部调用者的错误"温柔容错"，把 bug 变成错误答案
public static int dayOfYear(int month, int dayOfMonth, int year) {
    if (month < 1) month = 1;            // 静默修正：调用者的 bug 被吞掉
    if (month > 12) month = 12;          // 静默截断
    if (dayOfMonth < 1) dayOfMonth = 1;
    if (dayOfMonth > 31) dayOfMonth = 31;
    int total = dayOfMonth;
    for (int m = 1; m < month; ++m) total += 30;   // 又用 30 这个魔法数字掩盖了错误
    return total;
}
```

【错误代码的问题】 1. 静默容错把"调用者传错参数"这个 bug 转成了一个看起来合理的错误答案，错误会一路传播到很远的地方才爆发，定位成本极高（违反 fail fast）。 2. 参数被就地修改，违反"每个变量一个用途"和"参数不应被修改"的规则。 3. 用 `30` 这个统一的魔法数字代替真实月份长度，即使输入合法也算错，属于"为了容错而放弃正确性"。 4. 没有任何文档说明这些"修正"的存在，读者会以为方法是"宽容"的，从而更依赖这种未定义行为。

✅ *正确代码*

```
// 正确：内部代码相信前置条件，违反时立刻失败；外部边界处才做校验与转译
/**
 * @param month month of the year, where January=1 and December=12
 * @param dayOfMonth day of the month, 1 &lt;= dayOfMonth &lt;= days in that month
 * @param year the year; requires year &gt;= 1
 * @return the day of the year, counting from 1
 * @throws IllegalArgumentException if any argument is out of range
 */
public static int dayOfYear(final int month, final int dayOfMonth, final int year) {
    if (month < 1 || month > 12) {
        throw new IllegalArgumentException("month out of range: " + month);   // fail fast
    }
    if (dayOfMonth < 1 || dayOfMonth > monthLength(month, year)) {
        throw new IllegalArgumentException("day out of range: " + dayOfMonth);
    }
    int total = dayOfMonth;
    for (int m = 1; m < month; ++m) {
        total += monthLength(m, year);
    }
    return total;
}

// 只有真正的外部边界（这里是一个命令行前端）才做校验，并把失败转译成用户能懂的提示
public static void main(String[] args) {
    if (args.length != 3) {
        System.out.println("usage: DayOfYear <month> <day> <year>");
        return;
    }
    try {
        int month = Integer.parseInt(args[0]);
        int day   = Integer.parseInt(args[1]);
        int year  = Integer.parseInt(args[2]);
        System.out.println(dayOfYear(month, day, year));
    } catch (NumberFormatException e) {
        System.out.println("please enter numbers, not text");
    } catch (IllegalArgumentException e) {
        System.out.println("invalid date: " + e.getMessage());
    }
}
```

【为什么这样更好】 核心逻辑对"内部错误"零容忍：参数不合法立刻抛异常，错误在离成因最近的地方被报告；`final` 保证参数不被就地篡改；校验通过后，循环里使用的每一个月份长度都是真实值，正确性不再被"容错"牺牲。而**外部边界**（`main` 里的命令行解析）承担了防御与转译的职责：`Integer.parseInt` 的 `NumberFormatException` 与内部契约的 `IllegalArgumentException` 被转译成用户看得懂的提示，程序不会以难懂的堆栈崩溃。

【代码对比解说】 两段的差别不是"谁更健壮"，而是"谁把失败放在了正确的位置"。错误版本把容错放在最核心的计算逻辑里，代价是错误被掩盖；正确版本把防御集中在系统边界，核心逻辑保持纯粹和快速失败。这就是 6.031 的立场：**契约内部相信契约，契约边界验证契约**。异常转译（把低层异常转成高层异常）还额外带

来可修改性——将来核心逻辑换用别的日期库，边界层的提示语不用变。补充说明：若参数来自不可信来源，除抛异常外还应考虑日志与限流等措施，但那属于系统边界的设计范畴，不应侵入 `dayOfYear` 本身。

【设计原则透视】 这组对比把"快速失败"与 Reading 6（Specifications）的异常章节连起来：什么时候抛异常、抛什么异常、要不要写进规格说明，都是规格设计决策。它也预告了 Reading 9（Avoiding Debugging）中的断言：断言用于检查"本该永远为真"的假设（如表示不变量），而异常用于向客户报告可预期的失败——两者都是快速失败，但受众不同。进一步说，"谁会读这个错误"决定了手段：内部 bug 用断言/异常，用户输入用提示信息，二者不该混在一个方法里。

与其他设计原则的关联

- **与 Reading 1（Static Checking）**：快速失败有三个层级——静态检查快于动态检查，动态检查快于静默的错误答案。本讲中"用 `enum Month` 代替 `int month`""用 `final` 修饰参数"都是把检查提前到编译期的具体手段。`tmp.charAt(2) == 5` 这类静默 bug 之所以危险，正是因为它绕过了静态检查（`char` 与 `int` 的数值比较在 Java 中合法）。
- **与 Reading 2（Basic Java/TypeScript）**：本讲关于可变性、`static`、`final`、空值与非空的讨论，为后续 Reading 8（Immutability）的不可变对象与 Reading 6（Specifications）的 null 约定打基础。
- **与 Reading 3（Testing）**："函数应该返回结果而不是打印"直接决定了可测性：打印结果的方法无法用断言检查返回值，只能捕获标准输出，测试因此脆弱。代码审查与测试是互补的两道防线——测试发现行为错误，审查发现理解成本与修改成本。
- **与 Reading 5（Version Control）**：本讲反复强调"不要用注释保存旧代码"，其可行性完全依赖版本控制——历史版本已经在仓库里，删掉死代码比注释掉它更安全。同时，提交信息与差异审查是代码审查在时间维度上的延伸。
- **与 Reading 6（Specifications）**：本讲中最重要的注释类型就是规格说明。前置条件写进 `@param`、后置条件写进 `@return` 的做法在本讲只是点到，Reading 6 会给出契约的完整理论与异常、null、副作用的规定。
- **与 Reading 8（Immutability）**："不要全局变量"的正解之一是把可变状态收敛进对象；不可变对象可以被安全共享，因此"全局常量"是安全的，而"全局变量"不是。
- **与 Reading 9（Avoiding Debugging）**：防御性编程与快速失败的边界划分在本讲确立，Reading 9 会用断言（assertions）与表示不变量把它变成可执行的检查。
- **与 Reading 10–11（Abstract Data Types / Abstraction Functions & Rep Invariants）**：本讲要求规格说明不得谈论局部变量与私有字段；Reading 11 的表示不变量正是"私有字段上的约束"，两者共同构成"抽象边界"的两侧——对外是规格，对内是 RI。

关键要点

- **审查的两个目的同等重要**：改代码与改人。发现问题时优先解释"这条规则服务于哪个质量目标"，而不是宣布"这样写不好"。
- **快速失败优先于静默容错**：把检查尽量左移到编译期，其次放到方法入口，绝不用"修正参数"掩盖内部 bug；防御只发生在不可信边界。

- **DRY 是安全属性而非审美偏好**：任何一处重复都意味着未来有一个"只改了一半"的机会。用命名常量、数据表和辅助方法三件工具消除重复。
- **名字、空白、注释是给下一位读者的界面**：好名字消除注释，好排版暴露结构，注释只写代码无法自述的"为什么"，规格说明写前置/后置条件。
- **把状态和结果都局部化**：不用全局可变变量，不用打印代替返回，不为特例写分支——三条规则的共同目的是缩小 bug 的影响范围、扩大代码的复用范围。

常见陷阱与注意事项

1. **把"防御性编程"当成万能美德** → 在内部代码里做静默容错，调用者的 bug 被吞掉，错误在下游以完全不同的形式爆发，定位成本成倍增加。判据是：这个检查保护的是"我不信任的输入源"还是"我自己写的调用点"。
2. **用返回值 `-1` / `null` / `false` 表示失败** → 失败变成了一个看起来合法的值，客户端很容易忘记检查，于是错误数据继续向下流动。应采用异常或在签名中显式表达的"特殊结果"（见 Reading 6）。
3. **为"性能"提前写特例分支** → 代码变长、分支间行为可能不一致、bug 藏身处增多，而收益往往根本不存在（特例输入很少出现）。先写干净的通用算法，只有在有证据时才优化。
4. **把注释写成代码的英文翻译** → 注释与代码同步腐烂，读者多读一遍却没获得新信息；真正重要的假设（单位、范围、来源）反而没人记录。
5. **改写参数或复用变量** → 读者被"同名不同义"误导，未来需要原值的改动无法完成。用 `final` 参数与"一变量一用途"从语法上阻断这种写法。
6. **认为代码审查只是"找 bug"** → 忽略易理解性与可修改性的意见，团队会持续积累阅读成本；审查意见里"这段我看不懂"和"这里有个 NPE"同样重要。

思考题（带答案）

问题 1：下面的方法"看起来"很防御，请指出它的问题，并给出改进方案。

```
public static int countLongWords(String text) {
    if (text == null) return 0;
    if (text.length() == 0) return 0;
    String[] words = text.split(" ");
    int n = 0;
    for (String word : words) if (word.length() > 5) ++n;
    return n;
}
```

答案：三个问题。第一，`text == null` 的检查是"静默容错"：如果 `text` 是 `null`，那一定是调用者的 bug，返回 0 会让这个 bug 变成"这段文本没有长单词"的错误结论，违反快速失败；按照 Reading 6 的约定，参数非空是隐式前置条件，实现可以（也应该）让 `NullPointerException` 自然抛出，或显式 `Objects.requireNonNull(text)` 抛出更清楚的 `NullPointerException`。第二，`text.length() == 0` 是多余的特例分支——`"".split(" ")` 返回长度为 1、元素为 `""` 的数组，循环体同样不会计数，结果自然为 0；即使按其他语言的语义返回空数组，通用路径也照样得到 0。第三，`5` 是魔法数字，应提为 `public`

`static final int LONG_WORD_LENGTH = 5;`（或作为参数传入）。改进后的版本：保留 `final` 参数、删除两个 if、用命名常量、Javadoc 写明前置条件"text 非空"和返回值的定义。

问题 2：为什么 `leap()` 里的 `tmp.charAt(2) == 5` 不会产生编译错误，却会产生错误结果？这与"fail fast"有什么关系？

答案：在 Java 中 `char` 可以参与数值比较，`charAt(2)` 返回的 `char` 会被提升为 `int` 再与字面量 `5` 比较。字符 `'5'` 的 Unicode 码点是 53，因此 `'5' == 5` 恒为 `false`——这一支条件永远不会命中，编译器不会报错，程序也不会抛异常。它属于最慢的一类失败：**静默错误答案**。从 fail fast 的角度看，这段代码同时违反了"静态检查优先"（本该用 `int` 做算术，却退化成字符比较，把类型系统能帮的忙全部放弃）和"尽早暴露问题"（错误只在特定年份的结果里体现，可能需要很久才被发现）。修正方式是回到算术表达：`year % 4 == 0 && (year % 100 != 0 || year % 400 == 0)`。

问题 3：`countLongWords` 的"空数组特例分支"为什么不是"更健壮"，反而引入了 bug？请用"通用代码 vs 特例代码"的框架解释，并说明如何系统性地避免这类问题。

答案：特例分支与通用分支是两条独立演化的代码路径，只要它们对同一类结果负责，就必须保持行为一致——而人总会漏掉同步。原例中通用路径在循环前执行了 `longestWord = ""`，特例路径提前 `return` 跳过了它，于是"空数组"和"非空但没有长单词"这两种都应返回 0 的输入，在 `longestWord` 上产生了不同结果，这是典型的"分支不一致 bug"。系统性避免的办法有三条：一是先写通用路径并确认它能覆盖特例（本例中 `for` 循环对空数组天然不做任何事）；二是把特例视为"通用算法的退化情形"而不是"新的算法"，只在通用路径无法处理时才增加分支；三是如果确实必须保留快路径（例如排序对长度 0/1 的提前返回），也要让它与通用路径产生完全相同的外部可观察行为，并用测试覆盖边界输入。

Reading 5: 版本控制（Version Control）

> 说明：本讲 sp22 原版使用 TypeScript，本笔记按用户要求提供 Java 代码示例；类型/API 与 sp21（6.031 Java 版）原文保持一致。Git 命令与提交图沿用课程原文示例仓库 `ex05-hello-git` 的真实哈希值。

概述

版本控制（version control）回答的是一个看似朴素、实则决定团队生死的问题：**当我们不断修改项目时，如何可靠地保存、比较、回退、共享和合并这些修改？** 本讲先用"Alice 一个人写作业"的思想实验从零发明出版本控制需要支持的全部操作（回退到过去版本、比较两个版本、把完整历史推到别处、从别处拉回历史、合并同源分叉），再引入日志（谁、何时、改了什么）、分支与"多个共享位置"的概念，最后指出：这套东西就是 Git。

理解 Git 的正确姿势不是背命令，而是建立**两个模型**：一是 Git 把项目历史存成一张**有向无环图（directed acyclic graph, DAG）**，每个节点是一个提交（commit），也就是项目在那个时刻的完整快照；二是这张历史图只是完整**对象图（object graph）** 的骨架，完整对象图里还有代表"目录快照"的树对象与代表"单个文件内容"的 blob 对象，同一个文件版本被多个提交共享，从而避免重复存储。命令（`clone`、`add`、`commit`、`status`、`push`、`pull`、`log`、`show`）只是对这张图的操作。

版本控制直接服务于三大目标：**Safe from bugs（免于 bug）**——它能告诉你"什么时候、在哪里坏掉的"，让你发现有同类错误的其他位置，并给你信心"代码没有被人意外改动"；**Easy to understand（易于理解）**——提交日志回答了"这个改动为什么做""当时还改了什么""这段代码可以问谁"；**Ready for change（为变更而设计）**——版本控制本身就是关于"管理和组织变更"的：回退失败的改动、接受并整合他人的改动、把探索性工作隔离在分支上。它也承接 Reading 4（Code Review）：没有可靠的历史，就没有可信的"这次提交到底改了什么"可供审查。

核心概念与设计原则详解

为什么需要版本控制（Inventing Version Control） – **定义与目的**：版本控制系统（version control system, VCS）记录项目在时间上的全部版本，使回退、比较、共享与合并成为常规操作而不是危险动作。它服务全部三大目标。 – **直观解释（"它是什么？"）**：Alice 独自写作业，交作业前一分钟发现"最新改动把一切都搞坏了"，她只想回到过去的某个版本。于是她采用最朴素的纪律：把 `hello.ts` 存成 `hello.1.ts`、`hello.2.ts`、`hello.ts`，约定最新的那份就叫 `hello.ts`，这份最新版本称为**头（head）**。灾难避免了——但如果第 3 版里有好改动也有坏改动呢？她只能手工比较两个文件，把改动分类，再把好的搬回去。手工做得越多，出错概率越大。 – **关键规则与最佳实践**： – 一个人也需要版本控制：回退、比较、多机同步（笔记本与台式机通过云端交换）都是单人场景的真实需求。 – 一旦涉及多台机器或多个副本，就必须明确"谁覆盖谁"：Alice 在笔记本上做出 5L、在台式机上做出 5D，如果直接互相同步，就会丢掉一方的改动——她真正需要的是**合并（merge）**，基于两个第 5 版生成一个新版本。 – 判断"是否需要版本控制"的标准不是团队规模，而是"你是否在乎能不能回到过去、能不能说清改了什么"。 – 越是危险的操作（覆盖、删除、大重构），越应该先提交一个干净的状态。

版本控制系统应支持的操作与特性（Features of a VCS） – **定义与目的**：这些特性是本讲的"需求规格说明"，理解了需求才能理解 Git 的设计取舍。 – **直观解释（"它是什么？"）**：把版本控制当成一个必须同时满足"个人使用"与"多人协作"的数据结构服务。 – **关键规则与最佳实践**： – 可靠性：需要多久就保留多久，支持备份。 – 多文件：跟踪的是**项目**而不是单个文件——文件之间互相依赖，孤立地给单个文件编号会产生不一致的组合。 – 有意义的版本：每个版本要有"改了什么、为什么改"的记录（作者、时间、简短的人工撰写消息）。 – 回退、比较、查看历史：可以整体或部分恢复旧版本，可以对比版本差异，可以只看某个文件的历史。 – 不限于代码：散文、图片等一切文本或二进制资产都适用。 – 协作相关：合并同源分叉、追踪责任（哪一行是谁写的，即 annotate/blame）、支持并行工作、支持共享未完成的工作（work-in-progress）。 – 一致性约束：对每条日志记录，都应能方便地取出"当时完整可用的文件集合"——日志与实际文件集不能脱节。

集中式与分布式（Centralized vs. Distributed） – **定义与目的**：两种协作拓扑，决定了"什么算进入了版本控制"这个团队级判断。 – **直观解释（"它是什么？"）**：集中式系统（CVS、Subversion）有唯一的中央服务器，所有人的副本只与该服务器通信，协作图是一颗以中央仓库为心的星形：改动只有进了中央仓库才算"保存好了"，因为那是唯一的仓库。分布式系统（Git、Mercurial）允许任意协作图，团队与子团队可以各自实验替代版本与替代历史，觉得好再合并回来——**所有仓库生来平等**，是用户给它们分配不同的角色。 – **关键规则与最佳实践**： – 集中式的优点是规则简单；代价是中央仓库是单点，且离线工作受限。 – 分布式的优点

是灵活；代价是团队必须自己定义"什么算官方"：某个改动是否要先与指定协作者或服务器共享，才算被全队承认？ – 在 6.031 中，问题集仓库里所有仓库在 Git 眼里地位相同，但团队流程会赋予其中一个特殊角色（课程使用的托管仓库是布置与提交的官方位置）。 – 记住分布式的一个直接后果：**提交是本地操作**，`git commit` 不需要网络；只有 `push` / `pull` 才与远端交互。

版本控制术语（Terminology） – **定义与目的**：统一术语是讨论协作流程的前提。 – **直观解释（"它是什么？"）**： – **仓库（repository）**：项目所有版本的本地或远端存储。 – **工作副本（working copy）**：本地可编辑的项目拷贝，也就是你在编辑器里真正打开的那些文件。 – **文件（file）**：项目中的单个文件。 – **版本/修订（version / revision）**：项目在某一时刻内容的记录。 – **变更/差异（change / diff）**：两个版本之间的差别。 – **头（head）**：当前版本。 – **关键规则与最佳实践**： – 区分"仓库中的版本"与"工作副本中的文件"：仓库里存的是对象图中的对象，工作副本是"检出（check out）"出来的普通文件。 – 术语"分支（branch）"在 Git 中有特定含义（指向某个提交的可移动名字），与"历史图上的分叉"并不完全等同——分叉可以只是两个开发者从同一提交并行工作，而不需要任何人新建分支。 – 6.031 只使用默认分支 `main`（旧教程里可能叫 `master`，遇到时替换即可）。 – 与他人交流时讲清"我指的是哪个副本、哪个版本"，大多数协作事故来自这个歧义。

Git 对象图（The Git Object Graph） – **定义与目的**：Git 仓库由三部分组成——`.git` 目录、工作目录（working directory）、暂存区（staging area），而所有操作都是对存储在 `.git` 里的**图数据结构**的操作。 – **直观解释（"它是什么？"）**：想象一张图：历史图（提交 DAG）是骨架，每个提交节点挂着一棵目录树，树的叶子是文件内容的 blob。`git clone` 就是把这整张图从远端拷到本地 `.git`，再检出最新版本到工作目录；`git commit` 就是往图里加一个新节点。 – **关键规则与最佳实践**： – `git clone ssh://github.mit.edu/.../ps0-bitdiddle.git ps0` 做三件事：创建空的本地目录与 `ps0/.git`；连上远端并把对象图拷进 `ps0/.git`；检出 `main` 分支的当前版本到工作目录。 – 对象图在磁盘上以高效但人类不可读的形式存储；你编辑的文件是"检出"出来的普通副本。 – 图可以在多处存在副本（本地、远端托管）；它们是同一张图的拷贝，通过 push/pull 交换新节点。 – 不要把编辑器或 GUI 插件的菜单当作 Git 的唯一入口：出问题时课程助教无法帮你，因为工具改变了某些操作的实际语义；命令行的行为是公开、可复现的。

---

commit / tree / blob：提交、目录与文件内容（Commit, Tree, Blob） – **定义与目的**：这是 Git 数据模型的核心三层结构，直接解释了"Git 为什么这么快、这么省空间"。 – **直观解释（"它是什么？"）**：每个**提交（commit）** 是项目在那个时刻的**完整快照**，由唯一的十六进制 ID 标识；快照本身由一棵**树（tree）** 表示，树描述目录结构并指向子树的树对象或文件内容的 **blob** 对象。对任何规模合理的项目，绝大多数文件在一次修订中并未改变，因此 Git 不为每个提交复制一遍全部文件内容：**每个文件版本只存一次，多个提交共享同一份拷贝**。每个提交还带有日志数据（作者、时间、简短消息）。 – **关键规则与最佳实践**： – 提交 = 快照，不是差异；但 Git 默认向你展示差异，因为"大多数提交只改少量文件"这个统计事实让差异更有信息量。 – 内容寻址意味着相同内容只存一份：这正是 Reading 8（Immutability）所说"不可变数据可以自由共享"的绝佳范例——因为对象一旦写入就永不改变，共享它绝对安全。 – 想看清某个提交的完整快照，用 `git show <commit>:`（注意结尾的冒号，它彻底改变命令含义），想看某个文件在某个提交里的内容用 `git show <commit>:<path>`。 – 这条命令也是灾难恢复的最简单手段之一：从历史版本里取出被改坏的文件的完好版

本。 – 树/blob 是"存储实现"层的概念，日常不必直接操作，但理解它能让"为什么切换分支很快""为什么合并能自动进行"从魔法变成推理。

历史图：DAG、分支与 HEAD（History as a DAG） – **定义与目的**：项目历史是一张有向无环图，它是对象图的骨架。 – **直观解释（"它是什么？"）**：每个节点是一个提交；除初始提交外，每个提交都有一个指向父提交的指针（例如 `1255f4e` 的父是 `41c4b8f`，意思是后者先发生）。两个提交可以有同一个父提交——它们是从同一个先前版本分叉出来的两个版本（例如两位开发者各自独立工作）；一个提交也可以有**两个父提交**——这是把分叉的历史重新系在一起的合并提交。**分支（branch）** 只是一个指向某个提交的名字；而 HEAD 指向当前分支，当前分支再指向当前提交（所以 HEAD 是"指向当前提交——几乎是"）。 – **关键规则与最佳实践**： – 单人单机时历史图通常是一条序列：提交 1 → 提交 2 → 提交 3…… – 当多个提交共享同一父提交时，图从序列变为树（分叉）；当分叉被合并时，图从树变为真正的图（有节点有两个父）。 – "分叉"不要求任何人执行 `git branch`：只要两个人从同一提交各自继续提交，历史上就出现了分叉。 – 环是不可能的：历史上不可能存在"某个提交是自己祖先"的情形，因为时间只会向前，任何提交的父链必然终止于初始提交。 – 用 `git log --graph --oneline --decorate`（课程里常记作 `git lol` 别名）观察图形结构，比逐条读日志有效得多。

暂存区与提交（Staging Area, git add, git commit） – **定义与目的**：`git commit` 并不直接基于工作目录的当前内容创建提交，而是基于**暂存区（staging area，也叫 index）** 的内容。 – **直观解释（"它是什么？"）**：暂存区像是一个"半成品提交（proto-commit）"：你先用 `git add` 把想纳入本次提交的改动放进去，再用 `git commit` 把暂存区"刻成石头"。这让你能在一个混乱的工作目录里只提交一部分改动。 – **关键规则与最佳实践**： – 三步曲：修改文件 → `git add <file>` 暂存 → `git commit` 提交。 – `git status` 是核心工具，养成"每条 git 命令前后都跑一次"的习惯：它告诉你现在是"无改动""有未暂存改动""有已暂存改动"，以及本地是否领先于远端。 – 同一时刻可以**同时**存在已暂存与未暂存的改动；`git commit` 之后，已暂存的改动被提交，未暂存的改动原样保留。 – 暂存区让"一次提交只做一件事"成为可能（配合 `git add -p` 甚至能按代码块挑选）。 – 不要提交构建产物（Java 的 `.class`、TypeScript 的 `.js`）：它们是生成物，会持续与源码产生无意义的差异并制造冲突。

---

push、pull 与合并（push, pull, merging） – **定义与目的**：在多个仓库之间交换对象图的新节点，并把分叉的历史合并起来。 – **直观解释（"它是什么？"）**：`git clone` 时 Git 记住来源，把它命名为远端 **origin**。本地 `git commit` 产生新节点后，`git push origin main` 把它们送到远端；`git pull` 则反过来接收新节点**并且**更新工作副本（检出最新版本），如果远端与本地都变了，`pull` 会尝试合并。 – **关键规则与最佳实践**： – 典型并行场景：Alyssa 与 Ben 都从同一提交克隆；Alyssa 新建 `hello.scm` 提交 `6400936`，Ben 新建 `hello.rb` 提交 `82e049e`；两人各自的 `main` 指向不同提交。 – Alyssa 先 push 成功；此时 **Ben 的 push 会被拒绝**——如果服务器把 `main` 指向 Ben 的提交，Alyssa 的提交就会从项目历史里消失。 – Ben 必须先拉取（`git pull`），它做两件事：把新提交下载进本地对象图；把两段历史合并，产生一个新的合并提交（课程示例中的 `3e62e60`），这个提交和别的提交一样是一个快照——只是它同时包含了双方的改动。之后 Ben 才能 push。 – 当两人修改的是不同文件时，Git 能自动合并；若修改了同一文件的同一部分，Git

报告**合并冲突（merge conflict）**，需要人工把双方意图编织在一起后再提交合并结果。 – 自动合并成功 ≠ 语义正确：见下面的代码对比场景 1。

提交为什么"看起来像差异"（Why commits look like diffs） – **定义与目的**：理解"快照"与"差异"两种视角的切换，避免被 `git show` 的输出误导。 – **直观解释（"它是什么？"）**：`git show 1255f4e` 输出的是 diff，而不是完整快照——因为 Git 假设大多数文件在单次提交中没有变化，只显示差异更有用。这几乎总是对的，于是很多人误以为"Git 存的是差异"。 – **关键规则与最佳实践**： – `git show <commit>:` 列出该提交快照中的全部文件（课程示例输出 `hello.rb`、`hello.scm`、`hello.txt`）。 – `git show <commit>:<path>` 显示该提交中某个文件的内容，例如 `git show 3e62e60:hello.scm` 得到 `(display "Hello, version control!")`。 – 恢复被改坏的文件：用 `git show` 取出早期完好版本，这是最简单的灾难恢复手段之一。 – 记住"提交是快照、diff 是显示方式"这一点，才能理解为什么检出任意版本都很快，也才能理解合并需要的其实是"三个快照"（共同祖先与两个分支尖端），而不是"两串差异"。

快照图与提交图（Snapshot Diagram vs. Commit Graph） – **定义与目的**：两者都是"图"，但描述的是完全不同的世界，混淆它们是初学者的高频错误。 – **直观解释（"它是什么？"）**：**快照图（snapshot diagram）**画的是**程序运行时的内存状态**：变量、对象、指针、不可变引用、别名、栈帧——它回答"此刻堆和栈里有什么"（Reading 2 开始使用，Reading 8 讲不可变性时会大量使用）。**提交图（commit graph）**画的是**项目版本的演化**：节点是提交（整个项目的快照），边是父子关系——它回答"这些版本按什么顺序产生、在哪里分叉、在哪里合并"。前者的一个节点是一个对象，后者的一个节点是整个项目的一个版本。 – **关键规则与最佳实践**： – 快照图中的"不可变对象"要用双线箭头与双线框表示；提交图中的节点则总是不可变的——提交一旦创建就不能修改，这与 Reading 8（Immutability）中"不可变数据可安全共享"完全同源。 – 快照图解释"为什么两个变量是别名"，提交图解释"为什么两个分支能自动合并"。 – 提交图是有向无环图；快照图一般无环（除非你在画一个真的环形数据结构），两者的读图技巧不同。 – 讨论问题时先说明"我说的是快照图还是提交图"，可以省掉大量误解。

追踪责任与历史审查（Annotate / Blame, Review History） – **定义与目的**：日志使"某一行代码是谁、在哪个提交里引入的"可被自动查询。 – **直观解释（"它是什么？"）**：`git log` 可以限制到某个文件的历史；`git blame`（课程里也提到它"不幸地"叫这个名字）逐行标注责任人，出问题时知道该问谁。 – **关键规则与最佳实践**： – 看历史时优先用"图形 + 一行摘要 + 装饰引用"的视图，先建立整体形状，再看细节。 – 提交消息要写"为什么"，因为"改了什么"已经由 diff 记录了。 – `git log --follow` 可以跨越文件重命名追踪历史（补充说明：重命名检测是启发式的，重命名与内容大改混在同一提交里会让它失效——这正是下面场景 3 的教训）。 – 历史是给人读的：可读的提交序列是团队资产，拥塞的提交序列是负债。

代码示例与对比分析

场景 1：并行修改导致的"自动合并成功但语义错误"

假设两位开发者从同一个 `Hello.java` 出发。Alyssa 修改了 `greeting()` 的返回内容，Ben 修改了逗号放在哪里。

❌ *错误代码*

```
// 错误：起始版本把"问候语内容"和"标点格式"两件事耦合成两处独立可改的位置
public class Hello {
    public static void greet(String name) {
        System.out.println(greeting() + ", " + name);   // 标点在这里
    }
    public static String greeting() {
        return "Hello";                                  // 问候语在这里
    }
}

// Alyssa 的版本（改 greeting() 的内容）
//     public static String greeting() { return "Ciao"; }
// Ben 的版本（把逗号搬进 greeting()，让 greet() 不再拼标点）
//     System.out.println(greeting() + name);
//     public static String greeting() { return "Hello, "; }
// Git 自动合并后运行 Hello.greet("Eve") 的输出：
//     CiaoEve          <-- 未报任何静态/动态错误，但答案是错的
```

【错误代码的问题】 1. 自动合并成功但语义错误：Git 按"不同区域各自取新值"的规则合并，结果既用了 Alyssa 的 `"Ciao"`，又用了 Ben 的 `greeting() + name`，丢掉了逗号——产生 `CiaoEve`。这类"无错误、错误答案"的合并是最危险的一种，因为它不会有任何提示。 2. 职责耦合：格式（逗号与空格）与内容（问候语）分散在两个方法里，任何一方调整都会破坏另一方的前提，这是 DRY 原则在**语义**层面的违反（没有重复的代码，却有重复的知识）。 3. 缺少规格说明：`greeting()` 的契约到底"包含标点吗"从未写下来，两位开发者的理解不同，冲突只能在运行时暴露（这与 Reading 6 直接相关）。 4. 没有人审查合并结果：自动合并常被视为"Git 说没问题就没问题"，但 Git 只做文本层面的合并，不理解程序语义。

✅ *正确代码*

```
// 正确：用规格说明固定职责边界，让两处修改互不干扰、合并后语义正确
public class Hello {
    /**
     * Print a greeting to name.
     * @param name the name to greet; requires name is not null
     *        effects: prints exactly greeting() followed by ", " followed by name
     */
    public static void greet(String name) {
        System.out.println(greeting() + PUNCTUATION + name);   // 格式只有一处
    }

    /**
     * @return the greeting word, containing no punctuation
     *         for example, greeting() = "Hello"
     */
    public static String greeting() {
        return "Hello";                                        // 内容只有一处
    }

    private static final String PUNCTUATION = ", ";             // 命名常量，含义明确
}

// Alyssa 只改 greeting() 的返回值：return "Ciao";
// Ben 若想改标点，只改 PUNCTUATION = ": ";
// 两人的改动落在完全不同的位置，合并后输出 Ciao, Eve —— 语义正确
```

【为什么这样更好】 把"格式"与"内容"分别收敛到唯一的位置，使两类修改彼此独立，合并时不会互相吞掉对方的效果；`PUNCTUATION` 是命名常量，读者立刻知道它承担"标点与空格"的职责；Javadoc 明确写出 `greeting()` 返回**不含标点**的问候语，把隐式约定变成显式契约，客户与实现者从此不会各行其是；合并后的输出可被测试断言（`Ciao, Eve`）。

【代码对比解说】 这组对比的要点是：**合并是文本操作，正确性靠设计保证**。错误版本在文本层面看是两个不重叠的修改（一个改 `greeting()` 的返回字符串，一个改 `greet()` 的表达式），Git 有充分理由认为它们互补；但从语义看，二者都在定义"输出的格式"，属于同一个知识的两半。消除这种耦合有两种手段：一是把知识集中到单一位置（本例的 `PUNCTUATION`），二是把知识写进规格说明，让另一方能读到边界在哪里。实践中两者都要做——常量解决"改哪里"，规格解决"谁负责"。

【设计原则透视】 这组对比把版本控制与 Reading 6（Specifications）缝在一起：规格说明是"接口契约"，而合并冲突是"契约理解不一致"的物理表现。它也体现了 Reading 4 的 DRY 原则在知识层面的推广——重复的不只是代码，还有"谁负责加标点"这种设计知识。此外，它示范了 Reading 3（Testing）的必要性：合并后必须重新跑测试，因为 Git 不理解语义，"能自动合并"绝不等于"行为正确"。

场景 2：用注释保存旧实现 vs 删除它

❌ *错误代码*

```
// 错误：把历史塞进源文件，制造死代码与伪版本号
public class Greeter {
    public String greet(String name) {
        /* 2019-03-02 旧版本，先留着以防万一
        return "Hello, " + name;
        */

        /* v2 版本，2020-01-15，改用了 String.format
        return String.format("Hello, %s", name);
        */

        /* v3 版本 —— 2020-09-01，支持空名字
        if (name.isEmpty()) return "Hello!";
        return "Hello, " + name;
        */

        // 当前版本
        return "Hello, " + name + "!";   // v4 final final v2
    }
}
```

【错误代码的问题】 1. 死代码污染：四个版本堆在一个方法里，读者必须逐段排除才能找到真正生效的那一行；注释里的代码不会被编译器检查，会随时间腐烂成"看起来很权威的谎言"。 2. 文件名里的伪版本号（`v4 final final v2`）是手工版本控制的残留，既不可比较也不可回退——它恰恰证明了"没有版本控制时会发生的混乱"。 3. 无法比较、无法责任追踪：想知道"为什么支持空名字"、谁在什么时候改的，答案在注释里被压缩成了一句没有上下文的日期。 4. 违反 Reading 4 的 DRY 与"避免死代码"：同一逻辑的多个版本共存，任何 bug 修复都可能被误改到错误的那一份。

✅ *正确代码*

```
// 正确：源文件只保留当前实现，全部历史交给版本控制
public class Greeter {
    /**
     * Greet a person by name.
     * @param name the name to greet; requires name is not null,
     *        and name is not empty
     * @return a greeting of the form "Hello, <name>!"
     */
    public String greet(String name) {
        return "Hello, " + name + "!";
    }
}
```

```
$ git log --oneline -- Greeter.java
a1b2c3d Support empty names in greet
9f8e7d6 Rewrite greet using String.format
4c5b6a7 Initial greet implementation

$ git log -p --follow -- Greeter.java        # 随时取回任意历史版本的完整改动
$ git show 9f8e7d6:Greeter.java              # 甚至直接看当年那个文件长什么样
$ git revert a1b2c3d      # 撤销某个改动，并留下一个可审查的新提交
```

【为什么这样更好】 源文件只表达"现在是什么"，历史由仓库表达"曾经是什么、为什么变"；每一段旧实现都有作者、时间与提交消息，可以 `git show`、`git diff`、`git blame`，信息量远高于注释；删除旧代码是安全的——因为它在对象图里，不会丢；代码变短、变清晰，读者无需做考古。

【代码对比解说】 这组对比回答了一个非常常见的学生疑问："万一以后要改回来呢？"答案正是本讲的核心：**版本控制就是为这个问题而存在的**。没有版本控制时，注释掉旧代码是理性行为；有了版本控制后，它变成纯负债。注意 `git log -- <path>` 只列涉及该文件的提交，`--follow` 让它跨越重命名继续追踪，而 `git revert` 产生一个新提交而不是改写历史——后者对协作项目至关重要，因为改写历史会让别人的克隆失效。

【设计原则透视】 这条规则把 Reading 4 的"避免死代码"与 Reading 5 的"提交即快照"结合起来：可回退性由**对象图**提供，而不是由源码里的注释块提供。它同时依赖 Reading 8（Immutability）的思想——已提交的对象不可变，因此共享和检索它们绝对安全。反过来说，如果历史不可靠（例如把大量无关改动塞进同一个提交），"随时取回旧版本"的价值就会大打折扣，这正是下一组对比的主题。

场景 3：一个巨型提交 vs 一组原子提交

❌ *错误代码*

```
$ git commit -am "update"
# 这一个提交同时包含：
#   1) 把文件 Greeter.java 重命名为 Welcomer.java
#   2) 把方法 greet 改名为 welcome，并改变其行为（新增了感叹号）
#   3) 全文件重新格式化（缩进、括号位置）
#   4) 顺便把 MAX_NAME_LENGTH 从 20 改成 64
#   5) 删除了一段"看起来没用"的空值检查
#   6) 更新了 README 与三个测试文件
```

【错误代码的问题】 1. 无法审查：Reading 4 的代码审查在这种提交上完全失效——审查者面对上千行噪声，无法区分"语义改动"与"格式改动"，真正的风险改动（第 5 条）极易被忽略。 2. 无法回退：如果第 4 条被证明是错的，`git revert` 会把六件事一起撤销，包括那些正确的改动。 3. 责任追踪失效：`git blame` 会把整文件的行都归给这一次提交与这一个人，历史信息被抹平。 4. 重命名与内容大改混在一起，Git 的重命名检测（启发式）很可能判断失败，`--follow` 也就追不到历史——历史链在此断裂。

✅ *正确代码*

```
$ git commit -m "Rename Greeter to Welcomer (no behavior change)"
$ git commit -m "Rename greet() to welcome() (no behavior change)"
$ git commit -m "welcome() now appends '!' to the greeting"
$ git commit -m "Raise MAX_NAME_LENGTH to 64 to match the new ID format"
$ git commit -m "README: document the welcome greeting format"
# 每一步之后都运行一遍测试套件，确保每个提交自身是"绿色"的
$ git log --oneline
```

【为什么这样更好】 每个提交只做一件事，因此可以被独立审查、独立回退、独立理解；纯重命名与格式化提交不含行为变化，审查者可以快速扫过；行为变化的提交很短，diff 小到能被逐行读懂；`git blame` 与 `git bisect`（二分查找缺陷引入点，补充说明）都能给出精确答案；重命名单独成一步，重命名检测可以正常工作，历史链保持完整。

【代码对比解说】 核心原则是"提交是历史的原子单位"。判断一次提交是否原子，最快的标准是：**能不能用一句不含"并且"的话描述它？** 另一个标准是它能否被单独 revert 而不牵连其他意图。实践中常用的手法是在提交前用 `git add -p` 挑选代码块，把格式改动、重命名、行为改动分批暂存。注意格式化的批量改动最好单独成一次提交并在此之前与团队约定，否则它会淹没所有真正的改动——这也是 Reading 4 中"不要擅自重排别人的格式"在版本控制层面的又一次现身。

【设计原则透视】 这组对比把"提交历史"当作 Reading 4 代码审查的输入：审查的质量上限由提交的可读性决定。它也与 Reading 3（Testing）配合——每个提交保持测试通过，才能放心使用 `git bisect` 定位缺陷。更深一层，它体现了"为变更而设计"：一个组织良好的历史让回退、定位、理解的成本都从"小时级"降到"分钟级"。

场景 4：把环境相关的东西提交进仓库 vs 保持仓库可移植

❌ *错误代码*

```
// 错误：把开发者本机的绝对路径、编译产物与本地配置当作源码提交
public class ReportWriter {
    private static final String OUTPUT_DIR = "/Users/alice/6.031/ps1/build/out";
    private static final String DB_URL = "jdbc:postgresql://localhost:5432/alice_dev";
    private static final boolean DEBUG = true;   // 只对 Alice 有意义的本地开关

    public static void write(String text) throws Exception {
        java.nio.file.Files.write(
            java.nio.file.Paths.get(OUTPUT_DIR, "report.txt"),
            text.getBytes("UTF-8"));
    }
}
```

```
$ git status --short
A  ReportWriter.java
A  ReportWriter.class          # 编译产物：应被忽略
A  build/out/report.txt        # 运行产物：应被忽略
A  .DS_Store                   # 操作系统垃圾：应被忽略
A  local-settings.xml          # 本机配置：不应共享
```

【错误代码的问题】 1. 不可移植：其他开发者克隆后路径不存在，`write()` 直接失败；`DB_URL` 指向 Alice 的本地数据库，谁都连不上。 2. 编译产物进入历史：`.class` 文件是二进制，每次构建都会产生巨大且无意义的 diff，还会与源码冲突、制造合并冲突。 3. 运行产物与操作系统垃圾文件污染 `git status`，让你无法一眼看出"哪些是真正的改动"——这直接削弱了 `git status` 作为核心工具的价值。 4. 个人偏好（`DEBUG = true`）被固化进共享代码，他人的调试体验被你的设置覆盖，且无法通过配置调整。

✅ *正确代码*

```
// 正确：所有环境相关信息从外部注入，代码本身保持纯粹与可移植
public class ReportWriter {
    private final java.nio.file.Path outputDir;

    /**
     * @param outputDir directory to write reports into; requires outputDir
     *        is an existing, writable directory
     */
    public ReportWriter(final java.nio.file.Path outputDir) {
        this.outputDir = outputDir;
    }

    /** @return the file the report was written to */
    public java.nio.file.Path write(final String text) throws java.io.IOException {
        java.nio.file.Path report = outputDir.resolve("report.txt");
        java.nio.file.Files.write(report, text.getBytes(java.nio.charset.StandardCharsets.U
        return report;
    }
}

// 环境相关的选择集中在程序入口，并由命令行参数或环境变量提供
public static void main(String[] args) throws Exception {
    java.nio.file.Path out = java.nio.file.Paths.get(args[0]);   // 例如 args[0] = "build/ou
    new ReportWriter(out).write("hello");
}
```

```
$ cat .gitignore
build/
out/
*.class
.DS_Store
local-settings.xml

$ git add ReportWriter.java .gitignore
$ git commit -m "Write reports to a caller-supplied directory"
```

【为什么这样更好】 环境差异被推到程序边界之外，仓库里只剩下与机器无关的源码，任何人克隆后都能构建与运行；`.gitignore` 让 `git status` 只显示真正需要关心的改动，使它重新成为有效的状态检查工具（本讲反复强调的习惯）；编译产物不再进入历史，diff 干净、仓库更小、合并更少冲突。补充说明：Java 项

目中路径通常通过 `Path` 参数或系统属性传入；把"当前工作目录"当作隐含全局状态（例如依赖 `user.dir`）是同一类错误。

【代码对比解说】 这组对比把"不要使用全局变量"（Reading 4）与"工作副本卫生"连起来：`OUTPUT_DIR`、`DB_URL`、`DEBUG` 都是披着 `static final` 外衣的**环境依赖**，它们让同一份源码在不同机器上表现不同，等价于隐式的全局状态。正确做法是把它们变成显式的参数（构造器参数、方法参数、配置对象），让依赖关系在签名里可见——这与 Reading 6 中"参数是前置条件的一部分"完全一致：数据的来源与合法性应当写在契约里，而不是藏在常量中。`.gitignore` 则处理"什么该被跟踪"这一问题，它的判断标准是：**这个文件能由仓库里的其他文件重新生成吗？** 能重新生成的（`.class`、构建产物），就不该提交。

【设计原则透视】 这组对比跨越了版本控制与依赖管理：提交到仓库的东西定义了"这个项目是什么"。可移植的仓库让 Reading 29（Team Version Control）中的团队协作成为可能——每个人都能克隆、构建、测试。同时它呼应了 Reading 4 的"快速失败"：与其让程序在别人机器上因为找不到路径而神秘失败，不如让路径成为必须显式提供的参数，在编译期/启动期就暴露"你没有提供输出目录"这件事。

与其他设计原则的关联

- **与 Reading 1（Static Checking）**：版本控制与静态检查是两种"早发现"机制——前者在时间维度上尽早暴露回归，后者在编译期尽早暴露类型错误。两者都让修复成本降到最低。
- **与 Reading 3（Testing）**：每个提交应保持测试通过，这样历史才是可信的、`git bisect` 才有意义；测试文件与源码一起提交，才能让"历史版本"真正可复现。
- **与 Reading 4（Code Review）**：审查的对象是"提交"，因此提交的粒度与消息质量直接决定审查价值；上一讲中"不要用注释保存旧代码"的可行性，完全由本讲的对象图提供。
- **与 Reading 6（Specifications）**：合并冲突本质上是"契约理解不一致"的物理表现；场景 1 表明，把接口的职责边界写进规格说明，可以避免"文本合并成功、语义合并失败"。
- **与 Reading 8（Immutability）**：Git 的对象（commit、tree、blob）一旦写入就永不修改，因此可以被多个提交自由共享——这正是"不可变数据可以安全共享"的最有说服力的工程实例；本讲的"提交不可变性"与下一讲的"不可变对象"共享同一套推理。
- **与 Reading 9（Avoiding Debugging）**：`git log`、`git blame`、`git bisect` 是最有效的调试辅助工具之一；把"什么时候开始坏的"从猜测变成查询，能大幅缩短调试时间。
- **与 Reading 29（Team Version Control）**：本讲建立了单人/小规模协作的模型，Reading 29 会把分支策略、pull request、代码审查流程与团队工作流系统地接上去。

关键要点

- **Git 存的是对象图，不是差异**：提交是项目在该时刻的完整快照，树与 blob 让相同内容只存一份；`git show` 显示 diff 只是一种更有用的呈现方式，末尾加冒号就能看到完整快照。
- **历史是有向无环图，分支只是一个名字**：两个提交共享父提交就出现分叉，一个提交有两个父提交就是合并；环在语义上不可能（提交不能是自己的祖先）。
- **三区模型决定你的每一步**：工作目录（你编辑的文件）、暂存区（半成品提交）、仓库（对象图）；`git status` 是随时校准你对这三区认知的工具。

- **push 被拒绝是保护而非惩罚**：它防止远端历史丢失他人已推送的提交；正确反应是 `pull`（取回并合并）而不是强推。
- **提交要原子、消息要讲"为什么"**：能用一句不含"并且"的话描述的提交才是可审查、可回退、可 blame 的提交；"改了什么"已经由 diff 记录，消息应补上意图。

常见陷阱与注意事项

1. **以为"自动合并成功"就等于"正确"** → Git 只在文本层面合并，两个各自合理的改动可能组合出错误语义（`CiaoEve`）。补救：合并后必须重跑测试，并人工审查合并结果。
2. **把编译产物、运行产物、本机配置提交进仓库** → 二进制噪声让 diff 失去可读性、制造无谓冲突、暴露本机信息。补救：写 `.gitignore`，只提交能由源码重新生成之外的东西。
3. **在同一个提交里混入重命名、格式化与行为改动** → 无法回退、无法审查、`git blame` 失效、重命名检测失败导致历史断裂。补救：用 `git add -p` 分批提交，让每个提交只做一件事。
4. **直接使用 GUI 插件或编辑器菜单执行 add/commit/push** → 部分工具改变了操作语义，出问题时难以诊断，助教也无法复现。补救：用命令行，把 GUI 工具限制在"查看状态与历史"的用途上。
5. **把源文件当版本仓库用（`v2 final`、注释掉的旧实现）** → 死代码积累、历史不可查、bug 修复改错分支。补救：删除旧代码，需要时用 `git log -p`、`git show`、`git revert` 取回。
6. **从不运行 `git status`，或长期不提交** → 工作目录里堆积大量未提交改动，一旦误操作（覆盖、切分支、误删）就无法恢复，也写不出有意义的提交消息。补救：养成小步提交与"命令前后各看一次状态"的习惯。

思考题（带答案）

问题 1：Alice 与 Ben 从同一个提交开始，Alice 的提交先被 push 到远端。现在 Ben 执行 `git push origin main`，为什么会被拒绝？如果 Git 允许这次推送，会发生什么？Ben 的正确操作序列是什么？

答案：推送被拒绝是因为这不是快进（fast-forward）更新：远端的 `main` 现在指向 Alice 的提交，而 Ben 的本地 `main` 指向他自己的提交，两者从共同祖先分叉。如果服务器把 `main` 指向 Ben 的提交，Alice 的提交就会从项目历史中不可达——她的工作事实上丢失了。因此 Git 拒绝推送。Ben 的正确操作是先 `git pull`（等价于 `git fetch` 加合并）：把 Alice 的提交下载进本地对象图，并把两段历史合并，生成一个有两个父提交的新合并提交 `3e62e60`，其内容是"双方改动都已应用"的完整快照；如果两人改动了同一文件的同一部分则需要人工解决冲突并提交合并结果；之后 Ben 再 `git push origin main`，远端就能快进到新的合并提交，历史不会丢失。

问题 2：请解释"提交是一个快照"与 `git show` 输出 diff 之间的矛盾，并说明为什么这个设计对"共享存储"和"快速检出"都有好处。

答案：两者不矛盾，它们是**存储模型**与**展示方式**的区别。存储模型上，每个提交是一个 tree 对象，指向代表各文件内容的 blob 对象，即一棵完整的目录快照。展示上，Git 假设一次提交只改动项目中的少数文件，因此默认只显示与父提交的差异，信息密度更高。好处之一是可共享性：同一个文件版本只存一份 blob，多个提交复用同一个 blob 对象，因此仓库不会随提交数线性膨胀——这正是不可变数据可安全共享的直接应用（Reading 8）。好处之二是快速检出与比较：检出任意版本只需按 tree 展开 blob，不需要在差异链上重放历

史；比较两个版本只需比较两棵树的对应 blob。也正因为提交是快照，合并才能以"三方比较"（共同祖先 + 两个分支尖端）的方式自动完成。

问题 3：在一次 team project 中，一位同学提交了一个名为 `fix stuff` 的提交，其中同时改了 `.gitignore`、把两个类重命名、修了一个空指针 bug，还把三处格式从 4 空格改成 2 空格。请指出这个提交违反了本讲的哪些原则，并说明如何把它拆成更好的历史（不必真的改写已经推送的历史）。

答案：违反之处有四。其一，原子性：一个提交做了至少四件互不相关的事，无法用一句不含"并且"的话描述，因此无法被独立审查或独立回退。其二，可审查性：格式改动（2 空格）产生的噪声会淹没真正的语义改动（空指针修复），违反 Reading 4 中"审查要能看清语义变化"的期望。其三，责任追踪：`git blame` 会把大量未改动的行归给这次提交，历史信息被抹平。其四，重命名与内容改动混杂，Git 的重命名检测（启发式）可能失败，`--follow` 因此断链。更好的做法是：以后的改动按"配置""重命名""行为修复""格式化"分四次提交，每次提交前用 `git add -p` 挑选代码块并各自写清意图，且每个提交后跑一遍测试保持绿色。对于已经推送的历史，不要去改写（强推会破坏他人的克隆），而是在团队约定后补做：把格式化规则写进项目约定与自动格式化脚本，并在下一次相关改动时把格式与语义分开提交；必要时可在仓库文档中记录这次提交的复合意图，以免后人误读。

Reading 6: 规格说明（Specifications）

> 说明：本讲 sp22 原版使用 TypeScript，本笔记按用户要求提供 Java 代码示例；类型/API 与 sp21（6.031 Java 版）原文保持一致。凡属笔记额外补充的 Java 生态知识（如 `Optional`、`Objects.requireNonNull`、受检异常），均以「补充说明」标注。

概述

规格说明（specification）是团队协作的**关键枢纽（linchpin）**：不写规格说明，就不可能把"实现某个方法"的责任委派给别人。规格说明是一份**契约（contract）**——实现者负责满足契约，使用该方法的客户（client）则可以依赖契约；而且像真实的法律合同一样，契约对双方都提出要求：当前置条件存在时，**客户也有责任**。

本讲要回答的核心问题有四组：第一，什么是**行为等价（behavioral equivalence）**——什么时候可以把一个实现换成另一个而不破坏客户？答案取决于客户究竟依赖了什么，而"客户依赖什么"必须由规格说明精确写出。第二，规格说明的结构是什么——函数签名、`requires` 子句、`effects` 子句，它们合起来构成**前置条件（precondition）**与**后置条件（postcondition）**，整体是一条逻辑蕴含：若调用时前置条件成立，则函数返回时后置条件必须成立；若前置条件不成立，实现**不受任何约束**，可以做任何事，包括不返回、抛未被规格提及的异常、返回任意结果、做任意修改。第三，规格说明的**强弱（weaker / stronger）**如何比较——前置条件更弱、后置条件更强的规格更强，更强的规格对客户更有利、对实现者更苛刻；这一主题在 Reading 7（Designing Specifications）中正式展开，本讲先建立判据。第四，语言特性如何让规格更安全——异常（exceptions）用于区分"信号 bug"与"可预期的失败"，`null` 的默认禁用与 `Optional` 的克制使用让"值的缺失"不再模糊，而 TypeScript 的严格空检查在 Java 中对应着依赖静态类型与注解的同类保障。

本讲所有内容都对准三大目标：**Safe from bugs（免于 bug）**——程序中最难缠的 bug 来自"两个模块在接口处对行为的理解不一致"，规格说明把双方的共同假设写下来，用机器可检查的语言特性（静态类型、异常）替代纯人读的注释，能进一步减少 bug；**Easy to understand（易于理解）**——一份简短清晰的规格说明比

实现本身好懂得多，它让客户不必读源码；**Ready for change（为变更而设计）**——规格说明在客户与实现者之间筑起**防火墙（firewall）**，双方只要各自遵守契约，就能独立修改，这就是**解耦（decoupling）**。

核心概念与设计原则详解

行为等价（Behavioral Equivalence） – **定义与目的**：两个实现"行为等价"意味着可以把其中一个替换成另一个而不破坏任何客户。它决定了重构与优化的安全边界。 – **直观解释（"它是什么？"）**：课程用一个 `find` 方法说明。原版从左到右扫描，找不到返回 `-1`；"聪明版"同时从两端向中间扫描以加速。二者不仅性能不同，**输入输出行为也不同**：当 `val` 在数组中出现多次时，原版总返回最小的下标，聪明版可能返回最小或最大的下标，取决于哪一端先找到。 – **关键规则与最佳实践**： – 行为等价"取决于观察者"——也就是客户：如果客户从不依赖"返回最小下标"这一行为（例如他们总是传入恰好出现一次的 `val`），那么两个实现就是等价的。 – 要让"可以替换"这件事变成可判断的，就必须有一份规格说明，精确说明客户可以依赖什么。 – 规格说明不必描述所有输入下的行为，只需描述**合法调用**下的行为。 – 一旦规格说明存在，实现者就获得了"在不违反规格的前提下随意更换实现"的自由——这正是 Reading 4（Code Review）中"为变更而设计"的具体兑现。

规格说明即契约（Specification as a Contract） – **定义与目的**：规格说明是客户与实现者之间的契约，双方各有义务。它服务全部三大目标。 – **直观解释（"它是什么？"）**：契约是一堵**防火墙**：它把客户与模块实现细节隔开——作为客户，有规格就不必读模块源码；它同时把实现者与模块使用细节隔开——作为实现者，不必去问每个客户打算怎么用。墙的两侧可以各自改动，只要各自履行义务。 – **关键规则与最佳实践**： – 客户方的义务来自**前置条件**；实现者方的义务来自**后置条件**。 – 契约让"甩锅"变得客观：程序失败时能定位到是客户违反了前置条件，还是实现没有满足后置条件——归咎于代码片段，而不是人。 – 规格说明还能让代码更快：它排除了某些调用状态，使实现可以省掉昂贵检查（例如 `addAll` 的前置条件使人可以放心地边遍历边追加）。 – 团队里"每个人心里都有规格"是不够的：不写下来的结果就是不同人心里装着不同规格，程序失败时谁也说不清错在哪。 – 写规格说明不是官僚流程，而是把"接口处的共识"变成可传递的资产。

规格说明的结构（Specification Structure） – **定义与目的**：抽象地说，一个方法的规格说明由三部分组成，它们共同定义了前置条件与后置条件。 – **直观解释（"它是什么？"）**： – **函数签名（signature）**：方法名、参数类型、返回类型。 – **`requires` 子句**：对参数的额外限制。 – **`effects` 子句**：返回值、异常以及其他效果。 – **关键规则与最佳实践**： – 签名本身就是前置条件与后置条件的一部分，而且是**编译器自动检查**的那一部分。 – `requires` 的常见内容有两类：收窄参数类型（如"参数 `x` 必须是非负整数"）、参数之间的相互作用（如"`val` 在 `arr` 中恰好出现一次"）。 – `effects` 的常见内容有三类：返回值与输入的关系、抛哪些异常以及在什么条件下抛、是否以及如何修改对象。 – 在 Java 中，把前置条件写进 `@param`，把后置条件写进 `@return` 与 `@throws`，这是 Javadoc 约定，也是 6.031 要求的形式。 – 用 Javadoc 写规格还能让 IDE（Eclipse、IntelliJ）把信息显示给客户，并自动生成 HTML 文档。

前置条件（Precondition） – **定义与目的**：前置条件是**客户**的义务，是方法被调用时所处状态上的条件。它主要服务 Safe from bugs。 – **直观解释（"它是什么？"）**：前置条件回答"在什么情况下允许调用我"。它包括参数的数量与类型（由签名承载），以及额外的限制（写在 `requires` 里）。例如 `find` 的规格要求"`val`

在 `arr` 中恰好出现一次"。 – **关键规则与最佳实践**： – 前置条件不成立时，实现不受后置条件约束——这是契约的逻辑含义，不是实现者的恶意。 – 因此客户**不能**通过"故意违反前置条件"来测试实现的行为（如"空数组时它会怎样？"）：测试与所有客户一样必须遵守契约。 – 前置条件越弱（要求越少），客户越方便，但实现者的负担越重——这正是强弱权衡的核心。 – 如果前置条件是实现者**能够合理检查**的，往往应当把它从 `requires` 移到后置条件里（用异常或特殊结果来描述），这样契约更完整、更安全。 – 隐式前置条件同样存在：除非规格说明另作声明，对象与数组类型的参数**必须非 null**；集合的元素同样必须非 null。

后置条件（Postcondition） – **定义与目的**：后置条件是**实现者**的义务，是（假设前置条件成立时）方法返回后程序状态上的条件。它主要服务 Safe from bugs 与 Ready for change。 – **直观解释（"它是什么？"）**：后置条件回答"我必须保证什么"。它包含类型系统能静态检查的部分（尤其是返回类型），以及写在 `effects` 里的额外保证：返回值与输入的关系、会抛哪些异常、会修改什么。 – **关键规则与最佳实践**： – 整体结构是一条逻辑蕴含：**前置条件成立 ⇒ 后置条件必须成立**。 – 后置条件越强（承诺越多），客户越能依赖，但实现越难写——这与前置条件的方向正好相反。 – 后置条件必须说明**副作用**：除非明确写出会修改，否则默认不修改任何输入对象（与 null 的默认禁用同理）。 – 后置条件不能承诺超出必要的东西：承诺得越多，实现者越受束缚，未来越难优化——这正是下面 `find` 例子里"返回某个下标"优于"返回最小下标"的原因（在规格设计层面）。 – 后置条件只应谈论参数、返回值与对象状态，**不应**谈论局部变量或私有字段。

逻辑蕴含（Logical Implication） – **定义与目的**：理解"前置条件不成立时实现是自由的"这一条，是正确使用契约的关键。 – **直观解释（"它是什么？"）**：把规格写成 `pre → post`。当 `pre` 为假时，整个蕴含式为真，也就是"实现没有违反契约"，无论它做了什么。 – **关键规则与最佳实践**： – 当前置条件被违反时，合法行为包括：返回任意值、抛任意异常（包括规格里没提到的）、修改任意对象、进入死循环、永不返回。 – 因此客户绝不能依赖"违反前置条件时的行为"，也没法测试它。 – 对实现者而言，这条既是自由也是建议：可以在前置条件被违反时抛异常（快速失败，便于调试），但这不是契约要求。 – 课程的练习给出了很好的判据：`find` 的规格要求 `val` 恰好出现一次；那么"`arr` 为空时返回 0""`val` 出现两次时抛异常""不满足条件时把数组清零再抛异常"等都被规格允许。 – 注意区分两件事：**规格允许什么**（任意行为）与**好的实现应该做什么**（尽早失败、给出可诊断的信息）。

规格说明可以谈论什么（What a Specification May Talk About） – **定义与目的**：规格说明的"话题边界"定义了抽象屏障；越过它，客户就被迫依赖实现细节。 – **直观解释（"它是什么？"）**：规格可以谈论方法的参数与返回值，**绝不能**谈论方法的局部变量或所属类的私有字段。对读规格的人来说，实现应当是不可见的——它在防火墙之后。甚至源码本身都可能不可见：Javadoc 工具只抽取规格注释并渲染成 HTML。 – **关键规则与最佳实践**： – 不要在规格里描述算法（"我用 StringBuilder 逐个追加"），那是实现细节；算法可以自由更换。 – 不要用私有字段名出现在 `@param` / `@return` 中，否则一旦重构字段，规格就变成谎言。 – 规格要描述**可观察行为**：返回值、异常、对参数对象的修改。 – 规格中的"返回 `-1` 表示未找到"这类约定，实际上是**后置条件**的一种编码方式，必须写清楚；更好的做法是用异常或 `Optional` 让类型本身说明这件事（见下文）。 – 这条规则与 Reading 11（Abstraction Functions & Rep Invariants）正好构成抽象屏障的两侧：对外是规格说明，对内是表示不变量与抽象函数。

规格说明的强弱（Stronger and Weaker Specifications） – **定义与目的**：强弱是比较两份规格说明的判据，决定了"能不能替换""客户能依赖多少""实现有多少自由"。本讲先建立直觉与判据，Reading 7（Designing Specifications）会系统地展开。 – **直观解释（"它是什么？"）**：规格说明 = 前置条件 + 后置条件。若规格 S1 的**前置条件更弱**（对客户要求更少）且**后置条件更强**（对实现者承诺更多），则称 S1 **比 S2 更强（stronger）**；反向即更弱。 – **关键规则与最佳实践**： – 更强的规格对**客户**更好：可以依赖更多保证；对**实现者**更苛刻：必须处理更多输入、满足更多承诺。 – 因此实现者希望规格更弱，客户希望规格更强——这是接口设计中的核心张力。 – 一个满足更强规格 S1 的实现，也一定满足更弱的规格 S2：因为 S2 允许的输入集合被 S1 覆盖，S2 要求的承诺被 S1 的承诺蕴含。所以"用满足更强规格的实现去替换"是安全的。 – 注意术语陷阱：**"前置条件更强"意味着规格整体更弱**——因为它缩小了合法调用的集合。句子里的"强"修饰的是条件，不是规格。 – 课程在"测试与规格说明"一节里就用过这组措辞：规格 `requires: val occurs in arr` 加 `effects: returns index i such that arr[i] = val` 被认为有**强前置条件**（它要求客户保证 `val` 一定能被找到，即对客户提出了更多要求）与**相当弱的后置条件**（当 `val` 出现多次时，它完全不指定返回哪一个下标）。两者都指向同一个结论：**这条规格整体上偏弱，客户能依赖的东西很少**——所以客户的测试不能写成 `assert i == 0`。 – 补充说明：Reading 7 会进一步讨论如何在这条谱系上做设计决策（例如"前置条件该不该由实现者来检查"），本讲只需记住判据：**更弱的前置 + 更强的后置 = 更强的规格**。

规格说明决定客户能依赖什么（What Clients May Depend On） – **定义与目的**：规格说明是"可依赖清单"；超出清单的依赖是 bug 的源头，而不是运气问题。 – **直观解释（"它是什么？"）**：客户可以依赖规格承诺的一切，也**只能**依赖这些。实现可能碰巧提供了更强的保证（例如总是返回最小下标），但客户不能依赖这份"碰巧"：它是实现的偶然性质，随时可能在优化中消失。 – **关键规则与最佳实践**： – 客户不应依赖未写下的行为；如果某个行为重要到客户需要它，就必须把它写进规格。 – 客户不应违反前置条件，也不应依赖违反前置条件后的后果。 – 测试也是客户：即使玻璃盒测试（glass box test，见 Reading 3 Testing）了解实现细节，也必须遵守规格，不能断言超出规格的行为。 – 规格带来的好处是双向的：客户省下读源码的成本，实现者省下"问遍所有客户怎么用"的成本。 – 规格说明必须**完整到足以判断行为等价**：如果它漏写了客户真正依赖的行为，它就失职了。

规格说明与测试（Specifications and Testing） – **定义与目的**：测试必须针对规格编写；单元测试应聚焦单一规格。 – **直观解释（"它是什么？"）**：黑盒测试（black box test）只依据规格挑选用例；玻璃盒测试（glass box test）借助对实现的了解来挑选用例，但**断言仍必须依据规格**——目的是"用新用例覆盖实现的不同部分"，而不是"检查实现的私有行为"。 – **关键规则与最佳实践**： – 规格说"返回某个满足 `arr[i] = val` 的下标"时，`assert i == 0` 是错误测试（假设过多），`assert arr[i] == val` 才是正确断言。 – 不要写"违反前置条件会怎样"的测试——那超出了契约范围。 – 单元测试只应依赖被测方法与标准库的规格；`extract()` 的测试不应因为 `load()` 没满足它的后置条件而失败（课程中的搜索引擎例子：`load`、`extract`、`index`）。 – 集成测试（integration test）检验不同模块的规格是否兼容，但不能替代系统设计的单元测试：只通过 `index` 测 `extract`，就只覆盖了 `load` 可能产出的那一小部分输入空间，其余部分成为 bug 的藏身处。 – 规格说明为测试提供了"预期行为"的唯一权威来源——这正是测试与规格互为表里的原因。

异常（Exceptions） – **定义与目的**：异常是方法的一种可能输出，因此可能需要写进后置条件。关键是区分"信号 bug 的异常"与"信号可预期失败的异常"。 – **直观解释（"它是什么？"）**： – **信号 bug 的异常**：`IndexOutOfBoundsException`（下标越界）、`NullPointerException`（在 null 引用上调用方法）、`ArithmeticException`（如整数除以零）、`NumberFormatException`（`Integer.parseInt` 解析失败）。这类异常通常表示客户或实现有 bug，其信息用于帮助定位。**它们不是后置条件的一部分，因此不应出现在 `@throws` 中**——例如 `NullPointerException` 永远不该写进规格，因为"参数非 null"已经是隐式前置条件，实现可以在客户违反时自由抛出它。 – **信号可预期失败的异常**：用于让调用者能够捕获并响应的情况，例如 `lookup(name)` 在生日簿里找不到名字。这类异常**必须**用 `@throws` 记录，并说明在什么条件下抛出。`BirthdayBook` 的例子展示了这一点：与其返回 `9/9/99` 之类的"特殊值"（几十年来坏程序员的做法，而且他们错了），不如抛出 `NotFoundException`，让调用者用 `catch` 处理——既不需要特殊值，也不需要配套的检查。 – **关键规则与最佳实践**： – 信号可预期失败的异常总是写入 `@throws`；信号 bug 的异常则从不写入。 – **受检异常（checked exception）** 在 Java 中还必须在方法签名的 `throws` 子句中声明；来自 Python 或 TypeScript 的读者要特别注意：Java 编译器会静态要求调用者捕获或继续声明，这是 Java 相对 TypeScript 的一个显著优势（TypeScript 不提供异常处理的静态检查，用异常表达可预期失败容易漏掉 `try...catch`）。 – 对**未受检**但信号可预期失败的异常，Java 允许但**不建议**写入 `throws`——写上去会误导人类读者以为它是受检异常；只写 `@throws` 即可。 – 自定义异常时，继承 `Exception`（受检）或 `RuntimeException`（未受检）；**不要**继承 `Error`，那是 Java 保留给自身使用的。捕获时尽量用最具体的异常类，捕获 `Exception` / `RuntimeException` / `Error` 这类宽泛类型会隐藏 bug、破坏静态检查。 – **异常转译（exception translation）**：把低层异常（如 `PathNotFoundException`）转成更高层的异常（如 `RobotStuckException`），可以让模块更好地隐藏实现细节，从而更 ready for change。 – 要小心的反面例子：为了让编译器满意而用 `catch (Exception e) { return; }` 包住整个方法体——它会把真正的 bug（如 `IndexOutOfBoundsException`）也一起静默吞掉，既掩盖缺陷又不可能调试。 – 异常的另一个用途是"最后的手段"：当失败原因不需要客户处理时，返回特殊结果（`null`、`-1`）看似简单，却有两个问题——检查返回值的写法很繁琐，而且**很容易忘记检查**；用异常反而能得到编译器的帮助（在 Java 中尤其如此）。

Null 与 Optional（Avoid Null / Optional） – **定义与目的**：`null` 表示"引用不指向任何对象"，它是类型系统上的一个洞，最好完全避开；当确实需要表达"值缺失"时，用 `Optional<T>` 这类显式载体。 – **直观解释（"它是什么？"）**：Java 中基本类型不能为 `null`（`int size = null;` 是静态错误），但任何非基本类型变量都可以被赋值为 `null`：`String name = null; int[] points = null;` 编译器乐于接受，运行时报错——`name.length()` 与 `points.length` 都会抛 `NullPointerException`。注意 `null` **不等于**空字符串或空数组：空串与空数组是合法对象，`"".length()` 是 0；而 `null` 上的 `length()` 什么都不是，直接抛异常。还要注意"非 null 但元素是 null"的容器：`String[] names = new String[] { null };`、`List<Double> sizes` 里 `add(null)`——这些 null 会在别人使用容器内容时立刻引发错误。 – **关键规则与最佳实践**： – 约定：除非规格说明明确声明，参数与返回值**不允许 null**——包括集合（数组、列表、集合、映射）中的元素。 – 每个接受对象/数组参数的方法都隐式带有一个前置条件"非 null"；每个可能返回对象/数组的方法都隐式带有一个后置条件"返回值非 null"。 – 如果某个方法确实允许 null，必须在规格中显式写出（`@param` 里说明，或使用类型注解如 `@NonNull` / `@Nullable`；补充说明：Java 生态中用 `Objects.requireNonNull(x, "x")` 在入口处快速失败，是常用的落地方式）。 – Google 在 Guava 文档中

的说明很有说服力：在 Google 代码库中约 95% 的集合本不该含 null，让它们**快速失败**而不是静默接受 null 会对开发者有帮助；而且 `null` 语义模糊——`Map.get(key)` 返回 `null` 既可能是"键存在但值是 null"，也可能是"键不存在"。用别的东西代替 null 能让含义清晰。 – 当确实需要表示"缺失"时，用 **`Optional<T>`**：可以把它想象成一个长度至多 1 的受限制 `List<T>`——要么恰好含一个 `T`，要么为空。`isPresent()` 判断是否为空，`get()` 取出值。它的关键优势是**可以克制地使用**：只在规格确实允许缺失值的地方出现，从而清楚表达规格意图。（补充说明：Java 8+ 还提供了 `OptionalInt` / `OptionalLong` / `OptionalDouble` 以避免基本类型的装箱开销。） – 注意"空值"与"缺失"的区别：**空值永远是合法的**（空串、空列表、空映射都是普通对象），除非规格明确禁止；因此"允许空参数"不需要额外声明，而"允许 null 参数"必须声明。

可变方法的规格说明（Specifications for Mutating Functions） – **定义与目的**：到目前为止的例子都在描述返回值；可变对象上的方法还必须在后置条件里描述**副作用**。 – **直观解释（"它是什么？"）**：课程给出的例子是被简化过的 `List.addAll`：`requires` 写明"两个参数不是同一个对象"，`effects` 写明"把 `list2` 的元素追加到 `list1` 末尾，并返回 `list1` 是否因此改变"。后置条件因此有两条约束：如何修改 `list1`，以及返回值如何确定。 – **关键规则与最佳实践**： – 前置条件可以合法地禁止别名（`list1 != list2`）：它几乎不排除有用的用法，却让实现变得更简单——可以"从 `list2` 取一个元素追加到 `list1`，再取下一个"，直到取完。 – 如果两个参数是同一个列表，这个简单算法不会终止（实际中会因为对象膨胀耗尽内存而崩溃）；**无限循环与崩溃都被规格允许**，因为前置条件已被违反。 – 后置条件必须写清是"修改参数"还是"返回新对象"：`toLowerCase(list)` 的规格应当说明"返回一个新列表 `t`，长度相同且 `t[i] = list[i].toLowerCase()`"，而**不修改** `list`。 – 与 null 同理：除非规格说明写明，**默认不允许修改输入对象**。规格可以显式写"不修改输入"，但在缺少关于修改的后置条件时，我们一律要求不修改输入。 – 别忘了隐式前置条件（`list1`、`list2` 必须非 null），以及"取新变量而不是复用参数"的写法在实现层面的配合。

为什么规格说明有帮助（Why Specifications Help） – **定义与目的**：把前面所有概念收束到三大目标上。 – **直观解释（"它是什么？"）**：程序中最难缠的 bug 往往源于"接口处行为理解的错位"。每位程序员心里都有规格，但**不写下来**就意味着团队里存在多份互相冲突的规格——程序失败时，没人说得清该改哪里。 – **关键规则与最佳实践**： – Safe from bugs：好规格清晰记录客户与实现共同依赖的相互假设；用机器可检查的语言特性（静态类型、异常）替代纯注释，能进一步减少 bug。 – Easy to understand：简短简单的规格比实现本身更容易理解，省去别人读代码的功夫——请把 `find` 的一行规格与它的双端搜索实现对比一下。 – Ready for change：规格在代码的不同部分之间建立契约，只要各自继续满足契约要求，就能独立变化。 – 规格是**责任委派**的前提：没有规格，就无法把"实现这个方法"的责任交出去。 – 规格也是**沟通效率**的工具：它把"问你打算怎么用"与"读你的源码"两种昂贵沟通方式，替换成一次阅读。

代码示例与对比分析

场景 1：没有规格说明就更换实现——"优化"变成 bug

❌ *错误代码*

```
// 错误：没有写下任何规格，客户开始依赖实现中"恰好"出现的行为
public static int find(int[] arr, int val) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == val) return i;
    }
    return -1;
}

// 客户代码依赖了"返回最小下标"这一从未写下的行为：
int i = find(scores, 7);          // scores = {7, 7, 7}
assert i == 0 : "expected the first occurrence";

// 现在作者为了性能，把它换成"从两端同时向中间扫描"的实现：
public static int find(int[] arr, int val) {
    for (int i = 0, j = arr.length - 1; i <= j; i++, j--) {
        if (arr[i] == val) return i;
        if (arr[j] == val) return j;     // 可能返回较大的下标！
    }
    return -1;
}
// 客户的断言开始随机失败；更糟的是，若客户代码只在某些输入下才检查下标，
// 这个 bug 会在上线后以"偶发错误答案"的形式出现。
```

【错误代码的问题】 1. 缺失契约：两个实现的行为确实不同（重复元素时返回的下标可能不同），但没有任何文档说明客户可以依赖什么，"能不能替换"只能靠运气。 2. 客户依赖了未承诺的行为：`assert i == 0` 依赖"返回最小下标"，这从来不是契约的一部分；一旦实现变化，客户就崩溃。 3. 失败模式最糟：错误只在"`val` 出现多次"的输入下出现，属于"无异常、错误答案"，正是 Reading 1（Static Checking）里失败得最慢、最难定位的一类。 4. 作者失去了替换实现的自由：因为行为没有被约定，任何优化都要担心是否踩到某个客户的隐含假设——这正是"没有规格反而更不自由"的悖论。

✅ *正确代码*

```
/**
 * Find a value in an array.
 * @param arr array to search; requires val occurs exactly once in arr
 * @param val value to search for
 * @return index i such that arr[i] == val
 */
public static int find(final int[] arr, final int val) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == val) return i;
    }
    return -1;      // 在前置条件成立时永远不会执行到这里
}

// 优化后的实现同样满足这份规格：客户完全不必改动，因为契约没有变化
/**
 * Same specification as above: returns index i such that arr[i] == val.
 */
public static int findFromBothEnds(final int[] arr, final int val) {
    for (int i = 0, j = arr.length - 1; i <= j; i++, j--) {
        if (arr[i] == val) return i;
        if (arr[j] == val) return j;
    }
    return -1;
}
```

【为什么这样更好】 规格说明把"客户可以依赖什么"钉死在"存在某个下标 `i` 满足 `arr[i] == val`"上。在这一契约下，两个实现**行为等价**，可以自由替换；客户因此也不会写出 `assert i == 0` 这种越界依赖。注意规格中的 `return -1` 从未被提及——因为在前置条件成立时它不可达。这也回答了一个常见疑惑："实现里有 `return -1`，规格为什么不说？"答案是：**规格只需描述合法调用下的行为**。

【代码对比解说】 这份规格做了两件事：用前置条件（`val` 恰好出现一次）排除了"重复元素"这一分歧来源，用后置条件（存在性而非最小下标）明确了客户能依赖的全部内容。它同时是"更弱的后置条件"换"更强的实现自由"的范例：如果作者把后置条件写成"返回**最小**下标"，客户能依赖更多，但节省的那次扫描就不能省了——规格的每一分强度都有代价。这也是 Reading 7（Designing Specifications）将深入讨论的设计权衡，本讲只需记住：**规格的存在使"替换实现"从猜测变成推理**。

【设计原则透视】 这组对比直接对应"行为等价"与"规格决定客户能依赖什么"两个概念，并落到 Reading 3（Testing）的规则上：测试断言必须来自规格，`assert i == 0` 违反规格（假定过多），`assert arr[i] == val` 才合法。它同时展示了 Reading 4（Code Review）中"注释该写什么"的答案：最重要的注释就是规格说明。

场景 2：用魔法值表示失败 vs 用 `Optional` / 异常表达可预期失败

❌ *错误代码*

```
// 错误：用 -1 与 null 表示"失败"，客户很难不忘记检查，且语义模糊
public static int integerSquareRoot(int x) {
    if (x < 0) return -1;                    // -1 到底是"参数非法"还是"不是完全平方数"？
    for (int r = 0; r * r <= x; ++r) {
        if (r * r == x) return r;
    }
    return -1;                               // 两种完全不同的失败共用同一个魔法值
}

/** @return the birthday of name, or null if not found */
public static LocalDate lookup(String name) {
    return book.get(name);                   // 未找到与"值本身是 null"无法区分
}

// 客户代码
int root = integerSquareRoot(n);
int twice = root * 2;                        // 忘记检查 -1：得到 -2，错误悄悄传播
LocalDate bd = lookup("Alyssa");
System.out.println(bd.getMonth());          // 忘记检查 null：运行时 NullPointerException
```

【错误代码的问题】 1. 语义模糊：`-1` 同时表示"参数非法"和"不是完全平方数"，客户无法区分；`Map.get` 返回 `null` 也既可能是"值是 null"，也可能是"键不存在"。 2. 极易漏检：返回值检查是一种约定，编译器不会提醒；忘记检查就得到错误答案（`-2`）或运行时异常，而且错误位置离成因很远。 3. 规格不完整：`lookup` 的 Javadoc 只写了"或 null"，既没有写前置条件，也没有说明"未找到"是不是一种正常的、需要处理的失败。 4. 无法区分该处理与不该处理：参数非法属于客户的 bug（应当快速失败），"不是完全平方数"属于可预期的失败（应当可以被捕获或显式处理），把它们混成一个值就丧失了这种区分能力。

✅ *正确代码*

```
import java.time.LocalDate;
import java.util.HashMap;
import java.util.Map;
import java.util.Objects;
import java.util.Optional;
import java.util.OptionalInt;

/** name to birthday; contains no null keys and no null values */
private static final Map<String, LocalDate> book = new HashMap<>();

/**
 * Compute the integer square root.
 * @param x integer value to take the square root of; requires x &gt;= 0
 * @return the square root of x if x is a perfect square,
 *         otherwise OptionalInt.empty()
 * @throws IllegalArgumentException if x is negative (a client bug)
 */
public static OptionalInt integerSquareRoot(final int x) {
    if (x < 0) {
        throw new IllegalArgumentException("x must be nonnegative: " + x);  // fail fast
    }
    for (int r = 0; (long) r * r <= x; ++r) {
        if (r * r == x) return OptionalInt.of(r);
    }
    return OptionalInt.empty();               // 可预期的"没有结果"，由类型显式表达
}

/**
 * Look up a person's birthday.
 * @param name the person's name; requires name is not null
 * @return the birthday of name, or Optional.empty() if the birthday book
 *         has no entry for name
 */
public static Optional<LocalDate> lookup(final String name) {
    Objects.requireNonNull(name, "name");
    return book.containsKey(name) ? Optional.of(book.get(name)) : Optional.empty();
}

// 客户代码：缺失这件事必须被显式处理，语义清楚，无魔法值
OptionalInt root = integerSquareRoot(n);
if (root.isPresent()) {
    int twice = root.getAsInt() * 2;
}
Optional<LocalDate> birthday = lookup("Alyssa");
birthday.ifPresent(bd -> System.out.println(bd.getMonth()));
```

【为什么这样更好】 两种失败被彻底分开：违反前置条件（`x < 0`）立刻抛异常，属于快速失败；可预期的"没有结果"由返回类型 `OptionalInt` / `Optional<LocalDate>` 显式承载，客户必须调用 `isPresent()` 或 `ifPresent()` 才能取值，**漏检变得困难**。规格说明同时说明了前置条件（`x >= 0`）、后置条件（完全平方数时返回其平方根）与失败语义（否则为空），因此行为等价性可以被判断，客户也知道自己该处理什么。

【代码对比解说】 课程的立场是：**异常适合表达"调用者不太可能处理的问题"**（例如参数非法，说明调用方或其上游有 bug），而"可预期的失败"应该让客户能方便地处理。Java 用**受检异常**（编译器强制调用者 catch 或向上声明）来表达后一种情况，这是相对 TypeScript 的一个实质优势；TypeScript 没有异常处理的静态检查，用异常表达可预期失败容易漏掉 `try...catch`，因此更倾向用联合类型（`number | undefined`）这类"特殊结果"。Java 中对应的做法就是 `Optional`。（补充说明：如果失败确实罕见且客户几乎总是希望处理，也可以选择受检异常 `throws NotPerfectSquareException`——此时异常必须同时出现在 `@throws` 与签名 `throws` 中；两种做法都比魔法值好。）

【设计原则透视】 这一组把"后置条件必须完整描述可能的输出（包括异常）""Null 默认禁用""异常用于可预期失败"三条规则绑在一起，并落到 Reading 3（Testing）上：返回 `Optional` 的规格比返回魔法值的规格更容易写出正确的测试，因为"没有结果"是类型系统可见的分支，测试可以分别覆盖两条路径。它也预告了 Reading 8（Immutability）：`Optional` 与其承载的值都是不可变的，可以安全共享，不存在"谁负责拷贝"的问题。

场景 3：别名与前置条件——`addAll` 的无限循环

❌ *错误代码*

```
// 错误：实现依赖了"两个列表不是同一个对象"这一前提，却从未写下来，也不检查
public static <T> boolean addAll(List<T> list1, List<T> list2) {
    boolean changed = false;
    for (T element : list2) {
        list1.add(element);      // list1 == list2 时：边遍历边追加，永不终止！
        changed = true;
    }
    return changed;
}
```

【错误代码的问题】 1. 未文档化的前置条件：客户完全不知道"不能把列表加到自己身上"，会写出 `addAll(list, list)` 并期待得到"元素翻倍"。 2. 灾难性失败模式：`list1 == list2` 时迭代器永远取得到新元素，程序不终止；实际结果是列表膨胀到耗尽内存而崩溃。规格从未提醒过这种可能。 3. 后置条件不完整：也没有说明"当 `list2` 为空时返回 `false`"，客户只能靠猜。 4. 客户无法判断对错：因为没有规格，这条失败的调用既不能被称为"客户违规"，也不能被称为"实现有 bug"，追责无从谈起。

✅ *正确代码*

```
/**
 * Adds the elements of list2 to the end of list1.
 * @param list1 list to be modified; requires list1 is not null, and
 *              list1 and list2 are not the same object
 * @param list2 list whose elements are appended; requires list2 is not null,
 *              and list2 is not the same object as list1
 * @return true if list1 changed as a result of the call
 *         (false if list2 was empty)
 */
public static <T> boolean addAll(final List<T> list1, final List<T> list2) {
    Objects.requireNonNull(list1, "list1");
    Objects.requireNonNull(list2, "list2");
    if (list1 == list2) {
        throw new IllegalArgumentException("cannot add a list to itself");
    }
    boolean changed = false;
    for (T element : list2) {   // 前置条件保证 list1 != list2，因此遍历时修改 list1 是安全的
        list1.add(element);
        changed = true;
    }
    return changed;
}
```

【为什么这样更好】 前置条件被显式写下（非 null、且两参数不是同一对象），客户从此知道自己的义务；实现者在入口处用一个便宜的身份比较（`list1 == list2`）把违反前置条件变成**立刻抛出的异常**，而不是让程序陷入不可终止的循环——这正是"异常与前置条件的关系"的具体落地：规格可以只写前置条件、把违规行为留作未定义；也可以像这里一样，因为**检查成本极低**，把违规行为提升为明确的动态错误。后置条件完整描述了两种结果（修改了 `list1` 与否，以及返回值含义），客户可以放心依赖。

【代码对比解说】 这里体现了规格设计中的一个实用判据：**当前置条件可以被廉价地检查时，就把它检查出来并快速失败**；当检查代价高昂（例如"`val` 恰好出现一次"需要先扫一遍数组），就把它留在 `requires` 里，作为客户的义务。课程原文还展示了另一种等价设计：把前置条件从 `requires` 中删除，改为在 `effects` 中用 `@throws AliasingError if array1 === array2` 描述两种情形下的行为——这样契约对客户更友好（不必担心违规），代价是实现必须处理这种情况。两种写法都正确，选择取决于"谁更适合承担检查成本"。

【设计原则透视】 这组对比把"前置条件/后置条件""逻辑蕴含""规格与异常的关系"三条连在一起。它也示范了 Reading 4（Code Review）中的"每个变量一个用途"与"快速失败"如何与规格配合：`Objects.requireNonNull` 把隐式前置条件变成显式检查。此外，"遍历 `list2` 同时修改 `list1`"之所以安全，完全建立在规格的前置条件之上——**实现可以依赖前置条件**，这正是契约"对实现者也有利"的证明。

场景 4：规格说明谈论实现细节 vs 规格说明只谈论可观察行为

❌ *错误代码*

```
// 错误：规格谈论私有字段、局部变量与内部算法；隐含未声明的义务；用 null 表达"未找到"
public class TextSearch {
    private final StringBuilder buf = new StringBuilder();
    private boolean initialized = false;
    private static final int MAX_LEN = 100;

    public void init() {
        initialized = true;
    }

    /**
     * Uses a StringBuilder held in the private field buf, plus a local counter i;
     * if the loop falls through it returns -1. Assumes the caller has already
     * called init() and that MAX_LEN is at least 1.
     */
    public int indexOf(final String text, final char ch) {
        buf.setLength(0);
        buf.append(text);
        for (int i = 0; i < Math.min(buf.length(), MAX_LEN); i++) {
            if (buf.charAt(i) == ch) {
                return i;
            }
        }
        return -1;
    }

    /**
     * @param text the text; if text is null we return null instead of throwing
     * @return boxed result of indexOf(text, ch)
     */
    public Integer indexOfOrNull(final String text, final char ch) {
        if (text == null) {
            return null;                // 把"参数违规"和"未找到"混在一起，且未声明前置条件
        }
        return indexOf(text, ch);
    }
}
```

【错误代码的问题】 1. 规格谈论私有字段与局部变量（`buf`、`i`、`MAX_LEN`），把实现变成了契约的一部分：任何内部重构都会让规格变成谎言，客户也可能开始依赖这些内部结构。 2. 隐含未声明的义务（"调用者必须先调用 `init()`"）藏在文档里却不在前置条件中，客户很难发现，违反后表现为神秘崩溃。 3. 用 null 表达"未找到"（`if text is null we return null`）把两种完全不同的事情——参数违规与正常缺失——混为一谈，且默认违反了"null 必须显式声明"的约定（它声明了，但这是一个坏设计）。 4. 返回值定义成"局部变量的值"，客户完全无法据其编程；规格实际上没有提供任何可依赖的信息。

✅ *正确代码*

```
/**
 * Find the first occurrence of a character in a string.
 * @param text the string to search; requires text is not null
 * @param ch the character to search for
 * @return the lowest index i such that text.charAt(i) == ch,
 *         or -1 if ch does not occur in text
 */
public int indexOf(final String text, final char ch) {
    for (int i = 0; i < text.length(); i++) {
        if (text.charAt(i) == ch) {
            return i;
        }
    }
    return -1;
}

/**
 * Find the first occurrence of a character in a string.
 * @param text the string to search; requires text is not null
 * @param ch the character to search for
 * @return an index i such that text.charAt(i) == ch, or
 *         OptionalInt.empty() if ch does not occur in text
 */
public OptionalInt indexOfOrEmpty(final String text, final char ch) {
    int i = indexOf(text, ch);
    return i < 0 ? OptionalInt.empty() : OptionalInt.of(i);
}
```

【为什么这样更好】 规格只谈论参数、返回值与可预期的失败，客户读完就能正确使用，完全不需要知道内部用了什么数据结构；前置条件（`text` 非 null）被显式写出；"未找到"的语义由返回类型与文档共同固定（`-1` 或 `OptionalInt.empty()`），不再与"参数非法"混淆。规格稳定，因此实现可以自由演化。

【代码对比解说】 判断规格是否越界有个简单测试：**如果我把实现整个重写（换数据结构、换算法、改字段名），这份规格是否仍然完全正确？** 若答案是"否"，规格就写到了防火墙的错误一侧。同时注意"规格该说多少细节"的度：说少了客户无法编程（如"返回某个值"），说多了实现无法演化（如规定"返回最小下标"）。课程的判据是把**客户真正需要依赖的东西**写足，其余留白。

【设计原则透视】 这条规则是抽象屏障的直接体现，与 Reading 11（Abstraction Functions & Rep Invariants）互为表里：规格说明规定"外部可见行为"，表示不变量规定"内部表示必须满足的约束"，两者结合才能保证"实现可以自由更换而客户不受影响"。它也解释了为什么"在规格里写算法"是坏味道——那等于把 Reading 4 中"为变更而设计"的成果提前抵押掉。

场景 5：违反规格的测试——玻璃盒测试也不能越界

❌ *错误代码*

```
// 错误：测试依赖了规格从未承诺的行为，并且调用了违反前置条件的输入
@Test public void testFindBad() {
    int[] array = { 7, 7, 7 };
    int i = find(array, 7);
    assertEquals(0, i);                      // 规格只说"返回某个 i"，没承诺最小下标

    // 前置条件要求 val 在 arr 中出现；空数组违反前置条件，其行为未定义
    assertThrows(ArrayIndexOutOfBoundsException.class, () -> find(new int[0], 7));
}
```

【错误代码的问题】 1. 断言超出契约：`assertEquals(0, i)` 假定实现总是返回最小下标——如果实现优化成"从两端扫描"，这个测试会因为一个**并不违反契约**的改动而失败，测试成了重构的阻力。 2. 测试违反前置条件：空数组上调用 `find` 超出了规格定义范围，其行为是实现自由（`-1`、异常、任意值都合法），因此任何关于它的断言都没有依据；这类测试还会把"未定义行为"固化成客户可依赖的事实，破坏契约。 3. 测试与规格脱节：测试本应验证"实现是否满足规格"，这份测试却在验证"实现是否等同于当前这种写法"，失去了发现真正 bug 的能力。 4. 单元测试失去聚焦：它把"实现细节"与"契约行为"混在一处，失败时无法判断是规格变了、实现变了，还是测试写错了。

✅ *正确代码*

```
// 正确：只断言规格承诺的性质，用例选取遵循前置条件
@Test public void testFindHonorsSpec() {
    int[] array = { 7, 7, 7 };
    int i = find(array, 7);                  // 前置条件满足：7 在 array 中出现
    assertTrue(0 <= i && i < array.length, "returned index must be a valid index");
    assertEquals(7, array[i], "arr[i] must equal val");   // 这正是后置条件

    int[] single = { 42 };
    assertEquals(0, find(single, 42));       // 在"恰好出现一次"时，下标是唯一确定的
}

// 需要更强行为时，先改规格，再改实现与测试
/**
 * @param arr array to search; requires val occurs in arr
 * @return the LOWEST index i such that arr[i] == val
 */
public static int findFirst(final int[] arr, final int val) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == val) {
            return i;       // 从左向右扫描，因此第一个命中的就是最低下标
        }
    }
    return -1;              // 前置条件成立时不可达
}
```

【为什么这样更好】 断言直接来自后置条件——"存在下标 `i` 使得 `arr[i] == val`"，因此它对任何满足规格的实现都成立，不会成为重构的阻力；测试用例只使用满足前置条件的输入，因此从不依赖未定义行为；如

果某天客户真的需要"最小下标"这一更强保证，正确做法是**先加强规格**（如 `findFirst` 的 `@return the LOWEST index`），再让实现与测试跟上——契约是唯一的行为权威。

【代码对比解说】 玻璃盒测试的意义在于"用了解实现的知识去挑选**用例**（覆盖不同代码路径）"，而不是"用了解实现的知识去写**断言**"。例如知道实现是双端扫描，就应该分别构造"匹配元素在左半"与"在右半"的用例，但两者都只断言 `arr[i] == val`。同理，`extract()` 的测试不应依赖 `load()` 的行为（Reading 3 Testing 中的搜索引擎例子）：单元测试应只依赖被测方法的规格与标准库的规格。

【设计原则透视】 这一组把规格说明与测试的关系讲透：**规格是测试的预期来源，测试是规格的执行证据**。它也与 Reading 4（Code Review）呼应——审查者看到 `assertEquals(0, find(array, 7))` 时，第一反应应当是"规格承诺了最小下标吗？"如果没有，这条断言就是隐藏的技术债。

场景 6：Null 的静默传播 vs 显式的非空契约

❌ *错误代码*

```
// 错误：接受 null 并静默地把它传播下去，把错误推迟到很远的地方
public static List<String> toLowerCase(List<String> list) {
    if (list == null) {
        return null;                        // 静默传播：调用者迟早会遇到 NPE
    }
    List<String> out = new ArrayList<>();
    for (String s : list) {
        out.add(s == null ? null : s.toLowerCase());   // 元素里的 null 被悄悄保留
    }
    return out;
}
// 客户代码
List<String> lower = toLowerCase(names);
System.out.println(lower.get(0).length());  // 可能在完全无关的地方抛 NullPointerException
```

【错误代码的问题】 1. 静默传播：`null` 被当作合法输入接受又被当作合法输出返回，错误信息的"距离成因"被拉得很远，调试成本极高。 2. 规格缺失：既没有写"参数不得为 null"，也没有写"返回的列表不含 null 元素"，客户无从知道自己的义务与可依赖的保证。 3. 违反"避免 null"的默认约定：按照课程约定，除非规格明确声明，参数与返回值（包括集合元素）都不允许 null；这段代码既没有声明，也没有遵守。 4. 掩盖了两种不同的错误：参数是 `null`（客户 bug）与元素是 `null`（数据问题）应当以不同方式被暴露，这里两者都被无声吞下。

✅ *正确代码*

```
import java.util.Objects;

/**
 * Convert every string in a list to lower case.
 * @param list list of strings to convert; requires list is not null and
 *             contains no null elements
 * @return a new list t, same length as list, where t.get(i) is
 *         list.get(i).toLowerCase() for all valid indices i;
 *         list itself is not modified
 */
public static List<String> toLowerCase(final List<String> list) {
    Objects.requireNonNull(list, "list");              // 违反前置条件 → 立刻失败
    List<String> result = new ArrayList<>(list.size());
    for (String s : list) {
        result.add(Objects.requireNonNull(s, "element of list").toLowerCase());
    }
    return result;
}
```

【为什么这样更好】 隐式前置条件（非 null、元素非 null）被写进 `@param` 并用 `Objects.requireNonNull` 快速失败，客户在第一次调用时就会看到清晰的错误信息，而不是在几百行之后遇到 `NullPointerException`；后置条件明确写出"返回新列表、长度相同、逐元素小写、且**不修改**输入"，客户可以放心依赖；"不修改输入"这一条不再需要猜测，因为缺少关于修改的后置条件时默认就是不允许修改。

【代码对比解说】 Java 没有 TypeScript 那样的严格空检查（`strictNullChecks`），因此**约定 + 运行时检查 + 注解**三者合起来承担同样的职责：约定规定"默认非 null"，`Objects.requireNonNull` 提供廉价的运行时快速失败，`@NonNull` / `@Nullable` 注解（补充说明）让静态分析工具能在编译期给出提示。当确实需要表达"值可能缺失"时，使用 `Optional<T>` 而不是 `null`，并只在规格确实允许缺失的地方使用——这正是"克制地使用 Optional"的含义。

【设计原则透视】 这一组把 "Avoid null" 与 "Include emptiness" 两个概念联结起来：空列表、空字符串是**完全合法**的输入与返回（除非规格禁止），因此 `toLowerCase(new ArrayList<>())` 应当返回空列表而不是抛异常或返回 null；而 `null` 则是需要显式声明的例外。这也呼应 Reading 8（Immutability）：返回新对象而不是修改输入，使方法的行为更可预测，也让不可变值可以被安全共享。

与其他设计原则的关联

- **与 Reading 1（Static Checking）**：签名本身就是规格中"由编译器自动检查"的部分——参数类型、返回类型、受检异常的 `throws` 声明。规格说明则补齐编译器无法检查的那部分，两级检查共同构成"尽早失败"的体系。
- **与 Reading 3（Testing）**：测试必须依据规格编写，黑盒测试只用规格挑选用例，玻璃盒测试用实现知识挑选用例但仍只用规格写断言；单元测试聚焦单一规格，集成测试检验规格之间是否兼容。
- **与 Reading 4（Code Review）**：本讲回答"注释该写什么"——最重要的注释就是规格说明；上一讲的"避免魔法数字""返回结果而不是打印""避免特例代码"在接口层面表现为"不要用 `-1` / `null` 表示失败""后置条件要说明返回值"。

- **与 Reading 7（Designing Specifications）**：本讲建立了前置/后置条件与强弱判据，下一讲将系统讨论如何设计规格：前置条件该不该检查、如何决定规格的强弱、确定性（determinism）与欠定性（underdetermination）等。
- **与 Reading 8（Immutability）**：规格默认要求"不修改输入"，这与不可变对象的设计互为支撑；`Optional` 与不可变值可以安全共享，无需防御性拷贝。
- **与 Reading 11（Abstraction Functions & Rep Invariants）**：规格说明是抽象边界的外侧（客户可见的行为），表示不变量与抽象函数是内侧（实现必须维持的性质）；两者共同保证"实现可以自由更换"。
- **与 Reading 21–23（Concurrency / Locks）**：并发方法必须在线程安全的规格中写明（例如"this method is thread-safe"或"requires no other thread is mutating this object"），否则客户无法判断能否并发调用——规格说明是并发契约的唯一载体。
- **与 Reading 29（Team Version Control）**：规格说明是团队分解工作的前提：只有接口契约稳定，不同成员才能并行实现与修改不同模块。

关键要点

- **规格是一份双方契约**：客户承担前置条件，实现者承担后置条件；整体是 `pre → post` 的逻辑蕴含，前置条件不成立时实现完全自由（可以返回任意值、抛任意异常、做任意修改、甚至不返回）。
- **规格说明决定可替换性与可依赖范围**：只有当规格明确写出客户可以依赖什么，两个实现才可能"行为等价"；客户不能依赖规格之外的行为，测试也不能断言规格之外的结论。
- **写规格只谈可观察行为**：参数、返回值、异常、对参数的修改；绝不谈局部变量、私有字段与算法。判据是"重写实现后这份规格是否仍然正确"。
- **默认规则要记住三条**：参数与返回值（含集合元素）默认非 null；空值（空串、空列表）默认合法；除非写明修改，否则不得修改输入。
- **异常分两类，写法不同**：信号 bug 的异常（`NullPointerException`、`IndexOutOfBoundsException`）绝不写进规格；信号可预期失败的异常必须写 `@throws`，在 Java 中受检异常还要写进签名 `throws`。当失败难以被调用者处理时用异常，当失败是调用者必须处理的常见分支时用 `Optional` 这类显式结果类型。

常见陷阱与注意事项

1. **用 `-1`、`null` 或 `false` 表示失败** → 失败变成一个看起来合法的值，客户容易漏检，错误静默传播到很远的地方；语义也模糊（`null` 无法区分"值就是 null"与"键不存在"）。补救：用异常表达"调用者难以处理的问题"，用 `Optional`/联合类型表达"可预期的缺失"。
2. **在规格里描述实现**（提及私有字段、局部变量、具体算法） → 规格与实现绑死，任何内部重构都要同步改文档，客户还可能依赖内部细节；抽象屏障被击穿。补救：只写参数、返回值与副作用，并用"重写实现后规格是否仍正确"来检验。
3. **测试依赖规格未承诺的行为，或调用违反前置条件的输入** → 测试变成重构的阻力，还会把未定义行为固化成"事实"；真正的 bug 反而测不出来。补救：断言只来自后置条件，用例只使用满足前置条件的输入；需要更强保证时先改规格。

4. **忘记隐式前置条件与默认不修改约定** → 方法在 `null` 上神秘崩溃，或悄悄修改了客户传入的集合（客户以为它是只读的），引发极难复现的 bug。补救：在入口用 `Objects.requireNonNull` 快速失败，并在 `@param` 中写明非空要求；后置条件中要么声明修改，要么保证返回新对象。
5. **把"信号 bug 的异常"写进 `@throws`，或让未受检异常出现在 `throws` 子句里** → 前者让规格充满噪声（客户根本无法、也不应该处理这类异常），后者误导读者以为它是受检异常。补救：只把可预期失败写入规格；受检异常才写进签名。
6. **用 `catch (Exception e) { return; }` 让编译器闭嘴** → 连 `IndexOutOfBoundsException` 这类真正的 bug 都被静默吞掉，程序表现出难以理解的行为，且没有任何堆栈线索。补救：捕获最具体的异常类，捕获块要尽量窄；需要换层语义时做异常转译（如 `PathNotFoundException` → `RobotStuckException`）。

思考题（带答案）

问题 1：下面这份规格是否允许实现在"`arr` 为空"时抛异常？为什么？如果你希望客户能够依赖"空数组时返回 –1"，应该怎样修改规格?

```
/**
 * Find a value in an array.
 * @param arr array to search; requires val occurs exactly once in arr
 * @param val value to search for
 * @return index i such that arr[i] == val
 */
public static int find(int[] arr, int val)
```

答案：允许，而且实现可以做任何事。规格的前置条件是"`val` 在 `arr` 中恰好出现一次"；当 `arr` 为空时该前置条件必然不成立，于是实现不受后置条件约束——抛出规格未提及的异常、返回任意值、修改 `arr` 都是合法的。这解释了为什么规格完全不必提及实现中那句 `return -1`：在前置条件成立时它不可达，规格只描述**合法调用**下的行为。如果希望客户能依赖"空数组时返回 –1"，就必须把规格改强：删掉"`val` 恰好出现一次"这一前置条件（或改弱为"`arr` 非 null"），并在后置条件中写明"若 `val` 不在 `arr` 中则返回 –1"。注意这是"前置条件更弱 + 后置条件更强"的组合，因此新规格比原规格更强，实现者的负担更重，但客户获得了更多可依赖的保证。

问题 2：`countLongWords` 的两个版本——一个返回 `int` 并把最长单词写进全局变量，另一个返回一个不可变的小对象——哪一个更容易写出好的规格说明？请从"规格可以谈论什么"与"后置条件的完整性"两个角度解释。

答案：第二个版本容易得多。第一，规格只能谈论参数、返回值与副作用；第一版的第二个结果藏在**全局变量**里，规格根本无法用"返回值"的方式描述它，只能写"同时把 `longestWord` 全局变量设为……"，这既依赖了实现细节（全局状态），又让客户必须知道该去看哪个名字。第二，第一版还有"打印到控制台"这一副作用，规格必须额外描述它的输出格式，而后置条件本应描述状态而非人类可见的输出。第二版把两个结果打包进一个不可变的返回值对象，后置条件可以完整写成"返回一个 `WordStats`，其中 `count()` 是长度超过 `LONG_WORD_LENGTH` 的单词数，`longestWord()` 是最长单词（无单词时为 `""`）"，客户一读即懂，实现也能自由更换内部算法。这也印证了 Reading 4 的"函数应返回结果而不是打印"——那条规则的可维护性收益，本质上来自它让规格变得可写。

问题 3：某团队规定"任何异常都必须写进 `@throws`"，于是 `find` 的规格变成了：

```
/**
 * @throws NullPointerException if arr or val is null
 * @throws ArrayIndexOutOfBoundsException if the implementation has an index bug
 * @throws OutOfMemoryError if the array is enormous
 * @return index i such that arr[i] == val; requires val occurs exactly once in arr
 */
```

请评价这份规格，并说明"异常与前置条件的关系"应当如何正确理解。

答案：这份规格是错误的。`NullPointerException` 属于"信号 bug 的异常"：参数非 null 已经是**隐式前置条件**，实现可以在客户违反时自由抛出它，因而不属于后置条件，不该出现在 `@throws` 中；把它写出来反而会让读者以为需要处理它。`ArrayIndexOutOfBoundsException` 是**实现的 bug**，它根本不该发生，写进规格等于把缺陷当成契约的一部分。`OutOfMemoryError` 属于 `Error` 家族，是 JVM 层面的资源失败，同样不应作为方法契约的一部分（Java 约定：不要继承 `Error`，也不要把它当作正常的失败通道）。正确的理解是：**异常只有两种合法角色**——一是把"前置条件被违反"变成可诊断的失败（可写可不写，通常只需隐含在前置条件里；受检异常才必须写进签名），二是表达**可预期的失败**并让客户能够响应（必须写进 `@throws`，受检异常还要写进 `throws`）。因此 `integerSquareRoot` 的规格应当写 `@throws NotPerfectSquareException if x is not a perfect square` 并用 `OptionalInt` 或受检异常承载"缺失"，而不是罗列一堆 bug 型异常。

Reading 7: 设计规格（Designing Specifications）

> 说明：本讲 sp22 原版使用 TypeScript，本笔记按用户要求提供 Java 代码示例；类型/API 与 sp21（6.031 Java 版）原文保持一致。

概述

上一讲（Reading 6: Specifications）讲清了「规格是什么」：前置条件（precondition）是客户必须满足的义务，后置条件（postcondition）是实现者必须兑现的承诺，只要双方都守约，实现者就可以自由替换实现。本讲要回答的是更深一层的问题：**当我们可以为同一段行为写出多份不同的规格时，哪一份更好？** MIT 6.031 给出了三个比较维度——**确定性**（deterministic 还是欠定 underdetermined）、**声明性**（declarative 还是操作式 operational）、**强度**（strong 还是 weak），并进一步讨论「什么样的规格是连贯、有用、松紧得当的」。

这三个维度直接服务于课程的三大目标：欠定与合适强度的规格让实现者能在规格内部自由更换算法，从而 **Ready for change**；清晰、声明式、连贯的规格让客户不必读实现代码就能理解用法，从而 **Easy to understand**；而明确的前置/后置条件和「快速失败（fail fast）」的取舍，让误解与非法调用尽早暴露，从而 **Safe from bugs**。一句话总结本讲的核心命题：**写函数主要就是写规格，而规格的质量决定模块的可用性与可演化性。**

核心概念与设计原则详解

规格即防火墙（Specification as a Firewall） – **定义与目的**：规格是客户与实现者之间的一道屏障：客户无需阅读源码即可调用模块，实现者无需知道调用现场即可编写实现。它同时保护「安全性」（双方不越界）、「易理解性」（客户只读契约）与「可修改性」（实现可替换）。 – **直观解释（"它是什么？"）**：把规格想象成一块围起来的场地。实现者可以在场地内部任意走动——换算法、优化性能、重写代码——只要不出界，客户就不会受影响；客户则站在场地外，只能依据场地边界（规格）来规划自己的行为。围栏画得越粗糙（规格越弱），实现者的活动空间越大，但客户的确定性越小；围栏画得越精细（规格越强），客户越安心，实现者越受限。 – **关键规则与最佳实践**：规格只描述「客户端可观察到的行为」，不描述内部步骤；规格一旦发布，实现者只能在不改变规格的前提下改代码；如果要改规格本身，必须确认新规格「强于或等于」旧规格（见下文强弱规则），否则所有既有客户都需要复查。

确定性规格与欠定规格（Deterministic vs. Underdetermined Specs） – **定义与目的**：确定性规格（deterministic spec）对每个满足前置条件的输入只允许一个合法输出；**欠定规格（underdetermined spec）**允许多个合法输出，把「选哪一个」的自由留给实现者。它主要服务于 **Ready for change**：实现者可以在不改契约的前提下更换策略。 – **直观解释（"它是什么？"）**：确定性的规格像「点一份五分熟的牛排」——结果唯一；欠定的规格像「随便来一份主食」——只要端上来的是主食就算合格。注意 6.031 特意区分了「欠定（underdetermined）」与「非确定（nondeterministic）」：非确定指的是**代码**在相同输入下时而这样、时而那样（依赖随机数或并发时序）；而欠定的**规格**完全可以由一段完全确定的实现来满足，欠定性只是「契约允许的选择空间」。 – **关键规则与最佳实践**：写规格时先问「客户是否真的在意返回哪一个？」；如果不在意，用欠定规格换取实现自由（例如「返回任意一个满足 arr[i] == val 的 i」）；如果在意，就必须把它写进后置条件（例如「返回满足条件的最小下标」），否则客户不能依赖它；欠定不等于「没说清」——「返回任意一个出现位置」是精确的欠定，而「返回一个差不多对的下标」则是烂规格。

声明式规格与操作式规格（Declarative vs. Operational Specs） – **定义与目的**：操作式规格（operational spec）以「步骤」描述方法怎么做（伪代码式描述属于此类）；声明式规格（declarative spec）只刻画最终结果的性质以及它与初始状态的关系。声明式规格通常更短、更好懂，并且**不会意外泄漏实现细节**，因此同时服务于 **Easy to understand** 与 **Ready for change**。 – **直观解释（"它是什么？"）**：声明式说「结果是什么」，操作式说「过程怎么走」。就像菜谱：声明式是「一盘麻婆豆腐」，操作式是「先切葱、再下锅、翻三下、起锅」——后者一旦被别人当成验收标准，厨师就再也不能改用别的做法了。 – **关键规则与最佳实践**：规格注释里永远不要写实现解释（那是给维护者看的，应写在方法体内部）；同一份行为往往有多种声明式写法（用后缀拼接、用 substring、用前 prefix.length() 个字符等），要挑对客户和维护者最清晰的一种；当你想写「先……再……最后……」时，先停下来问：客户真的需要知道中途状态吗？只有像 `addAll` 那样「中途失败会留下部分效果」时，阶段信息才必须写进规格。

规格的强弱（Stronger vs. Weaker Specs） – **定义与目的**：若满足规格 S2 的实现集合是满足 S1 的实现集合的**真子集**，则称 S2 比 S1 强（stronger）。强弱是判断「能否安全替换规格」的唯一标准，直接服务于 **Ready for change**。 – **直观解释（"它是什么？"）**：强弱来自谓词逻辑——谓词 P 强于 Q，意味着满足 P 的

状态集合更小。规格整体是「关于实现的谓词」，所以规格越强 = 约束越多 = 合法实现越少。可以记成「强的更紧、弱的更松」。 – **关键规则与最佳实践**：核心判定规则是——**S2 强于或等于 S1，当且仅当 S2 的前置条件弱于或等于 S1 的前置条件，且 S2 的后置条件（在 S1 前置条件成立的输入上）强于或等于 S1 的后置条件**。由此得到两条操作口诀：**前置条件可以随时放宽**（对客户要求更少，永不伤害客户），**后置条件可以随时加强**（对客户承诺更多，也永不伤害客户）。两个规格可能互不可比（incomparable）：既不是子集关系，也没有重叠或完全分离，此时替换必须逐个客户审查。

规格空间图（Diagramming Specifications） – **定义与目的**：把所有可能的实现想象成一个巨大空间里的点集，一份规格就划出其中一片区域，实现要么落在区域内（满足规格），要么在区域外。这个心智模型用于直观理解「强弱」「不可比」与「防火墙」。 – **直观解释（"它是什么？"）**：`findFirst` 与 `findLast` 是两个**点**（实现），不是区域（规格）；它们都落在 `findOneOrMore,AnyIndex` 这片区域里，因此彼此可替换。加强后置条件会缩小区域（要求更高的输出），放宽前置条件也会缩小区域（要求处理更多输入，那些以前被排除的坏行为现在暴露了），所以**越强的规格区域越小，越弱的规格区域越大**。两个不可比的规格可能重叠、也可能完全分离。 – **关键规则与最佳实践**：判断包含关系时，分别看两个方向——「在 S1 区域里的实现是否一定在 S2 区域里」以及「在 S2 区域里的实现是否一定在 S1 区域里」；只要有一个方向不成立，就不存在单向的强于关系；用图来解释「为什么不能随手削弱后置条件」最有效：那会让一批既有客户脚下突然出现空白区。

前置条件还是后置条件（Precondition or Postcondition? Fail Fast） – **定义与目的**：设计规格时必须决定：某个要求是写成前置条件（客户的责任），还是写成后置条件（实现者负责检查并抛出异常）？这关系到 Safe from bugs：越早失败，越容易定位 bug。 – **直观解释（"它是什么？"）**：前置条件实质上是「把检查成本转嫁给客户」。所以前置条件最常见的用途，恰恰是那些**实现者检查起来很贵或很难**的性质：例如用二分查找实现 `find` 时要求数组已排序，如果强制实现者去验证有序性，就会把对数时间变成线性时间，二分查找的意义荡然无存。 – **关键规则与最佳实践**：非平凡的前置条件会给客户添麻烦——一旦违反，程序没有任何可预期的恢复方式，所以用户不喜欢前置条件；因此 Java API 与许多 JavaScript 库倾向于**用后置条件规定：参数不合适时抛出未检查异常（unchecked exception）**，让调用方的错误假设立刻暴露；通用的工程判断标准是两条——**检查的代价**（写代码与运行时的成本）与**方法的作用域**（只在类内部调用的私有方法可以用前置条件 + 小心审查所有调用点；对外公开的方法更应抛异常）；即使某个接口把要求写成前置条件，只要它同时承诺「违反时抛出特定异常」，那它在语义上就是后置条件。

好规格的准则：连贯、有信息量、足够强也足够弱、抽象类型（Designing Good Specifications） – **定义与目的**：形式（简短、清楚、结构好）容易做到，内容难有定规，但有几条可靠的经验准则，服务于 Easy to understand 与 Ready for change。 – **直观解释（"它是什么？"）**：一个**连贯（coherent）**的规格能让客户把它当作一个完整、单一的功能来理解——参数列表很长、用布尔开关切换行为、逻辑错综复杂，都是坏信号（例如 `sumFind(int[] a, int[] b, int val)` 同时「在两个数组里查找」和「把下标求和」，本应拆成两个方法）；一个规格的**结果应当有信息量（informative）**——如果返回 `null` 既表示「原来没有这个键」又表示「原来的值就是 null」，返回值就毫无用处；规格要**足够强**（特例不能毁掉一般用途，例如 `addAll` 若允许在抛异常前追加一部分元素，客户就不知道到底追加了什么）；也要**足够弱**（例如 `open` 不能保证「一定打开文件」，因为权限与文件系统故障不在程序掌控之中，它只能承诺「尝试打开，若成功则满足某些性质」）；

能用**抽象类型**就用抽象类型（返回 `List` 而不是 `ArrayList`，接口类型而不是具体实现类），这同时给客户和实现者留出自由。 – **关键规则与最佳实践**：先写规格再写实现（设计方法就是设计规格）；用「一个完整单元」的标准自查连贯性，发现多职责就拆分；检查特殊返回值是否与正常返回值语义冲突，冲突就用更明确的机制（显式查询、抛异常、`Optional`）；为每个特例问一句「这个特例会不会让整个方法对客户变得没用」；比较参数与返回值的类型，凡是「比行为所需更具体」的类型都换成抽象类型。

信息隐藏与自由度（Information Hiding, Access Control & Freedom） – **定义与目的**：规格是信息隐藏（information hiding）的载体：它只暴露客户需要知道的东西，把实现细节挡在契约之外，从而同时保护 **Ready for change** 与 **Easy to understand**。 – **直观解释（"它是什么？"）**：客户看模块是「透过规格这扇窗」看，看不到内部；只要窗上写的东西不变，屋里怎么装修都可以。对应到 Java 语言层面，`public` / `private` 的选择本身就是在画契约边界：把只供内部使用的辅助方法设为 `public`，等于向全世界承诺「我会一直提供这个方法」，将来就难以改动内部实现，还让公开接口变得杂乱（接口越小越连贯）。 – **关键规则与最佳实践**：默认把字段与内部辅助方法声明为 `private`，只在确实要对外提供服务时用 `public`；规格注释中不要提及任何内部变量、内部数据结构或算法步骤；用抽象类型（如 `List`、`Map`）而不是具体类型（如 `ArrayList`、`HashMap`）书写规格；当发现自己想用规格注释「顺便解释实现」时，把那段话移到方法体里。

代码示例与对比分析

场景 1：用实现步骤写 `find` 的规格，而且把「最小下标」这一实现细节写成了契约

❌ *错误代码*

```
// 错误：规格写成操作式，并且把 findFirst 的实现特征（返回最小下标、
// 找不到时返回 arr.length）当成了契约的一部分。
/**
 * 从头开始扫描数组：i = 0, 1, 2, ...,
 * 每次比较 arr[i] == val，一旦相等就返回 i，
 * 如果走完整个数组都没找到，就返回 arr.length。
 */
public static int find(int[] arr, int val) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == val) return i;
    }
    return arr.length;
}
```

【错误代码的问题】 1. **泄漏实现细节**：规格里写死了「从头开始扫描」「返回最小下标」「未找到返回 length」，任何客户只要读了这个注释就可能依赖这些细节。 2. **排除合法实现**：按这份规格，`findLast`（从尾部扫描、未找到返回 –1）不满足契约，但它其实是完全合理的实现，实现者的自由度被白白剥夺（违背 Ready for change）。 3. **语义含糊**：arr.length 这个「未找到」的哨兵值在 Java 中会被静默当作一个合法下标，客户若忘记检查就会得到 `ArrayIndexOutOfBoundsException`（违背 Safe from bugs）。 4. **文档与代

码重复：注释逐句翻译了方法体，一旦代码改动，注释立刻过期，成为误导读者的隐患（违背 Easy to understand）。

✅ *正确代码*

```
// 正确：声明式规格 + 允许两种实现的欠定后置条件。
// 两个实现都满足下面这一份规格，因此它们可以互相替换。
/**
 * 在数组中查找一个值。
 * @param arr 被搜索的数组
 * @param val 要查找的值
 * @return 满足 arr[i] == val 的下标 i
 * requires: val 在 arr 中至少出现一次
 */
public static int find(int[] arr, int val) { /* 见下方两种实现 */ return 0; }

// 实现 A：从前往后扫描
public static int findFirst(int[] arr, int val) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == val) return i;
    }
    return arr.length;   // 前置条件保证不会走到这里
}

// 实现 B：从后往前扫描
public static int findLast(int[] arr, int val) {
    for (int i = arr.length - 1; i >= 0; i--) {
        if (arr[i] == val) return i;
    }
    return -1;           // 前置条件保证不会走到这里
}
```

【为什么这样更好】 规格只声明「返回某个满足 arr[i] == val 的下标」，这是一个精确的**欠定**后置条件：它对客户是完整的信息（客户知道拿到的下标一定命中），对实现者则留出了「从哪头扫」的自由。`findFirst` 与 `findLast` 都落在这份规格划定的区域内，实现者可以在区域内任意改动（换算法、优化缓存局部性），客户一行代码都不用改。

【代码对比解说】 错误写法与正确写法的差别不在代码，而在**注释的语义层级**：前者把「实现的控制流」提升成了「契约」，后者把「结果的性质」写成了契约。判断标准很简单——问自己「客户有没有权利依赖这一点？」。客户有权依赖「返回的下标处确实是 val」，但无权依赖「返回的是最小下标」，除非规格明确承诺。如果业务确实需要最小下标，那也应该写成后置条件（`findOneOrMore,FirstIndex`：返回最低下标），而不是描述扫描方向——这两者结果相同，但前者是实现无关的声明，后者是操作式描述。

【设计原则透视】 这是「规格作为防火墙」的直接体现，也是抽象边界（abstraction boundary）的维护：契约层只谈可观察行为，实现层才谈步骤。它同时展示了「欠定规格」的正确用法——欠定不是模糊，而是把实现选择权明确地留给实现者。与 Reading 6（Specifications）中的前置/后置条件结构完全一致，也为 Reading 10、11（抽象数据类型、抽象函数与表示不变量）中「客户只能看见 AF 所描述的抽象值」埋下伏笔。

场景 2：`startsWith` 的规格写成伪代码步骤，而不是结果的性质

❌ *错误代码*

```
// 错误：操作式规格——把「怎么比」写了进去，还泄漏了内部实现可能用的下标范围。
/**
 * 令 i 从 0 开始，每次把 str.charAt(i) 与 prefix.charAt(i) 比较，
 * 若不同则立即返回 false，若相同则 i 加一，直到 i 等于 prefix.length()，
 * 此时返回 true。
 */
public static boolean startsWith(String str, String prefix) { /* ... */ return false; }
```

【错误代码的问题】 1. **不可用的契约**：客户无法从「i 从 0 开始循环」推断出任何可直接使用的结果性质，只能自己脑内模拟循环。 2. **绑定实现**：把「逐个字符比较」写进契约，排除了任何其他实现（例如先用 `str.substring(0, prefix.length()).equals(prefix)` 的写法），也排除了未来用更快的原生实现替换的可能。 3. **边界情况没交代**：操作式描述没有说明 `prefix` 比 `str` 长时会怎样，客户的疑问依然存在（违背 Easy to understand）。

✅ *正确代码*

```
// 正确：三种等价的声明式规格，任选其一（以下是推荐的第一种，最贴近“拼接”的直觉）。
/**
 * 判断 str 是否以 prefix 开头。
 * @param str    被检查的字符串
 * @param prefix 前缀
 * @return 当且仅当存在字符串 suffix 使得 prefix + suffix 等于 str 时返回 true
 */
public static boolean startsWith(String str, String prefix) {
    return str.startsWith(prefix);   // 也可以用下面的等价实现
}

// 等价的声明式写法之二：存在某个整数 i 使得 str.substring(0, i).equals(prefix)
// 等价的声明式写法之三：str 的前 prefix.length() 个字符与 prefix 的字符完全相同
```

【为什么这样更好】 后置条件是一句可以直接当数学性质使用的断言：「存在 suffix 使 `prefix + suffix = str`」。客户读一遍就知道所有边界情况（包括 `prefix` 为空串时恒为 true、`prefix` 比 `str` 长时不可能是后缀拼接的结果），实现者也可以自由选择任何等价实现。三种声明式写法在语义上完全等价，选择哪一种只关乎「对读者最清楚」。

【代码对比解说】 声明式写法都有一个共同特征：它们在描述**输出与输入的数学关系**（存在量词、下标约束、字符相等），而不是描述**机器的动作序列**。要注意「等价」并不意味着可以混着写——不要在声明式规格里夹一句「它先比较首字符」，那会把客户重新拉回实现细节。此外，如果一段实现解释对维护者真的有用（例如「这里用了 Boyer–Moore 以加速」），应该写成方法体内部的普通注释，而不是规格注释。

【设计原则透视】 这体现了抽象边界的两侧分工：规格面向客户，只谈可观察性质；实现注释面向维护者，才谈步骤与技巧。它与本讲「声明式优于操作式」的结论一致，也是 Reading 4（Code Review）中「注释要写规格而非代码翻译」原则的延伸。

场景 3：`put` 的返回值用 `null` 同时表示「原来没有键」和「原来的值就是 null」

❌ *错误代码*

```
// 错误：前置条件允许 val 与 map 中存 null，后置条件却用 null 表示“键不存在”，
// 返回值因此毫无信息量。
/**
 * requires: val 可以为 null，map 中也可以存放 null 值
 * effects:  把 (key, val) 插入映射，覆盖 key 原有的映射，
 *           并返回 key 原来的值；如果原来没有映射，则返回 null
 */
public static <K, V> V put(Map<K, V> map, K key, V val) {
    return map.put(key, val);   // 无法区分“没有键”与“原值为 null”
}
```

【错误代码的问题】 1. **信息丢失**：调用方拿到 `null` 时无法判断键是「从未存在」还是「存在且值为 null」，只能靠自己额外维护状态。 2. **契约自相矛盾**：前置条件主动允许 null 值存在，后置条件又拿 null 当哨兵，等于在同一份契约里让一个值承担两种互斥含义。 3. **诱导错误代码**：客户为了区分情况，往往会写 `if (map.containsKey(key)) {...} else {...}` 再调用一次 `put`，既重复又引入竞态窗口（在并发场景下更危险）。

✅ *正确代码*

```
// 正确：让“没有键”这一情况拥有自己的表示，而不是借用 null。
// 补充说明：Optional 来自 Java 8 的 java.util，不属于 6.031 原文用法，
// 这里作为“结果要有信息量”这一原则的 Java 生态实现方式给出。
/**
 * 把 (key, val) 插入映射，覆盖 key 原有的映射。
 * @return 键原有的值；若该键此前没有映射，返回 Optional.empty()
 */
public static <K, V> Optional<V> put(Map<K, V> map, K key, V val) {
    final boolean hadKey = map.containsKey(key);
    final V old = map.put(key, val);
    return hadKey ? Optional.ofNullable(old) : Optional.empty();
}
```

【为什么这样更好】 「没有键」与「值为 null」被编码成两个可区分的返回值（`Optional.empty()` 与 `Optional.of(null)`——实际上 `Optional.ofNullable(null)` 会得到 `Optional.empty()`，因此更稳妥的写法是让契约禁止 null 值，或用第三个状态、抛异常等机制），客户只需一次调用、一次判断就能得到完整信息。这也是「结果应当有信息量」这条准则的最直接应用。

【代码对比解说】 两份代码的实现几乎一样（都是 `map.put`），差别全在**返回值的语义设计**上。注意上面正确版本仍有一个残留陷阱：如果 `val` 与 `old` 都允许是 null，`Optional.ofNullable` 依旧会混同两者——所以在真实设计中通常要二选一：要么在前置条件中禁止 null（让 `Optional` 精确表达「键缺失」），要么用「是否包含键 + 值」双重查询。这正好说明：**规格设计不是加个新类型就完事，必须让新机制在同一份契约内保持语义无歧义**。

【设计原则透视】 这是「规格应当有信息量」与「前置条件/后置条件要一致」两条准则的交点。它也提醒我们：契约中出现的每个特殊值都必须有唯一定义，否则客户的推理链（以及 Reading 3 的测试设计）会出现

漏洞。这里的选择（禁止 null vs 引入显式缺失值）本质上是在权衡「前置条件强度」与「后置条件表达能力」。

场景 4：`addAll` 的规格太弱——抛异常前允许已追加一部分元素

❌ *错误代码*

```
// 错误：规格没有说明失败时的原子性，实现按“边检查边追加”写，
// 于是异常抛出时 list1 已经被改了一部分。
/**
 * effects: 把 list2 中的元素按顺序追加到 list1 末尾，
 *          除非在 list2 中遇到 null 元素，此时抛出 NullPointerException
 */
public static <T> void addAll(List<T> list1, List<T> list2) {
    for (T t : list2) {
        if (t == null) throw new NullPointerException("list2 contains null");
        list1.add(t);
    }
}
```

【错误代码的问题】 1. **失败后状态不明**：客户捕捉到异常后完全不知道 `list1` 里到底追加了哪些元素，必须自己写代码去比对，而这份信息理论上只有实现者才知道。 2. **规格太弱以致无用**：这份规格虽然比「不提 null 元素」的版本强，但仍不足以让客户安全地使用——常见做法只能整体拒绝这个结构。 3. **难以测试**：测试用例无法写出确定的期望结果（是「抛异常」还是「抛异常 + list1 部分改变」？），违背 Reading 3（Testing）中「测试要能断言确定结果」的要求。

✅ *正确代码*

```
// 正确：强化后置条件，明确失败时的原子性——先检查，再一次性追加。
/**
 * effects: 把 list2 中的元素按顺序追加到 list1 末尾。
 *          如果 list2 中含有 null 元素，则抛出 NullPointerException，
 *          并且不向 list1 追加任何元素。
 */
public static <T> void addAll(List<T> list1, List<T> list2) {
    if (list2.contains(null)) {
        throw new NullPointerException("list2 contains null");
    }
    list1.addAll(list2);   // 检查通过后一次性完成，不留部分效果
}
```

【为什么这样更好】 强化后的规格把「失败时的状态」也变成契约的一部分：要么完全成功，要么完全不改（原子性）。客户因此可以在 `catch` 块里安心地继续使用 `list1`，测试也能精确断言。注意这里的强化方向符合本讲的强弱规则——**加强后置条件对客户永远是无害的**，因为它只是多给了一个承诺。

【代码对比解说】 两份实现的行为差异只在「检查时机」：错误版本把检查与追加交织在一起（对应规格里的「unless it encounters…」），正确版本先整体验证再整体执行。从规格角度看，前者把「中途状态」暴露给

客户，后者把它隐藏起来；从性能角度看，`contains(null)` 多走一遍列表，属于用一点点开销换取契约的可用性——这正是「规格强弱是工程判断」的典型例子。

【设计原则透视】 这条对比把「规格应当足够强」落到了具体代码上：规格强度不足时，客户必须承担额外的推理负担甚至引入自己的容错代码。它还示范了「异常作为后置条件」的写法（与 Reading 6 一致），并说明了「先检查后执行」可以同时改善 Safe from bugs 与 Easy to understand。

场景 5：规格里用具体实现类型 `ArrayList`，把客户与实现者同时锁死

❌ *错误代码*

```
// 错误：规格依赖具体实现类型，客户必须传 ArrayList，
// 实现者也只能返回 ArrayList，即使它内部用的是更合适的 List 实现。
/**
 * @param list 待反转的列表
 * @return 反转后的新列表，newList[i] == list.get(n - i - 1)，n == list.size()
 */
public static ArrayList<String> reverse(ArrayList<String> list) {
    final ArrayList<String> result = new ArrayList<>();
    for (int i = list.size() - 1; i >= 0; i--) {
        result.add(list.get(i));
    }
    return result;
}
```

【错误代码的问题】 1. **无谓地限制客户**：行为与 `ArrayList` 的任何具体特性无关，客户却被迫把数据放进 `ArrayList`；传 `List.of(...)` 产生的不可变列表就直接编译失败。 2. **无谓地限制实现者**：实现者被迫构造并返回 `ArrayList`，无法返回 `Collections.unmodifiableList(...)` 之类的视图，也难以改成返回链表等更合适的结构。 3. **削弱可修改性**：将来若要把实现换成别的数据结构，签名变更会波及所有调用点（违背 Ready for change）。

✅ *正确代码*

```
// 正确：规格使用抽象类型 List，客户端与实现端都获得自由。
/**
 * @param list 待反转的列表
 * @return 反转后的新列表，newList[i] == list.get(n - i - 1)，n == list.size()
 */
public static List<String> reverse(List<String> list) {
    final List<String> result = new ArrayList<>();
    for (int i = list.size() - 1; i >= 0; i--) {
        result.add(list.get(i));
    }
    return Collections.unmodifiableList(result);   // 实现细节，客户无需知道
}
```

【为什么这样更好】 抽象类型是「规格层面」与「实现层面」的解耦点：客户只需提供任何 `List`（包括不可变视图），实现者可以自由挑选内部结构并返回 `List` 的任何合法实现。Java 中这意味着尽量用接口类型

（`List`、`Set`、`Map`、`Reader`）而不是具体类（`ArrayList`、`HashSet`、`HashMap`、`FileReader`）。

【代码对比解说】 两份代码的算法完全相同，唯一区别是签名里写 `ArrayList` 还是 `List`。但这一字之差改变了契约的「作用域」：具体类型把契约绑定到某个实现的细节上，抽象类型只绑定到行为。注意正确版本顺便演示了一个好习惯——返回不可变视图（见 Reading 8: Mutability & Immutability），这是实现者的自由，且不改变规格承诺。

【设计原则透视】 这是信息隐藏（information hiding）在签名层面的落实，也是 Reading 12（Interfaces, Generics, Enums）中「面向接口编程」的前置铺垫。它同时体现了本讲的用法：规格只承诺客户关心的东西，凡是行为不需要的具体类型都应当抽象掉。

场景 6：`countLongWords` 规格不连贯，一个方法干三件事

❌ *错误代码*

```
// 错误：规格不连贯——同时改全局变量、打印到控制台、还提到局部变量 words。
public static int LONG_WORD_LENGTH = 5;
public static String longestWord;

/**
 * 把 longestWord 更新为 words 中最长的元素，
 * 同时把长度大于 LONG_WORD_LENGTH 的元素个数打印到控制台。
 * @param text 要搜索的文本
 */
public static void countLongWords(String text) {
    // ... 实现略：既更新全局变量，又打印，还依赖未说明的 words
}
```

【错误代码的问题】 1. **不连贯**：把「统计长单词」和「找出最长单词」两件不相关的事塞进一个方法，客户为了其中一件必须承受另一件的副作用。 2. **通过全局变量通信**：结果写到 `public static` 字段里，任何代码都能在任意时刻改它，bug 的影响范围无法界定（违背 Safe from bugs，也为 Reading 9 中的「作用域最小化」提供了反面教材）。 3. **打印而非返回**：输出无法被测试断言，也无法被其他代码复用（违背 Reading 3 的测试原则）。 4. **规格提到局部变量 `words`**：契约暴露了实现内部的名字，客户读不懂，维护者也容易被过期名称误导。

✅ *正确代码*

```
// 正确：拆成两个连贯的方法，各自返回结果，不碰全局状态。
/**
 * @param text 要搜索的文本
 * @return text 中长度大于 minLength 的单词个数
 */
public static int countLongWords(String text, int minLength) { /* ... */ return 0; }

/**
 * @param text 要搜索的文本
 * @return text 中最长的单词；若没有单词则返回 Optional.empty()
 */
public static Optional<String> longestWord(String text) { /* ... */ return Optional.empty()
```

【为什么这样更好】 每个方法只做一件事，规格可以写成一句话；返回值取代了全局变量与打印，于是同一份逻辑既能被测试、又能被复用；`minLength` 作为参数取代全局常量，参数化之后方法在其他上下文中也能使用（Ready for change）。

【代码对比解说】 注意错误版本其实还违反了 DRY（它需要自己把文本切成单词两次），拆分之后「分词」这段逻辑可以抽成一个私有辅助方法被两者共用。这里的取舍是「方法数量变多」对「每个方法更容易理解、测试、复用」——6.031 明确选择后者，因为**模块化本身就是把 bug 局部化的手段**（见 Reading 9）。

【设计原则透视】 规格的连贯性直接对应抽象边界：一个方法应当只承诺一件事，这样它的前后置条件才可能被一句话说清。不连贯的规格几乎必然伴随全局状态与副作用，而这两者又会让表示不变量与并发推理变得困难（Reading 11、Reading 21/23）。

场景 7：规格强弱与安全替换——从 `findExactlyOne` 到 `findOneOrMore,FirstIndex`

❌ *错误代码*

```
// 错误：把弱规格当成强规格来“升级”——把前置条件改强、后置条件改弱，
// 结果是既有客户全部失去保护。
/**
 * requires: val 在 a 中恰好出现一次
 * effects:  返回满足 a[i] == val 的下标 i
 */
public static int findExactlyOne(int[] a, int val) { /* ... */ return 0; }

// 有人为了“更精确”，改成了下面这份规格：
/**
 * requires: val 在 a 中出现奇数次，且 a 已按升序排序
 * effects:  返回某个满足 a[i] == val 的下标 i，或者 -1
 */
public static int findOddSorted(int[] a, int val) { /* ... */ return -1; }
```

【错误代码的问题】 1. **前置条件变强**：原来是「恰好出现一次」，现在额外要求「出现奇数次」并且「数组升序」，一批原本合法的调用突然违约。 2. **后置条件变弱**：原来保证返回的下标一定命中，现在允许返回 -1，客户原来那句 `a[find(a, val)] == val` 可能直接抛数组越界。 3. **不可安全替换**：两个方向的改动都缩小

了客户能依赖的保证，等于同时攻击了所有既有调用点（违背 Ready for change 与 Safe from bugs）。 4. **规格之间不可比**：新旧规格既不是包含也不是被包含，属于 incomparable，无法用强弱规则一次性判断安全性，必须逐个调用点审查。

✅ *正确代码*

```
// 正确：沿“放宽前置条件、加强后置条件”的方向逐级强化，每一步都可安全替换。
// S1（最弱）：
//   requires: val 在 a 中恰好出现一次
//   effects:  返回 satisfying a[i] == val 的下标 i
//
// S2 强于 S1：前置条件放宽为“至少出现一次”，后置条件不变
/**
 * requires: val 在 a 中至少出现一次
 * effects:  返回满足 a[i] == val 的下标 i
 */
public static int findAnyIndex(int[] a, int val) { /* ... */ return 0; }

// S3 强于 S2：前置条件不变，后置条件加强为“最低下标”
/**
 * requires: val 在 a 中至少出现一次
 * effects:  返回满足 a[i] == val 的最低下标 i
 */
public static int findFirstIndex(int[] a, int val) {
    for (int i = 0; i < a.length; i++) {
        if (a[i] == val) return i;
    }
    throw new AssertionError("precondition violated: val not in a");
}
```

【为什么这样更好】 强化规格的两个方向都只**增加**客户能依赖的东西：S2 让更多输入变得合法（客户更容易满足前置条件），S3 让输出更确定（客户可以依赖「一定是最低下标」）。因此满足 S3 的实现自动满足 S2 与 S1，把旧规格替换为新规格永远不会让既有客户失效。要注意 S2 这类「至少出现一次」的规格是**欠定**的：它允许实现返回任意命中下标，`findFirst` 与 `findLast` 都合法。

【代码对比解说】 判断强弱的机械流程是：先比较两个前置条件的集合大小（谁更小谁更强），再在**旧前置条件成立的输入范围内**比较后置条件（谁约束更多谁更强）；只有「前置不更强」且「后置不更弱」时，才能宣布 S2 ⩾ S1。注意上面 S3 的实现里用 `throw new AssertionError` 表示前置条件被违反——这不是后置条件的一部分，而是「实现不再受契约约束」时的快速失败（见 Reading 9: Avoiding Debugging）。

【设计原则透视】 这条对比是本讲最强的可操作规则：**替换规格时只能放宽前置条件、加强后置条件**。它把「规格空间图」中的区域包含关系变成了可执行的代码演化策略，也解释了为什么 `findCanBeMissing`（前置条件为 nothing、但允许返回 −1）与 `findOneOrMore,FirstIndex` **互不可比**：前者前置更弱（区域更大），但后置也更弱（在共同输入上少了「最低下标」的保证），两个方向的变动互相抵消。

与其他设计原则的关联

- **Reading 6（Specifications）**：本讲是它的直接延续。Reading 6 建立了前置条件、后置条件与「客户—实现者契约」的基本结构，本讲则在此之上增加了三个比较维度（确定性、声明性、强度）与「如何写出好规格」的准则。没有 Reading 6 的契约语言，本讲的强弱规则无从表述。
- **Reading 3（Testing）与 Reading 4（Code Review）**：规格是测试的唯一依据——测试用例的合法性由「输入是否满足前置条件」决定，期望结果由后置条件决定（本讲场景 4 中「规格太弱无法写测试」正是这一点）。Reading 4 强调「注释要写规格、不要翻译代码」，与本讲「声明式优于操作式」是同一条规则的不同表述。
- **Reading 8（Mutability & Immutability）**：当方法会修改输入对象时，必须在 `effects`（或 `Modifies` 子句）中显式声明，否则「未声明的修改一律视作禁止」。本讲场景 5 中返回 `Collections.unmodifiableList` 的写法，正是把不可变性作为实现自由来使用。
- **Reading 9（Avoiding Debugging）**：本讲「前置条件还是后置条件」的讨论与 Reading 9 的断言（`assert`）直接衔接：把参数要求写成断言，是让违反契约的调用尽早、就近失败的标准做法；而「外部条件用异常、内部假设用断言」的分工也源自同一个「fail fast」原则。
- **Reading 10、11（Abstract Data Types；Abstraction Functions & Rep Invariants）**：本讲场景 5 的「抽象类型」要求在此彻底展开——ADT 的规格其实就是抽象函数（AF）的说明，而 ADT 的 `checkRep` 检查的是表示不变量（RI），二者共同构成「抽象边界」两侧的完整契约。
- **Reading 12（Interfaces, Generics, Enums）**：规格中优先使用接口类型而非实现类的做法，在泛型与接口章节成为语言级惯例（`List`、`Map`、`Function`）。
- **Reading 21、23（Concurrency；Locks）**：欠定规格在并发中会变成「允许哪些交错」的问题，而「放宽前置条件」在并发场景下需要额外小心——详见并发章节对线程安全规格的讨论。

关键要点

- **规格有三个可比维度：确定性、声明性、强度。** 先问「是否只允许一个输出」，再问「是在描述结果还是在描述步骤」，最后问「合法实现集合有多大」。
- **欠定规格 ≠ 模糊规格，也 ≠ 非确定实现。** 「返回任意一个命中下标」是精确的欠定承诺，通常由一个完全确定的实现来兑现；欠定的价值在于给实现者留出选择空间。
- **永远用声明式写法。** 后置条件只谈输出与输入的关系（存在量词、下标约束、等值关系），不谈控制流；实现解释写在方法体内部而不是规格注释里。
- **规格替换只允许单向操作：放宽前置条件、加强后置条件。** 判定公式是「S2 前置 ⩽ S1 前置 且 S2 后置 ⩾ S1 后置（在 S1 前置成立的输入上）」；方向相反或互不可比时，必须逐个调用点审查。
- **前置条件是「把检查成本转嫁给客户」的工具，只在检查代价高或方法作用域小时才使用。** 对外公开的方法宁可写「参数非法时抛出未检查异常」这样的后置条件，以便快速失败。
- **好规格的六条准则：连贯、结果有信息量、足够强、足够弱、使用抽象类型、形式简洁清晰。** 遇到长参数列表、布尔开关、一堆特例、与实现耦合的类型，都要重构规格本身。

常见陷阱与注意事项

1. **把实现步骤写进规格注释（操作式规格）** → 客户依赖实现细节；实现者一旦改算法就被指责「违反契约」，Readiness for change 直接归零。正确做法是把步骤描述移到方法体内部注释。
2. **用 `null`、`-1`、`arr.length` 之类的哨兵值兼作两种含义** → 客户无法区分「没有结果」与「结果就是特殊值」，返回值失去信息量；一旦客户据此写分支逻辑，后续任何语义调整都会引发连锁 bug。
3. **误以为「把前置条件改强 = 规格更严谨、更好」** → 实际上前置条件越强，客户越难调用、合法实现越多（区域越大、规格越弱），既有客户可能全部违约。记住口诀：**前置只能放宽，后置只能加强**。
4. **在规格中泄漏具体实现类型（`ArrayList`、`HashMap`、`FileReader`）** → 客户被迫使用特定结构，实现者也被锁死在某个返回类型上；签名一变，所有调用点都要改。
5. **认为「欠定 = 允许实现随便乱来」** → 欠定只放开了「多个合法输出之间的选择」，绝不放开后置条件本身；如果规格允许返回任意值，那不是欠定，而是没有规格。
6. **把「检查前置条件」的代码当成后置条件写进规格** → 例如声称「要求数组有序」却同时承诺「乱序时抛出特定异常」，这在语义上已经变成后置条件，实现者必须真的去检查；分不清这一点会导致实现与文档不一致。
7. **规格不连贯（一个方法既查找又打印又改全局变量）** → 无法一句话说清契约，客户为了用一半功能必须接受全部副作用；正确做法是按职责拆分成多个连贯方法。

思考题（带答案）

问题 1：设有两份 `find` 规格：（a）`requires: val 在 a 中恰好出现一次`；`effects: 返回满足 a[i] == val 的下标 i`；（b）`requires: nothing`；`effects: 返回满足 a[i] == val 的下标 i，若不存在则返回 -1`。请判断二者是否可比、哪一份更强，并说明 `findFirst`（返回最小下标、找不到返回 `a.length`）与 `findLast`（返回最大下标、找不到返回 −1）分别满足哪一份。

答案：二者**不可比（incomparable）**。相对（a），（b）的前置条件更弱（从「恰好一次」放宽到「无要求」），这一方向的改动使（b）更强；但在（a）前置条件成立的输入上，（b）的后置条件也更弱（允许返回 −1，而（a）保证一定命中），这一方向的改动使（b）更弱。两个方向相互抵消，因此不存在包含关系，也不存在完全分离，属于部分重叠。至于实现：`findFirst` 与 `findLast` 在满足（a）前置条件的输入上都返回正确下标，因此都满足（a）；它们对「找不到」的输入分别返回 `a.length` 与 −1，两者都不是 −1（`findFirst` 返回 `a.length`），所以 `findFirst` **不**满足（b），而 `findLast` 满足（b）。这也说明：同一个实现可以满足互不可比的多份规格，但一份规格的客户不能假定另一份规格的保证。

问题 2：某实现者想给 `sort(List<String> list)` 加一个更「聪明」的规格：`effects: 把 list 排成升序并返回 list 中的元素个数；若 list 中含有 null 元素，则抛出 NullPointerException`。请指出这份规格至少两个设计问题，并给出改进方向。

答案：第一，**不连贯**——方法既做排序又返回元素个数，这两件事没有关系，客户想数元素却被迫触发排序（或被迫接受排序带来的性能开销）。应拆成两个方法：`sort`（返回 void，修改 list）与 `size`（只读）。第二，**特例削弱了可用性**：规格没有说明抛出异常时 list 处于什么状态（有没有被排到一半），客户捕获异常后无法判断后续能否继续使用该列表；应像 `addAll` 那样强化后置条件，明确「若抛出异常，list 保持不变」并据此实现（先检查 null、再排序），或者干脆把 null 要求写成前置条件、让客户自己保证。第三（附加）：

返回元素个数在 Java 中与 `list.size()` 重复，属于无信息量的返回值。此外如果保留修改语义，规格里必须显式写出「修改 list」这一点，否则默认不允许修改输入。

问题 3：为什么不建议在没有特殊理由时为 `find` 写出「返回最低下标」这样的确定性后置条件？请结合「实现者自由度」与「客户可理解性」两方面权衡回答。

答案：确定性规格承诺更强，对客户当然更友好——客户可以依赖「返回的就是最低下标」做进一步的推理（例如推断前缀里没有 val），这是它作为**后置条件加强**的合法方向，任何既有客户都不会因此受损。但代价是**实现者自由度下降**：一旦承诺最低下标，任何实现都必须保证这一性质，例如想做「从尾部扫描 + 缓存」或并行分段查找的实现就会被迫额外处理下标比较，规格区域随之缩小（`findOneOrMore,FirstIndex` 区域小于 `findOneOrMore,AnyIndex`）。因此关键问题是：**客户是否真的需要确定性那一部分信息**。若客户只关心「拿到一个命中位置」，欠定规格即可，实现者可在区域内自由选择算法（这在 Reading 8、10 中体现为可以自由改用不同数据结构）；若客户确实依赖最低下标（例如要求「稳定」的行为或用于断言前缀属性），就应明确写成确定性后置条件并接受实现受限。这正是「规格应当足够强、也应当足够弱」这条准则的具体含义：**规格强度不是越高越好，而是恰好覆盖客户的真实需求**。

Reading 8: 可变性与不可变性（Mutability & Immutability）

> 说明：本讲 sp22 原版使用 TypeScript，本笔记按用户要求提供 Java 代码示例；类型/API 与 sp21（6.031 Java 版）原文保持一致。

概述

本讲要回答的问题是：**既然可变对象看起来「能力更强」，为什么我们仍然应当尽量使用不可变对象和不可重赋值的引用？** 6.031 的答案是：不可变类型 **更不容易出 bug（safe from bugs）、更容易理解（easy to understand）、更适应变化（ready for change）**；而可变性的真正危险来源不是「对象能改」，而是**别名（aliasing）**——同一个可变对象被程序的多处引用，任何一处都能在不通知其他人的情况下改变它，契约因此从「调用点附近的局部约定」退化为「贯穿整个程序生命周期的全局约定」。

围绕这一命题，本讲依次讨论：可变对象在「传入参数」与「返回结果」两条路径上引发的经典 bug（`List` 与 `Date`）、防御性拷贝（defensive copy）及其代价、修改型方法必须在规格中声明 `Modifies`、迭代器与「迭代中修改」导致的下标错位、不可变类的设计规则、`Collections.unmodifiableList` 这类**不可修改视图的局限**，以及不可变性在共享与性能上的真实优势。它上承 Reading 7（Designing Specifications）的契约语言，下启 Reading 10、11 的抽象数据类型与表示不变量、Reading 15 的相等性与 `hashCode`，并为 Reading 21、23 的并发安全打基础。

核心概念与设计原则详解

可变对象与不可变对象（Mutable vs. Immutable Objects） – 定义与目的：不可变对象（immutable object）一旦创建就永远表示同一个值，不存在能改变其值的方法；可变对象（mutable object）提供会改变自身值的方法。这一区分是 Safe from bugs 的第一道防线。 – **直观解释（"它是什么？"）**：Java 的 `String` 是不可变的——想「在末尾追加字符」必须创建一个新 `String`（`s = s.concat("b")`）；`StringBuilder` 是可变的——`sb.append("b")` 直接改变对象本身。两者最终都能表示 `"ab"`，差别只在

当存在多个引用时才显现：`String t = s; t = t + "c";` 不会影响 `s`，而 `StringBuilder tb = sb; tb.append("c");` 会连带改变 `sb` 所指向的内容。 – **关键规则与最佳实践**：默认选择不可变类型；只有「局部、单一引用」的场景才放心使用可变对象；把「对象不可变」与「引用不可重赋」严格区分（见下一条）；在快照图（snapshot diagram）中用双线边框表示不可变对象、双线箭头表示不可重赋值的引用，用来在脑内验证代码行为。

不可重赋值的引用与不可变对象（Unreassignable References vs. Immutable Objects） – **定义与目的**：`final`（Java）/ `const`、`readonly`（TypeScript）只保证**引用本身不能被重新赋值**，完全不保证被指向的对象不可变。区分这两件事是避免「我明明写了 final，为什么它还是变了」这类误解的关键。 – **直观解释（"它是什么？"）**：`final List<String> roster = new ArrayList<>();` 中的 `roster` 永远指向同一个列表对象，但这个列表里的元素可以被自由 `add`、`remove`（因此 `MyIterator` 的 `list` 字段声明为 `final`，表示迭代器一生都盯着同一个集合）。这与 `String` 那种「对象内部都改不了」的不可变性是两个层次。 – **关键规则与最佳实践**：把 `final` 用在所有能用的地方——方法参数、局部变量、字段，它既是给读者的文档（「这里不会重新指向别的对象」），也是编译器静态检查的对象；但它不能替代不可变性设计；`final` 字段是构造不可变类的**必要条件而非充分条件**（若字段指向可变对象，仍可能通过该对象泄漏可变性）。

别名（Aliasing）：可变性风险的根源 – **定义与目的**：别名指多个引用指向同一个对象。只有在存在别名时，可变性才会伤害他人；因此理解别名是理解本讲所有 bug 案例的统一钥匙（Safe from bugs）。 – **直观解释（"它是什么？"）**：如果某个可变对象从头到尾只有一个引用，且完全活在某个方法内部，那么随意修改它是安全的。问题出在两个程序员（或同一程序员的两个模块）各自持有一个别名：一个认为「我可以改」，另一个认为「它不会变」，后者就输了。 – **关键规则与最佳实践**：写代码时先问「这个对象现在有几个引用？谁能看见它？」；跨模块传递可变对象前，先决定是「明确共享」还是「隔离」；用快照图把别名画出来（先在纸上画，最终目标是能在脑中画）；一旦发现某个可变对象被两个以上模块持有，就要评估是否应该改成不可变或在边界上拷贝。

风险一：传入可变值（Risky Example #1: Passing Mutable Values） – **定义与目的**：把可变对象作为参数传入方法，等于把「它可能被改」的权利交给被调方；即使被调方动机良好（复用、性能），也可能改变调用方的数据（Safe from bugs、Easy to understand）。 – **直观解释（"它是什么？"）**：`sumAbsolute(list)` 为了复用 `sum(list)`，先把列表里每个元素改成绝对值再求和——从实现者角度这是 DRY 加性能双赢，但调用方 `main` 手里的 `myData` 被悄悄改成了正数，之后 `sum(myData)` 得到 10 而不是 –10。 – **关键规则与最佳实践**：如果方法不打算修改参数，就绝不修改它（并且在规格里不写 `Modifies`）；确实需要修改时，必须在规格中明确声明，让客户自己决定是否接受；怀疑有副作用时，优先构造新对象而不是原地修改；把「参数被修改」当作**潜伏 bug**：它不会立刻报错，而是在某次看似无关的改动后爆发。

风险二：返回可变值（Risky Example #2: Returning Mutable Values） – **定义与目的**：返回可变对象同样会制造别名：调用方持有返回对象，实现者可能把它存进缓存，两边互相踩踏（Safe from bugs、Ready for

change）。 – **直观解释（"它是什么？"）**：`startOfSpring()` 先返回 `askGroundhog()` 的结果；后来实现者为了少打扰土拨鼠加了缓存（`private static Date groundhogAnswer`）；同时某个客户为了把派对推迟一个月写了 `partyDate.setMonth(partyDate.getMonth() + 1)`——两处独立改动一交互，缓存里的「春天第一天」被永久改成一个月后，而**最先发现 bug 的往往是与这两处都无关的第三个调用者**（Ready for change 的反例：两个看起来独立的改动叠加出严重 bug）。 – **关键规则与最佳实践**：优先返回不可变类型（用 `java.time.LocalDate`、`Instant` 而不是 `Date`）；若必须返回可变对象，就在返回处做防御性拷贝；反过来，接收可变参数并保存为字段时也要拷贝（copy-in）；记住「不可变性让 bug 在设计上不可能发生」——这比事后打补丁更可靠。

防御性拷贝（Defensive Copying） – **定义与目的**：在「可变对象跨越模块边界」的两个方向上各拷贝一份（copy-in：构造器/设值方法入口拷贝；copy-out：返回前拷贝），从而切断别名。它是可变对象与不可变契约之间的折中方案。 – **直观解释（"它是什么？"）**：`return new Date(groundhogAnswer.getTime());` 让每个调用方拿到自己的副本，`partyPlanning` 想怎么改都不影响缓存。代价是**每个客户都要付一次拷贝的时间与空间**，哪怕 99% 的客户从不变更返回值。 – **关键规则与最佳实践**：拷贝必须在**所有**跨越抽象边界的方向上做，漏一处就前功尽弃；拷贝是浅拷贝还是深拷贝取决于内部结构（若元素本身可变，浅拷贝不够）；把「用拷贝换安全」与「用不可变类型换安全」作对比——不可变类型永远不需要防御性拷贝，因此常常**更快也更省内存**；防御性拷贝是可用的工程手段，但优先考虑不可变设计。

修改型方法的规格（Specifications for Mutating Methods） – **定义与目的**：如果方法会修改对象（参数或 `this`），必须在规格中显式声明（`Modifies:` 子句或 `effects` 中的描述）。这是 Reading 7 契约结构在可变性下的必然要求（Safe from bugs、Easy to understand）。 – **直观解释（"它是什么？"）**：`static void sort(List<String> list)` 的 `effects` 写「把 list 排成升序」，客户一看就知道自己传进去的列表会被改；而 `static List<String> toLowerCase(List<String> list)` 的 `effects` 写「返回一个新列表 t，其中 t[i] = list[i].toLowerCase()」，客户就知道原列表安然无恙。 – **关键规则与最佳实践**：**effects 里没有声明修改，就等于禁止修改**——「惊喜式修改（surprise mutation）」是 bug 的温床；对 `this` 的修改同样要写（例如 `next()` 的 `Modifies: this iterator`）；写测试时按规格断言「未被声明修改的对象不应改变」，例如对 `toLowerCase` 断言入参列表保持不变。

迭代器与「迭代中修改」（Iterators and Mutation During Iteration） – **定义与目的**：迭代器（iterator）是「依次取元素」的可变对象，`next()` 既返回元素又推进自身位置——它是**修改型方法**的典型例子。理解它与集合之间的别名关系，可以解释一大类难查的 bug（Safe from bugs）。 – **直观解释（"它是什么？"）**：Java 的 `for (String s : subjects)` 会被编译器改写成 `Iterator` 循环；迭代器内部记着一个下标。如果循环体里删除了集合中的元素，后面的元素会整体前移，而迭代器的下标不调整，于是**跳过某些元素**。例如 `dropCourse6(["6.045", "6.031", "6.036"])` 期望得到空列表，实际却留下 `["6.031"]`——第一轮删掉下标 0 后，原本在下标 1 的 `"6.031"` 移到下标 0，而迭代器已经推进到下标 1，永远不会再看它。 – **关键规则与最佳实践**：**迭代过程中禁止修改被迭代的集合**；需要边遍历边过滤时，改用「收集到新列表再替换/清空」的方式，或用 `Iterator.remove()`（迭代器会自行调整位置）、`removeIf`、`stream().filter(...)`；注意 `ArrayList` 的迭代器会主动检测这种修改并抛出

`ConcurrentModificationException`（快速失败，见 Reading 9），而自写的 `MyIterator` 不会，只会默默给出错误结果；即使 `Iterator.remove()` 也只解决了「只有一个迭代器」的情形——多迭代器并发、或做了排序之类的复杂修改时依然不安全。

不可变性的三大好处（The Three Benefits of Immutability） – **定义与目的**：不可变性是本讲的核心设计原则，它同时改进课程的三大质量目标。 – **直观解释（"它是什么？"）**：**Safe from bugs**——不可变对象不受别名 bug 影响，不可重赋值的变量永远指向同一个对象；**Easy to understand**——读者不必追踪「这个对象在整个程序里可能被谁改过」，因为答案恒为「没人能改」；**Ready for change**——既然运行期不可能变，依赖它的一切代码在程序演化时都不必跟着改。 – **关键规则与最佳实践**：把「不可变对象 + 不可重赋值引用」当作默认选择，可变性当作需要理由的例外；在代码评审中把「未经声明就修改参数」当作必须修复的问题；用不可变性把「全局推理」变成「局部推理」——这是它最深远的价值。

不可变类的设计规则（Design Rules for Immutable Classes） – **定义与目的**：如何真正写出一个不可变类？需要同时堵住三条泄漏通道，否则类会「看起来不可变、实际上可被改」（Safe from bugs）。 – **直观解释（"它是什么？"）**：三条规则分别是：（1）**所有字段用 `final` 并在构造器中初始化**——引用不再重赋；（2）**不提供任何修改字段的方法**，也不要把可变字段暴露出去（不要返回内部可变对象，也不要在构造器中保存外部传入的可变对象，即 copy-in/copy-out）；（3）**内部若持有可变对象，要么让它是私有的且从不外泄，要么它本身就是不可变的**。注意「所有字段都是 final」并不意味着类是安全的：`final List<String> animals` 指向的列表仍可被 `add`，所以还要配合（2）。 – **关键规则与最佳实践**：用 `private final` 声明字段；构造器对可变参数做防御性拷贝；观察器（getter）返回不可变视图或副本，绝不直接返回内部可变对象；类本身可以声明为 `final` 以防子类破坏不可变性；写完类后自查一句：**类外是否存在任何能改变本类实例可观察状态的代码路径？** 如果存在，它就不是不可变类。

`Collections.unmodifiableList` 的局限（The Limitation of Unmodifiable Views） – **定义与目的**：Java 的 `Collections.unmodifiableList/Set/Map` 返回的是**不可修改视图（unmodifiable view）**——一个包在底层集合外面的包装器，任何通过包装器的修改都会抛出 `UnsupportedOperationException`。但它**不是**不可变集合：底层集合若仍有别名存在，依旧可以被改，视图会跟着变。 – **直观解释（"它是什么？"）**：`List<String> view = Collections.unmodifiableList(roster);` 之后，`roster.add("bob")` 合法，并且 `view` 立刻显示 `["alice", "bob"]`——包装器挡住的只是「通过它自己」的修改。要真正隔离，必须**同时抛弃对底层集合的引用**（让持有 `roster` 的局部变量离开作用域），或者直接使用 `List.copyOf(...)` 得到一份不可修改的（浅）拷贝，或者用 `List.of(...)` 构造不可变集合。TypeScript 中对应的 `ReadonlyArray` 也是同样性质：它只是一个界面（interface），没有构造器，必须先有 `Array` 再赋给它，因此无法保证不可变。 – **关键规则与最佳实践**：把 `unmodifiableList` 当「防止客户误改的护栏」，而不是「不可变保证」；真正需要不可变性时用 `List.of` / `List.copyOf`（`copyOf` 产生不可修改的浅拷贝）；`Collections.emptyList()` 等不可变空集合可以放心使用（否则会出现「你确定很空的列表突然不空了」的诡异 bug）；返回视图的同时保留底层引用，是典型的半吊子封装。

不可变性与性能（Immutability and Performance） – **定义与目的**：很多人选择可变类型是为了性能，但 6.031 明确指出：**可变类型并不总是比不可变类型高效**。关键在于「能共享多少」与「必须拷贝多少」。 – **直观解释（"它是什么？"）**：不可变值可以被程序各处安全共享，因此在需要跨越边界传递时，往往只需传引用而无需防御性拷贝；相反，可变值在同样的场景下被迫反复拷贝，既费时间又费空间。反过来，若一个值需要被「大量小幅修改」，可变对象更省——例如用 `StringBuilder` 循环 `append` 替代 `String` 的 `+` 拼接，可把 $O(n^2)$ 的拷贝降为线性（Java 的 `+` 在循环里每次都生成新字符串，第一个字符会被反复拷贝 n 次）。此外，不可变值可以借助**内部共享**（structural sharing）降低拷贝成本：编辑一个百万字符字符串中间的一个字符，聪明的实现可以复用编辑点前后的未改动区域。Git 的对象图也是同一原理——因为提交（commit）不可变，后续提交可以放心指向父提交而不担心它被改写。 – **关键规则与最佳实践**：先问「这个值会被共享吗？会被跨界传递吗？」——会，就用不可变类型；再问「它会被频繁小幅修改吗？」——会，且在单一引用范围内，可考虑可变类型（如 `StringBuilder`）；永远不要在循环里用不可变类型做累积拼接；记住「不可变类型从不需要防御性拷贝」这一条本质上就是性能优势。

常用不可变类型清单（Useful Immutable Types in Java） – **定义与目的**：把「尽量不可变」落实为可执行的选型习惯（Safe from bugs、Ready for change）。 – **直观解释（"它是什么？"）**：Java 中：基本类型及其包装类（`Integer`、`Double`…）全部不可变；大数运算用不可变的 `BigInteger`、`BigDecimal`；**不要使用可变的 `java.util.Date`**，改用 `java.time` 包中按精度需求选择的 `LocalDate`、`LocalDateTime`、`Instant` 等（它们的规格保证不可变）；用 `List.of`、`Set.of`、`Map.of` 从已知值构造不可变集合；`Collections.unmodifiableList/Set/Map` 提供不可修改视图，`List.copyOf` 等提供不可修改的浅拷贝；`Collections.emptyList()` 等提供不可变空集合。 – **关键规则与最佳实践**：需要日期时间时默认选 `java.time`；需要「传出去不会被改」的集合时优先 `List.copyOf` 或 `List.of`；看到 `Date`、`Calendar`、`StringBuilder`、`ArrayList` 出现在方法签名里，就要停下来问「这里真的需要可变性吗？」；把「不可变优先」写进团队约定，而不是靠个人记忆。

代码示例与对比分析

场景 1：为了 DRY 与性能而修改传入的列表

❌ *错误代码*

```
// 错误：sumAbsolute 为了实现复用而原地修改参数列表，
// 调用方手里的 myData 被悄悄改成绝对值。
public static int sum(List<Integer> list) {
    int sum = 0;
    for (int x : list) {
        sum += x;
    }
    return sum;
}

public static int sumAbsolute(List<Integer> list) {
    // 复用 sum()，先就地取绝对值——看起来既 DRY 又高效
    for (int i = 0; i < list.size(); ++i) {
        list.set(i, Math.abs(list.get(i)));
    }
    return sum(list);
}

public static void main(String[] args) {
    List<Integer> myData = new ArrayList<>(Arrays.asList(-5, -3, -2));
    System.out.println(sumAbsolute(myData));   // 10
    System.out.println(sum(myData));           // 期望 -10，实际 10
}
```

【错误代码的问题】 1. **潜伏 bug**：程序不会报错，只会给出错误结果；真正的破坏发生在「另一个程序员本以为数据没变」的地方。 2. **规格违约**：`sumAbsolute` 的文档只承诺「返回绝对值之和」，并未声明会修改 `list`，因此修改属于未声明副作用（违反 Reading 7 的契约结构）。 3. **极难理解**：读 `main` 的人无法从调用形式看出 `myData` 会被改动，必须跳进实现里逐个检查，违背 Easy to understand。 4. **放大风险**：一旦有多个别名（例如 `myData` 也被别的模块持有），bug 会扩散到与 `sumAbsolute` 完全无关的代码。

✅ *正确代码*

```
// 正确：不修改参数；既保留 DRY 精神（借用 sum 的求和思路），又不产生任何副作用。
/**
 * @param list 待求和的列表
 * @return list 中各元素绝对值之和
 */
public static int sumAbsolute(List<Integer> list) {
    int sum = 0;
    for (int x : list) {       // 只读遍历，不改列表
        sum += Math.abs(x);
    }
    return sum;
}

public static void main(String[] args) {
    List<Integer> myData = new ArrayList<>(Arrays.asList(-5, -3, -2));
    System.out.println(sumAbsolute(myData));   // 10
    System.out.println(sum(myData));           // -10，myData 未被修改
}
```

【为什么这样更好】 方法对参数是**只读**的，客户可以放心地把同一个列表传给多个方法而无需担心相互干扰；行为在调用点即可推断（`sum(myData)` 之后 `myData` 不变），无需阅读被调方实现；两条独立的修改（一个是 `sumAbsolute` 的实现，一个是 `main` 里增加一次 `sum` 调用）不再可能互相破坏。

【代码对比解说】 错误版本其实蕴含一个合理的直觉：「如果列表有百万个元素，就地修改能省下一次百万级分配」。这个性能考量是真实的，但代价是把契约从局部推理变成全局推理。若性能确实关键，正确的折中是把修改**限制在局部**：在方法内部新建一个列表（`new ArrayList<>(list)`）再改，绝不改调用方的对象——即「把可变性关在方法里」。这也正是本讲的判断标准：**可变对象局部使用是安全的，跨边界共享才是危险的**。

【设计原则透视】 这是「别名 + 未声明修改」的典型组合。它把 Reading 7 的 `Modifies` 子句从纸面规则变成可观察的 bug：契约没有声明修改，客户就有权假定数据不变。同时它示范了抽象边界（abstraction boundary）的作用——方法边界是切断别名的天然位置，防御性拷贝通常就放在这里。

场景 2：返回可变的 `Date`，与内部缓存形成别名

❌ *错误代码*

```
// 错误：对外返回内部缓存的 Date 对象，客户一改，缓存就跟着坏。
private static Date groundhogAnswer = null;

/**
 * @return 今年春天的第一天
 */
public static Date startOfSpring() {
    if (groundhogAnswer == null) {
        groundhogAnswer = askGroundhog();   // 只问一次土拨鼠，之后走缓存
    }
    return groundhogAnswer;                 // 返回的是同一个 Date 对象
}

public static void partyPlanning() {
    Date partyDate = startOfSpring();
    partyDate.setMonth(partyDate.getMonth() + 1);   // 派对推迟一个月
    // …… 于是 startOfSpring() 的缓存也被推后了一个月
}
```

【错误代码的问题】 1. **缓存被污染**：`partyDate` 与 `groundhogAnswer` 是同一个对象的两个别名，客户端的合法调用修改了实现者的内部状态。 2. **bug 归属混乱**：错误发生在 `partyPlanning`，但受害者是后续任何调用 `startOfSpring()` 的无关代码——「谁的错」在工程上难以仲裁。 3. **契约变成终身契约**：若想用规格补救，就只能写上「调用方永远不得修改返回的数组/对象」，这让契约的效力延续到程序余生的每一刻，客户再也无法只靠「调用前看前置、调用后看后置」推理。 4. **`Date` 本身设计糟糕**：`Date.setMonth` 的月份取值是 0–11，而且文档允许越界参数（如 1 月 32 日被解释为 2 月 1 日），本该是前置条件的地方变成了「自动纠正」，违反了 fail fast（见 Reading 9）。

✅ *正确代码*

```
// 正确：改用 java.time 的不可变类型，缓存与客户永远不会互相干扰。
// 补充说明：java.time 的引入属于 Java 8 生态建议，6.031 sp21 原文也明确
// 建议“永远不要使用 Date，改用 java.time 中保证不可变的类型”。
private static LocalDate groundhogAnswer = null;

/**
 * @return 今年春天的第一天
 */
public static LocalDate startOfSpring() {
    if (groundhogAnswer == null) {
        groundhogAnswer = askGroundhog();   // 返回 LocalDate，不可变
    }
    return groundhogAnswer;                 // 共享同一个不可变对象，安全
}

public static void partyPlanning() {
    LocalDate partyDate = startOfSpring().plusMonths(1);   // 产生新对象，不改原值
    // ……
}
```

【为什么这样更好】 `LocalDate` 没有修改自身的方法：`plusMonths(1)` 返回**新对象**，缓存里的日期毫发无损。于是实现者获得了引入缓存的自由（性能改进），客户也获得了随意计算的自由，两者互不干扰——不需要任何一方仔细阅读规格注释，也不存在「终身契约」。

【代码对比解说】 如果因为历史原因必须沿用 `Date`，最低限度也要做**防御性拷贝**：`return new Date(groundhogAnswer.getTime());`。这样 `partyPlanning` 改的是自己的副本。但拷贝让**每个客户**都付出一次分配与复制的成本，哪怕 99% 的客户从不变更返回值；内存里还会散落大量「春天第一天」的副本。不可变类型则允许所有调用方安全共享同一个对象，拷贝次数为零——这就是「不可变性可以比可变性更高效」的具体含义。

【设计原则透视】 这条对比把「别名」从同一模块内提升到跨模块、跨开发者的尺度，并展示了 Reading 7 中「契约应当是局部可推理的」这一要点：一旦需要「终身契约」（调用方永远不得修改返回值），说明设计已经出了根本问题，正确做法是换用不可变类型而不只是加一条注释。它同时说明了后缀式 API（`plusMonths`）如何把可变性从设计里彻底移除。

场景 3：用 `char[]` 返回 9 位 MIT 学号

❌ *错误代码*

```
// 错误：返回可变数组，客户端“打码”与实现者的缓存互相踩踏。
private static Map<String, char[]> cache = new HashMap<>();

/**
 * @param username 要查询的用户名
 * @return 含 9 位 MIT 学号的字符数组
 * @throws NoSuchUserException 数据库中没有该用户时
 */
public static char[] getMitId(String username) throws NoSuchUserException {
    if (cache.containsKey(username)) {
        return cache.get(username);     // 缓存里的数组被直接交出去
    }
    char[] id = lookupInDatabase(username);
    cache.put(username, id);           // 同时把同一个数组存进缓存
    return id;
}

// 客户为了隐私，把前 5 位打码：
char[] id = getMitId("bitdiddle");
for (int i = 0; i < 5; ++i) {
    id[i] = '*';                       // 缓存里的学号变成了 "*****2033"
}
```

【错误代码的问题】 1. **缓存被破坏**：客户的打码操作写穿了缓存，此后所有调用者拿到的都是残缺学号。 2. **责任无法界定**：客户有义务不改返回值吗？实现者有权保留返回对象吗？双方各自都「合理」，冲突只能靠契约澄清——这就是共享可变对象带来的契约复杂化。 3. **两种补救规格都不理想**：「调用方永远不得修改返回数组」是终身契约；「返回一个新数组」虽然保证新鲜，但无法阻止实现者继续持有别名并在将来改写它。 4. **性能反而受损**：为了安全，实现者可能被迫在每次返回时拷贝数组，缓存省下的开销被抵消。

✅ *正确代码*

```
// 正确：返回不可变的 String，客户与实现者都能自由行动。
private static Map<String, String> cache = new HashMap<>();

/**
 * @param username 要查询的用户名
 * @return 含 9 位 MIT 学号的字符串
 * @throws NoSuchUserException 数据库中没有该用户时
 */
public static String getMitId(String username) throws NoSuchUserException {
    if (cache.containsKey(username)) {
        return cache.get(username);     // String 不可变，直接共享是安全的
    }
    String id = lookupInDatabase(username);
    cache.put(username, id);
    return id;
}

// 客户想打码，只能构造新字符串：
String id = getMitId("bitdiddle");
String obscured = "*****" + id.substring(5);   // 缓存中的学号不受影响
```

【为什么这样更好】 返回值类型本身提供了保证——`String` 不可变，所以「客户与实现者永远不会互相踩踏」这件事**不依赖于任何人仔细阅读规格注释**；同时实现者保留了引入缓存的自由（性能改进），客户也保留了任意加工返回值的自由。注意「返回新数组」那种规格做不到这一点：它只能保证数组是新鲜的，不能阻止实现者继续持有别名。

【代码对比解说】 三种规格写法可以排成一条优劣链：(a)「返回数组，调用方永不修改」——终身契约，最差；(b)「返回一个新数组」——保证新鲜但仍有别名风险，中等；(c)「返回 `String`」——从类型层面消灭问题，最好。这条链清楚说明：**用正确的类型表达不可变性，优先于用文档约束人的行为**。

【设计原则透视】 这是抽象边界上「返回值即契约」的经典案例：类型选择直接决定了契约的可表达性。它也预示了 Reading 11 中的表示暴露（rep exposure）问题——这里的缓存就是把内部表示直接交给了外部。同一个模式在并发场景下更危险：共享可变状态需要锁（Reading 23），而不可变对象天然线程安全（Reading 21）。

场景 4：构造器保存外部别名，观察器直接把内部集合交出去

❌ *错误代码*

```
// 错误：构造器保存外部传入的列表引用，getter 又直接把内部列表返回，
// 类的“动物园”完全不受自己控制。
public class Zoo {
    private List<String> animals;

    public Zoo(List<String> animals) {
        this.animals = animals;          // 保存别名（copy-in 缺失）
    }

    public List<String> getAnimals() {
        return this.animals;             // 泄漏内部表示（copy-out 缺失）
    }
}

// 客户端：
List<String> a = new ArrayList<>();
a.addAll(List.of("lion", "tiger", "bear"));
Zoo zoo = new Zoo(a);
a.add("zebra");                          // 动物园凭空多了一只斑马
System.out.println(zoo.getAnimals());    // [lion, tiger, bear, zebra]

List<String> b = zoo.getAnimals();
b.add("flamingo");                       // 客户绕过 Zoo 的规则直接改内部列表
System.out.println(a);                   // [lion, tiger, bear, zebra, flamingo]
```

【错误代码的问题】 1. **构造器未做防御性拷贝**：外部列表的任何后续修改都会「从后门」改变 `Zoo` 的状态，类无法维护自己的不变量。 2. **观察器泄漏表示（rep exposure）**：客户拿到内部列表后可以任意增删，类的全部规则形同虚设（若 `Zoo` 有「容量上限」之类的不变量，立刻会被绕过）。 3. **代码无法推理**：`zoo.getAnimals()` 返回的列表在何时可能变化，取决于程序里所有持有别名的位置，局部推理彻底失效。 4. **不可变性无法实现**：即使把所有字段都声明为 `final`，这个类依然不是不可变的（`final` 只锁住引用）。

✅ *正确代码*

```
// 正确：copy-in + copy-out，并用不可修改视图封住出口。
// 这同时演示了不可变类的三条设计规则：final 字段、不暴露可变内部对象、
// 不保存外部传入的可变对象。
public final class Zoo {
    private final List<String> animals;      // 规则 1：字段 final

    public Zoo(List<String> animals) {
        this.animals = new ArrayList<>(animals);   // 规则 3：copy-in，切断外部别名
    }

    /** @return 动物园中动物名的不可修改视图 */
    public List<String> getAnimals() {
        return Collections.unmodifiableList(animals);   // 规则 2：不交出内部可变对象
    }

    /** 增加一只动物。要求 animals.size() < capacity。 */
    public void add(String animal) {
        animals.add(animal);
    }
}
```

【为什么这样更好】 类的状态只由自己的方法改变，不变量随时可维护；外部列表后续怎么改都与 `Zoo` 无关；客户拿到的是不可修改视图，误写会立刻抛 `UnsupportedOperationException` 而不是悄悄破坏对象。配合 `final` 字段，这类「内部表示不泄漏」的类才可能进一步做成真正不可变的类。

【代码对比解说】 若确实希望类的实例不可变（没有 `add`），可把 `getAnimals` 改成 `return List.copyOf(animals);`（补充说明：`List.copyOf` 是 Java 10 引入的、产生不可修改浅拷贝的方法），或者用 `List.of(...)` 在构造时直接构造不可变列表。要注意 `Collections.unmodifiableList` 返回的是**视图**：只要还有对 `animals` 的别名，视图内容仍会被改变——在本例中 `animals` 是私有且只在内部使用，因此是安全的（详见场景 6）。

【设计原则透视】 这是表示不变量（rep invariant）与表示暴露（rep exposure）在 Reading 11 中正式定义的先声：**只要类的字段可以经由构造器输入或观察器输出被外部改动，类的抽象函数就无法保证成立**。copy-in/copy-out 是维护抽象边界的物理手段，而「不可变类型优先」是从根上免除拷贝的替代方案。

场景 5：迭代过程中修改被迭代的集合

❌ *错误代码*

```
// 错误：边迭代边删除元素，迭代器的下标不再对应正确的元素。
/**
 * 删除所有 Course 6 的课程。
 * Modifies: subjects —— 删除以 "6." 开头的课程号。
 * @param subjects MIT 课程号列表
 */
public static void dropCourse6(List<String> subjects) {
    MyIterator iter = new MyIterator(subjects);
    while (iter.hasNext()) {
        String subject = iter.next();
        if (subject.startsWith("6.")) {
            subjects.remove(subject);   // 通过另一个别名修改了正在被迭代的列表
        }
    }
}

// 测试用例：
// dropCourse6(["6.045", "6.031", "6.036"])
// 期望 []，实际 ["6.031"]
```

【错误代码的问题】 1. **静默跳过元素**：删掉下标 0 后，`"6.031"` 前移到下标 0，而迭代器已推进到下标 1，于是它被永久跳过——结果错误但不报错。 2. **前置条件无人声明**：迭代器的规格里根本没有写「迭代期间不得修改集合」（Java 集合类的文档也很难找到这条），于是责任在 `Iterator`、`List`、客户之间悬空。 3. **契约退化成了全局性质**：正确性不再取决于一次调用的前后置条件，而取决于「程序里是否有人恰好在此期间改了集合」，这类全局性质极难推理与测试。 4. **不可泛化修复**：就算换成 `Iterator.remove()`，也只能处理「只有一个迭代器、只做删除」的情形；排序之类更复杂的修改依然会错位。

✅ *正确代码*

```
// 正确：把“遍历 + 过滤”与新列表构造分开，迭代期间不修改被迭代的集合。
public static void dropCourse6(List<String> subjects) {
    final List<String> kept = new ArrayList<>();
    for (String subject : subjects) {          // 只读遍历
        if (!subject.startsWith("6.")) {
            kept.add(subject);
        }
    }
    subjects.clear();                          // 遍历结束后才修改
    subjects.addAll(kept);
}

// 补充说明：Java 8+ 还提供等价的惯用法（不在 6.031 原文中出现）：
// subjects.removeIf(subject -> subject.startsWith("6."));
```

【为什么这样更好】 「只读遍历 + 遍历后统一替换」把修改推迟到迭代结束，从根本上消除了下标错位；逻辑与结果都一目了然，测试用例 `["6.045","6.031","6.036"] => []` 能稳定通过。若确实需要原地删除，可改用 `Iterator.remove()`：它由迭代器自己调整位置，因此安全且更高效（迭代器已经知道要删哪个元素，不必再搜索一次），但仍只适用于简单删除场景。

【代码对比解说】 三种做法可作对比：自写 `MyIterator` + `list.remove` 会静默出错；Java 集合的 `for-each` + `list.remove` 会抛 `ConcurrentModificationException`（快速失败，见 Reading 9，症状不同但同样是「迭代中修改」这一病根）；「收集新列表」则完全绕开问题。这里也体现了「快速失败」的价值：抛异常比给出错误答案好得多。

【设计原则透视】 这条对比是「别名使契约复杂化」的最佳注解：正确性依赖的不再是某个方法的契约，而是一条**全局性质**（迭代期间集合不得变化），而这条性质在 Java API 文档里几乎无处安放。它也把本讲的结论与 Reading 9（Avoiding Debugging）连起来——把危险操作交给会快速失败的机制，而不是依赖程序员的自觉。

场景 6：把 `Collections.unmodifiableList` 当成不可变集合

❌ *错误代码*

```
// 错误：只包了一层不可修改视图，却保留着底层集合的别名，
// 于是“不可变”的 view 依然会被改动。
public static List<String> roster(String username) {
    List<String> roster = new ArrayList<>();
    roster.add("alice");
    List<String> view = Collections.unmodifiableList(roster);
    // view 看起来已经不可变了…… 但 roster 还活着
    roster.add("bob");                  // 通过别名修改底层集合
    System.out.println(view);           // [alice, bob] —— 视图跟着变了
    return view;
    // 更糟的是：view.add("carol") 会抛 UnsupportedOperationException，
    // 于是调用者以为“这里绝对改不了”，却没意识到内容仍会变。
}
```

【错误代码的问题】 1. **错误的安全感**：`UnsupportedOperationException` 让人以为对象不可变，但底层集合的任何别名都能继续改它。 2. **半吊子封装**：保存了底层引用却不打算共享，属于典型的信息泄漏；一旦将来有人在这个方法里复用 `roster`，行为会难以预测。 3. **规格无法表述**：返回类型是 `List`，无法在类型层面表达「永不改变」，只能靠注释——而注释挡不住代码。 4. **并发下更糟**：视图与底层集合并存会带来可见性与竞态问题（Reading 21、23）。

✅ *正确代码*

```
// 正确：要么用不可修改的拷贝彻底切断联系，要么用 List.of 直接构造不可变集合。
public static List<String> rosterCopy(String username) {
    List<String> roster = new ArrayList<>(List.of("alice", "bob"));
    return List.copyOf(roster);       // 不可修改的浅拷贝，与原列表无任何共享
}

public static List<String> rosterFixed(String username) {
    return List.of("alice", "bob");   // 从一开始就是不可变集合
}
```

【为什么这样更好】 `List.copyOf` 生成的是与原集合**没有共享**的不可修改拷贝，因此无论原列表后来怎么改，返回值都稳定；`List.of` 更进一步，从构造那一刻起就不存在可变别名。客户可以安全共享这个返回值，也不需要任何防御性拷贝。

【代码对比解说】 三种手段的定位应当分清楚：`Collections.unmodifiableList` 是**不可修改视图**——适合在「内部代码希望防止外部误改、但自己仍需要继续修改底层集合」的场景（例如类内部维护可变列表、对外只暴露只读视图，如场景 4）；`List.copyOf` 是**不可修改的浅拷贝**——适合跨模块传递、需要彻底隔离的场景；`List.of` 是**不可变集合**——适合常量数据。前者的关键词是「视图」，后者的关键词是「拷贝」。另外注意 `Collections.emptyList()` 之类的不可变空集合可以放心使用，它避免了「你确信很空的列表突然不空」的怪事。

【设计原则透视】 这里的关键区分是 **unmodifiable ≠ immutable**，与 TypeScript 中 `ReadonlyArray` 的局限完全对应（`ReadonlyArray` 只是接口、没有构造器，必须先有 `Array` 再赋值给它，因此不能保证不可变）。它又一次说明：**不可变性应当由类型与所有权结构保证，而不是由「不要通过这条路径修改」的约定保证**——后者只要存在第二条路径就会失效。

场景 7：在循环里用不可变 `String` 累积拼接（性能取向的可变性使用）

❌ *错误代码*

```
// 错误（性能问题）：循环里用 + 拼接不可变字符串，产生大量临时拷贝。
String s = "";
for (int i = 0; i < n; ++i) {
    s = s + i;          // 每次都新建一个 String，并整体复制已有内容
}
// 第一个数字被拷贝 n 次，第二个 n-1 次…… 总代价 O(n^2)
```

【错误代码的问题】 1. **$O(n^2)$ 时间**：即使只拼接了 n 个元素，复制总量却是平方级，规模稍大就明显变慢。 2. **大量垃圾对象**：每一步都产生一个立即被丢弃的中间 `String`，给垃圾回收带来压力。 3. **掩盖了真实意图**：读者看到的是「拼接」，但代码实际表达的是「反复重建」，意图与实现不匹配。 4. **隐含的性能回归风险**：在热路径上使用这种写法，一旦数据量增长就会成为瓶颈，而它并不会「报错」，因此极难被发现。

✅ *正确代码*

```
// 正确：使用可变的 StringBuilder 做累积，仅在最后生成一次不可变 String。
StringBuilder sb = new StringBuilder();
for (int i = 0; i < n; ++i) {
    sb.append(String.valueOf(i));     // 原地追加，不复制已有内容
}
String s = sb.toString();             // 只在这里产生一个不可变结果
```

【为什么这样更好】 `StringBuilder` 内部用可变结构避免中途拷贝，把总代价降到线性（O(n)）；最终仍然产出不可变的 `String`，因此「可变对象只活在方法内部、只有单一引用」这一安全条件完全满足。这正是 6.031 认可的可变性用法：**用可变类型做性能优化，但把它限制在局部**。

【代码对比解说】 这一组与场景 1 形成互补：场景 1 说明「可变的跨界共享很危险」，这里说明「可变的局部使用很合理」。两者的分界线是 **别名与作用域**：`StringBuilder` 没有被传到任何地方，迭代结束后就只留下不可变的 `String`；而 `sumAbsolute` 修改的是调用方持有的对象。补充说明：Java 的 `+` 在编译期常被优化为 `StringBuilder`（`String` API 文档也有相关实现说明），但在循环中每次迭代仍可能创建新对象，因此累积拼接仍应显式使用 `StringBuilder`。

【设计原则透视】 这条对比把「不可变性与性能」的结论落到实处：**不可变类型并不总是更快，但它从不需要防御性拷贝**；当共享占主导时它更快，当频繁小幅修改占主导时可变类型更快。真实工程判断应同时考虑「共享量」「拷贝量」「作用域」三个因素，而不是笼统地认为「可变一定快」或「不可变一定安全」。

与其他设计原则的关联

- **Reading 6（Specifications）与 Reading 7（Designing Specifications）**：本讲把契约语言用到了「对象会不会变」这件事上。修改型方法必须在规格中显式声明修改（`Modifies` 子句），否则默认禁止修改；而「返回易变对象 + 要求调用方永不修改」这种写法之所以坏，正是因为它把契约变成了覆盖程序余生的**终身契约**，违背了 Reading 7 中「前置条件在调用前检查、后置条件在调用后检查」的局部推理模型。
- **Reading 9（Avoiding Debugging）**：不可变性是第一道防线「让 bug 不可能」的核心手段之一；`final` / `const` 声明不可重赋值的引用，是把假设变成可静态检查的文档；而 `ArrayList` 在迭代中修改时抛出的 `ConcurrentModificationException` 正是「快速失败」的样板。本讲的 `final` 规则与 Reading 9 的作用域最小化规则合在一起，构成「局部化 bug」的完整策略。
- **Reading 10（Abstract Data Types）与 Reading 11（Abstraction Functions, Rep Invariants）**：本讲场景 4 的 copy-in/copy-out 在 Reading 11 中被系统化为「防止表示暴露（rep exposure）」；不可变类的三条设计规则正是维护表示不变量（RI）的前提。ADT 的实现几乎总是「可变 rep + 不可变抽象值」的组合，因此不理解别名与防御性拷贝就无法正确实现 ADT。
- **Reading 12（Interfaces, Generics, Enums）**：匿名类、lambda 与枚举常用于实现不可变值（如 `Comparator`、枚举常量），而接口类型优先的原则（`List` 而非 `ArrayList`）也来自本讲「不要用具体可变实现类型写规格」的结论。
- **Reading 15（Equality）**：不可变性与相等性密切相关——可变对象作为 `HashMap` 的键时，一旦被修改就会「消失」；同时 `hashCode` 只有在对象不参与相等性比较的字段被修改时才稳定，因此 6.031 建议用作键的对象应当是（或至少行为上表现为）不可变的。
- **Reading 21（Concurrency）与 Reading 23（Locks）**：不可变对象天然线程安全，可以自由共享；可变对象则需要锁来保护，而「谁拥有这个对象的别名」会直接决定死锁与竞态的风险。本讲关于「共享可变状态使契约复杂化」的分析，是并发章节的前提。
- **Reading 4（Code Review）**：本讲的 DRY（`sumAbsolute` 为复用而修改参数）与 Reading 4 中 `dayOfYear` 的重复代码，是同一原则的两面：DRY 是好事，但**为了 DRY 而破坏契约**（修改参数、暴露内部状态）是更严重的错误。

关键要点

- **可变性的危险来源是别名，不是「能改」本身。** 单一引用、完全局部的可变对象是安全且高效的；一旦跨越模块边界被多处引用，就需要显式声明、防御性拷贝，或干脆改成不可变类型。
- **始终区分「不可重赋值的引用」与「不可变对象」。** `final` / `const` / `readonly` 只锁住引用（`final List` 里的元素照样能增删），`String`、`java.time` 类型才锁住值。
- **默认不可变，例外需要理由。** 优先使用基本类型包装类、`BigInteger` / `BigDecimal`、`java.time` 类型、`List.of` / `Set.of` / `Map.of`、`List.copyOf`；避免 `java.util.Date`、`Calendar` 以及把 `ArrayList`、`StringBuilder` 放进方法签名。
- **跨界传递可变对象时，copy-in 与 copy-out 一个都不能少。** 构造器要拷贝传入的可变参数，观察器要返回副本或不可修改视图；只做一半等于没做。
- **`unmodifiable` ≠ `immutable`。** `Collections.unmodifiableList` 是不可修改**视图**（底层集合仍可被别名修改），`List.copyOf` 是不可修改**拷贝**，`List.of` 是**不可变集合**——按「是否需要与原集合隔离」选型。
- **迭代过程中禁止修改被迭代的集合。** 需要过滤时用「收集新列表再替换」、`Iterator.remove()` 或 `removeIf`；自写迭代器不会替你报错，只会静默给出错误结果。
- **性能上要比较「共享量」与「拷贝量」。** 共享占主导时不可变更快（无需防御性拷贝），频繁小幅修改占主导时可变更快（`StringBuilder` 代替循环 `+` 拼接）；不可变值还可以通过内部结构共享降低拷贝成本（git 的对象图即为例证）。

常见陷阱与注意事项

1. **把 `final` 当成不可变性的保证** → 例如 `private final List<String> items` 仍可被 `add` / `clear` / 被外部别名修改，类的状态照样会变。正确做法是 `final` 字段 + copy-in/copy-out（+ 必要时 `List.copyOf`）。
2. **只做 copy-in 或只做 copy-out** → 只拷贝入口，外部仍能通过 getter 拿到内部集合直接改；只拷贝出口，外部仍能通过构造时传入的列表改内部状态。抽象边界两侧都必须封住。
3. **用「调用方永远不得修改返回值」这类终身契约补救可变返回值** → 契约效力延伸到程序余生，任何一次疏忽都会破坏实现者缓存，而且责任的归属无法界定。应改用不可变返回类型（如 `String`、`LocalDate`）。
4. **认为 `Collections.unmodifiableList` 返回的就是不可变集合** → 通过它自身修改会抛 `UnsupportedOperationException`，但底层集合的别名仍可自由修改，视图内容随之改变；在需要真正隔离时误用它，会得到「以为不会变却变了」的隐蔽 bug。
5. **边遍历边增删集合元素** → `MyIterator` 这类自写迭代器会静默跳过元素并给出错误答案；Java 集合的 `for-each` 会抛 `ConcurrentModificationException`；即使改用 `Iterator.remove()` 也只是在有多个迭代器、或做了排序等复杂修改时失效。安全策略是遍历期间只读。
6. **为了复用或性能而在方法内修改传入的参数** → 未在规格中声明的副作用会污染调用方的数据，且错误结果往往在无关代码处显现；若要优化，就改动局部副本，或把修改写进规格让客户知情。

7. **在循环中用 `+` 累积拼接不可变字符串** → 形成 O(n^2) 的拷贝与大量临时对象；应改用局部 `StringBuilder`，最后一次性 `toString()`。

思考题（带答案）

问题 1：`Zoo` 类中有 `private final List<String> animals;`，构造器执行 `this.animals = animals;`，并提供一个方法 `public List<String> getAnimals() { return Collections.unmodifiableList(this.animals); }`。请问这个类是否已经安全？为什么？

答案：**不安全。** 三个问题：（1）构造器保存了外部传入列表的别名，因此外部代码此后仍可通过自己手中的列表（如 `a.add("zebra")`）改动 `Zoo` 的内部状态，类的表示不变量无法保证——缺少 copy-in；（2）`final` 只保证 `animals` 这个引用不被重赋，不保证列表内容不变；（3）`Collections.unmodifiableList` 只返回不可修改**视图**，它确实挡住了客户通过返回值的修改（会抛 `UnsupportedOperationException`），但由于第一个问题中仍存在外部别名，视图内容依然可能在客户不知情时变化。正确做法是构造器中 `this.animals = new ArrayList<>(animals);` 做 copy-in，观察器返回 `Collections.unmodifiableList(animals)`（或若类本身要不可变，则返回 `List.copyOf(animals)`）。这样两条泄漏通道同时封住，`Zoo` 才对自己的状态拥有完全控制权。

问题 2：有人在代码评审中提出：「我们干脆规定所有方法都不得修改自己的参数，需要修改时一律新建对象再返回。」这个规定会带来什么好处与什么代价？请至少各举一例。

答案：**好处**包括：（1）别名 bug 大幅减少——调用方可以假定数据在调用前后不变，从而只需局部推理（Safe from bugs、Easy to understand）；（2）契约更简单，无需写 `Modifies` 子句，也无需「调用方不得修改返回值」这类终身契约；（3）方法变得像数学函数，易于测试与并行化，天然线程安全。**代价**包括：（1）性能与内存——如 `String` 循环拼接退化为 O(n^2)，`List` 的原地排序、原地去重等高效算法每次都要整体复制，此时应改用 `StringBuilder`、或提供显式的原地修改版本；（2）某些操作本质上就是修改语义，例如迭代器的 `next()`（必须推进自身位置）、缓存填充、`removeIf` 之类的批量操作，强行「不改」会让 API 变得别扭；（3）大的对象图（如项目作业中的 `Graph`）每次都复制可能不可接受。结论：这条规定是**好默认值**，但必须允许有明确理由的例外，并把例外写进规格；判断标准仍是「别名有多少、作用域有多大、性能代价是否可接受」。

问题 3：为什么说「不可变类型从不需要防御性拷贝」，这个结论在什么时候反而会让不可变类型不如可变类型高效？

答案：因为不可变对象一旦创建就无法改变，任何持有者都不可能通过它影响别人，因此跨边界传递时只需传引用，不必每次调用都复制一份——这正是 `startOfSpring()` 返回 `LocalDate` 可以安全共享同一对象、而返回 `Date` 时若不拷贝就会污染缓存的原因。反过来，当**同一个值需要被频繁地小幅修改**时，不可变类型的每次修改都要生成新对象并复制未改动部分，代价随值的大小线性增长，而可变类型可以原地修改，只付出常数代价；典型例子是循环中累积拼接字符串（`String` 的 `+` 与 `StringBuilder` 的 `append`）以及需要原地排序、原地更新的大数组。此外，防御性拷贝的开销只有在「跨界传递」时才发生，如果某段代码里的可变对象完全局部、只有单一引用，它连拷贝都不需要，因而在这种场景下比不可变类型更省。因此最终的判断依据是：**这个值更多是被共享，还是更多被修改。**

Reading 9: 避免调试（Avoiding Debugging）

> 说明：本讲 sp22 原版使用 TypeScript，本笔记按用户要求提供 Java 代码示例；类型/API 与 sp21（6.031 Java 版）原文保持一致。

概述

本讲的主题是「调试」，但真正讲的是**如何写代码，使我们根本不需要调试，或者在不得不调试时能够轻松找到原因**。6.031 的原始结构是两道防线：**第一道防线——让 bug 不可能发生（make bugs impossible）**，用静态检查、动态检查、不可变性与 `final` 把一整类错误消灭在设计层面；**第二道防线——把 bug 局部化（localize bugs）**，用快速失败（fail fast）、断言（assertion）、增量开发、单元测试、模块化、封装与作用域最小化，把 bug 的影响限制在尽可能小、尽可能新的代码范围内。

为便于学习与自检，本笔记把它整理为四个层层递进的防御层次：**① 让 bug 不可能（静态/动态检查 + 不可变性 + `final`）→ ② 让 bug 显而易见（断言与类型即文档、把不变量写出来、作用域清晰）→ ③ 让 bug 尽早失败（防御式编程、前置条件检查、`checkRep`、失败时抛异常）→ ④ 把 bug 彻底隔离并测试（增量开发、单元测试与回归测试、模块化与封装、作用域最小化、DRY）**。这四个层次共同服务于三大目标：**Safe from bugs**（预防并消灭 bug）、**Easy to understand**（断言与 `final` 是机器可检查的文档）、**Ready for change**（断言让未来的改动一旦破坏假设就立刻报错）。

核心概念与设计原则详解

四层防御总览（Four Lines of Defense） – **定义与目的**：把「避免调试」的手段按「事前预防 → 显式表达 → 就近暴露 → 隔离与验证」排序，形成一个可操作的检查清单，覆盖 Safe from bugs、Easy to understand、Ready for change 三个目标。 – **直观解释（"它是什么？"）**：把它想象成防洪：第一层是**不让水出现**（类型系统与不可变性直接排除某类错误）；第二层是**让水位可见**（断言、类型、`final` 把隐含假设写在代码里，读者一眼可见）；第三层是**让堤坝在最初渗漏时就报警**（fail fast：错误在离原因最近的地方抛出，而不是让脏数据漂流到远处才爆炸）；第四层是**挖好分区的水渠并反复巡检**（增量开发、单元测试、回归测试、模块化与封装、作用域最小化，把任何 bug 限制在一个小格子里）。 – **关键规则与最佳实践**：写代码时按此顺序自问——「这个错误能不能让编译器/运行时替我拒绝？」「这个假设能不能用断言或类型写出来，让读者与编译器都看得见？」「一旦违反，这里能立刻失败吗？」「如果仍然出错，我要搜多大范围的代码？」；四层都要用，因为任何单一手段都有盲区：静态检查管不了语义错误，断言管不了外部故障，测试也不可能穷尽所有输入。

第一层：让 bug 不可能（Make Bugs Impossible） – **定义与目的**：最好的防御是设计上不可能出错。这一层包括静态检查（static checking）、自动动态检查（automatic dynamic checking）与不可变性（immutability），直接服务于 Safe from bugs。 – **直观解释（"它是什么？"）**：静态检查在编译期就拒绝类型错误、拼写错误、参数个数错误——这类 bug 根本没机会进入运行期。自动动态检查则在运行期主动拦截：Java 会在数组或 `List` 下标越界时立刻抛错（而 C/C++ 会静默地越界读写，造成 bug 与安全漏洞；TypeScript 在这点上也不如 Python/Java，因为它对越界读取只是安静地返回 `undefined`）。不可变性同样属于「让 bug 不可能」：`String` 没有任何方法能改变它所表示的字符序列，因此字符串可以随意传递共享，

无需担心被别的代码改动。 – **关键规则与最佳实践**：优先选择静态类型语言与静态检查友好的写法（用接口类型、避免强制类型转换、启用编译告警）；优先选择会主动报错的容器与 API；默认使用不可变类型（见 Reading 8）；把「这一整类错误能不能从设计上消除」当作设计评审的常规问题。

不可重赋值的引用与 `final`（Unreassignable References with `final`） – **定义与目的**：`final` 声明的变量只能被赋值一次，之后永不重赋。它是「让 bug 不可能 / 让假设显而易见」的低成本手段，也是给读者和编译器的文档。 – **直观解释（"它是什么？"）**：`final char[] letters = new char[]{'a','e','i','o','u'};` 之后，`letters = new char[]{'x','y','z'};` 会被编译器**静态拒绝**，而 `letters[0] = 'z';` 完全合法——因为 `final` 只让**引用**不可重赋，数组元素照样能改。这一点必须牢牢记住，否则会写出「我明明标了 `final`，为什么状态还是变了」的代码（详见 Reading 8 的不可变类设计规则）。 – **关键规则与最佳实践**：给方法参数、局部变量、字段尽可能加 `final`；在 TypeScript 中对应的是 `const`（局部）与 `readonly`（实例字段），约定是「能用 `const` 就用 `const`，绝不用 `var`」；把 `final` 与不可变性配合使用——`final` 锁引用，不可变类型锁值，两者共同生效才能保证「对象整个生命周期表示同一个值」。

第二层与第三层：防御式编程、快速失败与断言（Defensive Programming, Fail Fast, Assertions） – **定义与目的**：当我们无法阻止 bug 时，就让它**就近、尽早失败**。防御式编程（defensive programming）通过运行时检查减轻 bug 的破坏力，而断言是把这种检查标准化的手段，服务于 **Safe from bugs** 与 **Ready for change**。 – **直观解释（"它是什么？"）**：若 `sqrt(x)` 的规格写 `requires x >= 0`，调用方传了负数，那么按契约 `sqrt` 已不受约束，理论上可以返回任意值、死循环、甚至烧掉 CPU。但既然这是调用方的 bug，最有用的行为就是在**离 bug 最近的地方**指出它：插入对前置条件的运行时检查并抛出异常。真实的程序几乎不可能没有 bug，防御式编程提供了「即使不知道 bug 在哪，也能限制其影响」的手段。更早观察到问题，就更容易修复——这就是 fail fast。 – **关键规则与最佳实践**：在方法入口检查前置条件（参数要求）；在方法出口检查返回值要求（self check，如 `assert Math.abs(r*r - x) < .0001;`）；在对象的每次状态变化后检查表示不变量（`checkRep()`）；断言应当**边写代码边写**，而不是事后补——写代码时你脑子里正好装着那些不变量，事后再补一定会漏；把「抛异常」与「用断言」区分开：**内部假设用断言，外部故障用异常**。

Java 断言机制（The `assert` Statement in Java） – **定义与目的**：`assert` 是 Java 语言内置语句（不是一个方法），用于表达「此处程序状态应当满足某条件」。它既是可执行的检查，也是文档（**Easy to understand**），并让未来的改动一旦破坏假设就立刻暴露（**Ready for change**）。 – **直观解释（"它是什么？"）**：最简形式 `assert x >= 0;` 在布尔表达式为 false 时抛出 `AssertionError`；也可以附带描述表达式（通常是字符串，也可以是基本类型或对象引用），用冒号分隔：`assert x >= 0 : "x is " + x;`。当 `x == -1` 时，错误消息是 `x is -1`，并附带堆栈轨迹，告诉你断言在代码中的位置以及到达该处的调用序列——这些信息通常足以开始定位 bug。与注释的区别在于：**断言是可执行代码，会在运行时强制检查假设**。 – **关键规则与最佳实践**：Java 的断言**默认关闭**（语言设计者考虑到检查有时代价高昂，例如用二分查找要求数组有序，若真去验证有序性就把对数时间变成线性时间），必须显式用 `-ea`（enable assertions）打开；测试时应当乐意（甚至渴望）付出这个代价，发布给用户后则未必；在 Eclipse 中可通过 Run → Run Configurations → Arguments 的 VM arguments 填 `-ea`，或直接在 Preferences → Java → Installed

JREs → Edit → Default VM Arguments 中默认开启；始终确保跑 JUnit 测试时断言是打开的，可以用下面这个测试来验证。此外要区分两套机制：Java 的 `assert` 语句用于**实现代码内部的防御式检查**，JUnit 的 `assertTrue()`、`assertEquals()` 等方法用于**测试代码中检查测试结果**，前者不打开 `-ea` 就完全不起作用，后者永远生效。 – **补充代码（课程原文给出的断言启用自检）**：

```
java @Test public void testAssertionsEnabled() { assertThrows(AssertionError.class, () -> { assert false; }); }
```

说明：`assertThrows` 接收「期望的错误类型」与「应当抛出该错误的函数（此处用 lambda）」。若断言已开启，lambda 中的 `assert false` 会抛出 `AssertionError`，测试通过；若断言被关闭，lambda 什么也不做，不会抛出预期异常，测试失败——这样就能保证测试环境确实启用了断言。

应该断言什么（What to Assert） – **定义与目的**：断言要选在「假设最容易被破坏、而破坏后果最严重」的位置，从而在 **Safe from bugs** 上获得最大收益。 – **直观解释（"它是什么？"）**：两类最值得断言的检查是——**方法参数要求**（如 `sqrt` 的 `x >= 0`）与**方法返回值要求**（self check，如把结果平方回去看是否接近 `x`）。第三类是对象层面的**表示不变量**（rep invariant），即用 `checkRep()` 断言「对象的内部表示此刻是合法的」（见下文）。 – **关键规则与最佳实践**：在方法入口断言参数约束；在返回前断言结果性质（self check）；在构造器、生产者（producer）与修改器（mutator）末尾调用 `checkRep()`；观察器（observer）不必强制调用，但**顺手调用是好的防御实践**，因为这样更容易抓到由表示暴露（rep exposure）导致的不变量破坏——这正是 Reading 11 的结论；断言要写在**方法体内部**（面向实现者），而不是规格注释里（面向客户）；写断言的时机是**写代码的当下**。

不应该断言什么（What Not to Assert） – **定义与目的**：运行时断言不是免费的：它会增加代码噪声与执行开销，因此必须节制使用，否则反而损害 **Easy to understand**。 – **直观解释（"它是什么？"）**：四类应当避免的断言是——（1）**平凡断言**：`x = y + 1; assert x == y + 1;` 只能证明编译器或虚拟机有 bug，而这二者在你有充分理由怀疑之前都应当被信任；（2）**外部条件断言**：文件是否存在、网络是否可用、用户输入是否正确，这些不是程序内部状态，失败不表示程序出了 bug，而且无论你怎么改程序都无法阻止它们发生——外部故障要用异常处理（如 `FileNotFoundException`、`NoRouteToHostException`）；（3）**带副作用的断言表达式**：因为断言可能被关闭，`assert list.remove(x);` 在关闭断言时整个表达式被跳过，元素根本没被删除；（4）**用断言处理「不应到达」的分支**：`switch` 的 `default` 分支若用 `assert` 兜底，一旦断言被关闭就形同虚设，应当抛出异常（如 `throw new AssertionError(...)`），这样检查永远会发生。 – **关键规则与最佳实践**：断言只检查**程序内部状态**是否落在规格允许的范围内；**绝不要让程序的正确性依赖断言是否被执行**；凡是「这段代码绝不能走到」的地方，用抛异常而不是断言；断言一句话说不清时，先问「这是不是该用异常或直接删掉」。

`checkRep()` 与表示不变量（`checkRep()` and the Rep Invariant） – **定义与目的**：表示不变量（representation invariant, RI）是「对象的内部字段必须始终满足的性质」；`checkRep()` 是把它变成运行时断言的方法，用于在每次改动的当下抓住错误，是第三层防御（尽早失败）的核心（关联 Reading 11）。 – **直观解释（"它是什么？"）**：课程 Reading 11 给出的 `RatNum` 例子是：

`java // Check that the rep invariant is true // *** Warning: this does nothing unless you turn on assertion checking // by running Java with -enableassertions (or -ea) private void checkRep() { assert denominator > 0; assert gcd(Math.abs(numerator), denominator) == 1; }`

它断言「分母为正」且「分子分母互质」。每次创建或修改 rep 之后调用它，就能在表示被破坏的**那一刻**发现问题，而不是等到很久以后某个无关的运算得出荒谬结果。 – **关键规则与最佳实践**：在构造器、生产者、修改器的末尾调用 `checkRep()`；观察器也建议调用（用于抓表示暴露）；`checkRep()` 声明为 `private`，因为「检查并维护 rep 不变量」是**实现者**的责任，不是客户的责任；`checkRep()` 不是抽象函数（AF）——它检查的是 rep 是否合法，而不是 rep 代表什么抽象值；记住它同样只在 `-ea` 打开时有效，所以团队约定必须保证断言始终开启。

第四层：把 bug 局部化——增量开发与测试（Incremental Development, Unit & Regression Testing） – **定义与目的**：把 bug 限制在「刚写的那一小块代码」里，是让调试变便宜的最有效手段，服务于 Safe from bugs。 – **直观解释（"它是什么？"）**：增量开发（incremental development）的意思是一次只构建程序的一小部分，并在继续之前把这一部分测透；这样一旦发现 bug，它极可能就在你刚刚写的那部分里，而不是散落在成千上万行代码中。测试课（Reading 3）中的两个技术正好配合它：**单元测试**——孤立地测试一个模块，因此你找到的 bug 一定在这个单元里（或者就在测试用例本身里）；**回归测试**——给大系统加新功能时尽可能频繁地跑回归测试套件，一旦失败，bug 大概率就在你刚改的代码里。 – **关键规则与最佳实践**：先写规格与测试策略，再写实现（test-first programming）；小步提交、小步验证，不要一次写完整个模块才运行；每次改动后跑完整测试套件，把「测试失败」当作定位 bug 的指针；为被测单元提供足够小的接口，使其能被孤立测试（这正是模块化的价值）。

模块化与封装（Modularity & Encapsulation） – **定义与目的**：良好的软件设计本身就降低调试成本。模块化（modularity）把一个系统划分为可独立设计、实现、测试、推理与复用的组件；封装（encapsulation）在模块周围筑墙，使模块对自己的内部行为负责，其他部分的 bug 无法破坏它的完整性。二者共同服务于 Safe from bugs 与 Easy to understand。 – **直观解释（"它是什么？"）**：一个由单个超长函数组成的程序是**单体（monolithic）**的——难以理解，也难以隔离 bug；拆成许多小函数与小类则更加模块化。封装有两种主要形式：**访问控制**（`public` / `private`）与**变量作用域**。`public` 变量或方法能被任何代码访问，`private` 的只能在同一个类内访问；尽可能把东西（尤其是变量）设为 `private`，就限制了可能无意中造成 bug 的代码范围。同时，把只供内部使用的辅助方法设为 `public` 会让接口变得杂乱——公开接口越小、越连贯（只做一件事并做好），代码越容易理解；反过来，若让外部依赖了本应私有的辅助方法，将来就难以改动内部实现（Ready for change）。 – **关键规则与最佳实践**：默认 `private`，只在确实要对外提供服务时用 `public`；用「一个模块只承担一件事」的标准检查接口是否连贯；把内部状态保护起来（在后续讲持久内部状态的类时，这还会成为防 bug 的关键）；宁可多传参数，也不要为了让多个部分「方便共享」而扩大可见性。

作用域最小化（Minimizing Variable Scope） – **定义与目的**：变量的作用域（scope）是程序文本中该变量可见、可被引用的那部分。作用域越小，需要搜索 bug 的代码范围越小，服务于 Safe from bugs 与 Easy to understand。 – **直观解释（"它是什么？"）**：假设你发现某个循环永远跑不完（`i` 始终到不了 100），那

么一定有「某人」在改 `i`。如果 `i` 是全局变量，它的作用域是整个程序：可能是 `doSomeThings()` 改的，可能是 `doSomeThings()` 调用的另一个方法改的，可能是某个并发线程改的——你必须搜遍全局。如果 `i` 在 `for` 初始化器里声明为局部变量，那么能改它的地方只有 `for` 语句本身（实际上只有循环体里那些你没写出来的部分）；`doSomeThings()` 根本访问不到这个局部变量，因此可以直接排除。 – **关键规则与最佳实践**：始终在 `for` 初始化器中声明循环变量（`for (int i = 0; ...)`），而不是在循环之前声明（后者会把作用域扩大到外层整个花括号块）；在 TypeScript 中始终使用 `const` 或 `let`、**绝不用** `var`——`const` / `let` 的作用域是最小的花括号块，而 `var` 的作用域是整个函数；只在第一次需要时声明变量，并放在**包含所有使用点的最内层花括号块**里（不要在函数开头一次性声明所有变量）；避免全局变量——它们常被当成「给多处传参数的捷径」，正确的做法是把参数传给真正需要它的代码，而不是放在全局空间里等着被无意间重赋。

避免重复代码（Don't Repeat Yourself, DRY） – **定义与目的**：重复代码（duplicated code）是安全性的风险：如果两处有相同或相似的代码，根本风险就是「bug 存在于两份拷贝中，而某个维护者只修好了其中一份」。DRY 服务于 **Safe from bugs** 与 **Ready for change**。 – **直观解释（"它是什么？"）**：Reading 4（Code Review）里的 `dayOfYear` 是经典反例：它用一长串 `if` / `else if` 把每个月之前的天数硬编码进去，`31 + 28 + 31 + 30` 这样的累加表达式被反复抄写。假设日历改变（例如二月真的有 30 天），你要改的地方不止一处，改漏一处就会产生难以察觉的日期错误。复制粘贴是极具诱惑力的编程手段，但每当你按下粘贴键时，都应该感到一丝危险——拷贝的代码块越长，风险越大。 – **关键规则与最佳实践**：把重复的**值**抽成常量（如各月天数表），把重复的**控制流**抽成循环或辅助方法；用表驱动（table-driven）写法替代枚举式分支；把「同一份逻辑出现两次」当作必须重构的信号；注意 DRY 也有边界——不要为了复用而破坏契约（例如为了复用 `sum()` 而修改调用方的列表，见 Reading 8），**DRY 不能以牺牲正确性为代价**。

代码示例与对比分析

场景 1：不检查前置条件，让非法输入一路漂到远处才爆炸

❌ *错误代码*

```
// 错误：前置条件只写在注释里，实现完全不检查，
// 负数输入会得到毫无意义的结果，并且错误会在很远的地方才显现。
/**
 * @param x 要求 x >= 0
 * @return x 的平方根近似值
 */
public static double sqrt(double x) {
    double r = x / 2.0;                 // 对负数照样能算出结果
    for (int i = 0; i < 20; i++) {
        r = (r + x / r) / 2.0;          // 迭代到 NaN 也不报错
    }
    return r;                           // 传入 -4 时返回 NaN
}
```

【错误代码的问题】 1. **错误被掩盖**：调用方传了负数（违反前置条件的 bug 在调用方），但 `sqrt` 安静地返回 `NaN`，bug 的现场被破坏。 2. **影响范围扩散**：`NaN` 会继续参与后续计算并污染其他结果，最终在离 bug 原因很远的代码处爆发，调试成本骤增。 3. **规格与实现不一致**：注释承担了契约角色，却没有任何机制保证契约被遵守；对读者来说「要求」二字形同虚设。 4. **难以定位**：即便最终发现是 `NaN`，也需要反向追踪到这次调用，而调用点可能离得很远。

✅ *正确代码*

```
// 正确：在方法入口做防御式检查，让违反前置条件的调用立刻失败。
/**
 * @param x 要求 x >= 0
 * @return x 的平方根近似值
 */
public static double sqrt(double x) {
    if (!(x >= 0)) {
        throw new IllegalArgumentException("required x >= 0, but was: " + x);
    }
    assert x >= 0;                          // 与上面的显式检查二选一，或同时保留
    double r = x / 2.0;
    for (int i = 0; i < 20; i++) {
        r = (r + x / r) / 2.0;
    }
    assert Math.abs(r * r - x) < 0.0001 : "bad approximation: r = " + r;   // self check
    return r;
}
```

【为什么这样更好】 非法调用在**离 bug 最近的地方**（即调用点之后的第一条语句）就抛出带清晰消息的未检查异常 `IllegalArgumentException`，堆栈轨迹直接指向调用者；调用方的错误假设立刻暴露，`NaN` 不会流向程序的其余部分。返回前的 `assert` 进一步检查实现自身的正确性（self check），把「结果是 x 的平方根」这一后置条件也变成可执行检查。

【代码对比解说】 两种写法的差别是「检查放在哪里」。这里要注意选择机制的原则：`x >= 0` 是**方法参数要求**，代表调用方可能犯的错，因此使用异常（且必须是未检查异常，否则会强迫客户写 try-catch）更合适；而 `Math.abs(r*r - x) < 0.0001` 检查的是**实现自身的正确性**，属于内部假设，用 `assert` 更合适。两者都体现了 fail fast，区别在于「谁该为失败负责」。

【设计原则透视】 这正是 Reading 7「前置条件还是后置条件」讨论的落地：如果要求可以低成本检查，就把它写成「参数不合适时抛出异常」这样的后置条件，让错误尽早可见。它同时把契约从纸面变成了可执行代码——注释不会失败，断言会。

场景 2：断言表达式带有副作用

❌ *错误代码*

```
// 错误：把“删除元素”和“断言删除成功”合并成一句，
// 一旦断言被关闭（Java 默认如此），元素根本不会被删除。
public static void removeAllInstances(List<String> list, String target) {
    while (list.contains(target)) {
        assert list.remove(target);      // 断言里的副作用！
    }
}
```

【错误代码的问题】 1. **行为随 `-ea` 开关而改变**：开着断言时元素被删除，关闭断言时循环体什么都不做——`while (list.contains(target))` 变成死循环。 2. **违反「正确性不得依赖断言」原则**：程序的正确性绝不能取决于断言表达式是否被执行。 3. **测试环境与生产环境行为不一致**：本地（开了 `-ea`）测试通过，部署（未开 `-ea`）后挂起，属于最难排查的一类问题。 4. **可读性差**：把「做一件事」和「检查一件事」写在同一个表达式里，读者很难一眼看出副作用。

✅ *正确代码*

```
// 正确：先执行副作用，再用断言检查结果。
public static void removeAllInstances(List<String> list, String target) {
    while (list.contains(target)) {
        final boolean found = list.remove(target);   // 副作用在这里发生，一定会执行
        assert found;                                // 断言只负责检查
    }
}
```

【为什么这样更好】 副作用被移到断言之外，因此无论断言是否启用，删除都会发生；断言退化成了纯粹的检查，符合「断言表达式不得有副作用」的规则。这样即使生产环境关闭断言，程序语义也完全不变。

【代码对比解说】 这条规则常被误解为「断言不重要」，其实相反：正因为断言可能被关闭，才要求它**无副作用**，从而可以被安全地开关。同类错误还有 `assert (x = compute()) > 0;`（赋值写在断言里）等。更好的做法通常是连 `while` 也不需要：`list.removeIf(target::equals)`（补充说明，Java 8+ 惯用法）。

【设计原则透视】 这是「断言可能被关闭」这一事实的直接推论，也是 Reading 8 中「隐藏的副作用很危险」的另一种形态：写代码时必须能一眼看出「状态在哪里被改变」。它把「可读性」与「正确性」这两件事绑在一起。

场景 3：断言平凡事实，以及断言程序无法控制的外部条件

❌ *错误代码*

```
// 错误 1：平凡断言——只能证明编译器或 JVM 有 bug。
public static int increment(int y) {
    int x = y + 1;
    assert x == y + 1;        // 这句话对定位 bug 毫无帮助
    return x;
}

// 错误 2：断言外部条件——文件是否存在、网络是否可用不是程序的内部状态。
public static String readConfig(String filename) {
    File f = new File(filename);
    assert f.exists() : "config file missing";     // 断言挡不住外部故障
    return readAll(f);
}
```

【错误代码的问题】 1. **浪费且误导**：平凡断言没有发现 bug 的能力，只会让代码变吵；读者会误以为这里有某种不明显的风险。 2. **概念错位**：`assert f.exists()` 把「外部环境不满足」误当成「程序内部出了 bug」，而当程序真的遇到缺失文件时，断言失败的消息（"config file missing"）会与真正的 bug 报告混在一起，难以区分。 3. **无法预防**：无论怎么改程序，都无法阻止用户删掉文件或网络断开；用断言处理这类情况等于放弃处理。 4. **被关闭后形同虚设**：最需要报错的场景（生产环境、断言关闭）反而什么都不会发生。

✅ *正确代码*

```
// 正确 1：删掉平凡断言，或改为真正有价值的断言。
public static int increment(int y) {
    final int x = y + 1;
    return x;                 // 这种“不变式”无需断言，本地上下文已显然
}

// 正确 2：外部故障用异常处理，让调用方决定如何应对。
/**
 * @param filename 配置文件名
 * @return 文件内容
 * @throws IOException 文件不存在或读取失败时
 */
public static String readConfig(String filename) throws IOException {
    try (BufferedReader in = new BufferedReader(new FileReader(filename))) {
        StringBuilder sb = new StringBuilder();
        for (String line = in.readLine(); line != null; line = in.readLine()) {
            sb.append(line).append('\n');
        }
        return sb.toString();
    }
}
```

【为什么这样更好】 断言集中用于**程序内部状态**是否符合规格，噪声降低、信号变强；外部条件则通过异常显式声明在规格中（`@throws IOException`），调用方必须面对它，无法忽视。注意 `readConfig` 内部并未修改任何状态，因此也不需要 `checkRep`。

【代码对比解说】 判断标准只有一条：**失败的根源是「程序写错了」还是「环境不配合」**。前者用断言（且应当是内部假设），后者用异常。注意 Java API 中很多方法的文档把要求写成前置条件却又承诺特定异常，那

在语义上就是后置条件（Reading 7）——这正是「外部故障用异常」的标准做法。

【设计原则透视】 这是本讲两层防线的分工：断言负责「保持内部一致性」（内部状态是否仍在规格范围内），异常负责「与外部世界打交道」。混淆二者会让 bug 报告失去意义，也会让测试无法判断「这次失败是代码问题还是环境问题」（影响 Reading 3 的测试可信度）。

场景 4：用 `assert` 兜底「不可能到达」的 `switch` 分支

❌ *错误代码*

```
// 错误：default 分支用 assert 兜底，断言一旦关闭，非法输入会被静默接受。
public static String vowelClass(char vowel) {
    switch (vowel) {
        case 'a': case 'e': case 'i': case 'o': case 'u':
            return "A";
        default:
            assert false : "must be a vowel, but was: " + vowel;   // 关闭 -ea 后彻底失效
            return "A";                                            // 于是非法输入也返回 "A"
    }
}
```

【错误代码的问题】 1. **检查会消失**：Java 断言默认关闭，因此这个「兜底」在生产环境根本不存在。 2. **静默给出错误答案**：非法输入（例如 `'x'`）会被当成元音处理，错误结果继续传播。 3. **误导读者**：`assert false` 看上去像是严密的分支覆盖，实际提供了虚假的安全感。 4. **难以测试**：若测试环境开了 `-ea` 会通过，生产环境行为不同，形成「测不出来的生产 bug」。

✅ *正确代码*

```
// 正确：抛异常兜底，检查永远发生（即使断言被关闭）。
public static String vowelClass(char vowel) {
    switch (vowel) {
        case 'a': case 'e': case 'i': case 'o': case 'u':
            return "A";
        default:
            throw new AssertionError("must be a vowel, but was: " + vowel);
    }
}
```

【为什么这样更好】 抛出 `AssertionError`（或更贴切的 `IllegalArgumentException`）不依赖 `-ea`，因此无论运行配置如何，非法输入都会立刻失败；这既让 bug 尽早暴露，也让「这里绝不能到达」这一假设成为**始终生效**的可执行文档。

【代码对比解说】 语言层面的差别很小（`assert false` 换成一个 `throw`），语义差别却很大：断言是「可以被关掉的检查」，异常是「永远执行的检查」。因此凡是**正确性所必需**的检查，都必须用异常；只有「额外的自查」才适合用断言。这也是为什么本讲强调「断言用于内部一致性、异常用于所有必须保证的行为」。

【设计原则透视】 这条对比把 Reading 7 的契约语言接了过来：后置条件（「只接受 5 个元音字母，否则失败」）必须由始终生效的机制保障。同时它体现了「让 bug 尽早失败」的工程取舍——宁可立刻崩溃，也不要

带着错误结果继续跑。

场景 5：循环变量声明在循环外，作用域覆盖整个方法甚至整个程序

❌ *错误代码*

```
// 错误：i 的作用域是整个类（静态字段），谁都能改它。
public static int i;                    // 全局可变状态

public static void doSomeThings() {
    // 恶意或无意地改动了 i：
    i = 0;
}

public static void countTo100() {
    for (i = 0; i < 100; ++i) {         // 用的是全局 i
        doSomeThings();                 // 每次调用都把 i 归零 —— 死循环
    }
}
```

【错误代码的问题】 1. **死循环且原因隐蔽**：`i` 永远到不了 100，而要找出「谁改了 `i`」，你必须搜索整个程序——包括 `doSomeThings()` 调用的所有方法，甚至并发线程。 2. **无法并行/递归**：`countTo100` 不能重入，两次调用会互相干扰。 3. **修改的可见性失控**：`public static` 字段把内部计数暴露为全局状态，任何模块都能写（违背封装）。 4. **测试困难**：测试之间会通过共享状态相互污染，出现「单独跑可以、整套跑就挂」的现象。

✅ *正确代码*

```
// 正确：循环变量在 for 初始化器中声明，作用域仅限于循环本身。
public static void doSomeThings() {
    // 这里根本访问不到 countTo100 的局部变量 i，因此不可能破坏它
}

public static void countTo100() {
    for (int i = 0; i < 100; ++i) {   // i 的作用域仅限这个 for 语句
        doSomeThings();
    }
}
```

【为什么这样更好】 能修改 `i` 的代码范围被压缩到 `for` 语句内部，`doSomeThings()` 可以直接排除在嫌疑范围之外——这正是「局部化 bug」想要的效果；方法可以安全地重入与递归；不存在跨测试的状态污染。

【代码对比解说】 Java 中变量作用域以花括号块为单位，因此「在 `for` 初始化器中声明」能把作用域缩到最小；把变量集中声明在函数开头（老式 C 风格）会让作用域不必要地变大。TypeScript 还有一条对应的规则：**始终使用 `const` / `let`，绝不用 `var`**——`const` / `let` 的作用域是最小的花括号块，而 `var` 的作用域是整个函数。补充说明：Java 10+ 的 `var`（局部变量类型推断）与 JavaScript 的 `var` 完全不同，它只是省略类型名，作用域仍是所在块。

【设计原则透视】 作用域最小化是「封装」在方法内部的对应物：封装限制**谁能看到**内部状态，作用域限制**谁能访问**变量名。两者共同决定「一个 bug 可能来自多大的代码范围」，因此与 Reading 8 中「别名越多、推理越难」是同一个道理。

场景 6：表示不变量只写在注释里，没有任何运行时检查

❌ *错误代码*

```
// 错误：RatNum 的表示不变量（分母为正、分子分母互质）只存在于注释中，
// 字段可变且可以被任意设置，破坏不变量后要很久以后才会暴露。
public class RatNum {
    private int numerator;
    private int denominator;          // 可变字段，没有任何约束

    public RatNum(int numerator, int denominator) {
        this.numerator = numerator;
        this.denominator = denominator;   // 可能为 0，可能为负，可能未约分
    }

    public void setDenominator(int d) {
        this.denominator = d;             // 任何值都能设进去，包括 0
    }

    public double toDouble() {
        return (double) numerator / denominator;   // 分母为 0 时得到 Infinity
    }
}
```

【错误代码的问题】 1. **不变量被静默破坏**：`new RatNum(2, 4)` 没有被约分，`setDenominator(0)` 直接制造出非法对象，任何依赖「分母为正且互质」的代码都会出错。 2. **错误延迟暴露**：非法状态会在很久以后的某次运算中表现为荒谬结果（如 `Infinity`、错误的相等性判断），届时已难以追溯是何时被破坏的。 3. **可变字段使类无法成为不可变类型**：字段非 `final`，客户可以随时改变对象表示的值（违背 Reading 8 的不可变设计规则）。 4. **观察器无防护**：每个方法都必须自行假设 rep 合法，重复且容易遗漏。

✅ *正确代码*

```
// 正确：final 字段 + 构造器建立不变量 + checkRep() 在每次改动后立即验证。
// 以下 RatNum 与 Reading 11 的示例一致（checkRep 与其注释取自课程原文）。
public class RatNum {
    private final int numerator;      // 不可变字段：无修改器可破坏不变量
    private final int denominator;

    public RatNum(int numerator, int denominator) {
        if (denominator == 0) {
            throw new IllegalArgumentException("denominator is zero");
        }
        final int g = gcd(Math.abs(numerator), Math.abs(denominator));
        final int sign = denominator < 0 ? -1 : 1;
        this.numerator = sign * numerator / g;
        this.denominator = Math.abs(denominator) / g;
        checkRep();
    }

    public double toDouble() {
        return (double) numerator / denominator;
    }

    // Check that the rep invariant is true
    // *** Warning: this does nothing unless you turn on assertion checking
    // by running Java with -enableassertions (or -ea)
    private void checkRep() {
        assert denominator > 0;
        assert gcd(Math.abs(numerator), denominator) == 1;
    }

    private static int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }
}
```

【为什么这样更好】 不变量在**每一次创建对象的瞬间**就被建立（约分、把符号移到分子、强制分母为正），并立即由 `checkRep()` 验证；`final` 字段让对象创建后不可能被改坏，因此「表示不变量始终成立」由结构保证，而不仅仅靠断言。若将来增加了修改器（mutator），规则是：在每个修改器末尾再调用 `checkRep()`。

【代码对比解说】 注意这里同时用了两种机制：**用非法参数抛异常**（`denominator == 0` 是调用方的错误，属于契约层面）与**用断言检查内部不变量**（rep 是否合法，属于实现层面）。这与「外部故障用异常、内部假设用断言」的分工完全一致。另外，`checkRep()` 是 `private` 的——维护不变量是**实现者**的责任，客户端无权也无必要调用它；观察器方法虽然不改变 rep，但顺手调用 `checkRep()` 是好的防御实践，因为这样更容易抓到由表示暴露（rep exposure）引起的破坏。

【设计原则透视】 这是 Reading 11（Abstraction Functions & Rep Invariants）与 Reading 9 的交汇点：**RI 是「让 bug 不可能 / 尽早失败」在对象层面的具体形式**，而 AF 与 RI 的区分提醒我们——`checkRep()` 检查的是 rep 是否合法，而不是 rep 代表什么抽象值。把 `final`、异常、断言三者配合使用，才能让「对象永远处于合法状态」成为结构性保证。

场景 7：`dayOfYear` 中的重复代码（DRY）

❌ *错误代码*

```
// 错误：一模一样的累加表达式被反复抄写，同样的事实在多处重复。
public static int dayOfYear(int month, int dayOfMonth, int year) {
    if (month == 2) {
        dayOfMonth += 31;
    } else if (month == 3) {
        dayOfMonth += 59;
    } else if (month == 4) {
        dayOfMonth += 90;
    } else if (month == 5) {
        dayOfMonth += 31 + 28 + 31 + 30;
    } else if (month == 6) {
        dayOfMonth += 31 + 28 + 31 + 30 + 31;
    } else if (month == 7) {
        dayOfMonth += 31 + 28 + 31 + 30 + 31 + 30;
    } else if (month == 8) {
        dayOfMonth += 31 + 28 + 31 + 30 + 31 + 30 + 31;
    } else if (month == 9) {
        dayOfMonth += 31 + 28 + 31 + 30 + 31 + 30 + 31 + 31;
    } else if (month == 10) {
        dayOfMonth += 31 + 28 + 31 + 30 + 31 + 30 + 31 + 31 + 30;
    } else if (month == 11) {
        dayOfMonth += 31 + 28 + 31 + 30 + 31 + 30 + 31 + 31 + 30 + 31;
    } else if (month == 12) {
        dayOfMonth += 31 + 28 + 31 + 30 + 31 + 30 + 31 + 31 + 30 + 31 + 31;
    }
    return dayOfMonth;
}
```

【错误代码的问题】 1. **修一处漏一处**：假设日历改变（例如二月真的有 30 天），需要修改的数字散布在多个分支里，漏改任何一处都会得到错误的日期。 2. **没有前置条件检查**：传入 `month = 13` 或 `dayOfMonth = 40` 都会安静地返回一个「看起来合理」的日期，错误被掩盖（违背 fail fast）。 3. **难以阅读与验证**：读者必须逐条核对每一行的加法是否正确，而不是一次性验证一张天数表。 4. **难以扩展**：想支持闰年（二月 29 天）就必须再复制一套分支，重复度进一步上升。

✅ *正确代码*

```
// 正确：把各月天数抽成一张表，用循环累加；同时断言前置条件。
private static final int[] MONTH_LENGTH =
        { 0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };

/**
 * @param month 月份，要求 1 <= month <= 12
 * @param dayOfMonth 当月第几天，要求 1 <= dayOfMonth <= 该月天数
 * @param year 年份（本示例未处理闰年，与课程原文一致）
 * @return 该日期在当年是第几天
 */
public static int dayOfYear(int month, int dayOfMonth, int year) {
    assert 1 <= month && month <= 12 : "month out of range: " + month;
    assert 1 <= dayOfMonth && dayOfMonth <= MONTH_LENGTH[month]
            : "day out of range: " + dayOfMonth;

    int total = dayOfMonth;
    for (int m = 1; m < month; ++m) {     // 每个月的天数只写一次
        total += MONTH_LENGTH[m];
    }
    return total;
}
```

【为什么这样更好】 每个月的天数只出现一次（DRY），日历改变时只需改一处；`for` 循环取代了长长的 `if` 链，逻辑一眼可验证；入口处的断言把「月、日必须在合法范围内」这一前置条件变成可执行的检查，非法输入立刻失败而不是返回错误日期；将来要支持闰年，只需在计算前对二月长度做一次修正，而不必改动整个分支结构。

【代码对比解说】 两版程序对合法输入的输出完全相同，差别在**可维护性与安全性**。这也是 Reading 4（Code Review）中 `dayOfYear` 例子的核心教训：重复不只是「不好看」，而是「bug 被复制了多份」。同时要注意 DRY 的边界——本讲的 DRY 不应走向 Reading 8 场景 1 那种「为了复用而修改调用方数据」的歧路；**正确的做法是复用计算逻辑，而不是复用可变状态**。

【设计原则透视】 这条对比把本讲的多个层次串在一起：表驱动是「让 bug 不可能」的一种形式（数字只写一次），断言是「让 bug 尽早失败」，循环内的局部变量 `m` 与 `total` 是「作用域最小化」，而常量表的 `private static final` 是「封装 + 不可重赋值引用」。这正是四层防御在同一段小代码里协同工作的样例。

与其他设计原则的关联

- **Reading 2（Basic Java）与 Reading 6（Specifications）**：静态检查与类型系统（Reading 2）是「让 bug 不可能」的第一层；而本讲所有防御式检查的判断依据都是规格（Reading 6）——「检查什么」由前置条件决定，「检查结果是否符合预期」由后置条件决定。没有规格，断言就无从下手。
- **Reading 3（Testing）**：增量开发、单元测试与回归测试是「把 bug 局部化」的核心手段：单元测试保证找到的 bug 属于被测单元，回归测试保证新改动引入的 bug 出现在刚改的代码里。此外，本讲强调测试时必须开启 `-ea`（并可写 `testAssertionsEnabled` 这类测试来验证），否则断言全部形同虚设，测试的可信度会大幅下降。

- **Reading 4（Code Review）**：本讲 DRY 与「模块化/封装」的很多结论直接来自代码评审课：`dayOfYear` 的重复代码、`countLongWords` 的全局变量与打印副作用，都是同一批「坏味道」。断言与 `final` 是把 Code Review 中「要在注释里写清假设」升级为「用可执行代码强制假设」。
- **Reading 7（Designing Specifications）**：本讲「前置条件还是后置条件」与 Reading 7 完全衔接——当检查代价低、方法对外公开时，把要求写成「参数非法则抛出未检查异常」这样的后置条件，是本讲的 fail fast 在契约层面的表达；而 `sqrt` 的例子正是 Reading 7 讨论过的 `atan(y, x)` 前置条件检查的同类。
- **Reading 8（Mutability & Immutability）**：不可变性既是本讲第一层防御（让 bug 不可能），也与 `final` / `const` 一起构成「不可重赋值引用」的规则；而「为复用而修改调用方数据」类 bug（`sumAbsolute`）与本讲的副作用禁忌同源。表示不变量检查（`checkRep`）又要求 rep 尽可能不可变，才能保证不变量不被静默破坏。
- **Reading 11（Abstraction Functions, Rep Invariants）**：`checkRep()` 的完整规则在此展开：在构造器、生产者与修改器末尾调用，观察器也建议调用（用于抓表示暴露），必须声明为 `private`，且它检查的是 RI 而非 AF。本讲的场景 6 是这一章节的预习。
- **Reading 13（Debugging）**：本讲讲「如何避免调试」，下一讲则讲「当 bug 真实存在时如何系统性地定位它」——两者的关系是：本讲的所有手段（断言、作用域最小化、模块化、可复现的测试）都会显著缩短 Reading 13 中调试所需的时间。
- **Reading 21、23（Concurrency；Locks）**：全局可变状态、作用域过大的变量在并发下会变成数据竞争；断言与不可变对象在并发程序中的价值更高，因为并发 bug 难以复现，而「设计上不可能出错」与「就近失败」是最可靠的对策。

关键要点

- **防御要分层，四层都要用。** ① 让 bug 不可能（静态检查、自动动态检查、不可变性、`final`）→ ② 让 bug 显而易见（断言、类型与 `final` 作为机器可检查的文档）→ ③ 让 bug 尽早失败（防御式编程、前置条件检查、`checkRep`、非法分支抛异常）→ ④ 把 bug 隔离并验证（增量开发、单元测试、回归测试、模块化与封装、作用域最小化、DRY）。
- **断言是可执行的文档，但只在 `-ea` 打开时生效。** Java 的 `assert` 默认关闭；跑测试时必须开启，并可用 `assertThrows(AssertionError.class, () -> { assert false; })` 验证；程序的正确性绝不能依赖断言表达式是否被执行，因此断言表达式不得有副作用。
- **断言内部假设，异常处理外部故障与本该永不发生的分支。** 参数检查常用未检查异常（如 `IllegalArgumentException`）；返回值自查（self check）与表示不变量用断言；文件、网络、用户输入等外部条件用异常（`IOException`）；`switch` 的 `default` 分支要抛异常而不是 `assert`，因为断言可能被关闭。
- **不变量要写出来并检查。** `checkRep()` 在构造器、生产者、修改器末尾调用（观察器也建议调用），声明为 `private`；它检查的是表示不变量（RI），而不是抽象函数（AF）。
- **作用域最小化，避免全局状态。** 循环变量在 `for` 初始化器中声明；只在第一次使用时、在最内层花括号块中声明变量；TypeScript 中只用 `const` / `let`、绝不用 `var`；避免全局变量，改为显式传参——能改某个变量的代码范围越小，调试要搜的范围就越小。

- **模块化与封装是把 bug 关进笼子的结构性手段；DRY 是防止「bug 被复制多份」。** 默认 `private`、保持公开接口小而连贯；重复的值抽成常量、重复的控制流抽成循环或辅助方法，但绝不为复用而破坏契约（不要修改调用方的数据）。

常见陷阱与注意事项

1. **以为写了 `assert` 就一定在检查** → Java 断言默认关闭，不加 `-ea` 时所有断言被完全跳过；结果是「本地不报错的假设」在生产环境悄悄失效。对策：在 IDE 与测试配置中默认开启 `-ea`，并写 `testAssertionsEnabled` 之类的自检测试。
2. **把带副作用的表达式塞进断言** → 如 `assert list.remove(x);`，断言关闭时删除操作根本不会执行，程序行为随运行配置而变。对策：先执行副作用（`final boolean found = list.remove(x);`），再断言结果。
3. **用断言检查外部条件** → 如 `assert f.exists()`，把「环境不配合」误判为「程序有 bug」，且断言关闭时毫无保护。对策：外部故障一律用异常（`IOException`、`FileNotFoundException`），并在规格中显式声明。
4. **在「不可能到达」的分支里用 `assert false`** → 断言关闭后非法输入被静默接受，产生错误结果。对策：抛 `AssertionError` 或 `IllegalArgumentException`，让检查永远生效。
5. **把无用注释式的平凡断言当作文档** → 如 `x = y + 1; assert x == y + 1;`，只会增加噪声并暗示存在并不存在的风险。对策：只在「本地上下文看不出来」的地方写断言，并优先断言参数要求、返回值要求与表示不变量。
6. **在函数开头集中声明所有变量、循环变量声明在循环外、使用全局变量** → 作用域不必要地扩大，任何一行代码都可能成为嫌疑犯，调试时必须搜索整段甚至整个程序；还会造成测试之间的状态污染。对策：就近、就内层声明，循环变量写进 `for` 初始化器。
7. **为了复用（DRY）而修改传入的可变对象** → 复用本身值得鼓励，但修改调用方数据会造成隐藏副作用（Reading 8），属于「用正确性原则换取代码行数」的坏交易；对策：复用计算逻辑而不是复用可变状态。
8. **`checkRep()` 只在构造器里调用一次** → 后续修改器破坏了不变量却无人检查，问题延迟到很远的计算中才爆发。对策：每个会创建或修改 rep 的操作末尾都调用 `checkRep()`，并在测试时确保断言开启。

思考题（带答案）

问题 1：下面这段代码有两处与断言相关的错误，请指出并给出修改后的写法。

```
public static int quadratic(int a, int b, int c, int x) {
    assert a != 0;
    int value = a*x*x + b*x + c;
    assert value == a*x*x + b*x + c;
    return value;
}
```

答案：第一处错误是**平凡断言**——`assert value == a*x*x + b*x + c;` 只是在复述上一行的赋值，能发现的只有编译器或 JVM 的错误，而这两者应当被信任；应当删除。第二处问题在于**断言的类型选择**：`a != 0`

是调用方的参数要求（前置条件），用断言表达意味着「断言关闭时非法调用被静默接受」，而参数检查属于「正确性所必需」的约束，应当抛出始终生效的未检查异常，并在规格中把它写成后置条件。修改后的写法：

```
/**
 * @param a 二次项系数
 * @param b 一次项系数
 * @param c 常数项
 * @param x 自变量
 * @return a*x*x + b*x + c
 * @throws IllegalArgumentException 当 a == 0 时（此时不是二次式）
 */
public static int quadratic(int a, int b, int c, int x) {
    if (a == 0) {
        throw new IllegalArgumentException("a must be nonzero, but was: " + a);
    }
    return a*x*x + b*x + c;
}
```

若团队约定「断言始终开启」，保留 `assert a != 0 : "a == 0";` 作为额外自查也无妨，但它不能取代抛异常。（附带一提：题目给出的 `quadraticRoots` 类方法中，位置 A 合理的断言是 `assert a != 0;`，位置 B 合理的是 `for (double root : roots) { assert Math.abs(a*root*root + b*root + c) < 0.0001; }` 与 `assert roots.size() <= 2;`——前者是返回值自查，后者是后置条件检查；`assert roots.size() >= 0;` 与 `assert b != 0;`、`assert c != 0;` 都是无意义的平凡断言或与契约无关的检查。）

问题 2：某同学写道：「我给所有方法都加上了断言，所以我的程序已经很安全了。」请从本讲四个防御层次的角度评价这个说法，并说明还缺什么。

答案：这个说法**夸大了断言的作用**。断言只覆盖第三层防御的一部分——它能在运行时检查内部假设（参数要求、返回值自查、表示不变量），但存在三个明显局限：（1）Java 断言默认关闭，若没有 `-ea`，全部检查都不执行，因此断言不能替代始终生效的检查（正确性所必需的检查必须用异常）；（2）断言检查不了外部故障（文件、网络、用户输入），这些必须用异常处理；（3）断言只能在「程序跑到那一步」时发现问题，它不能阻止 bug 进入代码。完整的四层防御还需要：**第一层**——用静态检查（类型、`final`）与不可变性让某类 bug 不可能发生（例如用 `String` 而不是 `char[]` 返回学号，从根本上消除别名修改）；**第二层**——让假设对读者显而易见（类型、`final`、有意义的断言、清晰的命名）；**第四层**——增量开发、单元测试与回归测试、模块化与封装、作用域最小化、避免重复代码，从而把 bug 限制在小范围内并持续验证。此外还应检查断言本身的质量：是否出现了平凡断言、是否有副作用的断言表达式、是否把本该抛异常的分支用 `assert` 兜底——这些都会让「加了很多断言」变成虚假的安全感。

问题 3：为什么本讲把「避免在迭代过程中修改集合」「把循环变量声明在循环里」「避免全局变量」这三件事都归入「避免调试」，而不是仅仅当作编程风格问题？

答案：因为这三件事共同决定了**调试时你需要搜索多大的代码范围**，也就是 bug 的「局部性」。缺陷本身不可避免，但缺陷被发现时离原因越近、可能的嫌疑代码越少，修复成本就越低：迭代中修改集合并不会立刻报错（自写迭代器会静默跳过元素），错误结果与真正原因之间隔着整段循环逻辑；把 `i` 声明为全局变量后，能改它的地方是整个程序（包括被调用的其他方法、以及并发线程），而声明在 `for` 初始化器中后，嫌疑范围被压缩到循环体内部，`doSomeThings()` 可以直接被排除；全局变量不仅扩大作用域，还会让「谁能看见这份状态」失控，从而使测试之间相互污染、产生「单独跑通过、整套跑失败」的现象。三者都与 Reading 9

的第二道防线（把 bug 局部化）完全对应：**增量开发与单元测试把 bug 限制在「你刚写的那部分」，作用域最小化与封装把 bug 限制在「你能一眼看完的那部分代码」**。因此它们不是审美偏好，而是直接决定调试成本（也即 Safe from bugs 与 Ready for change）的工程决策。

Reading 10: 抽象数据类型（Abstract Data Types）

> 说明：本讲 sp22 原版使用 TypeScript，本笔记按用户要求提供 Java 代码示例；类型/API 与 sp21（6.031 Java 版）原文保持一致。

概述

本讲要解决的核心问题是：**客户端（client）对类型的内部表示（representation）做出假设**——这是软件构造中特别危险的一类耦合，因为一旦实现者更换数据结构，所有偷偷依赖内部字段的客户端代码都会失效，而且这类失效经常连编译器都发现不了（Family 的 `client3` 就是典型）。为此 6.031 提出两个紧密相连的思想：**抽象数据类型（abstract data type, ADT）**——一个类型由「你能对它执行哪些操作」以及每个操作的规格来完整刻画，而不是由它内部怎么存来刻画；以及**表示独立（representation independence）**——客户端只依赖公开操作的规格，实现者因此可以自由替换表示。这两个思想同时服务于课程的三大目标：ADT 用「**操作契约 + `private` 表示**」把非法状态挡在编译期和调用边界之外（Safe from bugs），用「**客户端只需读懂操作、不必读懂实现**」降低理解成本（Easy to understand），用「**换 rep 而不改客户端**」让系统能够应对变化（Ready for change）。

核心概念与设计原则详解

抽象数据类型（Abstract Data Type, ADT）

- **定义与目的**：ADT 是由**一组操作及其规格**完整刻画的数据类型：对类型 `T` 而言，「`T` 的全部含义」就等于「`T` 的这些操作所满足的规格」。它要解决的软件质量问题是：把「数据在程序里怎么被使用」与「数据本身以什么形式存在」分离开（sp22 原文的表述是 separate how we use a data structure in a program from the particular form of the data structure itself）。
- **直观解释（"它是什么？"）**：数字是什么？是你能对它做加、减、乘、除的东西；字符串是什么？是你能对它做拼接、取子串的东西；布尔值是什么？是你能对它取反的东西。我们从来不关心编译器把 `int` 存成几个字节、字节序如何——ADT 就是把这种「只用操作说话」的态度搬到用户自定义类型上。ADT 的值是**不透明的（opaque）**：客户端不能「看进去」，除非通过操作。原文建议把它想象成一堵**规格防火墙（specification firewall）**之后的硬壳：壳里藏的不只是某一个函数的实现，而是**一组相关操作的实现**以及它们共享的数据（`private` 字段）。
- **关键规则与最佳实践**：
- 定义类型时，先列**操作集合 + 每个操作的前置/后置条件**；这套东西就是类型的规格，也是客户端唯一被允许知道的全部信息。
- 值的内部数据必须不可被客户端直接读写；一切访问都走操作。

- 明确区分两个词：**操作集合 = 抽象（abstraction）**（public，客户端可见）；**实现类里的字段 = 一个具体的表示（representation）**（private，只有实现者可见）。
- 记住历史脉络能帮你理解为什么这么设计：Dahl（Simula）、Hoare（抽象类型的推理技术）、Parnas（提出 information hiding、主张按「模块封装了什么秘密」来组织程序）；MIT 的 Barbara Liskov 与 John Guttag 在抽象类型的规格与语言支持上做了奠基性工作，Liskov 因抽象类型相关研究获得图灵奖。
- 在 Java 里「内置类型」与「用户自定义类型」的界限是模糊的：`java.lang.Integer`、`Boolean` 等用与用户类完全相同的类/对象抽象来定义，但 `int`、`boolean` 这类**原始类型（primitive type）不能由用户扩展**。写 ADT 时要意识到这一点：你不能给 `int` 加操作，只能另建一个类型。

使用者与实现者的分离：抽象边界（Client–Implementer Separation / Abstraction Barrier）

- **定义与目的**：任何一个类型都有两类读者——只**使用**它的客户端，和**实现**它的人。抽象边界就是两者之间的那条分界线：线上只有操作与规格，线下才是字段、辅助类与算法细节。目的是让两边能够独立演化，把「误解」与「改一处坏一片」的耦合降到最低。
- **直观解释（"它是什么？"）**：像汽车的驾驶舱与发动机舱。司机只操作方向盘、油门、刹车（操作），不需要知道喷油策略、气门正时（表示）。只要这套操作的含义不变，换一台完全不同的发动机，司机毫无感觉。
- **关键规则与最佳实践**：
- 客户端**只**允许了解公开操作及其规格；任何对 `private` 字段、内部辅助类的依赖都算越界，即使它「现在能编译」。
- 实现者可以自由更换表示，前提是**操作的可观察行为不变**；这也是为什么规格必须写全前置条件与后置条件。
- 规格不完整时，抽象边界是假的：客户端不知道能依赖什么，实现者也不知道能安全改什么（这一点由 Reading 06 规格说明与 Reading 07 设计规格奠定）。
- 抽象边界同时是**测试边界**：客户端级测试是黑盒的（只通过操作），实现者级测试才是白盒的（可以检查 rep）。
- 在源码里，抽象边界具体体现为 `public` / `private` 这两个关键字——它不是一个口头约定，而是有编译器背书的机制。

操作的分类：构造者、生产者、观察者、修改者（Classifying Operations: Creators, Producers, Observers, Mutators）

- **定义与目的**：把 ADT 的操作按下述两个维度分成四类：**输入/输出里有没有出现本类型 `T`**、**执行后对象本身是否被改变**。目的是给出一张设计 ADT 时的检查清单——四类操作是否齐备、每一类是否都必要、某个操作是否放在了错误的类别里。
- **直观解释（"它是什么？"）**：把类型想象成一台机器。**构造者**是从零造出一台机器（不需要已有机器）；**生产者**是拿旧机器造出一台新机器（旧机器不动）；**观察者**是读表盘（机器不动，返回的是别的信息）；**修改者**是扳动开关（机器本身被改变）。

- **关键规则与最佳实践**（分类表格如下，重点看「输入是否含 T」「输出是否含 T」两列）：

操作类别（English）	签名形状	输入是否含该类型对象	输出是否含该类型对象	是否修改对象	Java 例子
构造者 creator	`t* → T`	**否**（只能含其他类型 `t`）	**是**（必须返回 `T`）	否	`new ArrayList<>()`、`List.of()`、`String.valueOf(int)`
生产者 producer	`T+, t* → T`	**是**（至少一个 `T`）	**是**（返回新的 `T`）	否	`String.concat`、`String.substring`、`String.toUpperCase`、`Collections.unmodifiableList`
观察者 observer	`T+, t* → t`	**是**（至少一个 `T`）	**否**（返回其他类型 `t`）	否	`List.size`、`List.get`、`String.length`、`String.charAt`
修改者 mutator	`T+, t* → void | t | T`	**是**（至少一个 `T`）	可有可无（`void`、其他类型或 `T` 都可以）	**是**	`List.add`、`List.remove`、`Collections.sort`

- 记号含义：`T` 是抽象类型本身，`t` 是别的类型；`+` 表示「出现一次或多次」，`*` 表示「零次或多次」，`|` 表示「或」。例如生产者可以吃两个 `T`，就像 `concat : String × String → String`。
- 观察者可以完全不吃其他类型：`size : List → int`；也可以吃很多：`regionMatches : String × boolean × int × String × int × int → boolean`。
- 构造者常常实现为**构造器**（`new ArrayList<>()`），但也可以只是**静态方法**——`List.of()`、`String.valueOf(...)` 都是；用静态方法实现的构造者通常叫**工厂方法（factory method）**。
- 修改者常常返回 `void`（返回 `void` 的方法必然是为了副作用而调用），但**不是必须**：`Set.add()` 返回 `boolean` 表示集合是否真的被改动；AWT 的 `Component.add()` 返回对象本身，以便链式调用。
- 分类并不完美：复杂类型里可能出现**同时是生产者和修改者**的操作。本课程把这类方法同时称作生产者和修改者，也有人只叫它修改者，把「生产者」一词留给不做修改的操作。
- 分类时别忘了**实例方法有一个隐藏的 `this` 参数**——`toUpperCase()` 看起来没有输入，实际上输入就是一个 `String`，所以它是生产者而不是构造者。

常见操作归类练习（可用作自测）：

操作	签名（含隐式 `this`）	归类	理由
`MyString.valueOf(boolean)`	`boolean → MyString`	构造者	输入里没有 `MyString`，输出是 `MyString`（工厂方法形式的 creator）
`MyString.length()`	`MyString → int`	观察者	返回其他类型，不改对象
`MyString.charAt(int)`	`MyString × int → char`	观察者	返回 `char`，不是 `MyString`
`MyString.substring(int, int)`	`MyString × int × int → MyString`	生产者	吃一个 `MyString`、返回**新的** `MyString`，原对象不变；它是本讲的示例 ADT **没有** mutator 的原因
`Integer.valueOf(int)`	`int → Integer`	构造者	输入不含 `Integer`，返回 `Integer`
`BigInteger.mod(BigInteger)`	`BigInteger × BigInteger → BigInteger`	生产者	吃一个 `T`、返回新的 `T`，不修改 `this`
`List.addAll(Collection)`	`List × Collection → boolean`	修改者	修改 `this`，返回值是其他类型（`boolean`）
`String.toUpperCase()`	`String → String`	生产者	返回新 `String`，原对象不变
`Set.contains(Object)`	`Set × Object → boolean`	观察者	返回其他类型，不改对象
`Map.keySet()`	`Map → Set`	观察者	返回的是**另一个** ADT（`Set`），不是 `Map` 本身
`StringBuilder.append(String)`	`StringBuilder × String → StringBuilder`	生产者 + 修改者	既改了 `this`，又返回 `this`
`List.size()`	`List → int`	观察者	只读，返回 `int`
`Collections.sort(List)`	`List → void`	修改者	静态方法，修改传入的列表
`BufferedReader.readLine()`	`BufferedReader → String`	修改者（兼有观察者的形态）	返回的是 `String` 而非 `BufferedReader`，但会推进内部读取位置，所以它有副作用

可变类型与不可变类型（Mutable vs Immutable Types）

- **定义与目的**：**可变类型**的对象可以被改变——存在某个操作，执行之后对**同一个对象**调用其他操作会得到不同的结果（`Date` 是可变类型：调用 `setMonth` 后，`getMonth` 的返回值变了）。**不可变类型**的操作则

创建新对象而不是修改已有对象（`String` 就是这样）。这个划分决定了别名（aliasing）与表示暴露（rep exposure）的风险等级，因而是 ADT 设计的第一决定。

- **直观解释（"它是什么？"）**：不可变对象像刻在石头上的字——别人拿到同一个引用也无所谓，因为他改不了；可变对象像白板，谁拿到引用谁都能擦，所以你必须谨慎决定把白板交给谁。
- **关键规则与最佳实践**：
- 有些类型同时提供可变与不可变两个版本：`String`（不可变）与 `StringBuilder`（可变）——注意二者不是同一个 Java 类型、不可互换；`List` 与 `Collections.unmodifiableList()` 包装出的视图也是这种关系。
- 不可变类型的字段应声明为 `private final`，并且**不提供任何 mutator**；但要牢记 `final` 只保证**引用不再改变**，不保证被引用对象的内容不变（`private final Date timestamp` 里的 `Date` 照样能被 `setTime` 改掉）。
- 可变类型只要把可变对象的引用**存进来**（构造器保存参数）或**发出去**（观察者返回内部对象），就产生表示暴露；必须做**防御性拷贝（defensive copy）**。
- 不可变性换来了一项重要的自由：**多个对象可以共享同一块内部数据**。`MyString` 的 `substring` 之所以能只记录 `start` / `end` 而不复制字符，正是因为它不可变；一旦加入 `reverse()` 这样的修改者，共享表示就会立刻变成 bug。
- 把可变对象同时交给两个别名持有，一处修改会「隔空」改变另一处看到的值——这就是 Reading 08 不可变性里的快照图与别名分析要练习的内容。

本讲原文正是用 Java 库里的真实类型来对照这两类：`int`、`String` 是不可变的（没有 mutator），`List`、`Date`、`StringBuilder` 是可变的。下面这段代码把原文 "Classifying types and operations" 与 "Abstract data type examples" 两节列出的操作原样归类在一起：

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.Date;
import java.util.List;

class AdtExamples {
    void examples() {
        // int: Java 的原始整数类型，不可变 —— 没有 mutator
        int n = 0;                          // creator: 字面量 0, 1, 2, ...
        int m = n + 1;                      // producer: 算术运算符 + 返回新值，n 仍然是 0
        boolean positive = n > 0;           // observer: 比较运算符 ==, !=, <, >

        // String: 不可变 —— 没有 mutator，所有"看起来会改"的操作都返回新串
        String s = String.valueOf(true); // creator: valueOf 静态工厂方法
        String t = s.substring(1, 3);    // producer: substring 返回新 String
        String u = s.concat("!");        // producer: concat 返回新 String
        String v = s.toUpperCase();      // producer: toUpperCase 返回新 String
        int len = s.length();            // observer: length
        char c0 = s.charAt(0);           // observer: charAt

        // StringBuilder: String 的可变版本 —— 注意二者不是同一个 Java 类型，不可互换
        StringBuilder sb = new StringBuilder("ab");  // creator: 构造器
        sb.append("c");                              // mutator: 返回 this，内容变成 "abc"
        sb.reverse();                                // mutator: 返回 this，内容变成 "cba"
        int sbLen = sb.length();                     // observer

        // Date: 可变 —— 可以调用 setMonth 再用 getMonth 观察到变化
        Date d = new Date();             // creator
        d.setTime(0L);                   // mutator
        long ms = d.getTime();           // observer

        // List: 可变，而且是接口 —— 真正的实现来自 ArrayList / LinkedList
        List<String> list = new ArrayList<>();       // creator: ArrayList 构造器
        list.add("x");                                // mutator: add
        int size = list.size();                       // observer: size
        String first = list.get(0);                   // observer: get
        List<String> ro = Collections.unmodifiableList(list);  // producer: 不可变包装
    }
}
```

sp22 原版 TypeScript 写法（对照）：sp22 用 `ReadonlyArray` 作为 `Array` 的不可变版本，Java 里没有独立的不可变数组类型，对应手段是 `Collections.unmodifiableList()` 或 `List.of()` 这类返回不可修改视图/列表的操作（在 Reading 11 中还会讨论：这种包装只在运行时阻止修改，编译期不报错）。

补充示例（非本讲原文，取自 Reading 02/06）：同一份材料里还常拿"点"来对比可变与不可变——本讲 Reading 10 原文只用 `MyString` / `List` / `String` / `StringBuilder` / `Date` 做可变性对比，并没有 `Point` 这个例子，下面两块代码是本笔记借 Reading 02/06 的 `Point` 话题补充的：

```
// 补充示例（非本讲原文，取自 Reading 02/06）：同一个"点"的两种 ADT 设计
public final class Point {                  // 不可变版本：只有 creator / observer / producer
    private final int x;
    private final int y;

    public Point(int x, int y) { this.x = x; this.y = y; }   // creator

    public int getX() { return x; }                          // observer
    public int getY() { return y; }                          // observer

    public Point translate(int dx, int dy) {                 // producer：返回新点，旧点不动
        return new Point(x + dx, y + dy);
    }
}

class MutablePoint {                        // 可变版本：多出两个 mutator
    private int x;
    private int y;

    MutablePoint(int x, int y) { this.x = x; this.y = y; }

    public int getX() { return x; }
    public int getY() { return y; }
    public void setX(int x) { this.x = x; }                  // mutator
    public void setY(int y) { this.y = y; }                  // mutator
}
```

可变/不可变的取舍并非"越不可变越好"：不可变类型要为新值分配新对象，修改频繁时成本更高，所以 Java 才同时提供 `String` 与 `StringBuilder`。课程的建议是：**默认选不可变**，只有确有性能或建模需要（例如一个会被大量就地修改的累积器）才提供可变版本，并把它封装好。

表示独立（Representation Independence）

- **定义与目的**：类型的**使用**与它的**表示**（实际采用的数据结构或数据字段）相互独立，因此表示的改变**不影响类型之外的任何代码**。这是本讲的第二个核心思想，直接对应「Ready for change」。
- **直观解释（"它是什么？"）**：`List` 提供的操作与它内部是链表还是数组无关；`MyString` 的 `charAt` / `length` / `substring` 与它内部是「独占字符数组」还是「共享字符数组 + `start` / `end`」无关。客户端写的是「取子串」，不是「复制数组的第 start 到 end 个元素」。
- **关键规则与最佳实践**：
- 表示独立的前提是**操作被完整规格化**（前置条件 + 后置条件齐全）：这样客户端才知道可以依赖什么，实现者才知道可以安全改什么。
- 表示独立 ≠ 表示不需要正确：客户端看不见 rep，但 rep 内部仍然必须满足不变量（表示不变量 RI 是 Reading 11 的主题）。
- 表示独立靠 `private` **强制**出来，而不是靠口头约定：只要字段是 `public`，客户端就**可能**依赖它（原文 Family 例子里 `client1`、`client2` 直接读写 `f.people`）。

- 违反表示独立有三级后果：最轻的是**静态错误**（编译就报错，例如把 `List` 换成 `Set` 后 `f.people.get(0)` 不存在）；中等的是**动态错误**（运行时抛异常）；最危险的是**静默给出错误答案**（能编译、不抛异常，只是结果变了——`client3` 依赖元素顺序就属于这一类）。
- 表示独立也让**客户端代码更易读**：读一个方法时不必跳进实现体去猜它是否修改了什么。

规格、表示与实现的三层划分（Specification / Representation / Implementation）

- **定义与目的**：原文的 Family 练习要求把每一段代码归类为「规格 / 表示 / 实现」。三者分清之后，code review 才能准确指出「这是实现细节泄漏进了规格」。这是理解抽象边界的具体抓手。
- **直观解释（"它是什么？"）**：规格是**合同**；表示是**仓库里货物实际怎么摆放**；实现是**干活的动作序列**。
- **关键规则与最佳实践**（对照原文 Family 的七段代码）：
- **规格**：类上方的 Javadoc（`/** Represents a family ... Families are mutable. */`）、方法签名（`public List<Person> getMembers()`）、方法上方的 Javadoc（`@return a list containing all the members ...`）。
- **表示**：`private List<Person> people;` 这条字段声明，以及描述 rep 性质的注释（「sorted from oldest to youngest, with no duplicates」）。
- **实现**：方法体 `return people;`，以及内部算法、辅助类。
- `class Family {` 这一行本身既不是规格也不是表示，它只是把三者打包在一起的语法外壳。
- 一条实用的自查：**如果一个客户端必须读懂某段代码才能正确使用这个类型，那么那段代码实际上已经是规格的一部分了**——要么把它写进 Javadoc，要么把它藏起来（改成 private 实现细节）。

访问控制：private 与 public（Access Control）

- **定义与目的**：Java 用 `private` / `public`（以及包级默认、`protected`）在语言层面建立抽象边界。`private` 表示「只有本类内部可见」，`public` 表示「任何地方可见」。目的是把越界访问变成**静态错误**，在编译期而不是运行时暴露问题。
- **直观解释（"它是什么？"）**：`private` 是给字段和内部辅助方法上的一把锁，唯一的钥匙是「同一个类里的代码」——注意钥匙是**按类**发的，不是按对象发的。
- **关键规则与最佳实践**（对照下面的 Wallet 例子）：
- Java 的 `private` 是**类级**授权：在 `Wallet` 的方法里写 `that.amount` 去访问**另一个** `Wallet` 对象的 private 字段是**合法**的。
- 换到 `Person` 类里写 `w.amount` 就是**静态错误**（`amount` 在另一个类中是 private）；写 `Wallet.amount == 0` 更是双重错误——既 private，又是**实例变量**而非 `static`，不能通过类名访问。
- `Wallet.main` 里写 `w.amount = 100` 在 sp21 的例子中是**合法**的，因为 `main` 恰好写在 `Wallet` 类内部。这说明可见性取决于「代码挂在哪一层」，而不是「谁在运行」。
- 因此结论很直接：**所有字段都应该是 `private`**；只有确实构成抽象的操作才 `public`。不要用包级可见（默认）当「半公开」的捷径。

- 注意构造器的可见性也是一个操作：不写任何构造器时 Java 会补一个 **public 的默认构造器**，等于凭空开放了一个 creator。若希望客户端只能通过工厂方法（或 `substring()`）拿到对象，就把构造器声明为 `private`。

```
class Wallet {
    private int amount;

    public void loanTo(Wallet that) {
        // put all of this wallet's money into that wallet
        that.amount += this.amount;   // 允许：private 按类授权，同类内部可见
        amount = 0;                   // 允许：等价于 this.amount = 0
    }

    public static void main(String[] args) {
        Wallet w = new Wallet();
        w.amount = 100;               // 允许：main 也写在 Wallet 类内部
        w.loanTo(w);                  // 允许：this 与 that 是同一个对象的别名
        System.out.println(w.amount); // 输出 0（先 += 变成 200，再被置 0）
    }
}

class Person {
    private Wallet w;

    public int getNetWorth() {
        return w.amount;              // 静态错误：amount 是 Wallet 的 private 字段
    }

    public boolean isBroke() {
        return Wallet.amount == 0;    // 静态错误：amount 既 private 又是实例变量
    }
}
```

sp22 原版 TypeScript 写法（对照）：

```
// sp22 原版 TypeScript 写法（节选，语义与上面的 Java 版一致）
class Wallet {
    private amount: number = 0;

    public loanTo(that: Wallet): void {
        that.amount += this.amount;   // 允许：同一类内部可以访问其它实例的 private 字段
        amount = 0;                   // 允许：省略 this 时默认指向本对象
    }
}
```

TypeScript 与 Java 在这里的规则几乎一样（都是类级授权、都允许省略 `this`），差别在语法与生态：TypeScript 没有"原始类型"这一层区分，而 Java 有 `int` / `boolean` 这些不可扩展的原始类型；TypeScript 用 `readonly` 表达"字段构造后不可再赋值"，Java 对应的是 `final`。

更有意思的对照是本讲的核心例子 `MyString`：sp22 与 sp21 给出的是**同一个抽象**、**两种语言外壳**，而且两边的 rep 都是 `private` 字段、两边的公开面都是那几个操作——这本身就是表示独立的最好演示（表示可以换，操作与规格不动）。

```
// sp22 原版 TypeScript 写法（原文的 MyString；rep 是 private 字段）
class MyString {
    private a: Uint16Array;          // 表示：16 位无符号整数数组

    /** @returns MyString representing the sequence of characters in s */
    public constructor(s: string) { /* ... */ }          // creator
    /** @returns number of characters in this string */
    public length(): number { /* ... */ }               // observer
    /** @param i character position (requires 0 <= i < string length) */
    public charAt(i: number): string { /* ... */ }       // observer
    /** @returns string consisting of charAt(start)...charAt(end-1) */
    public substring(start: number, end: number): MyString { /* ... */ }  // producer
}
```

```
// Java 对应写法（sp21 原文的 MyString）：同一个抽象，rep 换成 char[]，
// creator 换成静态工厂方法 valueOf —— 操作与规格的形态完全对应。
public class MyString {
    private char[] a;                // 表示：字符数组

    /** @param b a boolean value
     *  @return string representation of b, either "true" or "false" */
    public static MyString valueOf(boolean b) { /* ... */ }        // creator（工厂方法）

    /** @return number of characters in this string */
    public int length() { /* ... */ }                              // observer

    /** @param i character position (requires 0 <= i < string length)
     *  @return character at position i */
    public char charAt(int i) { /* ... */ }                        // observer

    /** @param start starting index
     *  @param end ending index. Requires 0 <= start <= end <= string length.
     *  @return string consisting of charAt(start)...charAt(end-1) */
    public MyString substring(int start, int end) { /* ... */ }    // producer
}
```

设计抽象类型的经验法则（Designing an Abstract Type）

- **定义与目的**：设计 ADT 就是「挑选好的操作集合，并决定它们应该如何表现」。这些法则用来判断一个操作集合是否**易于理解、不易出错、便于演化**。
- **直观解释（"它是什么？"）**：好的 ADT 像一套精简的工具箱——件数少、每件用途明确、能组合出很多活；坏的 ADT 像一个塞满专用扳手的抽屉，每个角落都要一把新扳手。
- **关键规则与最佳实践**：

- **少而简单 > 多而复杂**：宁可要少量简单操作，让它们能强力组合，也不要一大堆复杂操作。
- **每个操作目的明确、行为一致（coherent）**，而不是一堆特例。例如不该给 `List` 加 `sum`：对整数列表有用，那字符串列表怎么办？嵌套列表怎么办？这些特例会让 `sum` 变得难以理解和使用。
- **操作集合要足够（adequate）**：客户端想做的计算都要能做。原文给的检验方法是「对象的每一个性质都能被取出」——如果 `List` 没有 `get`，就根本取不出元素；`size` 严格说来并非必需（可以从下标 0 一直 `get` 到失败为止），但那样低效又难用，所以仍然值得提供。**基本信息不应该难以获得**。
- **通用（generic）与领域特定（domain-specific）不要混**：`Deck`（代表一副扑克牌的序列）不应该有接受任意对象的通用 `add` 方法（那会让 `Deck` 里混进整数或字符串）；反过来，把 `dealCards` 这种领域特定方法塞进通用的 `List` 也毫无意义。
- **四类操作齐备性检查**：能用 creator 造出来吗？producer 能否组合出需要的新值？observer 能否读出全部性质？mutator（如果是可变类型）是否覆盖了全部需要的变化？
- **小固定值集合用枚举（enum）**：一天的星期（原文举的例子）、罗盘方位、线段端点样式——这类 ADT 的值集小而有限，用 `enum` 一次定义所有命名值最省事，也最安全（原文的实现方式对照表把它列为 ADT 的三种实现手段之一）。

实现 ADT 概念的 Java 手段（Realizing ADT Concepts in Java）

- **定义与目的**：同一个「大思想」在 Java 里往往有多种实现途径。理解「概念」与「实现手段」的对应关系，才能在不同场景下做出合适选择。
- **直观解释（"它是什么？"）**：概念是「要做什么」，手段是「用哪个 Java 关键字/结构去做」。比如「构造者」这个概念，可以用构造器、工厂方法或常量三种手段实现。
- **关键规则与最佳实践**（对照表沿用 sp21 原文）：

ADT 概念	Java 中的实现方式	例子
抽象数据类型	类	`String`
抽象数据类型	接口 + 实现类	`List` 与 `ArrayList`
抽象数据类型	枚举	`DayOfWeek`
构造者操作	构造器	`new ArrayList<>()`
构造者操作	静态（工厂）方法	`List.of()`
构造者操作	常量	`BigInteger.ZERO`
观察者操作	实例方法	`List.get()`
观察者操作	静态方法	`Collections.max()`
生产者操作	实例方法	`String.trim()`
生产者操作	静态方法	`Collections.unmodifiableList()`
修改者操作	实例方法	`List.add()`

ADT 概念	Java 中的实现方式	例子
修改者操作	静态方法	`Collections.copy()`
表示	`private` 字段	（无）

- **接口 + 类**：`List` 只声明操作，`ArrayList`、`LinkedList` 各自提供表示。这是表示独立的**结构性**保证——客户端按 `List` 编程，实现可以随时换。
- **用常量作构造者**：常见于不可变类型——最简单或最空的那个值就是一个 public 常量，其余复杂值靠 producer 从它搭出来（`BigInteger.ZERO` 就是这个模式）。
- **用枚举作 ADT**：适合值集小而固定的类型，我们会在 Reading 12（接口、泛型与枚举）里展开。

测试抽象数据类型（Testing an ADT）

- **定义与目的**：ADT 的测试套件是「为每一个操作写测试」。但它与函数测试有一个关键差别：**ADT 的测试之间必然互相依赖**——测试 creator / producer / mutator 的唯一方式就是**对结果调用 observer**；测试 observer 的唯一方式就是**先造出对象让它观察**。
- **直观解释（"它是什么？"）**：你没法直接检查一个黑盒里装了什么，只能「做点什么」再「读出点什么」。所以一个测试用例天然是若干操作的合作演出。
- **关键规则与最佳实践**：
- 分区（partition）要基于**抽象状态**，**不要**基于 rep。原文明确要求：按字符串的**抽象长度**分区，而不是 rep 数组 `a` 的长度；按 `substring()` 的 `start` / `end` 入参分区，而不是 rep 里的 `start` / `end` 字段。
- 分区可以使用**客户端的知识**：这个实例是「由哪个创造者产生、经过哪些操作变成现在这样」的，这常常是不同代码路径的分界。可变 ADT 尤其要覆盖不同的操作序列。
- 在类型还没定义 `equals` 之前（见 Reading 15 相等性），不能直接用 `assertEquals` 比较两个 ADT 对象——只能用已经定义好的操作来断言（`length()`、`charAt()`）。
- 由 `substring()` 返回的对象要单独作为一类 `this` 来测试，因为不同构造路径可能走完全不同的代码。

```
// testing strategy for each operation of MyString:
//
// valueOf():
//   partition on the boolean argument: true, false
// length(), charAt(), substring():
//   partition on string length: 0, 1, >1
//   partition on this: produced by valueOf(), produced by substring()
// charAt():
//   partition on i = 0, 0 < i < len-1, i = len-1
// substring():
//   partition on start = 0, 0 < start < len, start = len
//   partition on end   = 0, 0 < end   < len, end   = len
//   partition on end - start: 0, > 0
```

```
@Test public void testValueOfTrue() {
    MyString s = MyString.valueOf(true);
    assertEquals(4, s.length());
    assertEquals('t', s.charAt(0));
    assertEquals('r', s.charAt(1));
    assertEquals('u', s.charAt(2));
    assertEquals('e', s.charAt(3));
}

@Test public void testSubstringIsWholeString() {
    MyString s = MyString.valueOf(false).substring(0, 5);
    assertEquals(5, s.length());
    assertEquals('f', s.charAt(0));
    assertEquals('e', s.charAt(4));
}

@Test public void testSubstringOfEmptySubstring() {
    MyString s = MyString.valueOf(false).substring(1, 1).substring(0, 0);
    assertEquals(0, s.length());   // 注意：this 由 substring() 产生，覆盖了那条分区
}
```

注意 `testValueOfTrue` 的「单元」不是一个操作：它同时测试了 `valueOf`、`length` 和 `charAt` 四个 `charAt` 调用。这不是设计缺陷，而是 ADT 测试的固有形态。

为什么 ADT 同时提升安全性、易理解性与可修改性

- **Safe from bugs（落到具体机制）**：① `private` 字段让客户端**无法**把对象置于非法状态——越界访问是编译期静态错误，而不是运行时惊喜；② 表示不变量（RI）因此只需要在类内部少量位置维护，容易出现「所有修改都经过同一道关卡」的结构；③ 每个操作的前置/后置条件构成了客户端与实现者之间的合同，边界处的契约违规可以 fail fast（这与 Reading 09 避免调试的精神一致）；④ 不可变 ADT 天生免疫别名攻击，也天生线程安全（Reading 21 并发的基础）。
- **Easy to understand（落到具体机制）**：① 客户端只需要读操作签名与规格，**不需要读实现**，阅读量的量级直接下降；② 操作分类给出清晰的心智模型：哪些操作会改状态（mutator）、哪些只是读（observer），读代码时能立刻判断「这行之后对象变了吗」；③ 一个类型只负责一个 concern（separation of concerns），不会把无关功能堆进来；④ 抽象边界把「读代码时要搜索的范围」压缩到了一个类的公开 API。
- **Ready for change（落到具体机制）**：① 表示独立让 rep 可以整体替换——`MyString` 从「独占数组」换成「共享数组 + `start` / `end`」，`Family` 从 `List` 换成 `Set`，客户端代码一行不改；② 只要规格不变，实现可以在不通知客户端的情况下做性能优化；③ 「接口 + 实现类」让「多种实现共存、按需切换」成为默认设计（Reading 12）。

代码示例与对比分析

场景 1：`Wallet` 与 `Person` 的访问控制——`public` 字段让抽象边界彻底消失

❌ *错误代码*

```
/** 错误：字段全部 public，"抽象边界"根本不存在。 */
public class Wallet {
    public int amount;                          // 错误：表示直接公开

    public Wallet(int amount) { this.amount = amount; }

    /** put all of this wallet's money into that wallet */
    public void loanTo(Wallet that) {
        that.amount += this.amount;
        this.amount = 0;
    }
}

class Person {
    public Wallet w;                            // 错误：连"人有钱包"这个字段也公开

    public Person(Wallet w) { this.w = w; }

    public int getNetWorth() {
        return w.amount;                        // 客户端直接读 Wallet 的 rep
    }

    public boolean isBroke() {
        return w.amount == 0;                   // 同一份 rep 依赖出现在多个类里
    }
}

class Client {
    void broken(Wallet mine, Person p) {
        mine.amount = -100;                     // 造出负余额，编译器不拦
        doubleIt(mine);                         // 别名：调用者手里的钱包被"隔空"改了
        p.w = null;                             // 连"这个人的钱包"都能被抹掉
        System.out.println(p.getNetWorth());    // 运行时 NullPointerException
    }

    /** 把传入钱包的钱翻倍 —— 它是个修改者，但没有任何操作约束 */
    void doubleIt(Wallet w) {
        w.amount *= 2;
    }
}
```

【错误代码的问题】

1. **抽象边界不存在**：所有字段都是 `public`，客户端（`Person`、`Client`）可以直接读写 `Wallet` 与 `Person` 的 rep，这既是表示暴露，也让"这个类型的值始终合法"这一承诺无法兑现。
2. **不变量无处安放**：`amount` 可以被设成负数（余额不变量被破坏），`p.w` 可以被置为 `null`（于是 `getNetWorth()` 抛 `NullPointerException`）。这些非法状态没有任何一个"关卡"能把它们挡住。
3. **表示再也换不掉**：客户端依赖"Wallet 里有一个 `int amount`"这个实现事实。将来想改成 `long`、想改成"余额 = 现金 + 信用额度"的复合表示、或想加一笔交易日志，所有写过 `w.amount` 的代码都要跟着改——表示独立被彻底摧毁。

4. **`private` 本可以在编译期就全部拦住**：只要把这些字段设为 `private`，`mine.amount = -100`、`p.w = null`、`doubleIt` 里的 `w.amount *= 2` 会**全部变成静态错误**。这是"错误在编译期暴露"与"错误在运行时爆炸"的区别。

✅ *正确代码*

```
/** 正确：private 字段 + 一组定义了抽象的操作（含前置条件校验）。 */
public class Wallet {
    private int amount;                     // 表示：只有本类内部可见

    /**
     * creator : int -> Wallet
     * @param amount 初始金额，要求 amount >= 0
     */
    public Wallet(int amount) {
        if (amount < 0) throw new IllegalArgumentException("negative amount");
        this.amount = amount;
    }

    /** observer : Wallet -> int */
    public int getAmount() { return amount; }

    /**
     * mutator : Wallet x Wallet -> void
     * @param that 收款方，要求 that != null
     */
    public void loanTo(Wallet that) {
        // put all of this wallet's money into that wallet
        that.amount += this.amount;         // 允许：private 按类授权，同类内部可见
        amount = 0;                         // 等价于 this.amount = 0
    }

    /**
     * mutator : Wallet x int -> void
     * @param delta 变动金额，要求 amount + delta >= 0
     */
    public void adjust(int delta) {
        if (amount + delta < 0) throw new IllegalArgumentException("would go negative");
        amount += delta;
    }
}

class Person {
    private Wallet w;                       // 表示：private

    public Person(Wallet w) { this.w = w; }

    /** observer : Person -> int */
    public int getNetWorth() {
        return w.getAmount();               // 通过操作读取，绝不碰别人的 rep
    }

    /** observer : Person -> boolean */
    public boolean isBroke() {
        return w.getAmount() == 0;          // 注意：原文写的 Wallet.amount == 0 是静态错误
    }
}

class Client {
```

```
    void correct(Wallet mine, Wallet yours, Person p) {
        mine.loanTo(yours);                 // 只有操作能改状态
        int worth = p.getNetWorth();        // 只有观察者能读状态
        // mine.amount = -100;              // 静态错误：amount has private access in Wallet
        // p.w = null;                      // 静态错误：w has private access in Person
    }
}
```

【为什么这样更好】

`private` 把表示关进类内部，客户端能用的只剩 `Wallet(int)`、`getAmount()`、`loanTo(Wallet)`、`adjust(int)` 与 `Person.getNetWorth()` / `isBroke()` 这几个操作，抽象边界由编译器强制。所有会改变状态的操作都集中在 `loanTo` 与 `adjust` 两处，`adjust` 在入口校验"不能变成负数"，构造器在入口校验"初始金额不能为负"——不变量因此有了单一守卫点。将来把 `amount` 换成 `long`、拆成复合表示、或加一层内部缓存，只要这些操作的规格不变，客户端一行都不用改。

【代码对比解说】

两个版本用的是**完全相同的类名与操作名**，差别只有两处：字段可见性，以及"改动状态是否必须经过带校验的操作"。这正说明抽象边界在 Java 里就是由 `public` / `private` 这两个关键字落地的。有三个细节值得记住：

- Java 的 `private` 是**类级**授权，不是对象级：`loanTo` 里写 `that.amount += this.amount` 访问**另一个** `Wallet` 对象的 private 字段是**合法**的；而 `Person` 类里写 `w.amount` 就是**静态错误**；`Wallet.amount == 0` 更是双重错误——既 private，又是实例变量而非 `static`，不能通过类名访问。
- 原文的例子把 `main` 写在 `Wallet` 类内部，于是 `w.amount = 100` 竟然**合法**。这提醒我们：可见性取决于"代码挂在哪一层"，而不是"谁在运行"。把这样的代码放在 `Wallet` 类里能编译，不代表它是一个好的设计——它只是把客户端逻辑塞进了实现者的地盘。
- 原文还留了一个"别名 + 修改者"的思考题：`w.loanTo(w)` 时 `this` 与 `that` 是同一个对象，`that.amount += this.amount` 先把余额变成两倍，紧接着 `amount = 0` 又把它清零。操作能编译不等于语义明确——规格应当说明这种自转账情形下的预期结果，客户端才不会依赖某个偶然实现。

【设计原则透视】

这一组对比是**抽象边界（abstraction barrier）**的最直接演示：`Wallet` 的**规格** = 四个操作及其前置/后置条件；`private int amount` 是**表示**；`loanTo` 的方法体是**实现**。三者分清之后，"这条 `w.amount = 100` 该不该存在"就有了判据：它把表示写进了客户端，属于越界。这也正是 Reading 06 规格说明所要求的"合同"、Reading 08 不可变性所强调的"不要让外部拿到可变状态"、以及 Reading 11 中"表示不变量与防表示暴露"的前置准备。注意 `adjust` 的 `throw new IllegalArgumentException(...)` 属于**前置条件被违反**时的快速失败（fail fast），与 Reading 09 避免调试中的思路一致：宁可响亮地崩，也不要静默地把对象弄坏。

场景 2：`Family` 的表示依赖——公开 rep 让客户端与「用 List 还是用 Set」死绑

❌ *错误代码*

```
import java.util.List;

class Person {
    private final String name;
    private final int age;
    Person(String name, int age) { this.name = name; this.age = age; }
    public String getName() { return name; }
    public int getAge() { return age; }
}

/**
 * Represents a family that lives in a household together.
 * A family always has at least one person in it.
 * Families are mutable.
 */
class Family {
    // the people in the family, sorted from oldest to youngest, with no duplicates.
    public List<Person> people;              // 错误：表示直接公开

    /** @return a list containing all the members of the family, with no duplicates. */
    public List<Person> getMembers() {
        return people;                       // 错误：把 rep 本身返回出去
    }
}

class Client {
    void client1(Family f) {
        // get youngest person in the family
        Person baby = f.people.get(f.people.size() - 1);   // 依赖"people 是 List，且按年龄排序
    }

    void client2(Family f) {
        int familySize = f.people.size();                  // 依赖 rep 的类型
    }

    void client3(Family f) {
        Person anybody = f.getMembers().get(0);            // 表面上走了操作，实际依赖返回顺序
    }

    void destroy(Family f) {
        f.people.clear();                                  // 客户端可以清空这个家庭
    }
}
```

【错误代码的问题】

1. **表示的变更会成为客户端的地震**：把 `people` 从 `List<Person>` 换成 `Set<Person>`（一个语义上完全合理的等价表示）后，`client1` 与 `client2` 立刻变成**静态错误**——它们编译不过了，必须逐个改写。这正是原文 Representation 1 / 2 练习要你识别的情形。

2. **最危险的一类依赖连编译器都发现不了**：`client3` 写的是 `f.getMembers().get(0)`，改动后依然能编译通过；如果 `getMembers` 的元素顺序变了（例如改用 `HashSet` 后），`client3` 会**静默地**得到另一个答案。原文 Representation 3 的答案就是这一类：依赖存在、无静态错误、无动态错误，但结果不同。
3. **不变量无法维护**：`getMembers()` 返回 rep 本身，客户端一次 `clear()` 或 `add()` 就可能破坏"至少一人""无重复""按年龄排序"这些 rep 的内部约定，而实现者对此毫无办法。
4. **规格被实现细节污染**：因为 `people` 是 public，它的类型与顺序事实上已成为规格的一部分，抽象边界形同虚设。

✅ *正确代码*

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

class Person {
    private final String name;
    private final int age;
    Person(String name, int age) { this.name = name; this.age = age; }
    public String getName() { return name; }
    public int getAge() { return age; }
}

/**
 * Represents a family that lives in a household together.
 * A family always has at least one person in it. Families are mutable.
 */
class Family {
    // rep: 成员按从长到幼排序，且无重复（对客户端不可见）
    private final List<Person> people;

    /**
     * creator : List<Person> -> Family
     * @param members 家庭的成员，必须至少有一人；
     *                调用者之后修改 members 不会影响本对象
     */
    public Family(List<Person> members) {
        if (members.isEmpty()) {
            throw new IllegalArgumentException("a family has at least one person in it");
        }
        this.people = new ArrayList<>(members);                                    // 防御性拷
        this.people.sort(Comparator.comparingInt(Person::getAge).reversed());    // 维护 rep
    }

    /** mutator : Family x Person -> void */
    public void add(Person p) {
        people.add(p);
        people.sort(Comparator.comparingInt(Person::getAge).reversed());
    }

    /**
     * observer : Family -> List<Person>
     * @return a list containing all the members of the family, with no duplicates.
     */
    public List<Person> getMembers() {
        return new ArrayList<>(people);   // 返回副本：客户端改不动 rep
    }

    /** observer : Family -> int */
    public int size() {
        return people.size();
    }
}
```

```
class Client {
    void client2(Family f) {
        int familySize = f.size();        // 只依赖操作，不依赖 rep 的类型
    }
}
```

【为什么这样更好】

`people` 变成 `private final`（并做了防御性拷贝）之后，客户端能用的只剩 `Family(...)`、`add`、`getMembers`、`size` 这几个操作。此时把 rep 从 `List<Person>` 换成 `Set<Person>`（或换成两个字段 `nuclear` + `others`）都**不影响客户端**：`size()` 对 `List` 和 `Set` 都成立，`getMembers()` 里的 `new ArrayList<>(people)` 对任何 `Collection` 都成立。这就是**表示独立**的直接收益。同时，"至少一人""无重复""有序"这些约定被集中在构造器与 `add` 两处维护，任何一次修改都必须过这两道关卡，不变量难以被绕过。

【代码对比解说】

两种写法的 `getMembers()` 签名一模一样，差别只在 `return people;` 与 `return new ArrayList<>(people);` 这一行。这一行是"是否暴露表示"的分水岭。但请注意一个更深的层次：**返回副本只解决了"结构被改"，没有解决"顺序被依赖"**。如果规格只承诺"返回全部成员、无重复"，那么 `client3` 里 `getMembers().get(0)` 取到的究竟是"最年长者"还是"任意一人"就取决于实现——这是**规格的充分性**问题，只能靠把顺序写进后置条件（或干脆提供一个语义明确的 `oldest()` 观察者）来解决。原文 Representation 4 的练习正是在训练这种分层判断：哪一行是规格、哪一行是表示、哪一行是实现。

【设计原则透视】

这一组对比是**表示独立（representation independence）**的标准教案：`private` 字段 + 规格完整的 public 操作 = 客户端与表示解耦。返回副本则对应 Reading 08 的**防御性拷贝**与 Reading 11 的**防表示暴露**。`Family` 是可变类型（有 `add` 这个 mutator），因此返回副本是必需的；如果 `Family` 设计成不可变类型（`add` 改成返回新 `Family` 的 producer），客户端拿到共享引用也仍然安全——这是"用不可变性替代拷贝"的另一条路。

场景 3：`MyString` 的 creator 与表示替换——从「独占数组」换成「共享数组 + 区间」

❌ *错误代码*

```
/** 错误：public 字段把表示变成规格；构造器保存调用者的数组；观察者返回内部数组。 */
public class MyString {
    public char[] a;                                    // 错误：rep 完全公开

    /** @param a 字符数组；长度为字符串长度 */
    public MyString(char[] a) {
        this.a = a;                                     // 错误：直接保存调用者的数组引用
    }

    /** @return number of characters in this string */
    public int length() { return a.length; }

    /** @param i character position (requires 0 <= i < string length) */
    public char charAt(int i) { return a[i]; }

    /** producer : MyString x int x int -> MyString */
    public MyString substring(int start, int end) {
        char[] sub = new char[end - start];
        System.arraycopy(this.a, start, sub, 0, end - start);
        return new MyString(sub);
    }
}

class Client {
    void broken(MyString s) {
        char c = s.a[0];                                // 直接读 rep：与"内部是 char[]"死绑
        s.a[0] = 'X';                                   // 直接改 rep：破坏对象内容
        char[] raw = new char[] { 'h', 'i' };
        MyString t = new MyString(raw);
        raw[0] = 'X';                                   // 别名：外部数组一改，t 也跟着变
    }
}
```

【错误代码的问题】

1. **表示无法更换**：客户端只要写过一次 `s.a`，实现者就再也不能把 rep 换成别的结构（例如带 `start`/`end` 的共享数组），否则客户端**编译不过**。原文的正向例子（MyString 的两种 rep）之所以成立，正是因为字段是 private。
2. **别名导致内容可被外部篡改**：`new MyString(raw)` 保存了 `raw` 的引用，随后 `raw[0] = 'X'` 会改变 `t` 看到的字符串——这个类型已经不再是"不可变的字符序列"了。
3. **不变量无守卫**：没有校验也没有拷贝，客户端可以构造出与内部约定冲突的对象（例如 `substring` 的调用者拿到数组后自行缩短它）。
4. **creator 集合失控**：public 的 `MyString(char[])` 加上隐式的默认构造器，意味着客户端可以用任何方式造出对象，实现者无法保证"每个 `MyString` 都处于合法状态"。

✅ *正确代码*

```
/** MyString represents an immutable sequence of characters. */
public class MyString {

    ////////////////////// 表示（private，客户端不可见） ////////////////////
    private char[] a;

    private MyString() { }   // 不开放默认构造器：客户端只能通过 valueOf / substring 得到对象

    ////////////////////// Example of a creator operation ////////////////////
    /**
     * @param b a boolean value
     * @return string representation of b, either "true" or "false"
     */
    public static MyString valueOf(boolean b) {
        MyString s = new MyString();
        s.a = b ? new char[] { 't', 'r', 'u', 'e' }
                : new char[] { 'f', 'a', 'l', 's', 'e' };
        return s;
    }

    ////////////////////// Examples of observer operations ////////////////////
    /**
     * @return number of characters in this string
     */
    public int length() { return a.length; }

    /**
     * @param i character position (requires 0 <= i < string length)
     * @return character at position i
     */
    public char charAt(int i) { return a[i]; }

    ////////////////////// Example of a producer operation ////////////////////
    /**
     * Get the substring between start (inclusive) and end (exclusive).
     * @param start starting index
     * @param end ending index. Requires 0 <= start <= end <= string length.
     * @return string consisting of charAt(start)...charAt(end-1)
     */
    public MyString substring(int start, int end) {
        MyString that = new MyString();
        that.a = new char[end - start];
        System.arraycopy(this.a, start, that.a, 0, end - start);
        return that;
    }

    /////// no mutator operations (why not?)
}

class Client {
    void correct() {
        MyString s = MyString.valueOf(true);      // s 表示 "true"
        MyString t = s.substring(1, 3);           // t 表示 "ru"
```

```
        char c = t.charAt(0);                    // 只能通过观察者取值
        int n = t.length();                      // n == 2
    }
}
```

表示变更后的版本（客户端代码一行不改）：

```
/** MyString represents an immutable sequence of characters. */
public class MyString {

    // 新表示：共享同一个字符数组，用 [start, end) 区间描述本对象代表的那一段
    private char[] a;
    private int start;
    private int end;

    private MyString() { }

    public static MyString valueOf(boolean b) {
        MyString s = new MyString();
        s.a = b ? new char[] { 't', 'r', 'u', 'e' }
                : new char[] { 'f', 'a', 'l', 's', 'e' };
        s.start = 0;
        s.end = s.a.length;
        return s;
    }

    public int length() { return end - start; }

    public char charAt(int i) { return a[start + i]; }

    public MyString substring(int start, int end) {
        MyString that = new MyString();
        that.a = this.a;                         // 共享数组：只有不可变类型才敢这么做
        that.start = this.start + start;
        that.end = this.start + end;
        return that;
    }
}
```

【为什么这样更好】

表示变成 private 之后，客户端能观察到的只有 `length()` / `charAt()` / `substring()` 的行为。把「独占数组」换成「共享数组 + `start`/`end`」时，`MyString.valueOf(true).substring(1,3).charAt(0)` 的结果仍然是 `'r'`——**客户端代码完全不需要修改，甚至不需要重新编译它的语义**。这正是原文的结论："Because MyString's existing clients depend only on the specs of its public methods, not on its private fields, we can make this change without having to inspect and change all that client code. That's the power of representation independence."同时，把默认构造器改成 `private` 收紧了她 creator 集合：对象只能由 `valueOf` 或 `substring` 产生，实现者因此可以保证每个 `MyString` 的 rep 都合法。

【代码对比解说】

三个关键差异：① 字段从 `public char[] a` 变成 `private char[] a`（外加 `private` 构造器）；② 客户端从「读数组」变成「调操作」；③ 实现者获得了替换表示的许可。注意第二个版本做了一件在可变类型里绝对不允许的事：**两个 `MyString` 对象共享同一个 `char[]`**。它之所以安全，前提是没有任何 mutator 能改写这个数组——`substring` 只读、`charAt` 只读、`length` 只读。这也回答了原文的提问"为什么没有 mutator 操作？"：一旦加了 `reverse()` 之类的修改者，共享表示会把一个对象的修改传播到所有共享它的对象上，共享优化必须立刻回退成独占拷贝。另外注意原文示例的一个细节：sp21 原文里 `valueOf` 内部写的是 `new MyString()`，依赖 Java 隐式的默认构造器——**那个默认构造器是 public 的**，等于漏了一个 creator。本笔记的正确版本显式声明了 `private MyString() { }` 来堵住它。

【设计原则透视】

这是本讲的核心画面：**操作集合 = 抽象（公开），字段 = 表示（私有）**，二者之间是抽象边界。`valueOf` 是构造者（`boolean → MyString`），`length` / `charAt` 是观察者（`MyString × int → int/char`），`substring` 是生产者（`MyString × int × int → MyString`），**没有修改者**——所以 `MyString` 是不可变类型。测试这个 ADT 时（Reading 03 测试与原文的 testing strategy），分区只能基于抽象状态（字符串长度、`this` 由 `valueOf` 还是 `substring` 产生），绝不能基于 rep 里的 `a.length` 或 `start` / `end`。

场景 4：`String` 与 `StringBuilder`——同一个"字符串"的可变版与不可变版，如何决定共享是否安全

❌ *错误代码*

```
import java.util.ArrayList;
import java.util.List;

/** 错误：把可变的 StringBuilder 当作"值"到处传递与共享。 */
public class TextLog {
    private final StringBuilder buffer = new StringBuilder();   // rep 本身可变

    /** mutator : TextLog x String -> void */
    public void add(String line) {
        buffer.append(line).append('\n');
    }

    /** 错误：把内部可变对象直接发出去 */
    public StringBuilder getBuffer() {
        return buffer;
    }
}

class Client {
    void broken(TextLog log) {
        log.add("first");
        StringBuilder shared = log.getBuffer();
        shared.append("injected\n");        // 别名：隔空改了 log 的内容

        // 把同一个 StringBuilder 交给两个"值"
        StringBuilder sb = new StringBuilder("ab");
        List<StringBuilder> list = new ArrayList<>();
        list.add(sb);                       // list 与 sb 是别名
        sb.append("c");                     // list.get(0) 的内容也变了
        String snapshot = sb.toString();    // 只得手动"快照"才能得到稳定值
    }
}
```

【错误代码的问题】

1. **表示暴露**：`getBuffer()` 返回内部 `StringBuilder` 的引用，客户端一次 `append` 就改掉了 `TextLog` 的内容，`TextLog` 对它自己的状态再也没有任何保证。
2. **别名导致"值"不稳定**：`sb` 与 `list.get(0)` 指向同一个可变对象，`sb.append("c")` 会改变列表里那个元素看到的内容。如果这段代码依赖"我先记下这个字符串，稍后再比对"，结果就会随执行顺序变化——这正是 Reading 08 里可变对象与别名分析的经典陷阱。
3. **共享变得不可能**：因为内容随时可能被改，任何"多处共享同一个 `StringBuilder`"的设计都要附带一整套纪律（谁都不许 append），而这种纪律无法由编译器强制。
4. **可变对象不能安全地"存起来当值用"**：把它放进集合、作为键、或跨方法传递，都需要额外约定；`StringBuilder` 的 `equals` 是引用相等，因此它作为值类型也没有正确的语义（见 Reading 15 相等性）。

✅ *正确代码*

```
import java.util.ArrayList;
import java.util.List;

/** 正确：把不可变的 String 当值用，producer 负责组合出新的 String。 */
public class TextLog {
    private String text = "";             // rep：不可变类型，因此"换值"就是换引用

    /** mutator : TextLog x String -> void */
    public void add(String line) {
        text = text.concat(line).concat("\n");   // String.concat 是 producer：产生新串
    }

    /** observer : TextLog -> String */
    public String getText() {
        return text;                      // 安全：String 不可变，发出去也没人能改
    }

    /** producer : TextLog -> TextLog（可选的不可变风格） */
    public TextLog withLine(String line) {
        TextLog that = new TextLog();
        that.text = this.text.concat(line).concat("\n");
        return that;
    }
}

class Client {
    void correct(TextLog log) {
        log.add("first");
        String snapshot = log.getText();   // snapshot 是稳定值，之后永不变
        log.add("second");                 // 不影响 snapshot
        boolean ok = snapshot.equals("first\n");   // 用 equals 比值（见 Reading 15）

        // 需要大量拼接时，才在"局部"使用 StringBuilder，用完立刻转成 String
        StringBuilder sb = new StringBuilder();
        for (String line : new ArrayList<String>()) {
            sb.append(line).append('\n');
        }
        String built = sb.toString();      // 到边界处立刻"不可变化"，不再外泄 sb
    }
}
```

【为什么这样更好】

`TextLog` 的 rep 换成不可变的 `String`：`add` 用 `concat` **产生新串**并把字段指向它，于是"值"一旦读出（`snapshot`）就永远不会变——客户端可以放心地保存、传递、比较。`getText()` 直接返回内部 `String` 也没有任何风险，因为 `String` 是**不可变类型**：没有 `append`、`setCharAt` 这类 mutator，任何人都改不了它。只有在"需要大量就地拼接"时（`String` 每次 `concat` 都要复制，成本 O(n)）才在**方法内部**使用 `StringBuilder`，并在返回边界上立刻 `toString()` 转成不可变值——把可变性限制在一个尽可能小的作用域里，不让它跨过抽象边界。

【代码对比解说】

两个版本的公开操作几乎一样（`add` + 读取内容），差别在于 **rep 的类型**与**是否有可变状态外泄的通道**。这正好对应本讲原文对类型的分类：`String` 是不可变类型，没有 mutator，所有"看起来会改"的操作（`concat`、`substring`、`toUpperCase`）都是 **producer**；`StringBuilder` 是可变类型，`append`、`reverse` 既是 **mutator** 又返回 `this`（因此也是 producer），而原文特别提醒：`StringBuilder` 是 `String` 的可变版本，但二者**不是同一个 Java 类型，也不能互换**。还有一个容易被忽略的点：不可变带来的收益不只是"安全"，还有"可以自由共享"。`MyString` 的 `substring` 之所以能共享底层字符数组、`String` 之所以能在语言层面被广泛传递，都是因为没人能改它们；而一个可变的 `StringBuilder` 一旦被两个地方持有，就必须靠纪律（而不是靠编译器）来维持正确性。

【设计原则透视】

这是 Reading 08 不可变性在**类型选择**层面的应用：先决定"这个类型的值会不会变"，再决定 rep 用什么类型、观察者能否直接把 rep 发出去。`TextLog` 的 rep 若是可变的 `StringBuilder`，就必须做防御性拷贝（返回 `new StringBuilder(buffer)` 或 `buffer.toString()`）；若换成不可变的 `String`，把 `private` 字段直接返回也完全安全——**不可变性可以替代拷贝**。另外注意 `StringBuilder` 的 `equals` 是引用相等（没有覆写），所以它作为值类型是残缺的，这一点由 Reading 15 相等性展开；而 `String` 的 `equals` 是按内容比较，才让它真正能当"值"用。

场景 5：`Deck` 与 `List`——把通用特征和领域特征混进同一个 ADT 的代价

本讲原文在"设计抽象类型"一节给出一条规则：**类型可以是通用的（generic，如 list / set / graph），也可以是领域特定的（domain-specific，如街道地图、员工数据库、电话簿），但不应该把两者混在一起**——"一个用来表示一副扑克牌的 `Deck` 类型，不应该有接受整数或字符串这类任意对象的通用 `add` 方法；反过来，把 `dealCards` 这种领域特定方法塞进通用的 `List` 里也毫无意义。"原文只给了这条规则，下面两块代码是它的具体化（`Deck` 属于领域特定类型，`List` 属于通用类型）。

❌ *错误代码*

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/** 错误一：领域特定的牌堆，却带了一个接受任意对象的通用 add。 */
public class Deck {
    private final List<Object> cards = new ArrayList<>();   // Object：什么都能塞

    /** 通用 add：签名上不限制类型，"一副牌"里可以混进整数和字符串 */
    public void add(Object item) {
        cards.add(item);
    }

    public void shuffle() {
        Collections.shuffle(cards);
    }

    public int size() { return cards.size(); }
}

/** 错误二：给通用的 List 加领域特定方法 —— 这里用工具类的形式演示同样的错误。 */
class ListUtils {
    /** 对"任意列表"做发牌，但发牌只对牌堆有意义 */
    public static List<Object> dealCards(List<Object> list, int hands) {
        List<Object> result = new ArrayList<>();
        for (int i = 0; i < hands; i++) {
            result.add(list.get(i % list.size()));
        }
        return result;
    }
}

class Client {
    void broken() {
        Deck deck = new Deck();
        deck.add("Ace of spades");   // 字符串，居然合法
        deck.add(42);                // 整数，也合法
        deck.add(new Deck());        // 连牌堆都能塞进牌堆
        deck.shuffle();              // 混着 Integer 和 Deck 的"牌"根本没法洗

        List<Object> notADeck = new ArrayList<>();   // 这是员工名单，不是牌堆
        ListUtils.dealCards(notADeck, 3);            // 却能被"发牌"
    }
}
```

【错误代码的问题】

1. **操作集合失去一致性（coherent）**：`Deck.add(Object)` 在签名上对"牌"没有任何约束，客户端可以往一副牌里塞整数、字符串甚至另一个 `Deck`。类型名承诺的"一副扑克牌"与它实际能表示的值完全脱节。
2. **不变量被架空**：`Deck` 想维护的任何性质（例如"只能有 52 张""每张牌唯一""花色只能是四种"）都无法在 `add(Object)` 这一层检查，因为参数类型里没有足够的信息。

3. **领域操作污染通用类型**：`dealCards` 只对牌堆有意义，却被挂在一个"对任意列表都适用"的位置上。任何看到它的程序员都要先判断"这个列表到底是不是牌堆"，而编译器一个都不拦——`List<Object> 员工名单` 照样能被"发牌"。
4. **表示与语义都难以演化**：因为 `cards` 是 `List<Object>`，将来想换成"按花色分桶"的表示、或想给 `Deck` 加 `sortBySuit()`，都会被"元素可能是任何东西"这个前提卡住。

✅ *正确代码*

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/** 领域特定类型：元素类型固定，操作只谈"牌"这一件事。 */
public final class Deck {
    private final List<String> cards = new ArrayList<>();   // rep：只装"牌"的表示

    /** creator : -> Deck（一副 52 张的标准牌堆） */
    public Deck() {
        for (String suit : new String[] { "clubs", "diamonds", "hearts", "spades" }) {
            for (int rank = 1; rank <= 13; rank++) {
                cards.add(rank + " of " + suit);
            }
        }
    }

    /**
     * mutator : Deck x int -> List<String>
     * @param n 要发出的牌数，要求 0 <= n <= size()
     * @return 发出的 n 张牌（从牌堆顶部取走）
     */
    public List<String> dealCards(int n) {
        if (n < 0 || n > cards.size()) throw new IllegalArgumentException("bad n");
        List<String> hand = new ArrayList<>(cards.subList(0, n));
        cards.subList(0, n).clear();
        return hand;
    }

    /** mutator : Deck -> void */
    public void shuffle() { Collections.shuffle(cards); }

    /** observer : Deck -> int */
    public int size() { return cards.size(); }
}

class Client {
    void correct() {
        Deck deck = new Deck();          // 52 张，类型固定
        // deck.add("whatever");         // 静态错误：Deck 没有通用 add
        List<String> hand = deck.dealCards(5);
        int rest = deck.size();          // 47
    }
}
```

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/** 通用类型：只提供少量能强力组合的简单操作，绝不掺入领域概念。 */
class ListUtils {
    /** producer : List<T> x int -> List<T>（通用：与"牌"毫无关系） */
    public static <T> List<T> take(List<T> list, int n) {
        return new ArrayList<>(list.subList(0, n));
    }

    /** mutator : List<T> -> void */
    public static <T> void sort(List<T> list, java.util.Comparator<? super T> cmp) {
        Collections.sort(list, cmp);
    }
}

class Client2 {
    void correct() {
        // "发牌"由 Deck 提供；"取前 n 个"由通用工具提供，二者互不侵入
        Deck deck = new Deck();
        List<String> hand = ListUtils.take(new ArrayList<>(deck.dealCards(5)), 2);
    }
}
```

【为什么这样更好】

`Deck` 的元素类型被钉死为"牌"，构造器一次性保证"标准 52 张"这个不变量，所有操作都只说牌堆的语言（`dealCards`、`shuffle`、`size`），领域概念不会泄漏到别处。通用的 `ListUtils.take` 则刻意不认识"牌"：它是一个泛型方法，对任何 `List<T>` 都成立，因此放在通用层完全合理。两者的分工恰好对应原文的两句话——`Deck` 不该有通用 `add`，通用 `List` 不该有 `dealCards`。此外 `Deck` 只有 `size` 与 `dealCards` 两个读取入口、一个 `shuffle` 修改入口，操作集合"少而简单"，客户端能做的组合却足够多。

【代码对比解说】

错误版本的问题不在"少写了校验"，而在**类型的粒度选错了**：`Deck` 被写成了一个"可以装任何东西的容器"，于是它不再是一个 ADT，只是一个恰好叫 `Deck` 的 `ArrayList`。正确版本把"牌"这个领域概念固化进类型，把"取前 n 个元素"这类真正通用的能力留在通用层，于是每一层都只有一个 concern（separation of concerns）。这也顺带解决了另一类隐患：`Deck.add(Object)` 让 `Deck` 的规格无法写清（"能加什么？"答不上来），而 `Deck.dealCards(int)` 的规格一句话就写完（前置条件 `0 <= n <= size()`，后置条件"返回取走的牌、牌堆少 n 张"）。判断一个方法该挂在哪里，有个简单测试：**把类型名遮住，这个方法还讲得通吗？** `take(list, n)` 遮住类型名依然成立；`dealCards(list, n)` 就不成立了——它偷偷假设了 list 是牌堆。

【设计原则透视】

这一组对比把本讲"设计抽象类型"的四条法则全部串了起来：① 少而简单（`Deck` 只有三个操作）；② 行为一致、无特例（`dealCards` 不需要"如果元素不是牌该怎么办"这种分支）；③ 操作集合足够（`size` + `dealCards` + `shuffle` 足够表达客户端要做的计算）；④ **通用与领域特定不混**。它同时是"表示独立"的前

提：只有当操作集合语义清晰，实现者才可能把 `List<String>` 换成"四个花色桶"甚至"一个 52 位掩码"而不影响客户端。这一点会在 Reading 12（接口与泛型）里以更形式化的方式出现——泛型参数 `T` 正是"通用 ADT"在类型系统里的写法，而领域特定类型则把它固定成具体的元素类型。

与其他设计原则的关联

- **与 Reading 06（规格说明 Specifications）**：ADT 的规格就是「操作集合 + 每个操作的前置条件与后置条件」。本讲反复强调，只有当操作被**完整规格化**时，实现者才敢更换表示——因此 Reading 06 的规格写作能力是表示独立的前置技能。
- **与 Reading 07（设计规格 Designing Specifications）**：规格的强弱、确定性、声明式写法决定了 ADT 留给实现者多少自由。规格写得过强（例如把"内部用数组"写进后置条件）会直接摧毁表示独立；写得过弱则客户端不敢依赖，抽象边界名存实亡。
- **与 Reading 08（不可变性 Immutability）**：本讲把 Reading 08 的"可变 vs 不可变""别名""防御性拷贝"从**单个对象**提升到**类型层面**：可变类型与不可变类型各自需要什么样的字段可见性、是否必须拷贝、能否共享内部数据（`MyString.substring` 的共享数组优化）。
- **与 Reading 09（避免调试 Avoiding Debugging）**：`private` 让越界访问变成编译期错误，这就是 fail fast 在类型边界上的版本；`Wallet` 例子里那几个"静态错误"正是这种保护的直接体现。
- **与 Reading 11（抽象函数与表示不变量 Abstraction Functions & Rep Invariants）**：本讲说"要有 rep 且必须是 private"，Reading 11 接着回答"rep 与抽象值之间如何精确对应"——抽象函数（AF）、表示不变量（RI）、以及如何系统地排查表示暴露。两讲合起来才是完整的 ADT 设计方法。
- **与 Reading 12（接口与泛型，兼枚举 Interfaces, Generics, Enums）**：`List` + `ArrayList` 展示了「接口声明操作、实现类提供表示」的结构，这是表示独立在语言层面的最强形式；泛型让 ADT 从"字符串的列表"变成"任意类型的列表"，也正是本讲 `Deck`（领域特定）与 `List`（通用）分工在类型系统里的写法；`enum` 则为小固定值集合（如原文举的 `DayOfWeek`）提供了一种现成的 ADT 实现方式。
- **与 Reading 15（相等性 Equality）**：原文明确指出，在 `MyString` 还没有定义 `equals` 之前，测试里不能直接 `assertEquals` 两个 `MyString` 对象。相等性与哈希本身也是 ADT 的观察者/操作，必须按规格谨慎实现。
- **与 Reading 17（递归数据类型 Recursive Data Types）**：ADT 的思想在递归数据类型里进一步升级为数据类型定义，例如 `ImList<E> = Empty + Cons(first:E, rest:ImList<E>)`——一个抽象类型由若干具体表示（`Empty` 与 `Cons`）共同实现，操作则按表示递归定义。
- **与 Reading 21 / 23（并发 Concurrency / 互斥 Mutual Exclusion）**：不可变 ADT 天然线程安全，是并发编程里最省心的共享方式；可变 ADT 一旦被多线程共享，就必须靠锁把 mutator 保护起来。

关键要点

- **用操作定义类型**：写一个类型时先写出它的**操作集合与每个操作的规格**；这套东西就是类型的全部含义，字段只是"若干种可能实现之一"。
- **字段一律 `private`，只有构成抽象的操作才 `public`**；同时检查构造器可见性——不写构造器时 Java 会补一个 public 默认构造器，等于凭空开放了一个 creator。

- **把每个操作归类**（creator / producer / observer / mutator），并用分类表核对：creator 的输入不含 `T`、输出是 `T`；producer 吃 `T` 返回新 `T`；observer 吃 `T` 返回别的类型；mutator 修改对象（返回值可以是 `void`、其他类型甚至 `T`）。别忘了实例方法的隐式 `this`。
- **可变类型必须做防御性拷贝**（构造器入口 + 观察者出口，两个方向都要），并且在加入 mutator 后重新审视所有"共享内部数据"的优化；**不可变类型**则可以把 `final` 字段直接共享出去。
- **先定规格，再改表示**：只要操作的规格不变，rep 就可以整体替换（`char[]` ↔ `char[] + start/end`、`List` ↔ `Set`）；反过来，只要客户端依赖了 rep，"改实现"就会变成"改所有客户端"。

常见陷阱与注意事项

- **把字段写成 `public` 图方便** → 表示暴露：客户端直接读写 rep，不可变类型的不可变性当场失效，而且从此**无法再更换表示**（原文的 `Wallet.amount`、Family 的 `client1` 直接读 `f.people`、`MyString.a` 都是这个坑）。
- **观察者直接返回内部的可变对象，或以为 `final` 就等于不可变** → 别名攻击：`getTimestamp()` 返回内部 `Date` 后，客户端一次 `setHours` 就改掉了对象内部状态；而 `private final Date timestamp` 也只保护"引用不被重新赋值"，`Date` 里的毫秒值照样能被 `setTime` 改掉。只在一侧做拷贝（只在构造器、或只在观察者）都不够，**必须双向**。
- **把隐式默认构造器当成"没有构造器"** → 你其实开放了一个 public 的 creator：客户端可以 `new MyString()` 造出 rep 未初始化（字段为 `null`）的坏对象。要限制创建途径，就把构造器显式声明为 `private`，只用工厂方法对外。
- **客户端依赖 rep 但仍能编译** → 最危险的坏味道：如 `f.getMembers().get(0)` 依赖返回顺序、`s.a[0]` 依赖内部是数组。这类依赖既不给静态错误也不给动态错误，只在某次实现调整后**静默给出错误答案**。判断标准很简单：客户端代码里出现任何 rep 的类型或顺序假设，就是越界。
- **测试时按 rep 分区** → 测试与实现死死绑定：按 rep 数组的长度、rep 里的 `start`/`end` 分区，会让"换表示就换测试"，一组本可用于验证表示独立的测试全部作废。应当按**抽象状态**（抽象长度、`this` 由哪个 creator 产生）分区。
- **把通用特征与领域特征混在同一个 ADT 里** → 操作集合失去一致性：给 `List` 加 `sum`（对字符串列表、嵌套列表无从定义），或给 `Deck` 加接受任意 `Object` 的 `add`（牌堆里混进整数）。正确做法是另建专门的类型，或让客户端用简单操作组合出所需计算。

思考题（带答案）

问题 1：原文说"每个操作应该有明确的目的、行为一致（coherent），而不是一堆特例"，并举例说**不应该给 `List` 加一个 `sum` 操作**。请解释为什么 `sum` 会破坏 `List` 的一致性；并说明如果一个真实项目确实需要"把一串数字加起来"，按本讲的设计法则应该怎么做。

答案：`List` 是一个**通用（generic）**的 ADT：它的操作集合必须对所有可能的元素类型都有意义（`get`、`size`、`add`、`remove`……）。而 `sum` 只在"元素是可相加的数"时才有定义：`List<String>` 上没有意义；`List<List<Integer>>` 上要么无意义、要么需要额外规则（是"展平后求和"还是"元素求和"？）；元素是自定义类型时又需要某种累加协议。于是 `sum` 的规格里必然出现大量特例，客户端必须先判断"这个列表能不能 sum"，每个特例都增加理解成本。本讲的四条法则正好逐条命中：① "少而简单 > 多而复杂"——`sum`

是一个能用 `get` + 循环组合出来的复合操作；② "每个操作行为一致"，`sum` 做不到；③ "通用与领域特定不要混"，`sum` 是领域特征入侵通用类型；④ "操作集合要足够（adequate）"，`List` 已有的 `get` / `size` 已经足够表达求和的全部信息，所以撤掉 `sum` 不会让客户端"做不到某事"。正确做法是：让客户端用 `get` + 循环组合（这也是"少量简单操作可以强力组合"的体现），或者定义一个**领域特定类型**，让"求和"成为它自己领域内的操作（原文举的 `Deck` 就是这种"只谈自己领域概念、不掺通用特征"的类型）。反过来，把 `dealCards` 塞进通用的 `List` 是同一个错误的镜像。

问题 2：原文给出一个 ADT `Bool`，操作为 `true : Bool`、`false : Bool`、`and : Bool × Bool → Bool`、`or : Bool × Bool → Bool`、`not : Bool → Bool`，其规格就是这三个运算的常规真值表。下列五种实现方式中，哪些**能够**满足这些操作的规格？(a) 用一个比特，`1` 表示 true、`0` 表示 false；(b) 用一个数值，`5` 表示 true、`8` 表示 false；(c) 用一个字符串引用，`"false"` 表示 true、`"true"` 表示 false；(d) 用一个数值，**所有**取值都表示 true；(e) 用一个大于 1 的整数，素数表示 true、合数表示 false。

答案：(a)、(b)、(c) 都可以；(d)、(e) 不可以。

推理依据是"ADT 由操作与规格定义，与表示无关"这一核心思想：表示长什么样、编码取名多奇怪，都不影响它是否合法，唯一的标准是**这些操作能否满足规格**。

- (a) 可以。这是最自然的编码：`and` 用按位与、`or` 用按位或、`not` 用取反，真值表逐条成立。
- (b) 可以。只要把三个操作按 `5 = true`、`8 = false` 重新映射即可：`not(5) = 8`、`not(8) = 5`、`and(5,5) = 5`、`and(5,8) = 8` 等等。表示的具体数值完全不进入规格。
- (c) 可以。这一点最能说明"表示与语义无关"：编码字符串的字面意思（`"true"` 竟然表示 false）纯粹是内部约定，客户端永远看不到它，只看得到 `and` / `or` / `not` 的行为。
- (d) 不可以。规格要求 `not(true) = false`，也就是要求"取反后的结果与原来的值不同"；如果所有值都表示 true，那么 `true` 与 `false` 实际是同一个值，`not` 不可能把 true 变成 false。这说明 `Bool` 的规格蕴含了"类型至少有**两个不同**的值"。
- (e) 不可以。虽然"素数 / 合数"提供了一个二值划分，但 `and(true, true)` 要求值为 true，而两个素数之积必然是**合数**，按该编码 `and` 会返回 false，与真值表冲突。可见能否满足规格，取决于**编码是否与所有操作相容**，而不取决于编码本身是否"看起来合理"。

这题的实际意义是：当你要替换一个 ADT 的实现时，唯一需要检查的是"新表示能否满足全部操作的规格"——这正是"表示独立"的另一面。

问题 3：考虑原文的 Family ADT（成员构成一个 `List<Person>`，`getMembers` 返回全部成员、无重复）。(1) 原文的 `client1`（`f.people.get(f.people.size()-1)`）、`client2`（`f.people.size()`）、`client3`（`f.getMembers().get(0)`）在把 rep 从 `List` 改成 `Set` 之后分别会怎样？(2) 如果把 `getMembers` 改成 `return new ArrayList<>(people);`，是否就完全没有表示暴露了？请说明理由。

答案：

(1) 三者结果截然不同，正好构成"依赖表示"的三级后果： – `client1` 直接读写 `f.people`，把 rep 当作 `List` 使用。改成 `Set` 后 `Set` 没有 `get(int)` 方法 → **静态错误**（编译失败）。这类依赖至少是响亮的。 – `client2` 也直接访问 `f.people.size()`。`Set` 同样有 `size()`，看起来没问题——但它依赖的是**rep 的类型**：改动后如果换成一个没有 `size()` 的表示（例如换成两个字段 `nuclear` + `others`），立刻静态错误；即便这次侥幸编译通过，它的合法性完全靠"新表示恰好也有同名方法"来支撑，这是运气而非设计。原文

把这一类归为"依赖表示、且**没有**被抽象边界保护"的情形。 – `client3` 走的是公开操作 `f.getMembers()`，改动后**仍然编译通过、不抛异常**，但结果可能不同：如果规格只承诺"包含所有成员、无重复"而未规定顺序，那么新实现返回的顺序变了，`get(0)` 取到的就不是原来那个人。这就是最危险的一类——**静默地给出错误答案**。它提醒我们：表示独立不仅仅是"不要读 private 字段"，还包括"不要依赖规格没有承诺的任何可观察性质"。

(2) 不够。返回 `new ArrayList<>(people)` 确实堵住了**一个**方向：客户端再也无法通过返回的列表改动 `Family` 的成员**结构**（不能 `clear()`、`add()`、`sort()`）。但它是**浅拷贝**：拷贝的只是"引用的序列"，`Person` 对象本身仍然是同一批对象。于是：① 如果 `Person` 是可变类型，客户端可以 `getMembers().get(0).setName("X")`，照样改掉 `Family` rep 里那个 `Person` 的内容，不变量（例如"按年龄排序"若依赖可变字段）被破坏；② 如果 rep 里存的是可变容器（例如 `List<Person>` 之外还有别的可变对象），浅拷贝同样挡不住。所以完整的做法是"两选一"：让元素类型**不可变**（`Person` 用 `private final` 字段、不提供 mutator），这样共享引用完全安全，浅拷贝足够；或者对可变元素做**深拷贝**（成本高、易漏，还要递归处理嵌套可变对象），并明确记录在规格里。此外还要注意：即使拷贝做全了，"返回顺序"这类性质仍需在规格里明确承诺，否则 `client3` 依然可能踩到静默错误。

Reading 11: 抽象函数与表示不变量（Abstraction Functions & Rep Invariants）

> 说明：本讲 sp22 原版使用 TypeScript，本笔记按用户要求提供 Java 代码示例；类型/API 与 sp21（6.031 Java 版）原文保持一致。

概述

本讲要回答一个此前一直被绕开的问题：当我们说一个类「实现」了某个抽象数据类型（abstract data type, ADT）时，究竟是什么意思？答案由两个数学对象给出：**抽象函数（abstraction function, AF）** 把每个合法的表示值映射到它所代表的抽象值，**表示不变量（representation invariant, RI）** 划出哪些表示值是合法的表示值；再补上一条纪律——**绝不暴露表示（rep exposure）**——三者合起来才使一个 ADT 真正「守护自己的不变量」。这三个概念是整个 6.031 最实用的一副理论工具：它让你在 Reading 15（相等性）里用抽象值而非表示值定义 `equals()`，让你在 Reading 13（调试）里把缺陷隔离在类内部。它与三大目标的关系是：**Safe from bugs**（不变量被 `checkRep()` 在运行时断言，数据结构的损坏当场暴露，而不是继续传播）、**Easy to understand**（AF/RI 注释把「表示如何被解释」写成可读的事实）、**Ready for change**（抽象与表示分离，替换表示不需要动任何客户端代码）。

核心概念与设计原则详解

不变量（Invariant） – **定义与目的**：不变量是程序的一个性质，在程序的**每一个可能的运行时状态**下都成立。它是「让代码可以被推理」的基本单位：只要你能依赖某个不变量，就不必再去检查与之矛盾的可能性。好的 ADT 之所以有价值，最核心的一条就是它会**保持自己的不变量**。 – **直观解释（"它是什么？"）**：想象一间图书馆，只要规则「书永远按索书号排列」始终为真，你就可以闭着眼睛用二分查找找书；一旦这条规则偶尔被打破，你就必须每次从头扫描。不变量就是这种「可以放心依赖的事实」。对对象而言，「始终为真」精确

地收缩为**该对象的整个生命周期**：从构造完成的那一刻起，直到它被回收。不变量的例子包括：变量的类型（`int i` 意味着 `i` 永远是整数）、变量之间的关系（用一个下标 `i` 遍历数组时，循环体内 `0 <= i && i < a.length` 是不变量）、以及我们已经见过的**不可变性**（不可变对象一旦创建，永远表示同一个值）。 – **关键规则与最佳实践**： – **把不变量所涉及的变量隐藏或保护起来**：用 `private` 修饰字段，只通过具有明确契约的操作访问——「守护自己的不变量」意味着责任在 ADT 自己身上，而不在客户端。 – **优先使用能写下强不变量的类型**：`String` 不可变，因此任何持有 `String` 的 ADT 都不必担心它被外人改掉；而想象一个可变的字符串类型，任何拿到它引用的代码都能修改它，于是「推理」退化为「必须检查程序中所有可能接触到它的地方」。 – **不变量必须被确立，也必须被保持**：只要有一个操作破坏它，整座大厦就塌了，所以每个构造者/生产者/修改者都要对不变量负责。 – **把不变量写成可执行的断言**，而不只是注释；注释会过期，`assert` 会当场喊出来。 – 注意：**禁止 `null` 是默认约定**——RI 隐含地对表示中每个对象引用 `x` 要求 `x != null`，因此不必在 RI 注释里重复写。

表示不变量（Representation Invariant, RI） – **定义与目的**：RI 是一个从表示值到布尔值的函数 `RI : R → boolean`。`RI(r)` 为真，当且仅当 `r` 是一个**合法（良构，well-formed）**的表示值，也即 `r` 位于抽象函数有定义的范围之内。等价的说法是：把 RI 看作表示值空间的一个子集——那些能映射到抽象值的表示值构成的子集。它的作用是**把「内部状态在什么条件下才有意义」这一隐含知识显式化**，并让损坏的数据结构尽早被抓住。 – **直观解释（"它是什么？"）**：RI 就是这份表示空间的「入场券」规则，或者说表示值空间里的绿灯区与红灯区。以用字符串表示字符集合 `CharSet` 为例，若规定字符串中不得出现重复字符，则 `RI("a") = true`、`RI("ac") = true`、`RI("acb") = true`，而 `RI("aa") = false`、`RI("abbc") = false`。绿灯区里的表示值一定映射到某个抽象值；红灯区里的表示值没有对应的抽象值，因为它们根本不是合法状态。这条「不许重复」的规则并非无用的洁癖：它让 `remove` 在遇到该字符的第一个实例时就可以收工，因为至多只有一个。 – **关键规则与最佳实践**： – **RI 是「字段值的合法条件」，不是「抽象值的性质」**：作为函数，把实际的字段值（合法或非法都行）代入文档化的 RI，必须得到一个布尔值。一旦你在 RI 里提到抽象值，就是本末倒置了——非法的表示值根本没有抽象值，RI 必须只谈表示本身。 – **RI 不能是空泛的话**：像「所有字段都有效」这样的 RI 毫无价值；RI 的职责是精确说明什么样的字段值组合是合法的。 – **同一个表示值空间可以有不同 RI**：同样用 `String` 表示字符集合，可以选择「无重复」的 RI，也可以选择「字符按非降序排列（允许重复）」的 RI，后者允许对字符串做二分查找，把 `contains` 从线性时间降到对数时间。**表示值的类型选择决定不了 RI**，这是 AF/RI 不是「冗余信息」的关键理由之一。 – **RI 写在表示（私有字段）声明的旁边**，用普通注释而不是 Javadoc 注释——写成 Javadoc 就等于把它当成公开规格的一部分，会破坏表示独立性与信息隐藏。 – **让 `checkRep()` 与 RI 一一对应**：RI 写了什么，`checkRep()` 就该断言什么（包括隐含的 `!= null`）。

抽象函数（Abstraction Function, AF） – **定义与目的**：AF 是一个从表示值到抽象值的映射 `AF : R → A`，说明「这堆具体字段值，被解释成哪个抽象值」。它使你能够在纸面上、也在代码里精确定义这个类型**是什么**，而不是它是怎么存的。AF 是 Reading 15 中为不可变类型定义 `equals()` 的基础，也是理解「表示独立」的钥匙。 – **直观解释（"它是什么？"）**：把表示值想成密码本里的一行密文，AF 就是解码规则。以 `CharSet` 中 `AF(s) = {s[i] | 0 <= i < s.length()}` 为例，代入一个合法表示值 `s="abbc"` 得到 `AF("abbc") = { "abbc"[i] | 0 <= i < "abbc".length() } = {'a','b','c'}`——注意右边**真的随着代**

入而计算出了唯一的结果。反例是含糊的 AF：`AF(s) = 一个字符集合`，代入 `s` 后右边毫无变化，这种 AF 完全没有说明 `"abbc"` 到底代表哪个集合，等于没写。 – **关键规则与最佳实践**： – **AF 只对合法表示值有定义**：`RI(r)` 为真 ⟺ `r` 被 AF 映射；非法表示值不在 AF 的定义域内。 – **AF 描述「表示值 → 抽象值」，且必须能代入求值**：AF 的写法应当像数学函数，等号右边要出现字段名。 – **AF 不是由两个值空间唯一决定的**：同一个抽象值空间可以有多种表示（字符集合既可以用字符串表示，也可以用一个位向量表示，每个可能字符占一位——显然需要两个不同的 AF）；即使表示值空间相同、RI 也相同，仍可以有不同 AF。例如对同样的「任意字符串」RI，我们可以把字符串解释为集合的元素，也可以把**相邻字符两两成对**解释为区间：`"acgg"` 被解释为 `[a-c]` 与 `[g-g]` 两个区间，代表集合 `{a,b,c,g}`，此时 RI 变为「`s.length()` 是偶数，且字符非降序」。 – **实现一个抽象类型，意味着三件事都要选**：抽象值空间（规格用）、表示值空间（实现用）、**哪些表示值合法（RI）以及如何解释它们（AF）**。 – **把 AF/RI 写进代码**：如果不同的实现者对表示的含义有分歧，这份表示就不再可靠；写下来是唯一可靠的沟通方式。

满射但不必单射（Surjective, Not Necessarily Injective） – **定义与目的**：用函数的术语精确描述 AF 的性质：它是**满射（surjective，也称 onto）**——每个抽象值都被某个表示值映射到；**不一定单射（injective，一对一）**，因此不一定双射（bijective）；并且**常常是部分函数（partial）**——不是所有表示值都被映射。这个性质解释了「同一抽象值可以有多种表示」，而这正是让 `equals()` 必须比较抽象值的根源。 – **直观解释（"它是什么？"）**：满射意味着「你想造的任何抽象值，我都造得出来」——实现抽象类型的目的就是支持对抽象值的操作，所以所有抽象值都必须可表示。不满射的实现等于有些合法值永远造不出来，是设计缺陷。不单射意味着「多个密码可以解出同一段明文」，例如把无序字符集合存成字符串时，`"abc"`、`"bca"`、`"cab"` 都可以表示同一个集合 `{a,b,c}`；这就是「编码不紧致（not a tight encoding）」。部分函数意味着「有些密码是废码」，例如 `"abbc"` 在「不许重复」的 RI 下没有意义。用有理数 `RatNum` 看得更清楚：在「分母为正且已约分」的 RI 下，`(1,2)` 是合法表示，`(2,4)` 落在红灯区（可用但被 RI 禁止），而 `(4,2)` 与 `(2,1)` 都映射到整数 2——不单射带来的「同一抽象值、多种表示」正是 `2/4` 与 `1/2` 这类相等性问题的根源。**本讲原文用 `CharSet` 演示了 AF 的这两条要求**：`AF(s) = {s[i] | 0 <= i < s.length()}` 是一个合格的 AF，因为代入合法表示值 `s = "abbc"` 可以直接算出 `AF("abbc") = {"abbc"[i] | 0 <= i < "abbc".length()} = {'a','b','c'}`；而含糊的 `AF(s) = a set of characters` 代入 `s` 后右边**毫无变化**，完全说不出 `"abbc"` 到底对应哪个集合，因此毫无信息量。（`Duration(1,2)` 与 `Duration(0,62)` 是同一现象的另一个例子，该示例出自 Reading 15（相等性），此处借用说明同一概念。） – **关键规则与最佳实践**： – **检查满射性**：每个抽象值都必须有表示值映到它，否则存在客户端「要不到」的合法值。 – **接受不单射，但要一致地处理它**：既然一个抽象值可以有多个表示，任何「比较两个对象」的操作都必须比较**抽象值**，绝不能比较字段。 – **窄化 RI 会改变合法表示集合、影响性能取舍**：`RatNum` 若采用更强的 RI（要求约分到最简），每次运算都要做 `gcd`；若采用更弱的 RI（只要求分母非零），连续运算可以免去约分，直到需要显示结果时再化简——两种设计取舍不同，但抽象值空间完全相同。 – **让 AF 保持「可判定的解释」**：AF 必须能机械地代入求值；不要写需要「意会」的 AF。 – **如果 RI 太严，考虑它是否真的必要**：过强的 RI 会让实现处处受限，过弱的 RI 会让不变量失去保护力，选择 RI 是一种设计权衡。

表示暴露（Representation Exposure）与防御性拷贝（Defensive Copying） – **定义与目的**：表示暴露指**类外部的代码能够直接修改（或至少直接取得）类的表示**。它同时威胁两件事：不变量（外部代码可以在

ADT 的任何一个操作之外改坏内部状态）和表示独立性（客户端会依赖具体的表示，于是你再也换不动实现）。防御性拷贝是修补手段：**拷贝一份可变对象，避免把表示中的引用泄漏出去。** – **直观解释（"它是什么？"）**：别名（aliasing）是核心机制：当两个变量指向同一个可变对象时，通过其中一个改，另一个也变了。`Tweet` 的例子最能说明问题——`getTimestamp()` 返回的 `Date` 与 `t.timestamp` 是同一个对象，于是这段「完全合理」的客户端代码 `Date d = t.getTimestamp(); d.setHours(d.getHours()+1);` 顺手把 `t` 里的时间也改了，`Tweet` 的不可变性轰然倒塌。反方向的泄漏同样是 bug：`Tweet` 的构造器直接把传入的 `Date` 存进表示，于是下面这段想造 24 条推文的循环，因为反复修改同一个 `Date` 对象，最终 24 个 `Tweet` 的时间戳全都一样。要养成一个习惯：**审视所有操作的参数类型与返回类型，只要其中有可变类型，就确认实现既没有把参数直接存进表示，也没有返回表示内部的直接引用。** – **直观解释续（为什么不用规格来免责）**：可以写「调用者此后绝不能再修改这个 `Date` 对象！」，在某些别无选择时（例如可变对象太大，拷贝代价高）确实会这么做，但它对「推理程序」和「避免 bug」的代价极大。除非有压倒性的理由，值得让 ADT 自己保证不变量，而**杜绝表示暴露是其中的必要条件。** – **本讲原文给出的三个代表性反例**： – **`RightTriangle`（表示暴露的经典反例）**：它的表示是 `private double[] sides;` 外加一个公开字段 `public final double hypotenuse;`。`getAllSides()` 的实现是 `return sides;`——直接把内部数组交给客户端，于是客户端可以随意改写边长（`/*E*/` 处的问题），而这个类型声称为「不可变的直角三角形」。此外构造器里的 `/*D*/` 直接把参数写进表示而没有做防御性拷贝，`/*B*/` 那个公开字段还让客户端依赖上了表示，破坏表示独立性。（原文的 `regularize()` 里把 `sides[0]` 误写成 `side[0]`，是一个原样保留的笔误；本笔记的示例代码中已改正为 `sides[0]` 以便编译。） – **`Identity[] getSigners()`（安全相关的表示暴露）**：设想一个只允许「已被可信身份签名」的类，它用自己的 `private Identity[]` 字段保存校验过的签名，而 `getSigners()` 直接把这个数组返回。客户端于是可以往数组里追加身份、把已有身份替换成别的身份，从而破坏一个安全相关的不变量——数组是可变类型，把它交出去等于把安全边界交出去。 – **`Date`（可变类型的对照）**：`java.util.Date` 是可变的，所以只要它出现在表示里，就必须在参数与返回值两处分别防御性拷贝；课程同时提醒，Java API 文档已把 `Date` 的大部分方法标为 deprecated，新代码不应使用它——换用不可变的 `java.time.ZonedDateTime` 才是根治。 – **关键规则与最佳实践**： – **构造器做入参防御性拷贝**：`this.timestamp = new Date(timestamp.getTime());`——在输入关口复制，把外部世界与你隔离。 – **观察者做出参防御性拷贝**：`return new Date(timestamp.getTime());`——在输出关口复制，防止客户端反手改你的内部状态。 – **返回新对象或不可变视图**：能返回 `String`、`Integer` 或不可变类型就不要返回可变容器；必须返回集合时考虑拷贝或不可变包装。 – **优先选择不可变类型**：如果日期用的是不可变的 `java.time.ZonedDateTime` 而不是可变的 `java.util.Date`，那么讲完 `private` / `public` 这一节就可以结束了——**不可能再有表示暴露**。这是最省心的方案。 – **防止可变对象被多个对象共享**：把客户端传入的数组、`Map`、`List` 直接存进表示，等于把表示的一部分放在客户端手里。一个同类型的例子见下面的 `Matrix`（**该示例出自 Reading 19（Programming with ADTs），不属于 Reading 11 原文，此处借用来说明表示暴露与深拷贝**）。

补充说明：另一个可变数组表示的示例 —— `Matrix`（出自 Reading 19，非本讲原文）。课程在后续的 Reading 19（Programming with ADTs）中给出了一个用二维数组当表示的矩阵类型，它的 AF/RI 与「构造器必须做防御性拷贝」这条纪律合在一起看，正好补全本讲的图景（原文在该构造器处留下了注释 `// note: danger!`）：

```
class Matrix implements MatrixExpression {
    private final double[][] array;

    // Rep invariant:
    //   array.length > 0，且所有 array[i] 的长度相同且非零
    // Abstraction function:
    //   AF(array) = 具有 array.length 行、array[0].length 列的矩阵，
    //   其 (row, column) 元素为 array[row][column]
    // Safety from rep exposure:
    //   字段 private final；但 double[][] 是可变类型，
    //   因此构造器必须做深拷贝（逐行复制），且不向任何操作返回内部数组。

    public Matrix(double[][] array) {
        // 课程原文此处只有 this.array = array; // note: danger!
        // 正确做法：深拷贝，切断与调用者之间的别名
        this.array = new double[array.length][];
        for (int row = 0; row < array.length; row++) {
            this.array[row] = array[row].clone();
        }
        checkRep();
    }

    /** @return 行数 */
    public int rows() {
        checkRep();
        return array.length;
    }

    /** @return 第 row 行第 col 列的元素 */
    public double get(int row, int col) {
        checkRep();
        return array[row][col];       // double 是原始类型，返回副本，不存在别名
    }

    private void checkRep() {
        assert array != null;
        assert array.length > 0;
        assert array[0] != null && array[0].length > 0;
        for (double[] row : array) {
            assert row != null && row.length == array[0].length;
        }
    }
}
```

注意三个要点：第一，**数组是可变类型**，所以「字段私有 + `final`」完全不够——`final` 只锁住 `array` 这个引用，锁不住数组里的元素，也锁不住客户端手里那份原始二维数组的引用，因此构造器必须做**深拷贝**（只复制外层数组是不够的，内层数组仍被共享）。第二，`checkRep()` 直接对应 RI 的每一条，包括「所有行长相同且非零」这条跨行关系——RI 常常是**字段之间的关系**，而不只是单个字段的性质。第三，返回 `double` 这类原始类型不会产生别名，是整个类里唯一不需要防御性拷贝的访问路径。另外，若某个字段本

身不该被客户端看到（例如缓存、索引），把它设为 `private` 并在 Safety 论证中单独说明其不可见性即可。

不可变包装器（Immutable Wrappers） – **定义与目的**：Java 集合库提供了一个折中方案：`Collections.unmodifiableList()`、`unmodifiableMap()`、`unmodifiableSet()` 等把可变的集合包装成一个「看起来一样、但所有修改者都抛异常」的对象。你可以用修改者把集合建好，然后用不可变包装把它封起来（并按 Reading 08 的建议丢掉对原始可变集合的引用），从而得到一个不可变的集合视图。 – **直观解释（"它是什么？"）**：像是给家里的电闸装了个只能看的玻璃罩：你还能读表，但伸不进去动手。它的代价是——**只有运行时不可变，没有编译期不可变**：编译期不会警告你调用 `sort()`，你只会在运行时拿到一个异常；而且如果谁还留着原始可变集合的引用并改了它，「不可变」视图的内容会随之改变，而且不会有任何报错。 – **关键规则与最佳实践**： – 用不可变包装**降低 bug 风险**，但别把它当成类型系统级别的保证。 – **包装之后立刻丢弃原始可变引用**，否则「不可变」只是假象。 – 返回集合的观察者中，`List.of(...)`、`Collections.emptyList()` 这类工厂也有同样的「仅运行时不可变」局限。 – 若需要编译期保证，请自定义不可变类型（如 `SortedCharSet`），或者使用库提供的真正不可变类型。 – **在 Safety from rep exposure 注释里如实写明用的是哪种机制**（拷贝 / 不可变包装 / 本身不可变），因为论证的说服力取决于机制的强度。

AF / RI / Safety 三件套注释（Documenting AF, RI, and Safety from Rep Exposure） – **定义与目的**：6.031 要求每个有实质表示的 ADT 在**表示（私有字段）声明旁边**写下三段注释：抽象函数、表示不变量、以及**免于表示暴露的安全性论证（safety from rep exposure）**。它们分别回答三个问题：这个表示被如何解释？什么样的表示是合法的？为什么客户端拿不到可变的内部表示？三者缺一不可——只写 RI 不写 Safety，你可能在「表示合法」的同时把它泄漏出去；只写 AF 不写 RI，`checkRep()` 无从下手；只写 Safety 不写 AF/RI，读者根本不知道你在保护什么。 – **直观解释（"它是什么？"）**：这是一份写给「未来的维护者」（包括三个月后的你自己）的**表示使用说明书**。它不是在描述类型做什么（那是公开规格的事），而是在描述实现内部约定，因此必须写在类体内部的普通注释里，而不是类上面的 Javadoc——写成 Javadoc 等于把它公开承诺为规格的一部分，会破坏表示独立性与信息隐藏。 – **关键规则与最佳实践**： – **AF 行写成 `AF(字段...) = 抽象值表达式`**，等号右边必须出现字段名，可代入求值。 – **RI 行写成对字段的断言式条件**，每条都应当能直接翻译成 `assert`。 – **Safety 行逐个字段交代**：字段是否 `private`；类型本身是否不可变；若可变，在哪里做了防御性拷贝或不可变包装；参数与返回值是否可能泄漏。注意 `Tweet` 的 `timestamp` 没有额外的 RI 条件，但它**仍然必须出现在 Safety 论证里**，因为整个类型的不可变性依赖于所有字段都不被改动。 – **写不清楚意味着什么**：如果你写不出精确的 AF/RI，通常说明你对这个表示的语义自己也还没想清楚，或者表示设计本身就有问题；含糊的注释会让不同实现者产生分歧（本讲的 `CharSet` 练习「Trying to implement without an AF/RI」正是展示这种灾难：Louis Reasoner 没写下 AF/RI，于是三位队友各自揣着 `SortedRep`、`SortedRangeRep`、`NoRepeatsRep`、`AnyRep` 四种不同理解去实现 `add()` / `remove()` / `contains()`，结果每种实现只对其中一部分 AF/RI 成立）。 – **注释与代码要同步**：修改表示时，AF/RI/Safety 三段注释是改动清单的第一项。

标准模板（必须完整书写，三行式注释缺一不可）：

```
public class SomeType {

    private final FieldType field1;   // 表示（rep）：私有字段
    private final OtherType field2;

    /**
     * ...
     * Abstraction function:
     *   AF(r) = ...
     * Representation invariant:
     *   ...
     * Safety from rep exposure:
     *   ...
     */
}
```

逐行解释：

- `Abstraction function:` 之后的 `AF(r) = ...` 描述**表示值 → 抽象值**的映射，`r` 应当被写成本类的字段（或一个元组），等号右边必须真的用到这些字段，且能对具体表示值代入求值，最终得到唯一的抽象值。
- `Representation invariant:` 之后描述**合法表示值的集合**：把任意字段取值代入，应当得到一个明确的真/假判断；只有为真的表示值才对应抽象值。它也是 `checkRep()` 里 `assert` 的逐条来源。
- `Safety from rep exposure:` 之后描述**为什么客户端拿不到可变内部表示的别名**：逐个字段说明可见性与可变性，并指出在哪些环节（构造器入参、观察者返回值）做了防御性拷贝或不可变包装。
- 上面三点合起来是「表示的完整语义」：**解释（AF）+ 合法性（RI）+ 隔离（Safety）**。少了任何一条，ADT 都不能算「守护住了自己的不变量」。
- 把注释放在字段声明旁边、用普通注释（`//`）而非 Javadoc（`/** */`）放在类上方，因为它们属于**实现**，不属于**规格**。

下面的 `FollowGraph` 是本讲原文用来检验 Safety 论证是否合格的可变类型：原文把 Safety 一栏留成 `..???..`，让读者自己补全——这正好说明**一段合格的 Safety 论证必须逐个字段交代，而不是一句「所有字段都私有」了事**：

```
// 可变类型：表示 Twitter 用户的关注关系
public class FollowGraph {
    private final Map<String, Set<String>> followersOf;

    // Rep invariant:
    //   followersOf 中的所有字符串都是 Twitter 用户名
    //    （即非空且仅含字母、数字、下划线的字符串）
    //   没有用户关注自己，即 x 不在 followersOf.get(x) 中
    // Abstraction function:
    //   AF(followersOf) = 这样的关注关系图：Twitter 用户 x 被用户 y 关注，
    //   当且仅当 followersOf.get(x).contains(y)
    // Safety from rep exposure:
    //   ..???..（原文留白，交由读者补全；合格答案必须逐个字段说明，
    //   并明确指出 getFollowers() 返回的是防御性拷贝还是不可变包装，
    //   以及 Map 是否可能作为参数或返回值出现）

    // 操作（规格与方法体从略）
    public FollowGraph() { ... }
    public void addFollower(String user, String follower) { ... }
    public void removeFollower(String user, String follower) { ... }
    public Set<String> getFollowers(String user) { ... }
}
```

原文给出了六种候选说法让读者判断能否用来补全 `..???..`：其中「Strings are immutable」不充分（漏掉了可变的 `Set` 与 `Map`）；「本类是可变类型，所以不存在表示暴露问题」是**错误**的（可变类型的表示同样需要保护，否则客户端可以绕过 `addFollower()` 的契约、破坏「没有用户关注自己」这条 RI）；「`followersOf` 从不出现在参数或返回值中」也要配合「`getFollowers()` 返回的 `Set` 做了什么处理」才成立。只有像「`String` 不可变；表示中的 `Set` 是可变类型，但 `getFollowers()` 返回的是全新的防御性拷贝而不是表示中任何集合的引用；表示中的 `Map` 是可变类型，但它从不出现在任何操作的参数或返回值中」这样**逐字段、逐边界**的表述，才构成完整而有力的论证。原文还提醒：`Tweet` 的 `timestamp` 没有任何额外的 RI 条件，却仍然必须写进 Safety 论证——因为整个类型的不可变性依赖于所有字段都不被改动（对照「一个关于可变 `Date` 的不合格论证」，本笔记在场景 2 中展开）。

---

checkRep()：在运行时断言表示不变量（Checking the Rep Invariant at Runtime） – 定义与目的：`checkRep()` 是一个**私有**方法，把 RI 逐条翻译成 `assert` 语句。RI 不只是漂亮的数学概念：只要在运行时断言它，你就能**在 bug 刚产生的第一时间**抓住它，而不是等损坏的数据结构继续传播、最后在完全无关的地方以莫名其妙的方式炸掉。 – **直观解释（"它是什么？"）**：它是你留在类内部的安检门。每次内部状态被重新构造或改动，都过一次安检；一旦某次改动让表示落到红灯区，程序立刻停在那里，而责任范围被限制在这个类内部。这也是为什么 `checkRep()` 必须是 `private`：**不变量由实现自己负责检查和强制**，而不是委托给客户端——客户端不该知道表示的存在，更不该被要求去验证它。 – **关键规则与最佳实践**： – **在每一个创建或修改表示的操作末尾调用**：构造者（creator）、生产者（producer）、修改者（mutator）。例如 `RatNum` 的两个构造器都在末尾调用 `checkRep()`。 – **观察者（observer）也应调用**：观察者本不需要，但这是良好的防御性实践——它让「由表示暴露引起的 RI 破坏」更早被发现（暴露出去的状态被外部改坏后，下一次观察时就会当场报错）。 – **用 `assert` 而不是抛异常**：`assert` 表达的正是「这里应当永远为真，若为假说明

实现有 bug」，语义精确，且可以被整体关闭。 – **注意断言默认是关闭的**：`assert` 只在以 `-enableassertions`（简写 `-ea`）启动 JVM 时才生效；原课代码特意注释道：`// *** Warning: this does nothing unless you turn on assertion checking by running Java with -enableassertions (or -ea)`。测试时务必打开，否则 `checkRep()` 形同虚设。 – **`checkRep()` 与 RI 严格对应**：RI 里写了什么就断言什么；`RI: true` 时 `checkRep()` 实际上没有业务断言（但隐含的 `!= null` 仍值得断言）；注意 `int` 这类原始类型字段不可能为 `null`，对它们写 `assert i != null` 是编译错误。 – **null 检查不该省**：Java 中 RI 隐含要求每个对象引用非 null，因此 `checkRep()` 应包含这些 null 检查——「尽早抓住 null bug」正是它的价值（sp22 用 TypeScript 的严格 null 检查在静态层面承担了这一职责）。

有益的可变性（Beneficent Mutation） – **定义与目的**：不可变的精确定义是「**抽象值**永不改变」，而不是「表示值永不改变」。既然 AF 是「多对一」的，实现完全可以**在保持抽象值不变的前提下修改表示值**——客户端观察不到任何差别。这种改动叫有益的可变性。它换来的往往是性能：缓存、数据结构再平衡、惰性清理。 – **直观解释（"它是什么？"）**：`RatNum` 的弱 RI 版本是最经典的例子：RI 只要求 `denominator != 0`，于是连续的算术运算可以不约分；等到要给人类看结果时，`toString()` 才把 `numerator`、`denominator` 同时除以 `gcd` 并调整符号，顺手把表示改成了最简形式。注意 `toString()` 在不可变类型上是个观察者方法，居然改写了两个 `private` 字段——但因为 `(1,2)` 和 `(2,4)` 通过 `AF(numerator, denominator) = numerator/denominator` 映射到**同一个**抽象值，这次改动对客户端完全不可见，因而是无害的、有益的。 – **直观解释续（迭代器为什么必须可变）**：`MyIterator` 是 Reading 08（可变性与不可变性）中给出的迭代器实现（**不属于 Reading 11 原文，此处借用**），它的 `next()` 规格明确写着 `Modifies: this iterator to advance it to the element following the returned element`。从「集合的元素序列」这个抽象视角看，`next()` 是在**观察**序列中的下一个元素；但从迭代器自身的抽象视角看，它同时又**修改**了迭代器的位置。于是它既是观察者又是修改者。`hasNext()` 与 `next()` 的关系是一份契约：`hasNext()` 为真 ⟺ 再调用一次 `next()` 会返回一个元素而不越界；`next()` 的前置条件正是「`hasNext()` 返回 true」。语义上，`hasNext()` 观察「是否还有元素」，`next()` 观察「下一个元素」并推进游标——正因为它推进了游标，同一个迭代器连续两次 `next()` 会返回不同元素（不像不可变对象那样可以重复观察同一结果），所以迭代器本身是可变的。 – **关键规则与最佳实践**： – **判据是抽象值，不是表示值**：只要 AF 结果不变，改动就是合法的有益可变性；反之，即使只改了表示的一部分，只要抽象值变了，就不是有益可变性。 – **有益可变性不改契约**：它不需要在公开规格里说明，客户端无从（也不应）观察它。 – **迭代器/游标类类型必须可变**：把「位置」放进表示，就注定 `next()` 是修改者；要重复遍历就新建一个迭代器。 – **修改者也要维护 RI**：`next()` 推进 `index` 时必须保证 `0 <= index <= list.size()`，并在返回前调用 `checkRep()`。 – **缓存是典型场景**：把已算出的结果记在表示里，重复查询直接命中（Reading 21 的 `isPrime` 记忆化缓存就是这种「用空间换时间的表示内变更」），但要记得它在并发环境下会引入新的线程安全问题。

用 AF 定义 equals() 与 toString()（Equality and String Representation via the AF） – **定义与目的**：既然 AF 把表示值映射到抽象值，「两个对象相等」的**正确判据就比较抽象值**：当且仅当 `AF(this.r) = AF(that.r)`。对不可变类型而言，这与「观察相等性」一致——两个对象若无法通过该 ADT 规格中的任何操作区分开，就应当相等。`toString()` 同理，应当输出**抽象值**（人类可读的形式），而不是把字段直接拼出来。 – **直观解释（"它是什么？"）**：以本讲原文的 `RatNum` 为例，`AF(numerator, denominator) =`

`numerator/denominator`。在「分母为正且已约分」的 RI 下，表示是唯一的，于是逐字段比较恰好等价于比较抽象值；但只要把 RI 放宽成「只要求 `denominator != 0`」（这是原文明确认可的另一种设计），`new RatNum(1, 2)` 与 `new RatNum(2, 4)` 就会表示**同一个**抽象值 1/2，而逐字段比较会把它们判为不等——正确的做法是 `this.numerator * that.denominator == that.numerator * this.denominator`，或先化为最简再比较，即真正比较抽象值。这正是 AF 非单射带来的必然后果：**表示不同不代表抽象值不同**。`toString()` 同理应当输出抽象值：原文中宽松 RI 版本的 `toString()` 先把结果化简（顺便改写字段，即有益的可变性），再输出 `numerator/denominator` 这样的人类可读形式，而不是把内部字段裸拼出来。（`Duration(1,2)` 与 `Duration(0,62)`、`LetterSet("abc")` 与 `LetterSet("aBc")` 是同一现象的另外两个例子；**这两个示例出自 Reading 15（相等性），不属于 Reading 11 原文，此处借用来说明同一概念**。） – **关键规则与最佳实践**： – **`equals()` 比较抽象值**，实现方式是「类型检查 + 私有 `sameValue()` 助手」：`return that instanceof RatNum && this.sameValue((RatNum) that);`。`instanceof` 在面向对象中通常是坏味道，**唯一被允许的例外就是实现 `equals()`**（`getClass()` 之类同理禁止）。 – **必须用 `@Override`**：签名写错会变成**重载（overload）**而不是**覆盖（override）**，于是 `r1.equals(r2)` 与 `r1.equals(o2)`（`o2` 静态类型为 `Object`）会给出不同答案，相等性变得不一致。`@Override` 让编译器替你检查签名。 – **`hashCode()` 必须与 `equals()` 一致**：相等对象必须有相同的哈希值。规则是「**覆盖 `equals()` 时就一定要覆盖 `hashCode()`**」；`RatNum` 在规范表示下可以用 `31 * numerator + denominator`（等价于对抽象值求哈希）。注意别把 `hashCode` 拼成 `hashcode`——那只是新加了个方法，根本没有覆盖 `Object.hashCode()`。 – **`equals()` 必须满足等价关系**：自反、对称、传递，且 `x.equals(null)` 对非 null 的 `x` 返回 `false`，并且结果稳定（只要对象没被改动）。原课 Reading 15 用给 `Duration` 加「时钟偏差容忍」的相等性来演示传递性如何被破坏，是必须避免的设计。 – **`toString()` 输出抽象值**：例如 `return (denominator > 1) ? (numerator + "/" + denominator) : (numerator + "");`，它显示的是有理数的值，而不是内部字段的裸拼装；也可以用它来检查「同一抽象值的两种表示是否打印一致」。 – **可变类型的处理不同**（详见 Reading 15）：可变类型一般不应覆盖 `equals()` / `hashCode()`，而应继承 `Object` 的引用相等；若确实需要「看起来一样」的概念，另起名如 `similar()` / `sameValue()` 作为公开操作。本讲的 `FollowGraph` 就是这种可变类型——它不应按抽象值定义 `equals()`。

不变量保持的证明规则（Establishing Invariants: Structural Induction） – **定义与目的**：这是本讲的理论结晶，教我们如何**证明**一个 ADT 的不变量在所有实例上都成立。不变量是「对整个程序为真」的性质，对对象而言就是「对该对象的整个生命周期为真」。要让它成立，需要做两件事：让它在对象的初始状态为真，并保证对该对象的所有改动都不破坏它。翻译成 ADT 操作的类型语言就是： – **构造者（creators）与生产者（producers）必须为新实例确立不变量**； – **修改者（mutators）、观察者（observers）、生产者（producers）必须为已有实例保持不变量**。 – **直观解释（"它是什么？"）**：这是一次**结构归纳（structural induction）**：把所有实例按「被哪个操作造出来」分层。基例是新对象——由构造者与生产者负责建立；归纳步是旧对象被操作——由修改者、观察者、生产者负责保持。证明某个方法保持 RI 的范式完全固定：**假设进入方法时 RI 成立（这就是前置条件），执行方法体，在每一个可被外部观察到（observable）的位置断言 RI 仍然成立**。对不同类型的操作用不同的具体目标： – **构造者**：没有输入对象，必须**建立** RI（还要让 AF 有定义，即产生合法表示）。 – **生产者**：输入旧对象、输出新对象。要做两件事——**保持**输入对象的 RI（不要改坏它），并**建立**新对象的 RI（若新对象的构造通过构造者完成，那么这一步通常由构造者承担，但要确认生产者

的参数传递没有引入非法值）。 – **观察者**：必须**保持** RI。观察者若能破坏 RI，唯一的现实原因就是它同时是修改者（例如签名里写着「读取并推进状态」的 `next()`），或者它把内部表示泄漏了出去，让别处的代码改坏了状态。 – **修改者**：必须**保持** RI，且这是最容易出错的一类——必须检查它在所有分支（包括提前 `return`、抛异常前）都把表示留在了绿灯区。 – **关键规则与最佳实践**： – **完整规则**（课程原文的判据）：如果一个 ADT 的不变量满足：**由构造者与生产者确立；由修改者、观察者与生产者保持；并且不发生任何表示暴露**——那么该不变量对**该 ADT 的所有实例**都成立。换句更简洁的话：**前置条件假设 RI；方法体执行；在每个可观察处断言 RI 仍成立。** – **表示暴露让证明失效**：这是第三条为什么必须写进来的原因——如果表示被暴露，对象可能在程序的**任意**位置被改动，而这些改动不在任何一个操作的「方法体」之内，你根本没有地方去断言 RI，归纳步的每个证明义务都随之作废。 – **观察者不能破坏 RI，除非它本身是修改者或发生了暴露**：给一个纯观察者写「它保持 RI」的证明通常是一行话（它不写任何字段）；真正需要警觉的是它是否泄漏了可变表示的别名。 – **注意参数别名**：即便是观察者，如果它返回内部可变对象，客户端随后修改该对象就等于在类外修改表示——RI 的证明从此不成立。 – **把证明义务落实为代码**：RI 的建立与保持最终体现为各方法末尾的 `checkRep()`；这不是形式化证明，但能在运行时以极低成本覆盖绝大多数数学证明的错误。

规格说明可以谈论什么 / 用 ADT 不变量替代前置条件（What a Spec May Talk About / ADT Invariants Replace Preconditions） – **定义与目的**：既然 AF/RI 属于实现内部，那么类型 `T` 的**规格**（即其各操作的规格）就只能谈论客户端可见的东西：参数、返回值、抛出的异常。凡规格中需要提到类型 `T` 的值时，都应当把它描述为**抽象值**（抽象值空间 `A` 中的数学值），而不提表示空间 `R` 的任何细节。把表示当作对客户端不可见，正如方法体与局部变量对客户端不可见一样——这也解释了为何 AF/RI 写作类体内部的普通注释，而不是类上方的 Javadoc。 – **直观解释（"它是什么？"）**：更好的设计是**把前置条件变成 ADT**。与其写一个前置条件冗长的方法 `static String exclusiveOr(String set1, String set2)`，并在文档里要求「`set1` 是排序且无重复的字符集」，不如让类型本身承担这个性质：`static SortedSet<Character> exclusiveOr(SortedSet<Character> s1, SortedSet<Character> s2)`。这样三个目标同时达成：**更安全**（「有序且无重复」这个条件只需在一个地方强制——`SortedSet` 类型本身，而且静态检查会在编译期拒绝不满足条件的值）、**更易理解**（签名更简单，类型名 `SortedCharSet` 已经传达了必要信息）、**更易修改**（表示可以随意更换，`exclusiveOr` 及其所有客户端都不必改动）。 – **关键规则与最佳实践**： – **规格里只出现抽象值**：不要在公开文档里写下私有字段名、表示细节或 `checkRep()` 的存在。 – **AF/RI/Safety 用普通注释写在字段旁**，不要写成 Javadoc。 – **能封装成类型的约束，就不要写成前置条件**：课程早期习题里大量用前置条件表达的约束，其实都更适合定义一个自定义 ADT。 – **类型名应当传达不变量**（如 `SortedCharSet`、`Username`），让编译器替你守住一部分契约。 – **为消除前置条件而抽取的 ADT 要小而专一**：例如把「不重名且非空的用户名」抽成 `Username`，把「时间戳互不相同的推文集合」抽成 `TweetList`。

代码示例与对比分析

场景 1：不可变直角三角形把内部 `double[]` 数组直接交给客户端（表示暴露的经典反例）

❌ *错误代码*

```
/** Represents an immutable right triangle. */
public class RightTriangle {
    private double[] sides;                    // /*A*/

    public final double hypotenuse;            // /*B*/

    /**
     * Make a right triangle.
     * @param legA, legB the two legs of the triangle
     * @param hypotenuse the hypotenuse of the triangle,
     *        requires hypotenuse^2 = legA^2 + legB^2
     *        (within the error tolerance of double arithmetic)
     */
    public RightTriangle(double legA, double legB, double hypotenuse) {
        this.sides = new double[] { legA, legB };   // /*D*/
        this.hypotenuse = hypotenuse;               // /*D*/
    }

    /**
     * Get the two sides of the right triangle.
     * @return two-element array with the triangle's side lengths
     */
    public double[] getAllSides() {
        return sides;                               // /*E*/ 泄漏内部数组
    }

    /**
     * @param factor to multiply the sides by
     * @return a triangle made from this triangle by
     * multiplying all side lengths by factor.
     */
    public RightTriangle scale(double factor) {
        return new RightTriangle(sides[0]*factor, sides[1]*factor, hypotenuse*factor);
    }

    /**
     * @return a regular triangle made from this triangle.
     * A regular right triangle is one in which
     * both legs have the same length.
     */
    public RightTriangle regularize() {
        // 原文此处写成了 double bigLeg = Math.max(side[0], side[1]);
        //  (sp21/sp22 原文的笔误，会编译失败；此处按本意更正为 sides[0]/sides[1])
        double bigLeg = Math.max(sides[0], sides[1]);
        return new RightTriangle(bigLeg, bigLeg, hypotenuse);
    }

    // 没有 AF / RI / Safety 注释，也没有 checkRep()
}
```

【错误代码的问题】 1. `/*E*/` 处的 `getAllSides()` 把内部数组本身返回出去，于是客户端写 `double[] s = t.getAllSides(); s[0] = 100;` 就改掉了这个「不可变」三角形的边长——而 `hypotenuse` 已经固定，

勾股关系这条不变量当场失效，且没有任何地方会报警。 2. `/*B*/` 处把 `hypotenuse` 声明为 `public final` 字段：客户端从此依赖具体表示（表示独立性被破坏），而 `final` 只保证这个字段不能被重新赋值，完全不保证表示可以被替换。 3. 构造器只把两个 `double` 参数（原始类型，不存在别名问题）装进新数组，看起来「已经拷贝过了」，但一旦参数类型换成可变对象（数组、`Date`、集合），同样的写法就会把调用者的对象直接存进表示——这个陷阱被 `/*D*/` 的写法掩盖了。 4. 完全没有 AF / RI / Safety 注释，也没有 `checkRep()`：类头声称自己是「immutable」的，而这条最重要的性质没有任何书面论证，也没有任何运行时的检查。

✅ *正确代码*

```java
/**
 * Represents an immutable right triangle.
 */
public class RightTriangle {
    private final double[] sides;   // sides[0], sides[1] 是两条直角边的长度

    // Rep invariant:
    //   sides != null, sides.length == 2
    //   sides[0] > 0 and sides[1] > 0
    // Abstraction function:
    //   AF(sides) = the right triangle whose two legs have lengths
    //   sides[0] and sides[1]
    // Safety from rep exposure:
    //   字段 sides 是 private final;
    //   double[] 是可变类型，因此接受数组的构造器对它做防御性拷贝，
    //   并且没有任何操作把内部数组本身交出去：
    //   getAllSides() 返回一份新数组，getSide(int) 返回原始类型 double（值拷贝）。

    /**
     * Make a right triangle.
     * @param legA, legB the two legs of the triangle; both must be > 0
     */
    public RightTriangle(double legA, double legB) {
        this.sides = new double[] { legA, legB };
        checkRep();
    }

    /**
     * Make a right triangle from an array of leg lengths.
     * @param legs two-element array of positive leg lengths;
     *             this object does not alias the caller's array
     */
    public RightTriangle(double[] legs) {
        this.sides = legs.clone();     // 入参防御性拷贝：切断与调用者的别名
        checkRep();
    }

    /**
     * Get the two sides of the right triangle.
     * @return a fresh two-element array with the triangle's side lengths
     */
    public double[] getAllSides() {
        checkRep();
        return sides.clone();          // 出参防御性拷贝
    }

    /**
     * @param i 0 or 1
     * @return length of leg i
     */
    public double getSide(int i) {
        checkRep();
        return sides[i];               // double 是原始类型，返回的是副本
```

```
    }

    /**
     * @return length of the hypotenuse
     */
    public double getHypotenuse() {
        checkRep();
        // 由两条直角边计算得出，而不是把斜边也存进表示
        return Math.sqrt(sides[0]*sides[0] + sides[1]*sides[1]);
    }

    /**
     * @param factor to multiply the sides by, must be > 0
     * @return a triangle made from this triangle by
     * multiplying all side lengths by factor.
     */
    public RightTriangle scale(double factor) {
        return new RightTriangle(sides[0]*factor, sides[1]*factor);
    }

    /**
     * @return a regular triangle made from this triangle.
     * A regular right triangle is one in which
     * both legs have the same length.
     */
    public RightTriangle regularize() {
        double bigLeg = Math.max(sides[0], sides[1]);
        return new RightTriangle(bigLeg, bigLeg);
    }

    private void checkRep() {
        assert sides != null;
        assert sides.length == 2;
        assert sides[0] > 0 && sides[1] > 0;
    }
}
```

【为什么这样更好】 1. 表示中的数组不再有机会离开类：接受数组的构造器用 `legs.clone()` 切断入参别名，`getAllSides()` 返回 `sides.clone()` 切断出参别名，`getSide(int)` 返回原始类型 `double`，根本不产生别名。三条路径合起来才构成完整的 Safety 论证。 2. 斜边不再作为字段存储，而是由两条直角边**计算**得出。这不只是省了一个字段：它把「勾股关系」这条原本需要被小心维护的不变量，变成了**由构造方式自动成立**的事实——破坏一条不可能被违反的规则，比事后检查它更可靠。 3. AF / RI / Safety 三段注释与 `checkRep()` 齐备，且 `checkRep()` 逐条对应 RI，在每个公开方法返回前调用。 4. `scale()` 与 `regularize()` 这两个生产者都通过构造器产生新对象，因此新对象的 RI 由构造器负责建立——这正是「生产者必须确立新实例的不变量」。

【代码对比解说】 两个版本的字段声明都写得「很像不可变」：一个是 `private double[]`，另一个是 `public final double`。问题恰恰在这里——**`final` 锁住的是引用而不是数组内容，`private` 锁住的是字段名而不是你已经交出去的别名**。错误版本在 `/*E*/` 一处失守，整条不可变性就作废了；而且失守之后 `hypotenuse` 无法跟着变化，类的抽象值直接进入自相矛盾的状态。原文把这段代码标上 `/*A*/` 到 `/*E*/`

五个位置让读者判断，按原文原则推理：`/*B*/`（公开字段让客户端依赖表示）与 `/*E*/`（返回内部数组威胁不可变性）确实成立；`/*A*/`（私有数组字段本身）只是「风险」而非暴露，暴露发生在引用外泄之时；`/*C*/` 不成立——构造者完全可以有前置条件（本讲 `Tweet` 的构造器就有）；`/*D*/` 也不成立——`legA`、`legB`、`hypotenuse` 都是原始类型 `double`，不存在别名，无需拷贝。最后一点尤其值得记住：**判断是否需要防御性拷贝，看的是类型是否可变，而不是看这个字段是不是「重要」。**

【设计原则透视】 本组是 **Safety from rep exposure** 与 **RI 保持证明**的交汇点。返回内部数组会让不变量在 ADT 的操作之外被破坏，于是「不变量由构造者与生产者确立、由修改者与观察者保持」这条证明规则的第三个条件（不发生表示暴露）失效，整个归纳证明随之崩溃——这就是原文为什么把「no representation exposure」单列为规则的一条。它也直接呼应 Reading 08（不可变性）中「共享可变对象会破坏不可变性」的结论，以及 Reading 10（抽象数据类型）中表示独立性的要求：只要 `getAllSides()` 返回内部数组，客户端就会开始依赖「内部就是一个 `double[2]`」，你再也换不动这个表示。

场景 2：`Tweet` 的可变 `Date` 字段未做防御性拷贝，且缺少 AF/RI/Safety 与 `checkRep()`

❌ *错误代码*

```
import java.util.Date;

/** Immutable type representing a tweet. */
public class Tweet {
    private final String author;
    private final String text;
    private final Date timestamp;

    public Tweet(String author, String text, Date timestamp) {
        this.author = author;
        this.text = text;
        this.timestamp = timestamp;        // 保存了调用者的 Date 引用
    }

    public String getAuthor()  { return author; }
    public String getText()    { return text; }
    public Date getTimestamp() { return timestamp; }   // 把内部 Date 交出去

    // 没有 AF / RI / Safety 注释，也没有 checkRep()
    // 也没有对 author 格式、text 长度 280 的任何检查
}
```

【错误代码的问题】 1. `getTimestamp()` 返回的 `Date` 与 `t.timestamp` 是**同一个对象**。原文的客户端代码 `Date d = t.getTimestamp(); d.setHours(d.getHours()+1);` 是完全合理的写法，却顺手改掉了 `t` 内部的时间——`Tweet` 的不可变性不变量当场被破坏，而 `Tweet` 的代码一行都没被执行。 2. 构造器直接保存传入的 `Date`，原文的 `tweetEveryHourToday()` 例子因此出错：它想用一个 `Date` 对象依次走过一天 24 小时、每小时造一条推文，但由于 24 个 `Tweet` 共享同一个 `Date`，最终**所有推文的时间戳都相同**。 3. 没有 Safety 论证，维护者无从知道 `timestamp` 需要拷贝。特别注意：`timestamp` 并没有额外的 RI 条件，但它**仍然必须出现在 Safety 论证中**，因为整个类型的不可变性依赖于所有字段都不被改动——省略它是最常

见的错误。 4. 没有 `checkRep()`，RI 里「`author` 是 Twitter 用户名」「`text.length <= 280`」这两条完全靠调用者自觉；违反后对象会带着非法表示继续存在。

✅ *正确代码*

```
import java.util.Date;

// Immutable type representing a tweet.
public class Tweet {

    private final String author;
    private final String text;
    private final Date timestamp;

    // Rep invariant:
    // author is a Twitter username (a nonempty string of letters, digits, underscores)
    // text.length <= 280
    // Abstraction function:
    // AF(author, text, timestamp) = a tweet posted by author, with content text,
    // at time timestamp
    // Safety from rep exposure:
    // All fields are private;
    // author and text are Strings, so are guaranteed immutable;
    // timestamp is a mutable Date, so Tweet() constructor and getTimestamp()
    // make defensive copies to avoid sharing the rep's Date object with clients.

    /**
     * Make a Tweet.
     * @param author Twitter user who wrote the tweet
     * @param text text of the tweet
     * @param timestamp date/time when the tweet was sent
     */
    public Tweet(String author, String text, Date timestamp) {
        this.author = author;
        this.text = text;
        this.timestamp = new Date(timestamp.getTime());   // 入参防御性拷贝
        checkRep();
    }

    /** @return Twitter user who wrote the tweet */
    public String getAuthor() {
        checkRep();
        return author;                 // String 不可变，无需拷贝
    }

    /** @return text of the tweet */
    public String getText() {
        checkRep();
        return text;
    }

    /** @return date/time when the tweet was sent */
    public Date getTimestamp() {
        checkRep();
        return new Date(timestamp.getTime());             // 出参防御性拷贝
    }

    // Check that the rep invariant is true
```

```
    // *** Warning: this does nothing unless you turn on assertion checking
    // by running Java with -enableassertions (or -ea)
    private void checkRep() {
        assert author != null && text != null && timestamp != null;
        assert author.matches("[A-Za-z0-9_]+") : "not a Twitter username: " + author;
        assert text.length() <= 280 : "tweet too long: " + text.length();
    }
}
```

【为什么这样更好】 1. 上面的三段注释**就是课程原文给出的完整写法**（RI 用 `text.length <= 280` 表述，在 Java 中即 `text.length() <= 280`）：AF 说明「字段被解释成什么」，RI 说明「什么样的字段值合法」，Safety 逐字段交代隔离方式。 2. 构造器与观察者各做一次防御性拷贝，于是 `retweetLater`、`tweetEveryHourToday` 这两段「完全合理」的客户端代码再也不可能改坏内部表示。 3. `checkRep()` 把 RI 的两条要求变成运行时可执行的断言，并在每个公开方法返回前调用；调用观察者也要检查，这样「由表示暴露引起的不变量破坏」会更早暴露。 4. 更好的做法是把 `Date` 换成不可变的 `java.time.ZonedDateTime`：那样连两次拷贝都可以省去，Safety 论证退化成「所有字段私有且表示中所有类型不可变」这一句话。原文还提醒，Java API 文档已把 `Date` 的大部分方法标为 deprecated，新代码不应使用它。

【代码对比解说】 两版代码的字段声明一模一样（`private final`），唯一的差别是**在边界处是否拷贝**，以及**是否把约定写下来**。这说明表示暴露的本质不是字段可见性，而是**引用的流向**：任何可变对象只要跨过类边界（作为参数进来、或作为返回值出去），就必须拷一份。原文给这段代码留了一个更早的版本：字段直接写成 `public String author; public String text; public Date timestamp;`，于是客户端一句 `t.author = "rbmllr";` 就能改掉推文作者——那是最直白的表示暴露，它同时破坏不变量与表示独立性（客户端从此依赖「作者存在 `author` 字段里」）。此外原文还讨论了一种「用规格免责」的诱惑：在文档里写「调用者此后绝不能再修改这个 `Date` 对象！」。这种做法只在别无选择时（例如可变对象太大、拷贝代价过高）才可接受，因为它把「保证不变量」的责任推给了每一个调用者，代价是极大的推理成本。

【设计原则透视】 本组是 **Safety from rep exposure 论证**的教科书范例，也是 Reading 08（不可变性）中「拒绝表示暴露」的具体落实：不可变性不是靠 `final` 拿到的，而是靠「不共享可变对象」拿到的。它同时把 Reading 10（抽象数据类型）的「ADT 守护自己的不变量」落到了参数与返回值这两个最容易被忽略的边界上；而 `checkRep()` 则把 Reading 09（避免调试）的策略——让 bug 在离根因最近的地方暴露——落实为一行断言。

场景 3：用表示值而非抽象值实现 `equals()`（同一有理数被判成两个值）

❌ *错误代码*

```java
/**
 * Immutable type representing a rational number.
 *
 * 这里刻意采用原文认可的「更宽松的 RI」：只要求分母非零，不要求已约分。
 */
public class RatNum {
    private final int numerator;
    private final int denominator;

    // Rep invariant:
    // denominator != 0
    // Abstraction function:
    // AF(numerator, denominator) = numerator/denominator
    // Safety from rep exposure:
    // All fields are private, and all types in the rep are immutable.

    public RatNum(int n, int d) {
        if (d == 0) throw new ArithmeticException("denominator is zero");
        this.numerator = n;
        this.denominator = d;
        checkRep();
    }

    public int getNumerator()   { return numerator; }
    public int getDenominator() { return denominator; }

    // 错误：逐个字段比较「表示值」
    @Override
    public boolean equals(Object that) {
        if (!(that instanceof RatNum)) return false;
        RatNum r = (RatNum) that;
        return this.numerator == r.numerator
            && this.denominator == r.denominator;
    }

    // 错误：哈希也只依赖表示值
    @Override
    public int hashCode() {
        return 31 * numerator + denominator;
    }

    private static int gcd(int a, int b) {
        a = Math.abs(a); b = Math.abs(b);
        while (b != 0) { int t = a % b; a = b; b = t; }
        return a;
    }

    private void checkRep() {
        assert denominator != 0;
    }
}
```

【错误代码的问题】 1. `new RatNum(1, 2)` 与 `new RatNum(2, 4)` 的抽象值完全相同——`AF(numerator, denominator) = numerator/denominator` 把两者都映射到 1/2——但逐字段比较把它们判为**不等**。这与 AF 定义的相等性直接矛盾，也违反观察相等性：除了 `getNumerator()` / `getDenominator()` 这两个把表示细节暴露给客户端的观察者，没有任何规格内的操作能区分它们。 2. `hashCode()` 同样基于表示值，于是这两个「本应相等」的对象哈希值不同，放进 `HashSet` / `HashMap` 后会分别落到不同的桶里——查找失败，而且不会有任何报错。 3. 相等性的正确性被 RI 的强弱绑架：本版本刻意使用宽松 RI（原文明确说这是合理的设计，某些操作更便宜、某些更贵），而宽松 RI 恰恰意味着「同一抽象值有多种表示」，此时逐字段比较必然是错的。 4. `getNumerator()` / `getDenominator()` 让客户端可以自己写出「比较表示」的逻辑，等于把表示写进了客户端的语义里——表示独立性的损失会在下一次更换表示时集中爆发。

✅ *正确代码*

```java
/**
 * Immutable type representing a rational number.
 */
public class RatNum {
    private final int numerator;
    private final int denominator;

    // Rep invariant:
    // denominator != 0
    // Abstraction function:
    // AF(numerator, denominator) = numerator/denominator
    // Safety from rep exposure:
    // All fields are private, and all types in the rep are immutable.

    public RatNum(int n, int d) {
        if (d == 0) throw new ArithmeticException("denominator is zero");
        this.numerator = n;
        this.denominator = d;
        checkRep();
    }

    /** @return 该有理数的近似浮点值 */
    public double value() {
        checkRep();
        return (double) numerator / denominator;
    }

    @Override
    public boolean equals(Object that) {
        return that instanceof RatNum && this.sameValue((RatNum) that);
    }

    // 返回 true 当且仅当 this 与 that 表示同一个抽象值
    private boolean sameValue(RatNum that) {
        // 比较抽象值：n1/d1 == n2/d2  等价于  n1*d2 == n2*d1
        //  （此处用 long 承接乘法；原文的 sp22 版本用 bigint 实现 RatNum，
        //    正是为了彻底避免这类溢出问题）
        return (long) this.numerator * that.denominator
            == (long) that.numerator * this.denominator;
    }

    @Override
    public int hashCode() {
        // 先把表示化为最简形式再求哈希，这样「抽象值相等 ⇒ 哈希相等」必然成立
        int g = gcd(numerator, denominator);
        int n = numerator / g;
        int d = denominator / g;
        if (d < 0) { n = -n; d = -d; }
        return 31 * n + d;
    }

    @Override
    public String toString() {
```

```
        // 输出抽象值：最简形式的人类可读写法，而不是内部字段的裸拼装
        int g = gcd(numerator, denominator);
        int n = numerator / g;
        int d = denominator / g;
        if (d < 0) { n = -n; d = -d; }
        return (d > 1) ? (n + "/" + d) : (n + "");
    }

    private static int gcd(int a, int b) {
        a = Math.abs(a); b = Math.abs(b);
        while (b != 0) { int t = a % b; a = b; b = t; }
        return a;
    }

    private void checkRep() {
        assert denominator != 0;
    }
}
```

【为什么这样更好】 1. `sameValue()` 通过交叉相乘比较**抽象值**，因此 `new RatNum(1, 2).equals(new RatNum(2, 4))` 返回 `true`——这正是 `AF` 定义的相等性；`new RatNum(1, 2)` 与 `new RatNum(2, 3)` 仍然不等。 2. `hashCode()` 先把表示化成最简形式再计算，保证「相等 ⇒ 哈希相同」，因此这些对象可以安全地作为 `HashMap` 的键或放进 `HashSet`。 3. `toString()` 输出抽象值（最简分式），于是同一个抽象值的不同表示打印结果一致——这也是检查 `equals` / `hashCode` 是否正确的一个实用手段。 4. `equals(Object)` 用 `@Override` 覆盖而不是重载，`(RatNum) that` 是类型转换，向编译器声明「经 `instanceof` 检查后我确信它是 `RatNum`」。

【代码对比解说】 两版代码的差别只在 `sameValue` 与 `hashCode` 的几行，但这几行决定了「AF 非单射」这一数学事实是否被正确对待。一个重要的细节：**如果采用原文那种「分母为正且已约分」的严格 RI，表示是规范化的，逐字段比较恰好等价于比较抽象值**——很多实现者因此「碰巧」写对了。但这种正确性是脆弱的：只要 RI 被放宽（原文明确指出这是完全合理的设计选择），或者某天有人加了一条不约分的快速路径，逐字段比较就立刻变成 bug。所以正确的纪律是：**永远按抽象值思考相等性**，需要时再借助「规范化表示」作为优化。另外要避免在 `equals` 里拿 `toString()` 做比较——它把相等性建立在另一个方法的具体实现上，还会带来「`new RatNum(1, 2).equals("1/2")` 返回 true」这类荒谬结果。修改变换时也别忘 `@Override`：Reading 15 中「`equals(Duration that)` 被误当作覆盖」的经典错误正是由它拦下的。

【设计原则透视】 本组是 **AF 的直接应用**：抽象函数是相等性的定义基础，因此**不可变类型必须覆盖 `equals()`，从而也必须覆盖 `hashCode()`**（Reading 15 的核心结论）。它也说明为什么表示独立性在相等性上尤其关键：只要 `equals` 比较抽象值，你就能把 `RatNum` 的表示从「分子/分母两个 `int`」换成「已化简易形式」或「分子分母两个 `BigInteger`」，而所有客户端与测试一行都不用改；反过来，一旦 `equals` 比较字段，客户端就把你的表示永久锁定了。它还回指本讲的 `FollowGraph`：**可变类型**一般不应按抽象值定义 `equals()`，而应继承 `Object` 的引用相等——原文与 Reading 15 都对这一点给出了明确结论。

场景 4：构造者没有建立 RI（未约分、分母可为 0 或负数），且没有 `checkRep()`

❌ *错误代码*

```java
/**
 * Immutable type representing a rational number.
 */
public class RatNum {
    private final int numerator;
    private final int denominator;

    // Rep invariant:
    // denominator > 0
    // numerator/denominator is in reduced form,
    // i.e. gcd(|numerator|,denominator) = 1
    // Abstraction function:
    // AF(numerator, denominator) = numerator/denominator
    // Safety from rep exposure:
    // All fields are private, and all types in the rep are immutable.

    /**
     * Make a new RatNum == (n / d).
     * @param n numerator
     * @param d denominator
     */
    public RatNum(int n, int d) {
        this.numerator = n;       // 未约分
        this.denominator = d;     // 分母可能为 0，也可能为负数
        // 没有调用 checkRep()
    }

    /** @return 该有理数的近似浮点值 */
    public double value() {
        return (double) numerator / denominator;
    }

    @Override
    public String toString() {
        return numerator + "/" + denominator;
    }
}
```

【错误代码的问题】 1. `new RatNum(2, 4)` 违反 RI 的「已约分」条件——`gcd(2, 4) = 2 ≠ 1`。表示落进红灯区之后，AF 与 RI 给出的承诺（例如「表示唯一，可以逐字段比较」）全部失效，任何依赖规范形式的代码都会出错。 2. `new RatNum(1, 0)` 构造出一个分母为 0 的对象：它的抽象值根本不是有理数，`value()` 返回 `Infinity`，`toString()` 打印 `"1/0"`。错误被制造出来时没有任何提示，直到很久以后在别处炸开。 3. `new RatNum(1, -2)` 让分母为负，同样违反 RI；`toString()` 打出 `"1/-2"` 这种与 `"-1/2"` 表示同一抽象值却有不同写法的结果。 4. 没有 `checkRep()`：上面三种非法表示没有任何一处在构造完成时被发现，类头声称的 RI 只是注释而已——「不变量没有被写下来并被检查，就不是安全的不变量」。

✅ *正确代码*

```
/**
 * Immutable type representing a rational number.
 */
public class RatNum {
    private final int numerator;
    private final int denominator;

    // Rep invariant:
    // denominator > 0
    // numerator/denominator is in reduced form,
    // i.e. gcd(|numerator|,denominator) = 1
    // Abstraction function:
    // AF(numerator, denominator) = numerator/denominator
    // Safety from rep exposure:
    // All fields are private, and all types in the rep are immutable.

    /**
     * Make a new RatNum == n.
     * @param n value
     */
    public RatNum(int n) {
        this.numerator = n;
        this.denominator = 1;
        checkRep();
    }

    /**
     * Make a new RatNum == (n / d).
     * @param n numerator
     * @param d denominator
     * @throws ArithmeticException if d == 0
     */
    public RatNum(int n, int d) {
        if (d == 0) throw new ArithmeticException("denominator is zero");

        // reduce ratio to lowest terms
        int g = gcd(n, d);
        int reducedNumerator = n / g;
        int reducedDenominator = d / g;

        // make denominator positive
        if (reducedDenominator < 0) {
            this.numerator = -reducedNumerator;
            this.denominator = -reducedDenominator;
        } else {
            this.numerator = reducedNumerator;
            this.denominator = reducedDenominator;
        }
        checkRep();     // 构造者必须确立不变量
    }

    /** @return 该有理数的近似浮点值 */
    public double value() {
```

```
        checkRep();
        return (double) numerator / denominator;
    }

    @Override
    public String toString() {
        return (denominator > 1) ? (numerator + "/" + denominator)
                                 : (numerator + "");
    }

    private static int gcd(int a, int b) {
        a = Math.abs(a); b = Math.abs(b);
        while (b != 0) { int t = a % b; a = b; b = t; }
        return a;
    }

    // Check that the rep invariant is true
    // *** Warning: this does nothing unless you turn on assertion checking
    // by running Java with -enableassertions (or -ea)
    private void checkRep() {
        assert denominator > 0;
        assert gcd(Math.abs(numerator), denominator) == 1;
    }
}
```

【为什么这样更好】 1. 构造器在返回前把表示**规约到唯一的合法形式**：先 `gcd` 约分，再统一符号使分母为正，`d == 0` 则立刻抛出异常——这就是「构造者必须确立不变量」的具体动作。 2. `checkRep()` 逐条对应 RI（分母为正、已约分），并在每个构造器与每个观察者返回前调用；构造时若哪一步写错，断言会立刻指出问题出在这里。 3. 表示被规范化之后，`toString()` 可以直接输出抽象值而不必再化简，`equals` / `hashCode` 也可以直接依赖规范形式（详见场景 3 的讨论）。 4. 两个构造器（`RatNum(int n)` 与 `RatNum(int n, int d)`）都调用 `checkRep()`，因此「每个构造者都要建立 RI」这条要求没有漏网之鱼。

【代码对比解说】 这一组的问题不在观察者，而在**构造者**——也就是「不变量确立」这一环。原文的完整规则是：不变量必须**由构造者与生产者确立**、**由修改者、观察者与生产者保持**，并且**不发生表示暴露**。错误版本把 RI 写得很清楚却完全不去建立它，于是每个对象从出生起就可能是非法的。同样的错误也会发生在修改者身上：例如一个用 `char[]` 表示字符集合的 `CharSet`（本讲原文的 `CharSet` 用 `String` 表示，这里借用同一个抽象），其 `add(char c)` 在写入 `chars[size] = c; ++size;` 之后**忘记调用 `checkRep()`**，那么一旦边界判断写错，非法状态就会静默留存，直到某个完全无关的地方爆出莫名其妙的错误。这也解释了为什么 `checkRep()` 的调用点应当是「每个可能改动表示的路径的出口」，而不是「记得起来的时候」。另一个现实提醒：`assert` 默认关闭，只有在 `-ea`（`-enableassertions`）下运行 JVM 时这些检查才真正生效——课程原文特意把这条警告写在了 `checkRep()` 旁边。

【设计原则透视】 本组对应 **RI + `checkRep()` + 不变量的建立/保持规则**，是「证明一个 ADT 的不变量对所有实例成立」这条结构归纳规则的基例部分：构造者没有基例，就必须亲手建立 RI。它同时展示了 RI 的强弱如何成为设计权衡——原文指出，完全可以用更宽松的 RI（只要求分母非零）实现同一个 ADT，代价是别的操作要付出更多（例如相等性必须比较抽象值而不能比较表示值，如场景 3 所示）。这与 Reading 06（规

格说明）和 Reading 07（设计规格）相连：规格只谈抽象值，而「什么表示合法、如何解释表示」属于实现文档，写在类体内部的普通注释里，不进入公开规格。

场景 5：迭代器的可变性、契约与 RI 维护（`next()` 同时是观察者与修改者）（该示例的迭代器 `MyIterator` 出自 Reading 08（可变性与不可变性），不属于 Reading 11 原文；本讲的可变性/不变量理论正好解释了它为什么必须可变、以及修改者如何保持 RI，故在此借用。）

❌ *错误代码*

```
import java.util.List;

/**
 * 可变类型：从前到后遍历 List<String> 中元素的迭代器。
 */
public class MyIterator {

    private final List<String> list;
    private int index;      // list[index] 是下一次 next() 将返回的元素

    public MyIterator(List<String> list) {
        this.list = list;
        this.index = 0;
    }

    /** @return true 表示还有元素可返回 */
    public boolean hasNext() {
        return index < list.size();
    }

    /** @return 下一个元素 */
    public String next() {
        return list.get(index++);      // 没有前置条件检查，越界时抛 IndexOutOfBoundsException
    }

    /** @return 剩余元素个数 */
    public int remaining() {
        return list.size() - index;    // 若 index 被推进过头，这里会返回负数
    }
}
```

【错误代码的问题】 1. 没有 RI 注释说明 `index` 的合法范围（`0 <= index <= list.size()`），也没有 `checkRep()`，于是 `index` 可以停留在非法状态（例如被推进到 `list.size()` 之外）而无人察觉。 2. `next()` 没有声明前置条件「`hasNext()` 返回 true」，越界行为的契约含糊；客户端可以在空迭代器上调用它，得到的是底层 `List` 的实现细节异常，而不是类型的契约。 3. `remaining()` 若不遵守 RI 就会返回负数，产生「逻辑上不可能」的观察结果——这是典型的 RI 破坏后静默传播错误的例子。 4. 规格里没有写明 `next()` 会**同时修改**迭代器状态，接手的程序员可能误以为可以重复观察同一结果。

✅ *正确代码*

```
import java.util.ArrayList;
import java.util.List;

/**
 * 可变类型：从前到后遍历 List<String> 中元素的迭代器。
 */
public class MyIterator {

    private final List<String> list;
    private int index;      // 表示：list[index] 是下一次 next() 将返回的元素；
                            // index == list.size() 表示已无元素可返回

    // Rep invariant:
    //   list != null，且 list 中不含 null 元素
    //   0 <= index <= list.size()
    // Abstraction function:
    //   AF(list, index) = 序列 list[index..list.size()-1]，
    //   即本迭代器接下来将依次返回的元素序列
    // Safety from rep exposure:
    //   list 与 index 都是 private；
    //   index 是原始类型 int，list 从不作为参数或返回值暴露；
    //   list 是构造时持有的引用，本类不修改它（它是共享的、由客户端持有的对象，
    //   但本类只读，且它不属于本类的可变表示 —— 本类的可变表示是 index）。
    //   注意：若客户端在迭代过程中修改该 list，迭代结果将不再有定义，
    //   这与 java.util.Iterator 的 "fail-fast" 契约属同一类问题。

    /**
     * 构造一个迭代器。
     * @param list 要遍历的列表，不允许为 null
     */
    public MyIterator(List<String> list) {
        this.list = new ArrayList<>(list);   // 防御性拷贝，隔离客户端的后续修改
        this.index = 0;
        checkRep();
    }

    /**
     * 测试迭代器是否还有元素可返回。
     * @return 若 next() 仍能返回一个元素则为 true，否则为 false
     */
    public boolean hasNext() {
        checkRep();
        return index < list.size();
    }

    /**
     * 取得列表的下一个元素。
     * 前置条件：hasNext() 返回 true。
     * 修改：将本迭代器推进到被返回元素之后的位置。
     * @return 列表的下一个元素
     */
    public String next() {
        checkRep();
```

```
        final String element = list.get(index);
        ++index;                        // 有益的状态推进：迭代器的抽象值随之改变
        checkRep();                     // 修改后立即断言 RI 仍成立
        return element;
    }

    /** @return 剩下还会返回的元素个数 */
    public int remaining() {
        checkRep();
        return list.size() - index;
    }

    private void checkRep() {
        assert list != null;
        assert 0 <= index && index <= list.size() : "index 越界: " + index;
    }
}
```

【为什么这样更好】 1. RI 明确了 `index` 的合法范围，`checkRep()` 把它变成可执行断言；`next()` 修改表示后**立即**断言 RI，正是「修改者必须保持不变量」的落地方式。 2. 规格里显式写出 `next()` 既是**观察者**（返回序列中的下一个元素）又是**修改者**（推进迭代器位置），并写出它的前置条件 `hasNext() == true` ——`hasNext()` 与 `next()` 的关系因此清晰：前者判断「还能不能再调用一次 `next()`」，后者要求这个判断为真。 3. `remaining()` 在 RI 成立时不可能返回负数，因为 `index <= list.size()` 由不变量保证。 4. 构造器对 `list` 做防御性拷贝，避免客户端在迭代过程中改动列表内容而使迭代器语义失去定义。

【代码对比解说】 迭代器是本讲中「可变性可以有益」的最好教材。若把 `index` 固定住，迭代器就只能反复观察同一个元素——因此**迭代器必须是可变的**：它的抽象值本身就随 `next()` 改变（这不是有益的可变性，而是正当的修改者行为）。与之相对的是 `RatNum` 弱 RI 版本中 `toString()` 改写字段却**不改变抽象值**，那才是「有益的可变性」（beneficent mutation）。两种情形都必须满足同一条纪律：**改动之后 RI 仍然成立**。`checkRep()` 放在 `next()` 内推进 `index` 之后，而不是只在方法开头，正是这个纪律的体现。最后要注意：对 `list` 做拷贝的取舍是设计选择——`java.util.Iterator` 选择不拷贝并采用 fail-fast 检测，两条路都要在 Safety 论证与规格中如实写明。

【设计原则透视】 本组把 **RI 的保持证明**落实到修改者身上，并展示了观察者与修改者边界的情形（`next()` 两者兼具）。它也串起 Reading 08（不可变性）中关于 `final` 与迭代器的讨论——`final` 锁住引用、锁不住被指向对象的可变内容；以及 Reading 21（并发）的伏笔：可变表示一旦被多个线程共享，`checkRep()` 与不变量的推理都会变得复杂得多。

---

sp22 原版 TypeScript 写法对照

sp22 用 TypeScript 讲授同一内容，机制与 Java 一一对应，注意以下三处差异：

```
// sp22 原版 TypeScript 写法：Tweet 的表示与不可变性
class Tweet {
    private readonly author: string;
    private readonly text: string;
    private readonly timestamp: Date;

    /**
     * @param author Twitter user who wrote the tweet
     * @param text text of the tweet
     * @param timestamp date/time when the tweet was sent
     */
    public constructor(author: string, text: string, timestamp: Date) {
        this.author = author;
        this.text = text;
        this.timestamp = new Date(timestamp.getTime());   // 与 Java 相同的防御性拷贝
    }

    /** @returns date/time when the tweet was sent */
    public getTimestamp(): Date {
        return new Date(this.timestamp.getTime());
    }
}
```

- **`private readonly`** ↔ **`private final`**：TypeScript 的 `readonly` 保证字段在构造后不被重新赋值，与 Java 的 `final` 语义对应；两者都**只锁引用，不锁被引用对象的可变内容**，因此 `Date` 的防御性拷贝在两个版本里都必不可少。
- **`Array<T>`** ↔ **`List<T>`**：sp22 用 `const names: Array<string> = [...]` 展示静态检查表达的不变量；Java 对应写法是 `List<String> names = List.of("Huey", "Dewey", "Louie");`（`List.of` 返回不可变列表，但仅是运行时不可变）。
- **严格 null 检查 ↔ 静态与断言双保险**：sp22 在 6.031 的 TypeScript 配置中打开 strict null-checking，由静态类型检查器保证表示中不含 `null` / `undefined`；Java 没有对应机制，因此这些 null 检查必须由 `checkRep()` 中的 `assert s != null` 承担（这正是 Java 版多写几行断言的原因）。
- **`Set<T>` 等可变容器**：sp22 中 `Map<string, Set<string>>` 这样的表示同样可变，其「Safety from rep exposure」论证需要 `getFollowers()` 做防御性拷贝；Java 版对应写法是 `private final Map<String, Set<String>> followersOf`，论证描述为「字段私有；`String` 不可变；`Set` 可变但 `getFollowers()` 返回全新的防御性拷贝；`Map` 可变但从不作为参数或返回值出现」——注意这正是课程练习中**唯一合格的论证方式**：必须逐个字段说明，而不是写「所有字段都私有」了事。

与其他设计原则的关联

- **Reading 06（Specifications，规格说明）**：本讲的 `checkRep()` 与 null 检查直接延续「前置条件/后置条件隐含对象非 null」的约定；`assert` 断言 RI 则是把规格中的前提转成可执行检查。
- **Reading 07（Designing Specifications，设计规格）**：规格中的前置条件一旦太多太复杂，就应该改用一个 ADT 来封装（本讲 `exclusiveOr` 的例子）；这类封装的安全性建立在「AF/RI 只属于实现、规格只

谈抽象值」的边界上。

- **Reading 08（Mutability & Immutability，可变性与不可变性）**：本讲的防御性拷贝、不可变包装器（`Collections.unmodifiableList`）、以及 `final` 只锁引用不锁内容的结论，全部是 Reading 08 的直接延伸；「有益的可变性」给出了不可变性的精确定义——**抽象值不变**，而非表示值不变。
- **Reading 09（Avoiding Debugging，避免调试）**：`checkRep()` 是「让 bug 尽早暴露」这一策略的典范——因为伤害发生在离根因最近的地方，而不是在数据结构被污染很久之后。
- **Reading 10（Abstract Data Types，抽象数据类型）**：本讲是 Reading 10 的理论深化。Reading 10 建立了抽象与表示的分离、表示独立性以及创造者/生产者/观察者/修改者的分类；本讲用 AF/RI 精确说明「表示如何被解释」，并给出了「如何证明不变量在所有实例上成立」的归纳规则。`MyString` 表示从「无冗余数组」换成「数组 + start/end」的例子，正是表示独立性带来的可修改性。
- **Reading 12（Interfaces & Generics，接口与泛型）**：把 AF/RI 写清楚是给类型换实现的前提，而接口/泛型是让客户端只依赖抽象的手段；`Map<String, Set<String>>` 这类泛型容器出现在表示中时，其 Safety 论证的难度也随之上升。
- **Reading 13（Debugging，调试）**：断言与 `checkRep()` 是调试工具；RI 明确了「合法状态」，因此调试一个损坏的数据结构时，你可以先定位到第一次违反 RI 的位置。
- **Reading 15（Equality，相等性）**：本讲为相等性打下理论基础——**抽象函数是相等性的定义基础**，因此不可变类型必须覆盖 `equals()`，从而也必须覆盖 `hashCode()`；`toString()` 应当输出抽象值。`RatNum` 是这条链路上本讲原文的主力例子；`Duration`、`LetterSet` 也属同一链路，但**它们是 Reading 15（相等性）中的例子**，本讲只在需要对照时借用并已逐一标注。
- **Reading 17（Recursive Data Types，递归数据类型）**：递归表示（树、列表）的 RI 通常需要递归地陈述（「左右子树都满足 RI」），而「建立与保持不变量」的证明相应地也就是对该递归结构做**结构归纳**。
- **Reading 21（Concurrency，并发）**：本讲的可变表示在多线程共享时，RI 的保持会变得更困难；Reading 21 中记忆化缓存（memoization）这类「有益可变性」正是新的线程安全风险源——修改者的并发交错可能破坏缓存自身的表示不变量。
- **Reading 23（Locks & Synchronization，锁与同步）**：把「修改者必须保持 RI」的推理扩展到并发场景，需要把操作的原子性一并纳入证明：只有互斥地执行修改者，RI 才能被保证。

关键要点

- **每个有实质表示的 ADT 都必须写下三件套**：`Abstraction function`（表示值 → 抽象值的映射，可代入求值）、`Representation invariant`（合法表示值的集合，可逐条断言）、`Safety from rep exposure`（逐个字段说明为什么客户端拿不到可变内部表示的别名）。三者缺一不可，并且写在字段声明旁边的普通注释里。
- **把 RI 变成可执行的 `checkRep()`，并在每个构造者、生产者、修改者返回前调用；观察者也建议调用。**记住 `assert` 默认关闭，测试时必须用 `java -ea` 启动。
- **消灭表示暴露**：凡参数或返回值中出现可变类型，就在边界处拷一份；能换成不可变类型（如用 `java.time.ZonedDateTime` 代替 `java.util.Date`）就优先更换；返回集合考虑不可变包装或拷贝。

- **一切相等性以抽象值为准**：AF 是满射但不必单射，因此 `equals()` 必须比较抽象值，`hashCode()` 必须与之一致，`toString()` 应当输出抽象值。
- **不变量由构造者与生产者确立、由修改者/观察者/生产者保持，且必须无表示暴露**——这三条同时成立，不变量才对该 ADT 的所有实例成立；有益的可变性是允许的，只要抽象值不变。

常见陷阱与注意事项

- **把 `private final` 当作「安全」** → 字段私有且 final 只锁引用；若类型可变（`Date`、数组、`List`）且引用被传入或返回，表示照样暴露，不可变性在不变量层面已经失效。
- **只在构造器里做防御性拷贝，忘了观察者也要拷贝** → `getTimestamp()` 这类方法把内部 `Date` 的别名交出去，客户端一个 `setHours()` 就能改坏你精心保护的表示。
- **AF/RI 写成空泛的话（「所有字段都有效」「表示一个字符集合」）** → 无法代入求值，`checkRep()` 无从编写，不同实现者对表示含义产生分歧，最终表现为「某人改了 `add`，另一个人写的 `contains` 就悄悄错了」。
- **在 RI 里引用抽象值，或完全不写 `checkRep()`、写了却从不在 `-ea` 下运行** → 前者本末倒置（非法的表示值根本没有抽象值，RI 必须只谈表示本身，否则既不能判定也不能检查）；后者让损坏的表示继续传播，bug 在离根因很远的地方爆发，因为 `assert` 在默认 JVM 配置下根本不会执行。
- **用表示值实现 `equals()` / `hashCode()`、覆盖 `equals()` 却忘记 `hashCode()`、或把 `equals(RatNum that)` 当作覆盖 `equals(Object)`** → 抽象值相同的对象被判为不等（或相等却哈希不同），放进 `HashSet` / `HashMap` 会出现找不到、重复存储等玄学 bug；签名写错则变成重载（overload），静态类型为 `Object` 的实参和静态类型为 `RatNum` 的实参会得到不同答案——务必写 `@Override`（Reading 15 用 `equals(Duration that)` 演示了同一个错误）。
- **以为可变类型不需要担心表示暴露** → 可变类型的 RI 同样需要保护：`getFollowers()` 直接返回内部 `Set` 会让客户端绕过 `addFollower()` 的契约（例如让自己关注自己），一个操作层面的不变量就这样被破坏。正确做法是返回防御性拷贝或不可变包装。

思考题（带答案）

问题 1：`RatNum` 使用了「分母为正且已约分」的 RI，而课程原文指出也可以用更宽松的 RI（只要求分母非零）实现同一个 ADT。请说明这两种设计各自的代价，并解释为什么在**宽松 RI** 的版本中，`sameValue()` 不能像严格版本那样直接比较 `numerator` 与 `denominator`。

答案：严格 RI 的代价是**每次构造（以及每次会产生新值的运算）都必须做一次 `gcd` 并调整符号**，好处是表示被规范化成唯一形式，于是 `equals`、`hashCode`、`toString` 都很简单（直接比较/使用字段即可），而且可以顺手保证「分母永远为正」这一对客户端友好的性质。宽松 RI 的代价是**同一个抽象值会有大量不同的表示**（`1/2`、`2/4`、`-3/-6`……），于是任何「比较抽象值」的操作都必须真正去算抽象值：`equals` 要么交叉相乘 `this.numerator * that.denominator == that.numerator * this.denominator`（注意溢出风险，课程原文的 `bigint` 版本正是为此），要么在比较前先化简；好处是**连续运算不必反复约分**，可以在需要显示结果时才化简一次（课程原文的 `toString()` 就是这么做的，它甚至直接改写了

`numerator` / `denominator` 字段——因为改前改后的表示映射到同一个抽象值，所以这是无害的、有益的可变性）。这件事的教益是：**AF 是不是单射取决于 RI 与表示的选取**，而「比较抽象值」这条纪律在任何选取下都不能违反。

问题 2：本讲原文给 `FollowGraph`（可变类型，表示是 `private final Map<String, Set<String>> followersOf`）留了一道 Safety 论证的填空题，把 `..???..` 交给读者补全。请判断下面三种说法能否用来补全，并说明理由：(a)「`String` 是不可变的。」(b)「本类是可变类型，所以不存在表示暴露的问题。」(c)「`followersOf` 是可变 `Map`，其中装着可变的 `Set` 对象，但 `getFollowers()` 返回的是防御性拷贝，而其他所有参数与返回值都是不可变的 `String` 或 `void`。」然后回答一个更一般的问题：为什么「三个字段都是 `private`，所以不会发生表示暴露」这样的论证对 `Tweet` 也不合格，以及 `timestamp` 为什么必须出现在 `Tweet` 的 Safety 论证里（尽管 RI 没有对它提出额外条件）。

答案：三种说法都不能单独用来补全。(a) 只交代了表示中 `String` 这一部分——它完全没有触及 `Set` 与 `Map` 这两个可变类型，而它们才是暴露风险的所在；Safety 论证的要求是**逐个字段**交代，漏掉字段就等于漏掉风险。(b) 是**错的**，而且错在最容易误解的一点上：可变类型的不变量同样需要保护。`FollowGraph` 的 RI 里有「没有用户关注自己，即 `x` 不在 `followersOf.get(x)` 中」这样的条件，如果 `getFollowers()` 把内部 `Set` 直接交出去，客户端就能绕过 `addFollower()` 的契约往集合里塞进 `x` 自己，从而破坏这条 RI——可变性从来不是免于表示暴露的理由。(c) 是三者中唯一接近合格的表述：它逐字段说明（`Map` 可变但从不出现于参数或返回值；`Set` 可变但 `getFollowers()` 返回防御性拷贝），并覆盖了参数与返回值这两个暴露发生的边界。原文还给出两条可用的替代写法供对比：用 `Collections.unmodifiableSet` 之类的不可变包装把表示中的 `Set` 封起来并据此论证，或者逐操作列举（「构造器不暴露表示；`addFollower` 不暴露；`removeFollower` 不暴露；`getFollowers` 不暴露」）——但后者只有在**逐个操作真的检查过**、并说清依据时才成立，否则只是一句空话。最后，为什么同样的道理适用于 `Tweet`：「三个字段都是 `private`，所以不会发生表示暴露」只考察了字段可见性这一个维度，而暴露的真正判定标准是**可变对象的引用有没有越过类边界**。`private` 只阻止通过字段名直接访问，并不阻止你把内部可变对象的引用交出去：`getTimestamp()` 返回的 `Date` 与 `t.timestamp` 是同一个对象，客户端一次 `setHours()` 就改掉了内部状态；构造器若不拷贝，调用者手里的 `Date` 也仍然指向表示里那个对象。而 `timestamp` 必须出现的原因在于：**整个类型的不变性依赖于所有字段都不被改动**，即使 RI 对它没有额外条件（除了所有对象引用都隐含的 `!= null`），只要它可能被外部改掉，`Tweet` 就不再是不可变的——「不可变」本身就是这个类型最重要的一条不变量。合格写法即原文那句：「All fields are private; author and text are Strings, so are guaranteed immutable; timestamp is a mutable Date, so Tweet() constructor and getTimestamp() make defensive copies to avoid sharing the rep's Date object with clients.」

问题 3：请用「建立与保持不变量」的规则，说明为什么「观察者（observer）不能破坏 RI，除非它同时是修改者」，并解释表示暴露如何让整个证明失效。再以 `MyIterator.next()` 为例，说明一个同时是观察者与修改者的方法需要满足哪些证明义务。

答案：证明规则是结构归纳：构造者与生产者必须为**新实例确立** RI；修改者、观察者与生产者必须为**已有实例保持** RI；并且不得发生任何表示暴露。对纯观察者来说，证明义务几乎自动成立——它的方法体**不写任何字段**（按定义它只读取与返回信息），所以执行前后表示完全相同，RI 若进入时为真，退出时必然为真；它也

可能通过调用其他观察者来间接读取，那些调用同样只读。唯一的例外是它**同时是修改者**（签名中带有「Modifies」或它调用了本类的修改者），此时它就必须像修改者一样证明「每个可被观察到的位置 RI 都仍然成立」。另一个例外情形是它把可变表示泄漏出去——严格说这时 RI 不是被观察者本身破坏的，而是被类外的代码破坏的，但后果一样：**在 ADT 操作的边界之外，没有任何地方能断言 RI**，于是归纳步中「所有改动都在某个操作的严格控制之下」这一前提不成立，整个证明随之作废；这正是「必须消灭表示暴露」成为证明规则第三条的原因。对 `MyIterator.next()` 而言，它的前置条件是 `hasNext() == true`（即进入时 `index < list.size()`），方法体做两件事：读取 `list.get(index)`（观察），然后 `++index`（修改）；它的证明义务是：在推进 `index` 之后，`0 <= index <= list.size()` 仍然成立（由前置条件 `index < list.size()` 与「`index` 每次只加一」共同保证），并在返回前用 `checkRep()` 断言这一点。此外，由于 `next()` 改变了迭代器的抽象值（它接下来将返回的元素序列变短了），这次修改必须在规格中如实声明为「Modifies」，而它不会破坏 RI，因为推进后的 `index` 恰好落在合法区间内。

Reading 12: 接口、泛型与枚举（Interfaces, Generics, Enums）

> 说明：本讲 sp22 原版使用 TypeScript，本笔记按用户要求提供 Java 代码示例；类型/API 与 sp21（6.031 Java 版）原文保持一致。凡课程原文**未涉及**、为回答本讲延伸问题而补充的 Java 生态知识（`EnumSet` / `EnumMap`、类型擦除与通配符细节、抽象类取舍），均单独放在标题中写明「**补充说明（超出 6.031 原文的 Java 生态知识）**」的小节里；原文中的 ADT 例子一律保留真实类名（`MyString` / `SimpleMyString` / `FastMyString`、`Curve` / `ArrayCurve`、`ImmutableRectangle` / `ImmutableSquare` / `MutableRectangle` / `MutableSquare`、`Set<E>` / `SimpleSet<E>` / `CharSet`、`Month` / `Semester`）。

概述

本讲回答的问题是：既然我们在 Reading 10（抽象数据类型）与 Reading 11（抽象函数与表示不变量）里已经学会「用一个类定义 ADT」，那么还有哪些语言机制可以定义 ADT？原文给出四条路：**接口（interface）**把 ADT 的规格与实现彻底分离，**泛型类型参数（generic type parameter）**让同一个 ADT 定义服务于一整族元素类型，**枚举（enumeration）**描述取值有限且不可变的小型 ADT，而**全局函数 + 不透明类型**（在 C 一类非面向对象语言里常见，原文用 `FILE` / `fopen` / `fputs` / `fclose` 举例）说明数据抽象本身并不依赖 class/interface 这些语言特性。贯穿全讲的核心原则是「面向接口编程」：客户端只依赖接口所声明的规格，实现类独占表示（rep）与抽象函数（AF）/表示不变量（RI），而**子类型（subtyping）关系必须由规格的强弱决定**——`implements` 关键字只保证签名兼容，绝不保证规格没有被削弱。

这直接服务于三大目标：**Safe from bugs**——接口让编译器替我们检查「是否实现了全部操作、签名是否正确、客户端是否只使用了规格允许的方法」，枚举让编译器拒绝取值集合之外的值、也拒绝把两个不同的枚举类型混用；**Easy to understand**——接口是客户端唯一需要读的文件，规格集中在接口里，不带任何实现细节（原文的 `FastMyString` 代码之所以难读，正是因为它把 ADT 层规格与实现细节混在一起）；**Ready for change**——新增一个实现类即可替换整个表示，客户端代码一行不改，而泛型让同一份实现服务于 `Set<String>`、`Set<Integer>` 等一整族类型。

核心概念与设计原则详解

接口与实现分离（Interface / Implementation Separation） – **定义与目的**：接口是一串**没有方法体的方法签名**，它只表达「这个 ADT 有哪些操作、这些操作的契约是什么」；实现类在 `implements` 子句里声明接口，并为接口中每个方法提供方法体。这样做的目的是建立一道**抽象屏障（abstraction barrier）**：Java 接口里连实例字段都不能出现，客户端根本无从依赖表示（rep），因此不可能产生「意外的表示依赖」这一类最难修的 bug。 – **直观解释（"它是什么？"）**：接口像餐厅的菜单，实现类像后厨。菜单上写的是「你能点什么、会得到什么」，而不是「厨师用哪口锅、先把哪个调料下锅」。原文的 `MyString` 就是菜单：`length()`、`charAt(i)`、`substring(start, end)`；`SimpleMyString` 与 `FastMyString` 是两个后厨，做法完全不同却端出同样的菜。 – **关键规则与最佳实践**： – 接口只放**公开方法签名与规格**（Javadoc），不放任何实例字段，也不放方法体（Java 8 之前；Java 8 起有了 `default` 与 `static` 两个例外，见下一个概念）。 – 接口不能被 `new`：原文明确指出 `new List<String>()` 是静态错误，因为接口没有 rep；要得到 `List`，必须实例化某个提供 rep 的类，例如 `new ArrayList<String>()`。 – 接口不能声明构造器，所以「创建者操作（creator）」要么由实现类的构造器承担，要么由接口的**静态工厂方法**承担（`List.of()`、`MyString.valueOf(boolean)`）。 – 实现接口的类必须提供接口里全部方法的方法体（否则是编译错误），并且规格**至少与接口一样强**；用 `@Override` 标注，它既让编译器检查签名匹配，又告诉读者「规格在接口里，不必在这里重复」（重复写规格违反 DRY）。 – 同一个 ADT 可以有多个实现并存于同一个程序：在 Reading 10 的 `MyString` 例子里，两种表示法无法同时存在于一个程序中；改成「接口 + `SimpleMyString` / `FastMyString` 两个实现类」后就可以。 – 编译器能替我们抓「少写方法」「返回类型写错」这类错误，但**抓不到**「后置条件被削弱」「前置条件被加强」这类规格层面的违约——原文强调那必须由人审。 – 原文的 `Curve` / `ArrayCurve` 例子暴露了一个反向陷阱：把实现类名写进接口（如让 `Curve` 的 `join` 返回 `ArrayCurve`）既造成循环依赖，又**不是 representation-independent**；同时 `ArrayCurve` 若少实现 `contains()` 就会直接编译失败，这说明签名层面的检查确实有效。

```
/**
 * MyString represents an immutable sequence of characters.
 */
public interface MyString {

    // We'll skip this creator operation for now
    // /**
    //  * @param b a boolean value
    //  * @return string representation of b, either "true" or "false"
    //  */
    // public static MyString valueOf(boolean b) { ... }

    /**
     * @return number of characters in this string
     */
    public int length();

    /**
     * @param i character position (requires 0 <= i < string length)
     * @return character at position i
     */
    public char charAt(int i);

    /**
     * Get the substring between start (inclusive) and end (exclusive).
     * @param start starting index
     * @param end ending index. Requires 0 <= start <= end <= string length.
     * @return string consisting of charAt(start)...charAt(end-1)
     */
    public MyString substring(int start, int end);
}
```

两个实现类提供完全不同的表示，却满足同一个接口规格——这正是接口的价值所在（代码取自 sp21 原文）：

```
class SimpleMyString implements MyString {   // 也可以放在独立的 SimpleMyString.java 中

    private char[] a;

    /**
     * Create a string representation of b, either "true" or "false".
     * @param b a boolean value
     */
    public SimpleMyString(boolean b) {
        a = b ? new char[] { 't', 'r', 'u', 'e' }
              : new char[] { 'f', 'a', 'l', 's', 'e' };
    }

    // private constructor, used internally by producer operations
    private SimpleMyString(char[] a) {
        this.a = a;
    }

    @Override public int length() { return a.length; }

    @Override public char charAt(int i) { return a[i]; }

    @Override public MyString substring(int start, int end) {
        char[] subArray = new char[end - start];
        System.arraycopy(this.a, start, subArray, 0, end - start);
        return new SimpleMyString(subArray);
    }
}

class FastMyString implements MyString {   // 原定义为 public class FastMyString implements M

    private char[] a;
    private int start;
    private int end;

    /**
     * Create a string representation of b, either "true" or "false".
     * @param b a boolean value
     */
    public FastMyString(boolean b) {
        a = b ? new char[] { 't', 'r', 'u', 'e' }
              : new char[] { 'f', 'a', 'l', 's', 'e' };
        start = 0;
        end = a.length;
    }

    // private constructor, used internally by producer operations.
    private FastMyString(char[] a, int start, int end) {
        this.a = a;
        this.start = start;
        this.end = end;
    }
```

```
    @Override public int length() { return end - start; }

    @Override public char charAt(int i) { return a[start + i]; }

    @Override public MyString substring(int start, int end) {
        return new FastMyString(this.a, this.start + start, this.start + end);
    }
}
```

`FastMyString` 的表示是「整串字符 + `start`/`end` 两个下标」——于是 `true` 这个值的 rep 是 `char[] {'t','r','u','e'}` 加上 `start=0`、`end=4`，而 `substring` 只改两个整数、连字符数组都不复制，这就是「同一 ADT 的两种实现可以有不同的性能特征」。注意原文的 rep 字段写成 `private char[] a; private int start; private int end;`（没有 `final`），而原文紧接着的 code review 练习正是追问「rep 字段是否应该声明为 `final` 使其不能被重新赋值」——答案是应当（Reading 08 不可变性与安全从表示暴露的要求）。

原文在讨论「隐藏实现类」时还用到 `Curve`/`ArrayCurve`：

```
/** Represents an immutable curve in the plane. */
public interface Curve {

    /** @return true if the point (x,y) lies on this curve */
    public boolean contains(double x, double y);

    /**
     * @return a curve formed by connecting this with that
     * 注意：返回类型必须是 Curve，不能写成 ArrayCurve，否则接口就不再 representation-independent。
     */
    public Curve join(Curve that);
}

/** Implementation of Curve. */
class ArrayCurve implements Curve {   // 也可以是独立的 ArrayCurve.java

    /** make a one-point curve */
    public ArrayCurve(double x, double y) { /* ... */ }

    @Override public boolean contains(double x, double y) { /* ... */ return false; }

    @Override public Curve join(Curve that) { /* ... */ return this; }

    /** extend this curve with a segment to the point (x,y) */
    public void add(double x, double y) { /* ... */ }
}
```

Java 接口的语法与规则（Rules of Java Interfaces：default 与 static 方法） – 定义与目的：Java 接口默认只有签名；Java 8 引入了两个例外——`default` 实例方法（带方法体，实现类可以继承它，也可以覆盖

它）与 `static` 方法（属于接口自身，通过接口名调用）。`default` 方法的目的是「在不破坏已有实现类的前提下给接口增加新操作」；`static` 方法的目的是「把创建者操作（工厂方法）放进接口，从而隐藏实现类的名字」。 – **直观解释（"它是什么？"）**：接口本来是纯粹的「合同」，`default` 方法相当于在合同里附上一段「标准做法」：不想自己写的人可以直接沿用，想自己写的人可以覆盖。`static` 方法则像菜单背面的「订餐电话」——它属于菜单，而不属于任何一道菜，所以 `MyString.valueOf(true)` 与 `List.of("glorp")` 都不需要任何实例。 – **关键规则与最佳实践**： – 接口**不能有实例字段**：Java 接口里声明的字段自动成为 `public static final` 常量，所以「在接口里放 rep」这件事在语法上就被禁止了——这是接口天然无表示泄漏的根据。 – 接口不能有构造器，因此创建者操作只能来自实现类构造器（`ArrayList()`、`LinkedList()`）或接口的静态方法（`List.of()`、`MyString.valueOf(boolean)`）。 – `default` 方法会带实现，因此它必须被当作**规格的一部分**来谨慎设计：一旦发布就很难撤回；只在确实希望所有实现共享同一行为时才使用。JDK 自身大量使用这一机制，例如 `Collection.removeIf` 与 `List.replaceAll` 都是 `default` 方法。 – 静态方法不能被实现类「继承后覆盖」，必须通过接口名调用：`MyString.valueOf(true)`，而不是 `new FastMyString(true)`。 – 一个类可以 `implements` 多个接口（原文的动机例子：一个下拉列表控件既是 widget 又是 list），但只能 `extends` 一个类。 – 接口继承接口用 `extends`，且只能**加强规格**或**新增操作**：原文的真实 JDK 签名是 `interface SortedSet<E> extends Set<E>`，它不提供任何 `Set` 方法的实现，只是加强规格（例如承诺元素有序）并新增操作。

```
import java.util.ArrayList;
import java.util.List;

class InterfaceMethodsDemo {

    static void demo() {
        // static 方法属于接口自身：List.of() 是 List 接口里的静态方法
        List<String> names = List.of("glorp", "fleeb");

        List<String> mutable = new ArrayList<>(names);

        // default 方法：接口提供的共享实现，实现类无需自己写
        mutable.removeIf(s -> s.startsWith("g"));   // Collection 的 default 方法
        mutable.replaceAll(String::toUpperCase);    // List 的 default 方法

        System.out.println(mutable);                // [FLEEB]

        // 静态错误：接口没有 rep，不能被 new（原文举的就是 new List<String>()）
        // List<String> bad = new List<String>();
    }
}
```

抽象屏障与工厂方法（Abstraction Barrier and Factory Methods） – **定义与目的**：用接口定义 ADT 后，客户端仍可能写 `MyString s = new FastMyString(true);`——这行代码虽然类型正确，却**打破了抽象屏障**：客户端必须知道表示类的名字。而接口里从来没有承诺过「每个实现都提供同样的构造器」：`SimpleMyString` 与 `FastMyString` 的构造器参数就完全不同，构造器的规格也不会出现在接口里。工厂方法解决这个问题。 – **直观解释（"它是什么？"）**：这像租房时只和中介签合同，而不是直接认识房东。房东可

以换（实现类可以换），你的合同（客户端代码）不用重签。原文用一句话概括这个模式的价值：「客户端可以在不打破抽象屏障的前提下使用 ADT」。 – **关键规则与最佳实践**： – 优先让客户端调用接口上的工厂方法（`MyString.valueOf(..)`），而不是实现类构造器。原文的 `MyString.valueOf` 就是接口里的静态方法：`public static MyString valueOf(boolean b) { return new FastMyString(b); }`。 – **静态方法必须自己声明类型参数**：原文特别指出这是一个「晦涩的 Java 语法要求」——`make` 这样的静态方法要在签名开头独立声明类型参数，写成 `public static <F> Set<F> make()`；原文故意用 `F` 而不是 `E`，就是为了强调「这是另一个类型参数」。实例方法则不需要这个额外的 `<...>`，因为它们总是使用外围接口声明的 `public interface Set<E>` 中的那个 `E`。 – 静态工厂的返回类型就是接口类型，因而可以把实现类完全藏起来：原文的练习要求给 `Set` 加上自己的创建者操作 `empty()`，正确签名是 `public static <E> Set<E> empty()`，正确实现体是 `return new HashSet<>();`（而不是 `return new Set<>();`，后者是静态错误，也不是 `return this;`，静态方法没有 `this`）。 – 完全隐藏实现在工程上是一种**取舍**：有时客户端确实需要按性能挑实现——这正是 Java 库同时暴露 `ArrayList` 与 `LinkedList` 的原因，因为它们的 `get()` 与插入操作性能不同。 – 也可以用**静态工厂方法**（如 `List.of()`、`Collections.unmodifiableList()`）替代构造器来承担创建者/生产者角色；原始 ADT 概念总表中「静态方法」这一栏正是它们。

```
/**
 * 原文的 MyString 接口加上创建者操作：静态工厂方法。
 * 客户端从此只看见 MyString，看不见 FastMyString。
 */
public interface MyString {

    /**
     * @param b a boolean value
     * @return string representation of b, either "true" or "false"
     */
    public static MyString valueOf(boolean b) {
        return new FastMyString(b);
    }

    /** @return number of characters in this string */
    public int length();

    /**
     * @param i character position (requires 0 <= i < string length)
     * @return character at position i
     */
    public char charAt(int i);

    /**
     * Get the substring between start (inclusive) and end (exclusive).
     * @param start starting index
     * @param end ending index. Requires 0 <= start <= end <= string length.
     * @return string consisting of charAt(start)...charAt(end-1)
     */
    public MyString substring(int start, int end);
}

class MyStringClient {
    static void demo() {
        MyString s = MyString.valueOf(true);        // 不打破抽象屏障
        System.out.println("The first character is: " + s.charAt(0));  // 't'
        System.out.println("The whole string is: " + s.substring(0, s.length()));
    }
}
```

子类型（Subtyping）与 Liskov 替换原则（Liskov Substitution Principle, LSP） – **定义与目的**：类型就是「一组值 + 一组操作」。子类型是超类型的**子集**：「B 是 A 的子类型」意思是「每一个 B 都是一个 A」，用规格的语言说就是「**每一个 B 都满足 A 的规格**」。因此 B 的规格必须**至少与 A 一样强**。LSP 就是这条要求的另一种说法：子类型的对象必须能够在任何期望超类型的地方替换超类型，而不破坏该超类型向客户端承诺的任何性质。 – **直观解释（"它是什么？"）**：把超类型想成一份「能力承诺书」：凡是被标注为 `ImmutableRectangle` 的东西，都保证「调用 `getWidth()` 我会给你宽度」。子类型只有真的做到全部承诺，才有资格自称超类型。原文用「每个正方形都是矩形吗？」这个问题把这个判断变成可操作的练习：不可变时答案是「是」，可变时答案就变成了「不是」。 – **关键规则与最佳实践**： – 规格的强弱规则：子类型**可以**

加强后置条件、可以削弱前置条件（对调用者更宽松）、**可以抛出更具体的异常**；**不可以加强前置条件、削弱后置条件**，也不可以「规格不可比」。 – `implements`（类实现接口，如真实 JDK 签名 `class ArrayList<E> implements List<E>`）与 `extends`（接口继承接口，如 `interface SortedSet<E> extends Set<E>`）都建立子类型关系；接口 `extends` 接口只能加强规格或新增操作。 – 编译器只检查**签名兼容**（是否实现了所有方法、返回类型是否匹配），**不检查规格的强弱**——「后置条件是否被削弱」必须由人审。 – `ImmutableSquare implements ImmutableRectangle` 是**合法**子类型：`getWidth()` 与 `getHeight()` 都完整满足接口规格，整个 `ImmutableSquare` 也满足 `ImmutableRectangle` 规格。 – `MutableSquare implements MutableRectangle` 却**不合法**：`MutableRectangle.setSize(width, height)` 承诺任意宽高都可用，而正方形必须保持宽高相等，于是 `MutableSquare` 只能加强前置条件（要求 `width == height`）或削弱后置条件（非正方形时行为未指定）——两者都违反「子类型规格至少一样强」。 – 运行时强制类型转换（`(List<String>) obj`）与 `instanceof` 分派是**危险信号**：它们通常意味着你在用运行时判断补救一个本该由规格层次表达的差别——即设计上「不是真正的子类型」。 – 静态类型 vs 动态类型要分清：原文的 `List<String> list = new ArrayList<>(List.of("abc")); Object obj = list;` 中，`list` 的静态类型是 `List<String>`、动态类型是 `ArrayList`；`obj` 的静态类型是 `Object`，动态类型仍是 `ArrayList`。因此 `obj.size()` 是**静态错误**（`Object` 没有 `size`），而 `obj.toString()` 通过动态分派返回 `"abc"`；`list = obj;` 是静态错误，`list = (List<String>) obj;` 才合法（且 `obj = "abc";` 合法、`list = "abc";` 是静态错误）。

```
/** An immutable rectangle. */
public interface ImmutableRectangle {
    /** @return the width of this rectangle */
    public int getWidth();
    /** @return the height of this rectangle */
    public int getHeight();
}

/** An immutable square: 每个正方形都真的是矩形，规格被完整满足。 */
class ImmutableSquare implements ImmutableRectangle {
    private final int side;
    /** Make a new side x side square. */
    public ImmutableSquare(int side) { this.side = side; }
    /** @return the width of this square */
    @Override public int getWidth() { return side; }
    /** @return the height of this square */
    @Override public int getHeight() { return side; }
}
```

可变版本则相反。原文给出的三种候选 `setSize` 规格，只有「加强后置条件」那一类才合法：

```
/** A mutable rectangle. */
public interface MutableRectangle {
    public int getWidth();
    public int getHeight();
    /** Set this rectangle's dimensions to width x height. */
    public void setSize(int width, int height);
}

// A mutable square. 原文写作：implements MutableRectangle /* hopefully? */
class MutableSquare implements MutableRectangle {
    private int side;

    public MutableSquare(int side) { this.side = side; }

    @Override public int getWidth() { return side; }
    @Override public int getHeight() { return side; }

    /**
     * Set this square's dimensions to width x height.
     * Requires width = height.                    // ← 违法：加强了前置条件
     */
    @Override public void setSize(int width, int height) {
        if (width != height) {
            throw new IllegalArgumentException("width != height");
        }
        this.side = width;
    }
}
```

原文对三种候选规格的判断结论如下，读者应当能独立复现这套推理：

1. `Requires width = height.` → **加强了前置条件**，不合法。
2. `@throws BadSizeException if width != height` → 同样是**加强了前置条件**（把原本合法的调用变成异常），不合法。
3. 「若 `width = height` 则设为 `width x height`；否则新尺寸未指定」→ **削弱了后置条件**（对客户端的保证变少），不合法。
4. 只有真正的加强后置条件（例如「保证设置成功后 `getWidth() == width`」这类更强的保证）才是合法的强化。

子类化（Subclassing）与动态分派（Dynamic Dispatch） – 定义与目的：子类化（`class B extends A`）定义一个类作为另一个类的扩展：子类自动继承父类的实例方法与**方法体**、可以覆盖它们、同时**继承父类的 rep**，并能新增自己的方法与字段。它与「实现接口」的关键差别就在于**继承方法体与 rep**。动态分派（dynamic dispatch）是 Java 决定「调用哪一份实现」的规则：看**对象的动态类型**，而不是引用的静态类型。 – **直观解释（"它是什么？"）**：实现接口像「签合同」——只继承义务，不继承家产；子类化像「继承家业」——你不仅拿到义务，还拿到房子和家具的摆放方式（rep），于是你的装修就会影响所有继承人。动态分派则像「按实际来的人提供服务」：无论你手里拿的是谁的会员卡（静态类型），服务都按你本人（动态类型）

来。 – **关键规则与最佳实践**： – 子类化**应当**蕴含子类型：若 `SpottedTurtle` 是 `Turtle` 的子类（Python 的 `class SpottedTurtle(Turtle)`，Java 对应 `class SpottedTurtle extends Turtle`），它的规格也必须至少与 `Turtle` 一样强，因为 Java 允许把子类对象用在任何期待父类的地方。 – 继承 rep 带来三个风险：父类与所有子类之间的**表示暴露（rep exposure）**、父类与子类之间的**表示依赖（rep dependence）**，以及父类与子类**互相破坏对方的表示不变量**。设计一个可安全子类化的父类，意味着它必须同时提供两份契约：一份给客户端，一份给子类——这些麻烦在接口上根本不会出现。 – 每个类都自动是 `Object` 的子类，因而继承了 `toString()`、`equals()`、`hashCode()`；调试时常常需要覆盖 `toString()`——`FastMyString` 的默认输出是 `"FastMyString@504bae78"`（只有类名与内存地址，毫无用处）。 – 覆盖方法时总是写 `@Override`：编译器会检查签名是否真的与父类/接口匹配，读者也知道规格在哪里。 – 记住：引用类型不影响分派结果——`Object obj = new FastMyString(true); obj.toString()` 得到 `"true"`，而不是 `Object` 的默认实现。

```
public class FastMyString implements MyString {
    // rep 与构造器见上文

    @Override
    public String toString() {
        String s = "";
        for (int i = 0; i < this.length(); ++i) {
            s += this.charAt(i);
        }
        return s;
    }
}

class DispatchDemo {
    static void demo() {
        FastMyString fms = new FastMyString(true);   // 这个值代表 4 字符串 "true"
        System.out.println(fms.toString());          // "true"

        Object obj = new FastMyString(true);
        System.out.println(obj.toString());          // 动态分派：仍然是 "true"
        // System.out.println(obj.length());         // 静态错误：Object 没有 length()
    }
}
```

泛型类型参数（Generic Type Parameters） – **定义与目的**：泛型类型是「规格里含有一个占位类型、稍后再填入」的类型。`Set<E>` 就是「某个其他类型 E 的有限集合」这一整族 ADT 的规格，而不必为 `Set<String>`、`Set<Integer>` 各写一份。它同时解决两个问题：**代码复用**（一份实现服务所有元素类型）与**静态类型安全**（`Set<String>` 里塞不进整数，取出元素也不需要强转）。原文还指出 `List<E>` 是泛型接口、`HashMap<K, V>` 是泛型类，而 `String<E>` 在 Java API 中根本不存在。 – **直观解释（"它是什么？"）**：泛型像带空格的标签模板：「这是装 ___ 的盒子」。你可以印出「装字符串的盒子」「装整数的盒子」，但每种盒子的标签一旦填好，装错东西就会被编译器当场抓住。原文用 `Set<E>` 演示了这一点：`Set<String> strings = Set.make();` 之后，编译器就知道这是一个字符串集合。 – **关键规则与最佳实

践： – 泛型接口的写法是 `public interface Set<E>`；**泛型实现类**保留占位符：`public class HashSet<E> implements Set<E>`（这就是 JDK 的做法）；**非泛型实现类**把它换成具体类型：`public class CharSet implements Set<Character>`。 – 静态方法必须自己声明类型参数：原文的 `public static <F> Set<F> make()` 故意用 `F` 而不是 `E`，以强调这是**另一个**类型参数；实例方法不需要额外声明，因为它们总是用外围接口的 `E`。 – 泛型实现只能依赖接口规格里**显式承诺**的性质：`HashSet<E>` 可以依赖 `E` 是 `Object`（因而有 `hashCode` 与 `equals`，这正是它做哈希所必需的），但**不能**调用只在 `String` 上才有的方法，因为 `E` 可能是任何类型。 – 非泛型实现（如用 `String` 做 rep 的 `CharSet`）通常**不适合**表示任意元素类型的集合：一个 `String` rep 无法表示 `Set<Integer>`，除非重新设计 RI 与 AF 来应付多位数——这正说明「换一个 rep 就要换一套 AF/RI」。 – 规格必须停留在抽象层：`contains(E e)`、`add(E e)` 的 Javadoc 说的是「元素」这一抽象概念，绝不能提「数组的哪个下标」之类表示细节；这些规格应当适用于 `Set` ADT 的任何合法实现。 – 接口的操作可以故意**欠定（underdetermined）**，实现可以**加强后置条件**但不能削弱：原文的 `Set.pick()` 只承诺「返回集合中的某个元素，空集时抛 `NoSuchElementException`」，于是 `SimpleSet` 可以说「返回最近加入且尚未移除的元素」甚至「返回最小元素」；但把 `@return` 写成「返回 `elementList` 末尾的元素」就暴露了 rep、也不再 representation-independent。 – 泛型也让「同一份代码服务一整族类型」成为可能：原文的练习要求把 `IntervalSet`（问题集 2 的接口）泛型化，此时原先写 `new RepMapIntervalSet()` 的地方要改成 `new RepMapIntervalSet<String>()`，也可以直接用菱形写法 `<>` 省掉重复的类型参数。

```
import java.util.ArrayList;
import java.util.List;
import java.util.NoSuchElementException;

/**
 * A mutable set.
 * @param <E> type of elements in the set
 */
public interface Set<E> {

    // example creator operation
    /**
     * Make an empty set.
     * @param <F> type of elements in the set
     * @return a new set instance, initially empty
     */
    public static <F> Set<F> make() {
        return new SimpleSet<F>();
    }

    // example observer operations

    /**
     * Get size of the set.
     * @return the number of elements in this set
     */
    public int size();

    /**
     * Test for membership.
     * @param e an element
     * @return true iff this set contains e
     */
    public boolean contains(E e);

    // example mutator operations

    /**
     * Modifies this set by adding e to the set.
     * @param e element to add
     */
    public void add(E e);

    /**
     * Modifies this set by removing e, if found.
     * If e is not found in the set, has no effect.
     * @param e element to remove
     */
    public void remove(E e);
}
```

泛型接口，泛型实现：下面是原文的 `SimpleSet<E>`，它把元素放在 `List<E>` 里，完全不关心 `E` 究竟是什么；`SimpleSet.pick()` 还演示了「实现加强接口的后置条件」：

```
/** A generic implementation that keeps its elements in a list. */
public class SimpleSet<E> implements Set<E> {

    private final List<E> elementList = new ArrayList<>();

    @Override public int size() { return elementList.size(); }

    @Override public boolean contains(E e) { return elementList.contains(e); }

    @Override public void add(E e) { if (!contains(e)) { elementList.add(e); } }

    @Override public void remove(E e) { elementList.remove(e); }

    /**
     * Picks an element from the set.
     * @return the element most recently added but not yet removed
     * @throws NoSuchElementException if set is empty
     */
    public E pick() throws NoSuchElementException {
        if (elementList.isEmpty()) {
            throw new NoSuchElementException("pick on empty set");
        }
        return elementList.get(elementList.size() - 1);
    }
}
```

泛型接口，非泛型实现：原文用 `CharSet` 演示「把 `E` 换成具体类型」；注意它的 rep 是一个 `String`，因此只适合字符集合：

```
/** Represents a mutable set of characters; rep is a String.
 * 原文只列出 contains/add 与 "// ..."；这里补全 size/remove 使其完整可编译。 */
public class CharSet implements Set<Character> {

    private String s = "";

    @Override
    public boolean contains(Character e) {
        checkRep();
        return s.indexOf(e) != -1;
    }

    @Override
    public void add(Character e) {
        if (!contains(e)) s += e;
        checkRep();
    }

    @Override
    public int size() { return s.length(); }

    @Override
    public void remove(Character e) {
        int i = s.indexOf(e);
        if (i != -1) s = s.substring(0, i) + s.substring(i + 1);
        checkRep();
    }

    /** Rep invariant: s has no repeated characters. */
    private void checkRep() {
        for (int i = 0; i < s.length(); ++i) {
            assert s.indexOf(s.charAt(i)) == i : "rep invariant violated: repeated characte
        }
    }
}

class SetDemo {
    static void demo() {
        Set<String> strings = Set.make();       // 编译器推断 Set<String>
        strings.add("glorp");
        strings.add("glorp");
        System.out.println(strings.size());     // 1

        // Set<Integer> numbers = Set.make();   // 同一份泛型实现服务另一种元素类型
        // strings.add(42);                     // 静态错误：int 不能转换为 String

        Set<Character> chars = new CharSet();   // 非泛型实现：E 已被替换为 Character
        chars.add('a');
        System.out.println(chars.contains('a'));  // true
    }
}
```

原文还给出 JDK 的真实签名作为对照：`class ArrayList<E> implements List<E>`（泛型类实现泛型接口）、`interface SortedSet<E> extends Set<E>`（泛型接口继承泛型接口）、`class HashSet<E> implements Set<E>`（泛型实现，其内部依赖每个元素正确实现 `Object.equals` 与 `Object.hashCode`）。

补充示例（该 ADT 名字与语义出自 Reading 15 相等性，不是 Reading 12 的原文示例）：下面用 `Map<E, Integer>` 实现一个「多重集」，用来复习「泛型实现 + AF/RI + 静态工厂 + 不暴露 rep」的组合。`Bag<E>` 是 Reading 15 用来讨论可变类型相等性的 ADT，其原文操作是 `add(e)` / `remove(e)` 返回新的 `Bag`，本笔记为呼应讨论把它写成变异式接口（`add` / `remove` 返回 `void`）：

```
import java.util.HashMap;
import java.util.Map;

/**
 * 补充示例：A mutable bag (multiset)，同一元素可以出现多次。
 * @param <E> type of elements in the bag
 */
public interface Bag<E> {

    /** @param e element to add; modifies this bag by adding one occurrence of e */
    public void add(E e);

    /** @param e element to remove; if e is not found, has no effect */
    public void remove(E e);

    /** @return the total number of occurrences of all elements in this bag */
    public int size();

    /** @param e an element @return true iff this bag contains at least one occurrence of e
    public boolean contains(E e);
}

/**
 * Rep: counts maps each element to the number of times it occurs.
 * Rep invariant: every value in counts is strictly greater than 0.
 * Abstraction function: AF(counts) = 每个元素 e 出现 counts.get(e) 次的多重集。
 * Safety from rep exposure: counts 是 private final，从不返回或别名给客户端。
 */
class MapBag<E> implements Bag<E> {

    private final Map<E, Integer> counts = new HashMap<>();

    @Override
    public void add(E e) {
        counts.put(e, counts.getOrDefault(e, 0) + 1);
        checkRep();
    }

    @Override
    public void remove(E e) {
        Integer n = counts.get(e);
        if (n == null) { return; }
        if (n == 1) { counts.remove(e); } else { counts.put(e, n - 1); }
        checkRep();
    }

    @Override
    public int size() {
        int total = 0;
        for (int n : counts.values()) { total += n; }
        return total;
    }
```

```
    @Override
    public boolean contains(E e) { return counts.containsKey(e); }

    private void checkRep() {
        for (int n : counts.values()) {
            assert n > 0 : "rep invariant violated: a count is not positive";
        }
    }
}

/** 静态工厂：客户端只看到接口，看不到 MapBag。 */
final class Bags {
    private Bags() { }                      // 不可实例化的工具类

    /** @param <F> type of elements in the bag @return a new, empty bag */
    public static <F> Bag<F> empty() { return new MapBag<F>(); }
}
```

泛型的实现限制：类型擦除与通配符——补充说明（超出 6.031 原文的 Java 生态知识） – 定义与目的：Java 的泛型是通过**类型擦除（erasure）**实现的：类型参数只在编译期存在，编译后 `Set<E>` 变成 `Set`、`E` 变成 `Object`（或它的上界），运行时**看不到**类型实参。这样做是为了与 Java 5 之前的旧代码保持二进制兼容，代价是若干语法限制。通配符 `? extends E` / `? super E` 是「协变/逆变」的写法，用于让方法参数或返回值接受「一族相关类型」，在保留静态类型安全的同时提高 API 的灵活性。 – **直观解释（"它是什么？"）**：擦除像请柬模板上的「+1 来宾」——印刷模板上只有「来宾」这一栏（`Object`），具体是谁只在你填写时（编译期）被检查；到了现场（运行时）没人再核对那一栏。通配符则像收货规则：「凡是 `E` 的子类都收」。`? extends E` 是只读视角（你只能从中取出 `E`），`? super E` 是只写视角（你只能往里放入 `E`）。 – **关键规则与最佳实践**： – **不能创建泛型数组**：`new E[10]` 是编译错误；要么用 `Object[]` 加显式转换（会产生 unchecked 警告），要么改用 `ArrayList<E>` 这样的泛型集合。 – 不能 `new E()`、不能对类型参数做 `instanceof`（`e instanceof E` 非法）、不能定义类型为 `E` 的 `static` 字段、不能按泛型实参重载（`contains(List<String>)` 与 `contains(List<Integer>)` 擦除后签名冲突）。 – 由于擦除，`List<String>` 与 `List<Integer>` 的 `getClass()` 返回同一个类；把 `List<String>` 强转成 `List<Integer>` 不会在转换处抛异常，而是在之后取元素时才抛——这就是泛型时代仍不能随意强转的原因。 – 通配符的使用惯例（PECS：Producer Extends, Consumer Super）：只从结构中**读取**元素时用 `? extends E`，只**写入**元素时用 `? super E`。 – 擦除还解释了为什么某些 API 必须显式传入 `Class` 对象（例如 `EnumMap` 需要 `Direction.class`）：运行时拿不到类型实参，只能靠一个显式的类对象来弥补。 – 这些限制**不影响**「接口 + 泛型定义 ADT」的建模能力：`Set<E>` 依然服务于一整族元素类型，只是实现内部要用集合或 `Object[]` 规避 `new E[]`。

```
/** 补充说明：擦除带来的常见限制。 */
class ErasureDemo<E> {

    // 不能写 new E[10]；只能用 Object[] 加转换，并且会得到 unchecked 警告
    @SuppressWarnings("unchecked")
    private E[] elements = (E[]) new Object[10];

    private int n = 0;

    /** @param e an element; requires n < elements.length */
    void add(E e) { elements[n++] = e; }

    /** @param index an index @return the element at index */
    E get(int index) { return elements[index]; }

    static void demo() {
        ErasureDemo<String> a = new ErasureDemo<>();
        ErasureDemo<Integer> b = new ErasureDemo<>();
        // 运行时看不到类型实参：两者的类对象完全相同
        System.out.println(a.getClass() == b.getClass());   // true

        Object obj = a;
        @SuppressWarnings("unchecked")
        ErasureDemo<Integer> lie = (ErasureDemo<Integer>) obj;  // 转换处不报错
        // lie.add(42);                                          // 之后才可能出现 ClassCastEx
    }

    /** 补充说明：通配符——只读取元素的“生产者”视角，可接受 List<? extends Number>。 */
    static double sumOf(java.util.List<? extends Number> numbers) {
        double sum = 0;
        for (Number x : numbers) { sum += x.doubleValue(); }
        return sum;
    }
}
```

sp22 原版 TypeScript 与 Java 在**子类型判定方式**上有一处根本差异，必须对照：TypeScript 的接口是**结构化类型（structural typing）**，只要一个类提供了接口要求的所有操作，即使它从未写过 `implements`，编译器也认为它是该接口的子类型（sp22 原文还专门用「结构子类型化」一节讨论它）；Java 是**名义类型（nominal typing）**，必须在 `implements` / `extends` 子句里显式声明，否则编译器完全不认。

```
// sp22 原版 TypeScript 写法：结构子类型化（structural subtyping）
interface MyString {
  length(): number;
  charAt(i: number): string;
  substring(start: number, end: number): MyString;
}

// 注意：这个类**没有**写 implements MyString
class AccidentalString {
  private a: string;
  public constructor(s: string) { this.a = s; }
  public length(): number { return this.a.length; }
  public charAt(i: number): string { return this.a.charAt(i); }
  public substring(start: number, end: number): MyString {
    return new AccidentalString(this.a.substring(start, end));
  }
}

// 合法：TypeScript 只看结构，AccidentalString 是 MyString 的结构子类型
const s: MyString = new AccidentalString("good morning");
```

```
// Java 对应写法：名义子类型化（nominal subtyping）
// 必须显式声明 implements；少写这一句就是静态错误，而不是“结构兼容”。
public class AccidentalString implements MyString {
    private final String a;

    public AccidentalString(String s) { this.a = s; }

    @Override public int length() { return a.length(); }
    @Override public char charAt(int i) { return a.charAt(i); }
    @Override public MyString substring(int start, int end) {
        return new AccidentalString(a.substring(start, end));
    }
}
```

结构化类型很方便，但会**在类型安全上开一个洞**：它允许 B 成为 A 的子类型，即使 B 的规格与 A 不兼容。sp22 原文用可变性举例——`Array` 是 `ReadonlyArray` 的结构子类型，所以可以写 `const readonlyArr: ReadonlyArray<number> = [1, 2, 3];` 把数组当作不可变值使用（反方向 `const arr: Array<number> = readonlyArr;` 是静态错误，因为 `ReadonlyArray` 不提供变异操作）；但如果同时保留了可变别名 `arr`，一句 `arr.push(4)` 就把 `ReadonlyArray` 的不可变性彻底破坏了——这说明 `Array` 只是**结构上的**子类型，**不是真正的**（规格意义上的）子类型。**补充说明**：Java 中没有结构子类型化，所以这一具体漏洞不会以同样形式出现，但有个高度相似的陷阱——`Collections.unmodifiableList(list)`（原文在集合接口那一节用过它）只返回一个不可修改的**视图**，原 `list` 一旦仍被任何别名持有并可改写，视图的内容就会跟着变；想真正不可变，应使用 `List.of(...)` / `List.copyOf(...)` 或丢弃全部可变别名（这与 Reading 08 不可变性的要求一致）。

枚举（Enumerations） – **定义与目的**：有些 ADT 只有**很小、有限、不可变**的一组取值：一年十二个月、一周七天、罗盘四个方向、线段端帽的 butt/round/square。把每个取值定义为命名常量，就叫枚举（enumeration）。Java 的 `enum` 不只是「一堆常量」——它定义了一个**新的类型名**（和 class、interface 一样），并且是**真正的类**：可以有字段、构造器、方法，因而也能有自己的 rep、抽象函数与表示不变量。 – **直观解释（"它是什么？"）**：枚举像一副只有固定几张的牌。你可以说「方块 K」，但你不能自己印一张新牌（客户端没有可用构造器）；也正因为只有一副牌、每张牌在内存里只有一个对象，判断两张牌是不是同一张，用「是不是同一张牌」（`==`）就足够了。 – **关键规则与最佳实践**： – `public enum Month { JANUARY, FEBRUARY, MARCH, ..., DECEMBER };` 定义了类型名 `Month` 与一组命名值；这些值实质上是 `public static final` 常量，所以按全大写命名。原文的另一个例子是 `public enum Semester { IAP, SPRING, SUMMER, FALL };`。 – 枚举值**天生不可变且唯一**：客户端没有构造器可用，无法制造新实例；因此 `==` 与 `equals()` 等价，而且 `==` **更 fail-fast**——它在编译期检查两侧是同一个枚举类型，而 `equals()` 要到运行时才发现类型不同。 – 与之对照，如果 `day` 用 `String` 表示，那么 `day == SATURDAY` 就非常不安全：`==` 判断的是两个表达式是否引用内存中同一个对象，而两个内容相同的 `"Saturday"` 未必是同一个对象，所以对象比较必须用 `equals()`；枚举没有这个问题。 – 枚举可以用在 `switch` 中（`switch` 本来只接受整型、其包装类型与 `String`，不接受其他对象），这让「按取值分派」写起来清晰又安全；原文的 `Month.startOfNextSemester()` 与 `switch (direction) { case NORTH: ... }` 都是实例。 – 枚举比 `int` 常量有更多静态检查：`Month firstMonth = MONDAY;` 是静态错误，因为 `MONDAY` 的类型是 `DayOfWeek` 而不是 `Month`。 – 枚举变量**可以为 `null`**（它仍是对象引用），必须像其他对象类型一样防范空引用。 – 枚举可以有 rep、观察者与生产者：`private Month(int daysInMonth)` 私有构造器为每个常量初始化字段；此外还有一个自动的、不可见的 `ordinal` 字段（取值 0、1、...）。 – 自动提供的操作（由 `Enum` 定义）：`ordinal()` 返回值在枚举中的下标（`JANUARY.ordinal()` 是 0）、`compareTo()` 按 ordinal 比较两个值、`name()` 返回常量名字符串（`JANUARY.name()` 是 `"JANUARY"`）、`toString()` 与 `name()` 行为相同。**补充说明**：Java 语言规范还自动提供静态方法 `values()`（按声明顺序返回全部取值）与 `valueOf(String)`（按名字取常量，名字非法时抛 `IllegalArgumentException`）。 – 枚举可以自成文件 `Month.java`，也可以作为另一个类型的从属声明：原文指出若某个 `Date` 类型需要它，写成 `Date` 内部的 `public enum Month`，外部客户端用 `Date.Month` 引用。 – 原文的结论很直接：相比「特殊整数值」或「特殊字符串」这些老办法，枚举让代码更安全（静态检查拒绝集合外的值与混用类型）、更好懂（命名常量不神秘，命名类型比 `int` / `String` 更能自我说明）、更易改（可以按枚举类型名搜索出所有使用点，IDE 还能自动重构）。

```
/** 一个真正的枚举类：有 rep、有观察者、有生产者（sp21 原文第 1540 行起）。 */
public enum Month {
    // the values of the enumeration, written as calls to the private constructor below
    JANUARY(31),
    FEBRUARY(28),
    MARCH(31),
    APRIL(30),
    MAY(31),
    JUNE(30),
    JULY(31),
    AUGUST(31),
    SEPTEMBER(30),
    OCTOBER(31),
    NOVEMBER(30),
    DECEMBER(31);

    // rep
    private final int daysInMonth;

    // enums also have an automatic, invisible rep field:
    // private final int ordinal;
    // which takes on values 0, 1, ... for each value in the enumeration.

    // rep invariant:
    //   daysInMonth is the number of days in this month in a non-leap year
    // abstraction function:
    //   AF(ordinal, daysInMonth) = the (ordinal+1)th month of the Gregorian calendar
    // safety from rep exposure:
    //   all fields are private, final, and have immutable types

    // Make a Month value. Not visible to clients, only used to initialize the constants ab
    private Month(int daysInMonth) {
        this.daysInMonth = daysInMonth;
    }

    /**
     * @param isLeapYear true iff the year under consideration is a leap year
     * @return number of days in this month in a normal year (if !isLeapYear)
     *         or leap year (if isLeapYear)
     */
    public int getDaysInMonth(boolean isLeapYear) {
        if (this == FEBRUARY && isLeapYear) {
            return daysInMonth + 1;
        } else {
            return daysInMonth;
        }
    }

    /**
     * @return first month of the semester after this month
     */
    public Month startOfNextSemester() {
        switch (this) {
```

```
            case JANUARY:
                return FEBRUARY;
            case FEBRUARY:   // cases with no break or return
            case MARCH:      // fall through to the next case
            case APRIL:
            case MAY:
                return JUNE;
            case JUNE:
            case JULY:
            case AUGUST:
                return SEPTEMBER;
            case SEPTEMBER:
            case OCTOBER:
            case NOVEMBER:
            case DECEMBER:
                return JANUARY;
            default:
                throw new RuntimeException("can't get here");
        }
    }
}

/** 原文的另一个枚举：用于注册选学期。 */
enum Semester { IAP, SPRING, SUMMER, FALL }

/** 罗盘方向；原文用它演示 switch 与枚举值。 */
enum Direction { NORTH, SOUTH, EAST, WEST }

class EnumDemo {
    /** @param day a day of the week @return true iff it is a weekend day */
    static boolean isWeekend(java.time.DayOfWeek day) {
        // 每个枚举值在内存中只有一个对象，所以 == 与 equals 等价，而且更 fail-fast
        return day == java.time.DayOfWeek.SATURDAY || day == java.time.DayOfWeek.SUNDAY;
    }

    /** @param direction a compass point @return the animal you would meet there */
    static String animalAt(Direction direction) {
        switch (direction) {
            case NORTH: return "polar bears";
            case SOUTH: return "penguins";
            case EAST:  return "elephants";
            case WEST:  return "llamas";
            default:    throw new AssertionError("unreachable");
        }
    }

    static void demo() {
        System.out.println(Month.JANUARY.ordinal());              // 0
        System.out.println(Month.JANUARY.name());                 // "JANUARY"
        System.out.println(Month.FEBRUARY.getDaysInMonth(true));  // 29
        System.out.println(isWeekend(java.time.DayOfWeek.SATURDAY));  // true
        System.out.println(animalAt(Direction.SOUTH));            // "penguins"
        // Month firstMonth = java.time.DayOfWeek.MONDAY;         // 静态错误：类型不同
```

```
    }
}
```

原文还用「报名选学期」的例子比较了三种方案：直接传字符串字面量 `startRegistrationFor("Fall", 2023)`（客户端拼错 `"FAll"` 也得不到静态错误，不能 fail fast）、用 `public static final String FALL = "Fall"` 命名常量（常量可被重新赋值，也拦不住客户端传 `"Autumn"`）、用 `public enum Semester { IAP, SPRING, SUMMER, FALL }`（客户端无法新增学期值，也无法把 `Month.JANUARY` 传进来——那是静态错误）。

`EnumSet` 与 `EnumMap`——补充说明（超出 6.031 原文的 Java 生态知识） – **定义与目的**：`EnumSet<E extends Enum<E>>` 与 `EnumMap<K extends Enum<K>, V>` 是 Java 标准库为枚举专门设计的集合与映射。**6.031 原文完全没有提到它们**——原文关于枚举只讲了 `enum` 本身（`Month`、`Semester`）以及它相比字符串/整数常量的优势。之所以值得补充，是因为枚举的取值既然只有固定几个、还自带 `ordinal` 编号，就没有必要为它们付出通用哈希容器的代价。 – **直观解释（"它是什么？"）**：`EnumSet` 内部是按 ordinal 索引的**位向量**（一个 `long` 可存 64 个取值），`EnumMap` 内部是按 ordinal 索引的**数组**。这就像点名册上只有固定的 4 个名字：你只需要 4 个勾选框，而不是一个能写任何名字的大本子再去找哪一页写了谁。 – **关键规则与最佳实践**： – 集合用法：`EnumSet.of(a, b, ...)`、`EnumSet.allOf(Semester.class)`、`EnumSet.noneOf(Semester.class)`、`EnumSet.range(a, b)`、`EnumSet.complementOf(set)`；遍历顺序固定为枚举的声明顺序，测试与调试因此可复现。 – 映射用法：构造时必须传入枚举类型对象 `new EnumMap<>(Direction.class)`（**类型擦除**导致运行时无法从 `EnumMap<Direction, V>` 推断出 `Direction`），键按 ordinal 顺序迭代，且不接受 `null` 键。 – 相对 `HashSet<String>` 的优势：不必为每个取值保存字符串对象、不必计算字符串哈希、也不会被任意字符串污染或混入另一个枚举类型；`EnumSet`/`EnumMap` 的类型安全由编译器保证。 – 相对 `boolean[]` 的优势：`EnumSet` 是一个真正的 `Set`，带规格、带类型名，不会因为枚举声明顺序调整而静默错位。 – 性能优势不代表可以牺牲封装：把 `EnumSet` 当字段时仍不要把它本身返回给客户端（那是表示暴露），应返回 `Collections.unmodifiableSet(...)` 或副本。 – `EnumSet`/`EnumMap` 都不是线程安全的，并发环境下需要外部同步（见 Reading 21 并发与 Reading 23 互斥）。 – 回到原文的结论：**「取值集合小而固定 → 用枚举」**这一步是课程内容；「枚举的集合用 `EnumSet` 而不是 `HashSet`」这一步是本补充说明。

```
// 补充说明：以下 API 超出 6.031 原文范围。
import java.util.Collections;
import java.util.EnumMap;
import java.util.EnumSet;
import java.util.Set;

/** 原文的 Semester 枚举 + 补充的 EnumSet 用法。 */
enum Semester { IAP, SPRING, SUMMER, FALL }

class RegistrationOffice {

    /** EnumSet：位向量实现，类型安全，迭代顺序固定为声明顺序。 */
    private final EnumSet<Semester> offered =
            EnumSet.of(Semester.IAP, Semester.SPRING, Semester.FALL);

    /**
     * @param semester the semester to register for
     * @param year the calendar year
     * @throws IllegalArgumentException if semester is not offered this year
     */
    public void startRegistrationFor(Semester semester, int year) {
        if (!offered.contains(semester)) {
            throw new IllegalArgumentException("semester not offered: " + semester);
        }
        System.out.println("Registering for " + semester + " " + year);
        // 对比：客户端无法传 "FAll" 或 Month.JANUARY —— 都是静态错误
    }

    /** @return an unmodifiable view of the semesters offered this year */
    public Set<Semester> semestersOffered() {
        return Collections.unmodifiableSet(offered);   // 不暴露 rep
    }
}

/** EnumMap：为枚举键优化的映射，内部就是按 ordinal 索引的数组。 */
class DirectionSurvey {

    /** 构造时必须显式传入枚举类型对象（类型擦除）。 */
    private final EnumMap<Direction, Integer> counts = new EnumMap<>(Direction.class);

    DirectionSurvey() {
        for (Direction d : Direction.values()) {   // values()：按声明顺序的全部取值
            counts.put(d, 0);
        }
    }

    /** @param d the direction observed; modifies this survey by counting one more observat
    void record(Direction d) {
        counts.put(d, counts.get(d) + 1);
    }

    /** @param d a direction @return the number of observations recorded in direction d */
    int count(Direction d) {
```

```
        return counts.get(d);       // 每个枚举值都已建键，不会返回 null
    }
}
```

抽象类与接口的取舍（Abstract Class vs Interface）——补充说明（6.031 原文没有单独一节讲抽象类） – **定义与目的**：抽象类（`abstract class`）不能实例化，可以包含抽象方法也可以包含具体实现与字段；接口只有规格、没有 rep。当多个类型**需要共享同一份实现或同一份字段**时用抽象类；当只需要共享**类型（规格）**、并且希望一个类能同时是多个抽象类型的成员时用接口。Java 8 引入 `default` 方法后两者能力接近，但接口仍不能持有实例字段、不能有构造器。 – **直观解释（"它是什么？"）**：接口像职业资格证：一个人可以同时是会计师和律师（多实现），但证件本身不发工资、不给你办公室。抽象类像家族企业：你继承它的资产与员工（rep 与方法体），但一个孩子只能继承一家的家业（单继承），而且你的经营方式会牵动整个家族（表示依赖）。 – **关键规则与最佳实践**： – **优先用接口**：sp21 Reading 12 的「子类化」一节说明了继承 rep 会带来表示暴露、表示依赖、父子互相破坏 RI 三个问题；课程在 sp21 问题集 2 中给出明确态度——「`IntervalSetTest` 是一个抽象类。抽象类与子类化有它们的用处，但一般应被避免。」换句话说，抽象类只在你确实要**共享实现**时才值得付出继承 rep 的代价。 – 抽象类的判断信号：多个实现要共享同一段方法体或同一批字段，而且这种共享是**表示层面**的共享；问题集 2 的 `IntervalSetTest` 正是这种用法：所有测试方法共享，唯一留给子类填空的是 `protected abstract IntervalSet<String> emptyInstance();`。 – 接口的判断信号：你需要**多重类型**（一个类同时是 widget 与 list）、或者只关心规格而不希望客户端受到任何表示影响——这是本讲的主线用法。 – 判断表的其余维度：接口不能有实例字段、不能有构造器；抽象类可以两者都有，还可以有 `protected` 成员。接口支持多实现，抽象类只支持单继承（但可以「继承一个抽象类 + 实现多个接口」）。 – 若接口的 `default` 方法开始承担大量实现，就该反问「这其实是不是想共享 rep？」——若是，考虑抽象类，或把共享实现抽成 helper 类，让实现类**组合**它而不是继承它（组合不会带来表示依赖）。 – 原文也提醒：接口继承（`SortedSet<E> extends Set<E>`）只会加强规格或新增操作，不会带来任何实现；想共享实现只能靠抽象类或 `default` 方法。

维度	抽象类（abstract class）	接口（interface）
实例字段 / rep	可以有（含 private 字段）	不能有（字段自动是 `public static final` 常量）
构造器	可以有，子类用 `super(..)` 调用	不能有
方法实现	普通方法 + 抽象方法	只有 `default` 与 `static` 方法可有方法体
继承数量	单继承（`extends` 一个类）	多实现（`implements` 多个接口）
建立的关系	子类化 + 子类型（**继承 rep**）	纯子类型（不继承 rep）
规格的位置	通常分散在类与其子类中	集中在接口里，客户端只读它
主要风险	表示暴露、表示依赖、父子互相破坏 RI	几乎没有（接口没有 rep）

```java
/** 问题集 2 的真实结构：IntervalSet 是接口，两个实现类各有自己的 rep。 */
public interface IntervalSet<L> {

    /** @param label a label @param start start of the interval @param end end of the inter
    public void insert(L label, long start, long end);

    /** @param label a label @return true iff this set contains label */
    public boolean contains(L label);

    /** @return a new, empty interval set; 客户端不必知道实现类是哪一个 */
    public static <L> IntervalSet<L> empty() {
        return new RepMapIntervalSet<L>();
    }
}

/** 一个实现：用 Map 做 rep（ps2 指定了它的表示）。 */
class RepMapIntervalSet<L> implements IntervalSet<L> {
    private final java.util.Map<L, long[]> intervals = new java.util.HashMap<>();
    @Override public void insert(L label, long start, long end) {
        intervals.put(label, new long[] { start, end });
    }
    @Override public boolean contains(L label) { return intervals.containsKey(label); }
}

/** 抽象类的合适用法：共享同一套测试逻辑，子类只填一个“空实例”钩子。 */
abstract class IntervalSetTest {

    /** @return a new empty instance of the implementation under test */
    protected abstract IntervalSet<String> emptyInstance();

    /** 共享的测试逻辑：对任何实现都成立。 */
    public void testInitialLabelsEmpty() {
        IntervalSet<String> set = emptyInstance();
        assert !set.contains("labelA");
    }
}

/** 子类只填空，从而把同一套测试跑在不同实现上。 */
class RepMapIntervalSetTest extends IntervalSetTest {
    @Override protected IntervalSet<String> emptyInstance() {
        return new RepMapIntervalSet<String>();
    }
}
```

用接口与实现类定义 ADT（Defining an ADT with an Interface and Implementation Classes） – 定义与目的：这是本讲的主线结论：ADT = **接口（操作集合 + 规格）** + **一个或多个实现类（表示 + AF + RI + 方法体）**。接口回答「这个类型能做什么、承诺了什么」，实现类回答「用什么数据表示、代码怎么写」，两者之间只通过规格相连。 – **直观解释（"它是什么？"）**：接口是 ADT 的「合同文本」，实现类是「按合同施工的施工

队」。`MyString` 接口说「你能取长度、按位置取字符、取子串」，`FastMyString` 说「我用 `char[]` 加起止下标施工，`substring` 不用复制字符」；`Set` 接口说「你能加入、查询、删除」，`HashSet` 说「我用哈希表施工，查找平均 O(1)」。客户端拿着合同办事，不关心哪家施工队。 – **关键规则与最佳实践**： – 接口里写操作与规格：`length()`、`charAt(int i)`、`substring(int start, int end)`；**绝不**提「数组」「下标」等表示细节，否则接口就不再 representation-independent（原文的 `Curve` 如果让 `join` 返回 `ArrayCurve`，就同时犯了「依赖实现类」与「不 representation-independent」两个错误）。 – 实现类里写 rep、AF、RI、`checkRep()`、`toString()`，并用 `@Override` 标注每个实现方法（原文的 `FastMyString` / `SimpleMyString` 都这样写）。 – 客户端应当只依赖接口类型：`MyString s = MyString.valueOf(true);`、`Set<String> strings = Set.make();`，而不再提 `FastMyString`、`SimpleSet`；更好的做法是用静态工厂连实现类名都藏掉。 – 当规格**故意欠定**时，接口应保持宽松，实现可以自行加强（但不能削弱）：`Set.pick()` 只承诺返回集合中某个元素，`SimpleSet` 可以承诺「返回最近加入且尚未移除的元素」。 – 同一个 ADT 的多个实现可以有显著不同的性能与互信度取舍：一个简单到显然正确的实现，和一个更快但更可能含 bug 的精致实现可以并存，让应用按「被 bug 咬到有多痛」来选择；原文的 `ArrayList` 与 `LinkedList` 就是库层面的例子。 – 原文的 ADT 概念总表（Java 版）值得记住：ADT 可以由单一类（`String`）、「接口 + 类」（`List` / `ArrayList`）、或 `enum`（`DayOfWeek`）实现；创建者可以是构造器 `ArrayList()`、静态工厂方法 `List.of()`、常量 `BigInteger.ZERO`；观察者可以是实例方法 `List.get()` 或静态方法 `Collections.max()`；生产者可以是 `String.trim()` 或 `Collections.unmodifiableList()`；变异者可以是 `List.add()` 或 `Collections.copy()`；而表示永远是 `private` 字段。

```
/** 接口定义 ADT 的操作集合；规格里不能出现任何表示细节。 */
public interface LabeledSet {

    /** @param label a label @return true iff this set contains label */
    public boolean contains(String label);

    /** @param label a label; modifies this set by adding label if not already present */
    public void add(String label);

    /** @param label a label; modifies this set by removing label if present, else no effec
    public void remove(String label);

    /** @return the number of distinct labels in this set */
    public int size();
}

/** 实现一：哈希表，查找平均 O(1)，元素无序。 */
class HashSetLabeledSet implements LabeledSet {
    private final java.util.Set<String> labels = new java.util.HashSet<>();
    @Override public boolean contains(String label) { return labels.contains(label); }
    @Override public void add(String label) { labels.add(label); }
    @Override public void remove(String label) { labels.remove(label); }
    @Override public int size() { return labels.size(); }
}

/** 实现二：有序树，单次操作 O(log n)，但可直接产出有序序列。 */
class TreeSetLabeledSet implements LabeledSet {
    private final java.util.Set<String> labels = new java.util.TreeSet<>();
    @Override public boolean contains(String label) { return labels.contains(label); }
    @Override public void add(String label) { labels.add(label); }
    @Override public void remove(String label) { labels.remove(label); }
    @Override public int size() { return labels.size(); }
    /** @return the labels in lexicographic order */
    public java.util.List<String> sortedLabels() {
        return new java.util.ArrayList<>(labels);
    }
}

class LabeledSetDemo {
    static void demo() {
        // 客户端只依赖接口：换实现只需改这一行
        LabeledSet set = new HashSetLabeledSet();
        set.add("glorp");
        set.add("glorp");
        System.out.println(set.size());        // 1

        // Java 集合类的对应事实：接口 + 多个实现
        java.util.Set<String> hashed = new java.util.HashSet<>();               // 无序，平均
        java.util.Set<String> sorted = new java.util.TreeSet<>();               // 有序，O(l
        java.util.Set<String> insertionOrder = new java.util.LinkedHashSet<>(); // 保持插入顺
        hashed.add("b"); sorted.add("b"); insertionOrder.add("b");
        System.out.println(hashed.size() + sorted.size() + insertionOrder.size());
```

```
    }
}
```

原文最后提醒：**数据抽象不依赖语言特性**。C 语言里没有类、方法、字段，甚至没有 `private`，但表示独立性依然可以实现——`FILE` 类型配合 `fopen`、`fputs`、`fclose` 就是「不透明类型 + 一组全局函数」的 ADT：客户端没有任何办法窥视 `FILE` 内部，只能把 `FILE` 交给这些操作使用（`fputs` 甚至把文件放在第二个参数）。这说明 ADT 是一种设计模式，而不是某个语言的语法糖。

代码示例与对比分析

场景 1：客户端必须知道具体实现类的名字才能创建对象，抽象屏障被打破

❌ *错误代码*

```
// 错误：客户端直接依赖具体表示类 FastMyString / SimpleMyString
class MyStringClient {

    static void demo() {
        // 客户端必须知道表示类的名字，还必须知道构造器参数代表什么含义
        FastMyString s = new FastMyString(true);     // "true"
        System.out.println("The first character is: " + s.charAt(0));

        // 另一个客户端挑了另一个表示类，两段代码无法互换
        SimpleMyString t = new SimpleMyString(false); // "false"
        System.out.println("length = " + t.length());

        // 更糟的是：两个实现的构造器签名根本不同，
        // 接口里也没有任何地方承诺“所有实现都提供同样的构造器”
    }
}
```

【错误代码的问题】 1. **抽象屏障被打破**：客户端代码里出现了 `FastMyString` 这个名字，从此任何「换实现」的改动都要改所有客户端，Ready for change 直接失效。 2. **没有静态保证**：接口里根本没有构造器，因此「所有实现都提供同样的构造器」这件事没有任何编译期保证——原文的 `FastMyString(boolean)` 与 `SimpleMyString(char[])`（私有构造器）就是反例。 3. **客户端可能意外依赖实现细节**：一旦客户端拿到具体类型，就很容易调用实现类独有的方法（例如 `ArrayCurve.add(x, y)`），此后这些细节都变成事实上的公开接口。 4. **测试与替换变难**：无法在不改客户端的前提下，用一个更简单（更可能正确）的实现替换性能更好的实现来做对照测试。

✅ *正确代码*

```
/** 客户端只依赖接口 MyString；创建者操作是接口上的静态工厂方法。 */
class MyStringClient {

    static void demo() {
        MyString s = MyString.valueOf(true);          // 只出现接口名
        System.out.println("The first character is: " + s.charAt(0));   // 't'
        System.out.println("The whole string is: "
                + s.substring(0, s.length()));        // "true"

        // 客户端无法写 new FastMyString(..)，因为它只见过 MyString 这个类型
    }
}
```

【为什么这样更好】 客户端只引用接口类型与接口的静态工厂方法，实现类名 `FastMyString` 只出现在接口内部（`return new FastMyString(b);`）。此后要把默认实现换成 `SimpleMyString` 或第三种表示，只需改接口里的那一行工厂代码，所有客户端零改动；同时接口里没有字段，客户端也无从依赖任何表示细节。原文的对应写法是「在 `MyString` 接口里加 `public static MyString valueOf(boolean b)`」，sp22 原版则在 TypeScript 里用工厂函数 `function makeMyString(s:string):MyString` 达成同一效果。

【代码对比解说】 两种写法的类型检查都不弱：错误代码也能编译、也能运行。差别在于**依赖的方向**。错误写法让客户端依赖「具体表示类」，正确写法让客户端依赖「规格」。这正是「接口是客户端唯一需要读的东西」这条原则的落地方式：客户端读 `MyString` 就知道全部契约，且无法越过契约去触碰实现。原文还提醒这是一种**取舍**：完全隐藏实现意味着客户端无法按性能挑实现；Java 库因此同时暴露 `ArrayList` 与 `LinkedList`，让需要选择性能的客户端仍有选择权——但即便如此，规格仍写在 `List` 接口里。

【设计原则透视】 这一组对比直接对应 **抽象屏障（abstraction barrier）**与 **表示独立性（representation independence）**：接口不含 rep，所以「接口 + 静态工厂」是唯一能同时满足「客户端只看规格」和「实现可替换」的设计。与 Reading 11（抽象函数与表示不变量）呼应：AF/RI 只存在于实现类里，客户端看不到；与 Reading 10（抽象数据类型）呼应：ADT 由操作定义，接口正是「操作集合」的代码化。静态工厂方法还额外解决了「接口不能有构造器」这一语言限制。

场景 2：一个可变的「正方形也是矩形」违反了 Liskov 替换原则

❌ *错误代码*

```
/** A mutable rectangle. */
interface MutableRectangle {
    int getWidth();
    int getHeight();
    /** Set this rectangle's dimensions to width x height. */
    void setSize(int width, int height);
}

// A mutable square: 原文写作 implements MutableRectangle /* hopefully? */
class MutableSquare implements MutableRectangle {
    private int side;

    MutableSquare(int side) { this.side = side; }

    @Override public int getWidth() { return side; }
    @Override public int getHeight() { return side; }

    /**
     * Set this square's dimensions to width x height.
     * Requires width = height.                    // ← 更强的前置条件：违反 LSP
     */
    @Override public void setSize(int width, int height) {
        if (width != height) {
            throw new IllegalArgumentException("width != height");
        }
        this.side = width;
    }
}

class RectangleClient {
    /** 把所有矩形放大一倍——这段代码在 MutableSquare 上会崩溃。 */
    static void doubleSize(MutableRectangle r) {
        r.setSize(r.getWidth() * 2, r.getHeight() * 2);   // 可能违反正方形的前置条件
    }

    /** 只能靠 instanceof + 强转“补救”，这就是设计坏味道。 */
    static void doubleSizeSafely(MutableRectangle r) {
        if (r instanceof MutableSquare) {
            MutableSquare s = (MutableSquare) r;          // 运行时强转：危险信号
            s.setSize(s.getWidth() * 2);
        } else {
            r.setSize(r.getWidth() * 2, r.getHeight() * 2);
        }
    }
}
```

【错误代码的问题】 1. **违反 LSP**：`MutableSquare` 的前置条件比 `MutableRectangle` 更强（`setSize` 只接受 `width == height`），于是「凡是 `MutableRectangle` 都能任意设置宽高」这一承诺被破坏，替换后客户端逻辑出错。 2. **强迫客户端写运行时分派**：`instanceof` + 强转是「本来就不该是子类型」的典型症状；每逢新增一个「特殊矩形」都要再改一次这段分派逻辑，Ready for change 变差。 3. **错误延迟到运行时**：前置条件被加强时编译器不会报错（它只检查签名），错误只会在某次 `setSize(3, 5)` 调用时以异常或错误

结果暴露——不符合 fail-fast。 4. **破坏规格的单一事实来源**：接口说「设成 width x height」，实现却偷偷附加了条件，读接口的人无法预知；这也会让后续依赖规格的逻辑（如 Reading 15 的 `equals`）出错。

✅ *正确代码*

```
/** An immutable rectangle: 不可变类型没有 setSize，因此不存在“改尺寸”的分歧。 */
public interface ImmutableRectangle {
    /** @return the width of this rectangle */
    public int getWidth();
    /** @return the height of this rectangle */
    public int getHeight();
}

/** An immutable square: 每个正方形都真的是矩形，规格被完整满足。 */
class ImmutableSquare implements ImmutableRectangle {
    private final int side;
    /** Make a new side x side square. */
    public ImmutableSquare(int side) { this.side = side; }
    /** @return the width of this square */
    @Override public int getWidth() { return side; }
    /** @return the height of this square */
    @Override public int getHeight() { return side; }
}

class RectangleClient {
    /** 任何 ImmutableRectangle 都能安全参与这段计算，无需任何 instanceof。 */
    static int area(ImmutableRectangle r) {
        return r.getWidth() * r.getHeight();
    }
}

/**
 * 若确实需要可变的矩形与正方形，就不要让正方形冒充矩形：
 * 定义两个彼此独立的类型，各自提供符合自身规格的操作。
 */
interface MutableRectangle2 {
    int getWidth();
    int getHeight();
    /** Set this rectangle's dimensions to width x height. */
    void setSize(int width, int height);
}

interface MutableSquare2 {
    int getSide();
    /** Set this square's side length to side. */
    void setSide(int side);
}
```

【为什么这样更好】 不可变版本里，`ImmutableSquare` 的每个方法的后置条件都与 `ImmutableRectangle` 完全一致（规格相同，因而至少一样强），所以它确实是合法子类型：任何期望「能取宽高」的地方都能用正方形，而且不需要任何运行时判断。原文的三个判断练习也印证了这一点：`getWidth()` 与 `getHeight()`

都满足接口规格，因此整个 `ImmutableSquare` 满足 `ImmutableRectangle` 规格。当确实需要可变性时，正确做法是**承认它们不是同一个抽象类型**，各写各的规格，而不是用 `instanceof` 掩盖设计缺陷。

【代码对比解说】 关键差别不在于「正方形数学上是矩形」，而在于**可变性让规格发生冲突**：矩形承诺「可以任意设置宽高」，正方形承诺「宽恒等于高」，两者不可能同时成立。所以「正方形是不是矩形」这个问题的答案是「**取决于是否可变**」——这正是 LSP 与规格强弱规则的用武之地。原文为此给出了四种候选 `setSize` 规格，其中三种被判为不合法：加强前置条件（`Requires width = height`）、加强前置条件（抛出 `BadSizeException`）、削弱后置条件（非正方形时行为未指定）。第二种写法还展示了替代方案：把「共享的行为」放在接口（`getWidth` / `getHeight`），把「冲突的行为」拆到不同接口。

【设计原则透视】 这组对比把 LSP 与 Reading 07（设计规格）中「前置条件不能加强、后置条件不能削弱」的规则直接连了起来：`implements` 只保证签名，规格的强弱必须由人保证。`instanceof` + 强转则对应 Reading 15（相等性）里同样危险的「按运行时类型分支处理」模式——它是抽象边界失效的信号。它还说明 Reading 08（不可变性）为什么是消除 LSP 冲突的利器：不可变类型没有变异者，因而不存在「子类无法满足变异者规格」的问题。

场景 3：用 `String` 常量代替 `enum`，并用 `HashSet<String>` 而不是枚举集合

❌ *错误代码*

```
import java.util.HashSet;
import java.util.Set;

/** 错误：用字符串常量模拟“学期”这一有限取值集合（原文的三种方案之一）。 */
class RegistrationOffice {

    public static final String IAP = "IAP";
    public static final String SPRING = "Spring";
    public static final String SUMMER = "Summer";
    public static final String FALL = "Fall";

    private final Set<String> offered = new HashSet<>();

    RegistrationOffice() {
        offered.add(IAP);
        offered.add(SPRING);
        offered.add(FALL);
    }

    /** @param semester the semester name @param year the calendar year */
    public void startRegistrationFor(String semester, int year) {
        if (!offered.contains(semester)) {
            throw new IllegalArgumentException("semester not offered: " + semester);
        }
        System.out.println("Registering for " + semester + " " + year);
    }
}

class BadClient {
    static void demo() {
        RegistrationOffice office = new RegistrationOffice();
        office.startRegistrationFor("FAll", 2023);    // 拼错也照样编译通过，运行时才炸
        office.startRegistrationFor("Autumn", 2023);  // 客户端“发明”了一个新学期

        Set<String> weird = new HashSet<>();
        weird.add("Fall");
        weird.add("Fall ");                           // 多一个空格就是另一个元素
        System.out.println(weird.size());             // 2
    }
}
```

【错误代码的问题】 1. **不能 fail fast**：`"FAll"`、`"Autumn"` 这类错误字符串是**合法**的 `String`，编译器无法拒绝；错误只能在运行时通过 `IllegalArgumentException` 暴露，甚至可能悄悄写入数据库。原文明确指出字符串字面量方案「不能 fail fast」。 2. **命名常量并不安全**：`public static final String FALL = "Fall"` 虽然名字固定，却拦不住客户端直接传 `"Autumn"`；若常量没写 `final`，它还能被重新赋值（原文把这个当作该方案的缺点之一）。 3. **表示与性能错配**：`HashSet<String>` 需要为每个字符串计算哈希、保存字符串对象；任何字符串都能被放进集合，需要额外检查来维持「集合只含合法学期」这一不变量。 4. **重构困难**：要找出「所有使用学期的地方」只能用字符串搜索，`"Fall"` 与代码里其他用途的 `"fall"` 无法区分，IDE 重构也无从下手。

✅ *正确代码*

```
import java.util.EnumSet;
import java.util.Set;
import java.util.Collections;

/** 正确：取值集合小而固定，用枚举而不是字符串常量（原文的第三种方案）。 */
enum Semester { IAP, SPRING, SUMMER, FALL }

class RegistrationOffice {

    /**
     * EnumSet 用法属于「补充说明（超出 6.031 原文的 Java 生态知识）」；
     * 原文只讲 enum 本身。若严格只用原文知识，可改用 Set<Semester>。
     */
    private final EnumSet<Semester> offered =
            EnumSet.of(Semester.IAP, Semester.SPRING, Semester.FALL);

    /**
     * @param semester the semester to register for
     * @param year the calendar year
     * @throws IllegalArgumentException if semester is not offered this year
     */
    public void startRegistrationFor(Semester semester, int year) {
        if (!offered.contains(semester)) {
            throw new IllegalArgumentException("semester not offered: " + semester);
        }
        System.out.println("Registering for " + semester + " " + year);   // FALL
    }

    /** @return an unmodifiable view of the semesters offered this year */
    public Set<Semester> semestersOffered() {
        return Collections.unmodifiableSet(offered);      // 不暴露 rep
    }
}

class GoodClient {
    static void demo() {
        RegistrationOffice office = new RegistrationOffice();
        office.startRegistrationFor(Semester.FALL, 2023);        // IDE 可补全，不可能拼错
        // office.startRegistrationFor(Month.JANUARY, 2023);     // 静态错误：类型不匹配
        // office.startRegistrationFor("Fall", 2023);            // 静态错误：String 不是 Seme

        // 枚举的相等性：每值唯一，所以 == 与 equals() 等价，而且更 fail-fast
        Semester s = Semester.FALL;
        System.out.println(s == Semester.FALL);                  // true
        System.out.println(Month.JANUARY.ordinal());             // 0
        System.out.println(Month.JANUARY.name());                // "JANUARY"
    }
}
```

【为什么这样更好】 枚举把「取值集合」变成类型系统的一部分：客户端**不可能**传入 `"FAll"`（那不是 `Semester`），也不可能传入 `Month.JANUARY`（类型不同，静态错误），更不可能发明一个新学期（枚举没有客户端可见的构造器）。原文对枚举方案的总结正是这三点：静态检查使客户端不能使用有限集合之外的值、也不能混淆两个不同的枚举类型；命名常量与命名类型比 `int` / `String` 更能自我说明；使用某个枚举的代码可以按枚举类型名搜索出来，改动时有据可依。附带的好处是，若采用 `EnumSet` / `EnumMap`（**补充说明**），还能得到位向量/数组级性能、固定的迭代顺序与编译期类型安全，而不是 `HashSet<String>` 的字符串哈希开销。

【代码对比解说】 两者都能「表示四个学期」，但承担的保证完全不同。字符串方案把「合法性」放在**运行时检查**里（`offered.contains(..)` 就是那个补丁），而枚举方案把合法性放进**类型**里，运行时检查只剩「本学期是否开设」这一真正的业务条件。此外 `values()` / `valueOf()` 让「枚举全部取值」「按名字解析」都有标准答案，而字符串方案只能靠手工维护的数组或反射。

【设计原则透视】 这一组直接对应原文枚举一节的总结（Safe from bugs / Easy to understand / Ready for change 三条全部成立），并与 Reading 15（相等性）相关：枚举值唯一，所以 `==` 与 `equals()` 等价且更 fail-fast；而字符串必须用 `equals()`，因为 `==` 判的是「是否同一个对象」。`EnumSet` 那部分只是 Java 生态的延伸（已在代码注释与上一小节中标注为补充说明），不能当作 6.031 原文内容。

场景 4：不使用泛型，容器只能装 `Object`，取出时必须运行时强转

❌ *错误代码*

```
import java.util.ArrayList;
import java.util.List;

/** 错误：非泛型容器，元素类型完全不受检查。 */
class ObjectSet {

    private final List<Object> elements = new ArrayList<>();

    public void add(Object e) { elements.add(e); }

    public int size() { return elements.size(); }

    public boolean contains(Object e) { return elements.contains(e); }

    /** @param i an index @return the element at index i, as an Object */
    public Object pick(int i) { return elements.get(i); }
}

class BadSetClient {
    static void demo() {
        ObjectSet set = new ObjectSet();
        set.add("glorp");
        set.add(42);                          // 编译通过：整数混进了“字符串集合”
        set.add(new int[] { 1, 2 });          // 什么都装得下

        String s = (String) set.pick(1);      // 运行时 ClassCastException!
        System.out.println(s.length());
    }
}
```

【错误代码的问题】 1. **类型错误延迟到运行时**：`set.add(42)` 违反使用者意图却完全合法；`(String) set.pick(1)` 才在运行时抛 `ClassCastException`，异常位置离真正的错误源（`add` 那一行）很远，调试成本高。 2. **每个调用点都重复强转**：客户端必须牢记每个位置装的是什么类型，强转代码散布各处，一旦类型变化就要全局修改。 3. **规格无法表达**：「这是一个字符串集合」这件事在类型上无法表达，只能写在注释里；签名 `add(Object)` 对客户端毫无约束力。 4. **与泛型集合互操作时会出现 unchecked 警告**：把 `Object` 塞进 `List<String>` 需要强制转换，编译器只能给出警告而非错误，类型安全从此靠运气。

✅ *正确代码*

```
import java.util.ArrayList;
import java.util.List;
import java.util.NoSuchElementException;

/**
 * 正确：用原文的泛型 Set<E> 把“元素类型”变成规格的一部分。
 * A mutable set.
 * @param <E> type of elements in the set
 */
public interface Set<E> {

    /**
     * Make an empty set.
     * @param <F> type of elements in the set
     * @return a new set instance, initially empty
     */
    public static <F> Set<F> make() { return new SimpleSet<F>(); }

    /** @return the number of elements in this set */
    public int size();

    /** @param e an element @return true iff this set contains e */
    public boolean contains(E e);

    /** @param e element to add; modifies this set by adding e to the set */
    public void add(E e);

    /** @param e element to remove; if e is not found in the set, has no effect */
    public void remove(E e);
}

/**
 * 泛型实现：元素存在 List<E> 里，完全不关心 E 是什么。
 * Rep invariant: elementList has no repeated elements.
 * Abstraction function: AF(elementList) = the set of elements in elementList.
 * Safety from rep exposure: elementList 是 private final，从不返回给客户端。
 */
class SimpleSet<E> implements Set<E> {

    private final List<E> elementList = new ArrayList<>();

    @Override public int size() { return elementList.size(); }

    @Override public boolean contains(E e) { return elementList.contains(e); }

    @Override public void add(E e) { if (!contains(e)) { elementList.add(e); } }

    @Override public void remove(E e) { elementList.remove(e); }

    /**
     * Picks an element from the set.
     * @return the element most recently added but not yet removed
     * @throws NoSuchElementException if set is empty
```

```
     */
    public E pick() throws NoSuchElementException {
        if (elementList.isEmpty()) { throw new NoSuchElementException("empty set"); }
        return elementList.get(elementList.size() - 1);
    }
}

class GoodSetClient {
    static void demo() {
        Set<String> strings = Set.make();   // 编译器推断 Set<String>
        strings.add("glorp");
        strings.add("glorp");
        // strings.add(42);                 // 静态错误：int 不能转换为 String
        System.out.println(strings.size()); // 1

        // 取出元素不需要强转：泛型接口的签名已经给出类型
        List<String> copy = new ArrayList<>();
        copy.add(strings.contains("glorp") ? "glorp" : "none");
        System.out.println(copy.get(0));    // glorp
    }
}
```

【为什么这样更好】 泛型把「元素类型」提升为类型参数，于是「`Set<String>` 里不能放整数」成为**编译期**保证，客户端不需要任何强转，也就不可能出现 `ClassCastException`（这正是泛型引入的首要动机：在集合 API 里消灭强转）。同一份 `SimpleSet<E>` 实现同时服务于 `Set<String>`、`Set<Integer>` 等一整族类型，无需为每种元素类型重写。而 `java.util.HashSet<E> implements Set<E>` 正是 JDK 用泛型实现 `Set` 的真实方式。

【代码对比解说】 两种写法的差异可以从两个维度看。维度一，**谁来检查**：`Object` 版本把检查推给运行时（强转），泛型版本把检查留在编译期（类型参数），并且错误信息直接指向出错的那一行 `add(42)`。维度二，**规格在哪里**：`Object` 版本的规格只能靠注释描述「这个容器里装的是字符串」，泛型版本的规格写进了签名 `Set<String>`，是机器可读的。代价是泛型带来若干限制（类型擦除、不能 `new E[]`、不能 `e instanceof E`），但这些限制只影响**实现内部**，不影响客户端看到的抽象。

【设计原则透视】 这组对比把本讲三条主线一次串起：**接口**定义了 ADT 的操作与规格（`Set<E>` 只谈「元素」这一抽象概念，绝不提数组或链表）；**泛型**让规格带占位符类型，从而适配整族元素类型并保持静态类型安全；**子类型与 LSP** 则要求 `SimpleSet<E>` 的每个方法规格至少与 `Set<E>` 一样强——例如 `remove` 在元素不存在时**必须**「无副作用」地返回而不许抛异常（那会加强前置条件），而 `pick()` 把后置条件从「某个元素」加强为「最近加入的元素」则是合法的。`checkRep()` 与 AF/RI 注释继续承担 Reading 11 的职责，`assert` 则呼应 Reading 09（避免调试）中 fail-fast 的要求。

与其他设计原则的关联

- **与 Reading 06（规格说明 Specifications）、Reading 07（设计规格 Designing Specifications）**：接口就是规格的容器，本讲的一切判断（子类型是否合法、`default` 方法能否加、实现能否加强 `pick()` 的后置条件）都归结为「前置条件不能加强、后置条件不能削弱」。接口把「规格写在哪里」这个问题的答案固定下来：写在接口里，而不是散落在各实现中。

- **与 Reading 08（不可变性 Immutability）**：不可变类型天然规避了本讲最尖锐的一类 LSP 冲突（`MutableSquare implements MutableRectangle`）——没有变异者，就没有「子类无法满足变异者规格」的问题；sp22 的 `ReadonlyArray` / `Array` 结构子类型漏洞也提醒我们，真正不可变必须做到「没有可变别名」，这也是 Java 中 `Collections.unmodifiableList` 只提供视图、而 `List.of` / `List.copyOf` 更可靠的原因。
- **与 Reading 10（抽象数据类型 Abstract Data Types）**：本讲把 Reading 10 的「用类定义 ADT」升级为「用接口 + 实现类定义 ADT」，并首次让同一 ADT 的多种表示（`SimpleMyString` 与 `FastMyString`）**并存于同一程序**；ADT 概念与 Java 实现方式的总表也在本讲给出。
- **与 Reading 11（抽象函数与表示不变量 Abstraction Functions & Rep Invariants）**：AF/RI/`checkRep()` 从此只属于实现类；接口里既不能有字段也不该有表示细节，所以「表示泄漏」在接口层面被结构性地排除（但实现类里仍要自己守 RI，例如 `CharSet` 的 rep 是「`String` 中没有重复字符」）。
- **与 Reading 09（避免调试 Avoiding Debugging）与 Reading 13（调试 Debugging）**：枚举与泛型都是「把错误提前到编译期」的工具——这正是 fail-fast 的核心手段；当 bug 仍然发生、需要切片时，接口带来的清晰边界也让「哪些代码可能影响这个值」更容易判断。
- **与 Reading 15（相等性 Equality）**：本讲的枚举 `==` 与 `equals()` 等价（每值唯一），而普通可变类型必须区分行为相等与观察相等；子类型改写 `equals` 语义会破坏「凡是超类型都能安全放入集合」的保证。Reading 15 还引入了 `Bag<E>` 这个可变多重集 ADT（本笔记只在明确标注的补充示例中使用它）。
- **与 Reading 16（map/filter/reduce）**：泛型接口是那一讲的基础设施——`List<E>`、`Set<E>`、`Optional<E>` 都靠类型参数让高阶函数保持静态类型安全，函数作为参数时接口又成了「函数类型」（见 Reading 20 回调中的单方法接口）。
- **与 Reading 17（递归数据类型 Recursive Data Types）**：递归 ADT（如语法树、图）通常需要接口与多个实现类配合，本讲的子类型与动态分派是它们的组织方式，而泛型让树可以携带任意元素或结果类型。
- **与 Reading 21（并发 Concurrency）、Reading 23（互斥 Mutual Exclusion）、Reading 24（队列 Queues）**：接口让「同一个抽象类型的线程安全实现与非线程安全实现」可以并存互换（这本身就是本讲「多个实现 + 性能取舍」的实例），而本讲涉及的集合类（`HashSet`、`EnumSet`、`EnumMap`）都不是线程安全的，需要外部同步。
- **与 Reading 05（版本控制）**：枚举带来的可搜索类型名让「修改取值集合」变成一次可审计的全局重构，这与版本控制中的可追踪变更相辅相成。

关键要点

- **接口只写规格，实现类只写表示**：接口里不出现字段、不出现表示细节（原文的诊断：`Curve.join` 若返回 `ArrayCurve` 就既不 representation-independent 又造成循环依赖）；客户端只依赖接口类型，创建者操作优先用接口的静态工厂方法（`MyString.valueOf(boolean)`），这样换实现时客户端零改动。
- **子类型由规格定义，不由关键字定义**：`implements` / `extends` 只保证签名兼容；子类型必须「前置条件不更强、后置条件不更弱」。`ImmutableSquare implements ImmutableRectangle` 合法，`MutableSquare implements MutableRectangle` 却会加强 `setSize` 的前置条件而违法，此时应拆成两个独立类型，而不是用 `instanceof` + 强转打补丁。

- **泛型是「一份实现服务一族类型」的机制**：写 `interface Set<E>`、`class HashSet<E> implements Set<E>`（泛型实现）或 `class CharSet implements Set<Character>`（非泛型实现）；静态方法必须自己声明类型参数（`public static <F> Set<F> make()`，原文故意用 `F` 强调它是不同的类型参数），泛型实现只能依赖接口规格里显式承诺的性质（如 `Object` 的 `equals` / `hashCode`）。
- **取值集合小而固定时一律用 `enum` 而不是字符串/整数常量**：枚举是真正的类，可以有 rep、私有构造器与方法；每个值唯一因而 `==` 与 `equals()` 等价且更 fail-fast，也支持 `switch` 与 `ordinal()` / `compareTo()` / `name()` / `toString()`；用 `EnumSet` / `EnumMap`（**补充说明，超出 6.031 原文**）替代 `HashSet<String>` / `HashMap<String, V>` 可获得位向量/数组级性能、固定迭代顺序与编译期类型安全。
- **优先用接口，抽象类只在确实要共享实现或字段时才用**：继承 rep 会带来表示暴露、表示依赖与父子互相破坏 RI（sp21 问题集 2 甚至明确说抽象类与子类化「一般应被避免」）；Java 8 的 `default` / `static` 方法缩小了两者的差距，但接口仍不能有实例字段与构造器。

常见陷阱与注意事项

1. 在接口里声明实例字段来「顺便共享」表示 → 违反接口无 rep 的规则（字段自动成为 `public static final` 常量），一旦误以为它能当 rep 用就会写出无法编译或语义错误的代码，而真正的表示必须留在实现类里（如 `FastMyString` 的 `char[] a` 与 `start` / `end`）。
2. 让客户端写 `MyString s = new FastMyString(true);` 或 `Set<String> set = new HashSet<>();` → 抽象屏障被打破，而接口里没有构造器规格，`FastMyString(boolean)` 与 `SimpleMyString(char[])` 这样的构造器根本不可能被接口统一承诺，换实现必须改所有客户端，客户端还会进一步依赖实现类独有的方法。
3. 让可变子类「假装」实现超类的变异者操作（`MutableSquare implements MutableRectangle` 并给 `setSize` 加 `width == height` 前置条件）→ 违反 LSP，客户端对超类型编程时会在运行时崩溃或得到错误结果，编译器不会预警；原文的另外两种候选规格（抛出 `BadSizeException`、削弱后置条件）同样不合法，正确做法是拆成两个独立类型或用不可变类型。
4. 用 `instanceof` + 强制类型转换来「兼容」不同子类型 → 这是抽象层次设计失败的信号：每新增一个子类型都要改这段分派代码，强转错误只会在运行时以 `ClassCastException` 暴露；应把差异上移到接口规格或拆分为不同抽象类型。
5. 用 `String` / `int` 常量 + `HashSet<String>` / `HashMap<String, V>` 表示有限取值集合 → 客户端可以传拼错的字符串或任意整数而得不到静态错误（不能 fail fast），命名常量还能被重新赋值，方法签名无信息量，重构只能靠字符串搜索；应改用 `enum`（配合补充说明中的 `EnumSet` / `EnumMap`，注意 `EnumMap` 构造时必须传 `Direction.class`，因为擦除导致无法推断枚举类型）。
6. 以为泛型在运行时会保留类型实参、于是把 `List<String>` 强转成 `List<Integer>`、或写 `new E[10]` / `e instanceof E` → 由于类型擦除，前者不会在转换处报错而是在稍后取元素时才抛异常，后两者直接是编译错误；泛型容器的元素类型检查只发生在编译期，运行时仍可能因 unchecked 强转出现 `ClassCastException`（补充说明范畴）。

思考题（带答案）

问题 1：原文的 `ImmutableSquare implements ImmutableRectangle` 被判定为合法子类型，而 `MutableSquare implements MutableRectangle` 被判定为不合法。请用「前置条件/后置条件」与 LSP 的语言解释这两者的差别；并说明原文给出的四种候选 `MutableSquare.setSize` 规格各自属于哪一种情况、为什么都不合法（或哪一种才合法）。

答案：子类型关系由**规格**决定：「B 是 A 的子类型」意味着「每一个 B 都满足 A 的规格」，也就是 B 的规格至少与 A 一样强，因而 B 的对象可以替换 A 的对象而不破坏 A 向客户端承诺的任何性质（这就是 Liskov 替换原则）。在不可变版本里，`ImmutableRectangle` 只承诺 `getWidth()` / `getHeight()` 返回宽高，而 `ImmutableSquare` 用 `final int side` 实现这两个观察者，后置条件完全一致（规格相同，因而至少一样强），所以它是合法子类型。到了可变版本，`MutableRectangle.setSize(width, height)` 承诺**任意**宽高都可以设置，而正方形的规格要求宽恒等于高，二者无法同时成立。原文的四种候选规格分别是：①「`Requires width = height`」——**加强了前置条件**（把原本合法的 `setSize(3, 5)` 变成违约），不合法；②「`@throws BadSizeException if width != height`」——同样是**加强前置条件**（原本合法的调用变成抛异常），不合法；③「若 `width = height` 则设为 `width x height`，否则新尺寸未指定」——**削弱了后置条件**（对客户端的保证变少），不合法；④ 只有真正的**加强后置条件**（例如承诺设置成功后宽高必然等于参数，或不引入任何新前置条件）才是合法的强化。因此当可变性使规格冲突时，正确做法是**承认它们是两个抽象类型**：把共享的观察者留在只读接口（`ImmutableRectangle`）里，把冲突的变异者分别放进各自的类型，而不是靠 `instanceof` + 强转掩盖设计缺陷。

问题 2：某团队用 `String` 保存「一学期的四种取值」（`startRegistrationFor("Fall", 2023)`），并用 `HashSet<String>` 记录本学期开设的学期。请指出至少三个具体后果，说明改用原文的 `enum Semester { IAP, SPRING, SUMMER, FALL }` 后分别如何被消除；并解释为什么 `EnumSet` / `EnumMap` 更适合枚举（注意：`EnumSet` / `EnumMap` 超出 6.031 原文范围，属于 Java 生态补充知识）。

答案：第一，**不能 fail fast**：`startRegistrationFor("FAll", 2023)` 或 `"Autumn"` 都是合法字符串，编译器无从拒绝，错误只能留到运行时（或更糟，写进数据库），而改成 `Semester` 后这两个调用都是静态错误，IDE 还能补全——这正是原文比较三种方案时的第一个结论。第二，**命名常量方案也不安全**：`public static final String FALL = "Fall"` 拦不住客户端传 `"Autumn"`，且常量若未声明 `final` 还能被重新赋值；枚举值则由语言保证唯一且不可构造新值。第三，**签名无信息量且容易写反**：两个标量参数可以互相写错而不被检出；用 `Semester` 后参数类型自带含义，而且把 `Month.JANUARY` 传进来也是静态错误（类型不同，这正体现「枚举比 `int` 常量有更多静态检查」）。第四，**表示与不变量不匹配**：`HashSet<String>` 可以装任意字符串（包括 `"Fall "` 这种多一个空格的变体），需要额外运行时校验来维持「只含合法学期」；枚举方案把合法性放进类型里。关于 `EnumSet` / `EnumMap`（补充说明）：它们内部是**位向量**与**按 ordinal 索引的数组**，`add` / `contains` / `remove` 是常数时间且常数因子极小，内存占用与枚举取值个数成位，迭代顺序固定为声明顺序（测试与调试可复现），并且类型安全（`EnumSet<Semester>` 装不进 `Month`）；而 `HashSet<String>` 要为每个取值保存字符串对象、计算字符串哈希、还可能被任意字符串污染。需要注意的是：**「取值集合小而固定 → 用枚举」是 6.031 原文内容，而「枚举的集合用 `EnumSet` 而不是 `HashSet`」是本笔记的补充说明**，`EnumMap` 的构造还必须显式传入 `Semester.class`，因为类型擦除使运行时无法推断键的枚举类型。

问题 3：说明 sp22 原版 TypeScript 的「结构子类型化（structural subtyping）」与 Java 的「名义子类型化（nominal subtyping）」的差别；并用 sp22 原文的 `Array` / `ReadonlyArray` 例子说明结构子类型化为什么会在类型安全上「开一个洞」，以及在 Java 中遇到类似情形应当如何应对。

答案：在 TypeScript 中，只要类型 B 提供了 A 所要求的全部操作（同样的公开方法与公开实例变量，且类型兼容），TypeScript 就认为 B 是 A 的子类型——B 的声明里**完全不必**提到 A（不需要 `implements` / `extends`），这叫结构子类型化。Java 则是名义子类型化：子类型关系必须由声明**显式建立**（类 `implements` 接口、接口 `extends` 接口、类 `extends` 类），否则编译器根本不承认两者有关系，即使方法签名逐一对得上——所以本讲的 `MyString` 实现类必须写 `implements MyString`，不存在「碰巧结构相同就被当作子类型」的情况。sp22 的例子是：`ReadonlyArray` 拥有 `Array` 的全部观察者与生产者但去掉变异者，于是 `Array` 是 `ReadonlyArray` 的**结构**子类型，因此 `const readonlyArr: ReadonlyArray<number> = [1, 2, 3];` 合法，反方向 `const arr: Array<number> = readonlyArr;` 则是静态错误（`ReadonlyArray` 不提供变异操作）。但「洞」在于：如果同时还保留可变别名（`const arr: Array<number> = [1,2,3]; const readonlyArr: ReadonlyArray<number> = arr;`），一句 `arr.push(4)` 就能改掉 `readonlyArr` 看到的内容——`Array` 虽然结构上是 `ReadonlyArray` 的子类型，却**不是真正的**（规格意义上的）子类型，因为它的契约不提供不可变性。**补充说明**：Java 没有结构子类型化，这一具体漏洞不会以同样形式出现，但有个相似陷阱——原文用过的 `Collections.unmodifiableList(list)` 只返回一个不可修改的**视图**，原 `list` 若仍被别名持有并可改写，视图内容就会跟着变；要真正不可变，应使用 `List.of(...)` / `List.copyOf(...)` 生成独立副本，并丢弃全部可变别名（与 Reading 08 的要求一致）。

Reading 13: 调试（Debugging）

> 说明：本讲 sp22 原版使用 TypeScript，本笔记按用户要求提供 Java 代码示例；类型/API 与 sp21（6.031 Java 版）原文保持一致。原文中出现在 Java 中不存在的机制（如 `console.log`、`RangeError`）已替换为 Java 对应写法（`System.out.println`、`ArrayIndexOutOfBoundsException`）。

概述

本讲要解决的问题是：当 bug 已经写进代码、测试已经失败、用户已经报告了异常时，我们该如何**系统地**把它找出来，而不是靠盯着屏幕发呆或随机改代码。核心方法是把调试当作**科学方法**来执行：重现失败 → 观察数据 → 提出假设 → 设计最小侵入的探针实验 → 根据观测修正假设；在此过程中用**二分查找**、**程序切片（slicing）**、**差分调试（delta debugging）**来缩小搜索空间，用**断言（assert）与 `checkRep()`**把 bug 的暴露点尽量推近它的源头。它与「Safe from bugs」直接相关——调试就是消灭 bug 并用回归测试防止它复活；同时它也服务于「Easy to understand」（受控的探针与最小作用域让推理更简单）与「Ready for change」（找到 bug 后判断是编码错误还是设计错误，并做定向修复而非贴补丁）。**注意：调试是最后手段，不是第一手段**——本讲的所有技巧都是为了在 Reading 09（Avoiding Debugging）的 fail-fast 设计与测试防线失效时兜底。

核心概念与设计原则详解

系统化调试（Systematic Debugging） – **定义与目的**：把「找 bug」这个活动从随机的、靠灵感的过程，变成一个可重复、可记录、可积累的工程流程。它解决的是「安全（Safe from bugs）」问题：随机的调试方式会引入新的 bug（治标不治本）、会浪费大量时间，还会让你在不理解原因的情况下「碰巧」让程序不再报错。 – **直观解释（"它是什么？"）**：想象修水管。门外汉会到处敲打，看看哪里不漏了；专业水管工会先确定漏水点，再判定是接头松了还是管子裂了，然后只动那一处。系统化调试就是「先定位、再理解、最后修复」，而不是「先修、再看还漏不漏」。 – **关键规则与最佳实践**： – 采用科学方法的四步循环：**Study the data → Hypothesize → Experiment → Repeat**。 – 使用 **10 分钟法则**：如果已经用非系统的方式找了 10 分钟，立即停下来，转向科学方法。 – 把调试过程从脑子里搬到纸面上：写下假设、实验、预测、观测四栏。 – 先让系统进入「可复现的失败状态」，再谈修复。 – 复现 → 理解原因 → 修复 → 加回归测试，四步都不能跳。

重现 bug（Reproduce the Bug） – **定义与目的**：找到一个**小的、可重复**的测试用例，让失败稳定地出现。它服务于「安全」：没有稳定失败，你的每一次「修改后的成功」都可能是巧合；有了稳定失败，你才能验证修复真的起了作用。同时它服务于「易理解」：小用例让数据流一目了然。 – **直观解释（"它是什么？"）**：把莎士比亚全集（100,000 行、800,000 多个词）喂给 `mostCommonWord()`，得到一个莫名其妙的结果 `"e"`。正常手段（打印调试、断点调试）在这种规模下几乎不可用。聪明的做法是不断缩小输入：前半部莎士比亚还出错吗？（二分查找！）单独一部戏呢？单独一段台词呢？直到得到 `"c c b"` 这样的小用例。 – **关键规则与最佳实践**： – 优先用二分查找缩小输入规模：一半、四分之一、单个函数、单个语句。 – 缩小后的用例要找**仍能触发同一个**（或极相似）bug 的最小输入，而不是「最简的单测」。 – 缩小用例的过程本身就是科学方法的应用：观察数据、假设「可能与出现次数有关而与具体单词无关」、用 `"a a a b"` 做实验。 – 找到并修复后，**回到原始的大输入确认修的是同一个 bug**。 – 修复后把这个用例放进回归测试套件（regression suite），让 bug 永不再现。 – 遇到 GUI 或多线程程序时bug 可能难以稳定复现，此时要先设法固定时序或加日志，再谈重现。

二分查找定位 bug（Binary Search） – **定义与目的**：在「问题可能出现的位置序列」（输入规模、数据流阶段、版本历史）上每次砍掉一半，用 O(log n) 次实验定位 bug。它服务的是调试效率——把「概率上要靠运气」变成「有保证的对数级收敛」。 – **直观解释（"它是什么？"）**：`mostCommonWord()` 的数据流是 `text → splitIntoWords → countOccurrences → findMostFrequent → winner`。如果在 `countOccurrences()` 里抛异常，那么它**下游**的 `findMostFrequent()` 根本还没执行，可以整段排除——搜索空间立刻小了一半。此后再把剩下的部分对半砍，一次实验就能判断 bug 在前半段还是后半段。 – **关键规则与最佳实践**： – 把程序视为**数据流**（一系列算法步骤），而不是一堆行号；这样一次就能排除整段代码。 – 在数据流的**中点**插探针（打印、断言、断点），看该处的值是好的还是坏的。 – 探针的结论必须能二值化：好 → 往下游找；坏 → 往上游找。 – 输入规模也能二分：把 800,000 词的输入砍成 400,000、200,000……直到失败消失，再取回上一层。 – 版本历史上同样可以二分：用版本控制系统（Reading 05）取出最近一次仍能通过测试的版本，在「工作版本」与「失败版本」的差异中定位引入 bug 的提交。

切片（Slicing） – **定义与目的**：找出「为计算某个具体值做出贡献」的那些代码行，这个集合就是该值的**切片**。导致该值出错的 bug 必然落在切片内，因此切片就是你的搜索空间——它同时告诉你「bug 可能在哪儿」和「bug 不可能在哪儿」。 – **直观解释（"它是什么？"）**：假设局部变量 `x` 不应为负，但某处的调试打印显示它是负数。那么所有**直接改写 x** 的行、所有**参与计算被加进 x 的值**（例如 `bonus`）的行、所有**控制这些语句是否执行**的条件与循环（`if (isWorthABonus(s))`、`for (final Sale s : salesList)`）、以及这些控制语句所依赖的数据来源，统统属于切片。而方法里那些与 `x` 无关的 `...` 代码，则可以放心排除。 – **关键规则与最佳实践**： – 切片可以在脑中完成，不需要工具；先写下「坏值的名字」，再顺着赋值与控制流往回追。 – **不可变性（immutability）极大加速切片**：看到 `final int bonus = getBonus();` 时，你可以立刻停止追踪——没有任何其他行能进入 `bonus` 的切片。而 `final Sale s` 若 `Sale` 是可变类型，你还得继续检查后面有没有人对 `s` 或其他别名调用了 mutator。 – **作用域最小化（scope minimization）同样加速切片**：局部变量的切片就在附近；实例变量要把整个类纳入搜索；全局变量（static 可变字段）则要把整个程序纳入搜索。 – 好设计让切片更省力，坏设计让切片代价爆炸——这是「设计影响调试效率」的直接体现。 – 切片得出的结论应转化为具体的、可检验的假设，例如「`getBonus()` 返回了负数」「`isWorthABonus()` 对太多销售返回 true 导致 `x` 溢出」。

差分调试（Delta Debugging） – **定义与目的**：通过**对比成功的运行与失败的运行之间的差异**来定位 bug。它解决的是「知道哪两种情形一好一坏，但不知道原因」的问题。 – **直观解释（"它是什么？"）**：也许 `mostCommonWords("c c, b")` 出错，而 `mostCommonWords("c c b")` 正常。两者只差一个逗号。于是问：哪些代码在通过的用例里执行了、在失败的用例里被跳过了（或反之）？差异点就是最可疑的地方。另一个版本是「时间上的差分」：回归测试开始失败时，取出最近一次仍然通过的版本，系统地探索两次之间的代码改动，直到找到引入 bug 的那一次改动。 – **关键规则与最佳实践**： – 主动构造**成对**的用例：一个通过、一个失败，且两者尽量只差一点点。 – 用版本历史做时间维度的差分，是定位「昨天还好好的」这类回归 bug 的首选方法。 – 差分调试与切片互为补充：切片给出静态的候选集合，差分调试给出动态的差异集合。 – 自动化工具（delta debugging、slicing 工具）存在，但课程明确指出它们目前**并不实用**，手工推理仍是主力。

假设的优先级（Prioritize Hypotheses） – **定义与目的**：不同代码的出错概率差别巨大，先怀疑最可能出错的部分，能把实验次数降到最低。 – **直观解释（"它是什么？"）**：可信度从低到高大致是：你刚写的新代码 < 老旧且被充分测试的代码 < Java 标准库 < 编译器与运行时 < 操作系统 < 硬件。就像家里的灯不亮，先怀疑灯泡和开关，最后才怀疑发电厂。 – **关键规则与最佳实践**： – 优先怀疑最近改动的代码、很少被测试覆盖的代码路径。 – 只有在你确实怀疑某个模块时，才做「替换组件」实验（例如把 `binarySearch()` 换成简单的 `linearSearch()`、把 `ArrayList` 换成 `LinkedList`、换 JDK 版本、换操作系统、换机器）。 – 替换组件很费时间，**不要盲目轮换**未出错的组件。 – 代码覆盖率工具可以用来发现问题：`quadraticRoots` 有时给出错误答案，也许是测试根本没覆盖到某些分支。 – 优先级排序的作用是「让每一次实验都有最大的信息增益」。

探针（Probes）：打印、日志、断言、调试器 – **定义与目的**：实验的形式就是「观察系统」；最好的实验叫探针——**尽可能少扰动系统**的温和观察。它服务于「安全」：改动越少，你越不会在调试过程中引入新 bug（海森堡 bug）。 – **直观解释（"它是什么？"）**：医生诊断时会先量体温、听心跳这些无创检查（探针），而不是先

开刀（修改代码）。四种常用探针： 1. **打印语句**：适用于几乎任何语言；缺点是需要事后撤销，容易在代码里留下成堆的垃圾。写的时候要写清楚上下文，例如打印 `start of calculateTotalBonus`，而不是在 15 个地方都打印同一句 `hi!`，否则你分不清哪句是哪句。 2. **日志（logging）**：把有信息量的打印语句永久留在代码里，用一个全局开关（`DEBUG` 常量或日志级别）控制开关。更成熟的框架（Java 生态中的 Log4j/SLF4J，属补充说明）可以把日志写到文件或网络服务器、记录结构化数据，并且可用于生产部署环境——大型系统没有日志几乎无法运维。 3. **断言**：直接检查变量值或内部状态，不需要人工读输出。优点是可以保留在代码中（若该断言普遍为真），缺点是**Java 默认不开启断言**，你必须用 `-ea`（enable assertions）运行，否则断言根本不执行，会出现「断言看起来通过了其实没运行」的欺骗。 4. **调试器断点**：在指定行暂停程序，单步执行并查看变量值。区分 **Step Over**（执行完当前行，停在下一行）与 **Step Into**（进入当前行调用的方法内部，粒度更细）；也常用 **Resume/Continue** 配合断点快速跳过无关代码。调试器功能强大，值得专门学习。 – **关键规则与最佳实践**： – 先做「只观察不修改」的探针，不要一上来就做「顺手把 bug 改掉」的实验。 – 打印语句要写清位置与语义；同一会话中保持探针可识别。 – 提交前清理所有调试探针：注释掉的代码、临时打印、只对某个用例成立的断言都要撤销。 – 找到原因并修复后，把「普遍为真」的检查转成正式的 `checkRep()` 或断言保留下来。

用 `assert` 与 `checkRep()` 缩小 bug 范围（Assertions and checkRep） – **定义与目的**：断言可以把「坏值被发现的位置」推到离它的产生点更近的地方。它服务于「安全」与「易理解」：越多不变量被显式检查，故障点就越接近 bug 源，同时不变量本身也成为可读的文档。 – **直观解释（"它是什么？"）**：如果 `x` 永远不该为负，就在每次可能改动 `x` 的地方后面加 `assert x >= 0;`。这样一旦某次加法出错，程序会**立刻**在那里炸掉，而不是等到几百行之后用一个荒唐的结果（或者一个莫名其妙的 `ArrayIndexOutOfBoundsException`）间接暴露。 – **关键规则与最佳实践**： – 把不变量写成 `checkRep()` 私有方法（表示不变量 RI 的代码化，见 Reading 11），在每个构造器、每个 mutator 的出口、每个 observer 的入口调用它。 – 断言应表达**规格与不变量**，例如 `assert fromCurrency != null && toCurrency != null;`、`assert !fromCurrency.equals(toCurrency);`。 – 不要写只在某个特定测试用例下成立的断言并把它留下——那会把「调试期的观察」误当成「类型的不变量」。 – 记住 Java 断言默认关闭；测试时务必加 `-ea`（JUnit 配置中也要打开），否则你的 fail-fast 防线是纸做的。 – 断言与 `checkRep()` 属于同一个思想：**让错误在最早、最局部的位置失败**，这与 Reading 09 的 fail-fast 设计一脉相承。

一次只处理一个 bug（One Bug at a Time） – **定义与目的**：调试过程中经常顺手发现别的问题（读自己代码时发现明显的错误——相当于一次自我代码评审）。此时要保持专注，避免「递归调试」。 – **直观解释（"它是什么？"）**：你的大脑栈很浅。开始调一个「顺带发现」的 bug，你可能就很难「弹栈」回到原来的 bug，而且你随手改的代码可能影响原实验的可解释性。 – **关键规则与最佳实践**： – 维护一份 **bug list**（纸、文本文件，或团队用的 issue tracker），把新发现的问题记下来，稍后处理。 – 思考新问题是否是当前 bug 的**信息性数据**（是否带来新假设），但不要立刻开始调试它。 – 调试期间**不要随意修改代码**；所有改动都应是为当前 bug 设计的、受控的探针。 – 如果新问题妨碍了当前调试（例如导致程序间歇性崩溃，实验无法稳定执行），则应重新排序：放下当前 bug（撤掉探针、确保记录在 bug list 上），先解决新 bug。

不要过早修复（Don't Fix Yet） – **定义与目的**：把「实验」误做成「修复」是调试中最常见、代价最大的错误之一。 – **直观解释（"它是什么？"）**：看到 `ArrayIndexOutOfBoundsException`，马上加 `try/catch` 吞掉，或先 `if (index < size)` 绕过——程序不报错了，但你并不知道为什么会越界。这是「治标不治本」，把疾病藏了起来。 – **关键规则与最佳实践**： – 先理解异常被抛出的原因，再改代码。 – 警惕「猜—试」式编程：它会产生难以理解的复杂代码，并且掩盖真正的 bug。 – 只有当假设被实验证实、原因被理解之后，才写下修复。 – 修复前先问一句：这是编码错误（拼错变量、参数顺序颠倒）还是**设计错误**（接口规格不足/不充分）？设计错误意味着要回头看设计，至少检查该接口的其他客户是否也中招。 – 修复时顺带搜索同类错误（这里除零了，别处是否也除零？）并评估修复的副作用（会不会破坏别的代码？）。

审计轨迹、检查插头、以及"不是你修好的就不算修好"（Audit Trail / Check the Plug / If YOU didn't fix it, it isn't fixed） – **定义与目的**：这三条来自 Agans 的《调试九条规则》，解决的是「调试过程本身失控」以及「bug 只是暂时隐藏」两类问题。 – **直观解释（"它是什么？"）**： – **Keep an Audit Trail**：只要一个 bug 花了超过几分钟、或超过了 study—hypothesis—experiment 循环的两三次迭代，就必须动笔。记录：当前假设、正在做的实验、实际观测（测试通过还是失败、程序输出特别是你自己的调试信息、任何栈轨迹）。缩小用例与差分调试往往需要多轮迭代，不记下来你很快就会忘记「刚才试的是 `"c c b"` 还是 `"c b"`，它到底通过没有」。 – **Check the Plug**：如果一切都不合逻辑，就质疑自己的假设。机器按了开关却不启动，也许该检查的不是开关而是插座有没有电。在编程中这意味着：**确认你运行的代码（class 文件）与你阅读的代码（源文件）一致**——从仓库拉取最新版本，删除所有编译产物并全量重新编译（Eclipse 中是 Project → Clean）。 – **If YOU didn't fix it, it isn't really fixed**：系统突然「好了」，而你说不出它为什么好，那它多半没好——bug 只是被环境变化掩盖了，随时可能重新抬头。并发 bug 尤其如此（Reading 21）。这正是系统化调试的价值：先复现失败、再理解原因、再修改、最后看到系统**因为你这次改动**从失败转为成功，你才有资格说它被修好了。 – **关键规则与最佳实践**： – 每轮循环都写下假设、实验、预测、观测四要素。 – 观察与预测不符 → 否定该假设；相符 → 精化假设、继续缩小范围。 – 出现「不合逻辑」的现象时，第一件事是检查构建产物是否最新。 – 不要因为「现在不报错了」就认为任务完成。

修复 bug 的收尾流程（Fix the Bug） – **定义与目的**：找到原因之后的第三步是设计修复，并且要做得干净、彻底、可回归。 – **直观解释（"它是什么？"）**：修复不是「贴个补丁走人」，而是一次小型的设计评审 + 清洁 + 加固。 – **关键规则与最佳实践**： – **撤销调试探针**：注释掉的代码、临时打印语句、为加速调试做的改动，在提交前全部撤销。 – **补一个回归测试**：把该 bug 的用例加入回归套件，并跑完整测试，确保 (a) bug 已修复，(b) 没有引入新 bug。 – **寻找相关 bug**：检查同类错误是否在别处出现，让代码对未来同类 bug 免疫。 – **评估副作用**：这次修复会不会破坏其他调用方？ – **换个视角（Get a fresh view）**：向别人（哪怕对方完全不懂）解释你的代码为什么应该工作、实际却在做什么，这就是橡皮鸭调试（rubber-duck debugging）。6.031 的助教与同学是更好的对象；在网上提问时，你最小化 bug 的努力正好帮你写出一份最小可复现示例（minimal reproducible example）。 – **睡一觉（Sleep on it）**：疲惫的调试者效率极低，用一点延迟换取效率是划算的。

代码示例与对比分析

场景 1：调试完成后，代码里还留着调试探针

❌ *错误代码*

```
/**
 * Convert from one currency to another.
 * @param fromCurrency currency that customer has (e.g. DOLLAR)
 * @param fromValue value of fromCurrency that customer has (e.g. $145.23)
 * @param toCurrency currency that customer wants (e.g. EURO).
 *                   Must be different from fromCurrency.
 * @return value of toCurrency that customer will get,
 *         after applying the conversion rate and bank fee
 */
public static double convertCurrency(Currency fromCurrency, double fromValue, Currency toCu
    assert fromCurrency != null && toCurrency != null;
    assert ! fromCurrency.equals(toCurrency);

    double rate = getConversionRate(fromCurrency, toCurrency);
    System.out.println("conversion rate is " + rate);      // 调试探针：忘了删

    double fee = getFee();
    assert fee == 0.01;   // right now the bank charges 1%   // 只对今天成立的旧观察

    return fromValue * rate * (1-fee);
}
```

【错误代码的问题】 1. `System.out.println` 会在生产环境持续污染标准输出，在批处理或服务端程序中可能造成日志洪水、性能损耗，且需要重新打包才能去掉。 2. `assert fee == 0.01;` 把「当前银行费率」这个**会变化的外部事实**写成了类型不变量。银行明天把费率改成 1.5%，这条断言会在正确的代码上失败，成为一颗定时炸弹（而且开启 `-ea` 时才炸，属于典型的「只在测试机上崩」的 bug）。 3. 断言与打印混在业务逻辑中，读者分不清哪些是规格的一部分、哪些是调试残留，破坏「Easy to understand」。 4. 一旦有人为了「让测试过」而把断言注释掉，团队就失去了这类 fail-fast 检查的可信度。

✅ *正确代码*

```
/**
 * Convert from one currency to another.
 * @param fromCurrency currency that customer has (e.g. DOLLAR)
 * @param fromValue value of fromCurrency that customer has (e.g. $145.23)
 * @param toCurrency currency that customer wants (e.g. EURO).
 *                   Must be different from fromCurrency.
 * @return value of toCurrency that customer will get,
 *         after applying the conversion rate and bank fee
 */
public static double convertCurrency(Currency fromCurrency, double fromValue, Currency toCu
    // 前置条件：与规格说明一致，普遍为真，保留下来长期防御
    assert fromCurrency != null && toCurrency != null;
    assert ! fromCurrency.equals(toCurrency);
    checkRep();

    double rate = getConversionRate(fromCurrency, toCurrency);
    double fee = getFee();
    assert fee >= 0 && fee < 1;   // 真正的不变量：费率落在合法区间

    double result = fromValue * rate * (1-fee);
    return result;
}

/** Rep invariant: 换汇结果不得为无穷大或 NaN（参数合法时）。 */
private static void checkRep() {
    // 这里的检查只依赖参数与常量，不依赖外部世界的当前取值
}
```

【为什么这样更好】 打印语句被删除，标准输出恢复干净；`fee == 0.01` 这种「只对当前世界成立」的观察被改写成「对任何时刻都成立」的区间不变量 `0 <= fee < 1`，因此断言可以长期保留并真正发挥 fail-fast 作用；把断言集中成 `checkRep()`，让「哪些条件是这个类型的规格」变成可读的文档。

【代码对比解说】 关键区别在于**断言的内容是「规格」还是「观察」**。调试期间我们常常临时写下只对某个用例成立的断言（例如「此时 `rate` 应该是 1.13」），它们是实验工具，实验结束必须撤销。而像「两个货币参数不能相同」「费率必须落在 [0,1)」这样的性质，是从规格直接推出来的不变量，可以永远留着。错误的做法混淆了两者，导致要么断言被全线关闭，要么生产环境频繁误报。

【设计原则透视】 这正是 Reading 09（Avoiding Debugging）中「fail fast + 显式不变量」的实践：`checkRep()` 是 Reading 11 里表示不变量（RI）的可执行形式，`assert` 是前置条件（precondition）的可执行形式。把断言写在方法入口/出口，等于把规格的抽象边界变成运行时检查点——bug 一旦跨越边界，就会在边界处被立即抓住，而不是在系统的另一端以奇怪的形式暴露。同时，Reading 06/07 的规格说明告诉你「哪些条件允许被断言」：断言只能覆盖**spec 保证的东西**，不能覆盖实现细节（implementation-specific），否则就把客户端不该依赖的实现暴露了出来。

场景 2：用一个笼统的 catch 把真正的 bug 吞掉

❌ *错误代码*

```
/**
 * @return true if and only if word1 is an anagram of word2
 *         (i.e. a permutation of its characters)
 */
boolean isAnagram(String word1, String word2) {
    if (word1.isEmpty() || word2.isEmpty()) {
        return word1.isEmpty() && word2.isEmpty();
    }

    try {
        word1 = sortCharacters(word1);
        word2 = sortCharacters(word2);
        return word1.equals(word2);
    } catch (StackOverflowError e) { return false; }   // 掩盖了真正的 bug
}
```

【错误代码的问题】 1. `catch (StackOverflowError e) { return false; }` 意味着：只要 `sortCharacters()` 因为递归不收敛而爆栈，这个方法就悄悄返回 `false`。调用者以为「这两个词不是变位词」，实际上程序已经崩过一次——**错误的答案比崩溃更危险**。 2. 它让「sortCharacters 的递归步没有缩小问题规模」这个真正的 bug 永远不被发现，`StackOverflowError` 也不会出现在日志或堆栈轨迹里，调试所需的「数据」被销毁了。 3. 用 `StackOverflowError`（`Error` 而非 `Exception`）做控制流，违反了 Java 的异常设计意图；在生产代码里捕获 `Error` 通常意味着放弃了程序的可恢复性。 4. 这段代码属于典型的「猜测—测试式编程」产物：每次报错就加一个 return，代码越来越复杂，却离正确越来越远。

✅ *正确代码*

```
/**
 * @return true if and only if word1 is an anagram of word2
 *         (i.e. a permutation of its characters)
 */
boolean isAnagram(String word1, String word2) {
    if (word1.isEmpty() || word2.isEmpty()) {
        return word1.isEmpty() && word2.isEmpty();   // 空串只与空串互为变位词
    }
    word1 = sortCharacters(word1);
    word2 = sortCharacters(word2);
    return word1.equals(word2);
}

/** @return a string with the same characters as s, in ascending order */
private static String sortCharacters(String s) {
    char[] chars = s.toCharArray();
    java.util.Arrays.sort(chars);   // 迭代实现：不会爆栈，也没有递归不收敛的风险
    return new String(chars);
}
```

【为什么这样更好】 去掉了 `try/catch` 这块「创可贴」，让任何残留的问题以真实异常的形式暴露出来（配合堆栈轨迹就能定位）；把 `sortCharacters()` 换成使用 `Arrays.sort()` 的迭代实现，从根上消除了「递归步不缩小问题规模导致 `StackOverflowError`」这一**原因**，而不是掩盖症状。

【代码对比解说】 这里体现了「Don't fix yet」与「理解原因后再修」的分工：错误的写法做了两个动作——先猜（用 try/catch 掩盖），再试（看测试过不过）。正确写法先问「栈为什么溢出」，答案是 `sortCharacters` 的递归没有收敛（例如把 `substring(1)` 误写成 `substring(0)`，永远处理同一个字符串）。找到这个原因之后，修复方式有两种：修好递归的收敛性，或者干脆改用库里的迭代排序。两者都比 catch 更好，因为两者都保留或消除了原因，而不是遮蔽它。

【设计原则透视】 这个例子把 Reading 14（Recursion）的「递归步必须把问题变小」与 Reading 13 的调试纪律连在一起：递归不收敛在迭代程序里表现为死循环（可能耗尽 CPU 但不报错），在递归程序里表现为 `StackOverflowError`——**递归的 bug 有时失败得更快**，这反而对调试有利，前提是你不把它 catch 掉。另一方面，`isAnagram` 的规格（`@return true iff ...`）是一个**完整的行为规格**：任何让它在「无法判断」时返回 `false` 的实现都违反了规格，因为 `false` 是一个有明确含义的答案。把 `Error` 当作 `false` 返回，等于让实现偷偷返回一个**错误的抽象值**，这是 ADT 设计中最严重的背离（Reading 10/11）。

场景 3：探针粗暴、作用域过大，妨碍切片

❌ *错误代码*

```
public class BonusCalculator {
    static int x = 0;                          // 全局可变状态：切片范围 = 整个程序
    static final List<Sale> salesList = new ArrayList<>();   // 可变别名，被多个方法共享

    int getBonus() { return computeBonus(); }
    boolean isWorthABonus(Sale s) { return s.amount() >= 100; }

    int calculateTotalBonus(List<Sale> sales) {
        salesList.addAll(sales);
        x = 0;
        int bonus = getBonus();                // 非 final：后续任何一行都可能改写它
        for (Sale s : salesList) {
            System.out.println("hi!");         // 15 个地方都打印 "hi!"，分不清谁是谁
            x += bonus;
            x = x - 0;                         // 调试时随手加的探测语句，忘了删
            bonus = bonus;                     // 为了打断点加的假赋值
        }
        return x;
    }

    int computeBonus() { return -5; }          // 真正的 bug：奖金为负
}
```

【错误代码的问题】 1. `static int x` 使 `x` 的切片范围扩大到**整个程序**——任何类、任何线程都可能改它，你无法用「读代码」的方式排除任何一行。 2. `bonus` 不是 `final`，观测者必须继续扫描循环体内部，确认没有别的地方改写 `bonus`；如果它是 `final int bonus = getBonus();`，切片在那一行就结束了。 3. `System.out.println("hi!")` 重复 15 次且没有上下文，输出无法对应到具体位置；`x = x - 0;` 与 `bonus = bonus;` 这类探测语句不但无信息量，还会改变程序行为、干扰后续实验的可解释性。 4.

`salesList` 是共享的可变静态字段，多个调用之间会互相污染，实验无法重复——这直接违反「重现 bug」的要求。

✅ *正确代码*

```
public class BonusCalculator {
    private int x = 0;                                      // 实例状态，作用域限于本类

    private int getBonus() { return computeBonus(); }
    private boolean isWorthABonus(Sale s) { return s.amount() >= 100; }
    private int computeBonus() { return 10; }

    /** Rep invariant: x >= 0. */
    private void checkRep() {
        assert x >= 0 : "x went negative: " + x;
    }

    /**
     * @param salesList 需要计算奖金的销售列表
     * @return 总奖金，要求 salesList 中每笔销售都非 null
     */
    public int calculateTotalBonus(final List<Sale> salesList) {
        x = 0;
        final int bonus = getBonus();               // final：切片到此为止
        checkRep();

        for (final Sale s : salesList) {            // s 不可重新赋值
            if (isWorthABonus(s)) {
                x += bonus;
                checkRep();                         // 探针：坏值刚产生就被抓住
            }
        }
        // 描述性探针（调试期临时使用，事后删除）
        // System.out.println("end of calculateTotalBonus: x=" + x + ", bonus=" + bonus);
        checkRep();
        return x;
    }
}
```

【为什么这样更好】 `x` 是实例字段而不是全局字段，切片只需在本类内搜索；`bonus` 与 `s` 是 `final`/循环变量，观测者在读到声明那一行就知道「没有别的行会影响它」；`checkRep()` 在每个可能改坏 `x` 的点上做检查，于是 bug 会在**第一次变负的那一刻**被抓住，而不是等到最后返回一个负数；调试打印语句带上下文并明确标注为事后删除，输出因此可读、可解释。

【代码对比解说】 两种写法功能「差不多」，但**可调试性完全不同**。错误的写法让「找出谁把 `x` 变负」变成在整个程序里的大海捞针；正确的写法把搜索空间压缩到几行代码内。再看二分查找：在这个循环里，我们可以只在循环**中点**（例如第 500 笔销售）插一个 `checkRep()`，如果此时 `x` 已经为负，bug 在前半段；否则在后半段——每次砍掉一半。这种「探针 + 二分」的组合，正是本讲的核心套路。

【设计原则透视】 这里把 Reading 08（Immutability）与调试效率显式联系起来：不可变引用（`final`）让切片在声明处终止，可变类型（如 `Sale`）则迫使你继续检查别名上的 mutator 调用。同时，作用域最小化

（scope minimization）是 Reading 04（Code Review）与 Reading 10（ADT）中的设计美德，它在调试阶段的回报就是「搜索空间更小」。此外，`checkRep()` 与 `assert x >= 0` 把 Reading 11 的表示不变量从注释升级为**可执行的契约**，保证任何违反 RI 的状态在产生的瞬间就 fail fast。

场景 4：在错误的地方找 bug，并用 try/catch 掩盖症状

❌ *错误代码*

```
/**
 * Find the most common word in a string.
 * @param text 零个或多个单词组成的字符串
 * @return 出现次数最多的单词（忽略大小写）
 */
public static String mostCommonWord(String text) {
    List<String> words = splitIntoWords(text);
    Map<String,Integer> frequencies = countOccurrences(words);
    String winner = findMostFrequent(frequencies);
    return winner;
}

// 现象：countOccurrences 内部抛出 NullPointerException。
// 错误反应：跑到下游的 findMostFrequent 里“找 bug”，并顺手加个保护：
private static String findMostFrequent(Map<String,Integer> frequencies) {
    try {
        return Collections.max(frequencies.entrySet(),
                Map.Entry.comparingByValue()).getKey();
    } catch (NullPointerException e) {
        return "";          // 治标：把异常变成空答案，原因仍然存在
    }
}
```

【错误代码的问题】 1. 失败点在下游之前的 `countOccurrences`，而 `findMostFrequent` **根本没有被执行**——在它里面找 bug 是纯粹的浪费时间（切片/数据流分析当场就能排除它）。 2. `catch (NullPointerException e) { return ""; }` 让方法在出错时返回一个「合法的抽象值」`""`，客户端无法区分「text 中没有单词」与「程序已经出错了」，bug 被藏进数据里。 3. 它破坏了 `NullPointerException` 携带的堆栈轨迹信息；堆栈轨迹是最有价值的「数据」，被吞掉后你连「哪一行解引用了 null」都不知道了。 4. 用「保护性判断」绕过异常（例如先检查 `index < size`）同样治标不治本：真正的原因可能是上游产出了一个 `null` 元素。

✅ *正确代码*

```
/**
 * Find the most common word in a string.
 * @param text 零个或多个单词组成的字符串
 * @return 出现次数最多的单词（忽略大小写）
 */
public static String mostCommonWord(String text) {
    List<String> words = splitIntoWords(text);

    // 二分查找：在数据流的中点插探针，检查上游的后置条件
    //  （调试期探针，定位后删除；或改写成长期保留的 checkRep 断言）
    assert words != null : "splitIntoWords violated its postcondition";
    for (String w : words) {
        assert w != null && !w.isEmpty() : "splitIntoWords produced a bad word";
    }

    Map<String,Integer> frequencies = countOccurrences(words);
    return findMostFrequent(frequencies);
}

/** @return 出现次数最多的单词；要求 frequencies 非空 */
private static String findMostFrequent(Map<String,Integer> frequencies) {
    // 不放 try/catch：让问题以真实异常暴露出来
    return Collections.max(frequencies.entrySet(),
            Map.Entry.comparingByValue()).getKey();
}
```

【为什么这样更好】 探针被放在了**数据流的中点**——`splitIntoWords` 与 `countOccurrences` 之间。这正是二分查找的做法：如果断言在这里就失败，说明假设「bug 在 splitIntoWords：输入合法但输出错误」成立，搜索空间立刻缩小到上游；如果断言通过，说明两个方法之间的**契约衔接**（前者的后置条件不满足后者的前置条件）或 `countOccurrences` 本身有问题，搜索空间缩小到下游。任何一条假设都对应一小块代码，与「在 `findMostFrequent` 里瞎找」形成鲜明对比。

【代码对比解说】 这段对比同时演示了三条原则：其一，**故障点 ≠ bug 位置**。程序在 `countOccurrences` 里崩，但 bug 可能在更早的 `splitIntoWords` 里（它产出了 `null`），坏值沿着好代码一路传播后才爆掉。其二，**探针应该放在能二值化判断的位置**：数据流的中点天然满足这一点。其三，**不要用 catch 掩盖**。把 `NullPointerException` 转成 `""` 之后，程序「不崩了」，但错误答案开始悄悄流动——这类 bug 往往要等到很久以后在完全无关的地方才暴露，代价高得多。

【设计原则透视】 这段代码把 Reading 06/07（Specifications）中的**前置条件与后置条件**当成了调试工具：`splitIntoWords` 的后置条件是「返回一个非 null、元素非 null 非空的单词列表」，把它写成断言就是在两个模块的**抽象边界**上设检查站。它也体现了 Reading 09 的 fail-fast：错误要么在源头立刻抛出，要么就会被传播到很远的地方再以晦涩的方式出现。最后，它示范了 Reading 03（Testing）中「回归测试」的闭环——定位并修复后，`mostCommonWord("c c b")` 这个最小用例要进入回归套件，让这个 bug 永不复活。

与其他设计原则的关联

- **Reading 03（Testing）**：调试的起点是「重现 bug」，而这通常就是写一个失败的测试用例；修复的终点是把它放进**回归测试套件**。调试与测试是一枚硬币的两面：测试负责发现 bug，调试负责消除 bug。
- **Reading 09（Avoiding Debugging）**：本讲所有技巧都是在「测试与 fail-fast 设计失效」时的兜底。断言、`checkRep()`、快速失败都是 Reading 09 的产物，它们在 Reading 13 中变成缩小搜索范围的探针。
- **Reading 11（Abstraction Functions & Rep Invariants）**：`checkRep()` 就是 RI 的可执行形式；切片时判断「哪些行可能影响这个值」时，AF 与 RI 帮你知道「哪些状态是合法的、哪些操作是允许的」。
- **Reading 06/07（Specifications / Designing Specs）**：调试时用前置/后置条件划分数据流阶段，是二分查找与切片的理论基础；而修复 bug 时要判断「这是编码错误还是规格/设计错误」，后者要回到规格层面重新设计。
- **Reading 05（Version Control）**：差分调试中的一个重要变体（找出「哪个提交引入了 bug」）完全依赖版本历史；`git bisect` 式的二分是这一思想的自动化。
- **Reading 04（Code Review）**：调试时阅读自己的代码本质上是一次自我代码评审；而「一次只处理一个 bug + 维护 bug list」的习惯也会反哺评审质量。
- **Reading 08（Immutability）**：不可变性让切片在声明处终止，极大降低调试成本；反之，可变别名的共享是「递归调用之间互相污染」这类难调 bug 的根源。
- **Reading 21（Concurrency）**：并发 bug 是「突然自己好了但其实没修好」的典型——时序相关的失败极易被环境变化掩盖，因此本讲的「如果不知道它为什么被修好，它就没被修好」在并发章节会被反复强调。

关键要点

- **先重现，再动手**：把失败变成一个小而可重复的用例，写进回归套件；在没有稳定失败之前，任何「修好了」都是幻觉。
- **10 分钟法则**：非系统化摸索超过 10 分钟就停下来，转为「观察数据 → 假设 → 探针实验 → 修正假设」的科学方法循环，并把每一步写进审计轨迹。
- **用二分与切片缩小搜索空间**：在数据流中点插探针；用 `final` + 最小作用域让切片在声明处终止；用成功/失败成对用例做差分调试。
- **探针最小侵入，修复要等理解原因**：优先用打印/日志/`assert`/`checkRep()`/调试器做只观察的实验，不要用 `try/catch` 把异常变成返回值；并且记住 Java 断言默认关闭，必须用 `-ea` 运行，否则 fail-fast 防线形同虚设。
- **收尾四件事**：撤销所有调试探针、补回归测试、检查同类 bug、评估副作用；只有当你**因为自己的改动**让系统从失败转为成功时，才叫修好了。

常见陷阱与注意事项

1. **把「不报错了」当成「修好了」** → 用 `try/catch` 或提前 return 掩盖异常，症状消失但原因仍在，错误答案在系统中悄悄传播，日后在完全无关的地方爆发。正确做法是先理解原因，再修改代码，并用回归测

试确认从失败到成功的转变。

2. **在故障点找 bug** → 程序在 `countOccurrences` 抛出异常，却在 `findMostFrequent` 或调用方里反复查看；坏值常常是在上游产生的，必须用数据流/切片向前回溯。
3. **调试期间随手改代码，或同时追多个 bug** → 为了「试试看」而改动无关逻辑会使实验失去可解释性；一发现问题就立刻开始递归调试则会导致「脑内栈」溢出、忘记原进展、实验互相干扰。调试期的改动应仅限于受控探针，新问题记入 bug list，除非它妨碍了当前调试否则不要切换。
4. **忘记撤销探针** → 打印语句、被注释的代码、只在特定用例下成立的断言被提交到仓库，污染日志、误导读者，甚至让断言在生产环境误报。`System.out.println` 尤其容易留在库里。
5. **依赖默认关闭的断言** → Java 断言只有加 `-ea` 才执行；以为「断言没报错就说明状态合法」，实际上它根本没运行（「看起来通过的断言」是最危险的假安全感）。
6. **未最小化就去求助** → 把 800,000 词的输入直接贴到论坛/助教面前，没人能帮你；先做二分缩小，得到一个最小可复现示例，求助效率与成功率都会成倍提升。

思考题（带答案）

问题 1：用户报告 `mostCommonWord("chicken chicken chicken beef")` 返回了 `"beef"` 而不是 `"chicken"`。为了在开始调试前把输入缩小、简化，下列哪些输入值得一试，为什么？ （a）`"chicken chicken beef"` （b）`"Chicken Chicken Chicken beef"` （c）`"chicken beef"` （d）`"a b c"` （e）`"c c c b"`

答案：（a）（b）（c）（e）都值得试，而且应当**全部**尝试，而不能只挑最简单的。理由：（a）减少重复次数，检验「次数是否为关键」；（b）改变大小写，检验「忽略大小写」这个规格点是否是 bug 所在；（c）是（a）的进一步缩减，检验「三次重复」是否为触发条件；（e）把单词缩写为单个字符，保留「三次 + 一次」的模式，若仍然失败，你就得到了最简洁的失败用例（本轮中最终正是 `"c c b"` 仍然出错）。至于（d）`"a b c"`，它取消了「某个单词重复出现」这一结构，很可能**不再触发** bug——但它依然值得一试，因为它能确认「平局/无重复」这条路径是正常的。这正是「不要只取最简输入」的原因：最简输入有时不再复现 bug。另外，缩小用例本身也是科学方法的实例：报告是 study the data，猜想「关键也许是出现次数而不是具体单词」是 hypothesize，运行 `"a a a b"` 是 experiment。

问题 2：假设 `calculateTotalBonus()` 返回了负数，而 `x` 本应始终 ⩾ 0。请描述你会如何用「切片 + 二分查找 + 断言探针」在最多几次实验内定位问题，并说明为什么 10 分钟法则要求你在此刻放弃「盯着代码看」。

答案：第一步（切片）列出所有能影响 `x` 的行：`x = 0;`、循环 `for (final Sale s : salesList)`、条件 `if (isWorthABonus(s))`、`x += bonus;`，以及 `bonus` 的来源 `final int bonus = getBonus();`。如果 `x` 与 `bonus` 都是 `final`/局部的最小作用域，切片就在这几行内闭合；如果 `x` 是 `static` 字段，切片就扩大到整个程序——所以先确认作用域。第二步（二分）把循环对半分：在第 250 笔、第 500 笔、第 750 笔销售处理完之后分别插一个 `assert x >= 0 : "x=" + x;`（或用调试器在这三处下断点观察）。第一次实验就能确定坏值是出现在前半段、后半段，还是**在循环之外**（若 `x` 在进入循环前就为负，则 bug 在 `getBonus()`：它返回了负的 `bonus`）。第三步（继续二分）在确定的半段里重复对半，直到锁定到某一次迭代、某一笔销售。整个过程只需 O(log n) 次实验，而不是 n 次。关于 10 分钟法则：人的短期记忆容量极小，靠「盯着代码猜」的方法没有任何收敛保证——你无法记住已经排除了哪些可能，也无法保证每次实验都

在缩小范围；而切成「假设——探针——观测」并写下来之后，每一次实验都能**确定性地**排除一半空间，这才是把随机搜索变成系统化搜索的关键。

问题 3：你在调试一个并发程序：某个偶发失败已经连续两天没有再出现，同事说「bug 已经自己好了」。请用本讲的规则说明为什么不能接受这个结论，以及你应该做什么。

答案：这直接对应 Agans 的规则「If YOU didn't fix it, it isn't really fixed」。系统「看起来正常」但有三种可能：你真的改了某个东西（那就应该能说出改的是什么、为什么它有效）；环境变了（例如机器更快、负载更低、JDK 升级），时序窗口关闭了；或者 bug 只是被掩盖（例如异常被别的路径吞掉）。在并发场景下，第二种和第三种极其常见——Reading 21 会展示数据竞争（data race）如何依赖于线程交错，而交错又依赖于负载、调度与硬件。因此正确的做法是：(1) **先复现**：用压力测试、`-ea` 断言、`Thread.sleep` / `CyclicBarrier` 人为放大交错窗口、降低并发度、在生产环境加入日志，把失败拉回可重复状态；(2) **在失败状态下观察**：收集堆栈轨迹、日志、断言信息，形成关于「哪个共享状态被谁改了」的假设；(3) **理解原因**后再改动，并**确认改动导致系统从失败转为成功**；(4) 把复现手段固化为回归测试/压力测试，纳入持续验证。如果做不到复现，就应该诚实地把该 bug 记录在 bug list 上并保留诊断线索，而不是宣称问题已解决。

Reading 14: 递归（Recursion）

> 说明：本讲 sp22 原版使用 TypeScript，本笔记按用户要求提供 Java 代码示例；类型/API 与 sp21（6.031 Java 版）原文保持一致。原文中的 `Array<string>`、`ReadonlyArray<string>`、`fs.Dir` 等 TypeScript/Node 机制已替换为 Java 对应写法（`List<String>`、`Collections.unmodifiableSet`、`java.io.File`）。

概述

本讲讨论「已经拿到规格说明之后，如何实现一个方法」，聚焦其中一个特定技术：**递归（recursion）**。递归函数由**基础情形（base case）**与**递归步骤（recursive step）**两部分定义：基础情形直接算出答案，递归步骤则通过调用自身解决一个**更小或更简单**的子问题，再组合子问题的结果。递归不是万能工具，但对于「问题本身天然递归」与「数据本身天然递归」的两类情形，它往往是更短、更清晰、更安全、更易修改的分解方式。它对「Safe from bugs」的贡献在于：理想的递归实现中所有变量都是 `final`、所有数据不可变、所有方法都是纯函数（pure function），因而**天然可重入（reentrant）**；对「Easy to understand」的贡献在于：递归结构与数学归纳法的证明结构同构，可以直接用「归纳假设成立」的方式推理；对「Ready for change」的贡献在于：可重入的代码可以在并发、回调、相互递归等更多场景下安全使用。代价是栈空间，以及需要警惕的几类典型错误。

核心概念与设计原则详解

递归的定义：基础情形与递归步骤（Base Case & Recursive Step） – **定义与目的**：**基础情形**是问题最简单、最小的实例，无法再分解，直接算出结果；**递归步骤**把一个较大的实例分解成一个或多个更简单的实例，用递归调用解决它们，再组合出原问题的解。它解决的是「如何把一个规格变成实现」的问题，并让实现与数学定义保持同构（易理解）。 – **直观解释（"它是什么？"）**：阶乘有两种定义——乘积形式与递推关系形式。

递推形式 `0! = 1`、`n! = n × (n-1)!` 直接翻译成代码就是：`if (n == 0) return 1; else return n * factorial(n-1);`。其中 `n == 0` 是基础情形，`n > 0` 是递归步骤。 – **关键规则与最佳实践**： – 递归实现**一定**包含基础情形与递归步骤两部分，缺一不可。 – 递归步骤必须把问题实例变换成**更小**的（或更简单的）实例，否则递归永不终止；若每一步都缩小、基础情形在底部，递归的有限性就有保证。 – 一个递归实现可以有**多个**基础情形（例如 Fibonacci 的 `n == 0` 与 `n == 1`）或**多个**递归步骤（例如 `subsequencesAfter` 一次调用自己两次）。 – 基础情形常常对应「空」：空串、空列表、空集合、空树、零。 – 「更小」不一定指数值变小：处理负数时递归到对应的正数（`stringValue(-n, base)`）也是把问题化简了——递归子问题可以在更微妙的意义上「更简单」。

调用栈（Call Stack）与递归的执行过程 – **定义与目的**：用调用栈图示理解递归的执行：栈随着递归调用不断增长，到达基础情形后开始**回卷（unwind）**，每一层把答案返回给调用者。它服务于「易理解」——递归代码短，但执行过程是动态的，图示能把动态过程变成可检查的对象。 – **直观解释（"它是什么？"）**：执行 `factorial(3)`：栈依次压入 `factorial(3)`、`factorial(2)`、`factorial(1)`、`factorial(0)`；`factorial(0)` 不再递归，直接返回 1；随后 `factorial(1)` 返回 1、`factorial(2)` 返回 2、`factorial(3)` 返回 6。而 Fibonacci 的栈形状不同：它不是「稳定增长到最大深度再收缩」，而是**反复地增长又收缩**，因为每一步会产生两个递归调用，其中左侧子树完全算完后才轮到右侧。 – **关键规则与最佳实践**： – 画栈图时要标注每一帧的**参数值**（如 `n = 2`），而不只是方法名。 – 追踪「回卷」阶段：返回值是如何被逐层组合的（`n * (返回值)`）。 – 对 Fibonacci 这类多分支递归，注意**同一条子问题会被重复计算多次**（`fibonacci(3)` 会执行基础情形 `return 1` 共 3 次），这是后面讨论性能与迭代替代的伏笔。 – 递归的可读性来自「相信递归调用会算对」——就像数学归纳法一样，你先假设子问题被正确解决，再关心如何组合。

为问题选择合适的分解（Choosing the Right Decomposition） – **定义与目的**：同一个规格可以有多种递归分解，好的分解「简单、短、易理解、安全、易于修改」；选错分解会让代码变得别扭而脆弱。 – **直观解释（"它是什么？"）**：`subsequences(word)` 要求返回单词的所有子序列（保持字母在原词中的顺序）并以逗号分隔，例如 `subsequences("abc")` 可能返回 `"abc,ab,bc,ac,a,b,c,"`（注意末尾那个逗号，它前面是空子序列，空子序列也是一个合法子序列）。优雅的分解是：取第一个字母，把所有子序列分成「包含这个字母」与「不包含这个字母」两族，两族合起来恰好覆盖全部子序列。 – **关键规则与最佳实践**： – 先写出规格，再尝试**多种**分解，比较哪一种产生的递归步骤最自然。 – 分解应当尽可能是「自我相似」的：子问题与原问题属于同一个规格家族（可直接用已有规格递归）。 – 注意边界与格式细节（如空子序列导致的尾随逗号），它们往往是规格的一部分。 – 如果发现某种分解需要额外的状态或别扭的特判，就考虑换一种分解，或引入辅助方法（下一节）。

辅助方法：用更强的递归假设让分解更简单（Helper Methods） – **定义与目的**：有时为了让递归分解更简单或更优雅，可以给递归步骤一个**更强（或不同）的规格**；实现方式是引入一个带额外参数的私有辅助方法。它服务于「易理解」与「易于修改」。 – **直观解释（"它是什么？"）**：`subsequences()` 的直接递归实现（先把剩余部分的所有子序列算出来，再逐个决定是否加上首字母）需要先 split 再拼接，代码较长。换一种思路：用一个参数 `partialSubsequence` 记录「已经构造到一半的子序列」，递归调用负责用单词剩下的字母把它补全。以 `"orange"` 为例：要么把 `"o"` 选进部分子序列并用 `"range"` 的所有子序列继续扩展，要么跳过

`"o"`（部分子序列仍为空串）并用 `"range"` 继续扩展。 – **关键规则与最佳实践**： – 辅助方法的规格与原方法**不同**——它多了一个参数，扮演迭代实现中「局部变量」的角色，在计算过程中保存临时状态。 – 用**私有（private）辅助方法**实现递归，让公共方法用正确的初值启动它：`subsequences(word)` 只是 `return subsequencesAfter("", word);`。 – **不要把辅助方法暴露给客户端**。递归分解方式纯粹是实现细节；让客户端去正确初始化 `partialSubsequence` 会把实现暴露出去，并削弱你未来修改实现的能力。 – Java 中辅助方法应写成 `private static`，参数用 `final` 修饰并保持不可变，这样它天然是可重入的。 – 判断是否需要辅助方法的信号：递归步骤需要携带「累积状态」或需要额外的边界参数（如 `start/end` 区间）。

不要用静态/全局变量保存递归状态（Reentrancy） – **定义与目的**：递归状态必须放在**参数与局部变量**里，绝不能放在 `static`（全局）字段中。这是「可重入代码（reentrant code）」的核心要求：代码可以安全地被重新进入，即在上一次调用尚未完成时再次被调用。 – **直观解释（"它是什么？"）**：Louis Reasoner 不想用辅助方法，于是把 `partialSubsequence` 写成静态字段。看似省事，实际上：`subsequencesLouis("xy")` 会产生 7 次递归调用，而 `partialSubsequence` 是**跨所有调用共享的一个变量**，每次调用看到的都是「被前面所有调用改过的值」。更糟的是，调用 `subsequencesLouis("c")` 之后再调用 `subsequencesLouis("a")`，第二次调用会被第一次调用残留的状态污染——这正是一次「以外部状态为记忆」的经典灾难。 – **关键规则与最佳实践**： – 可重入代码把状态**完全放在参数与局部变量**中，不使用 `static` 变量或全局变量，也不与其他代码或自身的其他调用共享可变对象的别名。 – 递归是「重入」的一种情形（直接递归）；相互递归是另一种。 – 重入性对「安全」有直接价值：重入代码可以安全用于并发、回调（callback）与相互递归等场景。在 Reading 21（并发）中我们会看到，一个方法可能被不同线程同时调用。 – 「把静态字段初始化好再开始递归」并不能解决问题：递归过程中的多次调用仍会互相踩踏。 – 想要「可重入」，就杜绝可变共享状态：`final` 参数 + 不可变对象 + 纯函数。

选择正确的递归子问题（Choosing the Right Recursive Subproblem） – **定义与目的**：即使算法方向正确，切分问题的**位置**也可能决定实现是自然还是别扭。它服务于「易理解」与「代码简洁」。 – **直观解释（"它是什么？"）**：把整数 `n` 转成某个进制（`2 <= base <= 10`）的字符串表示，`stringValue(16, 10)` 应得 `"16"`，`stringValue(16, 2)` 应得 `"10000"`，负数要带负号，且不得有前导零。从**最左边**（最高位）开始分解看上去符合我们书写的方向，但要先算出位数才能取出最高位；从**最右边**（最低位）开始则极其自然：`n % base` 就是最低位，`n / base` 就是剩下的高位，即 `stringValue(n/base, base) + "0123456789".charAt(n%base)`。 – **关键规则与最佳实践**： – 尝试多种切分方向，选择那个能让递归步骤**最自然**的（这里是「取余 + 整除」）。 – 基础情形要与递归步骤**配套**：若递归步骤最终会把 `n` 除到小于 `base`，那么基础情形就应该是 `n < base`，返回单个数字字符。 – 注意「分解方向」与「结果拼接顺序」的关系：从低位分解，结果要写成「递归结果的字符串 + 当前位字符」。 – 边界值要显式处理（负数、零、以及整型溢出的极端值——见下文 Java 补充说明）。 – 递归步骤的正确性依赖一个隐含事实：`n / base < n`（当 `n >= 1` 且 `base >= 2`），这正是「问题在缩小」的保证。

递归问题 vs 递归数据（Recursive Problems vs. Recursive Data） – **定义与目的**：使用递归有两个常见理由：**问题是天然递归的**（如 Fibonacci、阶乘），或者**数据本身是天然递归的**（如文件系统、树）。后者几乎必然要求递归实现。 – **直观解释（"它是什么？"）**：文件系统由命名文件组成；有些文件是文件夹，可以包含其

他文件。于是文件夹里有文件夹、文件夹里还有文件夹……直到最底层是普通（非文件夹）文件——这就是递归数据。Java 用 `java.io.File` 表示文件系统：`f.getParentFile()` 返回父文件夹（也是 `File`），`f.listFiles()` 返回它包含的文件数组（`File[]`），类型自身就是递归的。求一个文件的完整路径名就是天然的递归：`fullPathname(f.getParentFile()) + "/" + f.getName()`，基础情形是「已在文件系统根部」（`getParentFile() == null`）。 – **关键规则与最佳实践**： – 看到递归数据，就用递归实现；用迭代 + 显式栈虽然可能，但代码会复杂得多。 – 基础情形对应数据的「原子」层级（根目录、叶子节点、空树）。 – 注意 Java 补充说明：`java.nio.file.Path` / `Files` 提供了更清晰的 API，但数据结构本质依然是递归的。 – 递归遍历往往伴随「累积结果」的需求（打印、收集），这时要特别留意可变别名共享的问题（见下文场景 4）。

相互递归（Mutual Recursion） – **定义与目的**：如果方法 A 调用方法 B，B 又调用 A，则 A 与 B 互为递归。它常用于处理递归数据，能把两种「粒度」的遍历逻辑分开表达。 – **直观解释（"它是什么？"）**：遍历文件树可以拆成一对方法：`visitNode(File file)` 负责「处理单个节点」，如果它是目录就调用 `visitChildren(file.listFiles())`；而 `visitChildren(File[] files)` 负责「处理一批节点」，对每个文件调用 `visitNode(file)`。二者递归地互相调用，代码各自都很短。 – **关键规则与最佳实践**： – 相互递归常常出现在针对递归数据的代码中；Reading 17（Recursive Data Types）会大量使用它。 – 这种写法的一个优势是：客户端可以从「单个起点」（`visitNode`）或「多个起点」（`visitChildren`）开始遍历，而不需要任何重复代码。 – 相互递归通常是**有意设计**的；但**意外的**相互递归会导致 bug（无限递归、爆栈）。 – 需要跨调用共享的参数（如前缀 `pattern`）应当作为**参数**层层传递，而不是放在静态字段里——否则再次破坏可重入性。 – 收集结果的两种风格：调用者传入一个可变的 `Set` 供写入（模仿迭代风格），或者每个调用返回不可变集合再合并（更安全，见场景 4）。

累积结果的两种风格：可变容器 vs 不可变结果 – **定义与目的**：递归遍历常常需要把结果「收集起来」。可以用一个可变的容器（`Set`、`List`）一路传下去并就地修改，也可以让每次递归调用**返回新的不可变集合**，由调用者合并。后者更安全、更易理解，也更容易并行化。 – **直观解释（"它是什么？"）**：可变风格像一群人共用一块白板，每个人上去添一笔（要小心谁在什么时候擦掉）；不可变风格像每个人各自写一张便条，最后由上层的 `addAll` 把便条汇总成一份完整清单。 – **关键规则与最佳实践**： – 采用可变容器时，必须在规格中**明确写出**「该参数会被修改」（mutating 参数必须在 postcondition 中说明），并且它应当作为参数传递而不是静态字段。 – 采用不可变风格时，每个调用内部用局部 `Set<File> resultSet = new HashSet<>();` 累积，向上合并用 `resultSet.addAll(visitChildren(...))`，返回时用 `Collections.unmodifiableSet(resultSet)` 防止客户端修改（Java 中对应 sp22 的 `ReadonlyArray<string>`）。 – 结论（与 Reading 08 一致）：不可变性通常是递归中更安全、更好理解的选择。 – 无论哪种风格，都要保证**不会把参数里的可变对象改坏**——递归调用之间共享可变别名是最隐蔽的一类错误（场景 2 会展示它的破坏力）。

可重入代码（Reentrant Code） – **定义与目的**：可重入代码「可以被安全地重新进入」，即在上一次调用尚未完成时再次调用它也是正确的。递归是重入的一个特例。 – **直观解释（"它是什么？"）**：`factorial(n-1)` 可以在 `factorial(n)` 还没算完时被调用，是因为每次调用都有**自己的**参数与局部变量（各自的栈帧）。反之，如果实现依赖某个共享的「当前进度」，重入就会互相干扰。 – **关键规则与最佳实践**： – 可重入代码把状

态完全保存在参数与局部变量中，不使用 `static`/全局变量，不与其他代码或自身的其他调用共享可变对象别名。 – 直接递归与相互递归都会造成重入；并发程序则会造成「真正的」同时重入。 – 尽量让代码可重入：它更安全，并能在并发、回调、相互递归等更多场景下使用。 – 不可变性是可重入性的天然盟友：不可变参数无需防御性拷贝，也不会被别的递归调用改掉。

递归 vs 迭代：取舍（Recursion vs. Iteration） – **定义与目的**：两类实现各有代价与收益，选择取决于问题的自然形态与状态管理方式。它同时涉及「易理解」「安全」与资源消耗。 – **直观解释（"它是什么？"）**：阶乘与整数转字符串既可以用 `for` 循环写，也可以用递归写。迭代版本必然包含可重新赋值的变量（`fact = fact * i`）或在迭代中被修改的可变对象；要理解程序，你得在脑中想象各个时间点的状态快照。递归版本则可以让所有变量都是 `final`、所有数据都不可变、所有方法都是纯函数——行为可以仅由「参数 → 返回值」的关系来描述，没有任何副作用。 – **关键规则与最佳实践**： – 使用递归的三个理由：问题是天然递归的、数据是天然递归的、想更充分地利用不可变性（函数式风格，functional programming）。 – 递归的**代价是空间**：调用栈会临时占用内存，而栈的大小是有限的。若最大深度与输入规模成**对数**关系（如递归版二分查找），通常不是问题；若成**线性**关系（如阶乘、Fibonacci），栈深度就可能成为能处理的最大输入规模的限制。 – 迭代的代价是状态复杂：理解程序需要追踪随时间变化的状态快照。 – 工程上的折中：优先天然递归的分解；对深度线性增长的递归，考虑改写成迭代、引入累积参数（把递归变成尾调用形状）或使用显式栈。 – 关于**尾递归（tail recursion）**的补充说明（不属于 6.031 原文）：若递归调用是方法体里最后执行的动作（其返回值直接作为本方法的返回值），理论上可以复用当前栈帧，从而把空间降到 O(1)。但 **Java 虚拟机并不保证做尾调用优化**（JVM 规范允许但不要求），因此 Java 中的尾递归写法通常**仍会爆栈**；需要真正 O(1) 空间的写法时，应改写为循环，或使用支持尾调用优化的语言特性（如 Scala/Kotlin 的 `tailrec`，属补充说明）。6.031 的结论因此是：**在选择递归前先估计栈深度**。

递归与数学归纳法的对应（Recursion and Proof by Induction） – **定义与目的**：递归实现的结构与数学归纳法的证明结构一一对应，这既解释了「为什么可以相信递归写对了」，也给出了调试与验证的方法。 – **直观解释（"它是什么？"）**：归纳法有「基础情形」与「归纳步骤」；递归实现有 base case 与 recursive step。归纳步骤中我们假设命题对更小的 `n` 成立（归纳假设），递归步骤中我们假设递归调用会算对、只关心如何组合结果。数据结构方面，二叉树等递归数据结构同样具备「基础情形 + 递归步骤」的结构。 – **关键规则与最佳实践**： – 写递归时可以分两步推理：先验证基础情形正确（对应归纳基础）；再假设「对所有更小的实例递归调用都返回正确结果」（对应归纳假设），只验证组合逻辑正确。 – 归纳假设的「对更小实例成立」这一前提，正是递归步骤必须**缩小问题规模**的要求——如果递归步不缩小问题，归纳假设就不适用，而现实中对应的就是无限递归。 – 用归纳法还能帮助你设计**测试**：为每个基础情形写测试，为每个递归步骤写至少一个「比基础情形大一级」的测试（对应 Reading 03 的测试策略）。 – 归纳法只证明「若终止则正确」，终止性需要单独论证：每一步严格变小 + 基础情形可达。

递归实现的三类常见错误（Common Mistakes） – **定义与目的**：把调试经验固化成检查清单，能在写代码时就避开大部分递归 bug。 – **直观解释（"它是什么？"）**：三类典型错误是：基础情形完全缺失，或需要多个基础情形但没有全部覆盖（例如 Fibonacci 只写了 `n == 0`）；递归步骤没有缩小到更小的子问题，导致递归不收敛；以及在递归调用之间**无意间共享并修改**了指向可变数据结构的别名。 – **关键规则与最佳实践**： – 写

完后先自问：所有基础情形都覆盖了吗？每一步都严格变小了吗？有没有把可变对象作为参数并在递归中修改它？ – 调试递归时优先检查这三条，而不是盯着代码猜。 – 好消息是：迭代实现中的无限循环，在递归实现中通常会变成 `StackOverflowError`——**有 bug 的递归程序有时失败得更快**，前提是你不把它 catch 掉（参见 Reading 13 的调试纪律）。 – 「共享可变别名」也是最容易在**累积结果**模式中犯的错误：把一个 `List` / `Set` 一路传下去并在递归中修改，很可能把调用者给的数据改坏。

代码示例与对比分析

场景 1：用静态变量保存递归状态（Louis 的写法）

❌ *错误代码*

```
public class Subsequences {
    // 错误：把“进行中的子序列”放在静态字段里，所有递归调用共享它
    private static String partialSubsequence = "";

    public static String subsequences(String word) {
        partialSubsequence = "";                    // 试图“初始化好再开始递归”
        return subsequencesLouis(word);
    }

    public static String subsequencesLouis(String word) {
        if (word.isEmpty()) {
            // base case
            return partialSubsequence;
        } else {
            // recursive step
            String withoutFirstLetter = subsequencesLouis(word.substring(1));
            partialSubsequence += word.charAt(0);
            String withFirstLetter = subsequencesLouis(word.substring(1));
            return withoutFirstLetter + "," + withFirstLetter;
        }
    }
}
```

【错误代码的问题】 1. `partialSubsequence` 是 `static` 可变状态，所有递归调用（以及所有线程）共用同一个变量；`subsequences("xy")` 产生的 7 次 `subsequencesLouis` 调用看到的都是被前面调用改过的值，结果完全错误。 2. 它不是可重入的：先调用 `subsequences("c")` 再调用 `subsequences("a")`，第二次调用会受到第一次残留状态的污染（用户先看到的两个结果 `"c"` / `"c,ca"` 就是这样来的）。 3. 把 `partialSubsequence` 公开并要求客户端「调用前先置空」，等于把实现细节写进规格，客户端的正确性变成了实现正确性的前提——抽象边界被破坏。 4. 即使「在公共方法里先置空」，也不解决问题：递归过程中的多次调用仍会互相覆盖状态。

✅ *正确代码*

```
public class Subsequences {
    /**
     * @param word consisting only of letters A-Z or a-z
     * @return all subsequences of word, separated by commas,
     *         where a subsequence is a string of letters found in word
     *         in the same order that they appear in word.
     */
    public static String subsequences(String word) {
        return subsequencesAfter("", word);
    }

    /**
     * @param partialSubsequence a subsequence-in-progress, consisting only of letters A-Z
     * @param word consisting only of letters A-Z or a-z
     * @return all subsequences of word, separated by commas,
     *         with partialSubsequence prefixed to each one
     */
    private static String subsequencesAfter(final String partialSubsequence, final String w
        if (word.isEmpty()) {
            // base case
            return partialSubsequence;
        } else {
            // recursive step
            return subsequencesAfter(partialSubsequence, word.substring(1))
                 + ","
                 + subsequencesAfter(partialSubsequence + word.charAt(0), word.substring(1)
        }
    }
}
```

【为什么这样更好】 所有状态都在**参数**里：每次递归调用都有属于自己的 `partialSubsequence` 与 `word` 值（各自的栈帧），因此天然可重入，也不会被其他调用或线程污染。辅助方法是 `private` 的，客户端只看到原来的规格 `subsequences(String)`，实现细节被完整地隐藏在抽象边界之后。辅助方法的规格更强——它可以返回「以给定前缀开头的所有子序列」——正是这个更强的（更一般的）假设让递归步骤变得对称而简洁：要么不选首字母，要么选首字母，两条分支各自递归。

【代码对比解说】 两个版本「思路相同」，差别全在**状态放哪里**。放在静态字段里，状态就成了全局的、跨调用的、不可控的；放在参数里，状态随栈帧自然分层，函数的行为只由输入决定。注意正确版本还顺带修好了一个可重入性问题：它把 `partialSubsequence + word.charAt(0)` 写成**新的字符串**（字符串不可变），而不是原地 `+=`——原地修改会使同一层的两个递归调用互相干扰。另外，公共方法只有一行 `return subsequencesAfter("", word);`，这正是「用正确的初值启动递归」的标准模式。

【设计原则透视】 这是 Reading 08（Immutability）与 Reading 10（ADT）在本讲的核心落点：**可重入性 = 无共享可变状态**。`static` 可变字段在规格上等价于「隐式的、全局的前置/后置条件」，它让方法的规格无法用「参数 → 返回值」描述，从而使推理（和测试）都变得不可靠。辅助方法则展示了「抽象边界」的价值：把更强的规格留给私有辅助方法、把弱而稳定的规格留给公共 API，正是 Reading 06/07 中「规格应该足够强、但不过度暴露实现」的直接应用。Java 里若真有跨调用共享的需求，正确做法是通过参数显式传递（或使用不可变值），而不是依赖 `static`。

场景 2：递归调用之间共享可变别名并破坏它

❌ *错误代码*

```
/**
 * @param list must be nonempty
 * @return the maximum integer in list
 */
public static int maxList(List<Integer> list) {
    return maxOfRange(list, 0, list.size()-1);
}

// helper method -- finds the max of list[start]...list[end]
private static int maxOfRange(List<Integer> list, int start, int end) {
    if (start == end) {
        return list.remove(start);          // 在递归调用之间破坏了共享的 list
    } else {
        int midpoint = (start + end + 1)/2; // 切分点也错了：区间大小为 2 时右侧会越界
        return Math.max(maxOfRange(list, start, midpoint),
                        maxOfRange(list, midpoint+1, end));
    }
}
```

【错误代码的问题】 1. `list.remove(start)` 在基础情形**修改了所有递归调用共享的同一个列表**（调用者传入的 `list` 也被改坏），后续任何一次 `list.get(i)` 的下标含义都变了，结果不可预测；调用结束后调用者的列表也被清空，属于典型的「破坏参数」。 2. `midpoint = (start + end + 1)/2` 配合第二次调用 `(midpoint+1, end)`，在区间大小恰好为 2 时产生非法的 `(end+1, end)` 区间，`start == end` 判断不成立，递归**不收敛**，最终 `StackOverflowError`。 3. 这两类错误恰好命中递归「三大常见错误」中的两条：递归步未缩小问题、共享可变别名被破坏性修改。 4. 因为 `list` 的抽象值在递归过程中不断变化，这个方法的规格「返回 list 中的最大值」变得无法用输入/输出关系描述，测试结果也依赖调用顺序。

✅ *正确代码*

```
/**
 * @param list must be nonempty
 * @return the maximum integer in list
 */
public static int maxList(List<Integer> list) {
    return maxOfRange(list, 0, list.size()-1);
}

/**
 * helper method -- finds the max of list[start]...list[end]
 * @param list must be nonempty and unmodified during the call
 * @param start,end 满足 0 <= start <= end < list.size()
 */
private static int maxOfRange(List<Integer> list, int start, int end) {
    if (start == end) {
        return list.get(start);             // 只观察，不修改
    } else {
        int midpoint = (start + end) / 2;   // 两个子区间都非空且严格更小
        return Math.max(maxOfRange(list, start, midpoint),
                        maxOfRange(list, midpoint + 1, end));
    }
}
```

【为什么这样更好】 只用 `list.get(start)` 读取，绝不在递归中修改共享结构，于是「所有递归调用看到的 `list` 是同一个不可变内容」这一点成为可依赖的前提，方法的抽象值稳定、结果确定，也不会破坏调用者的数据。切分点改为 `(start + end) / 2` 后，两个子区间 `[start, midpoint]` 与 `[midpoint+1, end]` 都严格小于原区间且都非空，因此每一次递归调用都把问题**严格缩小**，终止性有保证。

【代码对比解说】 这组对比把「共享可变别名」的破坏力暴露得很直观：`remove` 不仅破坏了列表，还让 `start/end` 这两个下标失去意义——它们本来描述的是**原始列表**的位置区间，一旦列表被删掉元素，区间语义就崩了。即使修正了切分点（这是纯粹的算术 bug），共享可变结构的问题依然存在。正确版本体现了一个通用原则：**递归参数中出现的可变对象应该被当作只读**；如果确实需要在遍历中累积结果，就采用「不可变返回 + 上层合并」的模式（场景 4），或让被修改的容器成为显式文档化的参数。此外注意辅助方法的规格里写明了前置条件 `0 <= start <= end < list.size()`——即使它是私有方法，写清前置条件也能让「递归步是否合法」变得可检查。

【设计原则透视】 这是 Reading 11（AF/RI）中「抽象值稳定性」的问题：`List` 的抽象值（元素序列）在递归中被改变了，于是任何依赖「下标 ↔ 元素」对应关系的推理都失效。它也是 Reading 08 的核心警示：**别名（aliasing）是可变的代价**，当两个引用指向同一个可变对象时，通过一个引用做的修改会从另一个引用「意外地」可见。递归（尤其是分治式递归）天然会产生多个「同时活跃」的引用，因此是可重入性风险最高的场景之一。最后，它说明「三大常见错误」为什么值得背下来当检查清单：本场景同时命中两条，而这类 bug 往往表现为结果错误或爆栈，而不是清晰的异常信息。

场景 3：基础情形不配套、递归方向别扭（整数转字符串）

❌ *错误代码*

```
/**
 * @param n integer to convert to string
 * @param base base for the representation. Requires 2<=base<=10.
 * @return n represented as a string of digits in the specified base, with
 *         a minus sign if n<0. No unnecessary leading zeros are included.
 */
public static String stringValue(int n, int base) {
    if (n < 0) {
        return "-" + stringValue(-n, base);
    } else if (n == 0) {
        return "0";                                    // 基础情形与递归步不配套
    } else {
        return stringValue(n/base, base) + "0123456789".charAt(n%base);
    }
}
```

【错误代码的问题】 1. 基础情形 `n == 0 → "0"` 与递归步不配套：递归步把问题降到 `n/base`，但只有 `n` 除到 ⩽ 9 时才真正需要停下来。结果是 `stringValue(16, 10)` = `stringValue(1, 10) + "6"` = `("0" + "1") + "6"` = `"016"`，**产生前导零，违反规格**。 2. 规格说 `2 <= base <= 10`，但数字表 `"0123456789"` 只有 10 个字符；一旦调用者传入 `base = 16`（超出前置条件），`charAt(10)` 会抛 `StringIndexOutOfBoundsException`——错误信息离真正的原因很远。 3. `stringValue(-n, base)` 在 `n == Integer.MIN_VALUE` 时会**溢出**（`-Integer.MIN_VALUE == Integer.MIN_VALUE`，仍为负数），于是递归永不缩小，最终 `StackOverflowError`。补充说明：这是 Java 整型语义特有的边界情形，规格中应当明确 `n` 的取值范围或显式处理它。 4. 若把递归改成「从最高位开始」的分解，还需要先计算位数，递归步骤会变得复杂且容易错。

✅ *正确代码*

```
/**
 * @param n integer to convert to string
 * @param base base for the representation. Requires 2<=base<=10.
 * @return n represented as a string of digits in the specified base, with
 *         a minus sign if n<0. No unnecessary leading zeros are included.
 */
public static String stringValue(int n, int base) {
    if (n < 0) {
        // 递归子问题在更微妙的意义上更简单：化为正整数
        return "-" + stringValue(-n, base);
    } else if (n < base) {
        // base case：已是一位数字，直接取出；与递归步 n -> n/base 配套
        return "0123456789".substring(n, n + 1);
    } else {
        // recursive step：最低位用 n%base，高位用 n/base（严格变小）
        return stringValue(n / base, base) + "0123456789".charAt(n % base);
    }
}
```

【为什么这样更好】 基础情形 `n < base` 与递归步 `n / base` 严格配套：每次递归都保证 `n / base < n`（因为 `base >= 2` 且 `n >= base >= 2`），因此问题**必然收敛**到基础情形，且结果天然没有前导零（最高位一定落在基础情形里一次取出）。负数的处理把问题归约到正数，属于「更简单」而非「更小」的化简，同样合法。

【代码对比解说】 错误的版本错在一个非常典型的思路上：「我知道 0 是基础情形」——这对阶乘是对的，但对于「逐位输出」的问题，0 并不是**唯一**能直接返回的输入，任何 `0 <= n < base` 都能直接返回一位数字。基础情形的选择必须由递归步的形状反推：递归步每次做 `n / base`，那么「不能再整除」的状态就是 `n < base`。另一处对照是分解方向：从最高位开始分解需要先算位数（要么额外循环，要么引入辅助方法），而从最低位开始只需 `%` 与 `/`，因此递归步变成一行、拼接顺序也自然（先递归结果、再当前位字符）。

【设计原则透视】 这组对比同时展示了「基础情形与递归步必须配套」和「规格的边界要写清楚」两点。前置条件 `2 <= base <= 10` 之所以必须写明，正是因为实现只支持 10 个数字字符；若将来要支持 `base = 16`，正确做法是修改数字表并更新规格（Ready for change），而不是让 `charAt` 抛出一个含义晦涩的异常。另一个与 Reading 06/07 有关的点：规格里写「No unnecessary leading zeros」不是可有可无的修辞——它是一条**可测试的后置条件**，恰好就是两个版本的区别所在（`"016"` vs `"16"`）。最后，`Integer.MIN_VALUE` 提醒我们：递归的终止性论证依赖「每一步严格变小」，而算术溢出会悄悄破坏这个前提，因此在写递归时也要像 Reading 13 那样对边界值保持警觉。

场景 4：把可变容器一路传下去就地修改（收集遍历结果）

❌ *错误代码*

```
public class FileWalker {
    // 错误一：可变结果容器放在静态字段里，且“顺带”决定了客户端拿到的东西
    private static Set<File> resultSet = new HashSet<>();
    // 错误二：pattern 也放在静态字段里，调用者无法并行/嵌套使用
    private static String pattern = "";

    public static void visitNode(File file) {
        if (file.getName().startsWith(pattern)) {
            resultSet.add(file);
        }
        if (file.isDirectory()) {
            visitChildren(file.listFiles());
        }
    }

    public static void visitChildren(File[] files) {
        for (File file : files) {
            visitNode(file);
        }
    }

    public static Set<File> findMatching(File root, String p) {
        pattern = p;
        // 忘了清空 resultSet：上一次调用的结果会残留，结果集越来越大
        visitNode(root);
        return resultSet;       // 直接把内部可变容器交给客户端：客户端可以随意改坏它
    }
}
```

【错误代码的问题】 1. `resultSet` 与 `pattern` 都是 `static` 可变状态，方法不可重入：两次调用互相污染，第二次的结果里混着第一次的残留（忘记清空更是雪上加霜），并发调用会得到完全错误的结果。 2. 返回 `resultSet` 等于**泄露表示（rep exposure）**：客户端拿到内部容器后可以随意增删，甚至可以在别的线程里一边遍历一边修改，导致 `ConcurrentModificationException` 或更隐蔽的错误。 3. 调用者无法从「多个起点」（一批文件）开始遍历而不重复代码，因为遍历状态藏在全局。 4. 方法的规格无法用「参数 → 返回值」表达（它依赖并修改全局状态），因此难以测试、难以推理。

✅ *正确代码*

```
import java.io.File;
import java.util.Collections;
import java.util.HashSet;
import java.util.Set;

public class FileWalker {
    /**
     * 在以 file 为根的子树中，找出名字以 pattern 开头的文件或文件夹。
     * @param file    a file in the filesystem
     * @param pattern 前缀，非 null；空串表示匹配全部
     * @return 所有匹配的文件与文件夹；返回的集合不可修改
     */
    public static Set<File> visitNode(File file, String pattern) {
        Set<File> resultSet = new HashSet<>();
        if (file.getName().startsWith(pattern)) {
            resultSet.add(file);
        }
        if (file.isDirectory()) {
            resultSet.addAll(visitChildren(file.listFiles(), pattern));
        }
        return Collections.unmodifiableSet(resultSet);
    }

    /**
     * 从一批根文件出发做同样的搜索。
     * @param files   一批文件，非 null
     * @param pattern 前缀，非 null
     * @return 所有匹配的文件与文件夹；返回的集合不可修改
     */
    public static Set<File> visitChildren(File[] files, String pattern) {
        Set<File> resultSet = new HashSet<>();
        for (File file : files) {
            resultSet.addAll(visitNode(file, pattern));
        }
        return Collections.unmodifiableSet(resultSet);
    }
}
```

【为什么这样更好】 状态（结果集与模式）分别成为**局部变量**与**参数**，因此两个方法都可重入：可以安全地嵌套调用、并发调用，也可以从单个起点（`visitNode`）或多个起点（`visitChildren`）启动遍历而无需重复代码。上层用 `resultSet.addAll(...)` 合并子结果，符合不可变风格；返回时用 `Collections.unmodifiableSet(...)` 包一层，防止客户端修改内部集合（对应 sp22 的 `ReadonlyArray<string>` 语义）。Java 补充说明：更彻底的不可变做法是使用 `Set.copyOf(resultSet)`，它在 Java 10+ 可用，返回的集合同样是不可修改的快照。

【代码对比解说】 两种写法的差别是「谁持有状态」。可变风格像把所有东西写在一块共用的白板上，任何一次调用的结果都可能被下一次调用看到；不可变风格让每次调用各自准备一张便条，由调用者汇总。注意 `Collections.unmodifiableSet` 只提供**不可修改的视图**：它防止客户端改坏内部状态，但若内部集合之后被别的代码改动，视图也会跟着变；因此更严格的写法是返回一个**副本**（`Set.copyOf`）或返回前就不再持有该

集合的引用。另外，这一版把 `pattern` 变成了显式参数——这不只是「更干净」：它让「在哪棵树、用什么前缀搜索」成为完全由输入决定的事情，从而让方法可以委托、可以递归、可以测试，也让将来的改动（比如加上「忽略大小写」选项）只影响参数列表而不影响全局状态。

【设计原则透视】 这组对比把 Reading 08（Immutability）、Reading 10（ADT）与 Reading 11（AF/RI）串起来：返回内部可变容器属于典型的**表示泄露**，它直接破坏 ADT 的抽象边界（客户端可以绕过所有 observer 直接改状态）；同时，把状态放进静态字段使方法失去「纯函数」性质，也就失去了可重入性——而可重入性正是本讲反复强调的递归代码的核心优势，并且会在 Reading 21（并发）与 Reading 20（回调）中再次成为关键。用一个更强的规格（`visitChildren` 接受一批起点）来简化实现，也正是辅助方法思想的延续。

与其他设计原则的关联

- **Reading 03（Testing）**：递归与数学归纳法的对应直接给出测试策略——为每个基础情形写测试，再为「比基础情形大一级」的递归步骤写测试；`StackOverflowError` 也提示应加入边界/深度测试。
- **Reading 06/07（Specifications / Designing Specs）**：递归分解的目标是「让规格驱动实现」；辅助方法的存在正说明「更强的规格可以让递归更简单」，而把辅助方法私有化则是维护抽象边界的必然要求。
- **Reading 08（Immutability）**：本讲的核心盟友。理想的递归实现中所有变量为 `final`、所有数据不可变、所有方法为纯函数；反之，共享可变别名是递归 bug 的头号来源（场景 2、场景 4）。
- **Reading 09（Avoiding Debugging）**：递归的常见错误（缺基础情形、不收敛、共享可变别名）正是「让 bug 尽早暴露」的反面；`StackOverflowError` 比起死循环是更快的失败，配合 Reading 13 的系统化调试效率更高。
- **Reading 10（Abstract Data Types）**：递归遍历是 ADT observer 的常见实现方式；返回内部可变容器会破坏抽象边界，返回不可变集合则不会。
- **Reading 11（Abstraction Functions & Rep Invariants）**：递归中修改共享可变结构会改变其**抽象值**，让「下标 ↔ 元素」等基于 AF 的推理失效；写递归时用 RI 检查参数是否被改坏。
- **Reading 13（Debugging）**：递归不收敛通常表现为 `StackOverflowError`；用切片与断言可以在递归深度增加的过程中定位「哪一层开始出错」。
- **Reading 17（Recursive Data Types）**：本讲预告了递归数据（文件系统、树、语法树）与相互递归，下一部分会系统展开，并说明为什么对递归数据必须以递归方式访问。
- **Reading 20/21（Callbacks / Concurrency）**：可重入性是这两讲的前提——回调可能在原方法未返回时被触发，并发则会让方法真正被同时进入；本讲的「无静态可变状态」是可重入的必要条件。

关键要点

- **两部分结构**：任何递归实现都是「基础情形 + 递归步骤」；基础情形要覆盖所有「最小实例」（可能不止一个），递归步骤必须把问题**严格变小或变简单**。
- **状态放参数、辅助方法不暴露**：需要累积状态时使用**私有辅助方法 + 参数**，公共方法用正确初值启动递归；绝不用 `static` 可变字段保存递归状态，客户的规格也不应因你的分解方式而改变。

- **分解方向要选对**：从低位（`n % base`、`n / base`）而不是高位切分，往往能得到最自然的递归步与最简洁的拼接表达式。
- **优先不可变与纯函数**：所有变量 `final`、结果用不可变集合返回（`Collections.unmodifiableSet` / `Set.copyOf`），既安全又天然可重入。
- **先估栈深度再选递归**：深度随输入对数增长（如递归二分）通常安全；线性增长（阶乘、朴素 Fibonacci）要警惕 `StackOverflowError`，Java 不保证尾调用优化。

常见陷阱与注意事项

1. **基础情形缺失或覆盖不全** → 递归永不触底，抛出 `StackOverflowError`；典型例子是 Fibonacci 只写 `n == 0`，或「逐位输出」问题误用 `n == 0` 作为唯一基础情形（并因此产生前导零）。
2. **递归步骤没有缩小问题** → 递归不收敛（例如切分点算错导致子区间比原区间还大），表现为无限递归或错误结果；迭代写法中这相当于死循环。
3. **在递归之间共享并修改可变别名** → 通过一个引用做的修改对另一个引用可见，导致结果错误、下标语义失效、调用者的数据被破坏；`list.remove(start)` 是教科书级例子。
4. **用静态/全局变量保存递归状态，或把辅助方法暴露给客户端** → 前者使代码不可重入、多次调用互相污染、并发下结果完全错误（「先初始化再递归」并不能修好它）；后者迫使客户端正确初始化实现细节，破坏抽象边界。正确做法是 `private` 辅助方法 + 公共方法一行启动。
5. **忽略整型边界与深递归** → `-Integer.MIN_VALUE` 仍为负数导致递归不收敛；朴素 Fibonacci 的指数级重复计算与线性栈深度会让程序在大输入下爆栈或变得极慢。
6. **混用可变与不可变结果收集风格** → 一边传可变容器一边又试图返回不可变结果，容易出现「返回了内部集合的视图但内部仍在被修改」的隐蔽 bug；固定采用一种清晰风格。

思考题（带答案）

问题 1：对比 `subsequences()` 的两种实现——直接递归（先算 `subsequences(restOfWord)` 再逐个加上/不加首字母）与 `subsequencesAfter(partialSubsequence, word)` 辅助方法版本。它们的**规格**有什么本质区别？为什么说「辅助方法的规格更强」能让实现更简单？

答案：直接递归版本的规格与原方法完全一致：`subsequences(word)` 返回 `word` 的所有子序列。因此递归步骤必须在「子问题的答案」上做二次加工——把 `subsequencesOfRest` 的字符串结果按逗号 `split` 开，再对每个子序列分别生成「带首字母」和「不带首字母」两个版本并重新拼接，最后还要处理多余的前导逗号。辅助方法版本的规格是 `subsequencesAfter(partialSubsequence, word)`：返回「以 `partialSubsequence` 为前缀的、`word` 的所有子序列」。这个规格比原规格**更强/更一般**，因为它对任意前缀都成立（原规格只是前缀为空串的特例）。更强的规格让递归步骤变得对称而简单：每一步只需在两条分支中各选一次——不选首字母（前缀不变）或选首字母（前缀加上首字母），两条分支各自递归到 `word.substring(1)`；基础情形 `word.isEmpty()` 直接返回前缀本身。这样就不需要 split、不需要字符串预处理、也不需要额外的前导逗号特判。公共方法 `subsequences(word)` 退化为一行 `return subsequencesAfter("", word);`：分解方式是实现细节，不应污染客户端的规格，因此辅助方法必须是 `private`（在 Java 中还应当是 `static`，因为它不依赖实例状态——这同时让它天然可重入）。

问题 2：下面的辅助方法为什么既可能给出错误结果、又可能抛出 `StackOverflowError`？请指出两处不同性质的 bug，并说明如何用「三大常见错误」检查清单发现它们。

```
private static int maxOfRange(List<Integer> list, int start, int end) {
    if (start == end) { return list.remove(start); }
    int midpoint = (start + end + 1)/2;
    return Math.max(maxOfRange(list, start, midpoint), maxOfRange(list, midpoint+1, end));
}
```

答案：第一处 bug 是「共享可变别名被破坏性修改」：`list.remove(start)` 修改了所有递归调用共享的同一个列表（也是调用者传入的对象），此后 `start/end` 这些下标所描述的位置与原列表不再对应，同时还破坏了 `list.size()`，让后续 `list.get(i)` 可能越界或读到错误的元素；调用返回后调用者的列表也被改变了。第二处 bug 是「递归步没有严格缩小问题」：`midpoint = (start + end + 1)/2` 配合右侧区间 `(midpoint+1, end)`，当区间大小为 2（例如 `start=0, end=1`）时得到 `(2, 1)`，此时 `start == end` 为假、`start > end`，于是每次递归都停留在同一个非法区间并持续调用自己，最终 `StackOverflowError`。用三大常见错误清单检查：①基础情形是否存在且覆盖所有最小实例——存在（`start == end`），但区间被破坏成 `start > end` 后就不再可用了；②递归步是否缩小问题——不满足；③是否有共享可变别名被修改——满足（存在破坏性修改）。修复方式：把 `list.get(start)` 换成读取而非删除，并把切分点改为 `(start + end)/2`，使 `[start, midpoint]` 与 `[midpoint+1, end]` 都严格更小且非空。

问题 3：为什么说理想的递归实现「天然可重入」，而可重入性对并发程序（Reading 21）特别重要？请用 `factorial` 与 `subsequencesLouis` 两个例子对比说明，并解释为什么 Java 的尾递归通常仍然会爆栈（补充说明）。

答案：可重入代码的定义是「在上一次调用尚未完成时，可以再次被安全地调用」。`factorial(n)` 满足这一点：它的全部状态（参数 `n`、局部变量、返回值）都保存在自己的栈帧里，`factorial(n-1)` 的递归调用对 `factorial(n)` 没有任何影响，因此多个线程同时调用 `factorial` 也互不干扰——行为完全由「输入 → 输出」决定，没有副作用，属于纯函数。`subsequencesLouis` 则相反：它把进行中的部分子序列放在 `static` 字段 `partialSubsequence` 里，这个字段是**所有调用、所有线程共享**的可变状态；同一次递归中的 7 次调用会依次读到被前面调用修改过的值，两次连续调用之间也会互相污染，因而不可重入、不可并发。在并发场景下，不可重入的代码会直接导致数据竞争（data race）与不可重现的错误结果，所以 Reading 21 强调共享可变状态必须被保护或消除；而「把状态放进参数、保持对象不可变」是最简单的消除手段。补充说明：尾递归（递归调用是方法最后执行的动作，其返回值直接作为本方法返回值）在支持尾调用优化的语言中可以复用栈帧，把空间降到 O(1)；但 **Java 虚拟机规范允许而不要求尾调用优化**，主流 JVM 实现不做该优化，因此 Java 里写成尾递归形状的方法仍然会随着递归深度增长栈帧并最终抛 `StackOverflowError`。在 Java 中若需要 O(1) 空间，应改写为迭代循环（或使用显式的栈数据结构），而不是依赖尾递归；6.031 的实用结论是：**先估计最大栈深度与输入规模的关系（对数还是线性），再决定用递归还是迭代**。

Reading 15: 相等性（Equality）

> 说明：本讲 sp22 原版使用 TypeScript，本笔记按用户要求提供 Java 代码示例；类型/API 与 sp21（6.031 Java 版）原文保持一致。sp22 用 `===` / `equalValue()` 的场合，Java 对应 `==` / `equals()`；sp22 因语言限制改用 Python `__eq__` / `__hash__` 讨论哈希的部分，Java 对应 `equals()` / `hashCode()`。

概述

本讲要解决的问题是：在为一个抽象数据类型（ADT）定义「两个值什么时候相等」时，该如何做出正确的设计。核心结论有三条：任何相等操作都必须是**等价关系**（自反、对称、传递）；**不可变类型**的相等应以**抽象函数（abstraction function, AF）**为基础——即两个对象相等当且仅当它们表示同一个抽象值，这要求覆盖 `equals()` 与 `hashCode()`；**可变类型**的相等应以**引用相等（行为相等）**为基础——即不覆盖 `equals()`，因为「观察相等」会随时间变化，从而破坏哈希表的表示不变量。它与「Safe from bugs」直接相关：错误的 `equals` / `hashCode` 会让 `HashSet` / `HashMap` 静默丢失元素，也会让测试断言给出错误结论；与「Easy to understand」相关：客户端与读者都期待类型提供恰当的相等操作，否则会困惑于「明明一样却不相等」；与「Ready for change」相关：正确的不可变类型相等把「引用是否共享」这个实现细节对客户端隐藏起来。

核心概念与设计原则详解

等价关系（Equivalence Relation） – **定义与目的**：定义在类型 T 上的相等操作可看作二元关系 `E ⊆ T × T`；合格相等操作必须满足三条性质：**自反（reflexive）** `(t,t) ∈ E ∀t ∈ T`、**对称（symmetric）** `(t,u) ∈ E ⇒ (u,t) ∈ E`、**传递（transitive）** `(t,u) ∈ E ∧ (u,v) ∈ E ⇒ (t,v) ∈ E`。用 `==` / `equals` 的语言写就是 `t == t`、`t == u ⇒ u == t`、`t == u ∧ u == v ⇒ t == v`。它保证任何依赖相等的操作（集合成员判断、列表查找、去重、测试断言）行为一致、可预测。 – **直观解释（"它是什么？"）**：物理世界里每个对象都是独特的（两片「相同」的雪花在空间位置上终究不同），但它们只有「相似程度」；而数学与语言世界里可以有多个名字指同一个事物——`1+2`、`√9`、`3` 是同一个理想数学值的三种写法。等价关系就是「同一个事物」这个概念必须满足的形式条件：自己和自己一样（自反）、跟别人一样就是互相一样（对称）、一样的东西链条不会断裂（传递）。 – **关键规则与最佳实践**： – 相等操作**永远**必须是等价关系，否则会产生「令人惊讶的行为与 bug」。 – 违反自反的典型后果：把元素插入集合后，自己却查不到自己（`is_member(set[0], set)` 返回 false）。 – 违反对称的典型后果：集合的成员判断结果**依赖插入顺序**（sp21 的练习里，先插 `"007"` 再插 `7` 与反过来插，`len(set)` 结果不同）。 – 违反传递的典型后果：「差不多相等」的链式传播（a 与 b 差 5 秒算相等、b 与 c 差 5 秒算相等，于是 a 与 c 也算相等，尽管它们差 10 秒）导致查找、去重、排序全部失序。 – 检验「不满足自反」只需**一个**对象作为反例；检验对称需要两个；检验传递需要三个——这也解释了为什么传递性最容易被实现者忽略。

为什么必须严格：Python `None` 与自动类型转换的反例 – **定义与目的**：这个思想实验用最直白的方式说明「破坏等价关系会怎样」，而它是语言设计者真实面对的选择。 – **直观解释（"它是什么？"）**：假设我们规定 `None == x` 对**所有** x 都是 false（「None 太糟糕了，谁都不该等于它」）。那么这个规则首先违反**自反性**：

`None == None` 也是 false。后果：把一个值 `x` 插入集合两次，如果 `x` 是 `5`，集合长度为 1（正常去重）；但如果 `x` 是 `None`，因为 `is_member(None, set)` 永远为 false，`None` 会被重复插入，集合长度为 2，而且连 `is_member(set[0], set)` 都是 false——集合里明明放着 `None`，却说它不在里面。 – **直观解释（续）**：再假设 `==` 在类型不同时自动转换右操作数的类型（`"5" == 5` 变成 `"5" == str(5)`，为 true；`5 == "5"` 变成 `5 == int("5")`，也为 true）。由于 `int("007")` 忽略前导零，`7 == "007"` 为 true 而 `"007" == 7` 中右侧 `7` 转成字符串 `"7"`，`"007" == "7"` 为 **false——对称性被破坏**。后果是集合的去重行为依赖插入顺序：先插 `"007"` 再插 `7` 与先插 `7` 再插 `"007"` 得到不同的集合大小。 – **关键规则与最佳实践**： – 一旦相等不是等价关系，**所有**依赖它的数据结构（集合、映射、去重、查找）就都不可靠了，而且 bug 往往以「顺序相关」「时好时坏」的形式出现，极难调试。 – JavaScript/TypeScript 的 `==` 就是这种「自动类型转换」的反面教材：它有时做引用相等、有时做值相等，**甚至不是等价关系**，因此现代 TS/JS 程序员极力避免使用它（Java 中没有这种自动转换的 `==`，但 Java 有它自己的陷阱：自动装箱，见后文）。 – Java 的 `==` 在对象类型上始终是**引用相等**，在基本类型上始终是**值相等**，语义固定、不做隐式转换——这是好设计，但要注意「编译期类型」决定了调用哪一个版本。

不可变类型的两种相等定义：抽象函数视角 vs 观察视角 – **定义与目的**：对不可变类型，相等有两条形式化路线：**用抽象函数定义**——`a equals b` 当且仅当 `AF(a) = AF(b)`；**用观察定义**——两个对象相等当且仅当**无法通过观察区分**，即对 ADT 规格中的每一个操作，两者都产生相同结果。它服务于「安全」与「易理解」，并且两者**必须一致**，否则说明设计有问题。 – **直观解释（"它是什么？"）**：观察意义上的相等，就是「用规格允许的全部操作去戳它们，看能否分辨」。集合 `{1,2}` 与 `{2,1}` 用集合的观察操作（基数 `|…|` 与成员关系 `∈`）完全无法区分：基数都是 2，`1 ∈` 都为真，`2 ∈` 都为真，`3 ∈` 都为假……因此它们相等。 – **关键规则与最佳实践**： – 「观察」指的是**调用 ADT 规格中的操作**。Java 允许客户端突破抽象边界去观察与抽象值无关的差异（`==` 能看出两个表示同一集合的对象占不同内存、`System.identityHashCode()` 基于内存地址），但**这些操作不属于 ADT 规格**，因此不参与观察相等的判定。 – 只有当观察者集合与 AF 一致时，两种定义才给出相同答案。以 `Duration`（rep 为 `mins`、`secs`，AF 为「mins 分 secs 秒的时间跨度」，唯一观察者是 `getLength()`）为例：`d1=(1,2)` 与 `d4=(1,2)` 的 rep 完全相同，AF 必然相同；`d3=(0,62)` 表示的时间跨度与 `d1` 相同，按 AF 定义也相等（这正是 rep 允许 `secs > 60` 造成的非规范化表示）；`d2=(1,3)` 不等。换成观察视角，`getLength()` 对 `d1`、`d3`、`d4` 都返回 62，无法区分，结论一致。 – 反例是「观察者泄露 rep 细节」：考虑 `LetterSet`，其 AF 为「字符串中出现过的字母集合（忽略非字母与大小写）」。观察者 `contains()`、`size()` 与 AF 一致；但 `length()`（构造时字符串的长度）、`first()`（字符串的第一个字母）、`isAllLowercase()` 观察的是 **rep 的细节**而非抽象值——`new LetterSet("abc")` 与 `new LetterSet("1a2b3c")` 抽象值相同（都是 `{a,b,c}`），但 `length()` 不同。若把它们纳入观察者集合，观察相等就会与 AF 相等冲突，说明这些操作**根本不该出现在这个 ADT 的规格里**（它们属于 Reading 11 所说的「泄露表示」的观察者）。 – 同一个道理适用于 `MyLine`：如果该类只暴露 `slope()`，那么所有斜率相同的直线都无法被区分，观察相等的粒度就退化到「斜率相等」；此时若 AF 把 rep 解释为一条**具体的直线**，就会出现「AF 不同却无法观测区分」的不一致。要让两种定义一致，要么让 AF 与观察粒度匹配，要么把能区分直线的观察者补进规格。 – **小结**：观察相等与 AF 相等的关系是**设计约束**。它给出的实践指引是：先明确 AF，再检查每一个 observer/producer 是否与 AF 一致；不一致的操作要么删掉，要么修改 AF（并相应调整规格）。

引用相等 vs 对象相等（Reference Equality vs. Object Equality） – **定义与目的**：大多数语言提供两种相等操作。**引用相等**测试两个引用是否指向内存中的同一处存储（在快照图上就是两个箭头指向同一个对象气泡）；**对象相等（值相等）**测试两个对象是否表示同一个值。区分的意义在于：客户端常常希望「内容一样就算相等」（例如两个内容相同的 `Duration`），而语言层面的引用比对太强。 – **直观解释（"它是什么？"）**：各语言的对照：Python 用 `is` / `==`；Java 用 `==` / `equals()`；Objective-C 用 `==` / `isEqual:`；C# 用 `==` / `Equals()`；TypeScript/JavaScript 用 `===` / 无内置值相等（需要自己定义 `equalValue()`）。注意 `==` 的含义在 Python 与 Java 之间**恰好相反**，不要混淆：Java 的 `==` 只比较引用。 – **关键规则与最佳实践**： – 我们无法改变引用相等操作的含义（Java 中 `==` 永远是引用相等），但**新定义的数据类型有责任决定对象相等的含义**，并正确实现 `equals()`。 – 上表只适用于**对象类型**；基本类型遵循不同规则：Java 的 `int` 不能用 `equals()`，`==` 做值相等，也没有引用相等的概念。 – Java 中 `equals()` 由 `Object` 定义，默认实现就是 `return this == that;`，即默认含义就是引用相等。对不可变类型来说这**几乎总是错的**，因此必须覆盖。 – `Object` 的默认 `hashCode()` 与默认 `equals()` 是**一致的**（都基于地址），所以「两个默认实现」本身不违反契约；问题出在「覆盖了 `equals()` 却忘了覆盖 `hashCode()`」。

Object 契约（The Object Contract） – **定义与目的**：`Object` 的规格如此重要，以至于被称为 **Object 契约**。它规定了覆盖 `equals()` 时必须满足的四个条件：`equals` 必须定义**等价关系**（自反、对称、传递）；`equals` 必须**一致（consistent）**——只要对象没有被以影响比较的方式改变，重复调用必须得到相同结果；对非 null 引用 `x`，`x.equals(null)` 必须返回 false；**`hashCode` 对 `equals` 判定相等的一对对象必须返回相同结果**。 – **直观解释（"它是什么？"）**：把它理解为 ADT 作者与整个 Java 生态（集合库、测试框架、缓存、去重工具）之间的合同：你按合同实现，生态就能可靠地使用你的类型；你违约，生态就会在你完全想不到的地方出问题。 – **关键规则与最佳实践**： – 契约允许 `null` 作为参数，并在后置条件中明确规定结果：`x.equals(null)` 应为 false。若违反（例如让 `new Duration(0,0).equals(null)` 返回 true），会立刻破坏对称性——因为 `null.equals(...)` 根本无法被调用，也就无法「回敬」true。 – 「一致性」意味着相等的结果不能随时间变化（前提是对象没有被改动）。这正是可变类型采用观察相等会出问题的根源。 – 用 `@Override` 注解强制编译器检查你确实在覆盖而不是重载（见下文场景 1）。 – 覆盖 `equals` 时必须同时覆盖 `hashCode`（见场景 2）。 – 常见错误是把 `equals` 实现成「字符串比较」：`this.toString().equals(that.toString())` 既依赖 `toString` 的实现（默认实现无意义），又常常破坏对称性（`new Duration(1,30).equals("1:30")` 可能为 true，而 `"1:30".equals(new Duration(1,30))` 为 false）。

正确实现 `equals()`：`@Override`、`instanceof`、私有 `sameValue` – **定义与目的**：给出一个可以照抄的正确模板：覆盖 `equals(Object)`、用 `instanceof` 做类型检查、把真正的字段比较放进私有辅助方法 `sameValue`。它服务于「安全」——避免重载陷阱与类型转换错误。 – **直观解释（"它是什么？"）**：`java @Override public boolean equals(Object that) { return that instanceof Duration && this.sameValue((Duration)that); } private boolean sameValue(Duration that) { return this.getLength() == that.getLength(); }` 第一个方法覆盖并替换了从 `Object` 继承的 `equals(Object)`；它先检查 `that` 确实是 `Duration`（`(Duration)that` 是类型转换表达式，向编译器断言你的信心），再调用私有辅助方法做值比较。 – **关键规则与最佳实践**： – 签名必须与

`Object.equals(Object)` **完全一致**，并加上 `@Override`：签名写错时（例如参数类型写成 `Duration`）编译器会立刻报错，而不是悄悄做出一个重载版本。 – `instanceof` 是**动态类型检查**，在面向对象编程中通常是「坏味道」；好设计中 `instanceof` **只允许出现在 `equals` 的实现里**，`getClass()` 等其他运行时类型检查手段同样被禁止。 – `that instanceof Duration` 在 `that == null` 时返回 false，因此这一行**顺带处理了 `equals(null)` 的契约要求**（这也是回答「哪一行让 null 返回 false」的答案）。 – 用 `instanceof` 还是 `getClass()` 有取舍：`instanceof` 允许子类实例与父类实例相等（更符合「同一个抽象值」的直觉），`getClass()` 要求运行时类型完全相同（更严格、能保证对称性在继承体系下不被破坏）。6.031 采用 `instanceof` 的模板；若你的类会被继承并改变相等语义，需要重新审视这个选择。 – 私有 `sameValue(Duration)` 承担真正的字段比较：它接受具体类型、不必关心 null 与类型问题，也方便与既有比较逻辑（如 `getLength()`）复用。

可变类型的两种相等：观察相等 vs 行为相等 – **定义与目的**：对可变类型，相等仍然必须是等价关系，也仍然要尊重 AF 与操作；但多了一种新可能——**在观察之前调用 mutator**，可以改变对象状态，从而制造出差异。因此把「基于观察的相等」细分为两种：**观察相等（observational equality）**指两个引用**此刻**无法被区分，客户端只能调用不改变状态的观察者（observers/producers，不含 mutators）来比较，即「它们当前看起来一样吗」；**行为相等（behavioral equality）**指两个引用**现在与将来**都无法被区分，即使对其中一个调用 mutator 而对另一个不调用，即「它们在任何状态下都会表现一样吗」。 – **直观解释（"它是什么？"）**：`arrayA = {1,2,3}`、`arrayB = {1,2,3}`、`arrayC = arrayB`。用只读操作（`length`、`get(0)`…）去比较，三者当前看起来一样；但一旦执行 `arrayA.set(0, 0)` 或 `arrayB.clear()`，`arrayA` 与 `arrayB` 就会分道扬镳——因此它们只在「此刻」相等。而 `arrayB` 与 `arrayC` 指向同一个对象，任何 mutator 都会同时影响两者，因此它们在**一切未来状态**下都相等。 – **关键规则与最佳实践**： – 对**不可变类型**，观察相等与行为相等**完全相同**（没有 mutator 能改变状态），因此只需要一个相等操作。 – 对**可变类型**，两种相等都有用：Java 用 `==` 提供行为相等；观察相等则通过一个**单独的操作**提供（sp22 命名约定是 `equalValue()`；Java 中若需要，建议命名为 `similar()` 或 `sameValue()`，作为公开操作）。 – 实现方案：**不要**用 `equals()` 承载可变类型的观察相等。Java 库对可变类型混用了两种语义（`List`、`Set`、`Map` 采用观察相等，`StringBuilder` 与数组采用行为相等），这是一个历史遗留的不一致，不应模仿。 – 关键风险：观察相等**随时间变化**，因此把这种对象放进依赖哈希的结构（`HashSet`、`HashMap`）会破坏其表示不变量（见场景 3）。 – Bag（多重集）例子可以清晰区分两者：`b1 = {a,b}`、`b2 = {a,b}`、`b3 = b1.remove("b")` 后为 `{a}`、`b4 = {b,a}`。行为相等下只有 `b1 == b4` 不成立（它们是不同对象）、而每个对象只与自身相等；观察相等下 `b1`、`b2`、`b4` 三者两两相等（`count` 结果相同），`b3` 与它们都不等。

`hashCode()` 契约与正确实现 – **定义与目的**：`Set` 与 `Map` 的实现（`HashSet`、`HashMap`）基于**哈希表（hash table）**，要求元素类型或键类型提供**哈希函数**：把对象值映射成一个整数。契约要求：**`equals` 判定相等的两个对象必须具有相同的 `hashCode`**。它服务于「安全」与性能。 – **直观解释（"它是什么？"）**：哈希表内部是一个数组。插入键值对时，先计算键的哈希码，再把它映射到数组下标（例如取模），把值放进那个槽位；当两个键落到同一槽位（冲突）时，槽位里其实是一个**键值对列表**，称为**哈希桶（bucket）**。查找时先算哈希码定位槽位，再沿桶逐个比较，直到找到**`equals` 相等**的那个键。哈希表的表示不变量中包含一条根本约束：**键必须能从它的哈希码所决定的槽位出发被找到**。因此如果两个 `equals` 相等的对象有不同的

`hashCode`，它们可能被放进不同槽位——用与插入时相等的键去查找就会失败。 – **直观解释（续）**：`Object` 的默认 `hashCode()` 返回基于内存地址的整数，与默认 `equals()`（引用相等）严格一致。但一旦你覆盖 `equals()` 让「内容相同」的两个对象相等，默认 `hashCode()` 就会违约：`d1.equals(d2)` 为 true，而 `d1.hashCode()` 与 `d2.hashCode()` 不同。 – **关键规则与最佳实践**： – **覆盖 `equals` 就必须覆盖 `hashCode`**（反之不强制，但强烈建议）。 – 必须加 `@Override`：课程历史上曾有学生把 `hashCode` 拼成 `hashcode`，于是新方法根本不覆盖 `Object.hashCode`，出现了极难定位的怪异行为。 – 标准做法：对**参与相等判定的每个字段**分别取其哈希码，再用算术运算组合起来。`Duration` 的抽象值本身就是一个整数，因此 `return (int) getLength();` 即可。多字段场景可用 `Objects.hash(...)`（Java 补充说明），或手写 `31 * result + field` 的经典组合方式（Josh Bloch《Effective Java》有详细讨论）。 – 一种「简单粗暴」的合规做法是让 `hashCode` 永远返回常数（例如 42）：它满足契约，但所有键都挤在同一个槽位，查找退化为线性扫描，性能灾难性下降。 – 只要满足契约，具体哈希技巧**不影响正确性**，只影响性能：糟糕的哈希函数会产生不必要的冲突，但总比破坏契约好。 – **绝不要用可变字段计算哈希码**：对象的哈希码在生命周期内必须保持不变，否则它进入哈希表之后就无法再被找到（见场景 3）。 – 在像 Java 这样所有对象都支持相等比较的语言里：**对不可变类型，永远实现与 `equals` 一致的 `hashCode`**。补充说明：sp22 指出 TypeScript 的哈希函数是内置且不可覆盖的，因此 TypeScript 中不可变对象类型**不能**安全地用作 `Set` 元素或 `Map` 键；Java 没有这个限制，这反而凸显了「正确实现 `equals` / `hashCode`」的重要性。

用哈希表的不变量解释「可变字段不能进哈希」 – **定义与目的**：把哈希表原理与可变类型结合起来，得到本讲最重要的实践结论。 – **直观解释（"它是什么？"）**：把一个 `ArrayList` 放进 `HashSet`：插入时它按当时的 `hashCode()` 落到某个桶里。随后修改这个列表（例如 `list.add("goodbye")`），它的 `hashCode()` 变了，但 `HashSet` **不知道**需要把它搬到别的桶。于是再也找不到它：`set.contains(list)` 变为 **false**，而遍历集合时 `for (List<String> l : set) { set.contains(l); }` 又会发现**集合自己的迭代器和自己的 `contains()` 互相矛盾**——迭代器说元素在集合里，`contains` 说不在。集合明显已经损坏。`java.util.Set` 的规格里有一段名言：「如果把可变对象用作集合元素，必须极其小心。如果一个对象在集合中的期间被以影响 `equals` 比较的方式修改，集合的行为是未指定的。」 – **关键规则与最佳实践**： – 当 `equals()` / `hashCode()` 会被修改影响时，把该对象用作哈希表键就会破坏哈希表的表示不变量。 – 实践准则（本讲的最终规则）：需要放进 `HashSet` / `HashMap` 的类型，要么**不可变**（覆盖 `equals` / `hashCode`，基于抽象值），要么**采用引用相等**（不覆盖，继承 `Object` 的实现）。 – 如果一个可变类型确实需要「当前看起来一样」的判断，就把它做成**独立的公开操作**（`similar()` / `sameValue()`），不要污染 `equals()`。 – 另一种安全做法：放入哈希结构前先做**不可变快照**（`Set.copyOf`、`List.copyOf`、`Collections.unmodifiableList`），并确保快照本身不再被修改。

深相等（Deep Equality）与测试断言 – **定义与目的**：sp22 指出现代语言普遍缺少对 `Array` / `Set` / `Map` 的标准「观察相等」操作，于是一些库提供了**深相等**操作，可以逐层拆解嵌套集合进行比较。它服务于「易理解」与测试便利，但必须小心使用。 – **直观解释（"它是什么？"）**：TS/JS 世界里，`Assert.deepStrictEqual()`（Node）、`isEqual()`（Underscore/Lodash）能比较 `Array<Map<T,Set<U>>>` 这样的多层结构。Java 中的对应情况要更清楚一些（补充说明）：数组的 `equals` 是**引用（行为）相等**，因此比较数组内容要用 `Arrays.equals`（一维）或 `Arrays.deepEquals`（多维/嵌

套）；集合类型（`List`、`Set`、`Map`）自己实现了**观察相等**，所以 `listA.equals(listB)` 会比较元素序列，`setA.equals(setB)` 会比较元素集合；`Objects.deepEquals` 则会在数组与普通对象之间分派到合适的实现。 – **关键规则与最佳实践**： – 深相等操作对**内置集合**有特殊处理（例如忽略 `Map`/`Set` 的元素顺序），因此能正确比较这些集合的抽象值。 – 但对**用户自定义类型**，深相等操作只会**盲目地逐字段比较 rep**，完全不理解 AF。若 rep 是非规范化的（例如 `Duration(0,60)` 与 `Duration(1,0)` 表示同一个抽象值），深相等会得出错误结论。 – 因此：当集合的叶子类型是基本类型（`number`/`string`/`boolean`，Java 中即 `int`/`String`/`boolean` 等）时深相等是安全的；当叶子类型是自定义对象类型（如 `Duration`、`Bag`）时，深相等的表现可能出人意料，应改用类型自己的 `equals`（或预先定义好的比较操作）。 – 测试中要区分「引用相等断言」与「值相等断言」：`assert.strictEqual([1], [1])` 会失败（不同数组对象），`assert.deepStrictEqual([1], [1])` 会成功；Java 中对应的是 `assertSame` 与 `assertEquals`（`assertEquals` 会调用 `equals`）。

Java 陷阱：自动装箱与相等的交互 – **定义与目的**：Java 的基本类型与其包装类型（`int` 与 `Integer`）之间会自动转换（autoboxing / autounboxing），而 `==` 对引用类型是引用相等、对基本类型是值相等——于是同一次比较的语义取决于**编译期类型**。 – **直观解释（"它是什么？"）**：`Integer x = new Integer(3); Integer y = new Integer(3);` 时，`x.equals(y)` 为 **true**（`Integer` 正确实现了值相等），但 `x == y` 为 **false**（引用相等）；而 `(int)x == (int)y` 为 **true**（值相等）。更经典的例子是 `Map<String,Integer>`：`a.put(c, 130); b.put(c, 130);` 之后 `a.get(c) == b.get(c)` 的结果取决于自动装箱缓存（补充说明：Java 规范要求 `-128..127` 的装箱对象被缓存复用，超出该范围通常不会复用），而 `a.get(c).equals(b.get(c))` 永远为 true。 – **关键规则与最佳实践**： – 比较包装类型的值，**永远用 `equals()`**（或先拆箱成基本类型再用 `==`），不要直接用 `==`。 – 时刻清楚表达式的**编译期类型**：`130` 的编译期类型是 `int`；放进 `Map<String,Integer>` 后由自动装箱变成 `Integer`；`a.get(c)` 的编译期类型是 `Integer`。 – 用 `Integer.valueOf` 的心智模型（而非 `new Integer`，后者已废弃）理解装箱缓存，但不要依赖缓存的边界。 – 这条陷阱与 Java 的数组/`StringBuilder` 一样，属于「语言中相等语义不一致」的现实，写出正确代码的前提是明确每次比较用的是哪一种相等。

代码示例与对比分析

场景 1：重载（overload）而不是覆盖（override）`equals`

❌ *错误代码*

```
public class Duration {
    private final int mins;
    private final int secs;
    // Rep invariant: mins >= 0, secs >= 0
    // Abstraction function: AF(mins, secs) = the span of time of mins minutes and secs sec

    public Duration(int m, int s) { mins = m; secs = s; }

    /** @return length of this duration in seconds */
    public long getLength() { return (long)mins*60 + secs; }

    // 错误：参数类型写成了 Duration，这不是覆盖，而是重载
    public boolean equals(Duration that) {
        return this.getLength() == that.getLength();
    }
}
```

【错误代码的问题】 1. 签名与 `Object.equals(Object)` 不一致，因此这是一次**重载**：`Duration` 中同时存在新写的 `equals(Duration)` 和从 `Object` 继承来的 `equals(Object)`（后者做引用相等）。 2. Java 在**编译期**按参数的静态类型选择重载版本，于是 `d1.equals(d2)`（实参静态类型为 `Duration`）走新版本返回 true，而 `d1.equals(o2)`（`Object o2 = d2;`，静态类型为 `Object`）走继承版本返回 **false**——**同一个对象、同一个方法名，结果不同**，对称性与一致性都被破坏。 3. 集合、映射、断言等一切通过 `Object` 引用调用 `equals` 的代码（包括 `HashSet.contains`、`assertEquals`）都会走错版本，于是「内容相同的对象」在集合里查不到。 4. 这类错误极其常见，且编译器**不会**报错——除非你使用 `@Override` 注解。

✅ *正确代码*

```
public class Duration {
    private final int mins;
    private final int secs;
    // Rep invariant: mins >= 0, secs >= 0
    // Abstraction function: AF(mins, secs) = the span of time of mins minutes and secs sec

    public Duration(int m, int s) { mins = m; secs = s; }

    /** @return length of this duration in seconds */
    public long getLength() { return (long)mins*60 + secs; }

    @Override
    public boolean equals(Object that) {
        // that instanceof Duration 在 that == null 时为 false，满足 x.equals(null) == false
        return that instanceof Duration && this.sameValue((Duration)that);
    }

    // returns true iff this and that represent the same abstract value
    private boolean sameValue(Duration that) {
        return this.getLength() == that.getLength();
    }
}
```

【为什么这样更好】 `@Override` 让编译器**强制检查**确实存在同签名的父类方法：如果签名写错（写成 `equals(Duration)`），编译器立刻报错，而不是悄悄生成一个重载版本。参数类型是 `Object`，因此无论调用方用什么静态类型引用，都会走到同一个实现，从而恢复「同一个抽象值 → 同一个答案」的性质。`instanceof` 检查既保证了类型安全（随后的强制转换是合法的），又顺带满足了 `equals(null)` 返回 false 的契约；真正的值比较被放进私有方法 `sameValue(Duration)`，职责清晰、便于复用。

【代码对比解说】 这组对比的关键概念是**重载 vs 覆盖**：重载由**编译期**的静态类型决定（就像 `/` 在 `int` 与 `double` 之间选择整数除法或浮点除法），覆盖由**运行期**的动态分派决定。相等操作的本质是「抽象值之间的关系」，它必须与**运行时**对象绑定，因此绝不能依赖编译期类型。修复方式不是「改改参数类型试试」，而是「让签名与父类完全一致，并让编译器替你检查」——这也是为什么 6.031 把 `@Override` 当作强制性习惯。此外，把字段比较移入 `sameValue` 是一个重要模式：它让 `equals` 只负责「类型与非空」这类边界问题，而把「什么算同一个抽象值」集中到一个地方，将来修改相等的定义时只改一处。

【设计原则透视】 `equals(Object)` 的签名就是 **Object 契约的一部分**，属于 Reading 06/07 意义上的规格：客户端（`HashSet`、JUnit）只能通过 `Object` 引用来调用它。任何签名偏离都等于违反了规格的接口部分，即使实现逻辑本身正确。同时，`instanceof` 的使用限定体现了 Reading 12（Interfaces, Generics, Enums）中「用多态而不是运行时类型检查」的原则：`instanceof` 在面向对象设计中是坏味道，唯一被允许的例外就是实现 `equals`（`getClass()` 同样被禁止）。最后，`AF(a) = AF(b)` 这一判据直接落到了 `sameValue` 的实现里：比较的是 `getLength()`（抽象值的函数），而不是 `mins`/`secs`（rep 字段）——这保证了相等与 AF 一致（`Duration(0,60)` 与 `Duration(1,0)` 相等）。

场景 2：覆盖了 `equals` 却忘记覆盖（或写错）`hashCode`

❌ *错误代码*

```
public class Person {
    private final String firstName;
    private final String lastName;

    public Person(String first, String last) { firstName = first; lastName = last; }

    @Override
    public boolean equals(Object that) {
        return that instanceof Person && this.sameValue((Person) that);
    }

    // returns true iff this and that represent the same abstract value
    private boolean sameValue(Person that) {
        return this.lastName.toUpperCase().equals(that.lastName.toUpperCase());
    }

    // 错误一：完全没有覆盖 hashCode —— 继承了基于内存地址的实现
    // public int hashCode() { return super.hashCode(); }  // 隐含行为

    // 错误二（另一种常见写法）：用了不参与相等判定的字段
    // public int hashcode() { return firstName.hashCode() + lastName.hashCode(); }
}
```

【错误代码的问题】 1. 未覆盖 `hashCode` 时，`equals` 相等的两个 `Person` 会有**不同的哈希码**（基于地址），违反 Object 契约：把它们放进 `HashSet`，`add(p2)` 之后集合里会出现两个「相等」的元素；用 `p2` 去 `map.get()` 也取不到以 `p1` 为键存进去的值。 2. 第二种写法把方法名拼成了 `hashcode`（小写 c），这不是覆盖而是新增方法，`Object.hashCode` 依然生效——课程原文特别提到曾有学生为此花了数小时定位 bug。缺少 `@Override` 注解是根本原因。 3. 若用 `firstName.hashCode() + lastName.hashCode()` 作为哈希码，则**引入了不参与相等判定的字段**：两个姓氏相同（忽略大小写）、名字不同的 `Person` 会 `equals` 为 true，却可能有不同的哈希码——同样违反契约。 4. 任何违反契约的实现都会让 `HashSet` / `HashMap` / `equals` 相关的测试断言出现「静默错误」：不会抛异常，只是结果不对，极难调试。

✅ *正确代码*

```
import java.util.Objects;

public class Person {
    private final String firstName;
    private final String lastName;

    public Person(String first, String last) { firstName = first; lastName = last; }

    @Override
    public boolean equals(Object that) {
        return that instanceof Person && this.sameValue((Person) that);
    }

    // returns true iff this and that represent the same abstract value
    private boolean sameValue(Person that) {
        return this.lastName.equalsIgnoreCase(that.lastName);
    }

    @Override
    public int hashCode() {
        // 只使用参与相等判定的字段，并且用同一套“忽略大小写”的规范化方式
        return lastName.toUpperCase().hashCode();
    }
}
```

【为什么这样更好】 `hashCode` 只依赖**参与相等判定的字段**（`lastName`），并使用与 `sameValue` **一致的规范化规则**（忽略大小写），于是「`equals` 相等 ⇒ `hashCode` 相等」必然成立，契约得到满足：相等的对象落在同一个桶里，`HashSet` 去重与 `HashMap` 查找都能按预期工作。加上 `@Override` 后，任何拼写或签名错误都会在编译期暴露。

【代码对比解说】 这组对比的核心是「`equals` 与 `hashCode` 必须**基于同一组字段、同一套规范化规则**」。`lastName.toUpperCase().hashCode()` 与 `lastName.equalsIgnoreCase(...)` 表达的是同一个判定口径；若 `hashCode` 用原始 `lastName.hashCode()`，则 `"Smith"` 与 `"SMITH"` 会 `equals` 为 true 而哈希码不同，契约立刻被破坏。另一种「合规但糟糕」的选项是 `return 42;`——它满足契约（相等对象哈希码相同），但所有键挤进同一个桶，查找从 O(1) 退化为 O(n)。还有一种选项是 `return firstName.toUpperCase();`，它连编译都通不过（`String` 不能作为 `int` 返回）。真正需要记住的规则只有一句：**哈希码必须由抽象值决定**，而不是由 rep 的一部分或与相等无关的字段决定。

【设计原则透视】 `hashCode` 是 AF 的另一个出口：它把**抽象值**映射为一个整数（`Duration` 直接 `return (int) getLength();` 就是这个思想的最纯粹形式）。因此它天然与 Reading 11（AF/RI）绑定——如果 `hashCode` 依赖了 AF 之外的 rep 细节，就等于让哈希码携带了「抽象值无关的信息」，契约必然被破坏。同时，这也是 Reading 08（Immutability）的延伸：只有不可变类型才能保证哈希码在生命周期内稳定，因此「不可变类型覆盖 `equals`/`hashCode`」是一条可以无脑遵守的准则。Java 补充说明：多字段场景推荐 `Objects.hash(f1, f2, ...)`，它内部使用与《Effective Java》一致的做法（以 31 为基的组合），避免手写时漏字段或算错。

场景 3：把可变对象当作哈希表的键，并在其间修改它

❌ *错误代码*

```
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

public class BrokenSetDemo {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();
        list.add("a");

        Set<List<String>> set = new HashSet<>();
        set.add(list);                      // 用 list 当时的 hashCode 决定桶位置

        System.out.println(set.contains(list));   // true

        list.add("goodbye");                // 修改 list —— 它的 hashCode 变了
                                            // 但 HashSet 不知道要把它搬到别的桶

        System.out.println(set.contains(list));   // false ！元素还在集合里，却查不到

        for (List<String> l : set) {
            System.out.println(set.contains(l));  // false ！迭代器与 contains 自相矛盾
        }
    }
}
```

【错误代码的问题】 1. `List` 的 `equals` / `hashCode` 基于元素内容，因此**会被 mutation 影响**；插入时元素被放在与其当时哈希码对应的桶里，修改后哈希码改变，`HashSet` 不会重新分桶，于是**永远找不到**这个元素。 2. 破坏的严重性在于「同一个集合的两个操作互相矛盾」：迭代器认为元素在集合中，`contains()` 认为不在——集合的表示不变量显然已被破坏（这类不一致会导致去重失败、内存泄漏式的「幽灵元素」、以及难以复现的测试失败）。 3. 更糟糕的是，这种做法会产生**静默错误**而不是异常：程序继续运行，只是结果不对。 4. `java.util.Set` 的规格明确规定：若对象在集合中期间被以影响 `equals` 比较的方式修改，集合行为未指定——也就是说，写入这种代码的客户端已经站到了「未定义行为」的地面上。

✅ *正确代码*

```
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

public final class Point {                  // 不可变类型：安全地作为哈希键
    private final int x;
    private final int y;
    // Abstraction function: AF(x, y) = 平面上的点 (x, y)
    // Rep invariant: true

    public Point(int x, int y) { this.x = x; this.y = y; }

    public int getX() { return x; }
    public int getY() { return y; }

    @Override
    public boolean equals(Object that) {
        return that instanceof Point && this.sameValue((Point) that);
    }

    private boolean sameValue(Point that) {
        return this.x == that.x && this.y == that.y;
    }

    @Override
    public int hashCode() {
        return 31 * x + y;                  // 只依赖 final 字段：哈希码永不改变
    }
}

public class SafeSetDemo {
    public static void main(String[] args) {
        Set<Point> set = new HashSet<>();
        Point p = new Point(1, 2);
        set.add(p);
        System.out.println(set.contains(new Point(1, 2)));   // true：按抽象值查找

        // 若确实需要用可变集合做键，就先做不可变快照：
        List<String> list = new ArrayList<>();
        list.add("a");
        Set<List<String>> snapshotSet = new HashSet<>();
        snapshotSet.add(List.copyOf(list));    // 不可变副本：哈希码不会再变
        list.add("goodbye");
        System.out.println(snapshotSet.size());                  // 1
        System.out.println(snapshotSet.contains(List.of("a")));  // true
    }
}
```

【为什么这样更好】 `Point` 是不可变类型，其 `equals` / `hashCode` 都基于 `final` 字段，因此哈希码在对象整个生命周期内稳定，放在桶里的位置始终正确；`contains(new Point(1,2))` 能按**抽象值**找到元素，这

正是 ADT 相等应有的效果。可变集合若确实需要作为键，就先取一个**不可变快照**（`List.copyOf`）再放入，这样后续对原列表的修改不会影响哈希码，集合的不变量得以保持。

【代码对比解说】 两段代码的差别不在「写法技巧」，而在**相等语义与可变性的组合**。`Point` 采用的是不可变类型 + 覆盖 `equals` / `hashCode`（本讲的最终规则之一）；`ArrayList` 采用的是观察相等且**可变**（Java 库的历史选择），两者组合起来就会破坏哈希表。真正要记住的判断顺序是：先问「这个类型会不会被放进 `HashSet` / `HashMap`（或用在需要稳定哈希的地方）」；如果会，就要求它的相等与哈希在生命周期内不变——要么让类型不可变，要么采用引用相等（不覆盖 `equals`），要么在放入前做不可变快照。Java 补充说明：`List.copyOf` / `Set.copyOf`（Java 10+）返回不可修改的快照，`Collections.unmodifiableList` 返回的是**视图**而非副本，两者在「是否仍随原对象变化」上不同，需要注意区分。

【设计原则透视】 这是 AF/RI 在本讲最生动的一次应用：哈希表的 RI 是「每个键都能从其哈希码对应的槽位出发被找到」，而「修改会改变 `hashCode`」直接违反这条 RI。换句话说，**可变对象的观察相等会随时间变化，而哈希表要求相等关系在一段时间内稳定**，两个合法设计放在一起就产生了不合法。由此得出本讲的核心结论：可变类型的 `equals()` 应实现**行为相等**（也就是不覆盖，继承 `Object` 的引用比较），从而保证「相等」这个关系在任何未来状态下都不变；如果确实需要「此刻看起来一样」的语义，就把它做成独立的公开操作（`similar()` / `sameValue()`），让它与 `equals()` 各司其职。这同时呼应 Reading 08（不可变性是可复用的安全基石）与 Reading 11（RI 必须被每个操作维护）。

场景 4：为「容差」放宽相等，破坏传递性

❌ *错误代码*

```
public class Duration {
    private final int mins;
    private final int secs;
    // Rep invariant: mins >= 0, secs >= 0
    // Abstraction function: AF(mins, secs) = the span of time of mins minutes and secs sec

    public Duration(int m, int s) { mins = m; secs = s; }
    public long getLength() { return (long)mins*60 + secs; }

    private static final int CLOCK_SKEW = 5;   // seconds

    @Override
    public boolean equals(Object that) {
        return that instanceof Duration && this.sameValue((Duration) that);
    }

    // 错误：让“大致相等”也算相等
    private boolean sameValue(Duration that) {
        return Math.abs(this.getLength() - that.getLength()) <= CLOCK_SKEW;
    }
}
```

【错误代码的问题】 1. **破坏传递性**：`d_0_57`（57 秒）与 `d_1_00`（60 秒）相差 3 秒，判定相等；`d_1_00` 与 `d_1_03`（63 秒）相差 3 秒，判定相等；但 `d_0_57` 与 `d_1_03` 相差 6 秒，判定**不相等**。于是「a = b、b = c，但 a ≠ c」。 2. 传递性一旦失效，所有依赖相等的关系结构都会出错：`HashSet` 会出现「一个元素与集合中两个不同元素分别相等」的情形，去重结果依赖插入顺序；排序、查找、缓存命中判断都变得不可靠。 3. `equals` 还要求**一致性**：容差比较本身在对象未变化时是稳定的，因此这里侥幸没破坏一致性，但传递性的破坏已经足够致命。 4. 这种「好心放宽相等」在工程中非常常见（浮点比较、时间戳比较、字符串模糊匹配），它们看起来都更「实用」，实际上都在削弱整个生态所依赖的形式契约。

✅ *正确代码*

```
public class Duration {
    private final int mins;
    private final int secs;
    // Rep invariant: mins >= 0, secs >= 0
    // Abstraction function: AF(mins, secs) = the span of time of mins minutes and secs sec

    public Duration(int m, int s) { mins = m; secs = s; }
    public long getLength() { return (long)mins*60 + secs; }

    /** true iff this and that represent the same abstract value（精确的等价关系） */
    @Override
    public boolean equals(Object that) {
        return that instanceof Duration && this.sameValue((Duration) that);
    }

    private boolean sameValue(Duration that) {
        return this.getLength() == that.getLength();       // 精确比较：自反、对称、传递
    }

    @Override
    public int hashCode() {
        return (int) getLength();
    }

    /**
     * 如果一个“宽容比较”确实有业务价值，把它作为独立操作提供，
     * 而不是替换 equals —— 它不需要是等价关系。
     * @param that 另一个 Duration
     * @param tolerance 允许的秒数误差，要求 tolerance >= 0
     * @return true iff 两者的时间跨度之差不超过 tolerance
     */
    public boolean closeTo(Duration that, long tolerance) {
        return Math.abs(this.getLength() - that.getLength()) <= tolerance;
    }
}
```

【为什么这样更好】 `equals` 保持精确（`getLength()` 相等），因此它是一个真正的等价关系：自反（自己与自己差 0）、对称（绝对值）、传递（长度相等是等号，等号天然传递）。宽容比较被移到**独立的公开操作** `closeTo(that, tolerance)` 中——它不必是等价关系（在参数 `tolerance` 的同一取值下，差不超过

tolerance 的关系其实仍是等价关系，但当不同调用使用不同 tolerance 时就是「伪相等」，绝不能冒充 `equals`）。这样做同时满足：契约不被破坏、语义清晰（读者一眼看出哪种比较更严格）、并且 `hashCode` 可以与 `equals` 保持一致。

【代码对比解说】 这组对比说明了一个通用原则：**「相等」是形式契约，「相似」是应用需求，两者不能混为一谈**。`equals` 被 `HashSet`、`HashMap`、`List.contains`、JUnit 断言等大量代码以「等价关系」为前提使用，任何放宽都必须保证三条性质仍然成立；而「在 5 秒内算一样」这种需求天然是**有参数、有上下文**的，因此它的正确归属是独立的操作（sp22 也正是在可变类型上采用同样的策略：把观察相等命名为 `equalValue()` 而不是 `equals()` / `===`）。附带一点：实现 `equals` 时也不应把「宽容」的理由建立在浮点误差上——如果类型内部使用 `double`，`equals` 仍然应当基于确定的字段比较，浮点容差属于数值算法的范畴。

【设计原则透视】 传递性通过 `Math.abs(...) <= CLOCK_SKEW` 被破坏这件事，本质上说明**「相似」不是一个传递关系**——它只是「距离有界」，而距离链条可以任意长。等价关系要求的是「二值判定 + 三条公理」，任何度量式的概念都必须先离散化（例如「把时间量化到分钟后再比较」）才能成为等价关系。这一点与 Reading 06/07 的规格写作直接相关：`equals` 的规格里写着「must be an equivalence relation」并不是修辞，而是可以逐条验证的**义务**；实现者的工作就是把抽象值的相等（由 AF 决定）忠实地映射成一个数学上合格的关系。

与其他设计原则的关联

- **Reading 08（Immutability）**：本讲的最终规则几乎完全由不可变性决定——不可变类型**应当**覆盖 `equals` / `hashCode`（观察相等与行为相等一致，哈希码稳定）；可变类型**不应当**覆盖（只有引用相等才能保证关系不随时间变化）。哈希表的破坏案例就是「可变 + 观察相等」的组合。
- **Reading 10（Abstract Data Types）**：相等是 ADT 的一个操作，必须写进规格；客户端会根据规格期待「内容相同即相等」，因此缺失或错误的相等实现会让类型难以使用。
- **Reading 11（Abstraction Functions & Rep Invariants）**：AF 是相等定义的基础（`AF(a) = AF(b)`），`hashCode` 是 AF 到整数的另一条出口；观察相等与 AF 相等的冲突，正是「observer 泄露了 rep 细节」的信号。
- **Reading 06/07（Specifications / Designing Specs）**：`equals` 的前后置条件（等价关系、一致性、`x.equals(null)` 为 false）与 `@param` / `@return` 的书写方式完全一致；把「宽容比较」放进 `equals` 是典型的规格腐化。
- **Reading 12（Interfaces, Generics, Enums）**：`equals(Object)` 的签名与 `@Override` 是「接口/继承」机制的直接应用；`instanceof` 的禁令与「用多态代替运行时类型检查」相关，唯一例外就是实现 `equals`。
- **Reading 03（Testing）**：测试断言的行为由 `equals` 决定（Java 的 `assertEquals` 会调用 `equals`，`assertSame` 才比较引用）；相等实现错误会让测试要么虚假通过、要么虚假失败。
- **Reading 13（Debugging）**：`HashSet` 里「元素既在又不在」的矛盾是典型的**静默失败**；调试这类问题要先用切片与断言确认「相等/哈希契约是否被违反」，而在定位前不要靠猜。

- **Reading 14（Recursion）**：递归遍历集合时经常需要判断「是否已访问过」，这依赖相等的正确实现；不可变类型作为 `Set` 元素（例如 `Set<File>`、`Set<Point>`）也让递归实现保持可重入。
- **Reading 21（Concurrency）**：不可变且相等正确的类型可以安全地在多个线程之间共享作为键；而可变类型的观察相等在并发修改下会带来更严重的不一致（这也是库文档警告「可变元素 + 集合」的原因）。

关键要点

- **相等必须是等价关系**：自反、对称、传递，三者缺一不可；「约等于」式的宽松相等不能充当 `equals`，要另设操作。
- **不可变类型按抽象值定义相等**：覆盖 `equals(Object)`（加 `@Override`、用 `instanceof`、把字段比较放进 `sameValue`），同时覆盖 `hashCode` 并只用参与相等的字段。
- **可变类型按引用定义相等**：不要覆盖 `equals` / `hashCode`，让行为相等保证关系不随时间变化；确需观察相等时，提供独立的 `similar()` / `sameValue()` 操作。
- **`hashCode` 契约与哈希结构的前提**：`equals` 相等则 `hashCode` 必须相等；哈希码必须由抽象值决定、在对象生命周期内**稳定**（因此不能依赖可变字段），否则 `HashSet` / `HashMap` 依赖的「键能从其哈希码对应的桶被找到」这条 RI 会被静默破坏。
- **留意语言细节**：Java 的 `==` 在对象上是引用相等；数组与 `StringBuilder` 用行为相等而 `List` / `Set` / `Map` 用观察相等；包装类型比较永远用 `equals()`。

常见陷阱与注意事项

1. **`equals` 签名或契约写错（重载、`null`、一致性）** → 参数类型写成自己的类就变成重载，通过 `Object` 引用调用时走的是引用相等版本，`HashSet` / `assertEquals` 全部失效；违反 `x.equals(null) == false`（例如让 `new Duration(0,0).equals(null)` 返回 true）会立刻破坏对称性。必须写成 `equals(Object)` 并加 `@Override` 让编译器把关。
2. **覆盖 `equals` 而不覆盖 `hashCode`（或把方法名拼错、漏写 `@Override`）** → `equals` 相等的对象哈希码不同，`HashSet` 出现重复元素、`HashMap` 查不到键；这类 bug 不抛异常、只出错结果。
3. **`hashCode` 使用不参与相等判定或可变的字段** → 前者破坏「相等 ⇒ 同哈希」，后者让对象进表后「消失」；正确做法是只用参与相等判定的字段，并保证其为 `final` /不可变。
4. **把可变对象（如 `ArrayList`）放进 `HashSet` / `HashMap` 之后再修改它** → 元素无法再被 `contains` 找到，迭代器与 `contains` 互相矛盾；`java.util.Set` 规格明确说明此时行为未指定。应改用不可变类型或放入不可变快照。
5. **让 `equals` 做模糊/容差比较，或用 `toString()` 实现 `equals`** → 前者破坏传递性（差 5 秒 = 相等、差 10 秒 = 不相等），后者常常破坏对称性（`duration.equals("1:30")` 为 true 而 `"1:30".equals(duration)` 为 false）；宽容比较与字符串表示都不属于抽象值的相等。
6. **用 `==` 比较包装类型或数组/集合内容** → `Integer` 的比较因装箱缓存而「有时对有时错」；数组的 `equals` 是引用相等，比较内容要用 `Arrays.equals` / `Arrays.deepEquals`；集合比较用它们自己的 `equals`。

思考题（带答案）

问题 1：`Duration` 的 rep 是 `mins`、`secs`（RI 为 `mins >= 0, secs >= 0`），AF 为「mins 分 secs 秒的时间跨度」，唯一的观察者是 `getLength()`。现有 `d1 = new Duration(1, 2)`、`d2 = new Duration(1, 3)`、`d3 = new Duration(0, 62)`、`d4 = new Duration(1, 2)`。请分别用「抽象函数定义」与「观察定义」判断哪些与 `d1` 相等，并解释为什么这两种定义应当一致。

答案：按**抽象函数定义**（`a equals b` 当且仅当 `AF(a) = AF(b)`）：`d1` 与 `d4` 的 rep 完全相同，AF 必然相同；`d3 = (0, 62)` 表示的时间跨度与 `d1` 相同（都是 62 秒），因此按抽象值也相等——这正是 rep 允许 `secs > 60`（非规范化表示）导致的结果；`d2 = (1, 3)` 是 63 秒，显然不等。按**观察定义**（用规格中允许的观察者 `getLength()` 去区分）：`d1`、`d3`、`d4` 的长度都是 62，无法被区分；`d2` 是 63，可以区分。两种定义给出**相同**的结论——这是设计正确的一个信号。它们之所以必须一致，是因为「抽象值」的意义正是「客户端通过规格操作所能感知到的一切」：如果两个对象 AF 相同却能被某个规格内的观察者区分，说明该观察者泄露了 rep 细节（Reading 11 的典型错误）；反之，如果两个对象 AF 不同却完全无法区分，说明 AF 过于细化、或者规格缺少必要的观察者。反例可见 `LetterSet`：`length()`、`first()`、`isAllLowercase()` 观察的是构造时字符串的细节而非字母集合，把它们纳入观察者集合就会与 AF 相等冲突——正确做法是把这些操作从 ADT 中去掉。

问题 2：下面的 `equals` 与 `hashCode` 有哪些问题？请逐条指出并给出修正后的代码。

```
public class Person {
    private String firstName;
    private String lastName;
    public Person(String f, String l) { firstName = f; lastName = l; }
    @Override public boolean equals(Object that) {
        return that instanceof Person && this.sameValue(that);
    }
    private boolean sameValue(Person that) {
        return this.lastName.toUpperCase().equals(that.lastName.toUpperCase());
    }
    @Override public int hashCode() {
        return firstName.hashCode() + lastName.hashCode();
    }
    public void setLastName(String l) { lastName = l; }
}
```

答案：问题一：`hashCode` 用了 `firstName`，而 `sameValue` **忽略** `firstName`，于是两个 `Person("Alice","Smith")` 与 `Person("Bob","Smith")` 会 `equals` 为 true 却可能有不同的哈希码，违反 Object 契约（`equals` 相等 ⇒ `hashCode` 必须相等）。问题二：`hashCode` 用的是**原始大小写**的 `lastName`，而 `sameValue` 忽略大小写，于是 `Person("A","smith")` 与 `Person("B","SMITH")` 相等但哈希码不同——必须使用与 `equals` **相同的规范化规则**。问题三：字段 `firstName` / `lastName` 不是 `final`，并且存在 mutator `setLastName`，因此这个类型是**可变的**：一旦对象被放进 `HashSet` / `HashMap` 后再调用 `setLastName`，它的哈希码（以及相等性）都会改变，元素将无法再被找到。修正方向有两个：要么把类型变成不可变（字段 `final`、去掉 `setLastName`），并让 `equals` / `hashCode` 基于同一组字段与同一套规范化规则：

```
public final class Person {
    private final String firstName;
    private final String lastName;
    public Person(String first, String last) { firstName = first; lastName = last; }
    @Override public boolean equals(Object that) {
        return that instanceof Person && this.sameValue((Person) that);
    }
    private boolean sameValue(Person that) {
        return this.lastName.equalsIgnoreCase(that.lastName);
    }
    @Override public int hashCode() {
        return lastName.toUpperCase().hashCode();
    }
}
```

要么保持可变，但**不覆盖** `equals` / `hashCode`（采用行为相等 = 引用相等），另外提供一个公开的观察相等操作（如 `similar(Person that)`）供需要「当前看起来一样」的客户端使用。后一种选择是本讲对可变类型的推荐做法。

问题 3：为什么「把 `ArrayList` 放进 `HashSet` 之后再修改它」会让 `set.contains(list)` 从 true 变成 false，甚至让集合的迭代器与 `contains()` 互相矛盾？请结合哈希表的表示不变量解释，并说明两条正确的做法。

答案：`HashSet` 内部是哈希表：插入时先算键的 `hashCode()`，据此决定数组槽位（桶），把键放进那个桶；查找时同样先算哈希码定位桶，再沿桶用 `equals` 逐个比较。`ArrayList` 的 `equals` / `hashCode` 基于**元素内容**（这是 Java 库对可变集合采用的观察相等），因此修改列表（`list.add("goodbye")`）会改变它的 `hashCode`。但 `HashSet` 只在插入时用当时的哈希码决定桶位置，它**不会**在元素自身变化后重新分桶。于是对象仍留在旧桶里，而 `contains` 会去**新哈希码对应的桶**里找，什么也找不到，返回 false；同时迭代器是沿着整个数组（所有桶）扫描的，所以它仍然能看到这个「幽灵元素」，两者结论矛盾——哈希表的表示不变量（键必须能从其哈希码决定的槽位被找到）被破坏了。`java.util.Set` 的规格对此有明确警告：若对象在集合中期间被以影响 `equals` 比较的方式修改，集合的行为未指定。两条正确做法：① 让作为键的类型**不可变**（例如本讲的 `Point`：`final` 字段 + 基于抽象值的 `equals` / `hashCode`），这样哈希码在生命周期内稳定；② 若必须使用可变结构，就在放入前取**不可变快照**（`List.copyOf(list)` / `Set.copyOf(set)`），或改用不覆盖 `equals` / `hashCode` 的引用相等语义（可变类型的推荐做法），并避免在元素进入哈希结构后再修改它。

Reading 16: 映射、过滤与归约（Map, Filter, Reduce）

> 说明：本讲 sp22 原版使用 TypeScript，本笔记按用户要求提供 Java 代码示例；类型/API 与 sp21（6.031 Java 版）原文保持一致。

概述

本讲要解决的核心问题是：**如何对「元素的序列」编写函数而完全不写显式控制流**。课程给出的答案是三个操作——`map`、`filter`、`reduce`，它们把整个序列当作一个整体来处理，于是原来代码里的 `for`、`if`、`return` 以及 `filenames`、`f`、`result` 这些临时变量统统消失，程序员只需要专注于计算本身的含义。支撑这一模式的关键语言特性是**一等函数（first-class functions）**：函数可以像整数、字符串一样被存入变量、作为参数传递、动态创建；在此基础上才能写出**高阶函数（higher-order function）**来把控制流抽象掉。

这一讲与三大目标的关系非常直接。**Safe from bugs**：`map` / `filter` / `reduce` 只使用**纯函数（pure function）**与**不可变数据类型**，天然避免了循环下标越界、临时变量被意外修改、累加器初始化写错这类 bug，而且纯函数 + 不可变数据的组合可以自动并行化而不改变结果。**Easy to understand**：一行 `cameras.filter(c -> c.brand().equals("Nikon")).map(Camera::pixels).reduce(Math::max)` 把「筛选—投影—聚合」的意图直接写在代码表面，比三十行循环加分支更容易读懂。**Ready for change**：把控制流交给库实现之后，改变遍历方式（顺序流改并行流）、改变数据结构（`List` 换 `Set`）、改变元素类型（`Integer` 换 `Double`）都只需要改链条中的一个环节，而不用重写循环体。

核心概念与设计原则详解

一等函数（First-class Functions）

- **定义与目的**：若一种语言中的函数可以像其他值一样被传递、返回、存储和使用，就称函数在该语言中是**一等（first-class）**的。它解决的是「把行为本身参数化」的问题，让复用从数据层上升到行为层，直接服务于可修改性与易理解性。
- **直观解释（"它是什么？"）**：把函数想成一张写着操作步骤的卡片。传统语言里你只能照着卡片做事，却没法把卡片本身递给别人；一等函数允许你把卡片放进抽屉（变量）、交给同事（参数）、甚至现场写一张新卡片（动态创建）。很多语言构造并**不是**一等的：`public` / `private` 这类访问控制不能作为参数传递，`while` 循环和 `if` 语句也不能被单独引用或操纵——它们只是语法，不是值。
- **关键规则与最佳实践**：
- 在 Java 中函数本身不是严格意义上的一等值，但**函数式对象（functional object）**达到了同样的效果：`Function<T,R>` 表示一元函数，其核心操作是 `apply`；`Math::sqrt` 就是这样一个对象。
- 需要「返回一个函数」时，让方法返回 `Function` / `Predicate` / `Comparator` 等函数式接口，而不是返回某个已经算好的结果。
- 一旦某段逻辑需要按场景替换（比较规则、过滤条件、映射规则），就应把它抽成参数而不是写死在方法体里。
- 记住函数式接口的签名必须与使用点匹配：`filter` 需要 `Predicate<T>`（返回 `boolean`），`map` 需要 `Function<T,R>`，`reduce` 需要 `BinaryOperator<T>` 或累加器/组合器。

函数式编程与纯函数（Functional Programming & Pure Functions）

- **定义与目的**：**函数式编程（functional programming）**指用不可变数据和实现纯函数的操作来建模问题、实现系统，与「可变数据 + 有副作用（side effect）的操作」相对。它通过消除状态变化来提升安全

性与可并行性。

- **直观解释（"它是什么？"）**：纯函数就像自动售货机：投币、按键、掉出商品，除此之外世界没有任何变化，同样的输入永远得到同样的输出。有副作用的操作则像在公共白板上写字：谁先写、写了几次都会互相干扰。
- **关键规则与最佳实践**：
- 传给 `map` / `forEach` 的函数必须是**无状态（stateless）**的：其行为不能依赖在 map/forEach 执行过程中会变化的状态，因为库不保证元素上的函数调用顺序，并且可能在并行流中分不同线程执行。
- 不要在 `map` / `forEach` 里修改共享的可变对象；需要累加时用 `reduce`，需要收集时用 `collect`。
- 需要「对每个元素做同样的动作但不收集结果」时才用 `forEach`（如 `sockets.forEach(Socket::close)`）；`forEach` 的返回值被丢弃，因此它天然是一种有副作用的操作。
- 不可变 + 纯函数 = 立刻可并行：把顺序流换成 `parallel()` 或 `parallelStream()`，结果不变。

高阶函数（Higher-order Function）

- **定义与目的**：接受函数作为参数，或把函数作为返回值返回的函数称为高阶函数。它是「对函数这种数据类型做运算」的手段，是抽象掉控制流的杠杆。
- **直观解释（"它是什么？"）**：普通函数处理数字和字符串，高阶函数处理「动作」。工厂是典型类比：`endsWith(".java")` 不是一次判断，而是一台「以后缀为配方」的过滤器制造机，你给它一个后缀，它交给你一个可以反复使用的判断函数。
- **关键规则与最佳实践**：
- 高阶函数的签名要读得出来：sp21 中 `static Predicate<File> endsWith(String suffix)` 的签名是 `String → (File → boolean)`。
- 优先用高阶函数把「变化的部分」参数化，把「不变的部分」留在库里：这正是 `map` / `filter` / `reduce` 存在的意义。
- 高阶函数之间可以组合成链（chaining），链条每一步的输入输出类型必须首尾相接。
- 编写高阶函数时不要在内部提前调用参数函数——把函数原样传出去，否则退化成普通调用。

Lambda 表达式与方法引用（Lambda Expressions & Method References）

- **定义与目的**：Lambda 表达式是「匿名函数」的字面写法，方法引用是「直接指名一个已有方法」的简写。二者都用于在需要函数式对象的位置上提供实现，让代码聚焦于「做什么」。
- **直观解释（"它是什么？"）**：`x -> Math.sqrt(x)` 是现场写一张新卡片；`Math::sqrt` 则是直接指着墙上已有的那张卡片说「就用它」。后者少了一层无意义的中间环节——既然 lambda 只是把参数转交给 `sqrt` 再把结果返回，那它和 `sqrt` 本身在语义上完全等价。
- **关键规则与最佳实践**：
- 方法引用的写法是 `类名::方法名`（如 `Math::sqrt`、`String::toLowerCase`、`File::toPath`），注意中间是 `::` 而不是 `.`；`Math.sqrt` 是字段访问，`Math.sqrt(25)` 是方法调用，`Math::sqrt` 才是对

函数对象的引用。

- 方法引用既支持静态方法，也支持实例方法（含未绑定接收者的形式，如 `String::toLowerCase`）。
- 当 lambda 体只是一次直接转发调用时，改写为方法引用更短、更清晰；当需要额外计算、需要多参数重排或需要捕获局部变量时，仍用 lambda。
- lambda 只能捕获**事实上不可变（effectively final）**的局部变量，这也从语言层面鼓励无状态风格。

Map（映射）

- **定义与目的**：`map` 把一个一元函数应用到序列的每个元素上，并**按原顺序**返回由结果组成的新序列，用于「对每个元素做同样的变换」。
- **直观解释（"它是什么？"）**：流水线上的一排工人，每人拿到一个零件、做同一道加工，然后按原有顺序把成品放回传送带。
- **关键规则与最佳实践**：
- 类型签名是 `map : Stream<E> × (E → F) → Stream<F>`；输入是 `Stream<Integer>`、函数是 `Integer → Double` 时，结果是 `Stream<Double>`，元素类型可以改变，但**元素个数不变**。
- sp21 的例子：`List.of(1, 4, 9, 16).stream().map(Math::sqrt)` 得到 `1.0, 2.0, 3.0, 4.0`；`List.of("A", "b", "C").stream().map(s -> s.toLowerCase())` 得到 `"a", "b", "c"`。
- 想「映射一个有副作用的操作」时不要用 `map`：因为 mutator 通常返回 `void`，Java 要求改用 `forEach`。
- `map` 不修改输入序列，它返回一个新的流/集合。

Filter（过滤）

- **定义与目的**：`filter` 用一个一元**谓词（predicate）**测试每个元素，保留满足者、丢弃不满足者，返回新的序列；它用于「按条件挑选」。
- **直观解释（"它是什么？"）**：一道安检门，每个人经过时被问一个是非题，答「是」的进，答「否」的留下。
- **关键规则与最佳实践**：
- 类型签名是 `filter : Stream<E> × (E → boolean) → Stream<E>`；元素类型不变，元素个数可能减少（甚至变成空序列）。
- sp21 的例子：`List.of('x', 'y', '2', '3', 'a').stream().filter(Character::isLetter)` 得到 `['x', 'y', 'a']`；`List.of(1, 2, 3, 4).stream().filter(x -> x % 2 == 1)` 得到 `[1, 3]`。
- `filter` 的参数是 `Predicate<T>`（`T → boolean`），因此 `Character::isLetter`、`s -> !s.isEmpty()` 都可以直接使用。
- `filter` 同样不修改输入；空结果不是错误，而是合法的抽象值。

Reduce（归约）

- **定义与目的**：`reduce` 用二元函数把序列的元素合并成一个结果，用于「聚合」。它是三者中设计空间最大的一个，有三个关键设计选择，直接关系到正确性。
- **直观解释（"它是什么？"）**：把一叠账单一张张并入一个累计总额，最后手里只剩一个数字。累计的起点叫**初始值（identity / init）**，合并的规则叫**累加器（accumulator）**。
- **关键规则与最佳实践**：
- **设计选择一——是否要求初始值**：Java 允许省略，此时以第一个元素作为初始值；但空序列没有值可返回，所以省略初始值的 `reduce` 返回 `Optional<E>`（sp21 原文：`List.of(5, 8, 3, 1).stream().reduce(Math::max)` 返回含 `8` 的 `Optional<Integer>`）。sp22 的 TypeScript 版本则在空数组时抛 `TypeError`——这也是 `max` / `min` 这类**没有天然幺元（identity element）**的归约必须小心处理的原因。
- **设计选择二——结合方向**：Java 要求归约运算符**满足结合律（associative）**，如 `+` 和 `max`。满足结合律时组合顺序无关紧要，实现因此可以把 `((0+1)+2)+3` 计算成 `(0+1)+(2+3)` 等任意等价形式，并自动并行化。Python 的 `fold-left` 从左侧开始，对应的 `fold-right` 从右侧开始；对非结合运算符（如减法）两个方向结果不同：`fold-left([1,2,3], 0, –) = ((0–1)–2)–3 = –6`，而 `fold-right` 为 `2`。
- **设计选择三——归约到另一种类型**：结果类型 `F` 不必等于元素类型 `E`。Java 最一般的形式是 `reduce : Stream<E> × F × (F × E → F) × (F × F → F) → F`，即还要提供一个**组合器（combiner）** `⊗ : F × F → F` 来合并两个部分结果。累加器与组合器都必须满足结合律，且彼此一致：`(("" ⊙ 1) ⊙ 2) ⊙ 3`、`("" ⊙ 1) ⊗ (("" ⊙ 2) ⊙ 3)` 与 `("" ⊙ 1) ⊗ (("" ⊙ 2) ⊗ ("" ⊙ 3))` 都得 `"123"`。
- 初始值必须是该运算的**幺元**：求和的初始值是 `0`，求积的初始值是 `1`，拼接字符串的初始值是 `""`；把求积的初始值写成 `0` 会让结果恒为 `0`。
- 对没有天然幺元的运算（`min` / `max`），要么用 `Optional`（`reduce(Math::min)` 配合 `orElse` / `get`），要么选一个「极端值」当初始值（`Integer.MAX_VALUE` 配 `Math::min`），要么显式处理空序列——三者的语义差别必须在规格里说清。

Stream：惰性求值与一次性消费（Streams, Laziness, Single Consumption）

- **定义与目的**：`Stream<E>` 是 Java 中表示元素序列的抽象数据类型，来自「抽象掉控制流」这一设计目标：`List` / `Set` 等集合提供 `stream()`，`Arrays.stream` 由数组建流，`Stream` 自身还提供 `of`、`concat`、`IntStream.range(...).boxed()` 等工厂。
- **直观解释（"它是什么？"）**：把流想成一条「只能走一次的传送带」，而不是装满零件的箱子。链条上的 `map` / `filter` 只是挂上加工工位，真正推动零件的动作发生在终端操作（`reduce`、`collect`）被调用时。
- **关键规则与最佳实践**：
- **流只能被消费一次，不可重用**。用 `lines` 流造出 `words` 流后，就不能再用同一个 `lines` 流去找含注释的行；必须重新调用构建方法（如再次调用 `allFilesIn()`）获得新流。在这一点上 `Stream` 与 `Iterator`、`InputStream`、`OutputStream` 同类。
- **方法调用链（method call chaining）**是流的惯用写法：`List.of(1,4,9,16).stream().map(Math::sqrt)`，每一步的返回值直接用于调用下一步。

- 传给 `map` / `filter` / `reduce` 的函数**不能抛出受检异常（checked exception）**；需要调用 `Files.readAllLines` 这类会抛 `IOException` 的方法时，必须用 lambda 把受检异常包装成非受检异常（如 `UncheckedIOException`）。
- 想并行时用 `parallel()` 或 `parallelStream()`；这正是「纯函数 + 不可变数据」带来的红利。

何时用函数式、何时用命令式（Functional vs Imperative）

- **定义与目的**：`map` / `filter` / `reduce` 让代码更短更简单，使程序员专注于计算的核心而非循环、分支与控制流的细节；但并非所有代码都适合函数式改写，判断标准是「抽象后的代码是否更清楚地表达了意图」。
- **直观解释（"它是什么？"）**：这就像用电动工具还是手动工具：批量、同构、可组合的加工用电动工具（`map` / `filter` / `reduce`）又快又整齐；需要频繁试探、提前退出、复杂状态机的加工，手动工具（显式循环）反而更直白。
- **关键规则与最佳实践**：
- 只要循环体是「对每个元素做同一件事」或「按条件挑选」或「合并成一个值」，就优先考虑 `map` / `filter` / `reduce`。
- 数据库查询的经典类比：SQL 的 `select max(pixels) from cameras where brand = "Nikon"` 中，`cameras` 是序列，`where` 是 `filter`，`pixels` 是 `map`，`max` 是 `reduce`；关系数据库把这套范式称为 project/select/aggregate。
- TypeScript/Python 中惯用列表推导式（list comprehension）的地方，Java 中就用 `filter` + `map` 的组合；同样，能避免下标计数器就避免。
- 若改写后需要嵌套三层 lambda、需要临时状态、或可读性明显下降，就保留命令式写法并把它封装在一个有规格的方法里——**可读性优先于时髦**。

代码示例与对比分析

场景 1：求列表中奇数的乘积——`reduce` 的初始值必须是幺元

❌ *错误代码*

```
import java.util.List;

public class Products {
    /**
     * @param list list of integers
     * @return product of the odd integers in list
     */
    public static int productOfOdds(List<Integer> list) {
        // 错误：求积却把初始值写成 0，任何数乘以 0 都是 0
        return list.stream()
                   .filter(x -> x % 2 == 1)
                   .reduce(0, (x, y) -> x * y);
    }
}
```

【错误代码的问题】 1. 只要 `list` 非空，结果恒为 `0`（课程练习明确问过 `List.of(1,2,3).stream().reduce(0, (a,b) -> a*b)` 的结果就是 `0`），函数对所有正常输入都返回错误答案。 2. 这是一个**动态错误**：类型检查完全通过，编译器无法发现，只有运行测试时才暴露；而且空列表返回 `0` 「看起来恰好对」，更容易骗过随意写的测试。 3. 初始值的语义被误解成「累加的计数起点」而非「运算符的幺元」，一旦有人照抄这段代码去写求和、求最大值，错误会继续扩散。

✅ *正确代码*

```
import java.util.List;

public class Products {
    /**
     * @param list list of integers
     * @return product of the odd integers in list
     */
    public static int productOfOdds(List<Integer> list) {
        // 正确：乘法的幺元是 1，空列表返回 1（空乘积）
        return list.stream()
                   .filter(x -> x % 2 == 1)
                   .reduce(1, (x, y) -> x * y);
    }
}
```

【为什么这样更好】 初始值取乘法的幺元 `1`，保证「把初始值并入任何部分结果都不改变结果」这一性质成立，于是无论流被顺序处理还是并行地分成若干段再合并，答案都一致；空序列也自然得到数学上正确的空乘积 `1`，而不需要额外的特判分支。这正是 Java 要求归约运算符满足结合律、并要求初始值与其一致的直接体现。

【代码对比解说】 两种写法的**控制流完全相同**（都是 filter 后 reduce），差别只在一个字面量，但后果是「全错」与「全对」。这揭示了一个重要事实：函数式代码把 bug 压缩到了更少的自由度上——你不再有循环边界、局部变量、`return` 位置可以出错，但剩下的每一个参数（初始值、谓词、累加器）都必须与算子的代数性质严格吻合。经验法则是：写 `reduce` 前先问自己「这个二元运算的幺元是什么，它在空序列上代表什么含义」。

【设计原则透视】 从规格（Reading 06、Reading 07）的角度看，`reduce` 的初始值属于**调用契约的一部分**：它决定了空序列时的返回值，因此应当在方法规格中写明「@return 空列表时返回 1」。从不可变性（Reading 08）的角度看，这段代码没有修改 `list`，`filter` 和 `reduce` 都只读输入，所以它是纯函数；而纯函数是实现表示独立（Reading 11）与后续并行化（Reading 21）的前提。

场景 2：按后缀挑选文件——用高阶函数替代复制粘贴

❌ *错误代码*

```
import java.io.File;
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

public class Files1 {
    /** @return only the .java files in files */
    public static List<File> javaFiles(List<File> files) {
        List<File> result = new ArrayList<>();
        for (File f : files) {
            if (f.getName().endsWith(".java")) {
                result.add(f);
            }
        }
        return result;
    }

    /** @return only the .class files in files */
    public static List<File> classFiles(List<File> files) {
        List<File> result = new ArrayList<>();
        for (File f : files) {
            // 错误：从上一个方法复制粘贴而来，忘记把后缀从 ".java" 改成 ".class"
            if (f.getName().endsWith(".java")) {
                result.add(f);
            }
        }
        return result;
    }
}
```

【错误代码的问题】 1. 复制粘贴的第二个方法返回了错误的文件集合——这是最典型的「维护困难直接变成bug」：逻辑改动需要在三处同步修改，漏改一处就静默出错。 2. 每个新后缀都要新增一个几乎相同的方法，类的体积随需求线性膨胀，调用者还要记住众多同义方法名，Ready for change 被彻底破坏。 3. 循环里重复出现了 `for` / `if` / `result.add` 的样板代码，真正的意图（「按后缀过滤」）被淹没在控制流噪声里。

✅ *正确代码*

```
import java.io.File;
import java.util.List;
import java.util.function.Predicate;
import java.util.stream.Collectors;

public class Files1 {
    /**
     * @param suffix filename suffix to match, e.g. ".java"
     * @return a predicate that tests whether a file's name ends with suffix
     */
    public static Predicate<File> endsWith(String suffix) {
        // 高阶函数：签名是 String -> (File -> boolean)
        return f -> f.getName().endsWith(suffix);
    }

    /** @return only the .java files in files */
    public static List<File> javaFiles(List<File> files) {
        return files.stream().filter(endsWith(".java")).collect(Collectors.toList());
    }

    /** @return only the .class files in files */
    public static List<File> classFiles(List<File> files) {
        return files.stream().filter(endsWith(".class")).collect(Collectors.toList());
    }
}
```

【为什么这样更好】 「后缀」成了参数而不是硬编码常量，`endsWith` 每次调用动态生成一个新的谓词函数供 `filter` 使用，于是「按后缀过滤」这件事只有一份实现、一处可能出错。调用点 `files.filter(endsWith(".java"))` 直接读作「留下以 .java 结尾的文件」，被过滤的意图一目了然；新增后缀只需写一行新调用，不需要新增方法。

【代码对比解说】 关键区别在于抽象层次：错误版本把「过滤」这一通用模式与「.java 后缀」这一具体参数混在同一个方法体里重复了三遍；正确版本用高阶函数把通用模式交给库的 `filter`，把具体参数留在调用点。这也是本讲标题中「抽象掉控制流」的字面含义——程序员不再书写循环，而是制造并组合小函数。要注意 `Predicate<File>` 是 `File → boolean`，与 `filter` 的期望完全吻合；如果把它写成 `Function<File, Boolean>`，类型检查就会失败，这也是 Reading 01（静态检查）所强调的「让编译器替你抓错」。

【设计原则透视】 `endsWith` 是一个**函数工厂**：它的返回值是函数式对象，属于「对函数这种数据类型做运算」。从抽象边界（Reading 10、Reading 11）看，`endsWith` 的规格只承诺「返回一个判断文件名后缀的谓词」，完全不暴露内部是 lambda、方法引用还是匿名类，因此实现可以自由替换（例如改成预编译的正则，见 Reading 18），客户端代码不受影响。这与 `ImList` 用静态工厂 `empty()` 隐藏 `Empty` 构造器是同一个理念（Reading 17）：**客户端只应依赖抽象操作，不应依赖具体构造方式**。

场景 3：在流操作里累加共享可变状态——副作用与并行化冲突

❌ *错误代码*

```
import java.io.File;
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Stream;

public class WordCounter {
    private static final List<String> allWords = new ArrayList<>();

    /**
     * @param files files to read
     * @return every word found in files
     */
    public static List<String> wordsIn(Stream<File> files) {
        // 错误：在 forEach 里向共享的可变 ArrayList 追加，并依赖执行顺序
        files.parallel()
             .map(f -> f.getName().split("\\W+"))
             .forEach(words -> {
                 for (String w : words) {
                     if (w.length() > 0) {
                         allWords.add(w);   // 非线程安全 + 结果依赖调度
                     }
                 }
             });
        return allWords;
    }
}
```

【错误代码的问题】 1. `ArrayList` 不是线程安全的，而 `parallel()` 会让 `forEach` 在多个线程中执行，可能丢失元素、抛出 `ArrayIndexOutOfBoundsException` 或产生损坏的内部数组——bug 具有随机性，难以复现。 2. `allWords` 是静态字段，函数的行为依赖并改变跨越多次调用的外部状态，因此它**不是纯函数**：同样的输入在不同调用次序下得到不同结果，破坏了 Reading 06/07 中「规格—实现」必须一致的前提。 3. 把结果写进外部集合后，方法的返回值还可能与调用者的预期共享别名，调用者一次 `clear()` 就会毁掉别人拿到的列表——典型的表示暴露（rep exposure）问题。

✅ *正确代码*

```
import java.io.File;
import java.util.List;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class WordCounter {
    /**
     * @param files files to read
     * @return every word found in files
     */
    public static List<String> wordsIn(Stream<File> files) {
        // 正确：用 flatMap + filter 表达，用 collect 收集，全程无共享可变状态
        return files.parallel()
                    .flatMap(f -> Stream.of(f.getName().split("\\W+")))
                    .filter(w -> w.length() > 0)
                    .collect(Collectors.toList());
    }
}
```

【为什么这样更好】 计算被表达成一条纯函数的管道：每段的输入输出都是值，没有共享变量、没有对执行顺序的假设。`collect(Collectors.toList())` 由库负责安全地合并各线程的部分结果（其内部依赖与 `reduce` 相同的结合律要求），因此把 `parallel()` 去掉或加上都不改变答案，只是性能不同。方法返回值是新建的不可变意义上的结果列表，不存在跨调用的别名污染。

【代码对比解说】 错误版本的思想模型是「我先开一条空集合，然后逐个往里塞」；正确版本的思想模型是「这个结果是若干变换后的值的聚合」。前者把**状态**当作计算的主线，后者把**值**当作计算的主线。课程原文特别提醒：传给 `map` 和 `forEach` 的函数必须是无状态的，因为这两个方法对函数在元素上的执行顺序不作任何保证，并行流更会在不同线程里执行。凡是需要「合并」的地方，就应该交给 `reduce` / `collect` 去表达，而不是自己维护累加器。

【设计原则透视】 这段对比同时触及三条原则：其一，不可变性（Reading 08）——结果对象一旦构造就不再被修改，因此可以安全地共享和传递；其二，纯函数与副作用的分界——`forEach` 的定位是「执行动作、丢弃返回值」，一旦在它内部修改外部状态，就放弃了函数式风格的全部好处；其三，并发安全（Reading 21、Reading 23）——「纯函数 + 不可变数据」是免锁并行的充分条件，而共享可变状态则需要互斥（Reading 23）或消息传递（Reading 24）来保护，代价与复杂度都高得多。

场景 4：流操作里遇到受检异常——包装而不是吞掉

❌ *错误代码*

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class LineReader {
    /**
     * @param paths files to read
     * @return the lines of those files
     */
    public static List<String> allLines(Stream<Path> paths) {
        return paths.map(path -> {
            try {
                return Files.readAllLines(path);
            } catch (IOException ioe) {
                // 错误：吞掉异常并返回 null，把故障推迟到下游
                return null;
            }
        }).flatMap(List::stream).collect(Collectors.toList());
    }
}
```

【错误代码的问题】 1. 读取失败时 `flatMap(List::stream)` 立刻对 `null` 解引用，抛出与真实原因（文件不存在、权限不足）毫无关系的 `NullPointerException`，排错时被严重误导——这不是 fail fast，而是 fail late 且 fail 得含糊。 2. `catch` 块没有任何日志或状态传递，调用者无法区分「文件是空的」与「文件读失败了」，规格（Reading 06）中承诺的后置条件被静默违反。 3. 部分成功、部分失败时返回一个「看起来正常」的列表，错误被伪装成正常结果，属于最难发现的一类 bug。

✅ *正确代码*

```
import java.io.IOException;
import java.io.UncheckedIOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class LineReader {
    /**
     * @param paths files to read
     * @return the lines of those files
     * @throws UncheckedIOException if any file cannot be read
     */
    public static List<String> allLines(Stream<Path> paths) {
        return paths.map(path -> {
            try {
                return Files.readAllLines(path);
            } catch (IOException ioe) {
                // 正确：受检异常 -> 非受检异常，故障立即传播且保留原因
                throw new UncheckedIOException(ioe);
            }
        }).flatMap(List::stream).collect(Collectors.toList());
    }
}
```

【为什么这样更好】 传给 `map` 的函数不允许抛出受检异常，因此 `Files.readAllLines` 抛出的 `IOException` 必须先被捕获；但捕获之后正确的做法是**立刻把它转换成非受检异常抛出**，让失败以最快的速度、最完整的原因到达调用者——这正是课程原文所说的「用 lambda 把 `readAllLines` 包一层，把受检异常转成非受检异常」。`UncheckedIOException` 保留了原始 `IOException` 作为 cause，栈信息完整。

【代码对比解说】 两个版本都要写 `try` / `catch`，差别在于 catch 块里做什么：「返回哨兵值」与「转换后继续抛出」是两种截然不同的错误处理哲学。前者假设调用者会检查 `null`，可流管道里根本没有检查 `null` 的位置；后者把异常当作控制流之外的带外信号，流管道本身保持「要么全部成功，要么抛出」的干净语义。这也响应了 Reading 09（避免调试）中「bug 越早暴露越便宜」的原则。

【设计原则透视】 这里体现的是**规格的可检查性**：把 `@throws UncheckedIOException` 写进 Javadoc，等于把「读取失败会抛出异常」上升为契约，调用者可以据此设计处理逻辑；而返回 `null` 的版本没有任何可依赖的契约，因为 `null` 既可能表示失败也可能表示空文件。从抽象的角度看，方法的抽象值（Reading 11 的 AF）应当是「这些文件的所有行」，当这个抽象值无法构造时，用异常明确宣告构造失败，比返回一个非法或误导性的值更符合「表示不变量必须始终成立」的要求。

与其他设计原则的关联

本讲是**函数式风格**的集中体现，而它的技术前提在更早的章节已经铺好：Reading 02（Java 基础）介绍的接口与泛型让 `Function<T,R>`、`Predicate<T>`、`BinaryOperator<T>` 这样的函数类型可以被静态检查；Reading 12（接口、泛型与枚举）进一步说明「用一个接口表达一族行为、用若干实现或 lambda 提供具体

行为」正是接口的本义，函数式接口只是它的一个特例。没有静态类型（Reading 01）对函数签名的检查，`map` / `filter` 的误用就只能靠运行测试去发现。

本讲与 Reading 08（不可变性）互为因果：`map` / `filter` / `reduce` 之所以能自由地返回值、组合链、并行执行，是因为它们不修改输入；反过来，不可变数据结构一旦建立，用函数式操作消费它就成了最自然的选择。课程原文的总结句正是这个意思：本讲讨论的是**用不可变数据与纯函数来建模问题、实现系统**，而不是用可变数据与有副作用的操作。

本讲与 Reading 17（递归数据类型）关系紧密：`ImList` 这类不可变列表的 `size()`、`contains()`、`append()` 都可以用「每个 variant 一个 case」的递归方式定义，等价于对链表做 fold；理解 `reduce` 的「幺元 + 结合律」有助于读懂递归定义中 base case（如 `size(Empty) = 0`）的作用。本讲的 `split(/\W+/)`、`filter(s -> s.length() > 0)` 则直接依赖 Reading 18（正则表达式与文法）所介绍的字符串规格工具。

向后看，本讲是 Reading 21（并发）的重要地基：原文明确指出，「纯函数 + 不可变数据类型」上的 `map` / `filter` 天然可并行，「Maps and filters using pure functions over immutable datatypes are instantly parallelizable」，这就是 `parallel()` 敢自动开线程的理由；而一旦引入共享可变状态，就必须回到 Reading 23（互斥）与 Reading 24（消息传递）去解决竞争。Reading 20（回调与 GUI）与 Reading 22（Promise/async）中的事件处理大量使用函数式对象：把回调写成 lambda 正是本讲「一等函数」思想的直接延伸。最后，Reading 26/27（小语言）与 Reading 19（解析器）中「把语法树节点上的操作表达为一族小函数」也能看到 `map` / `reduce` 式抽象的影子。

关键要点

- **函数式接口就是「行为的类型」**：`Function<T,R>`（`apply`）、`Predicate<T>`（`test`）、`BinaryOperator<T>` 分别对应映射、过滤、归约所需的签名；方法引用的类型就是这些接口，可以存入变量、作为参数传递、作为返回值返回。
- **`map` 保长、`filter` 保型、`reduce` 归一**：记住三条签名 `Stream<E> × (E→F) → Stream<F>`、`Stream<E> × (E→boolean) → Stream<E>`、`Stream<E> × F × (F×E→F) × (F×F→F) → F`，绝大多数误用都能在写代码前被排除。
- **`reduce` 的三问**：初始值是什么（是否要求、是否为其运算的幺元）？运算符满足结合律吗？结果类型是否与元素类型相同（不同则要提供组合器）？三问中任何一问答错都会产生静默错误。
- **纯函数 + 不可变数据 = 可并行**：流操作中任何依赖共享可变状态或依赖元素处理顺序的写法都是错的，需要聚合时用 `reduce` / `collect` 而不是自己维护累加器。
- **流的生命周期只有一次**：`Stream` 消费后不可重用，需要再次遍历就重新构造；把「构造流」放进可重复调用的方法里。

常见陷阱与注意事项

- **把 `forEach` 当作 `map` 用**：需要结果却写成 `forEach(list::add)` → 依赖副作用与执行顺序，在并行流中丢数据或抛异常；正确做法是用 `map` + `collect`。
- **`reduce` 的初始值取错或不处理空序列**：求积写 `reduce(0, (a,b) -> a*b)` 会恒得 `0`；对没有天然幺元的运算（`min` / `max`）省略初始值后直接 `.get()`，在空序列上抛 `NoSuchElementException` → 类型

检查通不出任何警告，错误只在运行时显现；应当让初始值严格等于该运算的幺元，并用 `Optional` / `orElse` 或明确的极值哨兵把空序列语义写进规格。

- **在流管道里做有副作用的事**：打印日志、修改外部集合、递增计数器 → 顺序不确定、并行时结果不确定；副作用要么移到终端操作之后，要么改用 `forEach` 并接受「无返回值、不保证顺序」的语义。
- **把方法调用与函数对象混为一谈**：`files.map(File::toPath)` 是传函数对象，`files.map(File.toPath())` 既不是合法 Java 又表达了错误意图；`Math::sqrt` 与 `Math.sqrt(25)` 是两回事——前者是函数对象，后者是调用结果。同理，在 `forEach` 里写 `mySet::delete` 会丢失接收者 `this` 而不工作。
- **方法引用/ lambda 里调用有受检异常的方法却没包装**：代码无法通过编译（这是好事），但若为了通过编译而 `catch` 后返回 `null`，就把编译期问题换成了运行期灾难 → 应转换为 `UncheckedIOException` 之类的非受检异常并写进规格。
- **忘记 `Stream` 只能用一次**：复用已消费的流会抛 `IllegalStateException: stream has already been operated upon or closed` → 把流的构造封装成方法，每次需要时重新调用。

思考题（带答案）

问题 1：写出下面这段命令式代码的 `map` / `filter` / `reduce` 版本，并说明为什么改写后的版本在并行时结果不变。

```
static int productOfOdds(List<Integer> list) {
    int result = 1;
    for (int x : list) {
        if (x % 2 == 1) {
            result *= x;
        }
    }
    return result;
}
```

答案：

```
static int productOfOdds(List<Integer> list) {
    return list.stream()
               .filter(x -> x % 2 == 1)
               .reduce(1, (x, y) -> x * y);
}
```

`filter` 只保留奇数，`reduce` 以 `1`（乘法的幺元）为初始值把它们连乘。并行时结果不变的理由有两层：其一，`filter` 与 `reduce` 都不修改输入 `list`，也没有共享可变状态，因此「每个元素上做什么」与「谁先谁后」无关；其二，乘法满足结合律，`1` 是它的幺元，所以把序列切成若干段分别求积再用同一个运算符合并（这正是 Java 允许的实现自由度：`((1*a)*b)*c`、`(1*a)*(b*c)` 等等）得到的结果相同。空列表返回 `1`，即「空乘积」，符合数学约定。若把初始值写成 `0`，恰好会因为 `0` 不是乘法的幺元而对所有非空输入返回 `0`——这说明**初始值的选择不是风格问题，而是正确性问题**。

问题 2：`List.of(5, 8, 3, 1).stream().reduce(Math::max)` 的返回类型是什么？如果列表为空，三种常见写法的行为分别是什么？各自适合什么规格?

答案：省略初始值的 `reduce` 返回 `Optional<Integer>`，因为空序列没有值可以返回（sp22 的 TypeScript 版本在空数组时直接抛 `TypeError`，Java 用 `Optional` 把「可能没有结果」表达在类型里）。三种写法与空列表行为： 1. `list.stream().reduce(Math::max).get()`：非空时正确；空列表时 `get()` 抛 `NoSuchElementException`。适合「空输入是调用者的错误」的规格。 2. `list.stream().reduce(Math::max).orElse(0)`：空列表返回 `0`；但如果列表是 `List.of(-1,-2,-3)`，结果仍是 `-3`（正确），只是「空输入也得 0」这一语义必须写进规格，且当元素可能为负时 `0` 作为默认值容易误导。 3. `list.stream().reduce(Integer.MAX_VALUE, Math::min)` 用于求最小值时，空列表返回 `Integer.MAX_VALUE`——它确实是一个「哨兵值」，但如果元素集合可能包含比它更大的值（例如用 `Long` 元素）就会出错；用它求最大值则初始值应为 `Integer.MIN_VALUE`。

核心判据是：**`min` / `max` 没有天然的幺元**，所以要么用 `Optional` 把「没有结果」显式建模（推荐，可读性最好），要么选一个在该类型的取值范围内绝对不会被误认为合法结果的极值，并在规格中写清楚空序列的语义。

问题 3：为什么课程说「函数式写法让控制语句消失」是一种**好处**，而不是「把复杂度藏进库里」？请结合一条具体的流管道说明它分别如何改善三大目标。

答案：控制语句消失并不等于复杂度消失，而是把**通用**的复杂度（如何遍历、如何切分、如何在并行时合并部分结果、如何安全地收集）一次性放进经过充分测试的库实现里，同时让**特定**的复杂度（这次要映射什么、筛选什么、如何聚合）以最短的形式留在调用点。以

```
cameras.filter(camera -> camera.brand().equals("Nikon"))
       .map(Camera::pixels)
       .reduce(Math::max);
```

为例：`filter`、`map`、`reduce` 三段的意图与 SQL 的 `select max(pixels) from cameras where brand = "Nikon"` 一一对应（筛选—投影—聚合），读代码的人不需要在脑中模拟循环与下标。

对三大目标的作用分别是：**Safe from bugs**——没有下标、没有临时累加器、没有手写合并逻辑，因此没有越界与漏改状态的机会；纯函数 + 不可变数据使得把 `.stream()` 换成 `.parallelStream()` 不会改变结果，并行的正确性由库的结合律假设保证。**Easy to understand**——代码长度从几十行降到三行，意图（品牌过滤、取像素数、取最大值）直接可见，注释可以专注解释业务含义而非控制流。**Ready for change**——要换数据来源（`List` 换成 `Set`，只需改 `stream()` 的来源）、要加条件（插入一个 `filter`）、要改聚合方式（`max` 换 `average`）都只改一处，且改动局限在链条内，不会波及其他逻辑。唯一的代价是团队必须理解 `reduce` 的三条设计选择，否则「库帮了忙」会变成「错误被写得更短」。

问题 3 追问：上面的论述说明「纯函数 + 不可变数据 ⇒ 并行安全」，那么下面这段代码为什么可能出错？请给出修正，并说明这与本讲「无状态函数」的要求有何关系。

```
List<String> words = new ArrayList<>();
lines.stream().parallel().map(line -> line.split("\\W+"))
     .forEach(arr -> { for (String w : arr) if (!w.isEmpty()) words.add(w); });
```

答案：错误在于 `forEach` 内部的 lambda 修改了共享的可变 `ArrayList`。`ArrayList` 不是线程安全的，而 `.parallel()` 会让 `forEach` 在不同线程上并发执行，可能出现元素丢失、结果顺序混乱，甚至因内部数组扩容竞争而抛出异常；即便改成顺序流，代码也依赖 `forEach` 的执行顺序这一库**不保证**的性质，同时把结果通过副作用「输出」到外部变量，使方法不再是纯函数。课程原文的规则很明确：传给 `map` / `forEach` 的函数必须是**无状态**的，其行为不应依赖在 map/forEach 执行过程中变化的状态，因为实现可能并行执行它们。

修正方式是把「收集」表达为管道的终端操作：

```
List<String> words = lines.stream().parallel()
        .flatMap(line -> Stream.of(line.split("\\W+")))
        .filter(w -> !w.isEmpty())
        .collect(Collectors.toList());
```

（若要求去重与顺序稳定，可再加 `.distinct()` 并去掉 `parallel()`，或使用 `Collectors.toCollection` 指定集合类型。）改写后每个阶段都是纯函数，`collect` 负责安全合并部分结果，`parallel()` 的存废只影响性能而不影响正确性——这正是本讲反复强调的「不可变数据 + 纯函数 ⇒ 安全并发」的实践含义。

Reading 17: 递归数据类型（Recursive Data Types）

> 说明：本讲 sp22 原版使用 TypeScript，本笔记按用户要求提供 Java 代码示例；类型/API 与 sp21（6.031 Java 版）原文保持一致。

概述

本讲讨论**递归数据类型（recursive data type）**：一种用自身来定义自身的数据类型，正如递归函数用自身来定义自身一样。我们要回答四个问题：如何读写**数据类型定义（datatype definition）**；如何为这种类型的每个 variant 分别实现操作；**不可变列表** `ImList<E>` 这个经典例子长什么样；以及写 ADT 时应当遵循的「配方（recipe）」。核心设计原则是：把操作的**规格**声明在抽象接口上，把操作的**实现**按 variant 递归地分散到各个具体类里（这一模式有时被戏称为 interpreter pattern），从而使「数学定义」到「代码」的转换几乎是机械的。

它与三大目标的关系如下。**Safe from bugs**：递归数据类型没有下标、没有可变状态、没有 `null` 哨兵，`Cons` 与 `Empty` 分派由动态派发（dynamic dispatch）而非运行期类型检查完成，因此整类越界与空指针 bug 在类型层面被消除。**Easy to understand**：写好的代码「little more than the definition, with some semicolons to placate the compiler」——`size(Empty) = 0`、`size(Cons(elt, rest)) = 1 + size(rest)` 与 Java 实现几乎逐字对应，阅读者可以像读数学定义一样读代码。**Ready for change**：只要隐藏具体 variant、不违反表示不变量的前提下，内部表示（加缓存字段、改用数组支撑 `get`、换成别的实现）都可以自由替换，而客户端的规格不变。

核心概念与设计原则详解

递归数据类型与数据类型定义（Recursive Data Type & Datatype Definition）

- **定义与目的**：若一个数据类型在**自己的定义中出现在右端**（作为某个字段的类型），它就是递归数据类型。数据类型定义则把「抽象类型 = 若干 variant 的并集」这一结构写清楚，用来思考抽象类型、特别是递归抽象类型。
- **直观解释（"它是什么？"）**：想象俄罗斯套娃：一个套娃里面可以装另一个套娃，也可以什么都不装。`ImList` 就是这样——一个 `Cons` 里装着另一个 `ImList`，而最里层是一个什么也不装的 `Empty`。递归函数有 base case 与 recursive step，递归数据类型同样有 base case（`Empty`）与 recursive step（`Cons`）。
- **关键规则与最佳实践**：
- 数据类型的正式构成：左边是抽象数据类型，右边是它的**表示（representation，也称具体数据类型）**；表示由若干 **variant（变体）** 用并集运算符 `+` 组合而成；每个 variant 是「类名 + 零个或多个字段」，字段写成 `名字:类型`。
- `ImList<E> = Empty + Cons(elt:E, rest:ImList<E>)` 表示：`ImList` 的值要么由无字段的 `Empty` 对象表示，要么由字段为「一个元素 `elt` 与一个 `ImList` 类型的 `rest`」的 `Cons` 对象表示。
- 任何值都可以写成项（term）：`[0, 1, 2]` 写成 `Cons(0, Cons(1, Cons(2, Empty)))`。整个无限集合可以从 base case `Empty` 出发，反复应用 `Cons` 生成出来。
- 把递归数据类型定义**作为注释写在接口里**（sp21 原文用 `// Datatype definition:`），这样读接口的人立刻能看到全貌。
- 这种「variant 并集」在函数式编程中叫**代数数据类型（algebraic data type）**；Haskell/ML 用另一套语法，但思想一致。
- 常见的递归数据类型还有二叉树 `Tree<E> = Empty + Node(e:E, left:Tree<E>, right:Tree<E>)`、可选值 `Optional<E> = None + Some(value:E)`、布尔公式 `Formula = Variable(name:String) + Not(formula:Formula) + And(left:Formula, right:Formula) + Or(left:Formula, right:Formula)`。
- 定义必须保证**能构造出值**：`V = F(z:int, v:V) + G(z:int, v:V)` 无法构造任何实例（没有 base case），而 `W = K(n:int) + L(n:int, m:W)` 虽然递归但没有表示空序列的 variant，因此不能像 `X = M + N(here:int, there:X)` 那样当作列表使用。

不可变列表与四个基本操作（Immutable Lists: empty/cons/first/rest）

- **定义与目的**：`ImList<E>` 是递归数据类型的经典例子，它提供不可变的列表抽象，其不可变性不仅带来安全性，还带来**共享（sharing）**的可能，从而减少内存占用与复制时间。
- **直观解释（"它是什么？"）**：它不是数组，也不是可变链表，而更像一条「焊死了的链条」：每次添加元素都是在前端接上新的一节，原来的链条一动不动，别人手中的链条也不会变。
- **关键规则与最佳实践**：

- 四个基本操作：`empty: void → ImList`（返回空列表）、`cons: E × ImList → ImList`（在另一个列表前端加入一个元素并返回新列表）、`first: ImList → E`（返回第一个元素，要求非空）、`rest: ImList → ImList`（返回除第一个元素外的所有元素，要求非空）。
- 这四个操作历史悠久：在 Lisp/Scheme 中被称为 `nil`、`cons`、`car`、`cdr`；函数式编程中 `first`/`rest` 也常叫 `head`/`tail`。
- 它们之间的根本关系是 `first(cons(elt, list)) = elt` 与 `rest(cons(elt, list)) = list`——**cons 组合起来的，由 first 和 rest 拆开**。
- `cons` 加在**前端**，因此 `nil.cons(2).cons(1).cons(0)` 的结果是 `[0, 1, 2]`：后调用者位于更前面。
- 共享结构：`ImList<Integer> y = x.rest().cons(4);` 得到的 `[4, 1, 2]` 与 `x`（`[0, 1, 2]`）共享子列表 `[1, 2]` 的那份表示，内存中只有一份，两个引用都指向它；因为列表不可变，这种别名（aliasing）完全安全。
- 注意只有**尾部**能被共享：如果两个列表前缀相同而后缀不同，前缀必须各自存储（因为每个 `Cons` 只有一个 `rest` 字段，无法让两个不同的后缀共享同一个 `Cons` 前缀）。

两个实现类协作实现一个抽象类型（Empty and Cons Cooperate）

- **定义与目的**：`ImList` 的表示由 `Empty` 与 `Cons` **两个类合作**构成：它们不是 `ArrayList`/`LinkedList` 那种「同一抽象的两个可替换表示」，而是**同一个表示的两个 variant**，缺一不可。
- **直观解释（"它是什么？"）**：像拼装玩具的两半——「空槽」与「一节车厢」——只有合起来才能拼出任意长度的列车；而 `ArrayList` 与 `LinkedList` 更像是两种不同材质造出的同型列车，任选其一即可。
- **关键规则与最佳实践**：
- 接口 `public interface ImList<E>` 声明所有操作；`Empty<E>` 与 `Cons<E>` 各自实现同一批操作，但语义按 variant 不同。
- 不要在客户端代码里 `new Empty<>()`：那会牺牲**表示独立性（representation independence）**，因为客户端必须知道 `Empty` 这个类。正确做法是用静态工厂 `ImList.empty()`（Java 8+ 允许接口中的静态方法），并进一步把 `Empty`/`Cons` 声明为**包私有（package-private）**，让包外的类根本看不到它们。
- `ImList` 是接口：`new ImList()` 被 Java 禁止，因为它没有自己的对象值，也没有构造器。
- 任何 ADT 的**规格都不能谈论 rep**。递归 ADT 的具体 variant 就是它的 rep，因此规格中绝不能出现 `Empty`、`Cons` 这些名字。例如 `isEmpty` 的规格是「当且仅当本列表不含元素时返回 true」，而不是「当且仅当 this 是 `Empty` 的实例时返回 true」。
- `first`/`rest` 的规格写着「requires the list to be nonempty」；`Empty` 的实现应当**快速失败（fail fast）**，抛出 `UnsupportedOperationException` 之类的异常，而不是返回 `null` 或某个默认值。

递归数据类型的操作：每个 variant 一个 case（Functions with One Case per Variant）

- **定义与目的**：这种「把类型看成 variant 并集」的思考方式之所以有吸引力，不只是因为它能处理列表、树这类递归且无界的结构，还因为它提供了一个描述操作的方便途径：**每个 variant 一个 case 的函数**。

- **直观解释（"它是什么？"）**：像按食谱做菜：先写「空列表怎么做」（`size(Empty) = 0`），再写「非空列表怎么做」（`size(Cons(elt, rest)) = 1 + size(rest)`），两步合起来就完整规定了操作的含义，也天然对应到接口方法与两个实现类。
- **关键规则与最佳实践**：
- 实现一个操作的固定套路：① 在抽象接口里**声明**该操作；② 在**每个具体 variant里递归地实现**它。
- 用一系列「归约步骤（reduction steps）」来心算递归：`size(Cons(0, Cons(1, Empty))) = 1 + size(Cons(1, Empty)) = 1 + (1 + size(Empty)) = 1 + (1 + 0) = 2`。
- 常用操作的函数式定义：
  - `isEmpty(Empty) = true`；`isEmpty(Cons(elt, rest)) = false`。
  - `contains(Empty, e) = false`；`contains(Cons(elt, rest), e) = (elt = e) or contains(rest, e)`。
  - `get(Empty, n) = undefined`；`get(Cons(elt, rest), n) = if n = 0 then elt else get(rest, n - 1)`。
  - `append(Empty, list2) = list2`；`append(Cons(elt, rest), list2) = cons(elt, append(rest, list2))`。
  - `reverse(Empty) = empty()`；`reverse(Cons(elt, rest)) = append(reverse(rest), cons(elt, empty()))`。
- 注意 `reverse` 的这个递归定义产生的实现**性能很差**：其代价与列表长度的平方成正比（每次 `append` 都要走一遍左前缀），需要时可以用迭代方式重写。
- 实现 `contains` 时判等要用 `equals`（Reading 15），不要用 `==`；否则对 `String`、`Integer` 之外的引用类型会得到错误结果。

递归类型的抽象函数与表示不变量（AF and RI for Recursive Types）

- **定义与目的**：抽象函数（abstraction function, AF）说明「rep 的一个取值代表哪个抽象值」，表示不变量（representation invariant, RI）说明「哪些 rep 取值是合法的」。对递归数据类型，AF/RI 必须**按 variant 分别写**，因为每个 variant 的 rep 字段不同。
- **直观解释（"它是什么？"）**：AF 是「内部零件 → 外部含义」的翻译表，RI 是「内部零件必须满足的组装规则」。`Empty` 的 AF 只说一件事：它代表空列表；`Cons` 的 AF 要说清「第一个元素是 `elt`，其余元素是 `rest` 所代表的列表」。
- **关键规则与最佳实践**：
- `Empty`：`AF() = the empty list []`；`RI: true`（没有需要约束的字段）。注意 `AF(first) = an empty list` 这样的写法是错的，因为 `Empty` 没有名为 `first` 的字段。
- `Cons`：`AF(elt, rest) = 第一个元素是 elt、其余元素是 rest 所代表列表的那个列表`。像「`AF(elt, rest) = a non-empty list`」这种说法**信息量不足**（没有说明哪个元素在前），而「`AF(elt, rest) = a two-element list where the first element is elt and the second element is rest`」则是**错误的**（把 `rest` 说成了元素，而它其实是一个列表）。

- `Cons` 的 `RI`：`elt != null`、`rest != null`（以及实现缓存时新增的条款，见下文）。
- 规格中绝不提及 variant，AF/RI 中也绝不混入抽象值以外的承诺。
- 因为 `cons`、`first`、`rest` 都返回或接收 `ImList` 而非具体类，AF 的递归描述才能自然地把「子列表」交给下一层的 AF 处理。

表示独立性、表示暴露与受益人式修改（Rep Independence, Rep Exposure, Beneficent Mutation）

- **定义与目的**：表示独立性意味着客户端只依赖抽象操作，实现可以自由更换；表示暴露意味着内部 rep 的引用泄漏给了客户端，从而可能被破坏。**受益人式修改（beneficent mutation）**则是一种特殊技巧：不可变类型内部可以有可变的 rep，只要状态变化**不改变对象所表示的抽象值**。
- **直观解释（"它是什么？"）**：前两者像「只提供柜台服务，不让顾客进后厨」；受益人式修改则像「餐厅把算好的账单金额贴在墙上做备忘」——贴与不贴，顾客看到的账单一模一样。
- **关键规则与最佳实践**：
- `ImList` 的实现确实保持了表示独立性：`Empty` 构造器被 `ImList.empty()` 隐藏，客户端不需要也不应该直接使用 `Empty` / `Cons` 构造器。
- 因此实现有很大的自由度：可以给 `Cons` 加 `size` 字段，甚至可以在内部加一个数组让 `get()` 变快（代价是空间），这些取舍由实现者决定。
- 像 `isEmpty` 这样的操作**不会**破坏表示独立性：它的规格是抽象的（「列表是否不含元素」），任何实现方式（递归的 Empty/Cons、数组支撑、可变链表）都能满足。
- `Cons.rest()` 返回内部列表的引用，看似是表示暴露——但因为内部列表**不可变**，任何人都无法通过它威胁 `Cons` 的不变量（既不能破坏不可变性，也不能让缓存的 `size` 失效）。
- 引入缓存字段时必须同步更新 RI，明确写出缓存正确性条款，例如 `size > 0 implies size == 1 + rest.size()`。

哨兵对象与 null 的对比（Sentinel Objects vs null）

- **定义与目的**：用一个真实对象（`Empty`）而不是 `null` 引用去表示数据结构的 base case 或端点，这一设计模式称为**哨兵对象（sentinel objects）**。它的巨大优势在于「它像数据类型中的普通对象一样工作，因此可以在它上面调用方法」。
- **直观解释（"它是什么？"）**：就像队列里放一个「空位」的牌子而不是把这一格挖掉——你依然可以对牌子做操作，不必每次都先判断「这里有没有东西」。
- **关键规则与最佳实践**：
- 若用 `null` 表示空列表，代码里就会充满 `if (list != null) n = list.size();` 这类检查，它们污染代码、掩盖意图、而且容易忘记写。
- 有哨兵对象时可以直接写 `n = list.size();`，对空列表也永远有效。
- 「把 `null` 值赶出你的数据结构，你的日子会好过得多」——这条规则与 Reading 15（相等性）中对 `equals(null)` 处理的要求、以及 Reading 07（设计规格）中「避免用 `null` 表示缺失」的建议一脉相承。

- 需要表达「可能没有值」时，用 `Optional<E>` 这类显式类型（其数据类型定义为 `Optional<E> = None + Some(value:E)`），而不是 `null`。

静态类型与实际类型；`instanceof` 反模式（Declared Type vs Actual Type）

- **定义与目的**：编译期每个变量有一个**声明类型（declared type，也叫静态类型、编译期类型）**，运行期每个对象有一个由构造器赋予的**实际类型（actual type，也叫动态类型、运行期类型）**。理解这个区分，才能理解动态派发如何让我们「按 variant 分派实现」。
- **直观解释（"它是什么？"）**：变量的声明类型像「合同上写的职位」，对象的实际类型像「这个人实际会做什么」。`ImList<String> words2 = ImList.empty();` 中变量的声明类型是 `ImList`，而它指向的对象的实际类型是 `Empty`。
- **关键规则与最佳实践**：
- `String hello = "Hello"` 中 `hello` 的声明类型是 `String`、实际类型也是 `String`（对基本类型与不可变值类型，两者一致）；`List<String> words1 = new ArrayList<>()` 中变量的声明类型是 `List`、实际类型是 `ArrayList`。
- 动态派发使客户端调用 `size()` 时自动执行 variant 对应的实现，这就是我们「免费」得到按 variant 分派的原因。
- **不要用 `instanceof` 检查运行期类型**。课程明确说：`instanceof` 是运行期类型检查，比静态类型检查既更不安全（less safe from bugs）也更难适应变化（less ready for change）。`if (this.rest instanceof Empty) { return this.first; }` 这样的写法是反模式。
- 当 `instanceof` 看起来很诱人时，正确反应是**回头重新思考问题**：`Cons` 并不关心 `rest` 的表示，只关心它的抽象值。如果类型提供的操作不够用，就给类型**增加操作**，而不是去窥探 rep。
- 唯一可能「least-bad」的例外是为不可变类型定义 `equalValue`（Reading 15）：由于客户端只知道 `ImList`，需要在接口上声明 `equalValue(ImList<E>)`，此时或者用 `size()` + `get()` 等观察者操作在不看 variant 的前提下判等（笨重但完全在抽象屏障之上，更安全），或者检查运行期类型（优雅、递归结构与数据同形，但引入了运行期类型检查的风险）。

不可变链表的性能特征与回溯搜索（Performance & Backtracking）

- **定义与目的**：不可变列表的共享结构决定了它的性能画像；理解这一点，才能在「何时用 `ImList`、何时用数组或可变结构」上做出正确取舍，并理解为什么回溯搜索特别适合不可变结构。
- **直观解释（"它是什么？"）**：`cons` 像在前端接一节车厢，代价恒定；而 `append` 像要把整列车拆开重接到另一列车前面，代价与左列表长度成正比。共享则像两列火车共用同一段尾轨。
- **关键规则与最佳实践**：
- `cons`、`first`、`rest` 都是 O(1)；`contains` 是 O(n)；`get(i)` 是 O(i)；朴素的 `size()` 是 O(n)。
- `append` 是 O(this 的长度)：它复制左列表的「脊柱」，而右列表被**完整共享**（`append(Empty, list2) = list2` 直接返回 `list2` 本身，这是共享的直接体现）。

- 按递归定义实现的 `reverse` 是 $O(n^2)$，因为每一步 `append` 都要走一遍前缀；需要时改用迭代（用一个累加器从前往后 `cons`）可降到 O(n)。
- 给 `size` 加缓存可把后续查询降到 O(1)，这是**受益人式修改**的典型用法；因为 `Cons` 的 size 永远不为 0，sp21 用 `0` 作为「尚未计算」的哨兵值，并在 RI 中写明 `size >= 0` 与 `size > 0 implies size == 1 + rest.size()`。
- 回溯搜索（如布尔公式的可满足性问题）是这类列表的绝佳应用：搜索空间中的每一步只需在前端 `cons` 一次就能共享此前全部信息；回溯时「停止使用当前状态」即可，而之前的状态仍然被引用着，不需要像可变 `Map` 那样逐个撤销绑定。
- 但「完全没有共享的不可变结构」并不好：如果每一步都要完整复制环境，空间开销会随步数平方增长，因为你必须保留路径上所有历史环境以便回退。
- 用不可变数据结构实现的搜索**立刻可并行**：可以把多条路径分给多个处理器，不必担心它们在共享可变结构上互相踩踏（见 Reading 21 并发）。

代码示例与对比分析

场景 1：空列表上的 `first()` / `rest()`——快速失败而不是返回默认值

❌ *错误代码*

```
public class Empty<E> implements ImList<E> {
    public Empty() {
    }
    public ImList<E> cons(E elt) {
        return new Cons<>(elt, this);
    }
    public E first() {
        return null;          // 错误：用 null 掩盖「不允许调用」
    }
    public ImList<E> rest() {
        return this;          // 错误：返回一个看似合理的空列表
    }
}
```

【错误代码的问题】 1. 违反规格。`first` / `rest` 的规格写着「requires the list to be nonempty」，在空列表上调用是**调用者的 bug**；返回 `null` / `this` 把调用者的错误伪装成正常结果，bug 会沿着调用链继续传播到很远的地方才以 `NullPointerException` 或错误答案的形式爆发。 2. `first()` 返回 `null` 之后，调用点往往写成 `if (list.first() == null) ...`，于是「空列表」与「首元素恰好是 null」两种完全不同的情况被混为一谈——这正是 Reading 15 关于 `equals(null)` 所警告的语义混淆。 3. `rest()` 返回 `this` 让 `rest().rest().rest()` 在任何列表上都永远不报错，掩盖了下标越界式的逻辑错误。

✅ *正确代码*

```
public class Empty<E> implements ImList<E> {
    // Abstraction function:
    //   AF() = the empty list []
    // Representation invariant:
    //   true
    // Safety from rep exposure:
    //   no fields at all

    public Empty() {
    }
    public ImList<E> cons(E elt) {
        return new Cons<>(elt, this);
    }
    public E first() {
        throw new UnsupportedOperationException("first() of an empty list");
    }
    public ImList<E> rest() {
        throw new UnsupportedOperationException("rest() of an empty list");
    }
}
```

【为什么这样更好】 明确地把「前置条件被违反」这一事实立刻暴露出来：异常在错误发生的**现场**抛出，栈轨迹直接指向出错的那一行，调试成本极低（Reading 09、Reading 13）。同时 `Empty` 与 `Cons` 的分工变得清晰：`Empty` 只负责「空」这一 variant 的语义，其余全部拒绝。

【代码对比解说】 两种实现都「通过编译」、都能满足 `size()`、`isEmpty()` 这些不需要 `first()` 的操作；差别只在**违反规格时**的表现。这里体现了一条通用原则：**对违反前置条件的输入，要么用未检查异常快速失败，要么用规格明确允许的默认行为；绝不要返回一个语义含糊的值。**`Empty.rest()` 返回 `this` 看起来「友好」，实际上破坏了 `first(rest(x))` 与 `cons`/`first`/`rest` 的基础等式所隐含的结构信息——它让空列表变成了一个「无限长的空列表」，与数据类型定义 `ImList<E> = Empty + Cons(elt:E, rest:ImList<E>)` 所刻画的有限结构不符。

【设计原则透视】 `Empty` 的 AF 是 `AF() = the empty list []`，RI 是 `true`：它没有任何字段，因此没有可违反的约束。抛出异常的实现完全遵守这两条；而返回 `null` 的实现在 AF 上已经说不通——`Empty` 的抽象值只能是空列表本身，不能是「没有首元素的某个值」。从抽象屏障看，客户端只需知道 `first` 要求非空，而不需要知道 `Empty` 的存在（`Empty` 应当被声明为包私有，并通过 `ImList.empty()` 这个静态工厂暴露），因此这里抛出的是标准库异常而不是自定义类型，以免泄漏 rep 细节。

场景 2：用 `null` 表示空列表——哨兵对象 `Empty` 才是正解

❌ *错误代码*

```
public class ImListOps {
    /**
     * @param head first element, may be null to mean "no elements"
     * @param tail the rest of the list, may be null
     * @return the size of the list
     */
    public static int size(Node head, Node tail) {
        int n = 0;
        Node cur = head;
        // 错误：用 null 表示空列表，于是每处使用都要判空
        while (cur != null) {
            n = n + 1;
            cur = cur.next;   // next 为 null 表示结束
        }
        if (tail != null) {
            // 忘记处理 tail 的情况也时有发生
        }
        return n;
    }
}
```

【错误代码的问题】 1. 每一处使用都要写 `!= null` 判断：代码被判空语句淹没，真正的含义（「求列表长度」）被掩盖，而且**很容易漏写一处**，漏写就是空指针异常。 2. `null` 的含义被重载：它既表示「列表结束」，又可能表示「调用者传了 null 参数」，两种语义无法区分，规格里只能写「may be null」，把不确定性推给所有调用者。 3. 无法在空列表上调用方法，因此所有操作都要么是静态工具方法、要么在入口处特判，无法真正做到「操作属于类型」（`list.size()` 这样的写法根本不可能实现）。

✅ *正确代码*

```
public interface ImList<E> {
    // Datatype definition:
    //   ImList<E> = Empty + Cons(elt:E, rest:ImList<E>)

    /**
     * @return an empty list
     */
    public static <E> ImList<E> empty() {
        return new Empty<>();
    }

    /**
     * @param elt element to add
     * @return a new list with elt at the front of this list
     */
    public ImList<E> cons(E elt);

    /**
     * @return the first element; requires this list to be nonempty
     */
    public E first();

    /**
     * @return the list of all elements except the first;
     *         requires this list to be nonempty
     */
    public ImList<E> rest();

    /**
     * @return the number of elements in this list
     */
    public int size();
}
```

【为什么这样更好】 空列表是一个**真实对象**（`Empty` 的实例），因此可以像任何列表一样接收 `size()`、`isEmpty()`、`cons()` 等调用，客户端代码里彻底不需要判空分支：`n = list.size();` 永远有效。`null` 从此退出这个数据类型的表示，`E` 的取值也不必再被 `null` 污染，AF/RI 得以简单清晰地陈述。

【代码对比解说】 这是「哨兵对象」模式的标准收益：把「没有元素」这一**抽象概念**用对象表达，而不是用语言的空引用表达。课程原文的论证很直接——若空列表用 `null` 表示，代码就会充满 `if (list != null) n = list.size();` 这样的测试，它们扰乱代码、模糊含义、且容易忘记；有了哨兵对象就可以写 `n = list.size();`。同理，`static` 工具方法把列表操作变成了「外部函数」，违背了面向对象的封装；而把方法放进接口后，`Empty` 与 `Cons` 各自实现自己的那一份语义，动态派发替我们完成分派。

【设计原则透视】 这直接关系到 AF/RI：`Empty` 的 AF 是 `AF() = the empty list []`，它必须是一个**对象**才能成为 `ImList<E>` 的一个合法值；如果空列表是 `null`，那么 `ImList` 的抽象值集合中有一部分根本无法用合法对象表示，RI 也只能写成「this == null 或 ...」，这是设计上的失败。从 Reading 10（抽象数据类型）的角度看，把 `size` 声明在接口上意味着它成为**类型的操作**而不是**外部过程**，客户端只依赖抽象；从

Reading 12（接口、泛型与枚举）的角度看，`static <E> ImList<E> empty()` 中的 `E` 是方法自己的类型参数（静态方法看不到实例的类型参数），读作「对任意 E，`empty()` 返回一个 `ImList<E>`」。

场景 3：`last()` 的实现——不要用 `instanceof` 窥探 variant

❌ *错误代码*

```
public class Cons<E> implements ImList<E> {
    private final E elt;
    private final ImList<E> rest;

    public Cons(E elt, ImList<E> rest) {
        this.elt = elt;
        this.rest = rest;
    }
    public ImList<E> cons(E elt) { return new Cons<>(elt, this); }
    public E first() { return elt; }
    public ImList<E> rest() { return rest; }
    public int size() { return 1 + rest.size(); }

    /**
     * @return the last element; requires this list to be nonempty
     */
    public E last() {
        if (this.rest instanceof Empty) {   // 错误：运行期类型检查
            return this.first();
        }
        return this.rest.last();
    }
}
```

【错误代码的问题】 1. `instanceof` 是运行期类型检查，比静态类型检查既更不安全、也更难适应变化：只要有人新增第三个 variant（例如一个共享后缀的 `Cons` 或数组支撑的 `Chunk`），这里的分支就会静默失效，而编译器不会给出任何提示。 2. 它把 `Cons` 与 `Empty` 的具体表示**焊死**在一起，破坏了表示独立性与「两个类合作实现抽象类型」的设计：`Cons` 本应只依赖 `ImList` 的抽象操作，却开始关心 `rest` 的 rep。 3. 由于 `last()` 只声明在 `Cons` 上而接口中没有声明，客户端拿到 `ImList<E>` 类型时无法调用它，只能做向下转型，进一步引入运行期类型判断。

✅ *正确代码*

```
public interface ImList<E> {
    // ... empty(), cons(), first(), rest(), size() ...

    /**
     * @param index index into the list, requires 0 <= index < size()
     * @return the element at that index
     */
    public E get(int index);

    /**
     * @return the last element; requires this list to be nonempty
     */
    public E last();
}
```

```
public class Empty<E> implements ImList<E> {
    // ...
    public E get(int index) {
        throw new IndexOutOfBoundsException("empty list has no elements");
    }
    public E last() {
        throw new UnsupportedOperationException("last() of an empty list");
    }
    public int size() { return 0; }
    public boolean isEmpty() { return true; }
}

public class Cons<E> implements ImList<E> {
    private final E elt;
    private final ImList<E> rest;
    // ...
    public E get(int index) {
        // get(Cons(elt, rest), n) = if n = 0 then elt else get(rest, n - 1)
        return (index == 0) ? elt : rest.get(index - 1);
    }
    public E last() {
        return get(size() - 1);   // 只用抽象操作，不看 variant
    }
    public int size() { return 1 + rest.size(); }
    public boolean isEmpty() { return false; }
}
```

【为什么这样更好】 `last()` 被实现在抽象层之上：它通过 `size()` 与 `get()` 这两个已经规定好的抽象操作表达「最后一个元素」的含义，完全不触及任何具体 variant。因此无论 `rest` 内部是 `Empty`、`Cons` 还是未来某种新 variant，这段代码都继续正确——这正是「规格写在接口、实现随 variant 分派」带来的可修改性。同时把 `last` / `get` 声明在接口上，客户端无需向下转型即可使用。

【代码对比解说】 两种写法的**递归结构与数据的递归结构**不同：`instanceof` 版本是「边递归边窥探表示」，抽象操作版本是「把递归下放到 `get` / `size`，`last` 只做组合」。课程原文的建议是：每当 `instanceof` 显得诱人时，就退一步重新思考——`Cons` 不关心 `rest` 的表示，只关心其抽象值；如果现有操作不足以表达

需求，就给类型**增加操作**（这里增加了 `size()`、`get()`、`isEmpty()`、`contains()`、`append()`、`reverse()` 这类通用操作），而不是去检查运行期类型。代价是 `last()` 现在需要先算 `size()`（O(n)）再 `get()`（O(n)），如果不加缓存则是 O(n) 的两次遍历；若性能敏感，可以给 `Cons` 加 `size` 缓存（见场景 5），或者改用带累加器的递归实现——但绝不回到 `instanceof`。

【设计原则透视】 这一组对比精准地展示了 Reading 11（抽象函数与表示不变量）中的**抽象屏障（abstraction barrier）**：`Cons` 既是 `ImList` 的实现者，又是其（递归的）客户端，作为客户端它必须只使用接口承诺的操作。用 `instanceof` 等于越过屏障去看 rep，一旦 rep 改变（新增 variant、把 `Empty` 换成单例、把链式结构换成数组块），越过屏障的代码立刻失效；而站在抽象层之上的实现只依赖规格，规格不变则代码不变。这也是 `isEmpty` 的规格必须是抽象的（「不含元素」）而不能是「是 `Empty` 的实例」的原因。

场景 4：为让 `append` 变快而就地修改 `rest`——破坏共享的致命诱惑

❌ *错误代码*

```
public class Cons<E> implements ImList<E> {
    private final E elt;
    private ImList<E> rest;      // 错误：不是 final，可以被就地改写

    public Cons(E elt, ImList<E> rest) {
        this.elt = elt;
        this.rest = rest;
    }
    public ImList<E> cons(E elt) { return new Cons<>(elt, this); }
    public E first() { return elt; }
    public ImList<E> rest() { return rest; }
    public int size() { return 1 + rest.size(); }

    /**
     * @param other list to append to this list
     * @return list with the elements of this followed by the elements of other
     */
    public ImList<E> append(ImList<E> other) {
        // 错误：为了「原地」拼接而改写自己的 rest 字段
        this.rest = this.rest.append(other);
        return this;
    }
}
```

【错误代码的问题】 1. 破坏不可变性，并**污染所有共享者**：`ImList<Integer> x = nil.cons(2).cons(1).cons(0);`（`[0,1,2]`）与 `ImList<Integer> y = x.rest().cons(4);`（`[4,1,2]`）共享子列表 `[1,2]`；若对 `x` 调用一次 `append`，`y` 的内容会**追溯性地改变**，此前创建的所有引用的含义全部失效——这类 bug 极难定位。 2. 使缓存字段（如 `size`）失效，违反 RI 中 `size == 1 + rest.size()` 的条款；一旦并发使用（Reading 21、Reading 23），`rest` 的读写还会产生数据竞争。 3. 违反 `append` 的规格语义：规格承诺返回「this 的元素后接 other 的元素」，并未允许修改 this；客户端合理地假设 `x` 不变，程序其余部分因此崩溃。

✅ *正确代码*

```
public class Cons<E> implements ImList<E> {
    private final E elt;
    private final ImList<E> rest;   // 正确：final，永不改写

    // Abstraction function:
    //   AF(elt, rest) = the list whose first element is elt,
    //                   followed by all the elements of rest
    // Representation invariant:
    //   elt != null, rest != null
    // Safety from rep exposure:
    //   all fields are private and final; E and ImList<E> are immutable,
    //   so exposing this rep through rest() cannot threaten the invariant

    public Cons(E elt, ImList<E> rest) {
        this.elt = elt;
        this.rest = rest;
    }
    public ImList<E> cons(E elt) { return new Cons<>(elt, this); }
    public E first() { return elt; }
    public ImList<E> rest() { return rest; }
    public int size() { return 1 + rest.size(); }

    /**
     * @param other list to append to this list
     * @return a new list with the elements of this followed by those of other
     */
    public ImList<E> append(ImList<E> other) {
        // append(Cons(elt, rest), other) = cons(elt, append(rest, other))
        return new Cons<>(elt, rest.append(other));
        // 等价写法：return rest.append(other).cons(elt);
    }
}
```

```
public class Empty<E> implements ImList<E> {
    // ...
    public ImList<E> append(ImList<E> other) {
        // append(Empty, other) = other   —— 直接返回 other，实现完整共享
        return other;
    }
}
```

【为什么这样更好】 `append` 是**非破坏性**的：它复制左列表的「脊柱」（每一个 `Cons` 换成一个新 `Cons`），而把右列表 `other` 原封不动地接在末尾——`Empty.append(other)` 直接返回 `other` 这一行就是共享的证明。于是所有既有列表保持不变，共享结构继续安全，RI 与缓存都成立。注意 `new Cons<>(elt, rest.append(other))` 与 `rest.append(other).cons(elt)` 是等价的正确写法，而 `new Cons<>(other, rest.append(elt))` 把参数顺序弄反（既类型错误又语义错误：`cons` 是「把元素放在前端」，不是「把列表放在前端」）。

【代码对比解说】 这一组的核心是**共享与可变的冲突**：不可变列表的性能优势正是建立在结构共享之上，而就地修改会从根上摧毁这一前提。课程原文说得很清楚：共享意味着更少的内存与更少的复制时间；而共享之所以**安全**，完全依赖不可变性——「this aliasing is perfectly safe because the list is immutable」。一旦放弃不可变性，别名就从「优化」变成了「定时炸弹」。性能上也要算清楚账：`append` 的代价是 O(this 的长度)，无法靠就地修改变成 O(1)（除非引入 `Seq`/差分列表之类的结构，那属于另一层设计）。

【设计原则透视】 从 RI 的角度看，把 `rest` 改成非 `final` 并允许改写，等于放弃了「所有 `Cons` 的 `rest` 字段在其生命周期内恒定」这一条隐含不变量，而这条不变量正是 `size` 缓存正确、`first`/`rest` 等式成立的基础。从 Reading 08（不可变性）的角度看，这正是「不可变类型必须防御性地不泄漏可变 rep」的另一种表现：这里不是泄漏，而是自己内部破坏；结论相同——不可变类型的字段应当 `private final`。从 Reading 21/23 的角度看，可变 rep 还需要同步机制，而不可变 + 共享完全不需要，这也是课程把「回溯搜索用不可变列表」当作范例的原因。

场景 5：给 `size()` 加缓存——受益人式修改必须同步更新 RI

❌ *错误代码*

```
public class Cons<E> implements ImList<E> {
    private final E elt;
    private final ImList<E> rest;
    public int size = 0;   // 错误：public 且可变，RI 没有任何说明

    public Cons(E elt, ImList<E> rest) {
        this.elt = elt;
        this.rest = rest;
    }
    public ImList<E> cons(E elt) { return new Cons<>(elt, this); }
    public E first() { return elt; }
    public ImList<E> rest() { return rest; }

    public int size() {
        if (size == 0) size = 1 + rest.size();
        return size;
    }
}
```

【错误代码的问题】 1. `size` 是 `public` 的可变字段，客户端可以写 `cons.size = -5;` 或直接读到一个尚未计算的 `0`，于是「`size()` 返回列表长度」这一抽象承诺被彻底破坏——典型的表现暴露。 2. 缓存的使用约定（`0` 表示「尚未计算」、`size` 要么为 0 要么等于 `1 + rest.size()`）只存在于作者脑中，RI 未文档化，后续维护者（哪怕是同一个人几个月后）无法判断哪些操作会更新它、哪些不变式必须保持。 3. 若有人把 `Cons` 的 `size` 传给外部代码或做序列化，就必须连同「缓存是否为 0」的内部状态一起解释，抽象与表示的边界变得模糊。

✅ *正确代码*

```
public class Cons<E> implements ImList<E> {
    private final E elt;
    private final ImList<E> rest;

    private int size = 0;

    // Abstraction function:
    //   AF(elt, rest) = the list whose first element is elt,
    //                   followed by all the elements of rest
    // Representation invariant:
    //   elt != null, rest != null, size >= 0
    //   size > 0 implies size == 1 + rest.size()
    //   (size == 0 means "not yet computed"; a Cons is never empty)
    // Safety from rep exposure:
    //   all fields are private; elt and rest are final and immutable,
    //   and size is a primitive that is never exposed

    public Cons(E elt, ImList<E> rest) {
        this.elt = elt;
        this.rest = rest;
    }
    public ImList<E> cons(E elt) { return new Cons<>(elt, this); }
    public E first() { return elt; }
    public ImList<E> rest() { return rest; }

    public int size() {
        // 受益人式修改（beneficent mutation）：
        // 写 size 不改变本对象所表示的抽象值，因此类型仍然是不可变的
        if (size == 0) {
            size = 1 + rest.size();
        }
        return size;
    }
}
```

【为什么这样更好】 缓存被彻底私有化：客户端只能通过 `size()` 观察长度，无法破坏它；RI 明确写出了字段间的约束与哨兵值 0 的含义，任何人读到这段注释都能推理实现的正确性。缓存把重复查询从 O(n) 降到 O(1)（只在首次计算时付出 O(n)），而**抽象值完全没变**——这正是 `beneficent mutation`（受益人式修改）的定义：不改变对象所表示抽象值的状态变化，因此该类型仍然是不可变的。

【代码对比解说】 两种写法的算法完全一样，区别全在**封装与文档**上。课程原文特意点出这个例子的趣味之处：「this is an immutable datatype, and yet it has a mutable rep」——不可变类型的内部可以有可变状态，前提是变化对抽象值不可见、且不影响其他共享者。因此判断标准不是「字段是否 `final`」，而是「任何观察者能否区分变化前后」。要注意 sp22 的 TypeScript 版本用 `number|undefined` 表示「尚未计算」，而 sp21 的 Java 版本用 `0` 这个哨兵值，因为 `Cons` 的长度永远大于 0；如果把同样的技巧用到 `Empty` 上就会出错（空列表的长度正是 0）。**补充说明**：这个技巧在 Java 内存模型下不是无锁安全的——并发调用 `size()` 构成数据竞争；这里之所以在实践中无害，是因为两个线程计算出的值必然相同且 `int` 写入是原子的。若确实要多线程共享，应按 Reading 23 的方式做同步，或干脆在读多写少的场景使用 `volatile` / `AtomicInteger` 并写清规格。

【设计原则透视】 这组对比把 AF/RI 的价值展示得最充分：AF 说明「这个对象代表哪个抽象列表」，因此只要 `size` 不影响 AF，改它就是受益人式修改；RI 说明「哪些 rep 是合法的」，因此新增字段必须同步新增约束条款。它也体现了 Reading 11 中「先写 AF/RI 再写实现」的配方价值：如果先写下 RI，`size == 0` 与「Cons 非空」的兼容性、以及缓存与 `rest` 的关系会立刻暴露出来，而不会成为一个隐藏的坑。

与其他设计原则的关联

本讲站在多条前置线索的交汇处。**Reading 14（递归）**提供了思维方式：递归数据类型与递归函数一样需要 base case（`Empty`）与 recursive step（`Cons`），「用归约步骤心算递归」的技巧直接来自那里。**Reading 10（抽象数据类型）**确立了「抽象类型 + 具体表示」的框架，而本讲第一次让**两个具体类共同实现一个抽象类型**，并强调这与 `ArrayList` / `LinkedList` 同实现 `List` 的情形有本质区别。**Reading 11（抽象函数与表示不变量）**是本讲的直接基础：AF/RI 必须按 variant 分别书写，表示独立性与表示暴露的讨论（隐藏构造器、`rest()` 返回不可变内部列表为何安全）都从这里来。

Reading 08（不可变性）解释了为什么 `ImList` 可以放心共享结构，也引出受益人式修改这一微妙话题；**Reading 12（接口、泛型与枚举）**提供了 `ImList<E>` 的泛型语法、接口静态方法 `empty()`，以及「用一组类表达一组 variant」这一手法的语言支持。**Reading 06/07（规格说明与设计规格）**规定了「规格不得谈论 rep」这条铁律，因此 `isEmpty` 不能写成「是否是 `Empty` 的实例」；`first` / `rest` 的「requires nonempty」前置条件与 `UnsupportedOperationException` / `IndexOutOfBoundsException` 的选择也属于规格设计。**Reading 15（相等性）**在实现 `contains`、`equalValue` 时立刻派上用场：判等要用 `equals`，而为递归类型定义 `equalValue` 时存在「用观察者操作判等」与「检查运行期类型」的取舍。

向后看，**Reading 18（正则表达式与文法）**与**Reading 19（解析器）**把文法产生的解析树建模为递归数据类型（`Formula`、语法树节点都是 variant 并集），本讲的「每个 variant 一个 case」正是遍历语法树的标准做法；**Reading 26/27（小语言）**会在更大规模上重复这一模式。**Reading 21（并发）**使用本讲的结论：不可变数据 + 纯函数天然可并行，回溯搜索因此可以直接并行化；**Reading 23（互斥）**则解释了为什么受益人式修改的缓存需要额外考虑。**Reading 16（Map、Filter、Reduce）**与 `ImList` 关系密切：对列表的 `size` / `contains` / `append` 的递归定义本质上就是 fold，理解「幺元 + 结合律」有助于看清 base case 的作用。最后，**Reading 09/13（避免调试与调试）**所倡导的 fail fast，正是场景 1 中「抛异常而非返回 `null`」的依据。

关键要点

- **先写数据类型定义，再写代码**：在接口里用注释写下 `ImList<E> = Empty + Cons(elt:E, rest:ImList<E>)`，然后让接口、variant 类与操作一一对应；数学定义到实现几乎是机械翻译。
- **操作声明在接口、实现按 variant 分派，且绝不用 `instanceof` 窥探 variant**：`size` / `isEmpty` / `contains` / `get` / `append` / `last` 都应在 `ImList` 上声明，在 `Empty` 与 `Cons` 中分别实现，客户端永远只看见抽象类型；需要新能力时给类型**增加抽象操作**，而不是检查 `rest` 的具体表示。
- **规格与 AF/RI 都不得提及 variant**：`isEmpty` 的规格是「不含元素」，AF 是「第一个元素是 elt，其余是 rest 所代表的列表」，RI 是「字段非空」（加缓存时补上缓存条款）。
- **拒绝 `null`，使用哨兵对象**：`Empty` 让空列表也能接收方法调用，避免遍地判空；`first` / `rest` 在空列表上必须快速失败而不是返回默认值。

- **共享依赖不可变**：`cons` / `first` / `rest` 是 O(1)，`append` 是 O(this 的长度) 且完整共享右列表，`reverse` 的递归定义是 $O(n^2)$；内部缓存属于受益人式修改，必须写进 RI。

常见陷阱与注意事项

- **在客户端直接 `new Empty<>()` 或 `new Cons<>(...)`** → 表示独立性被破坏，客户端与具体 variant 耦合，日后替换实现会连带破坏所有调用点；应当只通过 `ImList.empty()` 与 `cons()` 构造，并把两个类设为包私有。
- **用 `null` 表示空列表或空尾部** → 每一处使用都要判空、极易漏写导致 `NullPointerException`，并且无法在空列表上调用方法；应当使用 `Empty` 哨兵对象。
- **用 `instanceof` 判断 variant 来分支** → 运行期类型检查比静态检查更不安全、更难适应变化；新增 variant 时静默失效。正确做法是增加抽象操作（如 `size`、`get`）或把分支下放到各 variant 的 `accept` 式方法里。
- **把 `last()`、`reverse()` 之类只写在某一个 variant 上** → 客户端拿到 `ImList` 类型时无法调用，被迫向下转型并再次引入运行期类型检查；所有公共操作都必须在接口上声明（在不适用的 variant 上抛异常即可）。
- **AF/RI 写得含糊或写错** → 例如 `AF(elt, rest) = a two-element list where the second element is rest`（把列表当元素）、`AF() = an empty list` 写成 `AF(first) = ...`（不存在的字段）；这类文档错误会让后续维护者做出错误假设，进而写出真正违反不变量的代码。
- **给缓存加字段却不更新 RI、或把缓存字段设为 `public`** → 缓存与 `rest` 的一致性无人保证，客户端可任意破坏抽象值；必须写成 `private`、在 RI 中写明 `size > 0 implies size == 1 + rest.size()`，并说明 `0` 的哨兵含义。反过来也别误以为「不可变类型的所有字段都必须 `final`」：受益人式修改允许不改变抽象值的字段变化，但在并发场景下这种缓存仍可能构成数据竞争（补充说明：需要共享时按 Reading 23 加同步）。

思考题（带答案）

问题 1：为下面这段 `ImList` 的片段写出合适的抽象函数与表示不变量，并说明 `isEmpty` 的实现为什么看起来「像」`instanceof Empty` 却并不等价。

```
public class Cons<E> implements ImList<E> {
    private final E elt;
    private final ImList<E> rest;
    public Cons(E elt, ImList<E> rest) { this.elt = elt; this.rest = rest; }
    public boolean isEmpty() { return false; }
    // ...
}
```

答案：抽象函数与表示不变量应写为：

```
// Abstraction function:
//   AF(elt, rest) = the list whose first element is elt,
//                   followed by all the elements of rest
// Representation invariant:
//   elt != null, rest != null
// Safety from rep exposure:
//   all fields are private and final; E and ImList<E> are immutable,
//   so rest() returning the internal list cannot threaten the invariant
```

注意 `AF` 不能写成「`AF(elt, rest) = a non-empty list`」（信息不足，没说明谁在前），也不能写成「`AF(elt, rest) = 两元素列表，第二个元素是 rest`」（错把列表当元素）。`RI` 至少要包含两个字段的非空约束——它们是「AF 是良定义的」的前提。

`isEmpty` 与 `instanceof Empty` 的区别在于**层次**：`isEmpty` 是**抽象操作**，其规格是「当且仅当本列表不含任何元素时返回 true」；`instanceof Empty` 是**表示层的判断**，它只对「用 Empty/Cons 表示列表」这一种实现成立。二者的外延在当前的实现下恰好一致，但这只是巧合的副产品：如果我们把 `ImList` 换成数组支撑的实现、或者引入一个表示「共享后缀的若干元素」的新 variant，`isEmpty` 的规格仍然成立，`instanceof Empty` 的代码却会失效。因此规格绝不能提及 variant——**具体 variant 就是 rep**。`Cons.isEmpty()` 返回 `false` 只是「某个 variant 对抽象操作的具体实现」，它与抽象操作的定义不是一回事。

问题 2：`size()` 的朴素实现在长度为 n 的列表上是 O(n)，加上缓存后首次仍是 O(n)、之后是 O(1)。请说明为什么这个缓存不违反不可变性，以及要让这段代码在多个线程中共享还需要做什么。

答案：缓存不违反不可变性，因为它是**受益人式修改（beneficent mutation）**：状态变化（把 `size` 从 0 改为 `1 + rest.size()`）**不改变对象所表示的抽象值**——`AF(elt, rest)` 只与 `elt` 和 `rest` 有关，`size` 完全不在 AF 中出现；因此任何客户端通过抽象操作观察到的行为，在修改前后完全一致。这也是「不可变类型可以有可变 rep」的标准含义（课程原文：this is an immutable datatype, and yet it has a mutable rep）。前提是缓存字段必须 `private`（否则客户端可以直接改它，抽象值就被破坏了），并且必须在 RI 中写下它与其他字段的关系：`size >= 0`，`size > 0 implies size == 1 + rest.size()`，以及 `size == 0` 表示「尚未计算」（这是因为 `Cons` 永远不空，长度不可能为 0；若把同样的哨兵技巧用在 `Empty` 上就会错，因为空列表的长度正是 0）。

至于线程安全：**补充说明**，上述缓存是普通的非 `volatile` 非同步字段，多线程并发调用 `size()` 按 Java 内存模型构成数据竞争。在这里它「碰巧」无害，因为两个线程计算出的值必然相同，而 `int` 的写入是原子的，所以最坏情况只是重复计算。但这依赖具体实现细节而非规格保证：如果缓存值可能是引用类型、可能是 64 位 `long`（非 `volatile` 时写入不是原子的），或者计算过程依赖其他可变状态，就必须按 Reading 23（互斥与同步）的方式加锁、使用 `AtomicInteger` / `volatile`，或者干脆放弃缓存——这正是「不可变数据天然线程安全」这一红利的边界：**受益人式修改把一部分安全性让渡给了实现细节**。

问题 3：为什么课程说 `append` 的递归实现是「非破坏性」的，而它为什么无法做到 O(1)？请结合共享结构解释 `append(Empty, list2) = list2` 这一条基例的意义。

答案：`append` 的递归定义是 `append(Empty, list2) = list2` 与 `append(Cons(elt, rest), list2) = cons(elt, append(rest, list2))`。翻译成 Java 时，`Cons` 的每一步都**新建**一个 `Cons`（`return new Cons<>(elt, rest.append(other));`），而 `Empty` 的基例**直接返回 `list2` 本身**。因此左列表的每一个

`Cons` 都被复制成新节点（「复制脊柱」），而右列表 `list2` 的节点一个也没有复制，被完整共享。这就是非破坏性：所有既有列表对象在操作前后完全不变，因此对共享同一子列表的多个引用都安全——这也回应了课程原文对共享的强调（「Only one copy of this sublist exists in memory, and both x and y point to it, but this aliasing is perfectly safe because the list is immutable」）。

无法做到 O(1) 的原因是**表示的限制**：每个 `Cons` 只有一个 `rest` 字段，指向唯一一个后继，而且要把新列表的「第一个元素」暴露给 `first()`。若想让 `append` 变成 O(1)，就必须改写某个既有 `Cons` 的 `rest` 字段（例如把左列表最后一个节点的 `rest` 指向右列表），而那会破坏不可变性，让所有共享该子列表的引用**追溯性地改变含义**——前面场景 4 已经展示过这种 bug 的破坏力。换句话说，O(1) 的 `append` 与「多引用安全共享」在同一表示下不可兼得；需要频繁拼接时应改用其它结构（如数组支撑的 `List`、`ArrayList` 的 `addAll`，或专门的 `Seq`/差分列表），并接受不同的性能画像。

`append(Empty, list2) = list2` 这一基例的意义有两重：其一，它定义了「空列表拼接任何列表」的语义，使 `append` 在左参数递减到 base case 时终止；其二，它是**结构共享的入口**——正因为基例原样返回右列表，`append` 才能在不复制右列表的前提下完成拼接。这与 `size(Empty) = 0` 作为「空乘积/空和的幺元」在 Reading 16 中扮演的角色异曲同工：base case 不只是终止条件，它还定义了运算的单位元语义。

Reading 18: 正则表达式与文法（Regular Expressions & Grammars）

> 说明：本讲 sp22 原版使用 TypeScript，本笔记按用户要求提供 Java 代码示例；类型/API 与 sp21（6.031 Java 版）原文保持一致。

概述

本讲解决的是**如何为「字符序列」写规格**这一问题。很多程序模块以字节序列或字符序列作为输入或输出：存于内存时叫字符串（string），流入流出模块时叫流（stream）；具体形式可能是字符串本身、磁盘文件（此时规格叫文件格式 file format）、网络消息（规格叫线协议 wire protocol）、或用户在控制台输入的命令（规格叫命令行接口 command line interface）。课程给出的工具是**文法（grammar）**：它不仅能区分合法与非法序列，还能把序列**解析（parse）**成程序可操作的数据结构，这个数据结构往往就是 Reading 17 所讲的递归数据类型。文法的一个特殊子类叫**正则表达式（regular expression, regex）**，它除用于规格与解析外，还是各种字符串处理任务（拆分、抽取、替换）的常用工具。

与三大目标的关系如下。**Safe from bugs**：文法与正则表达式是字符串和流的**声明式规格（declarative specification）**，可以被库与工具直接使用；这种规格通常比手写的解析代码更简单、更直接、更不容易出错。**Easy to understand**：文法把序列的形状以比手写解析代码更容易理解的形式固定下来；但正则表达式往往**不**易理解，因为它把本可读的正则文法压缩成了一行。**Ready for change**：文法很容易修改，正则表达式则难得多，因为复杂的正则表达式晦涩难懂——这也是本讲反复强调「能用带名字的非终结符就用文法」的原因。

核心概念与设计原则详解

文法、产生式、终结符与非终结符（Grammars, Productions, Terminals, Nonterminals）

- **定义与目的**：**文法（grammar）**是描述字符串集合的紧凑表示：它定义一组字符串，用来判断某个序列是否属于该集合，并为解析提供结构依据。
- **直观解释（"它是什么？"）**：把文法想成一份「填词游戏」的规则表：一部分词是固定不能变的（终结符），另一部分是可以继续展开的（非终结符），最终展开到只剩固定词时，就得到了一条合法字符串。
- **关键规则与最佳实践**：
- **终结符（terminals）**是文法中的字面字符串，之所以叫终结符是因为它们不能再被展开；书写时通常加引号，如 `'http'` 或 `':'`。
- 文法由一组**产生式（productions）**描述，每条产生式定义一个**非终结符（nonterminal）**。可以这样类比：非终结符像一个代表某组字符串的**变量**，产生式则是这个变量用其他变量（非终结符）、运算符与常量（终结符）写出的定义；在表示字符串的树中，非终结符是内部节点。
- 产生式的形式是 `非终结符 ::= 由终结符、非终结符与运算符构成的表达式`。
- 文法中要指定一个非终结符作为**根（root，也叫 start，甚至直接叫 S）**；文法识别的字符串集合正是**匹配根非终结符**的那些字符串。课程建议给根取可读的名字，如 `url`、`html`、`markdown`。
- 单例文法可以只有一条产生式，右侧全是终结符：`url ::= 'http://mit.edu/'` 只识别这一个字符串。
- 用法示例：`url ::= 'http://' hostname '/'` 与 `hostname ::= 'mit.edu' | 'stanford.edu' | 'google.com'` 合起来正好表示三个字符串 `http://mit.edu/`、`http://google.com/`、`http://stanford.edu/`。

三个核心运算符：重复、连接、选择（Repetition, Concatenation, Union）

- **定义与目的**：产生式右侧用运算符把终结符与非终结符组合起来。最重要的三个运算符是重复 `*`、连接（不用符号，仅用空格）与选择 `|`，它们足以表达任意正则语言。
- **直观解释（"它是什么？"）**：`*` 是「任意多份」，连接是「一份接一份」，`|` 是「二选一」。三者组合起来就像用最少的积木搭出所有形状。
- **关键规则与最佳实践**：
- 重复：`x ::= y*`，`x` 匹配零个或多个 `y`。
- 连接：`x ::= y z`，`x` 匹配「`y` 后接 `z`」。
- 选择（也叫 alternation，交替）：`x ::= y | z`，`x` 匹配 `y` 或 `z`。
- **优先级约定**：后缀运算符（如 `*`）优先级最高、最先应用；连接次之；选择 `|` 优先级最低、最后应用。用括号可以覆盖优先级：`m ::= a (b|c) d` 匹配「a，然后 b 或 c，然后 d」；`x ::= (y z | a b)*` 匹配零个或多个「yz 或 ab」对。
- 用 `*` 要当心它允许**零次**：`word ::= letter*` 会让整个文法也能匹配 `http://./` 这样并不合法的 URL；让单词至少一个字母的笨办法是 `word ::= letter letter*`。

更多文法运算符：语法糖（More Grammar Operators）

- **定义与目的**：除三个核心运算符外，还有一批**语法糖（syntactic sugar）**——它们都等价于核心运算符的组合，只是写法更紧凑，用于精确表达「出现次数」与「字符集合」。
- **直观解释（"它是什么？"）**：就像 `*` 的亲戚：`?` 是「零或一次」，`+` 是「一次或多次」，`{n,m}` 是「区间次数」，而 `[...]` 是「一个字符的候选清单」。
- **关键规则与最佳实践**：
- `x ::= y?` 表示零或一次，等价于 `x ::= | y`（注意这里有一个空串分支）。
- `x ::= y+` 表示一次或多次，等价于 `x ::= y y*`。
- `x ::= y{3}` 等价于 `x ::= y y y`；`x ::= y{1,3}` 等价于 `x ::= y | y y | y y y`；`x ::= y{,4}` 等价于 `x ::= | y | y y | y y y | y y y y`（含空串）；`x ::= y{2,}` 等价于 `x ::= y y y*`。
- **字符类（character class）** `x ::= [aeiou]` 等价于 `x ::= 'a'|'e'|'i'|'o'|'u'`；用 `-` 可以写字符**范围**：`[a-ckx-z]` 等价于 `'a'|'b'|'c'|'k'|'x'|'y'|'z'`；**反向字符类（inverted character class）** `[^a-c]` 匹配不在括号中列出的单个字符（即 `'d'|'e'|...|'!'|'@'|...` 等所有其他字符）。
- 有了这些运算符，`word` 可以写得既紧凑又精确：`word ::= [a-z]+`。

文法中的递归（Recursion in Grammars）

- **定义与目的**：要表达「主机名可以有多段」「可以带可选端口号」这类结构（例如 `http://didit.csail.mit.edu:4949/`），产生式右侧需要**递归地引用自己**。递归使文法能表达无界的嵌套与重复结构。
- **直观解释（"它是什么？"）**：就像一棵树的画法——「主机名 = 单词 + '.' + 主机名」，一直向右展开，直到用 base case「单词 + '.' + 单词」收尾。
- **关键规则与最佳实践**：
- 递归写法：`hostname ::= word '.' hostname | word '.' word`；其中 `word '.' word` 是 **base case**，`word '.' hostname` 是 **递归步骤**。它允许任意多段（≥2 段）的主机名。
- 用重复运算符可以**消去**这种递归：`hostname ::= (word '.')+ word`，两者识别的语言相同，但后者没有递归。文法中的递归**有时**能被运算符消掉，但**并非总能**（HTML 的嵌套标签就不能）。
- 完整 URL 文法：`url ::= 'http://' hostname (':' port)? '/'`、`hostname ::= word '.' hostname | word '.' word`、`port ::= [0-9]+`、`word ::= [a-z]+`。
- 文法通常**不**表达数值范围约束：`port` 允许任何数字串，而「端口必须在 0 到 65535（$2^{16}-1$）之间」这样的约束应当在**使用该文法的程序里**检查，而不是写进文法。
- 想继续推广还可以：支持更多协议（`https`、`ftp`，写法如 `protocol ::= ('http' 's'?) | 'ftp'`）、把末尾的 `/` 推广成斜杠分隔的路径、允许主机名使用完整的合法字符集而不只是 `a-z`。

解析树（Parse Trees）

- **定义与目的**：把文法与字符串匹配的过程记录下来，就得到**解析树**：它展示字符串的哪些部分对应文法中的哪些部分，是把线性字符串转成结构化数据（递归数据类型）的桥梁。

- **直观解释（"它是什么？"）**：解析树像句子的语法分析图：叶子是实际写出来的字，内部节点是「这些字一起扮演了什么角色」。
- **关键规则与最佳实践**：
- 解析树的**叶子**标注终结符，代表被解析出的字符串片段；它们没有子节点、不能再展开；把所有叶子按顺序拼接起来就能还原原字符串。
- 解析树的**内部节点**标注非终结符；某个非终结符节点的直接子节点必须符合该非终结符产生式的模式。例如主机名节点的子节点必须符合 `word '.' word`。
- 递归文法会生成**更深的树**：递归版 `hostname ::= word '.' hostname | word '.' word` 会为每个 `hostname` 生成一个内部节点，而非递归版 `hostname ::= (word '.')+ word` 只生成一个 `hostname` 节点（其下是若干 `word` 节点）。同一个字符串在两个文法下会得到不同形状的解析树，节点数也不同。
- 解析树的结构直接对应 Reading 17 的递归数据类型，因此「每个 variant 一个 case」的实现方式可以天然地遍历它。

正则文法与正则表达式（Regular Grammars & Regular Expressions）

- **定义与目的**：**正则文法**有一种特殊性质：把除根之外的每个非终结符都用其右侧内容替换掉，就能把它化简成**只有根的一条产生式**，右侧只剩终结符与运算符。这种化简后的紧凑写法就叫**正则表达式**。
- **直观解释（"它是什么？"）**：文法像「带中间变量的多步算式」，正则表达式像「把所有中间变量代入后的最终一行」——更短，但也更难读，因为中间变量的名字（那些说明了每部分含义的非终结符）全都不见了。
- **关键规则与最佳实践**：
- 化简示例：URL 文法可以化简为 `url ::= 'http://' ([a-z]+ '.')+ [a-z]+ (':' [0-9]+)? '/'`；Markdown 文法可以化简为 `markdown ::= ([^_]* | '_' [^_]* '_' )*`。
- 正则表达式**去掉终结符的引号、去掉终结符与运算符之间的空格**，只剩下终结符字符、用于分组的括号与运算符字符：Markdown 的正则表达式就是 `([^_]*|_[^_]*_)*`。
- 正则表达式远不如原文法可读，因为它缺少了说明各子表达式含义的非终结符名字；但**许多编程语言只有正则库而没有文法库**，而且正则匹配比文法匹配快得多。
- 常见的额外元字符：`.` 匹配任意单个字符（视库而定可能不含换行）；`\d` 等价于 `[0-9]`；`\s` 匹配空白字符（空格、制表、换行）；`\w` 匹配单词字符（含下划线），等价于 `[a-zA-Z_0-9]`。
- 反斜杠用于「转义」运算符或特殊字符使其按字面匹配：常见的需要转义的字符有 `\. \( \) \* \+ \| \[ \] \\`。例如 URL 正则里的 `.` 是终结符，必须写成 `\.`：`http://([a-z]+\.)+[a-z]+(:[0-9]+)?/`。
- 另一种转义方式是把特殊字符放进字符类括号：用 `[.]` 也能匹配字面点号。在字符类**内部**，大多数特殊字符失去特殊含义而按字面处理；但字符类语法自身的特殊字符 `[`、`]`、`^`、`-`、`\` 仍需反斜杠转义。

在实践中使用正则表达式（Regular Expressions in Practice）

- **定义与目的**：正则是日常编程的必备工具；在 Java 中，字符串操作可以使用 `String.split`、`String.matches`、`String.replaceAll`，更需要控制力时使用 `java.util.regex.Pattern` 与 `java.util.regex.Matcher`。
- **直观解释（"它是什么？"）**：`Pattern` 是「编译好的规格」，`Matcher` 是「拿着这份规格在某个具体字符串上走的匹配器」——前者可反复使用，后者记录匹配进度与捕获结果。
- **关键规则与最佳实践**：
- 把连续空格替换为单个空格：`String singleSpacedString = s.replaceAll(" +", " ");`（sp22 的 TypeScript 版本写作 `s.replace(/ +/g, " ")`，其中 `g` 表示全局匹配；Java 的 `replaceAll` 本身就是全局替换）。
- 匹配 URL：`if (s.matches("http://([a-z]+\\.)+[a-z]+(:[0-9]+)?/")) { ... }`。注意这里出现了**双重反斜杠**：先写 `\.` 让正则把点号当字面量，再把反斜杠写成 `\\` 以躲过 Java 字符串的转义——「频繁需要双反斜杠转义让正则更加难读」是课程的原话。
- 抽取日期 `"2020-03-18"` 的各个部分：用 `Pattern.compile("(?<year>\\d{4})-(?<month>\\d{2})-(?<day>\\d{2})")`，再用 `Matcher` 的 `matches()` 与 `group("year")` 取值。`(?<name>...)` 是**命名捕获组（named capturing group）**：它匹配括号内的正则，并把匹配到的字符串赋给名字 `name`。注意这里的 `?` **不是**「零或一次」的意思——紧跟在左括号之后，它表示这组括号有特殊含义，而不仅仅是分组。
- `Matcher.group(name)` 在匹配成功后返回对应片段：把上面的正则匹配到 `"2025-03-18"` 上，`group("year")` 得到 `"2025"`、`group("month")` 得到 `"03"`、`group("day")` 得到 `"18"`。
- 匹配范围要明确：`Matcher.matches()` 要求**整串**与正则匹配，而 `Matcher.find()` 只要求在串中**找到**一个匹配子串；把二者混用是最常见的语义 bug 之一。
- 字符串解析示例：`Pattern.compile("[0-9]+ .* (Rd|St|Ave|Ln)")` 可以匹配 `"77 Rose Court Ln"`，把各部分改写成命名捕获组 `(?<houseNumber>[0-9]+) (?<streetName>.*) (?<streetType>Rd|St|Ave|Ln)` 就能解析出 `"77"`、`"Rose Court"`、`"Ln"` 三段。要注意**空格在正则中是有含义的**，不能随意增删。
- **补充说明**：`Pattern` 是不可变且线程安全的，可以安全地声明为 `static final` 常量并复用；`Matcher` 保存匹配状态、**不是**线程安全的，应当每次匹配时新建。频繁使用 `String.matches` / `String.split` 会在每次调用时重新编译正则，带来不必要的开销。

上下文无关文法与正则表达式的表达能力对比（Context-Free Grammars）

- **定义与目的**：用本讲的这套文法系统能表达的语言统称为**上下文无关（context-free）**语言。**并非所有上下文无关语言都是正则的**：有些文法无法化简为「单条非递归产生式」。
- **直观解释（"它是什么？"）**：正则表达式像一台「没有记忆」的机器，它只能数「有多少个」，不能数「配对了几层」；而嵌套结构要求「记住已经打开了几个括号」，这就需要递归（也就是上下文无关文法）。
- **关键规则与最佳实践**：
- HTML（简化版）文法**不是**正则的：`html ::= ( normal | italic )*`、`italic ::= '<i>' html '</i>'`、`normal ::= text`、`text ::= [^<>]*`。替换非终结符后得到 `html ::= ( [^<>]* | '<i>'`

`html '</i>' )*`，右侧对 `html` 的递归引用**无法消除**，也无法简单换成重复运算符。

- 对比 Markdown：`italic ::= '_' normal '_'` 中的 `normal` 不会递归回到 `markdown`，因此**可以**化简为正则 `([^_]*|_[^_]*_)*`——也就是说，**同一个「斜体」概念，Markdown 版本可有正则表达，HTML 版本不能**，差别只在定界符之间匹配的是哪个非终结符。
- 一般规律：**任何具有嵌套结构的语言（嵌套括号、嵌套花括号、成对标签）都是上下文无关但非正则的。**
- 大多数编程语言的文法都是上下文无关的。课程给出的 Java `statement` 产生式片段就包含 `'{' statement* '}'`、`'if' '(' expression ')' statement ('else' statement)?`、`'while' '(' expression ')' statement`、`'synchronized' '(' expression ')' '{' statement* '}'`、`'try' ...` 等分支——它们都用「每个 variant 一行」的方式写出了语句的全部形状（sp22 的 TypeScript 版本对应地列出 `'{' statement* '}'`、`'if' ...`、`'for' ...`、`'switch' ...` 等）。
- 决策建议：**需要嵌套结构 → 用文法（配解析器，见 Reading 19）；只需要扁平的字符模式匹配/抽取/替换 → 用正则表达式。**

代码示例与对比分析

场景 1：校验 URL——手写字符扫描 vs 声明式正则规格

❌ *错误代码*

```
public class Urls {
    /**
     * @param s a string
     * @return true iff s is a legal http URL
     */
    public static boolean isHttpUrl(String s) {
        // 错误：用 startsWith/indexOf/substring 手写「解析」，规格散落在控制流里
        if (!s.startsWith("http://")) {
            return false;
        }
        int slash = s.indexOf('/', 7);
        if (slash < 0) {
            return false;
        }
        String host = s.substring(7, slash);
        return host.length() > 0;   // 只要主机名非空就算合法?
    }
}
```

【错误代码的问题】 1. 检查远弱于规格：`"http://this is not a host/"`、`"http://??/"`、`"http://A_B/"` 都会被判为合法，因为它只检查了前缀、斜杠位置与主机名非空；主机名字符集、端口号格式完全没有校验。 2. 逻辑无法复用：这段代码只能回答「是/否」，无法把主机名、端口、路径**抽取**出来；需要解析时只能再写一遍（于是两份实现必然漂移）。 3. 规格与实现混在一起：真正的规则（主机名由若干 `[a-z]+` 段用 `.` 连接、端口是可选的数字串、末尾是 `/`）只存在于 if 语句的排列中，读代码的人必须自己反推规则，修改规则时也没有单一改动点。

✅ *正确代码*

```
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class Urls {
    // 把文法/正则作为一条可读的、可复用的规格常量：
    //   url      ::= protocol '://' hostname (':' port)? '/'
    //   protocol ::= ('http' 's'?) | 'ftp'
    //   hostname ::= ([a-z]+ '.')+ [a-z]+
    //   port     ::= [0-9]+
    private static final Pattern URL = Pattern.compile(
        "(?<protocol>(?:https?|ftp)):" +
        "(?<host>(?:[a-z]+\\.)+[a-z]+)" +
        "(?::(?<port>[0-9]+))?/");

    /**
     * @param s a string
     * @return true iff the whole string matches the URL grammar above
     */
    public static boolean isHttpUrl(String s) {
        return URL.matcher(s).matches();     // matches() 要求整串匹配
    }

    /**
     * @param s a string
     * @return the host part of s, or empty if s is not a URL of that form
     */
    public static java.util.Optional<String> hostOf(String s) {
        Matcher m = URL.matcher(s);
        return m.matches() ? java.util.Optional.of(m.group("host"))
                           : java.util.Optional.empty();
    }
}
```

【为什么这样更好】 校验与抽取共用同一份规格：`Pattern` 是编译好的声明式规格，`matches()` 回答「是否合法」，`group("host")` 回答「其中主机名是什么」，两者不可能不一致。规则以命名捕获组的形式写在正则里，与课程文法逐条对应，读代码时能直接对照 `url ::= protocol '://' hostname (':' port)? '/'`；要放宽或收紧规则（比如允许 `ftp`、允许大写字母）只需改一处常量。

【代码对比解说】 手写扫描的代码是**命令式**的：它描述「先看前缀、再找斜杠、再取子串」，读者必须执行一遍才能知道规则是什么；正则版本是**声明式**的：它直接陈述「合法 URL 长什么样」。此外，手写版本的失败模式是静默的过宽匹配（把非法串判成合法），而正则版本的失败模式通常是明确的（不匹配），配合单元测试很容易覆盖。注意两组 `(?:...)` 是**非捕获组**（补充说明：Java 正则的语法扩展），用它可以避免为不想抽取的部分创建多余的捕获组；命名捕获组 `(?<name>...)` 中的 `?` 不是量词，而是「这组括号有特殊含义」的标记。

【设计原则透视】 文法/正则在这里扮演的是 Reading 06/07 中**方法规格**的角色：`isHttpUrl` 的规格可以写成「当且仅当 `s` 匹配上述文法时返回 true」，规格与实现分离且可独立评审。从 Reading 17 的角度看，课程文法图中的 `url`、`hostname`、`port`、`word` 节点正是解析树的非终结符节点，而解析树的递归结构可以直接实现为一个递归数据类型；与之对应的 Java 代码则是「每个 variant 一个 case」的操作集。把

`Pattern` 声明为 `static final` 并用 `matches()` 判定整串，还体现了「让失败可预测」的调试友好性（Reading 09）。

场景 2：忘记转义 `.`——正则比规格更宽松

❌ *错误代码*

```
import java.util.regex.Pattern;

public class Hosts {
    // 错误：本意是「若干由点号分隔的小写单词」，但 . 没有转义，
    // 它变成了「任意单个字符」的元字符
    private static final Pattern BAD =
        Pattern.compile("http://([a-z]+.)+[a-z]+");

    public static boolean looksLikeUrl(String s) {
        return BAD.matcher(s).matches();
    }
}
```

【错误代码的问题】 1. 语义被悄悄放宽：`[a-z]+.` 的含义是「一个或多个小写字母后跟**任意一个字符**」，因此 `"http://abcXdefYghi"`、`"http://aaa$bbb"` 都会被判为合法——规格说「点号分隔」，实现却接受任何分隔符。 2. 这类错误**不会**被编译器或类型检查发现，测试若只覆盖「正常 URL」就永远看不到差异；而一旦下游据此放行数据，注入类风险随之而来。 3. 因为 `.` 也在 `[a-z]+` 之外，它还可能吞掉本应属于后续部分（如端口、斜杠）的字符，使整串匹配（`matches()`）的结果与预期完全不符，排错时会先怀疑「正则库有问题」。

✅ *正确代码*

```
import java.util.regex.Pattern;

public class Hosts {
    // 正确：\. 让点号按字面匹配；Java 字符串里写成 "\\."
    private static final Pattern URL = Pattern.compile(
        "http://([a-z]+\\.)+[a-z]+(:[0-9]+)?/");

    public static boolean looksLikeUrl(String s) {
        return URL.matcher(s).matches();
    }
}
```

【为什么这样更好】 用 `\.` 明确告诉正则引擎「这里要匹配一个真正的点号」，于是非法分隔符立即被拒绝。也可以写成 `[.]`：在字符类括号内部，大多数特殊字符失去特殊含义，因此 `[.]` 与 `\.` 等价，而且不必写双反斜杠——这在可读性上略有优势。规格与实现的差距被消除，`matches()` 的判定结果与文法描述严格一致。

【代码对比解说】 这一组说明「正则表达式不是自然语言，每个字符都有含义」：`[a-z]+.` 与 `[a-z]+\.` 只差一个反斜杠，语义却从「小写字母加点号」变成「小写字母加任意字符」。Java 还叠加了**第二层转义**：字符

串字面量里的 `\` 本身要写成 `\\`，于是正则的 `\.` 在源码中必须写成 `"\\."`；这正是课程原文所说的「频繁需要双反斜杠转义让正则更难读」。相比之下，如果这段规格用文法写（`hostname ::= (word '.')+ word`），点号被引号括起来，就完全不存在转义问题——这是文法在**易理解性**上的直接优势。

【设计原则透视】 这组对比是「规格—实现一致性」问题的正则版本：正则表达式本身就是规格，但它是一门**隐晦的规格语言**，一处转义缺失就让规格与作者意图分岔，且没有任何工具会警告。从三大目标看，它伤害的是 Safe from bugs（静默放宽）与 Ready for change（难以审阅、难以修改）；而把 `Pattern` 作为常量集中声明、并在 Javadoc 里写出对应的文法（如 `hostname ::= (word '.')+ word`），可以让规格与实现互相对照，这是「文档即规格」的实践。补充说明：`Pattern` 线程安全，把它放进 `static final` 是安全且高效的；若把 `Matcher` 也做成共享字段，就会引入 Reading 21/23 才讨论的并发问题。

场景 3：文法写得过宽——`letter*` 允许空串导致匹配非法 URL

❌ *错误代码*

```
public class Words {
    // 文法:
    //   url      ::= 'http://' hostname '/'
    //   hostname ::= word '.' word
    //   word     ::= letter*          <- 错误：允许「零个字母」
    // 对应的正则:
    private static final java.util.regex.Pattern URL =
        java.util.regex.Pattern.compile("http://[a-z]*\\.[a-z]*/");

    public static boolean isUrl(String s) {
        return URL.matcher(s).matches();
    }
}
```

【错误代码的问题】 1. `word ::= letter*` 匹配零个或多个字母，因此 `hostname` 可以是两个空单词加一个点号——整个文法会接受 `http://./` 这样**并不合法**的 URL。 2. 这个 bug 完全来自「`*` 允许零次」这一细节，作者的本意是「单词由字母组成」，却无意中允许了「空单词」；在只测正常 URL 的测试下不会暴露。 3. 一旦这个过宽的模式被用于输入校验或路由分发，空主机名会一路传到下游（DNS 解析、HTTP 请求构造），故障点与原因相距很远。

✅ *正确代码*

```
public class Words {
    // 文法：
    //   url      ::= 'http://' hostname '/'
    //   hostname ::= word '.' word
    //   word     ::= [a-z]+          <- 正确：至少一个字母
    //                （冗长但等价的写法：word ::= letter letter*）
    private static final java.util.regex.Pattern URL =
        java.util.regex.Pattern.compile("http://[a-z]+\\.[a-z]+/");

    public static boolean isUrl(String s) {
        return URL.matcher(s).matches();
    }
}
```

【为什么这样更好】 `[a-z]+` 明确要求「一个或多个小写字母」，空单词被排除，`http://./` 不再匹配。可以把它视为对 `*` 与 `+` 之差的显式选择：`*`（零或多次）适合「可以为空」的部分（如可选的路径段），`+`（一次或多次）适合「必须存在的成分」（如协议名、主机名的每一段、单词本身）。选择哪一个，是**规格的一部分**，应当有意识地决定并写进注释。

【代码对比解说】 这是「语法糖不只是糖，它带着语义」的典型例子：`letter*` 与 `letter letter*` 都等价于 `+` 的含义，但作者往往只想着「一个单词」，就顺手写了 `*`。同样的陷阱在 `{n,m}` 家族里也存在：`y{,4}` 的等价形式里**含空串**（至多四个），而 `y{1,3}` 不含空串。写文法时的自检方法是：把运算符替换成它的等价展开形式（`* → 零或多次`、`? → 含空串分支`），然后问「允许空串在这里合理吗」。若规格确实允许空串（例如 Markdown 里两段定界符之间可以为空），那就应当保留 `*` 并在注释中写明理由。

【设计原则透视】 文法是**规格**，因此它的松紧直接决定安全性：过宽 = 接受了规格本不允许的输入（违反前置条件检查的职责），过窄 = 拒绝了合法输入（同样破坏规格）。在 Reading 17 的框架里，文法定义的是解析树这一递归数据类型的合法值集合——若文法允许空单词，那么数据类型的某个 variant 就可能持有空字符串，AF/RI 若不写明（例如「`word` 至少含一个字符」），后续所有基于它的假设都会松动。课程给出的建议也适用于这里：把中间的非终结符保留下来（`word`、`letter` 各有名字）能让这种错误一眼可见，而压成一行正则后，`*` 与 `+` 的差别就淹没在符号里了。

场景 4：用正则解析嵌套标记——正则做不到，必须用文法

❌ *错误代码*

```
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class Italicizer {
    // 错误：企图用正则匹配「配对」的 <i>...</i>
    private static final Pattern ITALIC = Pattern.compile("<i>(.*)</i>");

    /**
     * @param s a string
     * @return the text inside the first italic element
     */
    public static String firstItalic(String s) {
        Matcher m = ITALIC.matcher(s);
        if (!m.find()) {
            throw new IllegalArgumentException("no italic in " + s);
        }
        return m.group(1);
    }
}
```

【错误代码的问题】 1. 贪婪量词 `.*` 会跨过**独立的**多个斜体元素：对 `"a<i>b</i>c<i>d</i>e"`，`find()` 匹配到的是 `<i>b</i>c<i>d</i>`，把两段斜体之间的普通文字 `c` 也吞了进去——结果取决于「串里还有没有另一个 `<i>`」，而不是规格所说的「第一个斜体元素」。 2. 改成非贪婪 `.*?` 也不能解决嵌套：对 `"a<i>b<i>c</i>d</i>e"`，`find()` 得到的是 `<i>b<i>c</i>`，既不是最内层也不是最外层，配对完全错乱。 3. 根本原因不是量词选得不好，而是**正则语言无法表达配对嵌套**：简化 HTML 文法 `html ::= ( normal | italic )*`、`italic ::= '<i>' html '</i>'` 化简后右侧仍含 `html` 自身，无法消除，因此它**不是正则文法**。用正则去解析它，属于用错工具。

✅ *正确代码*

```
import java.util.ArrayList;
import java.util.List;

/**
 * 用递归下降解析简化 HTML 的子集。文法（见注释）与 Reading 17 的递归数据类型对应：
 *   html   ::= ( normal | italic )*
 *   italic ::= '<i>' html '</i>'
 *   normal ::= text
 *   text   ::= [^<>]*
 * 返回结果使用 Reading 17 的不可变列表 ImList<E>：
 *   ImList<E> = Empty + Cons(elt:E, rest:ImList<E>)
 */
public class ItalicParser {
    /**
     * @param input a string
     * @return the contents of every italic element; an enclosing element's content
     *         precedes the contents of elements nested inside it
     * @throws IllegalArgumentException if the tags are not balanced
     */
    public static ImList<String> italicContents(String input) {
        int[] pos = { 0 };                      // 游标：与 pos[0] 共享，供递归调用推进
        ImList<String> result = parseSeq(input, pos, false);
        return result;
    }

    /** 解析 ( normal | italic )*，nested 为 true 时在遇到 "</i>" 处停下 */
    private static ImList<String> parseSeq(String s, int[] pos, boolean nested) {
        ImList<String> out = ImList.empty();
        while (pos[0] < s.length()) {
            if (nested && s.startsWith("</i>", pos[0])) {
                return out;                     // 交给调用者消费 "</i>"
            }
            if (s.startsWith("<i>", pos[0])) {
                pos[0] += 3;                    // 消费 '<i>'
                int innerStart = pos[0];
                ImList<String> inner = parseSeq(s, pos, true);   // 递归解析内部 html
                if (pos[0] >= s.length() || !s.startsWith("</i>", pos[0])) {
                    throw new IllegalArgumentException("unclosed <i> at " + innerStart);
                }
                out = inner.cons(s.substring(innerStart, pos[0]));
                pos[0] += 4;                    // 消费 '</i>'
            } else {
                pos[0]++;                       // 普通字符：属于 text ::= [^<>]*
            }
        }
        if (nested) {
            throw new IllegalArgumentException("missing </i>");
        }
        return out;
    }
}
```

【为什么这样更好】 递归下降解析器与文法**同形**：`parseSeq` 对应 `html ::= ( normal | italic )*`，遇到 `<i>` 就递归调用自己并在返回后消费 `</i>`——配对关系由**调用栈**维护，因此嵌套多少层都能正确处理，而正则的有限状态无法记录「当前还差几个 `</i>`」。对 `"a<i>b<i>c</i>d</i>e"`，它会返回 `["b<i>c</i>d", "c"]`：外层斜体的内容在前、嵌套在其中的内层内容在后，与规格一致；标签不配对时立刻抛出异常（fail fast）。

【代码对比解说】 这组对比的教训是「**选择与语言能力相匹配的形式化工具**」。正则表达式等价于有限状态自动机，注定无法处理需要计数的配对结构；而上下文无关文法（配解析栈）可以。实践中的判据很简单：如果模式里出现「成对定界符」并且它们可以互相嵌套（括号、标签、`begin/end`），就必须用文法；如果只是扁平的字符模式（日期、邮箱、单行日志字段），正则是更轻便的选择。注意 sp22 的 Markdown 文法之所以**可以**用正则，是因为它的 `italic ::= '_' normal '_'` 不允许嵌套（内层只能是 `normal`）——同一个「斜体」概念，两种语法的可表达性因此完全不同。

【设计原则透视】 这段代码同时用到 Reading 17 的两个要点：解析结果用不可变列表 `ImList` 表示（`cons` 在前端追加，O(1)），以及递归数据类型与递归文法一一对应。从 Reading 19（解析器）的角度看，这正是「解析器生成器」要自动完成的事：把文法交给工具，它会生成这样的解析器，并额外处理左递归、优先级等细节。从易理解性看，文法 + 递归下降的代码可以逐行对照文法阅读，而那个贪婪正则的 bug 却需要读者在脑中模拟回溯才能发现——这正是课程所说「正则表达式把本可读的文法压成一行，代价是可读性与可修改性」的具体体现。

场景 5：字符串解析——滥用 `split` 与 `String.matches` vs 预编译 `Pattern` + 命名捕获组

❌ *错误代码*

```
public class Dates {
    /**
     * @param s a date string like "2020-03-18"
     * @return the year part of s
     */
    public static String yearOf(String s) {
        // 错误 1：没有校验格式，任何带 '-' 的串都「成功」
        // 错误 2：split 的参数是正则，每次调用都要重新编译
        // 错误 3：结构不匹配时抛 ArrayIndexOutOfBoundsException，而非规格中的失败语义
        return s.split("-")[0];
    }

    /**
     * @param s a date string
     * @return true iff s has the form YYYY-MM-DD
     */
    public static boolean isDate(String s) {
        // 错误 4：String.matches 每次调用都重新编译正则；且这里的 \d 未写成 \\d 会编译不过
        return s.matches("\\d{4}-\\d{2}-\\d{2}") || true;
    }
}
```

【错误代码的问题】 1. `yearOf` 对 `"3-18"` 返回 `"3"`、对 `"not-a-date"` 返回 `"not"`、对 `"2020-3-8"` 也「成功」，完全不符合「解析日期」的规格；对 `"20200318"` 抛 `ArrayIndexOutOfBoundsException`——用异常表达了一个**预期之中**的失败，把正常的输入校验变成了崩溃。 2. `split` 与 `matches` 的参数都是**正则表达式**，每次调用都会新建并编译 `Pattern`；在高频路径（日志解析、逐行处理）上这是可观的浪费。 3. `isDate` 里那个 `|| true` 使整个表达式恒为真（示例化的严重 bug），说明「把校验写成一行大表达式」时极易出现逻辑错误且难以审阅；此外正则里的反斜杠必须写成 `\\d`，否则 Java 编译器直接报错——这是把正则嵌进 Java 源码的常见摩擦。

✅ *正确代码*

```
import java.util.Optional;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class Dates {
    // 预编译一次，反复使用；Pattern 不可变且线程安全
    private static final Pattern DATE =
        Pattern.compile("(?<year>\\d{4})-(?<month>\\d{2})-(?<day>\\d{2})");

    /**
     * @param s a string
     * @return the year if s has the form YYYY-MM-DD, otherwise Optional.empty()
     */
    public static Optional<String> yearOf(String s) {
        Matcher m = DATE.matcher(s);
        if (!m.matches()) {          // matches() 要求整串匹配：不会漏掉多余的尾巴
            return Optional.empty();
        }
        return Optional.of(m.group("year"));   // 命名捕获组：规格与抽取共用一处定义
    }

    /**
     * @param s a string
     * @return true iff s has the form YYYY-MM-DD
     */
    public static boolean isDate(String s) {
        return DATE.matcher(s).matches();
    }
}
```

【为什么这样更好】 格式规则只写一次（那个 `static final Pattern`），校验与抽取由同一份规格驱动，不可能出现「校验通过但抽取结果不对」的不一致。失败被表达为 `Optional.empty()` 而不是异常，与 Reading 07 关于「用返回值表达可预期的失败、用异常表达违反前置条件」的建议一致。`Matcher.matches()` 的整串语义避免了 `find()` 那种「串里有一处像日期就通过」的漏检。用命名捕获组后，`group("year")` 的语义与文法中的 `year` 字段同名，读代码即可对照。

【代码对比解说】 三种解析路线的对比很清楚：**手工 `split` / `substring`** 最省事但最脆弱（隐含假设、异常语义错位、无复用）；**`String.matches` / `String.split` 内联正则** 稍好（声明式）但每次编译且容易被引号反斜杠搞乱；**预编译 `Pattern` + 命名捕获组** 兼顾声明式、性能与可读性，是生产代码的默认选择。还要

注意 `matches()` 与 `find()` 的区别：前者等价于整串锚定，后者像「搜索」。若确实想用 `find()`（例如在长文本里找第一个日期），就必须自己在正则两端加锚点或检查 `start()/end()`，否则会得到「部分匹配」的结果。补充说明：`Matcher` 持有匹配状态（`group`、`start`、`end` 都依赖最近的匹配），**不是线程安全的**，不要把它放进共享字段；而 `Pattern` 是线程安全的，可以安全复用。

【设计原则透视】 这里体现的是 Reading 06/07 中「规格的可判定性」：一个好的规格应能让实现者明确回答「这个输入是否满足」，并让失败有明确语义。用 `Optional<String>` 作为返回类型把这个语义放进了**类型**里（Reading 12 的泛型与 `Optional` 协作），调用者无法忽略它；用预编译的 `Pattern` 常量则把「规则」从散落的控制流中提炼成一处可评审、可测试的声明式规格。从 Reading 03（测试）的角度看，围绕这条规格应覆盖的等价类是：合法输入、格式正确但数值越界（如 `"2020-13-40"`——文法允许，语义不许，需要在程序里另行检查，正如课程对端口范围的处理）、缺位/多位、含多余尾巴（验证 `matches()` 而非 `find()` 的语义）、空串与 `null`。这正是「文法管形状、程序管约束」这一分工的具体落点。

与其他设计原则的关联

本讲与 **Reading 06（规格说明）**、**Reading 07（设计规格）** 同源：文法与正则表达式就是**序列的规格**，只不过描述对象从「方法的输入输出」换成了「字符序列的形状」；课程所说的「文件格式」「线协议」「命令行接口」全都是规格在不同场景下的别名。**Reading 17（递归数据类型）**是解析的落点：解析树与语法树都是递归数据类型，因此「在接口上声明操作、在每个 variant 上递归实现」的做法可以原样搬到语法树上；本讲场景 4 的解析器就直接返回 `ImList`。**Reading 19（解析器）**紧接着本讲，讨论把文法**自动**翻译成解析器的工具（parser generators），从而省掉手写递归下降的工作。

向前追溯，**Reading 02（Java 基础）**提供的字符串与 API 基础（`String.split`、`matches`、`replaceAll`）是使用正则的前提；**Reading 12（接口、泛型与枚举）**解释了 `Pattern`/`Matcher` 这种「不可变规格 + 有状态会话」的组合为何常见，以及 `Optional` 为何适合表达解析失败。**Reading 16（Map、Filter、Reduce）**中「把项目里所有 Java/TypeScript 文件的单词抽出来」的例子用到 `split(/\W+/)` 与 `filter(s -> s.length() > 0)`，正是本讲正则的实战应用；反过来，本讲也是 Reading 16 的一个注脚——那里只用了正则最简单的一面。**Reading 03（测试）**要求为文法与正则设计覆盖等价类的测试（空串、最长/最短匹配、非法尾随字符）；**Reading 09（避免调试）**与 **Reading 13（调试）**提醒：正则表达式难以调试，写的时候应尽量拆分成带注释的片段或改用带名字的文法。

向后看，**Reading 21（并发）**与 **Reading 23（互斥）**解释了为什么 `Pattern` 可以安全共享而 `Matcher` 不行；**Reading 25（网络）**中的消息格式即「线协议」，其规格写法与本文法完全同构；**Reading 26/27（小语言）**把「文法 + 解析树 + 递归求值」组织成完整的解释器，本讲的 `url`、`html`、`markdown` 文法只是它在小规模上的预演。最后，本讲关于「正则 vs 文法」的取舍判断，本身就是课程反复训练的**设计取舍能力**：同一个字符串集合可以有多种规格，选择的标准是三大目标——安全性、易理解性、可修改性。

关键要点

- **文法 = 一组产生式 + 一个根非终结符**：每条产生式用 `::=` 定义一个非终结符，右侧由终结符（加引号、不能再展开）、非终结符与运算符组成；文法识别的正是匹配根非终结符的那些字符串。
- **三个核心运算符加优先级**：重复 `*`（零或多次）、连接（空格）、选择 `|`；后缀运算符优先级最高、连接次之、`|` 最低，用括号覆盖优先级。`?`、`+`、`{n,m}`、`[...]`、`[^...]` 都是它们的语法糖。

- **`*` 允许空串，`+` 不允许**：`word ::= letter*` 会让 `http://./` 也合法；要「至少一个」就用 `word ::= [a-z]+`（或冗长的 `letter letter*`）。运算符的选择是规格的一部分。
- **正则 = 化简后的文法，有嵌套就必须回到文法**：把非终结符逐个代入直到只剩根，去掉引号与空格，就得到正则表达式——它更快、库支持更广，但缺少说明性的名字，可读性与可修改性都差得多。HTML 的 `italic ::= '<i>' html '</i>'` 无法消去递归（上下文无关但非正则），Markdown 的 `italic ::= '_' normal '_'` 可以化简为正则；因此扁平的字符模式用正则，成对可嵌套的结构用文法 + 解析器。
- **Java 中的实践要点**：`\\` 双层转义、`matches()` 整串匹配 vs `find()` 搜索匹配、命名捕获组 `(?<name>...)`、`Pattern` 预编译且线程安全而 `Matcher` 有状态。

常见陷阱与注意事项

- **忘记转义元字符**：把 `.`、`*`、`+`、`|`、`(`、`)`、`[`、`]`、`\` 当作字面字符使用却不加反斜杠 → 正则悄悄匹配了远多于规格的字符串（如 `[a-z]+.` 接受任意分隔符），且编译器毫无提示；正确做法是 `\.`（或用 `[.]`），并在 Java 中写成 `"\\."`。
- **`*` 与 `+` 用错导致允许空串**：`word ::= letter*`、`y{,4}`（含空串）被用在「必须存在」的位置 → 文法接受了空主机名、空标签等非法输入；写完后把语法糖展开检查一遍空串是否可接受。
- **用正则解析嵌套结构，或把复杂正则当作可维护的规格**：`<i>(.*)</i>` 或 `<i>(.*?)</i>` 处理 HTML/XML/括号嵌套时，贪婪会跨越多个同级元素、非贪婪会让嵌套配对错乱；而一个两百字符、改一处就崩一片的正则本身也是维护灾难 → 正则语言无法表达配对嵌套，应当先写带名字的文法（`url`、`hostname`、`port`、`word`），用解析器处理嵌套，只在模式确实扁平且简单时才把文法机械化简为正则并保留文法注释。
- **混淆 `matches()` 与 `find()`**：想在整串上校验却调用 `find()` → 只要串中某处像日期/URL 就通过校验；反之想在长文本中搜索却用 `matches()` → 永远匹配失败。选择哪一个必须与规格一致。
- **每次调用都重新编译正则**：把正则写在 `String.matches` / `String.split` 的实参里，或在方法体内反复 `Pattern.compile` → 高频路径上性能明显下降；应把 `Pattern` 提为 `static final` 常量（它是线程安全的）。同时注意不要共享 `Matcher`（它有状态、非线程安全）。
- **把数值范围约束写进文法**：试图用文法精确表达 `0 ≤ port ≤ 65535` → 文法急剧膨胀且难以维护；课程的做法是让文法只描述**形状**（`port ::= [0-9]+`），范围检查放到使用该文法的程序里。

思考题（带答案）

问题 1：下面这个文法识别哪些字符串？请判断 `617`、`617-253`、`617-253-1000`、`---`、`integer-integer-integer`、`5--5`、`3-6-293-1` 是否匹配，并说明 `integer` 扮演的角色。

```
root    ::= integer ('-' integer)+
integer ::= [0-9]+
```

答案：`root` 由 `integer`、一个**至少出现一次**的「`'-' integer` 组」构成，因此它识别的是「由两个或更多个非空数字串、用单个连字符连接」的字符串（类似美国电话号码格式）。

- `617`：不匹配。`('-' integer)+` 要求至少一组「连字符 + 数字」，这里一组都没有。
- `617-253`：匹配（恰好一组）。

- `617-253-1000`：匹配（两组）。
- `---`：不匹配。`integer ::= [0-9]+` 要求至少一个数字，连字符本身不是 `integer`。
- `integer-integer-integer`：不匹配。字面量单词 `integer` 不是数字串。
- `5--5`：不匹配。`[0-9]+` 在第一个 `-` 之前只吃到 `5`，随后 `('-' integer)+` 需要「连字符后紧跟数字」，而这里连字符后面又是一个连字符。
- `3-6-293-1`：匹配（三组）。

`integer` 是一个**非终结符**，它把「一个或多个数字」这条子规则命名为 `integer`，于是 `root` 的产生式可以用这个名字而不是重复写 `[0-9]+`。这正是文法优于正则表达式的地方：名字承载了含义，读文法时能看到「整数—连字符—整数」的结构。注意 `[0-9]+` 中的 `+` 是 `[0-9][0-9]*` 的语法糖，它保证每段数字非空——若误写成 `[0-9]*`，`--` 甚至 `-` 都可能被接受（取决于具体写法），这正是场景 3 讨论过的空串陷阱。

问题 2：为什么简化 HTML 文法不是正则的，而简化 Markdown 文法可以化简为正则表达式？请写出两者的化简结果，并说明这对「用正则还是用文法」的实践选择意味着什么。

答案：两个文法的差别只在 `italic` 产生式中「定界符之间匹配哪个非终结符」：

```
markdown ::= ( normal | italic )*      html ::= ( normal | italic )*
italic   ::= '_' normal '_'             italic ::= '<i>' html '</i>'
normal   ::= text                       normal ::= text
text     ::= [^_]*                      text   ::= [^<>]*
```

Markdown 的 `italic` 内部只允许 `normal ::= text ::= [^_]*`，即「不含下划线的任意文本」，它**不会回到** `markdown`，所以替换所有非终结符后可以得到只含终结符与运算符的单一产生式：

```
markdown ::= ([^_]* | '_' [^_]* '_' )*
```

去掉引号与空格就是正则表达式 `([^_]*|_[^_]*_)*`——Markdown 文法是**正则**的（它的斜体不能嵌套，`a_b_c_d_e` 中只有 `b`、`d` 位于 `italic` 内部）。

HTML 的 `italic` 内部是 `html` 自身，替换后得到：

```
html ::= ( [^<>]* | '<i>' html '</i>' )*
```

右侧对 `html` 的**递归引用无法消除**，也无法用重复运算符代替（因为需要记住「打开了几个 `<i>`」才能正确配对），所以 HTML 文法**是上下文无关的但不是正则的**。

实践含义：**能否用正则取决于是否需要「配对/嵌套」的记忆**。扁平模式（日期、URL、日志字段、Markdown 式不嵌套的定界符）用正则，短、快、库支持好；含成对定界符且可嵌套的结构（HTML/XML、括号、`{}` 代码块）必须用文法加解析器（Reading 19），正则在这里无论怎么改都无法正确处理——场景 4 中 `<i>(.*)</i>` 在 `"a<i>b</i>c<i>d</i>e"` 上跨越两个同级元素、在 `"a<i>b<i>c</i>d</i>e"` 上配对错乱，就是这个结论的具体证据。补充一点实践判断：如果一个「正则」已经长到需要写注释才能读懂，那它多半应该先写成文法，再机械化简；而一旦发现化简时卡在递归引用上，就等于发现「这里必须用文法」。

问题 3：下面的 Java 代码想从 `"77 Rose Court Ln"` 这样的地址中取出门牌号、街道名与街道类型。它有什么问题？请给出改进版本，并说明 `matches()` 与 `find()` 的区别在这段代码里为什么重要。

```
public static String streetTypeOf(String s) {
    return s.split(" ")[s.split(" ").length - 1];
}
```

答案：问题有四类。其一，**没有校验**：`split(" ")` 对任何含空格的串都「成功」，`"hello world"` 会被当成地址、`"77-Rose-Court-Ln"` 会直接返回整个串（因为没有空格，数组长度为 1）。其二，**结构假设过强**：它假设街道类型永远是最后一个空格分隔的词，但规格并未如此规定，一旦地址里出现楼层、单元号等后置信息就出错。其三，**语义错位**：`split` 的参数是正则表达式（这里恰好也是字面量空格），每次调用都要编译；而且连写两遍 `s.split(" ")` 意味着两次完整拆分，效率与可读性都差。其四，**失败语义不明**：空串会返回 `""`（`split` 对空串返回长度 1 的数组），而 `null` 会抛 `NullPointerException`，规格里没有任何说明。

改进版本使用预编译的 `Pattern` 与命名捕获组（对应课程中 `Pattern.compile("[0-9]+ .*(Rd|St|Ave|Ln)")` 的加命名组写法）：

```
import java.util.Optional;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class Addresses {
    private static final Pattern ADDRESS = Pattern.compile(
        "(?<houseNumber>[0-9]+) (?<streetName>.*) (?<streetType>Rd|St|Ave|Ln)");

    /**
     * @param s a string
     * @return the street type if s is a street address of the form above
     */
    public static Optional<String> streetTypeOf(String s) {
        Matcher m = ADDRESS.matcher(s);
        if (!m.matches()) {
            return Optional.empty();
        }
        return Optional.of(m.group("streetType"));
    }
}
```

注意正则里的空格是**有含义的**（匹配字面空格），不能随意增删；`(?<name>...)` 中的 `?` 不是「零或一次」的量词，而是「这组括号有特殊含义」的标记。

`matches()` 与 `find()` 的区别在这里很关键：`matches()` 要求**整串**与正则匹配，等价于给整个模式加上了 `^...$` 锚定，因此 `"77 Rose Court Ln"` 通过、`"77 Rose Court Ln Apt 3"` 会失败（尾部多出内容），这正是「这条串是否是这种形式的地址」这一规格的正确语义；而 `find()` 只要求在串中**找到**一个匹配子串，用它来校验就会把 `"见 77 Rose Court Ln。"` 这类串也判为合法地址，即「校验被降级成搜索」。反过来，如果需求真的是「在长文本里找出第一个地址」，那就应当用 `find()`，并通过 `m.start()` / `m.end()`

取得位置——**用哪个方法取决于规格想要「整体判定」还是「搜索定位」**，这也是把正则当作规格时最容易被忽视的一条细节。

Reading 19: 解析器（Parsers）

> 说明：本讲 sp22 原版使用 TypeScript，本笔记按用户要求提供 Java 代码示例；类型/API 与 sp21（6.031 Java 版）原文保持一致。

概述

本讲要解决的问题是：**如何把一串字符（character sequence）可靠地变成一个程序可以直接操作的数据值**。课程给出的答案是一条三步流水线：先用一份**文法（grammar）**声明式地描述「合法的字符序列长什么样」，再让**解析器生成器（parser generator）**把文法自动编译成**解析器（parser）**，解析器把输入匹配成一棵**解析树（parse tree）**；最后我们写一个跟随文法结构递归的函数，把解析树翻译成一个**递归数据类型**——也就是**抽象语法树（abstract syntax tree, AST）**。这条流水线在软件构造中的角色是「把外部世界的非结构化输入变成内部世界的结构化的、有类型的数据」，是所有编译器、配置文件读取器、协议解析器、查询语言解释器的第一道关卡。

它与三大目标的关系是直接的：**Safe from bugs**，文法是一种声明式的规格说明（declarative specification），比手写解析代码更简单、更直接、更不容易出错，而且文法的错误在生成阶段就能被发现；**Easy to understand**，一份紧凑的文法比几十行手工 `substring` / `indexOf` 代码更能说明「这个序列的形状是什么」；**Ready for change**，要支持新语法时，改文法再重新生成解析代码即可，不需要重写整段解析逻辑。本讲也是 Reading 17（递归数据类型）在真实工程里的第一次大规模应用：解析的终点就是递归 ADT。

核心概念与设计原则详解

解析器生成器（Parser Generator） – **定义与目的**：解析器生成器是一个把「文法」当作输入、把「解析器」当作输出的工具；它生成的解析器接受字符序列，并尝试把该序列与文法匹配。它服务的质量目标是安全性（自动生成的匹配逻辑比人手写的状态机更可靠）与可修改性（文法改动可重新生成）。 – **直观解释（"它是什么？"）**：它像一台「语法翻译机」：你交给它一份「什么样的句子算合法」的说明书，它替你造出一个能把句子拆解成语法结构的工人。6.031 使用课程组自研的 **ParserLib**（sp21 为 Java 版），它与工业界广泛使用的 **Antlr** 思想相同，但接口更简单。ParserLib 采用的是自顶向下的**递归下降解析器（recursive descent parser）**。 – **关键规则与最佳实践**： – 把解析器生成器当作工具箱的常备工具：凡是「解析文本」的需求，先想「能不能用文法描述」，而不是先想「怎么写循环」。 – 文法用 `::=` 定义规则，规则以分号结束，规则名即**非终结符（nonterminal）**，习惯全小写。 – **终结符（terminal）**是带引号的字面串（如 `'<i>'`）或正则表达式式的字符类（如 `[^<>]+`）；ParserLib 要求字面字符必须加引号或放进 `[...]`，所以 `(alpha|beta|[c-z])*` 要写成 `('alpha'|'beta'|[c-z])*`。 – 规则中可用选择 `|`、重复 `*` `+` `?`、分组 `(...)`。 – 文法文件里的空白（引号与字符类之外的）不具意义，`html::=(italic|normal)*;` 与带空格写法等价。 – 习惯把**根非终结符（root / starting symbol）**的规则写在最前面，方便人自顶向下阅读；真正决定根的是 `compile` 时传入的那个枚举常量。

根非终结符与文法的不变式（Root Nonterminal） – **定义与目的**：根非终结符就是「整段输入必须匹配的那个非终结符」。它决定了解析的入口，也决定了「什么算解析成功」。它保护的是安全性：一个明确的入口避免了「部分匹配」被误认为成功。 – **直观解释（"它是什么？"）**：把文法想成一张语法地图，根非终结符是你出发的城市；解析器必须从这座城市出发走完全程，走不到终点就是解析失败。 – **关键规则与最佳实践**： – 根通常是语法上最外层、最概括的那个非终结符（如整数表达式文法里的 `expr`、HTML 文法里的 `html`）。 – 在 Java 中，ParserLib 用一个 `enum` 列出文法的全部非终结符，`compile` 时用 `XXXGrammar.EXPR` 指定根；枚举也帮助 ParserLib 检查文法里是否有拼错或漏掉的规则。 – ParserLib 对非终结符名大小写不敏感（内部统一转小写），但 **Java 枚举常量按惯例全大写**，两者要能在心里对上号。 – 枚举中要包含**所有**非终结符（包括只在 `@skip` 里使用的那个，如 `WHITESPACE`），但**不包含**终结符（`'Fall'` 不是枚举值）。

空白处理与 `@skip` 指令（Whitespace and `@skip`） – **定义与目的**：真实输入里到处是空格、制表符和换行，但语法结构通常「不关心」它们。`@skip` 让文法作者声明「某个非终结符的匹配结果应当被自动忽略」，从而把空白处理从每条规则里抽出来，提升可理解性与可修改性。 – **直观解释（"它是什么？"）**：`@skip N { ... }` 相当于在块内每条规则的右侧、每个终结符/非终结符/字符类的**前后**，都自动插入 `N*`。就像排版时说「这里到那里，空白一律不计」。 – **关键规则与最佳实践**： – `@skip` 不专属于空白：任何在文法中定义过的非终结符都能被 skip，例如 `@skip spacesAndComments`。 – **关键设计抉择**：把 `number` / `constant` 这类规则**放在 `@skip` 块外**，才能接受 `42 + 2` 却拒绝 `4 2 + 2`。若把 `number` 放进块内，它实际会变成 `number ::= (whitespace* [0-9] whitespace*)+`，于是 `4 2` 会被当成一个数。 – 反过来，把使用 `number` 的 `primary` 放进 `@skip` 块内，`number` 的**前后**就能有空白。 – 被 skip 的子树不会出现在 `children()` 里，因此转换函数里**永远不需要处理 `WHITESPACE` 这一 case**。 – `@skip` 的写法会改变节点 `text()` 的内容：块内规则的节点其 `text()` 可能包含首尾空白（例如 `"Fall "`），块外的不会。

解析树（Parse Tree）与遍历 – **定义与目的**：解析树展示「文法产生式是如何展开成与输入匹配的句子的」。根节点对应文法的根非终结符，每个节点展开成一条产生式。它是解析的中间产物，也是「解析到底哪里出错」的调试依据。 – **直观解释（"它是什么？"）**：解析树像一棵「语法的家族树」：每个内部节点是一个语法范畴（nonterminal），它的孩子是这条产生式右侧被匹配到的那些子范畴。终结符（`+`、`(`、`54` 里的数字字符）不单独成节点。 – **关键规则与最佳实践**： – 在 Java 中 `ParseTree<NT>` 是**泛型类型**，由你定义的非终结符枚举参数化：`ParseTree<IntegerGrammar>`。 – 四个核心方法：观察者 `name()`（本节点对应的非终结符）、`children()`（有序的孩子，**不含被 skip 的子树**）、`text()`（本子树匹配到的原始子串），以及查询方法 `childrenByName(NT)`（等价于对 `children()` 做一次 filter）。 – 遍历解析树的自然方式是**递归函数**，并让函数结构跟随文法结构。 – 调试时用 `toString()` 打印，或用 `Visualizer.showInBrowser(tree)` 在浏览器里可视化（打不开浏览器时会打印一个可复制的 URL）。 – 只看 `children()` 的节点名，就能验证「终结符没有独立节点」「孩子的名字必是该产生式右侧提到的非终结符」「`@skip` 的非终结符不会出现在孩子里」。

抽象语法树（AST）与具体语法树（Concrete Syntax Tree） – **定义与目的**：把解析树翻译成递归数据类型，就得到**抽象语法树**。它保留语言表达式的**重要特征**（分组结构与其中的数值），丢弃「这串字符具体是怎

么写的」这种无关细节。这是把「文本」正式转成「有类型的值」的一步，直接服务于安全性（类型系统从此可以检查）与可修改性（后续处理只依赖 AST，不依赖书写形式）。 – **直观解释（"它是什么？"）**：解析树（也叫**具体语法树**）记录「作者怎么写的」，AST 记录「意思是什么」。`2+2`、`((2)+(2))`、`0002+0002` 会生成三棵不同的具体语法树，但它们都对应同一个 AST 值 `Plus(Number(2), Number(2))`。 – **关键规则与最佳实践**： – 转换函数**一个 case 对应一条文法规则**，并显式处理每种产生式；文法重大改动时这个函数通常也要跟着改。 – 恰当使用 `map` / `reduce`（Java 中用循环或 `stream()`）避免手写索引，因为 `childrenByName(NT)` 就是一次 filter。 – 用 `switch` 处理 `primary` 这类「多选一」节点时，务必**每个 case 都 return 或 throw**——`switch` 会从匹配的 case 开始向下贯穿（fall through），忘记 return 会静默执行下一个 case 的代码。 – 文法的 n 元结构（`sum ::= primary ('+' primary)*`）与 AST 的二元结构（`Plus(left, right)`）经常不一致，需要在转换函数里显式地把 n 元折叠成二元（或改动文法使解析树本身至多二元）。 – 解析树的形状由文法决定：`@skip` 放在哪一层，直接决定你比较 `text()` 时要不要 `trim()`。

递归下降解析与左递归（Recursive Descent & Left Recursion） – **定义与目的**：ParserLib 生成的是自顶向下的递归下降解析器：它从根非终结符出发，为每条规则尝试匹配右侧的各个成分。理解这个实现方式，才能理解它为什么**不能**接受某些文法，从而避免在这类文法上浪费时间。 – **直观解释（"它是什么？"）**：递归下降解析器像人照着地图走迷宫：每到一处，就按规则顺序试着走一步。**左递归（left recursion）**就是「第一步要求你先走到你现在所在的位置」——永远原地打转，问题规模不变小，递归无法终止。 – **关键规则与最佳实践**： – 左递归的定义：某非终结符的定义中，它自己出现在**最左符号**位置，例如 `sum ::= number | sum '+' number ;`。 – 左递归可以是**间接的**：`sum ::= number | thing number ; thing ::= sum '+' ;` 同样致命。 – 非左递归的递归是安全的：`expr ::= number | '(' expr ')' ;` 每次递归前先消耗掉一个 `(`，问题在变小。 – 消除办法：把左递归改写成重复，`sum ::= (number '+')* number ;`。 – 若把左递归文法交给 ParserLib，解析时会以 `UnableToParseException`（sp22 中为 `ParseError`）失败，并列出有问题的非终结符。 – **贪婪性（greediness）**：ParserLib 在每一点都尝试为当前规则匹配**最长**的串，因此 `g ::= ab threeb ; ab ::= 'a'*'b'* ; threeb ::= 'bbb' ;` 无法解析 `'aaaabbb'`（`ab` 先把整串吃掉）。这是该类解析器的固有局限，不像左递归那样容易修。

错误处理（Handling Errors） – **定义与目的**：解析是「外部输入」进入程序的第一道门，必须明确失败语义。ParserLib 用异常把三类失败分开，让调用者能区分「我的文法文件有问题」与「用户的输入不合法」。 – **直观解释（"它是什么？"）**：解析错误像海关查验：要么是你的查验手册写错了（文法错误），要么是旅客的证件不合规（输入错误）。异常里给出的位置信息只是**可能**的位置，因为解析器并不知道你原本想写什么。 – **关键规则与最佳实践**： – 文法文件打不开 → `compile` 抛 `IOException`。 – 文法本身有语法错误 → `compile` 抛 `UnableToParseException`。 – 输入串无法用该文法解析 → `parse` 抛 `UnableToParseException`。 – 异常里的位置信息需要人工排查，不要期望它精确指向你心里的那一个字符。 – 在 Java 中 `UnableToParseException` 是**受检异常（checked exception）**，调用处必须 `try/catch` 或声明 `throws`——编译器会强制你面对「解析可能失败」这个事实。

解析与 ADT / 递归数据类型的关系（Parsing as ADT Construction） – **定义与目的**：解析的终点不是树，而是**值**。AST 是用递归数据类型定义的 ADT，它的**抽象函数（AF）**把「内存里的对象图」映射为「一个数学上的表达式」。 – **直观解释（"它是什么？"）**：解析树是「过程性的中间产物」，AST 是「结果性的抽象值」。就像做菜时案板上的半成品与端上桌的那道菜：前者记录了你切了几刀，后者才是「一道菜」本身。 – **关键规则与最佳实践**： – AST 类型应当在**接口/抽象类**里写下 datatype definition 注释，把「ADT 的取值集合」显式化，例如 `IntegerExpression = Number(n:int) + Plus(left, right)`。 – 产生 AST 的转换函数应当写清晰的 specs：前置条件是「该解析树由本讲文法生成」，后置条件是「返回与之对应的 AST 值」。 – 让 AST 的类型选择反映**语义**而不是**书写形式**：`Number` 存 `int` 而不是原字符串，`Plus` 存两个子表达式而不是 token 列表。 – 一旦有了 AST，后续所有分析（求值、优化、类型检查）都只面对这个小而清晰的数据类型，形成清晰的抽象边界。

代码示例与对比分析

场景 1：把「Fall15」这学期字符串变成有意义的数据类型——手写字符扫描 vs 文法 + 解析器生成器

❌ *错误代码*

```
// 错误：手工按下标扫描字符串，把语法知识散落在 if 判断里
public static Semester parseSemester(String input) {
    String s = input.trim();
    int i = 0;
    while (i < s.length() && Character.isWhitespace(s.charAt(i))) i++;
    String season;
    if (s.startsWith("Fall", i)) { season = "Fall"; i += 4; }
    else if (s.startsWith("Spring", i)) { season = "Spring"; i += 6; }
    else throw new IllegalArgumentException("bad season: " + input);
    while (i < s.length() && Character.isWhitespace(s.charAt(i))) i++;
    int start = i;
    while (i < s.length() && Character.isDigit(s.charAt(i))) i++;
    String digits = s.substring(start, i);
    if (digits.length() != 2) throw new IllegalArgumentException("bad year: " + input);
    if (i != s.length()) throw new IllegalArgumentException("trailing junk: " + input);
    return new Semester(season, Integer.parseInt(digits));
}
```

【错误代码的问题】 1. **规格说明被埋在代码里**：合法输入的形状（`season` 后跟两位 `year`、允许空白）只能靠读代码反推，无法单独检视，违反了「清晰沟通」的目标。 2. **极易漏掉边界**：忘记检查尾部多余字符（`i != s.length()`）、忘记 `Spring` 与 `Fall` 之后的空白、把 `[0-9] [0-9]` 误写成「一或多个数字」，都可能漏检或误检。 3. **不可修改**：一旦要支持 `Winter` / `Summer`，或者要求年份恰好两位，需要重写多段索引逻辑，改动点分散。 4. **难以测试**：没有一组「这个文法的语言包含/不包含哪些串」的清单，测试用例只能靠直觉补。

✅ *正确代码*

```
// 正确：文法独立于代码，且由解析器生成器生成匹配逻辑
// semester.g 文件内容：
//   @skip spaces {
//     semester ::= season year ;
//     season ::= 'Fall' | 'Spring' ;
//     year ::= [0-9] [0-9] ;
//   }
//   spaces ::= ' '+ ;

import edu.mit.eecs.parserlib.*;   // ParserLib (sp21 Java 版)

public enum SemesterGrammar { SEMESTER, SEASON, YEAR, SPACES }

public static Semester parseSemester(String input)
        throws IOException, UnableToParseException {
    Parser<SemesterGrammar> parser = Parser.compile(
            new File("src/semester/semester.g"), SemesterGrammar.SEMESTER);
    ParseTree<SemesterGrammar> tree = parser.parse(input);
    return convertToSemester(tree);
}

/** @param node must be a match to the semester rule
 *  @return corresponding Semester value */
private static Semester convertToSemester(ParseTree<SemesterGrammar> node) {
    if (node.name() != SemesterGrammar.SEMESTER) {
        throw new AssertionError("expected SEMESTER node");
    }
    List<ParseTree<SemesterGrammar>> seasons = node.childrenByName(SemesterGrammar.SEASON);
    List<ParseTree<SemesterGrammar>> years   = node.childrenByName(SemesterGrammar.YEAR);
    if (seasons.size() != 1 || years.size() != 1) {
        throw new AssertionError("semester should have exactly one season and one year");
    }
    return new Semester(convertToSeason(seasons.get(0)),
                        Integer.parseInt(years.get(0).text()));
}
```

【为什么这样更好】 文法是**声明式规格**：语言是什么，一眼可见，而且它就是可以被评审、被测试的文档。匹配逻辑由生成器产出，不存在手写索引漏检 `trailing junk` 的机会。新增 `Winter` 只需在文法的 `season` 规则里加一个选择，解析器重新生成即可。

【代码对比解说】 两种写法的真正差别不在「谁写的循环更短」，而在**知识放在哪里**。手写扫描把「语言的定义」拆散成若干条 `if` 与 `i += 4`，这些常量与文法知识是同一份信息的两种表示，容易不同步。文法写法把这份信息集中成一份可执行的规格。代价是必须理解一套新的工具链（`.g` 文件、枚举、泛型 `Parser<NT>`），并且文法要遵守递归下降解析器的限制（不能左递归）。在 6.031 的尺度上，这个代价是值得的。

【设计原则透视】 这是「规格说明（Reading 6）先行」的直接应用：文法即规格，转换函数即实现。转换函数还是一个典型的**抽象函数**：它把产生式结构（representation）映射成 `Semester` 这个抽象值。原子性/边界检查的思路也是 **Reading 9（避免调试）** 的实践——不要依赖「我小心一点」，而要让工具替你把关。

场景 2：整数表达式文法中的空白——把 `number` 放进 `@skip` 块 vs 放在块外

❌ *错误代码*

```
// 错误：整个文法都在 @skip 块内，constant 也被跳过空白
@skip whitespace {
  expr ::= sum ;
  sum ::= primary ('+' primary)* ;
  primary ::= constant | '(' sum ')' ;
  constant ::= [0-9]+ ;
}
whitespace ::= [ \t\r\n]+ ;
```

```
// 后果：下面这行代码把 "4 2 + 2" 里的 "4 2" 错当成一个常量
ParseTree<IntegerGrammar> tree = parser.parse("4 2 + 2");
IntegerExpression expr = makeAbstractSyntaxTree(tree); // 得到 Plus(Number(4), Number(2)) ..
```

【错误代码的问题】 1. **接受了不该接受的输入**：块内的 `constant` 实际展开为 `constant ::= (whitespace* [0-9] whitespace*)+`，于是 `4 2` 被当成一个常量，语言边界被悄悄放宽。 2. **缺陷极难察觉**：文法「看起来」完全正常，错误只在特定输入上暴露，属于典型的「规格被悄悄改写」。 3. **下游语义污染**：AST 里出现的 `Number(4)`、`Number(2)` 与用户本意（一个叫 42 的数？还是两个数？）不符，错误会传播到求值阶段。 4. **`text()` 里带空白**：块内节点的 `text()` 会含空白，日后若要直接比较 `text()`，很容易写成 `text() == "Fall"` 之类的错误。

✅ *正确代码*

```
// 正确：把 constant 的规则移到 @skip 块之外
@skip whitespace {
  expr ::= sum ;
  sum ::= primary ('+' primary)* ;
  primary ::= constant | '(' sum ')' ;
}
whitespace ::= [ \t\r\n]+ ;
constant ::= [0-9]+ ;
```

```
// 于是 "42 + 2" 可以解析（constant 的前后有空白由 primary 所在的块负责），
// 而 "4 2 + 2" 会被拒绝（constant 内部不允许空白）
try {
    parser.parse("42 + 2");   // OK
    parser.parse("4 2 + 2");  // 抛 UnableToParseException
} catch (UnableToParseException e) {
    // 解析失败被显式暴露，而不是产生一个语义错误的 AST
}
```

【为什么这样更好】 `@skip` 的作用范围是「块内规则右侧各成分的前后」。把 `constant` 移出块，就切断了「常量内部允许空白」这条被意外引入的规则，同时因为 `primary` 仍在块内，`constant` 作为一个整体仍

可被空白包围——恰好是我们要的语言。

【代码对比解说】 这是一个关于**抽象边界画在哪里**的精彩案例。同一条 `constant ::= [0-9]+`，放在块内还是块外，得到的是两种不同的语言。很多学生以为 `@skip` 只是「美化空白的语法糖」，其实它是**规格的一部分**：它精确决定了哪些位置可以有空白。判断标准很简单——问自己「这个记号内部允许空白吗？」不允许，就不要把它的规则放进 `@skip` 块。

【设计原则透视】 对应 RI 的思想：文法是语言的表示不变量，`@skip` 的位置选择属于 RI 的一部分，必须与「这个语言应该长什么样」严格一致。也呼应 Reading 7（设计规格说明）：规格的**强弱**非常关键，过强的规格会拒绝合法输入，过弱的规格会接受非法输入——本例正是「过弱」。

场景 3：把 n 元的 `sum` 节点转成二叉 `Plus`——错误地假设孩子个数与顺序 vs 用 `childrenByName` 与折叠

❌ *错误代码*

```
// 错误：假设 SUM 恰好有两个孩子，且第一个一定是数字
private static IntegerExpression makeAbstractSyntaxTree(ParseTree<IntegerGrammar> t) {
    switch (t.name()) {
        case EXPR:
            return makeAbstractSyntaxTree(t.children().get(0));
        case SUM: {
            // 文法 sum ::= primary ('+' primary)* 是 n 元的：
            // "19+23+18" 的 SUM 节点有 3 个孩子，这里只取前两个
            IntegerExpression left  = makeAbstractSyntaxTree(t.children().get(0));
            IntegerExpression right = makeAbstractSyntaxTree(t.children().get(1));
            return new Plus(left, right);
        }
        case PRIMARY:
            return makeAbstractSyntaxTree(t.children().get(0));
        case NUMBER:
            return new Number(Integer.parseInt(t.text()));
        default:
            throw new AssertionError("should never get here");
        // 注意：SUM 分支忘了 return 时，Java 会继续执行 PRIMARY 分支的代码
    }
}
```

【错误代码的问题】 1. **静默丢数据**：`"19+23+18"` 的 `Sum` 节点有三个 `Primary` 孩子，这段代码只取前两个，第三个 `18` 被丢弃，得到错误的 AST 且不报错。 2. **依赖孩子的个数与顺序**：一旦文法改成 `sum ::= primary ('+' primary)*` 之外的形状，索引假设立刻失效，属于典型的**表示依赖**（rep exposure 的思维错误）。 3. **`switch` 的贯穿风险**：若某个 case 忘记 `return`，执行会落入下一个 case，产生难以定位的错误结果。 4. **无法处理 n = 1 的情形**：单个数字 `42` 的 `Sum` 只有一个孩子，`get(1)` 直接抛 `IndexOutOfBoundsException`。

✅ *正确代码*

```
/**
 * Convert a parse tree into an abstract syntax tree.
 *
 * @param parseTree constructed according to the grammar in IntegerExpression.g
 * @return abstract syntax tree corresponding to parseTree
 */
private static IntegerExpression makeAbstractSyntaxTree(final ParseTree<IntegerGrammar> par
    switch (parseTree.name()) {
        case EXPR: // expr ::= sum;
        {
            final ParseTree<IntegerGrammar> child = parseTree.children().get(0);
            return makeAbstractSyntaxTree(child);
        }

        case SUM: // sum ::= primary ('+' primary)*;
        {
            final List<ParseTree<IntegerGrammar>> children =
                    parseTree.childrenByName(IntegerGrammar.PRIMARY);
            IntegerExpression expression = makeAbstractSyntaxTree(children.get(0));
            for (int i = 1; i < children.size(); ++i) {
                expression = new Plus(expression, makeAbstractSyntaxTree(children.get(i)));
            }
            return expression;   // n 元折叠成左结合的二叉 Plus
        }

        case PRIMARY: // primary ::= number | '(' sum ')';
        {
            final ParseTree<IntegerGrammar> child = parseTree.children().get(0);
            // 检查实际匹配的是哪一个选择分支（number 还是 sum）
            switch (child.name()) {
                case NUMBER:
                    return makeAbstractSyntaxTree(child);
                case SUM:
                    return makeAbstractSyntaxTree(child); // 本例两种情形处理相同
                default:
                    throw new AssertionError("should never get here");
            }
        }

        case NUMBER: // number ::= [0-9]+;
        {
            final int n = Integer.parseInt(parseTree.text());
            return new Number(n);
        }

        default:
            throw new AssertionError("should never get here");
    }
}
```

【为什么这样更好】 `childrenByName(PRIMARY)` 直接表达「我要的是这条产生式里所有的 `PRIMARY` 孩子」，把 `+` 号这类终结符（它们本来也不出现在 `children()` 里）和结构性细节一起屏蔽掉；循环折叠对任意 n ⩾ 1 都成立，`"19+23+18"` 得到左结合的 `Plus(Plus(Number(19), Number(23)), Number(18))`。每个分支都 `return` 或 `throw`，`switch` 绝不贯穿。

【代码对比解说】 关键洞察是：**解析树的形状是文法的直接映射，不是你的 AST 想要的形状**。文法 `sum ::= primary ('+' primary)*` 是 n 元的，而 `Plus` 是二元的（恰好左右各一）。转换函数就是这两者之间的翻译层，翻译策略（本例取左结合）必须是有意识的选择，而不是从「孩子只有两个」的错觉里无意产生的。若希望解析树本身至多二元，可以把文法改成 `sum ::= primary | primary '+' sum ;`（右结合），此时 `SUM` 孩子最多两个——但要注意这时就不能再用 `sum ::= sum '+' sum`，那会引入左递归。

【设计原则透视】 这是 **Reading 17（递归数据类型）** 与 **Reading 10（ADT）** 的交汇：AST 的 datatype definition 决定了取值的集合，而转换函数必须覆盖整个取值集合。`childrenByName` 的使用体现了「不要暴露表示的细节给调用者」——用语义查询而非下标访问。同时注意函数里对「不可能情况」抛 `AssertionError`，是 RI 的外部化表达。

场景 4：从 `season` 节点取季节——用 `==` 比较字符串 vs 用 `equals`，并正确处理 `@skip` 带来的空白

❌ *错误代码*

```
// 错误一：用 == 比较字符串内容
static Season convertToSeason(ParseTree<SemesterGrammar> node) {
    if (node.name() != SemesterGrammar.SEASON) throw new AssertionError();
    return node.text() == "Fall" ? Season.FALL : Season.SPRING;   // 永远为 false!
}

// 错误二：即使改用 equals，块外规则拿到的 text() 可能带空白
static Season convertToSeason2(ParseTree<SemesterGrammar> node) {
    return node.text().equals("Fall") ? Season.FALL : Season.SPRING;
    // 当 season 规则也在 @skip 块内时，node.text() 可能是 "Fall " 或 " Fall"，比较失败
}
```

【错误代码的问题】 1. **`==` 比较的是引用**：`node.text()` 返回新构造的 `String`，与字面量 `"Fall"` 不是同一对象，判断恒为 false，于是 `convertToSeason` 永远返回 `SPRING`——一个完全不报错的错误答案。 2. **忽略 `text()` 的空白语义**：`text()` 返回「本子树匹配到的原始子串」；如果这条规则在 `@skip` 块内，首尾的空白也在匹配范围内，`equals` 会失败。 3. **失败方式恶劣**：两个错误都表现为「结果错但不抛异常」，属于典型的 silent bug，测试若只覆盖一种输入就发现不了。

✅ *正确代码*

```
/** @param node must be a match to the season rule
 *  @return corresponding Season value */
static Season convertToSeason(ParseTree<SemesterGrammar> node) {
    if (node.name() != SemesterGrammar.SEASON) {
        throw new AssertionError("expected a SEASON node");
    }
    final String text = node.text();
    // 视 season 规则是否位于 @skip 块内决定是否需要 trim
    return text.trim().equals("Fall") ? Season.FALL : Season.SPRING;
}
```

【为什么这样更好】 用 `equals` 比较内容；`trim()` 把「这条规则是否在 `@skip` 块内」这个文法细节与比较逻辑解耦，使函数在两种文法下都正确。更稳妥的做法是针对真正的语义分支写测试：`"Fall15"`、`" Spring 23 "`、`"Spring 9 9"`（应当被拒绝）都要覆盖。

【代码对比解说】 这个例子把两讲的内容缝合在一起：`==` vs `equals` 是 **Reading 15（相等性）** 的核心教训，而 `text()` 是否含空白是**本讲 `@skip` 语义**的直接后果。学生常见的第三种错误是「看到比较失败就加 `trim()`」，却不理解为什么——如果 `season` 规则在块外，`text()` 本来就不含空白，`trim()` 只是无害的冗余；如果在块内，`trim()` 就是必需的。理解原因，才能在文法变化时正确判断。

【设计原则透视】 `ParseTree` 的规格（spec）明确说明 `text()` 是「原串的子串」，这不是「规范化后的记号」，调用者不能假设它已去除空白——典型的**规格边界**问题。而 `==` 的错误则说明「相等性语义」必须依规约（Reading 15 中的等价关系）来用，不能凭语法直觉。

场景 5（补充）：左递归文法 vs 用重复改写

❌ *错误代码*

```
// 错误：左递归，递归下降解析器会陷入无限递归
sum ::= number | sum '+' number ;
number ::= [0-9]+ ;
```

【错误代码的问题】 1. 解析 `sum` 时先要匹配 `sum` 本身，问题规模不缩小，递归永不终止（ParserLib 会以异常失败并指出违规的非终结符）。 2. **间接左递归同样致命**：`sum ::= number | thing number ; thing ::= sum '+' ;` 依然把 `sum` 放在最左位置。 3. 这类文法是**自然书写顺序**的产物（人写算式就是「左边再加一个数」），所以学生很容易不自觉写出来。

✅ *正确代码*

```
// 正确：用重复 (*) 表达「若干个 number，用 + 连接」
sum ::= (number '+')* number ;
number ::= [0-9]+ ;
```

【为什么这样更好】 重复算子 `*` 由解析器生成器实现为一个循环，不再需要「先递归到自己」；同时它保留了「至少一个 number」的语义，语言集合与直觉一致。

【代码对比解说】 消除左递归的一般手法是引入尾递归/迭代；在 ParserLib 的语境下最简单的是把左递归改写成 `(X op)* X` 形式。注意代价：改写会改变**解析树的形状**，因而转换函数也要跟着改（n 元折叠的问题又回来了）。

【设计原则透视】 这是「工具的抽象代价」：声明式文法并非万能，生成器有它自己的**前置条件**（grammar must not be left-recursive）。程序员必须理解抽象边界之下的实现约束，才能正确使用抽象（Reading 11 的思想：抽象不隐藏它承诺之外的义务）。

与其他设计原则的关联

- **Reading 18（正则表达式与文法）**：本讲是它的直接延续。正则表达式适合描述**词法**（token 的形状，如 `[0-9]+`），文法适合描述**语法**（token 如何组成结构）；`@skip whitespace` 就是把「词法层面的空白」从语法层剔除的机制。ParserLib 语法里「字面字符必须加引号」也是相对普通正则的一处差异。
- **Reading 17（递归数据类型）**：AST 就是递归 ADT。`IntegerExpression = Number(n:int) + Plus(left, right)` 的 datatype definition 直接决定了转换函数要处理的 case 集合；本讲是递归数据类型的第一次真实应用。
- **Reading 10（抽象数据类型）与 Reading 11（抽象函数、表示不变量）**：转换函数本质上是**抽象函数**：把「由文法定义的表示结构」映射为「表达式这一抽象值」。解析树的结构约束（哪些孩子可能出现、`@skip` 的孩子不出现）就是它的 RI。
- **Reading 6/7（规格说明与设计规格）**：文法是文本输入的**声明式规格**，`@skip` 的位置决定规格的强弱；转换函数则需要写清前置条件（「输入是由本文法生成的解析树」）与后置条件（「返回对应的 AST」）。
- **Reading 15（相等性）**：`text().equals("Fall")` 与 `text() == "Fall"` 的差别是本讲最常见的 bug 之一，正确处理它依赖相等性的等价关系概念。
- **Reading 20（回调与 GUI）与 Reading 25/21（并发）**：解析器常被用在事件处理中——用户在 GUI 里输入一段表达式，回调里调用 `parser.parse(...)`。这时异常处理与执行时间（不要阻塞事件循环）就成为新的关注点。
- **Reading 26/27（Little Languages）**：本讲是「解释器/小语言」项目的基础：文法 → 解析树 → AST → 求值器，正是后续课程与 psets 中反复出现的流水线。

关键要点

- **先写文法，再写代码**：任何「解析文本」的任务都先问「这个语言是什么」，用文法把它写下来；匹配逻辑交给解析器生成器，你只负责文法与「解析树 → AST」的翻译。
- **`@skip` 的位置是规格的一部分**：它决定哪些位置允许空白；记号内部不允许空白时，该记号的规则必须放在 `@skip` 块外。
- **转换函数必须跟随文法结构**：一个 case 对应一条产生式，每个 case 都要 `return` 或 `throw`，并且要处理 n 元结构到二元 AST 的折叠这类形状差异。
- **了解你的解析器的限制**：递归下降解析器不能处理左递归（包括间接左递归），用 `*` 改写；贪婪匹配是更根本的局限，需要靠文法设计规避。

- **解析的终点是有类型的值**：把 `ParseTree` 尽早转成 AST，之后所有代码都面对小而清晰的递归数据类型，而不是面对解析树的表示细节。

常见陷阱与注意事项

- **把 `number` / `constant` 这类记号规则放进 `@skip` 块** → 记号内部允许空白，`"4 2 + 2"` 被错误接受，语言边界被悄悄放宽。
- **用 `==` 比较 `text()` 与字符串字面量** → 比较的是引用，判断恒为 false，得到静默的错误结果（永远走另一个分支）。
- **假设 `SUM` （n 元）节点恰好有两个孩子** → 对 `"19+23+18"` 静默丢弃第三个操作数，或对单个数字抛 `IndexOutOfBoundsException`。
- **在 `switch` 的某个 case 忘记 `return`** → Java 从匹配的 case 向下贯穿，执行到下一个 case 的代码，产生看似莫名其妙的行为。
- **写出左递归文法（含间接左递归）** → 递归下降解析器无限递归，ParserLib 以异常失败；应改写为 `(X op)* X` 形式。
- **与解析器 API 打交道时疏忽**：枚举里漏掉非终结符（`WHITESPACE` 这类只在 `@skip` 中使用的也必须列出）或误把终结符写进去 → `compile` 报缺失规则；忘记 `parse` / `compile` 会抛受检异常（`IOException`、`UnableToParseException`）→ 代码无法编译，或用空 `catch` 吞掉解析失败，让非法输入以「默认值」的形式流进系统。

思考题（带答案）

问题 1：给定文法

```
@skip spaces {
  semester ::= season year ;
}
season ::= 'Fall' | 'Spring' ;
year ::= [0-9] [0-9] ;
spaces ::= ' '+ ;
```

（注意 `season` 与 `year` 的规则都在 `@skip` 块**外**）请问 `" Spring 23 "` 能否被匹配？如果 `@skip` 块把 `season` 和 `year` 也包进去，答案会改变吗？这对 `convertToSeason` 的实现有什么影响？

答案：`@skip spaces { semester ::= season year ; }` 只对 `semester` 这条规则右侧的成分生效，也就是说 `semester` 右侧的 `season` 与 `year` 的前后允许出现空格，因此 `" Spring 23 "` 可以被匹配（开头的空白属于 `semester` 之前的位置，也会被跳过）。如果 `season`、`year` 的规则本身也被移入 `@skip` 块，那么 `season` 的 `text()` 就可能包含首尾空白（如 `"Spring "`），`convertToSeason` 中直接 `text().equals("Fall")` 就会失败，必须写成 `text().trim().equals("Fall")`。这正是课程练习里「To every thing… there is a season」两组题目的差别：`@skip` 的覆盖范围改变了解析树节点 `text()` 的内容，从而改变了转换代码的正确写法。

问题 2：为什么 `sum ::= primary ('+' primary)*` 的解析树中，`Sum` 节点的孩子**不包含** `'+'`，而 `primary ::= constant | '(' sum ')'` 的 `Primary` 节点的孩子可能是 `Sum` 也可能是 `Constant`？请从

「终结符/非终结符」与「产生式展开」的角度解释，并说明这对转换函数意味着什么。

答案：解析树的每个节点对应一条产生式的展开，节点的名字来自**左侧的非终结符**；孩子则是右侧被匹配到的成分。终结符（`'+'`、`'('`、`')'`）在 ParserLib 的解析树里**不单独成节点**，因此不会出现在 `children()` 中；`childrenByName(PRIMARY)` 恰好等价于「对 `children()` 做 filter，只保留名字为 `PRIMARY` 的孩子」。所以 `Sum` 的孩子只有一串 `Primary`，`+` 的存在只能从 `text()` 或孩子的个数推断出来。而 `Primary` 的孩子是「实际匹配的那一个选择分支」对应的非终结符节点：匹配到 `constant` 时孩子是 `Constant`，匹配到 `'(' sum ')'` 时孩子是 `Sum`（括号作为终结符不出现）。对转换函数的直接后果是：处理 `Primary` 必须用 `switch` 判断孩子的 `name()` 来区分分支（并且每个分支都要 `return` 或 `throw`），而处理 `Sum` 则应当用 `childrenByName` 取出所有 `Primary` 再折叠，不能假设固定个数。

问题 3：下面的文法为什么不能让 ParserLib 正常工作？请给出两种修改方案，并说明它们会如何改变解析树的形状。

```
sum ::= number | sum '+' number ;
number ::= [0-9]+ ;
```

答案：这是**左递归**：`sum` 的一个选择分支 `sum '+' number` 把 `sum` 放在最左位置。递归下降解析器在匹配 `sum` 时必须依次尝试每个选择，而尝试 `sum '+' number` 的第一步又要匹配 `sum`，问题规模不缩小，递归无法终止，ParserLib 会以 `UnableToParseException` 失败并指出违规的非终结符。修改方案一：用重复消除左递归，`sum ::= (number '+')* number ;`，此时 `Sum` 节点是 n 元的（孩子全是 `Number`），解析树变「平」，转换函数需要把 n 元折叠成二叉 AST。修改方案二：改成右递归，`sum ::= number | number '+' sum ;`（或等价地 `sum ::= number ('+' sum)? ;`），这不是左递归，可以正常工作；此时解析树是右倾的（每个 `Sum` 最多两个孩子），得到的 AST 天然是右结合的 `Plus(Number(19), Plus(Number(23), Number(18)))`，与方案一得到的左结合结果语义不同——这说明「消除左递归的方式」会同时决定结合性，必须与语言规格保持一致。

Reading 20: 回调与图形用户界面（Callbacks and Graphical User Interfaces）

> 说明：本讲 sp22 原版使用 TypeScript，本笔记按用户要求提供 Java 代码示例；类型/API 与 sp21（6.031 Java 版）原文保持一致。

概述

本讲讨论**回调（callback）**：由**客户端（client）**提供给**模块（module）**、由模块在合适的时机去调用的函数。这与我们熟悉的控制流方向恰好相反——平时总是客户端向下调用模块提供的函数，而回调是客户端把一段代码「交出去」，让实现者来调用它。课程用**图形用户界面（GUI）**作为主要语境：GUI 里的一种回调叫**监听器（listener）**，用来响应用户产生的输入事件。回调背后的更大思想是**一等函数（first-class functions）**：把函数当成数据一样传递、返回、存储。

它与三大目标的关系是：**Ready for change** 最明显——回调让客户端自己提供「事件发生时该做什么」，行为不必被硬编码进实现；**Easy to understand**，比起「写一个巨大的输入循环处理系统里所有可能的事件」，

让每个模块负责自己的事件显然更清晰；**Safe from bugs**，因为那个「无所不知、无所不控」的巨型分派函数是最容易出错的代码。本讲同时埋下了后面几讲的伏笔：异步回调意味着**控制权不在你手里**，因此必然引出并发（Reading 21）、锁与线程安全（Reading 23）以及异步编程（Reading 22）的问题。

核心概念与设计原则详解

回调（Callback） – **定义与目的**：回调是客户端提供给模块、供模块调用的函数。它把「事件发生时做什么」这一决策从实现者手里交给客户端，直接服务于可修改性（行为可插拔）与易理解性（每个模块只管自己的事件）。 – **直观解释（"它是什么？"）**：课程给的类比是给银行打电话。普通函数调用像你主动打电话到银行问余额，银行查到后告诉你，你挂断——你是客户端，银行是你调用的模块。如果银行很慢，你被要求「别挂，稍等」，这就是**阻塞式**调用。但有些任务太慢，银行不愿意让你一直等，于是问你一个**回拨电话（callback number）**，承诺将来某个不可预测的时刻打回来——这就是异步回调。 – **关键规则与最佳实践**： – 回调有两种调用模式：一次性回答（如查询余额）与被反复触发（如账户盗刷提醒——对应监听器模式）。 – 用回调意味着**控制权反转（inversion of control）**：实现者决定何时调用你的代码，你必须按规格假设「它可能在任何时刻被调用」。 – 回调函数的规格必须写清：调用次数（至多一次 / 每次事件一次）、调用时机（同步还是异步）、参数含义与取值范围。 – 把回调参数放在参数列表的**末尾**（`setTimeout` 把回调放第一位是课程明确批评的「不幸设计」，因为在单行 lambda 写法下难以分辨参数边界）。

同步回调与异步回调（Synchronous vs Asynchronous Callbacks） – **定义与目的**：同步回调只在接收它的那个调用执行期间被调用，调用返回后不再使用；异步回调会被模块**保存下来**，在接收它的函数**已经返回之后**的某个时刻再调用。区分的目的是让「时间」这个维度显式化，避免写出依赖错误时序的代码。 – **直观解释（"它是什么？"）**：Reading 16 里传给 `map` / `filter` / `reduce` 的函数就是同步回调——它在 `map` 执行完之前被逐元素调用，`map` 一返回，它的使命就结束了。而「三秒后响铃」的定时器回调是异步的：登记完就立刻返回，三秒后才被叫醒。 – **关键规则与最佳实践**： – 同步回调的异常可以用**包围调用点**的 `try/catch` 捕获；异步回调的异常**不能**——登记回调的那行代码早已返回，异常发生时没有任何栈帧在处理它。 – 同步回调的副作用在调用返回时**保证已发生**；异步回调的副作用只保证「将来会发生」，代码不能依赖它已经发生。 – 规格里必须写明首次调用是同步还是异步（例如 countdown 规格：第一次同步调用，之后每次计时跳动异步调用）。 – 异步回调可以被调用**零次、一次或多次**，取决于规格；客户端的代码必须对「还没发生」有正确假设。

一等函数与函数对象（First-Class Functions & Functional Objects） – **定义与目的**：一等函数指函数可以像其他值一样被传递、返回、存入变量与数据结构。它是实现回调的**语言前提**：如果函数不能作为值传递，就无法把「一段代码」交给模块。 – **直观解释（"它是什么？"）**：语言里很多东西**不是**一等公民：访问控制（`public` / `private` 不能当参数）、`while` 循环、`if` 语句都无法被单独引用或在运行时操纵。而在一等函数的语言里，函数本身就是可以拿在手上的东西。 – **关键规则与最佳实践**： – Java 里唯一的类型化一等值是**原始值与对象引用**；函数必须**搭在对象上**（方法）。 – 于是 Java 用**函数对象（functional object）**实现一等函数：一个「目的是表示某个函数」的对象，其规格由一个**单抽象方法接口（Single Abstract Method, SAM interface）**给出。 – 课程里已经出现过多次：传给 `Thread` 的 `Runnable`（即 `void run()`）、传给有序集合的 `Comparator<T>`（即 `int compare(T,T)`）、传给按钮的 `ActionListener`（即 `void`

`actionPerformed(ActionEvent)`）、传给 `HttpServer` 的 `HttpHandler`（即 `void handle(HttpExchange)`）。 – 一等函数的谱系可以追溯到 Lisp（MIT 的 John McCarthy）与更早的 λ 演算（Alonzo Church），这也是 lambda 一词的来源。

Lambda 表达式（Lambda Expressions） – **定义与目的**：lambda 是创建函数对象的**简洁语法**，它让回调的书写成本降到接近「就地写一段代码」，从而鼓励使用回调（可修改性）而不牺牲可读性。 – **直观解释（"它是什么？"）**：`new Thread(new Runnable() { public void run() { ... } })` 与 `new Thread(() -> { ... })` 表达的是同一件事，后者只是把「创建一个只为了装这段代码的对象」这件事的噪声去掉了。 – **关键规则与最佳实践**： – 这里**没有魔法**：Java 仍然没有一等函数。lambda 只在编译器能验证两件事时可用：(1) 能推断出函数对象的类型（例如 `Thread` 构造器接受 `Runnable`）；(2) 该类型是**函数式接口**——只有一个抽象方法的接口。 – 因此 `Runnable`、`Comparator`、`ActionListener`、自定义的 `NumberListener` 都能写成 lambda；而含两个抽象方法的接口不行。 – 方法引用（如 `System.out::println`）是更进一步的简写，含义是「把这个已有方法当作函数对象」。 – 短小的 lambda 提升可读性；语句过多、逻辑复杂的 lambda 应当抽成命名方法或命名类，否则会打断周围代码的阅读。

监听器模式 / 发布–订阅（Listener Pattern / Publish–Subscribe） – **定义与目的**：监听器模式描述「事件源产生离散事件流，一个或多个监听者订阅这个流并提供事件发生时被调用的函数」这一结构。它让 GUI 库组件（按钮、滚动条、文本框、菜单）各自**自包含地处理自己的输入**，是实现「模块化 GUI」的关键。 – **直观解释（"它是什么？"）**：想象一家出版社（事件源）与许多订户（监听器）：出版社有新刊就发给所有订户，订户各自决定怎么读；出版社不需要知道订户是谁、要做什么。课程称之为 Listener pattern，也叫 Publish–Subscribe。 – **关键规则与最佳实践**： – 一次典型的注册：`playButton.addActionListener(回调)`。这里 `JButton` 是事件源，事件是「按钮被按下」，监听器是那个 `ActionListener` 实例，被调用的函数是 `actionPerformed`。 – 事件往往带**附加信息**，既可能打包成一个**事件对象**（`ActionEvent`、`MouseEvent`、`HttpExchange`），也可能直接作为回调参数传入。 – 事件发生时，事件源把事件**分发给所有已订阅的监听器**，逐个调用它们的回调方法。 – 不只按钮有事件：`JButton` 在按下时发 action 事件（无论鼠标还是键盘）、`JList` 在选择变化时发选择事件、`JTextField` 在文本变化时发变更事件；HTML 中 `<button>` 的 `click` 事件是它收到 `mousedown` 与 `mouseup` 之后合成的。 – 事件源通常提供 `addXxxListener` / `removeXxxListener`（Java）或 `addEventListener` / `on`（TypeScript、Node）来注册与注销。

事件循环与事件队列（Event Loop & Event Queue） – **定义与目的**：事件循环是 GUI 与 JavaScript 运行时的核心：一个 FIFO 的**事件队列**存放各种来源到达的事件，事件循环反复从队列取出事件并调用相应回调。理解它才能理解「为什么监听器必须快速返回」。 – **直观解释（"它是什么？"）**：事件循环像一位只有一个窗口的银行柜员：柜台前有一条队伍（事件队列），柜员每次只服务一位客户（调用一个回调），服务完必须叫下一位。如果某位客户赖着不走（回调长时间不返回），后面所有人都在等待——用户界面就「卡住」了，鼠标变成转圈的沙漏或风车。 – **关键规则与最佳实践**： – Java 的 GUI 库在创建第一个 GUI 对象时就会自动创建一个**事件处理线程**（Swing 中即事件分派线程 event dispatch thread, EDT），它与程序的 `main` 线程不同，负责事件循环并调用监听器。 – 在 Java 里这个循环被隐藏在工具包内部（常常运行在独立的线程上），监听器「看起来像是被魔法调用的」。 – 所有 GUI 代码都跑在事件循环上，因此**所有代码都必须及时回到事件循**

环。 – 定时器到期、GUI 的鼠标/键盘事件、文件与网络 I/O 完成，都是事件的来源，都要经这条队列排队。 – 事件循环的存在意味着：GUI 程序在你看不到的地方**已经**是并发的——这直接引出 Reading 21。

控制反转（Inversion of Control, IoC，"Don't call us, we'll call you"） – **定义与目的**：控制反转指「谁调用谁」的决定权从调用方转移到被调用方：客户端不再主导控制流，而是把代码交给模块，由模块决定调用时机。它服务于可修改性与易理解性，但也把「时序责任」转移给了程序员。 – **直观解释（"它是什么？"）**：课程练习的标题就是那句好莱坞名言「别打给我，我会打给你」。注册监听器之后，你的代码什么时候被执行不由你决定，而由事件源与事件循环决定。 – **关键规则与最佳实践**： – 在客户端视角，这段关系是：客户端创建回调函数 → 传给实现者 → 事件发生时实现者调用它。要能明确回答「客户端是谁、那段代码是什么、模块是谁」。 – 控制反转让**时序不再可预测**：回调可能在你没预期的时候发生（用户点得快、网络来得慢、定时器先到）。 – 因此回调里**不要假设**任何关于「其他回调是否已经跑过」「共享对象处于什么状态」的隐含条件。 – 控制反转是后续 Web 服务器（路由处理器）、异步计算（Reading 22）、并发（Reading 21/23）等所有「回调式系统」的共同前提。

回调中的共享可变状态与再入（Shared Mutable State & Reentrancy） – **定义与目的**：这是本讲最重要、也最容易踩坑的部分。实现者要维护「监听器集合」与「计数/状态」这些**可变的 rep**；当回调在遍历这个集合的过程中反过来修改它（再入，reentrancy），或当回调由另一个线程调用时，共享可变状态就会出问题。 – **直观解释（"它是什么？"）**：`callListeners()` 正在「挨个给订户打电话」，某个订户在电话里说「把我从名单上划掉」——如果名单正在被遍历，这个划掉动作会让遍历器失效。又或者，你一边在遍历名单一边另一个线程在往名单里加人，同样会出问题。 – **关键规则与最佳实践**： – 用**防御性拷贝（defensive copy）**让遍历发生在一个独立的集合对象上：`for (NumberListener l : new HashSet<>(listeners))`，这样 `addNumberListener` / `removeNumberListener` 在回调内被调用也不会破坏遍历（否则会抛 `ConcurrentModificationException`）。 – 让事件源的 rep 从设计之初就**为线程安全做准备**：课程给出的方案是把所有公共方法都标 `synchronized`，采用**监视器模式（monitor pattern）**，rep 由对象锁保护。（锁与监视器模式的系统讲解属于 Reading 23。） – 在监听器规格里说明：监听器**不应注册/注销监听器**，或者说明实现支持这种再入行为；二者必须与实现一致。 – 监听器里的长耗时计算要交给后台线程（Java 中如 `SwingWorker`、或自建 `Thread`），并把界面更新切回事件分派线程（`SwingUtilities.invokeLater`），不要阻塞事件循环。（补充说明：`SwingWorker` 与 `SwingUtilities.invokeLater` 是 Java Swing 生态的标准做法，课程原文以 `setTimeout` 与「使用并发」的方式表达同一思想。）

阻塞事件循环与忙等待（Blocking the Event Loop / Busy-Waiting） – **定义与目的**：事件循环是单线程的服务者，任何「占着控制流不还」的代码都会让它停摆。认识这一点是为了让 GUI 保持响应性（responsiveness），这是可用性层面的硬性要求。 – **直观解释（"它是什么？"）**：`busyWait(milliseconds)` 死盯着时钟自旋到时间到达，期间事件在队列里越堆越多、一个都不处理；等它终于返回，堆积的定时器回调全部**迟到**触发。这种「自旋等待外部变化」的写法叫**忙等待**，在事件循环运行时里是严重的代码坏味道。 – **关键规则与最佳实践**： – 监听器必须**快速运行、快速返回**，通常在几毫秒以内。 – 需要等待外部变化时，应当**注册一个由该变化触发的回调**（如定时器），而不是自旋检查。 – 监听器里做长时间计算（例如一次画一百万个图形）会让整页冻结：鼠标点不动、滚动无效、按键没反应。 – 若监听器确

实要长时间计算，必须使用并发（Reading 21 起的主题）。 – 判断标准很简单：问「这个回调要跑多久？」，超过几毫秒就要重新设计。

代码示例与对比分析

场景 1：GUI 的输入处理——写一个集中式的巨型输入循环 vs 让每个组件自己注册回调

❌ *错误代码*

```
// 错误：把所有输入分派逻辑写在一个无限循环里（伪代码风格，真实 GUI 里这样写会毁掉模块化）
public static void mainLoop(JButton playButton, JButton stopButton, JSlider volumeSlider) {
    while (true) {
        MouseEvent click = readMouseClick();          // 假设存在这样一个底层调用
        int x = click.getX(), y = click.getY();
        if (playButton.contains(x, y)) {
            playSound();
        } else if (stopButton.contains(x, y)) {
            stopSound();
        } else if (volumeSlider.contains(x, y)) {
            volumeSlider.setValue(volumeSlider.getValueFromPosition(x, y));
        } else if (/* ... 系统里还有几十个组件 ... */ false) {
            // ...
        }
    }
}
```

【错误代码的问题】 1. **不模块化**：`mainLoop` 必须知道系统里每一个组件的存在、位置与语义。新增一个按钮就要改这个函数，违反「对修改封闭」的期望，属于可修改性的直接损失。 2. **责任错位**：按钮本该「自包含地处理自己的输入」（它最清楚自己被点中意味着什么），现在却要由外部代码去猜。 3. **无法复用**：这段逻辑只适用于这个特定的界面；GUI 库组件（滚动条、文本框、菜单）本来是通用的，一旦要求外部循环分派，通用性就没了。 4. **不可维护**：成千上万行 `else if` 是典型的「无所不知、无所不控的巨兽」，既不易读也极易出错（漏判、顺序错、坐标判断写错）。

✅ *正确代码*

```
// 正确：每个组件自己处理输入，只把「按下时做什么」作为回调交出去
JFrame frame = new JFrame("Sound Player");
JButton playButton = new JButton("Play");
JButton stopButton = new JButton("Stop");

// 匿名类写法（sp21 原文风格）
playButton.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent event) {
        playSound();
    }
});

// lambda 写法（Java 8+，同一个函数对象，更简洁）
stopButton.addActionListener(event -> stopSound());

frame.setLayout(new FlowLayout());
frame.add(playButton);
frame.add(stopButton);
frame.pack();
frame.setVisible(true);
// 框架内部的输入事件循环负责把鼠标/键盘事件分派到正确的组件，
// 再由组件调用我们注册的回调 —— 我们不再写任何分派代码
```

【为什么这样更好】 分派逻辑被下沉到 GUI 工具包内部，客户端只声明「这个按钮被按下时做什么」。新增按钮不需要改动任何既有代码，只需为新按钮注册回调——这正是「Ready for change」。按钮作为一个自包含组件，自己处理鼠标与键盘输入，客户端代码不依赖坐标或输入设备细节。

【代码对比解说】 两种写法的根本差异是**依赖方向**。巨型循环要求「中心知道所有组件」，是自上而下的强耦合；监听器模式让「组件知道自己的行为」，中心只需要把输入事件广播到组件树。前者的复杂度随组件数线性（甚至超线性）增长在**一个函数**里；后者把复杂度分散到各自独立的注册点。代价是控制流变得「不可见」——监听器看起来像被魔法调用，阅读代码时需要知道事件循环的存在。

【设计原则透视】 这是**抽象边界**的选择：GUI 组件把「输入设备细节」封装起来，只暴露一个「被激活」的抽象事件（action event），客户端不必依赖它的表示（坐标、绘制方式）。它同时体现了**可修改性**的核心手段——把「变化的部分」（行为）以参数（回调）的形式注入，而不是写在实现里。可以说，回调就是「把策略作为一等值传递」，与 Reading 8（不可变性）中「把不变的部分固定、把变化的部分参数化」是同一思路的两种体现。

场景 2：定时回调——忘记递减导致无限循环 vs 递归式倒计时

❌ *错误代码*

```
/** 每 1000 毫秒调用一次 callback，但 ticks 从不递减 —— 会永远打印同一个值。 */
public static void countdown(int ticks, IntConsumer callback) {
    final int millisecondsPerTick = 1000;

    // javax.swing.Timer 是 Java 中与 sp22 的 setTimeout 对应的定时器回调机制，
    // 它的监听器在事件分派线程上被异步调用
    Timer timer = new Timer(millisecondsPerTick, null);
    timer.addActionListener(event -> {
        callback.accept(ticks);              // ticks 永远不变
        if (ticks > 0) {
            timer.restart();                 // 条件永远为真（当 ticks > 0 时），永不停止
        }
    });
    timer.setRepeats(false);
    timer.start();
    callback.accept(ticks);                  // 规格要求第一次同步调用
}
```

【错误代码的问题】 1. **永远不终止**：`ticks` 从不变化，`if (ticks > 0)` 恒为真，回调每秒触发一次直到进程结束——既浪费资源也违反规格（应当在 0 处停下）。 2. **难以察觉**：程序不崩溃、不报错，只是「一直在响」；若在单元测试里运行，测试会直接挂死（也说明回调式代码的测试需要考虑终止性）。 3. **递减位置敏感**：即使补上 `--ticks`，放错位置也会出错——例如在调用 `callback` **之前**递减，会漏掉最后的 0；放在 `if` 判断之后太晚，则又会多跑一轮。位置决定了「最后一次回调能否打印 0」。

✅ *正确代码*

```
/**
 * Start a timer that ticks once per second until it expires.
 *
 * @param ticks duration of the timer in seconds, must be an integer >= 0
 * @param callback callback function, initially called synchronously, then called
 *                 again after each tick of the timer, each time passing the number
 *                 of seconds left until the timer expires.
 *                 ticksLeft must be an integer in [0, ticks].
 */
public static void countdown(int ticks, IntConsumer callback) {
    if (ticks > 0) {
        // javax.swing.Timer：延迟 1000 毫秒后异步调用一次监听器
        Timer timer = new Timer(1000, null);
        timer.setRepeats(false);
        timer.addActionListener(event -> countdown(ticks - 1, callback));
        timer.start();
    }
    callback.accept(ticks);   // 第一次是同步调用：在 countdown 返回之前发生
}
```

【为什么这样更好】 采用**递归式**设计：每次计时到点就调用 `countdown(ticks - 1, callback)`，把「剩余秒数」变成**新的参数**，而不是去修改一个被闭包捕获的可变变量。递归天然地保证了终止（`ticks` 降到 0 时不再登记定时器），也精确实现了规格要求「第一次同步调用，之后每次异步调用，值依次为 ticks, …, 0」。

【代码对比解说】 这里有两个关键教学点。第一，**代数式的状态传递优于可变状态的就地修改**：`countdown(ticks - 1, ...)` 把状态放在参数里，杜绝了「忘记递减」这类错误；可变版本则要求程序员记住在正确的位置、正确的时机改它。第二，规格明确区分了「同步的第一次」与「异步的后续」——`callback.accept(ticks)` 在 `countdown` 返回前执行，因此输出顺序中会先出现 `ticks`，再出现随后的递减值。这种「先同步一次再异步若干次」的模式在真实的进度回调里非常常见。

【设计原则透视】 这体现了**规格的完整性**：调用次数、调用时机、参数取值范围都被写进 `@param callback` 的说明里，客户端的实现才有依据。也预演了 Reading 21 的核心教训——异步回调意味着「共享可变状态」会在不可预测的时刻被读写；此处通过「不共享可变状态」（把状态作为参数传递）回避了该风险，是并发设计的第一个技巧：**优先消除共享可变状态**。

场景 3：事件源的实现——直接遍历监听器集合 vs 防御性拷贝（再入 bug）

❌ *错误代码*

```
public class Counter {
    private BigInteger number = BigInteger.ZERO;
    private Set<NumberListener> listeners = new HashSet<>();

    public interface NumberListener {
        /** Called when the counter changes.
         *  @param number the new number */
        void numberReached(BigInteger number);
    }

    public synchronized void increment() {
        number = number.add(BigInteger.ONE);
        callListeners();
    }

    public synchronized void addNumberListener(NumberListener listener) {
        listeners.add(listener);
    }

    public synchronized void removeNumberListener(NumberListener listener) {
        listeners.remove(listener);
    }

    // 错误：直接在 listeners 字段上迭代；若某个监听器在回调里 remove 自己，迭代器立刻失效
    private void callListeners() {
        for (NumberListener listener : listeners) {
            listener.numberReached(number);
        }
    }
}
```

【错误代码的问题】 1. **再入导致 `ConcurrentModificationException`**：客户端若写成「打印后把自己注销」（`counter.removeNumberListener(this)`），`HashSet` 的迭代器会在下次 `next()` 时抛出

`ConcurrentModificationException`，程序中断。 2. **部分监听器可能收不到通知**：迭代被中断意味着排在其后的监听器**不会被调用**，事件被静默地漏发。 3. **锁不能救它**：Java 的锁是**可重入的**（reentrant），同一个线程再次进入 `synchronized removeNumberListener` 不会被阻塞——所以这不是死锁问题，`synchronized` 解决不了它。这是学生最容易误判的地方。 4. **在别的线程里更糟**：如果计数器由后台线程持续递增，主线程同时注册/注销监听器，问题会以 `ConcurrentModificationException` 或更隐蔽的数据竞争形式出现，且难以复现。

✅ *正确代码*

```
public class Counter {
    private BigInteger number = BigInteger.ZERO;
    private final Set<NumberListener> listeners = new HashSet<>();
    // Abstraction function
    //   AF(number, listeners) = a counter currently at `number`
    //     that sends events to the `listeners` whenever it changes
    // Rep invariant
    //   true
    // Thread safety argument
    //   uses the monitor pattern -- the rep is guarded by this object's lock,
    //   acquired on entering every public method

    public interface NumberListener {
        /** Called when the counter changes.
         *  @param number the new number */
        void numberReached(BigInteger number);
    }

    public synchronized BigInteger number() { return number; }

    public synchronized void increment() {
        number = number.add(BigInteger.ONE);
        callListeners();
    }

    public synchronized void addNumberListener(NumberListener listener) {
        listeners.add(listener);
    }

    public synchronized void removeNumberListener(NumberListener listener) {
        listeners.remove(listener);
    }

    // 正确：在集合的防御性拷贝上迭代，回调中的注册/注销不会破坏本次遍历
    private void callListeners() {
        for (NumberListener listener : new HashSet<>(listeners)) {
            listener.numberReached(number);
        }
    }
}
```

【为什么这样更好】 `new HashSet<>(listeners)` 产生一个独立的快照集合：遍历发生在快照上，`addNumberListener` / `removeNumberListener` 即便在回调内被调用，修改的也是原集合，不会让正在迭代的迭代器失效。语义上这还带来一个明确的规定：**本次事件只通知「事件开始时已注册」的监听器**，本次新注册的监听器从下一次事件起生效。这是可测试、可写进规格的行为。

【代码对比解说】 这个 bug 的隐蔽性在于「单线程、无锁竞争」也会发生：它纯粹是**回调再入**（回调在实现者的临界区内反调实现者的公共方法）造成的。修复方式有两种取向：一是让遍历基于**快照**（本例），二是禁止监听器在回调里注册/注销（在规格里禁止）。前者更宽容、更常用；后者更简单但把责任推给客户端，一旦客户端违反就是 bug。课程选的是前者，并用「防御性拷贝」这个老朋友来实现——同一技巧在 Reading 8（不可变性）里用来保护 rep 不被泄露，这里用来保护**遍历过程**不被并发修改，思想完全一致。

【设计原则透视】 与 AF/RI 显式关联：注释里写出了 `AF(number, listeners)` 与 RI（此处平凡为 `true`，因为 `HashSet` 与 `BigInteger` 自身没有额外的约束）。与**线程安全论证**显式关联：所有公共方法 `synchronized`，rep 由对象锁保护，属于**监视器模式（monitor pattern）**；而 `callListeners()` 是私有方法、只在持锁的公共方法内被调用，因此不需要再同步。（锁、监视器模式与线程安全论证的完整讨论属于 Reading 23。）特别注意：监听器本身**在锁内被调用**，若监听器又去获取别的锁，可能引发死锁——这属于 Reading 23「锁的顺序」要处理的问题。

场景 4：回调的异常处理——以为能在登记处捕获异步回调的异常 vs 明确区分同步/异步

❌ *错误代码*

```
static void alwaysThrows() {
    throw new RuntimeException("boom!");
}

// 错误：以为把登记调用包在 try 里就能捕获回调抛出的异常
public static void main(String[] args) {
    try {
        // javax.swing.Timer：登记回调后立刻返回，一秒后才由事件循环调用
        Timer timer = new Timer(1000, null);
        timer.setRepeats(false);
        timer.addActionListener(event -> alwaysThrows());
        timer.start();
    } catch (RuntimeException e) {
        System.out.println("caught: " + e.getMessage());   // 永远不会执行
    }
    System.out.println("main returns");
}
```

【错误代码的问题】 1. **catch 块永远不执行**：`addActionListener` / `start` 只是**登记**回调，异常要等到一秒后事件循环调用回调时才抛出，那时 `try` 所在的栈帧早已退栈。 2. **异常落进事件循环**：它会由事件分派线程向上传播，而不是回到你的 `main`；程序可能打印栈迹、也可能静默地继续运行后面的 GUI 代码，行为难以预料。 3. **错误的安全感**：这段代码读起来「已经处理了异常」，实际上没有任何处理，属于典型的假防御。

✅ *正确代码*

```
static void alwaysThrows() {
    throw new RuntimeException("boom!");
}

public static void main(String[] args) {
    Timer timer = new Timer(1000, null);
    timer.setRepeats(false);
    timer.addActionListener(event -> {
        try {
            alwaysThrows();                 // 同步回调：可以在调用点周围捕获
        } catch (RuntimeException e) {
            System.out.println("caught in listener: " + e.getMessage());
        }
    });
    timer.start();
    System.out.println("main returns");     // 立刻打印，不等一秒
}
```

【为什么这样更好】 异常处理必须发生在**异常实际抛出的那个调用栈**里：同步回调的调用点在 `main` 的 `try` 内（如 `list.forEach(alwaysThrows)` 的情形），可以被外层捕获；异步回调的调用点在事件循环里，唯一能捕获它的地方是**回调自身的方法体**（或事件循环提供的统一异常处理钩子）。

【代码对比解说】 这条规则可以推广为一个判断方法：**问「这行代码执行时，回调函数在栈上吗？」**。`list.forEach(f)` 会立刻调用 `f`，所以 `f` 的异常会沿 `forEach` 向上传回调用者；`timer.start()` 与 `addEventListener` 不会调用回调，因此包住它们的 `try` 毫无作用。另外注意 `main` 会立刻返回：异步的 `main` 不会等回调——如果进程因为 `main` 返回而退出，回调可能永远没有机会执行（这也是「回调可能被调用零次」的现实来源）。

【设计原则透视】 这仍然是**规格边界**问题：回调接口的规格必须说明调用发生在哪个执行上下文（哪个线程、同步还是异步），调用者才能正确放置异常处理与同步措施。它也直接指向 Reading 21：异步回调的执行上下文往往是**另一个线程**，因此「回调里访问的对象」变成了共享可变状态。

场景 5（补充）：匿名类 vs lambda——同一函数对象的两种写法

❌ *错误代码*

```
// 错误：同一个监听器实现被 Ctrl-C/Ctrl-V 了三份，修改时必须同时改三处
playButton.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent event) { player.play(); }
});
stopButton.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent event) { player.play(); }   // 复制粘贴来的
});
pauseButton.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent event) { player.play(); }   // 又一份
});
```

【错误代码的问题】 1. **违反 DRY**：同一段逻辑有三份副本，任何修改都必须同步三处，遗漏一处就是 bug。2. **噪声淹没了意图**：三处 `new ActionListener() { public void actionPerformed(...) }` 的样板代码让「按下时播放」这个真正的意图不显眼。 3. **可读性下降**：读者需要越过语法噪声才能看清「谁在按下时做什么」。

✅ *正确代码*

```
// 正确：把可复用的行为提取成一个具名方法对象，需要时用方法引用或 lambda 注入
ActionListener play = event -> player.play();

playButton.addActionListener(play);
stopButton.addActionListener(play);
pauseButton.addActionListener(play);

// 只在一个地方使用的一次性行为，就地写 lambda 即可
exitButton.addActionListener(event -> System.exit(0));
```

【为什么这样更好】 用「一个具名的函数对象」表达被复用的行为，既消除重复，又让三个注册点各自只有一行、意图一目了然；一次性行为用就地 lambda，读起来最直接。这也和 Reading 20 的结论呼应：**回调让行为不必硬编码进实现**，那么行为本身也应该像数据一样被合理地组织、命名与复用。

【代码对比解说】 匿名类与 lambda 表达的是同一个对象，取舍在于信息密度与可读性：匿名类适合需要（在 Java 8 之前）明确写出接口与方法的场合，或需要多条语句且有状态的长实现；lambda 适合短小的一次性实现。真正的坏味道不是「用匿名类」或「用 lambda」，而是**复制粘贴**——那说明这段行为应当被提取、命名与复用。

【设计原则透视】 这里对应 Reading 4（代码评审）与 DRY 原则：代码应当只在一个地方表达一个知识。也体现函数对象的价值：既然函数是对象，它就可以被赋给变量、被命名（`ActionListener play`）、被传递多次——这正是「一等函数」带来的组织能力。

与其他设计原则的关联

- **Reading 16（Map/Filter/Reduce）**：本讲明确以它作为「同步回调」的先例——传给 `map` / `filter` / `reduce` 的函数就是一等函数与回调的第一次亮相。两讲的对比（同步 vs 异步）是本讲的核心时间维度。
- **Reading 12（接口、泛型、枚举）**：函数对象由**单抽象方法接口**（`Runnable`、`Comparator<T>`、`ActionListener`、`NumberListener`）定义，lambda 只能用于这类函数式接口；泛型还会出现在事件源的类型签名里（如 `Comparator<Dog>`）。
- **Reading 6/7（规格说明与设计规格）**：回调参数的规格必须写清调用次数、时机（同步/异步）、参数范围与线程；「监听器必须快速返回」也是一条应当写进规格的**性能约定**。
- **Reading 8（不可变性）与 Reading 11（AF/RI）**：`callListeners()` 的防御性拷贝与 Reading 8 中「保护 rep 不被泄露」是同一技巧；`Counter` 的 AF/RI/线程安全论证是 Reading 11 报告格式的直接应用。

- **Reading 9（避免调试）**：忙等待与大段阻塞代码让 GUI 变得不可调试；「避免使用全局可变状态」在本讲体现为「不要在回调之间共享可变状态」。
- **Reading 21（并发）**：本讲直接为之铺垫——GUI 工具包自动创建的事件处理线程、`HttpServer` 创建的网络线程都意味着程序**已经并发**；回调可能在任何时刻、在另一个线程上被调用，「共享可变状态」的风险由此而来。事件处理系统的交错（interleaving）是 Reading 21 的第一个案例。
- **Reading 22（异步编程）与 Reading 23（锁）**：异步回调的异常处理、`Promise` / `CompletableFuture` 式的组合，是 Reading 22 的主题；本讲中 `Counter` 的 `synchronized` 与「监听器在锁内被调用」的死锁风险，将在 Reading 23 系统展开。
- **Reading 25（Socket 与网络）**：sp21 的 `HttpServer` 例子与本讲共用「回调处理输入事件」的骨架：路由处理器 `HttpHandler.handle(HttpExchange)` 与按钮监听器是同一个模式的两个实例。

关键要点

- **回调 = 客户端交给模块调用的代码**；它把控制流方向反转（IoC），因此必须按规格处理「何时被调用、被调用几次、在哪个线程被调用」。
- **区分同步与异步回调**：同步回调的异常可在调用点周围捕获、副作用保证已发生；异步回调两者都不成立，只能在回调自身内部处理异常。
- **在 Java 里用单抽象方法接口 + lambda/方法引用来表达一等函数**；短小的就地 lambda 与具名复用的函数对象各得其所，避免复制粘贴。
- **监听器必须快速返回**：它是事件循环上的一环，长耗时应交给后台线程/并发，绝不要忙等待。
- **事件源的实现要防再入、防并发**：遍历监听器集合时用防御性拷贝；从设计之初就写出线程安全论证（监视器模式）。

常见陷阱与注意事项

- **在监听器里做长耗时计算或忙等待** → 事件循环被阻塞，界面冻结、定时器回调迟到，用户看到沙漏/风车光标。
- **以为把 `addEventListener` / `addActionListener` / `start` 包在 `try` 里能捕获回调异常** → 异步回调的异常不在那条调用栈上，`catch` 永不执行；正确做法是在回调体内捕获。
- **在回调中注销监听器（再入）却让实现直接在集合字段上迭代** → `ConcurrentModificationException`，并且排在其后的监听器收不到事件；用防御性拷贝（`new HashSet<>(listeners)`）解决。
- **以为给方法加 `synchronized` 就能修复再入 bug** → Java 锁可重入，同一线程重入不会阻塞，问题依然存在；同步解决的是跨线程问题，不是再入问题。
- **在回调之间共享可变状态（例如一个共享的 `List` /计数器字段）** → 回调可能由事件线程调用、可能被重复调用，状态会以不可预测的顺序被修改；优先用参数传递状态（如 `countdown(ticks - 1, callback)`）。
- **监听器环与监听器泄漏**：`JTextField` 与 `JSlider` 互相更新对方的监听器若由代码修改触发事件，就可能无限循环（注意规格中「事件仅在用户操作时发出」，并避免在回调里触发自己监听的事件）；另一方

面，长生命周期的 `HttpServer` / `Timer` 持有短生命周期对象的回调引用却忘记注销，会让这些对象无法被回收。

思考题（带答案）

问题 1：下面两段代码都让「一秒后抛出异常」的回调执行。为什么只有第二段的 `catch` 能生效？请从「回调执行时栈上有什么」的角度解释，并说明这对「回调的规格」提出了什么要求。

```
// 第一段：同步回调
try { List.of(1, 2, 3).forEach(n -> { throw new RuntimeException("boom"); }); }
catch (RuntimeException e) { System.out.println("caught"); }

// 第二段：异步回调
try { timer.addActionListener(event -> { throw new RuntimeException("boom"); }); timer.star
catch (RuntimeException e) { System.out.println("caught"); }
```

答案：关键在于回调函数被调用时，哪些栈帧还在。`forEach` 是同步的：它在**执行过程中**调用回调，此时 `try` 所在的 `main` /当前方法的栈帧仍然活着，异常沿 `forEach → 回调` 的调用栈向上传播，能被外层的 `catch` 捕获。`addActionListener` / `start` 是异步的：它们只是把回调对象登记进去并立刻返回，一秒后由**事件分派线程的事件循环**调用回调，那时登记处所在的栈帧早已退栈，异常只会在事件循环的栈上传播，`catch` 自然不生效——唯一能捕获它的地方是回调自己的方法体（或事件循环的统一异常处理钩子）。因此回调的规格**必须**说明：调用是同步还是异步、在哪个线程上发生、可能被调用多少次；否则客户端无法知道异常处理该放在哪里、共享状态需要怎样保护。这也解释了为什么「回调让控制流不再属于你」是本讲反复强调的结论。

问题 2：`Counter` 的实现里，所有公共方法都标了 `synchronized`，`callListeners()` 却仍然可能抛出 `ConcurrentModificationException`。请解释原因，并给出修复方案，说明修复后「一次事件通知哪些监听器」的语义。

答案：原因是**再入（reentrancy）**：`increment()` 持锁调用 `callListeners()`，后者正在用 `HashSet` 的迭代器遍历 `listeners`；某个监听器在自己的回调里调用 `counter.removeNumberListener(this)`（或 `addNumberListener`），这会在**同一个线程**里直接修改 `listeners`，从而使正在使用的迭代器失效，下一次 `next()` 抛 `ConcurrentModificationException`。锁救不了它，因为 Java 的 `synchronized` 锁是**可重入的**：同一线程再次获取同一把锁会立即成功，不会阻塞，所以再入照样发生。修复方案是在一份**防御性拷贝**上迭代：`for (NumberListener listener : new HashSet<>(listeners)) { listener.numberReached(number); }`。这样回调中的注册/注销修改的是原集合，遍历使用的是独立快照，不会失效。由此得到的明确语义是：**一次事件只通知该事件开始时（快照建立时）已注册的监听器**；在本次回调中新注册的监听器从下一次事件开始生效，已被移除的监听器在本次事件中仍会被通知（如果它出现在快照里且尚未被访问）。这个语义应当写进 `Counter` 的规格说明，让客户端可以依赖它。

问题 3：GUI 程序里为什么说「回调自然引出并发」？请结合 Java Swing 的事件处理线程与 sp21 中 `HttpServer` 的行为说明，并指出由此带来的两个具体的共享可变状态风险。

答案：因为回调的**调用方不再是你写的代码**，而是事件源与事件循环，而事件循环通常运行在**另一个线程**上。Java 的 GUI 库在创建第一个 GUI 对象时就会自动创建一个事件处理线程（事件分派线程），它与 `main` 线

程是两个线程，负责读取鼠标/键盘事件并调用监听器；同样地，`HttpServer` 会创建新线程来监听连接、解析 HTTP 请求并调用路由处理器回调。因此即使程序员没有显式写 `new Thread(...)`，程序也已经是并发的：`main` 线程与事件线程同时在运行同一个程序、共享同一片堆内存。由此产生的两个具体风险是：(1) **共享可变对象的竞态**——如果 `main` 线程与事件线程都读写同一个数据结构（例如同时往一个 `ArrayList` 里添加元素、或同时更新同一个计数器字段），交错执行可能导致数据丢失、`ConcurrentModificationException` 或更隐蔽的不一致状态；(2) **再入与锁内调用回调**——实现者常常在持锁的状态下调用监听器（如 `synchronized increment()` 内调用 `callListeners()`），监听器又在回调里回到同一对象，这既可能触发上面的再入 bug，也可能因为监听器去获取另一把锁而形成锁顺序问题，进而在 Reading 23 的语境下引发死锁。这两个风险正是 Reading 21 要正式引入的主题：共享内存模型、交错执行与竞态条件。

Reading 21: 并发（Concurrency）

> 说明：本讲 sp22 原版使用 TypeScript，本笔记按用户要求提供 Java 代码示例；类型/API 与 sp21（6.031 Java 版）原文保持一致。

概述

本讲回答三个问题：**并发是什么**（多个计算同时进行）、**并发单元如何组织与通信**（共享内存模型 vs 消息传递模型；进程 vs 线程；时间分片）、以及**并发为什么危险**（交错执行使「顺序执行」的假象被打破，从而产生**竞态条件（race condition）**）。核心结论是：并发是现代软件不可回避的必需品，但它同时是**安全性**（race condition 极难发现、极难复现）、**易理解性**（人几乎无法预测交错）与**可修改性**（共享内存的选择、消息协议的设计会长期约束系统的演化）三个目标的重大威胁。本讲的主要目的之一是「先让你害怕」——后续 Reading 22（异步编程）与 Reading 23（锁与线程安全）会给出有原则的应对方法。

核心概念与设计原则详解

并发（Concurrency）的定义与必要性 – **定义与目的**：并发意味着**多个计算同时进行**。它不是一个可选的高级特性，而是现代编程的基本处境：网络中的多台计算机、一台机器上的多个应用、一颗芯片上的多个处理器核心，都是并发。写并发程序的能力直接决定系统能否满足性能与交互需求。 – **直观解释（"它是什么？"）**：把程序想成一条流水线，单线程的程序只有一条传送带；并发就是同时开动多条传送带，它们可能共享同一个仓库（内存），也可能通过传送消息协作。 – **关键规则与最佳实践**： – 并发**无处不在**，无论你喜不喜欢：网站必须同时服务多个用户；移动应用需要把部分处理放到云端的服务器上；GUI 几乎总需要「不打断用户」的后台工作（例如编辑器一边让你改代码一边在后台编译）。 – 处理器主频不再增长，取而代之的是每一代芯片**更多的核心**；因此将来想让计算更快，就必须把计算**拆成可并发的片段**（这是课程给出的对未来程序员的核心判断）。 – 判断你的程序是否已经并发，不要只看有没有 `new Thread`：GUI 工具包、Web 服务器、定时器、框架都会在背后创建线程。 – 并发是**手段**（为了响应性、吞吐量、利用多核），不是目的；引入并发必须有明确的理由，因为它会显著提高设计与调试成本。

共享内存模型（Shared Memory Model） – **定义与目的**：在共享内存模型中，并发模块通过**读写共享对象**来交互。它是最自然、最容易上手的模型（因为「通信」就是普通的读写变量），但也因此把「数据被谁改了、什么时候改的」这一负担留给了程序员。 – **直观解释（"它是什么？"）**：几个模块站在同一间房间里，都能看到并改动房间中央的一块白板。图上 A、B 是两个并发模块，蓝色对象是各自私有的（只有一个模块能访问），橙色对象是共享的（两个模块都有它的引用）。 – **关键规则与最佳实践**： – 典型例子：同一台机器上共享同一块物理内存的两个处理器/核心；共享同一个文件系统的两个程序；同一个程序里共享同一批对象的两个**线程**。 – 共享内存的危险范围是「所有可达的共享对象」，包括容器里的元素、对象的字段，而不只是那一个显眼的变量。 – 在 Java 里，线程**自动**就绪于共享内存：线程共享进程内的全部内存；想要「线程私有」的内存反而需要额外努力。 – 共享内存模型的正确性依赖于**同步机制**（锁、原子变量、不可变对象等，见 Reading 23），而不是「小心地写代码」。

消息传递模型（Message Passing Model） – **定义与目的**：在消息传递模型中，并发模块通过**通信通道互相发送消息**来交互：模块把消息发出去，到达某个模块的消息被**排队**等待处理。模块之间不共享可变内存，从结构上减少了「谁什么时候改了共享数据」这类问题。 – **直观解释（"它是什么？"）**：模块之间像邮局通信：你寄一封信，对方按顺序拆信、回信；双方各自的房间互不进入。发信人**不会停下来等回信**——它继续处理自己队列里的其他请求，回信会作为另一条消息到达。 – **关键规则与最佳实践**： – 典型例子：网络中的两台计算机；浏览器与 Web 服务器；即时通信的客户端与服务器；同一台机器上用管道（pipe）连接输入输出的两个程序，如命令行里的 `ls | grep`。 – 进程**自动**就绪于消息传递：新进程天生带有标准输入输出流（Java 中的 `System.in` / `System.out`），可以直接通信。 – 消息传递**并不能消除竞态条件**：交错依然存在，只不过交错的是**消息的顺序**而不是指令的顺序。 – 关键设计教训：**精心选择消息接口的操作粒度**。有 `get-balance` 与 `withdraw` 两个操作并不够——需要 `withdraw-if-sufficient-funds` 这样「判断与修改合为一条消息」的操作。 – 在 Java 中显式使用消息传递需要自己建立队列数据结构（Reading 24 会讲队列）。

进程（Process）与线程（Thread） – **定义与目的**：上面两个模型说的是「模块如何通信」，而模块本身有两种：**进程**与**线程**。理解二者的差异，才能正确判断「共享什么、隔离什么」，从而做出正确的并发设计。 – **直观解释（"它是什么？"）**： – **进程**是运行中程序的实例，与同一台机器上的其他进程**相互隔离**，尤其拥有自己私有的那一段内存。进程抽象是一台**虚拟计算机**：程序感觉整台机器只为自己服务，内存是崭新的。任何程序启动时都会创建一个新进程来承载它。 – **线程**是运行中程序内部的**控制点（locus of control）**：可以理解为「程序正在执行的那个位置，加上通向该位置的调用栈」（所以遇到 `return` 时它能沿栈返回）。线程抽象是一台**虚拟处理器**：创建线程就像在这台虚拟计算机里新造一个处理器，它运行**同一个程序**、共享**同一片内存**。Java 程序启动时有一个线程，它第一步就调用 `main()`，称为**主线程（main thread）**。 – **关键规则与最佳实践**： – 进程之间通常**不共享内存**：一个进程根本无法访问另一个进程的内存或对象；在多数操作系统上共享内存是可能的，但需要特殊手段。跨进程通信天然适合消息传递。 – 线程之间**共享进程内全部内存**：「线程本地（thread-local）」的私有内存需要额外努力；要用消息传递则必须显式建立队列。 – 口诀：**进程 ≈ 虚拟计算机，线程 ≈ 虚拟处理器**。选进程得到隔离（更安全、更重），选线程得到共享（更快、更危险）。 – 在 Java 中启动线程：`new Thread(runnable).start()`，新线程要做的第一件事是调用 `Runnable.run()`；也可以继承 `Thread` 或使用 lambda。

时间分片（Time Slicing） – **定义与目的**：当线程数多于处理器数时，并发是通过**时间分片**「模拟」出来的：处理器在多个线程之间来回切换。它解释了「为什么单核机器上也有并发」，也解释了「为什么交错是不可预测的」。 – **直观解释（"它是什么？"）**：课程配图展示了三个线程 T1、T2、T3 在只有**两个**真实处理器的机器上如何被分片：时间向下流动，起初一个处理器跑 T1、另一个跑 T2，随后第二个处理器切换去跑 T3；T2 就那样暂停，等待它在同一个或另一个处理器上的下一个时间片。图的最右侧显示从**每个线程自己的视角**看：有时它在处理器上活跃运行，有时它被挂起、等待下一次运行机会。 – **关键规则与最佳实践**： – 在大多数系统上，时间分片**不可预测且不确定（nondeterministically）**：线程可能**在任何时刻**被暂停或恢复。 – 因此不能依赖「这两行相邻的代码之间不会被插入别的线程的动作」——交错可以发生在任意指令边界上。 – 分片意味着「线程的执行速度」不是程序能控制的东西：负载、调度策略、其他程序、时钟频率都会影响它。 – 时间片切换点的不可预测性正是**竞态条件难以复现**的根源（heisenbug 的成因之一）。

交错（Interleaving）与「顺序执行的假象」被打破 – **定义与目的**：交错指并发执行时，实际执行序列会把 A 的操作与 B 的操作**任意穿插**。（某些操作甚至可能真正同时发生，但本讲先只讨论交错。）理解交错是理解竞态条件的前提。 – **直观解释（"它是什么？"）**：高层看起来是「一个动作」的语句，其实会分解成若干条更底层的指令。课程用银行账户的例子展示 `deposit()`（把余额加一）的内部步骤：读余额（`get balance`）、加一（`add 1`）、写回结果（`write back`）。两个并发的存款操作可能交错成两种结果： – **安全交错**：A 完整做完（余额 0→1），B 再完整做完（余额 1→2），得 2，两笔存款都在。 – **危险交错**：A 读到 0，B 也读到 0，各自加一，各自写回 1——**A 的一块钱丢了**。原因是两个操作都基于同一个「过时的读」，写回时没有把对方的更新考虑进去（典型的 read–modify–write 丢失更新）。 – **关键规则与最佳实践**： – 交错发生在**低层指令**之间，而不是 Java 语句之间；你看到的一行代码可能对应多条不可分割性未知的机器指令。 – 「顺序执行的假象」是指：单线程时我们可以按代码顺序推理，而并发下这条推理链失效，正确性变成「对**所有**可能交错都成立」。 – 交错的组合数是天文数字，人无法穷举；正确的做法是让代码**不依赖交错**（用同步或消息把临界区变成不可分割的单位）。 – 交错不是罕见事件：只要时序合适就会发生，而且往往在负载高的生产环境才暴露。

竞态条件（Race Condition） – **定义与目的**：竞态条件指**程序的正确性（规格的后置条件与不变量的满足）依赖于并发计算 A 与 B 中事件的相对时序**。当这种情况发生时，我们说「A 与 B 处在竞态之中」。它是本讲要传达的核心危险。 – **直观解释（"它是什么？"）**：有些交错像单进程的顺序执行一样「合法」，会得到正确结果；另一些交错则会产生错误答案——违反规格的后置条件或表示不变量。程序在多数时候跑对，只是因为「运气好」遇上了安全交错。 – **关键规则与最佳实践**： – **改写法救不了它**：`balance = balance + 1`、`balance += 1`、`++balance` 三个版本具有**完全相同的竞态**。你不能从 Java 源码看出处理器会生成什么指令，也不能判断哪些是**原子操作（atomic，不可分割的步骤）**。仅仅因为它是「一行 Java」并不意味着它原子；仅仅因为标识符 `balance` 只出现一次也不意味着它只被触碰一次。典型的现代 Java 编译器为这三个版本生成的代码**完全相同**。 – 核心教训：**你不能靠「看一眼表达式」判断它是否免于竞态**。 – 竞态可以出现在共享内存模型的**指令交错**上，也可以出现在消息传递模型的**消息交错**上（同一枚硬币的两面）。 – 处理竞态的正道是设计（不可变性、限制共享、同步、消息接口设计），而不是「多测几次」。

重排序与内存可见性（Reordering） – **定义与目的**：比交错更糟的是：当使用多个变量与多个处理器时，你甚至不能指望对这些变量的修改**按代码顺序**出现。编译器与处理器为了性能会在寄存器/缓存里做临时副本，写回（storeback）的顺序可能与代码顺序不同。这直接威胁「用一个标志位通知另一个线程」这类设计的正确性。 – **直观解释（"它是什么？"）**：课程的例子中，`computeAnswer()` 先写 `answer = 42`，再写 `ready = true`；另一个线程 `useAnswer()` 忙等 `while (!ready)`，看到 `ready` 为真后检查 `answer`，却发现它仍是 0，于是抛出 `RuntimeException("answer wasn't ready!")`。原因可以理解为处理器实际上创建了两个临时变量 `tmpr`、`tmpa` 分别摆弄 `ready` 与 `answer`，并且先把 `ready` 写回、把 `answer` 留在后面——于是出现了「ready 已置位、answer 还未写入」的窗口。 – **关键规则与最佳实践**： – 不要用「一个共享标志位」在线程之间传递「准备就绪」信号，除非配合明确的同步原语。 – 要等待另一个计算完成，使用 `Thread.join()`、阻塞队列、消息回复、`Future.get()` 这类**有同步语义**的机制，而不是忙等。 – 忙等（beyond 是坏味道）在这里还是**错误**的：它既不保证可见性，也不保证顺序。 – 「先写数据、再写标志」这种直觉在单线程里成立，在并发里必须由内存模型与同步手段来保障。

并发单元之间的关系：并发 / 并行 / 交错（Concurrency, Parallelism, Interleaving） – **定义与目的**：这三个词经常被混用，但它们描述不同层面的事实，区分它们才能准确推理与沟通。 – **直观解释（"它是什么？"）**： – **并发（concurrency）**是「多个计算在时间上重叠发生」这一**结构**性质：多个计算单元同时存在并推进。单核机器上两个线程也是并发的。 – **并行（parallelism）**是「**真正同时**在多个处理器/核心上执行」这一**物理**事实：它需要多个执行资源。并发不要求并行，并行是并发的一种实现方式。 – **交错（interleaving）**是「把多个执行流合并成一个观察到的操作序列」的**观察模型**：当并行度有限（线程数 > 处理器数）时，时间分片把并行模拟成并发，使得从外部看指令序列像是被任意穿插的。并行执行在观察层面同样可以（在同步点之间）被理解为某种交错。 – **关键规则与最佳实践**： – 「并发」不等于「更快」：只有可分解为并行片段的工作才因多核而加速；I/O 密集与交互式程序受益于并发是因为**响应性**，而不是吞吐量。 – 推理正确性时用的模型是**交错**（考虑所有可能的操作顺序），而不是「真同时发生」——交错模型更保守也更安全。 – 线程数多于处理器数时，时间分片是常态；因此即使代码「逻辑上并行」，也可能被分片成任意交错。 – 进程与线程是**并发单元**的两种形态（虚拟计算机 vs 虚拟处理器），共享内存与消息传递是**通信方式**的两种模型——这两组概念相互正交，可以两两组合。

代码示例与对比分析

场景 1：启动一个线程——调用 `run()` 或忘记 `start()` vs 正确调用 `start()`

❌ *错误代码*

```
public class Parcae {
    public static void main(String[] args) {
        Thread nona = new Thread(new Runnable() {
            public void run() { System.out.println("spinning"); }
        });
        nona.run();   // bug! 直接调用 run()，没有启动新线程

        Runnable decima = new Runnable() {
            public void run() { System.out.println("measuring"); }
        };
        decima.run(); // bug? 也许本意是要创建一个 Thread?
    }
}

public class Moirai {
    public static void main(String[] args) {
        Thread clotho = new Thread(new Runnable() {
            public void run() { System.out.println("spinning"); }
        });
        clotho.start();
        new Thread(new Runnable() {
            public void run() { System.out.println("measuring"); }
        }).start();
        new Thread(new Runnable() {
            public void run() { System.out.println("cutting"); }
        });
        // bug! 这个线程对象被创建了，但从未 start()
    }
}
```

【错误代码的问题】 1. **`nona.run()` 完全没有创建新线程**：它只是**在当前线程（主线程）里同步地**执行了 `run()` 的方法体。输出的 "spinning" 会出现在 `main` 的执行序列中，与其他代码顺序执行，没有任何并发，也没有交错的可能（程序仍然「看起来对」，所以这个 bug 极其隐蔽）。 2. **忘记 `start()`**：`Moirai` 里第三个 `Thread` 对象被创建了却从未启动，于是 "cutting" **永远不会**被打印。程序不报错，只是少了一部分行为。这属于「行为悄悄丢失」类 bug。 3. **可运行线程数被误判**：`Moirai` 创建了 3 个 `Thread` 对象，但只有 2 个线程真正运行；最大同时运行的线程数是 3（主线程 + 两个新线程）而不是 4。对并发规模的错误估计会让性能分析与正确性推理全盘偏离。 4. **测试假象**：在 JUnit 中，如果新线程抛出自旋异常，它**不会**让测试失败（见场景 3 的解说），这类「线程没跑/线程跑挂了」的错误更难被发现。

✅ *正确代码*

```
public class MoiraiFixed {
    public static void main(String[] args) {
        Thread clotho = new Thread(new Runnable() {
            public void run() { System.out.println("spinning"); }
        });
        clotho.start();   // 启动新线程，异步执行 run()

        new Thread(new Runnable() {
            public void run() { System.out.println("measuring"); }
        }).start();

        // 常见惯用法：把短的、一次性的实现写成 lambda
        new Thread(() -> System.out.println("cutting")).start();
    }
}
```

【为什么这样更好】 `start()` 才会真正创建一个新的**虚拟处理器**：它让新线程异步地调用 `run()`，并立刻返回，主线程继续执行。这样三个打印分别由三个线程完成，顺序不确定（`spinning measuring cutting` 的任意排列都可能），这才是本讲想让你看到的真并发。

【代码对比解说】 `run()` 与 `start()` 的区别是 Java 并发里最经典的一课：`run()` 只是一个普通方法，直接调用它得到的是**顺序执行**；`start()` 才是「创建线程」这个语义动作。还要理解创建 `Thread` 对象与启动线程是**两件事**：`new Thread(...)` 只是造了一个对象，`start()` 才让它跑起来。两处 bug 合起来说明：并发代码的失败模式往往是**静默的**（少打印一行、顺序变了），而不是抛异常。

【设计原则透视】 这体现了**抽象边界**：`Thread` 这个抽象承诺「模拟一个新的处理器」，而 `run()` 是运行在该处理器上的入口点；绕过 `start()` 直接调 `run()`，等于越过抽象边界使用了实现细节。也对应 Reading 6（规格说明）：`start()` 的后置条件包含「新线程已经开始执行」，而 `run()` 只是普通方法调用，两者的规格完全不同。

场景 2：银行账户的共享内存竞态——`balance = balance + 1` 的三个版本 vs 使读–改–写成为临界区

❌ *错误代码*

```
// 共享内存模型：所有「柜员机」共享同一个账户
// 版本 1
private static int balance = 0;
private static void deposit()  { balance = balance + 1; }
private static void withdraw() { balance = balance - 1; }

// 版本 2
private static void deposit2()  { balance += 1; }
private static void withdraw2() { balance -= 1; }

// 版本 3
private static void deposit3()  { ++balance; }
private static void withdraw3() { --balance; }

// 每个柜员机跑一串「存一块、取一块」的交易，余额本该不变
public static void cashMachine() {
    new Thread(new Runnable() {
        public void run() {
            for (int i = 0; i < TRANSACTIONS_PER_MACHINE; ++i) {
                deposit();    // 放一块钱进去
                withdraw();   // 再取出来
            }
        }
    }).start();
}

public static void main(String[] args) {
    for (int i = 0; i < NUMBER_OF_CASH_MACHINES; ++i) {
        cashMachine();
    }
    // 期望 balance == 0，但常常不是 0
}
```

【错误代码的问题】 1. **丢失更新（lost update）**：`deposit` 在低层分解为「读 balance → 加 1 → 写回」。两个线程若都先读到 0、各自加一、各自写回 1，则只净增了 1，另一笔存款凭空消失。本题里存入与取出各半，最终余额常偏离 0。 2. **三个版本完全一样**：`balance + 1`、`+= 1`、`++balance` 具有**相同的竞态**，典型的现代 Java 编译器为三者生成完全相同的代码——所以这不是「写法不够小心」的问题。 3. **不可判断性**：仅凭源码无法得知哪些步骤是原子的；因此无法通过「读代码」来确认安全性。 4. **难以复现**：错误依赖时序，往往在高负载、多核、恰好分片切换的时刻才发生，测试时经常「跑一次对、跑一次错」。

✅ *正确代码*

```
/**
 * 共享内存模型的正确做法（其完整理论在 Reading 23 展开）：
 * 让「读-改-写」这个复合动作成为不可分割的临界区，
 * 使得并发的交错无法插入到它内部。
 */
public class Account {
    private int balance = 0;   // 受 this 对象的锁保护

    public synchronized void deposit()  { balance = balance + 1; }
    public synchronized void withdraw() { balance = balance - 1; }
    public synchronized int balance()   { return balance; }

    // Thread safety argument:
    //   the monitor pattern —— rep 由 this 对象的锁保护，
    //   每个公共方法在进入时获取锁、退出时释放；
    //   因此「读 balance - 加一 - 写回」整体对其他线程不可分割。
}
```

```
// 补充说明：Java 还提供了原子变量类（java.util.concurrent.atomic），
// 用一条不可分割的指令完成读-改-写（这类 API 的完整讨论超出本讲）
private static final AtomicInteger balance = new AtomicInteger(0);
private static void deposit()  { balance.incrementAndGet(); }
private static void withdraw() { balance.decrementAndGet(); }
```

【为什么这样更好】 竞态的根源不是「两行代码写得太近」，而是「一个复合动作被别的线程插进来了」。把整个读-改-写放进临界区，交错就只能发生在临界区**之间**，而不再能插入其**内部**，于是余额的每一次变化都是不可分割的，任何交错都得到与顺序执行一致的结果。注意：`synchronized` 是 Java 语言层面的机制，**锁、临界区、监视器模式与线程安全论证属于 Reading 23**，此处作为「本讲问题的标准答案预告」给出；本讲本身只要求你认识到问题的存在与严重性。

【代码对比解说】 本例的教学价值在于**打破一个直觉**：你把 `deposit()` 和 `withdraw()` 写成两行相邻的代码，并不等于它们会作为整体被执行；甚至 `balance = balance + 1` 这一行也不等于一个原子动作。课程用「低层指令交错表」把这一点可视化：安全的交错（A 全部做完再做 B）与危险的交错（两边都读到 0）会产生不同结果，而程序**无法选择**哪一种交错。还有一个重要教训：插入一句 `System.out.println(balance)` 常常让 bug「消失」——因为打印比算术慢 100–1000 倍，时序被改变了，交错被掩盖了，但**并没有修好**。

【设计原则透视】 这是**表示不变量（RI）**在并发下的失效：单线程时「余额等于所有已完成交易之和」是一个不变量，竞态会让它被违反。修复的本质是让不变量在「操作的边界」上成立——这正是 Reading 23 中「用锁保护 RI」的思想。也可从**抽象边界**看：`deposit()` 的规格承诺「余额增加一元」，这一原子性承诺必须由实现来保证，而不能靠客户端的调用方式。

场景 3：用共享标志位通知「算好了」——忙等 + 重排序 vs `join()` 或消息传递

❌ *错误代码*

```
// 错误：用共享可变标志位在线程之间传递「准备就绪」，还用了忙等
private boolean ready = false;
private int answer = 0;

// 在线程 1 中运行
private void computeAnswer() {
    // ... 计算很久 ...
    answer = 42;
    ready = true;
}

// 在线程 2 中运行
private void useAnswer() {
    while (!ready) {          // 忙等：既耗费 CPU，也不保证可见性与顺序
        Thread.yield();
    }
    if (answer == 0) {
        throw new RuntimeException("answer wasn't ready!");   // 真的可能抛出来
    }
}
```

【错误代码的问题】 1. **重排序导致「标志已置位、数据未写入」**：编译器和处理器会在寄存器/缓存里做临时副本，写回顺序可能与代码顺序不同。课程给出的等价画面是：`computeAnswer` 先把 `ready` 写回、把 `answer` 的写回留在后面，于是另一个线程看到 `ready == true` 时 `answer` 仍是 0，抛出 `RuntimeException("answer wasn't ready!")`。 2. **可见性没有保证**：即便顺序没问题，一个线程的写也不保证被另一个线程及时看到——除非有同步原语建立 happens-before 关系。 3. **忙等是坏味道也是坏实现**：`while (!ready) Thread.yield()` 空转耗费 CPU，且在本例中它**不能修复**正确性，只是碰运气。 4. **错误依赖「代码顺序即执行顺序」**：这是单线程直觉被错误迁移到并发场景的典型表现，属于易理解性层面的陷阱（代码看起来很对，读代码的人却无法判断它是否对）。

✅ *正确代码*

```
// 正确一：用 join() 等待另一个线程结束，由 JVM 建立同步关系
public class Answer {
    private int answer = 0;   // 只在 compute 线程写、在 join 之后读

    public void compute() {
        // ... 计算很久 ...
        answer = 42;
    }

    public static void main(String[] args) throws InterruptedException {
        Answer a = new Answer();
        Thread worker = new Thread(() -> a.compute());
        worker.start();
        worker.join();               // 阻塞直到 worker 结束；join 返回后能看到它的全部写入
        System.out.println(a.answer);   // 保证打印 42
    }
}
```

```
// 正确二：用消息传递代替共享标志位 —— worker 把结果作为消息发回，主线程从队列取
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.ArrayBlockingQueue;

public class AnswerByMessage {
    record AnswerMessage(int value) { }   // 消息：结果本身，而不是「结果已就绪」的标志

    public static void main(String[] args) throws InterruptedException {
        BlockingQueue<AnswerMessage> inbox = new ArrayBlockingQueue<>(1);
        new Thread(() -> {
            int result = 42;              // ... 计算很久 ...
            inbox.add(new AnswerMessage(result));   // 把答案作为消息发出
        }).start();

        AnswerMessage message = inbox.take();       // 队列的 take() 自带同步语义
        System.out.println(message.value());        // 保证打印 42
    }
}
```

【为什么这样更好】 两种正确写法都把「等待」交给**具备同步语义的原语**：`join()` 保证返回之后能看到目标线程的全部写入；`BlockingQueue` 的 `put` / `take` 保证消息的传递伴随可见性与顺序。更重要的是第二种写法在**设计上**消除了「共享可变标志位 + 共享数据」这个脆弱组合——传的是**消息（结果本身）**，而不是「一个标志说别的地方有数据」。

【代码对比解说】 错误版本有两个独立缺陷：**可见性/顺序**（需要同步原语）与**忙等**（浪费 CPU、也让代码更难理解）。正确写法用一个原语同时解决二者。要注意 `join()` 与 `Thread.yield()` 的区别：`yield()` 只是「提示调度器让出时间片」，不建立任何同步关系；`join()` 是一个有明确规格的同步操作。同理，`BlockingQueue.take()` 在队列空时会**阻塞并挂起线程**（不消耗 CPU），这正是消息传递模型里「模块把到达的消息排队等待处理」的 Java 落地方式（队列的深入讨论属于 Reading 24）。

【设计原则透视】 共享内存模型下，「谁先写谁后写」需要同步手段来保证，而不能靠代码顺序；消息传递模型把这件事变成**通信本身**的语义——用「消息」而不是「共享变量 + 标志」来传递结果，等于把状态**限制在单个线程内**（线程限制，thread confinement），从而大幅降低竞态面。这也是 Reading 21 结尾强调的：好的并发设计应当让程序员**不必去思考交错**（易理解性目标）。

场景 4：消息传递的接口设计——两步式 `get-balance` + `withdraw` vs 一步式 `withdraw-if-sufficient-funds`

❌ *错误代码*

```
// 错误：账户模块暴露两个独立的消息，客户端自己组合成「检查再取款」
get-balance
if balance >= 1 then withdraw 1
```

```
// Java 侧的等价错误：把「检查」与「扣款」分成两次调用
public class Account {
    private int balance = 0;
    public int getBalance() { return balance; }
    public void withdraw(int amount) { balance -= amount; }   // 不检查余额
}

// 两个柜员机 A、B 同时想取走账户里唯一的一块钱
int b = account.getBalance();          // A、B 都可能读到 1
if (b >= 1) {
    account.withdraw(1);               // 于是两笔取款都执行，账户被透支！
}
```

【错误代码的问题】 1. **检查与使用之间的时间窗（TOCTOU）**：`getBalance()` 与 `withdraw()` 是两条独立消息，二者之间可以插入 B 的取款。若账户起初只有 1 元，A 与 B 都读到 1，都认为「余额足够」，于是**透支**——这在银行业务中是严重的正确性事故。 2. **交错的对象变成了消息**：共享内存里交错的是低层指令，这里交错的是**发往账户的消息顺序**。危险依然存在，只是换了形态——「消息传递能消除竞态」是错误认知。 3. **把不变量的维护责任推给了客户端**：「余额不得为负」是账户的不变量，却要由每个客户端自己检查，一旦有客户端忘了检查就破坏不变量。 4. **抽象边界过窄**：只暴露 `withdraw` 迫使客户端做「读—判断—写」这一复合动作，而复合动作恰恰是不安全的来源。

✅ *正确代码*

```
// 正确：把「判断资金是否充足」与「扣款」合成一条消息
withdraw-if-sufficient-funds 1
```

```
/**
 * 账户模块：把复合动作封装成单一操作，
 * 由账户自己（在其内部）原子地完成「检查 + 扣款」。
 */
public class Account {
    private int balance;

    public Account(int initialBalance) {
        if (initialBalance < 0) throw new IllegalArgumentException("negative balance");
        this.balance = initialBalance;
    }

    /** @param amount must be > 0
     *  @return true if the withdrawal succeeded and the balance was reduced;
     *          false if the balance was insufficient (balance unchanged) */
    public synchronized boolean withdrawIfSufficientFunds(int amount) {
        if (amount <= 0) throw new IllegalArgumentException("amount must be positive");
        if (balance < amount) return false;   // 不变量：余额永不为负
        balance -= amount;
        return true;
    }

    public synchronized int balance() { return balance; }
}

// 客户端只需要发一条消息，并根据布尔回复决定行为
boolean ok = account.withdrawIfSufficientFunds(1);
if (!ok) System.out.println("insufficient funds");
```

【为什么这样更好】 复合动作被移进账户模块内部，成为**一条消息、一个原子操作**，客户端之间再也无法插进「检查」与「扣款」中间。不变量「余额永远不为负」由账户自己守护，而不是寄希望于每个客户端都记得检查。返回值让客户端仍能得知结果（消息传递中「回复」也是一条消息）。

【代码对比解说】 这是本讲最有工程价值的教训：**并发设计很大程度上是接口设计**。当接口把一个复合动作拆成几步暴露出去时，客户端就必须自己去保证那几步之间不被干扰——而客户端**做不到**这件事（它无法控制其他客户端的消息何时到达）。把动作合并成一条消息，就让「不可分割性」成为服务方的职责。这也是消息传递模型优于共享内存的地方之一：你可以在协议层面**只提供安全的操作**，而不像共享内存那样必须暴露每一个字段的读写。同样的思路在阅读材料的练习里被明确点出：`withdraw-if-sufficient-funds` 比单纯的 `withdraw` 是更好的操作。

【设计原则透视】 与**规格说明**直接相关：「余额不为负」是账户的**不变量**，应当由实现来保证（前置条件只在客户端可控时才有意义——而这里客户端不可控）。与**抽象边界**相关：账户的抽象应当提供「有意义的原子业务操作」，而不是把表示层的读写原语暴露出去（对比 Reading 10/11 中「不做表示暴露」的思想）。与**线程安全**相关：`synchronized` 使这条消息不可分割，属于监视器模式的预览（Reading 23）。

与其他设计原则的关联

- **Reading 20（回调与 GUI）**：本讲是它的直接延续。GUI 工具包在创建第一个组件时就会创建事件处理线程，`HttpServer` 也会创建网络线程，因此**回调式系统天生并发**：回调可能在任意时刻、在另一个线程上被调用，监听器之间共享的可变状态就成了本讲要讨论的竞态对象。
- **Reading 8（不可变性）**：不可变对象没有「写」，因此不可能出现本讲的丢失更新问题——「让共享数据不可变」是避免竞态最有力的手段之一，会在后续并发设计中反复使用。
- **Reading 11（抽象函数、表示不变量；以及线程安全论证）**：竞态的本质是**不变量被并发操作破坏**；本讲中 `Counter` / `Account` 的线程安全论证（监视器模式）是 Reading 11 报告格式在并发语境下的延伸。
- **Reading 6/7（规格说明与设计规格）**：本讲中「正确性依赖于相对时序」的定义本身就是以规格的后置条件与不变量为参照的；场景 4 更说明接口（规格）设计直接决定并发安全性。
- **Reading 22（异步编程）与 Reading 23（锁与线程安全）**：本讲只负责「吓你一下」——指出问题；Reading 23 提供锁、临界区、监视器模式、死锁与锁顺序等系统性答案；Reading 22 讨论异步回调与 `Promise` / `CompletableFuture` 式的组合（Java 对应写法见该讲）。
- **Reading 24（队列）**：消息传递模型在 Java 中落地需要队列数据结构（`BlockingQueue` 等），那是 Reading 24 的主题；本讲的「消息排队等待处理」正是它的语义来源。
- **Reading 25（Socket 与网络）**：跨机器的并发是消息传递模型最自然的实例，也是 Reading 20 中 `HttpServer` 是并发系统的原因。

关键要点

- **并发是必需品但也是危险源**：处理器主频不再提升、核心数不断增加，要让计算更快就必须拆出并发；但并发直接威胁 Safe from bugs / Easy to understand / Ready for change 三个目标。
- **通信有两种模型，单元有两种形态**：共享内存 vs 消息传递描述「怎么通信」，进程（虚拟计算机）vs 线程（虚拟处理器）描述「谁在并发」；两组概念正交，可两两组合。
- **交错打破了顺序执行的假象**：一行代码不等于一个原子操作，`balance+1`、`+=1`、`++balance` 三个版本编译器生成同样的代码——**无法凭代码外观判断是否免于竞态**。
- **竞态条件 = 正确性依赖事件的相对时序**：有些交错得到正确结果，有些违反后置条件或不变量；重排序还会让「先写数据再写标志」这类直觉失效。
- **设计胜于测试**：并发 bug 是 heisenbug，不可复现、打印与调试器会让它消失；正确道路是消除共享可变状态、用不可变对象、用同步原语、把复合动作设计成单一消息。

常见陷阱与注意事项

- **调用 `run()` 而不是 `start()`，或创建了 `Thread` 却忘记 `start()`** → 代码根本不并发（前者）或部分行为永不发生（后者）；程序不报错，只是结果或输出静默地不对。
- **以为「一行 Java 代码」是原子的** → `balance = balance + 1` 在并发下丢失更新；`+=` 与 `++` 同样不安全，三种写法等价。
- **用共享标志位 + 忙等在线程间传递「已就绪」** → 重排序与可见性问题可能导致「标志已置位而数据未写入」，从而抛出莫名其妙的异常；应改用 `join()`、阻塞队列、消息回复等有同步语义的原语。

- **认为消息传递就没有竞态** → 交错只是从「指令交错」变成「消息交错」；`get-balance` 后再 `withdraw` 依然会透支，接口必须提供 `withdraw-if-sufficient-funds` 这类原子操作。
- **用打印语句或调试器「验证」并发代码** → 打印比正常操作慢 100–1000 倍，会显著改变时序从而**掩盖** bug（heisenbug 看起来消失了），但并未修复；生产环境时序一变就会重现。
- **把「测试通过」当作并发正确性的证据**：在线程里抛出的异常不会传播到 JUnit 的测试线程，`new Thread(() -> { throw new Error("oops"); }).start();` 的测试照样通过（若 JUnit 结束时调用 `System.exit()`，慢线程中的栈迹甚至可能来不及打印）；而并发 bug 本身也极难用测试发现、更难定位，需要显式收集线程内的失败（如 `Future`、队列回传异常）并做有针对性的并发设计。

思考题（带答案）

问题 1：假设 `int balance = 0`，两个线程 A、B 各自执行 `balance = balance + 1;`。请写出所有可能的最终值，并解释为什么 `balance += 1` 与 `++balance` 并不会让情况变好。

答案：`deposit` 在低层分解为三步：读 `balance`、加 1、写回结果。设 A 的三步为 a1（读到 0）、a2（算得 1）、a3（写回 1），B 同理。若交错使得两个「读」都发生在任何「写」之前（例如 a1、b1、a2、b2、a3、b3，或 a1、b1、a3、b3 交错在中间），则两次写回的都是 1，最终 `balance == 1`——一次更新被丢失；若 A 的写回发生在 B 的读之前（a1、a2、a3、b1、b2、b3），则最终 `balance == 2`。因此最终值可能是 1 或 2，取决于交错的相对时序，这正是**竞态条件**：正确性依赖于并发计算中事件的相对时序。`balance += 1` 与 `++balance` 并不会更好，因为「复合赋值」与「自增」只是 Java 语法上的简写，它们同样必须完成「读—改—写」这个复合动作；Java 语言并不承诺把一个复合赋值编译成一条原子指令。事实上课程明确指出：典型的现代 Java 编译器会为这三个版本生成**完全相同**的代码。这就得出本讲的核心教训——你不能通过观察表达式的写法来判断它是否安全，它是否原子取决于编译器与处理器生成的底层操作。要修复它，必须让「读—改—写」整体成为不可分割的临界区（如 `synchronized`），或使用原子变量类（`AtomicInteger.incrementAndGet()`）等专门提供原子性的设施。

问题 2：课程说「进程像一台虚拟计算机，线程像一个虚拟处理器」。请据此说明：为什么线程天生适合共享内存模型，而进程天生适合消息传递模型？并解释「并发」「并行」「交错」三者的关系。

答案：进程是运行中程序的实例，与同机其他进程相互隔离，拥有自己私有的内存段，因此一个进程根本无法访问另一个进程的内存或对象——它在抽象上就是一台**独立的计算机**，跨进程通信自然只能靠网络式/管道式的**消息传递**（新进程天生带标准输入输出流）。线程则是进程内部的**控制点**：创建线程相当于在这台虚拟计算机里新造一个处理器，它运行同一个程序、共享同一个进程的全部内存——因此在抽象上它天生就绪于**共享内存**（想要线程私有的内存反而需要额外努力）。二者恰好互补：进程得到隔离（更安全、更重），线程得到共享（更快、更危险）。关于三个概念的关系：**并发**是结构性质，指多个计算在时间上重叠地推进——单核上的两个线程也构成并发；**并行**是物理事实，指多个处理器/核心**真正同时**执行，它需要多个执行资源，是并发的一种实现方式（并发不要求并行）；**交错**是观察模型，指把多个执行流合并成一个操作序列来看，当线程数多于处理器数时由**时间分片**模拟出并发，使指令序列看起来被任意穿插。推理正确性时应当采用交错模型（考虑所有可能的操作顺序），因为它比「真同时发生」更保守、更安全：任何在交错模型下正确的实现，在真并行下也正确。

问题 3：账户模块原本提供 `getBalance()` 与 `withdraw(amount)` 两个操作，两个客户端可能同时透支账户。请说明错误发生的机制，给出修复后的接口设计，并解释为什么「客户端的代码写得再小心」也解决不了

这个问题。

答案：机制是「检查与使用之间的时间窗」：客户端先发送 `get-balance` 消息读到余额为 1，判断「足够取 1 元」，然后发送 `withdraw 1`。在这两条**独立消息**之间，另一个客户端的消息可以插进来，它也读到 1、也判断足够、也取走 1 元，于是账户余额变成 −1，破坏了「余额不得为负」这个不变量。这与共享内存里的指令交错是同一个问题的两种形态：交错的对象由「指令」变成「消息」，竞态并未消失。修复方式是**把复合动作合并成单一操作**：账户只提供 `withdrawIfSufficientFunds(amount)`，由账户自己在内部原子地完成「检查资金 + 扣款」，并返回布尔值告诉调用者成功与否（例如 `public synchronized boolean withdrawIfSufficientFunds(int amount)`：先检查 `amount > 0`，若 `balance < amount` 返回 `false` 且不改动余额，否则扣款并返回 `true`）。这样「检查」与「扣款」之间不存在任何可被其他消息插入的窗口。客户端写得再小心也没用的原因在于：客户端**无法控制其他客户端的消息何时到达**，它也无法让自己的两条消息「一起」送达——原子性是服务方的职责，只有把动作合并在一条消息内部，服务方才能保证它不可分割。反过来说，接口只暴露 `withdraw` 就等于把「维护不变量」的责任推给所有客户端，任何一个客户端疏忽（或未来新增的客户端疏忽）都会破坏账户的不变量。这正是本讲最有价值的工程结论：**并发安全性很大程度上是接口设计与规格设计的问题**，而不只是加锁的问题。

Reading 22: 承诺与异步计算（Promises）

> 说明：本讲 sp22 原版使用 TypeScript，本笔记按用户要求提供 Java 代码示例；类型/API 与 sp21（6.031 Java 版）原文保持一致。凡属 Java 生态的补充机制（`Future`、`CompletableFuture`、`ExecutorService`、`ReentrantLock` 等）均显式标注为「Java 类比 / 补充说明」，而 promise 三态（pending / fulfilled / rejected）、`then()` 组合、`async` / `await` 语法糖、`Promise.all` / `Promise.any` / `Promise.race`、事件循环与协作式并发等概念以 sp22 原文为准。

概述

本讲讨论如何用「承诺（promise）」这一抽象来表示**已经启动但可能尚未完成的计算**，以及如何用 `await` 运算符和 `async` 函数声明，让并发代码写起来几乎和顺序代码一样自然。核心设计原则有三条：异步计算的结果必须被封装成一个**一等值**（promise）而不是靠回调层层嵌套；读取这个值只能通过 `then()` 或 `await`（绝不允许忙等待或窥探状态）；以及所有交错都只发生在 `await` 这个明确的让出点上，因此可以靠「互斥区间」推理。它与软件构造三大目标的关系是：**Safe from bugs**——promise 的静态类型与「必须 await/then 才能取值」的规则，保证依赖异步结果的代码不可能在结果就绪前继续运行；**Easy to understand**——`await` 把异步代码还原成直线式的同步代码，避免了回调地狱；**Ready for change**——promise 可以组合、聚合、串联，这是线程与 worker 难以做到的。

核心概念与设计原则详解

并发、并行与异步（Concurrency, Parallelism, Asynchrony） – 定义与目的：**并发（concurrency）**指多个计算在时间上重叠地推进；**并行（parallelism）**指它们在物理上真的同时执行（需要多核）；**异步（asynchronous）**是接口层面的性质——一个异步函数在计算完成**之前**就把控制权还给了调用者。三者互不等同：单线程的 JavaScript 可以有大量并发但不是并行；一个同步函数调用也可以发生在多线程程序里。区

分它们是理解本讲的起点，因为 promise 解决的是「如何表示并组合异步计算」，而不是「如何制造更多 CPU 并行度」（后者是 Reading 21 与 Reading 23 中的线程话题）。 – **直观解释（"它是什么？"）**：把并发想成「餐厅里一个服务员同时照看五桌客人」（重叠推进），把并行想成「雇了五个服务员」（同时物理执行），把异步想成「点了菜之后服务员不会站在你桌边等厨房做好，而是先去招呼别人」。异步是一种**礼貌的返回方式**：我先把「菜会端上来」这个承诺交给你，然后我就去干别的了。 – **关键规则与最佳实践**： – 先问「我需不需要并发」，再问「我需不需要并行」；异步不等于更快，它只是不浪费等待时间。 – 异步函数的返回值不是一个值，而是一个**尚未就绪的值的表示**；调用方必须显式地处理这种「未就绪」。 – 在单线程环境中，任何一处阻塞都会冻结**整个程序**；在多线程环境中，阻塞只冻结**当前线程**。混淆这两种后果是并发 bug 的头号来源。 – 不要在同步函数里偷偷做长时间的异步工作；接口的同步/异步性质必须写进规格说明。

承诺抽象与三种状态（Promise and Its Three States） – **定义与目的**：sp22 原文：*A promise represents a concurrent computation that has been started but might still be unfinished, whose result may not be ready yet.* 类型是泛型的：`Promise<T>` 表示一个「将来应当产生类型 T 的值」的并发计算。它是可变的，且有且仅有三种状态：**pending（进行中）**、**fulfilled（已完成，持有 T 类型的值）**、**rejected（已失败，持有描述失败的 Error 对象）**。它把「结果还没到」这件事变成了一个可以传递、可以组合、可以等待的一等值，从而支撑起安全性与可理解性。 – **直观解释（"它是什么？"）**：promise 就是一张**取餐号小票**。你不会站在柜台前死死盯着厨房（忙等待），而是拿着小票去坐下；厨房做好了会叫你的号（fulfilled），做失败了会告诉你原因（rejected）。小票一旦被叫号，就再也不会变回「等待中」——它没有「重置」这个操作。也可以把 `Promise<T>` 类比成一个**最终只装一个元素的列表**：`then(g)` 就像 `map(g)`，元素到达后立刻被 `g` 处理。 – **关键规则与最佳实践**： – 状态转移是**单向的、一次性的**：pending → fulfilled 或 pending → rejected，之后不再改变；一个 promise 也可能永远停在 pending（例如等待一个永不发生的事件）。 – **消费者（consumer）**只能通过 `then()` 或 `await` 读取值；**承诺者（promiser）**才拥有 `resolve()` / `reject()` 这两个修改器。这个「读写权限分离」是 promise 设计的关键安全属性。 – promise 上**没有任何直接观察器**：没有 `isPending()`，也没有 `get()`。这不是疏漏，而是刻意的设计（见下文「绝不忙等待」）。 – 创建 promise 的函数**立即返回**：`readFile`、`diskSpace`、`fetch`、`timeout` 都是启动计算后马上返回小票，而不是等计算结束。 – 若一个 promise 被 reject 而无人处理，就是一个被静默丢弃的异常——在 Java 类比里对应「`Future` 的异常永远没人去 `get()` 解包」。

异步函数与 async/await 语法糖（Asynchronous Functions and async/await） – **定义与目的**：sp22 原文：*An asynchronous function is a function that returns control to its caller before its computation is done.* 用 `async` 声明的函数**必须**返回 `Promise`；`await` 是一个内置运算符，把 `Promise<T>` 变成 `T`：它等到 promise 被 fulfilled，然后拆包取出值；如果 promise 被 rejected，`await` 就**抛出**那个 Error 对象。它的目的是让异步代码在读者眼里退化为普通的直线代码。 – **直观解释（"它是什么？"）**：`await` 常被误解成「启动计算」。恰恰相反：**计算早已在进行中**，是创建 promise 的那个函数启动的；`await` 只是在处理一次「延迟的返回」。更好的心智模型是：`await` = 等这个函数的返回值 / 异常，和调用普通函数拿返回值是一回事，只不过中间隔了一段时间。 – **关键规则与最佳实践**： – `await` 只能在 `async` 函数内部使用，TypeScript 会**静态检查**这一点；在非 async 函数中使用 `await` 是编译错误。 – `async` 函数体里的 `return v` 会自动 fulfills 该 promise，`throw e` 会自动 rejects 它；函数「走到末尾自然结束」也会

fulfills promise。 – `async` / `await` 是**语法糖**：`const data = await promise; ...rest...` 语义上等价于 `promise.then(function(data) { ...rest... })`。理解这个变换，才能理解交错发生在哪里。 – **`void` 和 `undefined` 是两个不同的类型**：`void` 的取值集合为空，专用于「没有返回值」的函数，因此 `Promise<void>` 用于只为了副作用（如定时器延时）而运行的计算；`undefined` 则恰好有一个值。 – 一旦某一行引入 `await`，它就会「传染」：调用者必须 `await` 或 `then`，否则拿不到值（Java 中的对应物是 `Future`，见下）。

---

then() 组合与回调地狱（then() as Composition; Callback Hell） – **定义与目的**：`then` 是 promise 的**基础操作**，既是生产者也是观察器：`then(callback: T => U|Promise<U>): Promise<U>`。它把一个「将产生 T」的计算与一个「消费 T、产生 U」的计算**组合**起来。目的有两个：一是让异步计算像函数组合 `g ∘ f` 一样被逐段搭起来；二是取代「回调地狱（callback hell）」——层层嵌套的回调金字塔既难读也难做错误处理。 – **直观解释（"它是什么？"）**：普通函数组合把 `f: S → T` 与 `g: T → U` 组合成 `g ∘ f: S → U`；`then` 做的是同一件事，只不过类型上都套了一层：`Promise<T>` 与 `T → U` 组合成 `Promise<U>`。回调还**不一定要返回已算好的 U**，它可以返回 `Promise<U>`（例如 `fetch` 只完成了连接，真正下载正文还要等 `response.text()`），此时 `then` 会自动把内外两层 promise 拍平（flatten）。 – **关键规则与最佳实践**： – `then` 可以调用任意多次，挂上多个互不影响、各做各事的回调。 – `then` 在 promise **任何状态下**都可调用：pending 时回调将来才跑；已经 fulfilled 时回调立即运行。 – `then()` 是**唯一**访问 promise 所计算之值的途径——这既是安全属性（客户端永远不可能看到「缺值」的内部状态），也是并发设计特性：一个并发计算由一串受控、可预测的交错点上的 `then()` 组成。 – 用 `thenCompose` / `then` 链接「需要前一步结果的下一步计算」；如果只是并行独立的任务，**不要**串成链，那会白白损失并发性。

事件循环与协作式（非抢占）并发（Event Loop and Cooperative Concurrency） – **定义与目的**：JavaScript 每个全局环境只有一个控制线程（Workers 会创建新的全局环境，并且通常只靠消息传递通信）。那么「多个异步函数同时进行」是怎么实现的？答案是**事件循环（event loop）**：异步函数在 `await` 处不是忙等，而是**给 promise 挂一个回调，然后交出控制权**返回调用者；promise 最终被 resolved 是一个事件，由事件循环处理，事件循环再调用那个回调，把控制权还给该异步函数，并从 `await` 之后恢复执行。这种并发模型称为**协作式（cooperative）或非抢占式（non-preemptive）**并发。 – **直观解释（"它是什么？"）**：把事件循环想成一位**只有一个服务员的餐厅**。服务员永远不会被客人从背后拍醒（没有抢占），但每当客人说「我去等菜，你先忙别的」（`await`），他就去招呼别人。所以只要没人主动让出，其他人就全都饿着——这正是忙等待致命的根源。 – **关键规则与最佳实践**： – **每一个 `await` 都是一个可能让出控制权的地方**；但如果 promise 已经 fulfilled，也可能不让出、直接继续。 – 异步函数的**第一个** `await` 让出控制权的方式，是把自己的 promise 返回给调用者；**之后的** `await` 则直接把控制权还给事件循环。 – `await` 恢复时，控制权是**从事件循环回来的**。因此如果事件循环因为被阻塞而永远拿不到控制权，异步函数也永远无法推进。 – 一个异步函数在语义上被切分成「`await` 之间的若干段」，这些段可以与其他异步函数、其他回调交错执行——这也解释了为什么**每一段没有 `await` 的代码就是一个天然的互斥区间**（这一点在 Reading 23 中被系统化）。

错误传播（Error Propagation and Rejection） – **定义与目的**：异步计算失败时，异常**不能**像同步代码那样直接沿调用栈向上抛——因为调用栈早已返回了。promise 的解法是把失败也变成一种状态：rejected，并把 Error 对象**存进 promise**。`await` 一个 rejected promise 会重新抛出该异常；`then` 链会把失败沿着链条传下去，直到有人处理。目的：让异步错误处理和同步的 `try` / `catch` 在写法上尽量一致（可理解性），同时保证异常不会被无声吞掉（安全性）。 – **直观解释（"它是什么？"）**：同步世界里异常是「沿栈向上跳」；异步世界里没有栈可跳，于是把异常装进一个信封，随小票一起传下去，谁拆包谁面对它。 – **关键规则与最佳实践**： – `async` 函数里的 `throw` 就是「reject 我自己的 promise」；`return v` 就是「fulfill 我自己的 promise」。 – 在 `async` 函数内部用 `try` / `catch` 包住 `await`，可以像捕获同步异常那样捕获 rejected promise——这是 `await` 相对裸 `then` 的最大可读性优势。 – Java 类比（**补充说明**）：`Future.get()` 会把任务中的异常包成 `ExecutionException` 抛出，必须用 `e.getCause()` 解包；`CompletableFuture` 则提供 `exceptionally` / `handle` 做**恢复**。注意区分「传播」与「恢复」：前者让失败继续上浮，后者产出替代值。 – 永远不要用 `catch (Exception e) {}` 把失败静默吃掉；也不要在 `catch (InterruptedException e)` 里不恢复中断标志。

承诺聚合：all / any / race（Aggregating Promises） – **定义与目的**：并行跑多个计算时，常常需要把它们的结果像逻辑与/或一样合并。`Promise.all()` 相当于**逻辑与**：把一组 promise 合成一个，等全部 fulfilled 后返回结果数组，但只要有**任何一个**失败，整个 `Promise.all` 也失败；`Promise.any()` 相当于**逻辑或**：等**任意一个**成功 fulfill，只有全部失败才失败（适合冗余计算）；`Promise.race()` 也是逻辑或，但它等**任意一个** settle（fulfill 或 reject 都算），立即以同样的方式 settle——常用于给操作加超时。目的：把「多个并发计算」的协调工作交给库，而不是手写状态机。 – **直观解释（"它是什么？"）**：`all` 是「人到齐了才开饭」，`any` 是「谁先到谁点菜，全不来才散伙」，`race` 是「谁先有结果（不管好结果坏结果）就按谁来」。 – **关键规则与最佳实践**： – 用 `Promise.all` 取代「一个一个 `await`」，既避免串行化又表达意图；Java 类比是 `CompletableFuture.allOf(...)`（返回 `CompletableFuture<Void>`，需要再逐个 `join()` 取值）与 `anyOf(...)`（**补充说明**）。 – `Promise.all` 的失败语义是「一票否决」，如果你希望「部分成功也要结果」，要用 `Promise.allSettled` 之类的语义或自己处理每个 promise 的失败。 – 用 `Promise.race([fetch(url), timeout(5000)])` 实现超时的写法很常见，但要记住：落败的那个计算**并没有被取消**，它只是没人等了。 – `await` 的求值顺序是从左到右的（TS/JS 如此，并非所有语言都如此），所以 `(await a) + (await b)` 先等 `a`。

Deferred：把 promise 的修改器打包（Deferred） – **定义与目的**：promise 有两种客户：**消费者**用 `then` / `await` 安排后续计算；**承诺者**负责算出值并 resolve/reject 它。普通 promise 的修改器是通过 `new Promise((resolve, reject) => {...})` 这个「构造函数 + 回调」的形式交给承诺者的。另一种更直观的设计模式是 `Deferred<T>`：把 `Promise<T>` 与它的两个修改器打包成**一个对象**——`deferred.promise` 交给消费者，`deferred.resolve(t)` / `deferred.reject(err)` 留给承诺者。目的：让「由自己代码中的某个事件来完成的 promise」写起来干净、不易把修改器泄露给消费者。 – **直观解释（"它是什么？"）**：`Deferred` 就是**呼叫器（pager）**：前台把呼叫器给你（`promise`），服务员拿着主机（`resolve` / `reject`）。只有服务员能按下呼叫键。图书馆例子里「预约（hold）」就是这样一个 Deferred——这与 Reading 23 中 `checkout` 等待书籍归还的实现完全一致。 – **关键规则与最佳实践**： – `Deferred` 不属于标准 Node/JS 库，但可以**用**

`Promise` 构造函数实现；同理 `timeout` 也不在标准库里，要用 `setTimeout` + `Promise` 构造函数（或 `Deferred`）自己写。 – 修改器**绝不能**作为 promise 对象上的公开实例方法暴露出去，否则消费者就能自己 resolve 别人的 promise。 – Java 类比（**补充说明**）：`CompletableFuture` 本身就兼作 `Deferred` —— `new CompletableFuture<T>()` 创建后，持有者调用 `complete(v)` / `completeExceptionally(e)` 来满足它，而把该对象交给消费者去 `thenApply(...)`。二者是同一模式在不同语言里的呈现。 – 一个 Deferred 只能被 resolve 一次；重复 resolve 是设计错误（TypeScript 里后续调用会被忽略，Java 的 `complete` 返回 `false`）。

绝不忙等待（Never Busy-Wait） – **定义与目的**：**忙等待（busy-waiting）**指在一个紧凑循环里空转，等待某个事件发生，期间**不让出控制权**。promise 刻意不提供观察器，就是为了让忙等待**写不出来**。忙等待在并发编程中通常是坏主意（自旋锁等少数例外除外），在 promise/async-await 代码中尤其致命：在单线程模型下它会**冻结整个程序**，而且往往连自己想等的事件都等不到。 – **直观解释（"它是什么？"）**：`busyWait(2000)` 就像一个客人在柜台前站着不走、不停问「好了没」。他堵住了唯一的服务员，于是厨房的消息永远没人传出来——越等越等不到。 – **关键规则与最佳实践**： – 只能通过 `await` 或 `then` 与 promise 交互；不要试图轮询其状态（`promise.isPending()`、`promise.get()` 这些假想 API 不存在是有意为之）。 – 在 Java 中轮询 `future.isDone()`（**补充说明**）同样会浪费 CPU；应改用 `get()`、`get(timeout, unit)` 或 `thenAccept` 风格的回调组合。`CompletableFuture` 的 `orTimeout` / `completeOnTimeout`（Java 9+）比手写 `race` 风格更清晰。 – 忙等待不仅浪费 CPU，还会**掩盖时序 bug**（heisenbug）：加了打印或断点后行为大变，问题反而更难定位（参见 Reading 13 调试）。 – 需要「等一会儿」时使用定时器 promise（`timeout`）而不是空转循环；需要「等条件成立」时应该等一个由他人 resolve 的 Deferred。

Java 对应机制：Future、CompletableFuture、ExecutorService（Java 类比 / 补充说明） – **定义与目的**：Java 没有与 sp22 完全对应的孪生章节（sp21 的对应阅读是「Concurrency」，讲的是线程与共享内存）。在 Java 中表示「异步计算结果」的一等值主要有两种：`java.util.concurrent.Future`（老式、以阻塞式 `get()` 为主）与 `CompletableFuture`（可组合、可回调，最接近 promise/`then`）。`ExecutorService` 则负责在哪个线程上执行这些计算。 – **直观解释（"它是什么？"）**：`ExecutorService` 是「后厨团队」，`submit()` 是「下单」，返回的 `Future` 就是取餐号。`Future.get()` 相当于**站在柜台前等**（阻塞线程），而 `CompletableFuture.thenApply(...)` 相当于**留下电话号码**（注册回调，不占线程）。 – **关键规则与最佳实践**： – `Future.get()` 会**阻塞调用线程**，并抛出受检异常 `InterruptedException`（中断）与 `ExecutionException`（任务失败），必须显式处理。 – `CompletableFuture` 默认在 `ForkJoinPool.commonPool()` 上执行；生产代码应显式传入自己的 `Executor`，避免与库代码争抢公共池。 – `ExecutorService` 必须 `shutdown()`，否则程序可能无法退出（non-daemon 线程仍在运行）。 – **`await` ≈ `thenApply` 的语法糖，而不是 `get()` 的语法糖**：`await` 不占用线程（在 JS 中根本不阻塞线程），`get()` 占用线程。这是本讲最需要辨析的一处对照，详见「代码示例与对比分析」的场景 5。

代码示例与对比分析

场景 1：三个并行的取数任务——先提交全部，还是提交一个等一个？

❌ *错误代码*

```
import java.util.List;
import java.util.concurrent.*;

/** 读取三份文件并拼接，使用线程池并行取数。 */
public class Fetcher {

    /** 模拟一次耗时的远程读取。 */
    private static String fetch(String url) {
        // 假设这里是一次需要数秒的网络或磁盘操作
        return url + ":200";
    }

    /** 错误：提交一个任务就立刻 get 一个，任务被强行串行化。 */
    public static String fetchAll(List<String> urls, ExecutorService executor)
            throws Exception {
        StringBuilder out = new StringBuilder();
        for (String url : urls) {
            Future<String> future = executor.submit(() -> fetch(url));
            // 阻塞等待这一个完成后，才去提交下一个任务
            out.append(future.get());
        }
        return out.toString();
    }
}
```

【错误代码的问题】 1. **完全丧失并发性**：第二个任务的 `submit` 发生在第一个任务的 `get` 返回之后，三个计算实际上变成了串行执行，总耗时是三者之和而不是最大值。 2. **违背了本讲的中心思想**：sp22 原文明确指出，`totalBalance` 之所以并发，正是因为它**先把两个 promise 都拿到手，把 `await` 留到后面**；这段代码把顺序写反了。 3. **语义上「看起来」是并行的**：代码里有线程池、有 `Future`，很容易骗过代码评审——bug 是性能问题而非正确性问题，测试不会红。 4. **异常会立刻中断整个循环**：第一个任务的失败会让后面两个任务根本不会被提交，哪怕它们本来可以成功。

✅ *正确代码*

```java
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.*;

/** 读取三份文件并拼接，使用线程池并行取数。 */
public class Fetcher {

    private static String fetch(String url) {
        return url + ":200";
    }

    /** 正确：先把所有计算都启动，再统一收割结果。 */
    public static String fetchAll(List<String> urls, ExecutorService executor)
            throws Exception {
        List<Future<String>> futures = new ArrayList<>();
        for (String url : urls) {
            // 第一步：只提交，不等待——所有计算同时在跑
            futures.add(executor.submit(() -> fetch(url)));
        }
        StringBuilder out = new StringBuilder();
        for (Future<String> future : futures) {
            // 第二步：逐个收割；此时它们多半已经完成，get 立即返回
            out.append(future.get());
        }
        return out.toString();
    }
}
```

【为什么这样更好】 提交与收割被明确拆成两个阶段，于是所有 `fetch` 调用**同时**在池中的线程上推进，墙钟耗时从「三者之和」降到「三者最大值」。这段代码与 sp22 的 `totalBalance` 一一对应：`const checkingPromise = getBalance('checking'); const savingsPromise = getBalance('savings'); return (await checkingPromise) + (await savingsPromise);`——先把 promise 收集起来，再 await。若把 `await` 挪到每一个赋值语句上（`const checking = await getBalance('checking')`），就退化成本场景的错误版本。

【代码对比解说】 关键差别只有一个字：`await` / `get` 的位置。sp22 用一组练习专门训练这种眼力，比如 `const checking = getBalance('checking'); const savings = getBalance('savings'); return checking + savings;` 是**静态类型错误**（把两个 `Promise<number>` 相加），而 `const checking = await getBalance('checking'); const savings = await getBalance('savings');` 能编译但**不并发**。在 Java 中这一课同样成立，而且更危险：`Future` 是值、可以相加、可以放进 `List`，编译器不会拦你，静默的性能损失只有压测才能发现。顺带一提，`CompletableFuture.allOf(f1, f2, f3).join()` 是「聚合」版本的写法，语义上等价于 `Promise.all`。

【设计原则透视】 这属于**规格说明与性能契约**的交叉点：`fetchAll` 的方法规格应当写明「三个取数并发进行」，否则调用者无从判断实现是否兑现了承诺。它同时暴露了 ADT 设计的一条老规矩（Reading 10、Reading 11）：**操作的语义要能被客户端推理**。在并发语境下，「什么时候发生交错」也是语义的一部分，因此必须在规格或注释里写清楚，这也就是 sp22 反复强调的「正确地推理交错点」。

场景 2：等待结果——忙等待轮询状态，还是阻塞/回调?

❌ *错误代码*

```
import java.util.concurrent.*;

/** 错误：轮询 Future 的状态，同时在单线程池里做「后台」工作。 */
public class BusyWaiter {
    public static void main(String[] args) throws Exception {
        ExecutorService executor = Executors.newSingleThreadExecutor();
        Future<Integer> future = executor.submit(() -> {
            Thread.sleep(2000);
            return 42;
        });

        // 忙等待：空转消耗 CPU，而且完全不做别的事
        while (!future.isDone()) {
            // 什么也不做，只是不停地问「好了没」
        }
        System.out.println(future.get());
        executor.shutdown();
    }
}
```

【错误代码的问题】 1. **白白烧掉 CPU**：这两秒里处理器被一个什么都不做的循环占满，其他线程/进程被拖慢。 2. **无法被中断**：这个循环不响应中断，也无法设置超时；程序卡住时只能强杀。 3. **在单线程模型下会彻底死锁**：sp22 的 `busyWait` 例子正是这个后果——`busyWait` 的循环体里没有 `await`，因此事件循环永远拿不到控制权，**其他所有异步函数都停摆**；更糟的是，它等待的事件本身要靠事件循环来处理，所以那个「promise 是否已 fulfilled」的检查永远为假，循环永远出不来。 4. **掩盖时序 bug**：忙等待对时序极其敏感，加一行 `println` 行为就变（Reading 13 中的 heisenbug）。

✅ *正确代码*

```
import java.util.concurrent.*;

/** 正确：让出 CPU 去等，或用回调/超时把等待交给库。 */
public class Waiter {
    public static void main(String[] args) throws Exception {
        ExecutorService executor = Executors.newSingleThreadExecutor();

        // 写法一：阻塞式等待——线程被挂起，CPU 交给别人用
        Future<Integer> future = executor.submit(() -> {
            Thread.sleep(2000);
            return 42;
        });
        System.out.println(future.get(5, TimeUnit.SECONDS)); // 可设超时

        // 写法二：回调式等待——不占用调用线程，最接近 sp22 的 then()
        CompletableFuture<Integer> promised = CompletableFuture.supplyAsync(() -> {
            sleepQuietly(2000);
            return 42;
        }, executor);
        promised.thenAccept(answer -> System.out.println("answer = " + answer));

        executor.shutdown();
    }

    private static void sleepQuietly(long millis) {
        try {
            Thread.sleep(millis);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

【为什么这样更好】 `get()` 把线程**挂起**（不消耗 CPU），`get(timeout, unit)` 还给出了失败上界；`thenAccept` 则连调用线程都不占，语义上与 sp22 的 `promise.then(function(data) {...})` 完全同构。这段正确代码同时演示了 sp22 的核心纪律：**永远用 await/then 与 promise 交互，绝不用轮询**。原文还解释了为什么 promise 索性不提供观察器：*If a Promise provided observers, then programmers would be tempted to write busy-waiting code like this* ——把危险 API 直接删掉，是从设计上「让错误写法写不出来」。

【代码对比解说】 两种等待在「谁占着资源」上截然不同：忙等待占着 **CPU**，`get()` 占着**线程**（但释放 CPU），`then` 回调两者都不额外占用。在 JavaScript 的单线程世界里，占着「线程」就等于占着「整个事件循环」，所以 promise 必须提供 `then` 这种不占线程的机制，并且必须禁止忙等待。Java 可以「奢侈地」用阻塞 `get()`，因为线程便宜（相对 JS 的唯一线程而言）——但请记住 Reading 22 的错误示范：**在单线程 executor 上，阻塞就等于 JS 里的忙等待**，见场景 5。

【设计原则透视】 这是**表示独立性与抽象边界**的典范：promise 对外只暴露 `then` / `await` 这条「注册后续计算」的通道，把状态与值藏在抽象边界内，客户端因此无法写出依赖内部状态的脆弱代码。用 Reading 6

的话说，promise 的规格规定了「什么时候能得到值」，而把「值以什么形式暂存」留给实现——这使得库可以自由优化，而客户端代码不受影响。

场景 3：异步失败——静默吞掉，还是显式传播/恢复？

❌ *错误代码*

```
import java.util.concurrent.*;

/** 错误：把所有失败都悄悄变成默认值。 */
public class BalanceReader {
    private static String readFile(String account) throws Exception {
        // 文件不存在时抛出异常
        throw new java.io.FileNotFoundException(account);
    }

    public static int getBalance(String account, ExecutorService executor) {
        Future<String> future = executor.submit(() -> readFile(account));
        try {
            return Integer.parseInt(future.get());
        } catch (Exception e) {
            // 出什么事都当作余额 0
            return 0;
        }
    }
}
```

【错误代码的问题】 1. **异常被静默吞掉**：调用者拿到 `0` 却毫不知情，与「账户真的是 0 元」无法区分——这正是 sp22 所说的「无人处理的 rejection」。 2. **异常类型信息丢失**：`ExecutionException` 只是包装，真正的 `cause`（文件不存在 / 权限不足 / 网络中断）被丢弃，故障无法定位。 3. **错误地统一了可恢复与不可恢复的失败**：`InterruptedException`（线程被要求停止）与「业务失败」被同样对待，中断信号被吞，线程池的关闭逻辑会失效。 4. **掩盖了「格式非法」与「读取失败」的区别**：`parseInt` 的 `NumberFormatException`（对应 sp22 `getBalance` 里的 `throw new Error('account does not contain a number')`）本应是明确的一类 rejection。

✅ *正确代码*

```
import java.io.IOException;
import java.util.concurrent.*;

/** 正确：分类处理中断与失败，并保留原始异常。 */
public class BalanceReader {
    private static String readFile(String account) throws IOException {
        throw new java.io.FileNotFoundException(account);
    }

    /**
     * @param account 账户文件名
     * @return 账户余额
     * @throws IOException 如果账户无法读取或内容不是数字
     */
    public static int getBalance(String account, ExecutorService executor)
            throws IOException {
        Future<String> future = executor.submit(() -> readFile(account));
        final String data;
        try {
            data = future.get();
        } catch (InterruptedException e) {
            // 有人在要求本线程停止：恢复中断标志并向上传递
            Thread.currentThread().interrupt();
            throw new IOException("interrupted while reading " + account, e);
        } catch (ExecutionException e) {
            // 解包任务内部的真实原因，不要丢掉它
            throw new IOException("could not read " + account, e.getCause());
        }
        try {
            return Integer.parseInt(data);
        } catch (NumberFormatException e) {
            throw new IOException(account + " does not contain a number", e);
        }
    }
}
```

【为什么这样更好】 三种结局被清楚地区分开：中断被恢复标志后继续上浮，任务内部的失败被解包成带 `cause` 的领域异常，格式错误被明确标注。调用者可以针对性地处理，而不是面对一个来历不明的 `0`。这正是 sp22 中 `await` 的语义：*If the promise is rejected, then await throws an exception instead, using the Error object that the computation stored in the promise* ——失败**必须**以异常的形式重新出现在等待者面前，而不是变成某个「看起来正常」的返回值。`CompletableFuture` 的 `exceptionally`（**补充说明**）则适用于确实有合理默认值的场景。

【代码对比解说】 用 `CompletableFuture` 写同一件事会更接近 sp22 的写法：

```
// 传播：把失败沿 then 链传下去（对应 await 抛异常）
CompletableFuture<Integer> balance =
        CompletableFuture.supplyAsync(() -> readFileQuietly(account), executor)
                         .thenApply(Integer::parseInt);

// 恢复：给失败提供一个替代值（对应 try/catch 包住 await）
CompletableFuture<Integer> safe =
        balance.exceptionally(err -> 0);
```

`thenApply` 只在成功时运行，失败会**自动跳过**后续的 `thenApply` 直到遇到 `exceptionally` / `handle` ——这就是 promise 链上的「错误传播」，和 sp22 描述的行为一致。注意 `exceptionally` 不是「吞掉异常」，它是**显式声明**「这类失败我接受默认值」，这是有意的设计决定而非疏忽。

【设计原则透视】 这是**规格说明（Reading 6）与异常设计**的结合：方法的 `@throws` 子句就是它的失败契约，也是客户端唯一能依赖的信息。把 `ExecutionException` 原样抛出会把「实现用了线程池」这一实现细节泄露到抽象边界之外，等于让表示泄漏（rep exposure 的异常版本）；正确做法是翻译成领域异常。同时，静默返回 `0` 违反了「让错误尽早、响亮地暴露」的原则，会让 bug 漂移到离现场很远的地方才爆发。

场景 4：手写状态标志，还是用一个可以被外部事件完成的 promise?

❌ *错误代码*

```
/** 错误：用共享可变标志位 + 轮询来模拟「由别人完成的结果」。 */
public class Downloader {
    private boolean ready = false;       // 没有同步，且被多个线程读写
    private String content;

    /** 在另一个线程里被调用。 */
    public void finishDownload(String text) {
        this.content = text;
        this.ready = true;
    }

    /** 在等待的线程里被调用。 */
    public String await() {
        while (!ready) {
            // 忙等待，而且根本看不到 ready 的变化（可见性问题）
        }
        return content;
    }
}
```

【错误代码的问题】 1. **忙等待 + 冻结执行流**：与场景 2 同样的病，且这次连「等的事件由谁触发」都依赖同一个被堵住的线程。 2. **可见性问题**：`ready` 与 `content` 都不是 `volatile`，等待线程可能永远读到旧值（sp21 的 `computeAnswer` / `useAnswer` 例子演示了这一点）——甚至可能先看到 `ready == true` 再看到空的 `content`（重排序）。 3. **没有失败通道**：下载失败时无法通知等待者，只能永远等待。 4. **接口把实现细节暴露给所有人**：任何人都可以调用 `finishDownload`，消费者与承诺者的权限边界荡然无存。

✅ *正确代码*

```
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.CompletionStage;

/**
 * 正确：CompletableFuture 就是 Java 里现成的 Deferred。
 * 承诺者持有它并调用 complete/completeExceptionally，消费者只拿到 CompletionStage。
 */
public class Downloader {
    private final CompletableFuture<String> promise = new CompletableFuture<>();

    /** 交给消费者：只能注册后续计算，不能修改状态。 */
    public CompletionStage<String> content() {
        return promise;
    }

    /** 只有承诺者能调用。 */
    public void finishDownload(String text) {
        promise.complete(text);
    }

    /** 失败时也必须有人来 settle 它，否则等待者会永远 pending。 */
    public void failDownload(Throwable cause) {
        promise.completeExceptionally(cause);
    }
}
```

【为什么这样更好】 `CompletableFuture` 把「创建」与「完成」分开：创建者拿到可写的对象，消费者只拿到 `CompletionStage`（只读视图，只能挂回调），于是**权限边界由类型系统保证**——这正是 sp22 对 `Deferred` 的描述：*only the promiser … ends up having access to these mutators*。失败也有一等表示（`completeExceptionally`），等待者不会无限挂起。同步问题交给库：`CompletableFuture` 内部保证可见性与线程安全，不需要手写 `volatile`。

【代码对比解说】 sp22 中的 `Deferred<T>` 由三部分组成：`deferred.promise`（给消费者）、`deferred.resolve(t)`（承诺者）、`deferred.reject(err)`（承诺者）。Java 的 `CompletableFuture` 把这三件事放在一个对象上，靠**返回视图的类型**（`CompletionStage` vs `CompletableFuture`）来区分权限；TypeScript 则靠「不把修改器做成公开方法」来区分。这一个小例子几乎就是 Reading 23 图书馆例子中 `hold` 的雏形：`checkout` 创建 `new Deferred<void>()` 放进预约列表并 `await hold.promise`，`checkin` 找到它并 `deferred.resolve()`。

【设计原则透视】 这是**抽象边界 + 权限最小化**的实践：谁可以改变对象状态，应当由**类型**而不是文档和自觉来约束。同时它体现了 Reading 11 的表示不变量思想——promise 的 RI 是「状态单调地从 pending 走向 fulfilled/rejected，且不会再变」，而保证这条 RI 的唯一手段就是把两个修改器锁在抽象边界内。

场景 5：在单线程执行器里「等自己」——把 await 误当成 Future.get

❌ *错误代码*

```
import java.util.concurrent.*;

/** 错误：在单线程池里，任务内部又提交任务并等待它完成。 */
public class SelfDeadlock {
    public static void main(String[] args) throws Exception {
        ExecutorService executor = Executors.newSingleThreadExecutor();

        Future<Integer> outer = executor.submit(() -> {
            // 这个任务运行在唯一的线程上
            Future<Integer> inner = executor.submit(() -> 42);
            // 排在前面的 inner 只能等这条线程空出来——可这条线程正在等它
            return inner.get();
        });

        System.out.println(outer.get()); // 永远打印不出来
        executor.shutdown();
    }
}
```

【错误代码的问题】 1. **永久死锁**：唯一的线程被外层任务占着并阻塞在 `inner.get()`，而 `inner` 只有等这条线程空出来才能运行。系统就此冻结。 2. **这正是 sp22 描述的「事件循环被阻塞」的 Java 版本**：*if the event loop never gets control, because it is blocked, then asynchronous functions won't be able to make progress either*。单线程池 + 阻塞等待 = 单线程事件循环 + 忙等待。 3. **难以察觉**：代码在 `newFixedThreadPool(4)` 下往往能正常工作，一换成单线程池就挂，属于典型的环境相关 heisenbug。 4. **没有超时与失败出口**：一旦发生，程序既不报错也不退出，只是静静地卡住。

✅ *正确代码*

```
import java.util.concurrent.*;

/** 正确：用组合而不是「在线程上阻塞等待同一条线程」。 */
public class NoSelfDeadlock {
    public static void main(String[] args) throws Exception {
        ExecutorService executor = Executors.newSingleThreadExecutor();

        // 写法一：把两步计算组合成一个异步流水线，中途不让出线程去阻塞
        CompletableFuture<Integer> pipelined =
                CompletableFuture.supplyAsync(() -> 42, executor)
                                 .thenApply(n -> n + 0);
        System.out.println(pipelined.get());

        // 写法二：确实需要嵌套提交时，给内层换一个执行器
        ExecutorService inner = Executors.newSingleThreadExecutor();
        Future<Integer> outer = executor.submit(() -> inner.submit(() -> 42).get());
        System.out.println(outer.get());

        executor.shutdown();
        inner.shutdown();
    }
}
```

【为什么这样更好】 写法一把「提交并等待」换成「注册后续计算」，任务之间不再互相占线程，因此单线程池也能跑得动——这正是 sp22 中 `await` / `then` 在单线程 JS 里可行的根本原因。写法二承认「阻塞等待」的代价，用独立资源（第二个线程）来消化它，避免循环等待。两种写法都保证了「没有线程被要求等待自己」。

【代码对比解说】 本场景把前面所有对照收束成一个精确的类比：

sp22（TypeScript）	Java
`await promise`	语义等价于 `thenApply` 组合；**不**等价于 `Future.get()`
事件循环（唯一线程）	单线程 `ExecutorService`
阻塞事件循环 → 全部停摆	单线程池里阻塞 → 死锁
`Promise.all`	`CompletableFuture.allOf`
`Promise.race`	`applyToEither` / `orTimeout`（Java 9+）
`new Deferred<T>()`	`new CompletableFuture<T>()` + `complete` / `completeExceptionally`
busy-wait 循环	`while (!f.isDone()) {}`

差异的根源是：JS 的 `await` **不占线程**，Java 的 `get()` **占线程**。所以在 Java 里判断「能不能在这儿阻塞」必须回到 Reading 21 与 Reading 23 的问题：这条线程是不是还有别的事要做（比如驱动别的任务）？

【设计原则透视】 这是**死锁（liveness）**问题的第一个具体面孔，sp22 把它归入 liveness：*Does the program keep running and eventually do what you want, or does it get stuck somewhere waiting forever for events that will never happen?* 判断依据是「等待图里有没有环」——本场景的环是「外层任务 → 内层任务 → 同一条线程 → 外层任务」。这个「画等待图找环」的方法在 Reading 24 分析消息传递死锁时会再次使用，在那里环出现在两个队列之间。

与其他设计原则的关联

本讲的 promise 抽象建立在 **Reading 20（回调函数）** 之上：回调是「在计算完成时被调用」的最朴素机制，而 promise 是把它包装成一等值的改进版；sp22 明确指出，在 promise 出现之前回调是 JavaScript 实现异步行为的最常见方式，至今仍广泛存在于生态中。**Reading 21（并发）** 提供了两个并发模型（共享内存与消息传递）、竞态条件、交错与 heisenbug 的背景知识，并解释了为什么「正确性不该依赖时序的偶然」；本讲则在单线程、协作式并发的语境下把交错点精确到 `await`。

本讲的直接后续是 **Reading 23（互斥）**：那一讲把 promise 的 ADT 补全（引入 `Deferred` 的 resolve/reject 修改器），并用图书馆预约（hold）的完整例子展示「用 promise 等待一个由自己代码触发的事件」；它还系统化了本讲最后提到的直觉——**没有 `await` 的代码段是一个天然互斥区间**。再往后是 **Reading 24（消息传递）**：那里的 `put` / `take` 才是真正会阻塞线程的操作，与本讲的 `await`（不占线程）形成鲜明对照；理解这个差别，才能理解为什么消息传递中「阻塞是双刃剑」。**Reading 25（套接字与网络）** 会把消息传递搬到网络上，而网络 I/O 天然是异步的，正是 promise 的用武之地。

与更早的章节也有清晰依赖：**Reading 6、Reading 7（规格说明与设计规格）** 要求异步 API 的规格写明「何时 fulfills、何时 rejects、是否可取消」；**Reading 8（不可变性）** 是异步结果能被安全共享的前提（多个回调可能持有同一份结果）；**Reading 10、Reading 11（ADT、AF 与 RI）** 提供理解 promise 的语言——promise 是可变类型，其三态转移就是它的表示不变量，而「只有承诺者能改状态」就是防止表示泄漏的边界；**Reading 12（接口、泛型与枚举）** 解释了 `Promise<T>` 的泛型与「消息/结果类型用带标签的联合」这两件事；**Reading 13（调试）** 提醒我们，异步与时序 bug 极易成为 heisenbug。

关键要点

- **promise 表示「已启动但可能未完成的计算」，且只有三态**：pending → fulfilled / rejected，单向、一次性、不可重置；`await` 把 `Promise<T>` 变成 `T`，rejected 时抛异常。
- **`await` 不是启动计算，而是处理延迟返回**；计算在创建 promise 的那一刻就开始了。`async` / `await` 只是 `then()` 与 promise 状态转移的语法糖。
- **绝不允许忙等待，也不允许窥探 promise 状态**：只能通过 `await` 或 `then` 与之交互；promise 不提供观察器是刻意的设计，因为「把危险 API 删掉」比「写文档劝阻」更可靠。
- **交错只发生在 `await` 点**：（单线程协作式并发下）没有 `await` 的代码段必不被打断——这条直觉是 Reading 23 互斥的基础。
- **异步 ≠ 并发 ≠ 并行**：异步是接口性质（提前返回），并发是结构性质（重叠推进），并行是硬件性质（同时执行）；Java 里 `Future.get()` 阻塞线程，`CompletableFuture.thenApply` 不占线程，而 sp22 的 `await` 属于后者。

常见陷阱与注意事项

- **把 `await` 写进循环的赋值里，导致假并发 → 性能损失**：`for (u : urls) results.add(executor.submit(...).get())` 与 `const x = await f(i)` 都是「提交一个等一个」。后果是任务被静默串行化，测试全绿但墙钟时间翻倍；正确做法是先全部启动，再统一等（`Promise.all` / `allOf`）。
- **在单线程执行器里阻塞等待同一条线程上的任务 → 死锁**：把 `await` 的直觉直接套到 `Future.get()` 上，在 JS 里表现为阻塞事件循环（全部停摆），在 Java 里表现为单线程池自锁。后果是程序既不报错也不推进，只能强杀。
- **忙等待或轮询状态（`while (!f.isDone())`、假想的 `promise.isPending()`）→ 烧 CPU 且等不到结果**：在单线程模型中，被等的事件本身要靠事件循环处理，而事件循环正被这个循环堵死，于是形成「越等越等不到」的活锁式冻结；同时它会掩盖真正的时序问题，形成 heisenbug。
- **忘记给失败留通道（不 reject、不 `completeExceptionally`、`catch (Exception e) {}`）→ 等待者永远 pending，或错误被静默吞掉**：前者是 liveness 问题（程序卡住），后者是 safety 问题（错误结果被当成正常结果继续传播），而且都无法通过测试发现。
- **把 promise 的修改器暴露成公开方法，或把 `CompletableFuture` 直接交给消费者 → 抽象边界失守**：任何客户端都能 resolve 别人的 promise，或调用 `complete`、`cancel` 破坏状态机，表示不变量从此不可维护；应只交出 `CompletionStage` 这样的只读视图。

- **用「上帝式」`Promise.all` 吞掉部分失败，或以为 `Promise.race` 会取消落败者 → 语义误解**：`all` 是一票否决，`race` 只是不再等待、**不会**停止落败的计算（超时场景下后台请求仍在跑，可能仍在写入共享状态）。

思考题（带答案）

问题 1：下面两段 TypeScript 代码（sp22 原文风格）都用来读两个账户的余额并求和。请分别说明它们的并发行为与静态类型结果，并给出对应的 Java 写法。

```
// 版本 A
const checking = getBalance('checking');
const savings = getBalance('savings');
return (await checking) + (await savings);
```

```
// 版本 B
const checking = await getBalance('checking');
const savings = await getBalance('savings');
return checking + savings;
```

答案：版本 A 是**并发**的：两次 `getBalance` 调用先各自启动一个后台计算并立刻返回 promise，`await` 被推迟到最后，因此「读 checking」与「读 savings」可以重叠进行，总耗时接近两者最大值。这里两个 `await` 的顺序并不影响并发性，只影响**恢复的顺序**（TS/JS 从左到右求值，所以先恢复 `checking`）。版本 B 是**串行**的：第一个 `await` 必须等到 checking 读完，控制权才回到这一行，`getBalance('savings')` 才被调用；总耗时是两者之和。两者都能通过静态检查（都得到 `number`），所以错误不会被编译器发现，只会表现为性能问题——这正是本讲要训练的「看 await 位置判断并发性」的眼力。对应的 Java 写法：版本 A 是「先把两个 `Future` 收进变量，再依次 `get()`」（或 `CompletableFuture` 的 `allOf` / `thenCombine`）；版本 B 是「提交一个、`get()` 一个」。反过来，若写成 `const checking = getBalance('checking'); const savings = getBalance('savings'); return checking + savings;`（少了 `await`），就是**静态类型错误**：`Promise<number> + Promise<number>` 不合法——这是静态检查在我们这边的唯一一次帮忙。

问题 2：为什么 `Promise` 不提供 `isPending()`、`get()` 这类观察器？请说明至少两个具体危害，并解释 Java 中对应的「观察器」是什么、该如何避免。

答案：sp22 原文给出的理由是：一旦有了观察器，客户端就会忍不住**忙等待**——`while (promise.isPending()) { }` 然后 `promise.get()`。危害有两个层面：(1) **程序冻结**：忙等待循环从不交出控制权，在单线程 JS 模型下事件循环永远拿不到控制权，其他异步函数全部停摆；(2) **越等越等不到**：被等的 promise 之所以能变成 fulfilled，恰恰需要事件循环去处理它的回调；事件循环被堵住，promise 永远不会 fulfilled，`isPending()` 永远返回 true，循环永远不退出——这是一个自我封闭的死局。此外还有 (3) 破坏 promise 的安全设计：任何客户端都能窥见「尚未有值」的内部状态，很容易写出「先检查再取值」的竞态代码。Java 中对应的观察器是 `Future.isDone()` / `Future.isCancelled()` 加 `get()` 的轮询组合；避免方式是改用 `get()`（阻塞但释放 CPU）、`get(timeout, unit)`（有上界），或 `CompletableFuture.thenAccept` / `orTimeout`（不占线程、由库负责唤醒）。核心原则一致：**把「等待」表达成注册后续计算，而不是表达成反复询问状态**。

问题 3：sp22 把 `Promise<void>` 用于 `timeout(milliseconds)` 这样的纯计时器。为什么一个「空 promise」是有用的？`void` 与 `undefined` 有何区别？并说明 `Deferred` 在其中的作用。

答案：`timeout` 的 promise 不产生任何有用的值，但**它的完成本身就是有用的事件**：`await timeout(2000)` 之后我们就知道「2000 毫秒确实已经过去」，可以在那一刻触发后续计算。所以空 promise 完全对应「返回 `void` 的函数」——它存在的意义是副作用与时序，而不是数据。`void` 与 `undefined` 的区别是类型论层面的：`void` 是「取值集合为空」的类型，专门用作无返回值函数的返回类型，没有任何值属于它；而 `undefined` 是「恰好有一个值 `undefined`」的类型，那个值是货真价实的一等值，可以赋给变量、放进数据结构、作为参数传递。因此「不会产生值的计算」用 `Promise<void>`，而不是 `Promise<undefined>`。`Deferred` 的作用在于：`timeout` 这类函数既不在标准库里（Node 没有现成的 `timeout`），也无法靠外部设备完成——它必须由**我们自己的代码**在计时器触发时完成，因此需要一个持有 `resolve` 修改器的对象。实现就是 `const deferred = new Deferred<void>(); setTimeout(() => deferred.resolve(), milliseconds); return deferred.promise;`——把 promise 交给消费者，把修改器留在承诺者手里。Java 类比是 `new CompletableFuture<Void>()` 配合一个定时任务调用 `complete(null)`，或直接用 `CompletableFuture.delayedExecutor`（**补充说明**）。

Reading 23: 互斥与锁（Mutual Exclusion）

> 说明：本讲 sp22 原版使用 TypeScript，本笔记按用户要求提供 Java 代码示例；类型/API 与 sp21（6.031 Java 版）原文保持一致。sp22 的「互斥」概念是用单线程协作式并发（`await` 交错点）来讲的，而 sp21 的孪生阅读（Locks and Synchronization）讲的是多线程下的 `synchronized`、监视器模式与死锁；本笔记以 sp22 的概念框架（交错、互斥、临界区、竞态、死锁、safety/liveness）为主结构，用 sp21 的 Java 代码（`synchronized`、`ReentrantLock`、`SimpleBuffer`、`GapBuffer`、`findReplace`、`Wizard`/`Castle`、`ConcurrentMap`）作为实现示例。凡属 Java 生态的补充内容（`ReentrantLock` 的全部细节、死锁的四个必要条件、`System.identityHashCode` 等）均显式标注为「补充说明」。

概述

本讲把并发与承诺两条线索汇合到**异步抽象数据类型（asynchronous ADT）**上：这个 ADT 的操作可能彼此并发运行，并访问同一份共享的（可变）表示，于是产生了**竞态条件（race condition）**与**死锁（deadlock）**两类威胁抽象函数、表示不变量与规格说明的 bug。防御手段是两手：一是**推理交错**（在哪里可能发生危险的交错？），二是**构造互斥**（哪里必须绝对禁止交错？），从而得到只允许一个计算独占地访问共享可变数据的区间——**临界区（critical section）**。在 sp22 的单线程模型里，互斥区间就是「不含 `await` 的代码段」；在多线程的 Java 里，互斥区间由**锁（lock）**提供，最常用的是 `synchronized` 的**监视器模式（monitor pattern）**。它与三大目标的关系是：**Safe from bugs**——互斥是防止竞态的根本手段，但锁本身又引入死锁，威胁**liveness**；**Easy to understand**——锁的纪律（哪个锁保护哪些数据）必须写进代码注释，否则后来者无法维护；**Ready for change**——锁的粒度、是否把锁暴露给客户端、是否改用消息传递，都是影响可修改性的重大设计决策。

核心概念与设计原则详解

线程安全与四种并发策略（Thread Safety and Four Strategies） – **定义与目的**：**线程安全**指一个数据类型或函数在被多个线程使用时，无论这些线程如何被执行，都表现正确，且不需要调用方做额外协调。贯穿全课的总原则是：**并发程序的正确性不应依赖时序的偶然**。sp21 归纳了四种策略：**限定（confinement）**——不共享，把变量与其指向的数据限制在单个线程内可访问；**不可变性（immutability）**——共享但不可变（`final` 字段、不可变类型）；**使用已有的线程安全类型**——让库替你协调；**同步（synchronization）**——阻止线程同时访问共享数据。本讲的主题是第四种。 – **直观解释（"它是什么？"）**：想象一间只有一支笔的办公室。限定 = 每人发一支笔（不共享）；不可变 = 笔上刻死了字，谁看都一样；用现成的线程安全类型 = 用一台自动售笔机；同步 = 笔上栓了把锁，谁用谁锁上。前三种都「不靠时序」；第四种则明确规定了**时序**（谁先拿到锁谁先用），因此它最强大也最危险。 – **关键规则与最佳实践**： – 优先考虑前三种策略，因为它们不引入阻塞，也就不会死锁；只有在必须共享可变数据时才用同步。 – 使用同步意味着**接受阻塞**，而阻塞意味着可能死锁——这是 Reading 21 中「并发很难」的具体化。 – 线程安全论证（thread safety argument）必须**写进代码**：*that discipline needs to be written down, or maintainers won't know what it is*。 – 不要指望测试：线程交错的数量是天文数字，测试不可能覆盖，且竞态 bug 往往是 heisenbug（加 `println` 就消失）。

竞态条件与交错（Race Condition and Interleaving） – **定义与目的**：**竞态条件**指程序的正确性（后置条件与不变量的满足）取决于并发计算中事件的**相对时序**。**交错（interleaving）**是理解它的工具：把并发执行看成「各模块的低层操作可以被任意地穿插排列」。一个实现若**对某些交错正确、对另一些交错错误**，它就含有竞态条件。目的：把「时序相关」这个模糊的担忧，变成可以逐个交错检查的具体问题。 – **直观解释（"它是什么？"）**：`balance = balance + 1` 在处理器层面是「读—加—写」三步。两个线程各自读到 0，各自算出 1，各自写回 1——结果只存进了 1 美元，另一美元凭空消失。sp21 特别强调：`balance = balance + 1`、`balance += 1`、`++balance` 三个版本**有同样的竞态**，现代编译器甚至生成完全相同的代码——所以**看一行 Java 代码根本判断不出它是否安全**。更糟的是**重排序（reordering）**：处理器可能把变量缓存到寄存器再回写，回写顺序与代码顺序不同，所以 `answer = 42; ready = true;` 之后，另一个线程可能先看到 `ready == true` 却仍看到旧的 `answer`。 – **关键规则与最佳实践**： – 「一行 Java 代码」不等于「一个原子操作」；**原子操作的边界由处理器与内存模型决定**，不由源代码的行数决定。 – 任何「读—改—写」序列（自增、`check-then-act`、`get-modify-set`）都是竞态的高危候选。 – 在 sp22 的 TS 模型里，交错只发生在 `await` 点；在 Java 多线程模型里，线程可以在**任意点**被抢占（preemptive），所以推理难度大得多——这正是必须用锁这类显式机制的原因。 – 消息传递**不能**消除竞态：当一个客户端必须向服务端发送多条消息才能完成一件事时，这些消息与其他客户端的消息会交错（见 Reading 24 的「LOOK before you TAKE」）。

互斥与临界区（Mutual Exclusion and Critical Section） – **定义与目的**：**互斥**指「一段代码在同一时刻只有一个计算在运行，其他可能访问同一份共享数据的并发计算被排除在外」。这样的代码区间称为**临界区**。互斥是防止竞态的根本思想。sp22 给出了两个必须反复自问的问题：(1) **哪里可能发生危险的交错？**（找出需要防御的地方）；(2) **哪里必须绝对禁止交错？**（建造抵御 bug 的墙）。 – **直观解释（"它是什么？"）**：互斥就像**手术室的独占时段**：医生（一个线程）在手术期间，别人不能进来动同一具身体；其他手术室（其他数据）互不影响。sp22 的关键观察是：在单线程协作式并发里，**只要一段代码里一个 `await` 都没有，就可以**

确信没有任何异步回调或异步函数会与它交错——代码可能提前返回或抛异常，但如果控制流干净地走到末尾，那它一定是**不被打断地**跑完的。 – **关键规则与最佳实践**： – 先找交错点，再决定在哪里建墙；不要盲目地把 `synchronized` 撒满全程序。 – 在 TS 中：**每一个 `await` 都是可能失去控制权的地方**；恢复控制权时可能需要对条件**重新检查**。 – 把「变更（mutation）」推迟到所有条件都就绪之后，然后**一次性做完，中途不失去控制权**。 – 在 Java 中：临界区由 `synchronized` 块/方法（或 `Lock.lock()/unlock()`）划定；互斥只对**获取同一把锁**的其他线程有效。

锁作为抽象数据类型：acquire 与 release（Lock as an ADT） – **定义与目的**：**锁**是一种抽象，允许「至多一个线程」在某一时刻拥有它。它有两个操作：**acquire** 获取所有权（若已被别的线程持有，就**阻塞**直到对方 release，然后与其它竞争者争抢，**胜者不确定**）；**release** 释放所有权。持有锁是一个线程向其他线程宣告：「我正在处理这个东西，现在别碰。」使用锁还会告诉编译器与处理器「这里在并发使用共享内存」，从而避免寄存器/缓存回写顺序导致的重排序问题。 – **直观解释（"它是什么？"）**：锁就是**更衣室的钥匙**。你拿走钥匙（acquire）才能进去（访问共享数据），出来还钥匙（release）别人才进得去。注意：钥匙放在那儿本身不会阻止任何人**硬闯**——必须所有进更衣室的人都遵守「先拿钥匙」的约定。 – **关键规则与最佳实践**： – **锁只是一种约定（a convention）**：如果有一个写得不好的客户端没有获取正确的锁，系统就不再是线程安全的。 – **阻塞（blocking）** 的一般含义是「线程不做别的事，一直等到某个事件发生」；`acquire` 阻塞时等的事件就是「持有者 release」。 – 锁**只对获取同一把锁的线程**提供互斥；不获取锁的代码可以随意破坏数据。 – 锁通常还要**保护数据**，而不是只保护代码：持有某对象的锁，并不能阻止别的线程访问那个对象——它只能阻止别的线程进入它们自己的、以同一对象为锁的 `synchronized` 块。 – 用锁的代码必须保证 release 一定发生（Java 的 `synchronized` 由语言保证；`ReentrantLock` 必须写在 `finally` 里）。

Java 的内置锁与 synchronized（Intrinsic Locks and synchronized） – **定义与目的**：Java 把锁做成了语言内建特性：**每一个对象都隐式地关联着一把锁**——`String`、数组、`ArrayList`、你自己创建的每个类的实例，甚至一个平凡的 `Object` 都有锁，所以 `Object lock = new Object();` 常被用作纯粹的锁对象。Java 不允许你直接调用 `acquire`/`release`，而是用 **`synchronized` 语句块**在块的作用域内自动获取与释放锁。 – **直观解释（"它是什么？"）**：`synchronized (lock) { ... }` 就是「进门拿钥匙、出门还钥匙」的自动门：进门时若钥匙不在就等着，出门时（无论是正常结束、`return` 还是抛异常）一定还钥匙。这种块提供**互斥**：在由同一个对象锁保护的临界区里，同一时刻只有一个线程——就与那个对象相关的其它 `synchronized` 区而言，你又回到了「顺序编程的世界」。 – **关键规则与最佳实践**： – `synchronized (obj) { ... }` 只做一件事：阻止其他线程进入它们自己的、以**同一个对象**为锁的 `synchronized` 块。仅此而已。 – **读也要加锁**，不只是写：如果读不加锁，读线程可能看到表示被修改到一半的状态。 – 你必须显式地、小心地把**每一次访问**都用恰当的 `synchronized` 块或方法关键字保护起来。 – 构造方法**不允许**加 `synchronized` 关键字（语法上被禁止），因为构造中的对象应当被限定在单个线程内，直到构造方法返回；确实需要时可以在构造方法体内写 `synchronized (this) { ... }`。 – **Java 的锁是可重入的（reentrant，补充说明）**：同一个线程可以重复获取自己已持有的锁，因此 `synchronized (obj) { synchronized (obj) { ... } }` **不会死锁**，内层退出后线程仍然持有该锁。这条性质使得「对象方法互相调用」很自然，但也让死锁更容易在**两个不同对象**之间悄悄发生。

监视器模式与锁的纪律（Monitor Pattern and Locking Discipline） – **定义与目的**：写类的方法时最方便的锁就是对象实例自身（`this`）。**监视器模式**的做法是：把整个表示（rep）用一把锁保护起来，所有访问 rep 的方法都在 `synchronized (this)` 内执行；**监视器（monitor）** 就是「方法之间互斥、同一时刻只有一个线程能进入其实例」的类。Java 提供了语法糖：在方法签名上加 `synchronized`，效果等同于把整个方法体包在 `synchronized (this)` 里。配套的**锁的纪律（locking discipline）**有两条：每个共享可变变量都必须被某把锁保护，除了在该锁的 `synchronized` 块内不得读写；如果一个不变量涉及多个共享可变变量（甚至跨对象），那么**所有相关变量必须由同一把锁保护**，并且在释放锁之前必须重建该不变量。 – **直观解释（"它是什么？"）**：监视器模式是「**一个房间一把钥匙**」：类的实例就是一整间房，所有方法（包括看似微不足道的 `length()`、`toString()`）都必须拿同一把钥匙进门。为什么不给 `length()` 免检？因为它读的正是别人正在改的东西。 – **关键规则与最佳实践**： – **每一个**公开方法都加锁，包括观察器（observer）；不要有例外。 – 把线程安全论证**写在类里、紧挨着表示不变量**：例如「对 `text` 的所有访问都发生在 `SimpleBuffer` 的方法内，而这些方法全部由 `SimpleBuffer` 的锁保护」。 – **封装是论证成立的前提**：如果 `text` 是 `public` 的，客户端就能不加锁地读写它，监视器模式立刻失效（这是表示泄漏在并发语境下的后果）。 – 锁的对象必须是**所有客户端都能拿到且都不会换的**：`this`、专用的 `private final Object lock`、或文档中明示可用的对象。 – 别把 `synchronized` 加到 `static` 方法上指望它保护实例数据：那会获取**整个类的静态锁**，既伤害性能又保护不到正确的东西。

原子性与复合操作（Atomicity and Compound Operations） – **定义与目的**：一个操作是**原子的（atomic）**，意指它相对于其他线程不可被拆分、不可被打断。互斥的意义正在于把「若干步骤」变成一个原子区。危险之处在于：**每个方法各自原子，不等于「若干个方法的组合」也原子。** `findReplace` 就是典型：它先 `buf.toString()` 找到下标，再 `delete`，再 `insert`——三次调用各自原子，但整个方法不是，因为别的线程可能在中途改动缓冲区，导致删错区域、插错位置。 – **直观解释（"它是什么？"）**：原子性像**复印合同**：只签一页、只盖一个章，都不算签完；必须「检查—签字—盖章」一口气完成，中途别人不能把合同抽走。 – **关键规则与最佳实践**： – 需要「多个操作合起来原子」时，必须让它们位于**同一个** `synchronized` 区域——可以扩大方法内的同步区，也可以在客户端 `synchronized (buf) { ... }` 把三次调用包起来。 – 若要在客户端加锁，**必须在接口的规格/注释里明确写出**「客户端之间可以用该对象本身互相同步」，否则锁的约定不成立（*Clients may synchronize with each other using the EditBuffer object itself.*）。 – 更好的做法是从 ADT 设计上消灭这种需求：为并发设计的数据类型应当提供**语义良好的原子操作**，例如 `ConcurrentMap.putIfAbsent(key, value)` 是 `if (!map.containsKey(key)) map.put(key, value);` 的原子版本，`map.replace(key, value)` 是 `if (map.containsKey(key)) map.put(key, value);` 的原子版本（**补充说明**：这两个方法来自 `java.util.concurrent`）。 – 不要用「加锁」来掩盖「接口本身对并发不友好」：`EditBuffer` 依赖整数下标，而下标对别人的增删极其脆弱；更友好的设计是引入 `Position`（游标位置）或 `Selection`（选区）类型，让位置能在周围文本被修改时保持含义并主动报告冲突。

锁的粒度（Lock Granularity）与并发性能 – **定义与目的**：**粒度**指一把锁保护多少数据。细粒度锁（每个对象一把锁）允许更多并行，但需要同时获取多把锁，容易死锁；**粗粒度锁（coarse-grained locking）** 用一把锁保护许多对象实例甚至整个子系统，简单、不易死锁，但会牺牲并行度。目的是在「性能」与「正确性/可维护性」之间做出有意识的取舍。 – **直观解释（"它是什么？"）**：细粒度像**每间办公室一把钥匙**（互不干

扰，但你要进两间就可能和人对撞）；粗粒度像**整层楼一把钥匙**（绝不会对撞，但同一时间只能有一个人在这层楼里干活）。 – **关键规则与最佳实践**： – 应用层编程通常优选**粗粒度锁或限定**；细粒度锁主要用于操作系统内核与设备驱动，那里需要极致性能并配合锁顺序（**补充说明**：这是 sp21 原文的原话归纳）。 – 库数据结构通常**不加同步**（把协调留给调用者以保证单线程性能），或者采用**监视器模式**。 – 图形界面工具包（如 Java Swing）常采用**线程限定**：只允许一个专用线程访问整棵组件树，其他线程必须通过消息传递请求它代劳。 – 搜索类问题常用**不可变数据类型**：没有可变状态，就没有竞态也没有死锁。 – 同步是有代价的：一次同步方法调用可能显著变慢，因为要获取锁、操作共享存储、与其他处理器通信——**不需要同步时就不要同步**。

死锁：成因、四个必要条件与预防（Deadlock） – **定义与目的**：**死锁**发生在并发模块互相等待对方做某件事时；可能涉及两个以上模块（A 等 B、B 等 C、C 等 A）。**死锁的本质特征是依赖关系中存在环**。它不威胁正确性（safety），而威胁**存活性（liveness）**：程序不再推进。经典的操作系统理论给出**四个必要条件（补充说明）**：互斥（资源独占）、持有并等待（hold and wait）、不可抢占（no preemption）、循环等待（circular wait）——四者同时成立才可能死锁，因此预防策略就是打破其中至少一条。 – **直观解释（"它是什么？"）**：银行转账是标准剧本：A 与 B 同时做两账户之间的转账，A 先锁住「转出账户 1」，B 先锁住「转出账户 2」，然后 A 等账户 2 的锁、B 等账户 1 的锁——**致命拥抱（deadly embrace）**，两人都卡住。sp22 的图书馆版本是：Frodo 持有 Two Towers 等着 Return of the King，Gandalf 持有 Return of the King 等着 Two Towers。 – **关键规则与最佳实践**： – **预防方案一：锁顺序（lock ordering）**。给需要同时获取的锁定一个全序，所有代码都按该顺序获取。这样 A 若先拿到 Harry 的锁，也必然先拿到 Snape 的锁，等待图中不可能出现环。（sp21 用的是「按人名首字母排序」，并留了一个思考：真实社交网络里人名会重复，更好的排序键是**稳定唯一的标识**，如账户号或对象身份哈希。） – 锁顺序的缺点：**不模块化**（代码必须知道系统里所有的锁），而且**在拿到第一把锁之前往往无法知道还需要哪些锁**（例如对图做深度优先搜索）。 – **预防方案二：粗粒度锁**。用一把锁保护多个对象甚至整个子系统（如让所有 `Wizard` 共用所属 `Castle` 的锁），简单可靠，但可能把程序退化成「同一时刻只有一个线程能推进」。 – **重入性**可以避免「自己等自己」（同一线程重复获取同一把锁），但不能避免两个对象之间的循环等待；`Wizard.friend` 的经典死锁正是后者。 – 值得注意：死锁常常**可以**不发生（例如 A 在 B 拿到第一把锁之前就完成了两把锁的获取与释放），这种「时有时无」使它和竞态一样难以复现与调试。

ReentrantLock 与 synchronized 的取舍（ReentrantLock vs synchronized，补充说明） – **定义与目的**：sp22 原文在「其它互斥技术」中提及 locks、mutexes、semaphores 属于更底层的原语；在 Java 中，除了内建的 `synchronized`，`java.util.concurrent.locks.ReentrantLock` 提供了显式的锁对象：`lock()` / `unlock()`、`tryLock()`、`tryLock(timeout, unit)`、`lockInterruptibly()`、公平锁选项，以及可以绑定多个 `Condition`（用于等待/通知）。**补充说明**：这些是 Java 生态的机制，不在 6.031 的必讲范围内，但它们解释了「为什么有时要用显式锁」。 – **直观解释（"它是什么？"）**：`synchronized` 是**自动门**（进出自动上锁/解锁，简单但只能在块结构内结束）；`ReentrantLock` 是**手动门**（可以试着推一下看能不能进、可以定个等待上限、可以被中断叫停，但你必须记得出来时把门锁上）。 – **关键规则与最佳实践**： – 用 `ReentrantLock` 时，`unlock()` **必须**放在 `finally` 里，否则一旦抛异常，锁将永久泄漏。 – 需要「尝试获取、失败就做别的事」或「设定等待上限」时用 `tryLock`——这是打破死锁四条件中「持有并等待/不可

抢占」的实用手段。 – 默认选择仍应是 `synchronized`（或干脆用监视器模式）：更少的代码、更少的出错机会、更容易维护。 – 无论用哪种锁，**锁顺序与锁的纪律的论证都要写下来**。

安全性、存活性与其它互斥技术（Safety, Liveness, and Other Techniques） – **定义与目的**：把并发程序的正确性拆成两类性质：**安全性（Safety）**——程序是否满足其不变量与规格说明？即「能否证明坏事永不发生」（竞态威胁安全性）；**存活性（Liveness）**——程序是否会持续运行并最终做到你想做的事？即「能否证明好事终将发生」（死锁威胁存活性）。此外还有**公平性（fairness）**：模块是否被给予推进所需的处理能力，主要由操作系统的线程调度器决定，但可以通过线程优先级施加影响。 – **直观解释（"它是什么？"）**：安全性是「绝不闯红灯」，存活性是「最终能到达目的地」。一个既不撞车也永远开不动的程序，是「安全但不活」的。 – **关键规则与最佳实践**： – 分析并发程序时，分别问「会不会把数据搞坏」（safety）与「会不会卡住」（liveness），两者需要不同的推理与不同的防御。 – sp22 指出其他互斥技术包括：**锁/互斥量/信号量**（多线程抢占式环境），以及**数据库事务**——事务为一组读写提供互斥式的原子效果，广泛用于分布式客户端/服务器系统；事务不一定要显式加锁，冲突时可以失败并回滚，数据库还能自动管理加锁顺序。 – 协作式并发（`async` / `await`）不只存在于 TypeScript/JavaScript：Python、Swift、Rust、C# 都有类似机制（**补充说明**，sp22 原文提及）。 – 抢占式并发超出 6.031 范围，sp21 原文指向 6.033（计算机系统工程）与 6.039（操作系统工程）进一步学习。

代码示例与对比分析

场景 1：银行账户——裸的共享可变字段，还是监视器模式？

❌ *错误代码*

```
/** 一台可以被多个取款机共享的银行账户。 */
public class BankAccount {
    // 错误：共享可变数据，没有任何保护
    private long balance;

    public BankAccount(long initial) {
        this.balance = initial;
    }

    /** 存钱。 */
    public void deposit(long amount) {
        balance = balance + amount;      // 读–加–写，不是原子操作
    }

    /** 取钱。 */
    public void withdraw(long amount) {
        balance = balance - amount;      // 同样不是原子操作
    }

    /** 查询余额。 */
    public long getBalance() {
        return balance;                  // 读也不安全：可能读到改了一半的状态
    }
}
```

【错误代码的问题】 1. **丢更新（lost update）**：两个线程同时 `deposit(1)`，可能都读到 0、都算出 1、都写回 1，最终余额只增加了 1。sp21 的取款机例子中，成对的存/取交易本该让余额保持为 0，实际却经常不为 0。 2. **不可复现**：竞态是 heisenbug，取决于调度、其他进程、机器负载；加一行 `System.out.println` 常常就让 bug「消失」（只是被掩盖）。 3. **`getBalance()` 也不安全**：`long` 在现代 64 位 JVM 上通常是原子的，但 Java 语言规范**不保证**这一点（非 `volatile` 的 `long` / `double` 允许被撕裂读取），而且一旦 rep 变成多个字段（例如还要维护交易计数），读操作就可能看到「改到一半」的表示，违反表示不变量。 4. **没有任何线程安全论证**：代码里既没有锁也没有注释说明它是单线程专用的，维护者无法判断能否安全地在多线程环境使用它。

✅ *正确代码*

```
/**
 * 一台可以被多个取款机共享的银行账户。
 *
 * Rep invariant:
 *   balance >= 0
 * Abstraction function:
 *   AF(balance) = 一个余额为 balance 分的银行账户
 * Safety from rep exposure:
 *   balance 是 private 的，且方法不返回 rep 的别名
 * Thread safety argument:
 *   所有对 balance 的访问都发生在 BankAccount 的方法内，
 *   而这些方法全部由 BankAccount 实例自身的锁（监视器模式）保护；
 *   由于每一次读–改–写都在同一个临界区内完成，操作是原子的。
 */
public class BankAccount {
    private long balance;

    public BankAccount(long initial) {
        this.balance = initial;
        checkRep();
    }

    private void checkRep() {
        assert balance >= 0;
    }

    public synchronized void deposit(long amount) {
        balance = balance + amount;
        checkRep();
    }

    /** @throws IllegalArgumentException 如果余额不足 */
    public synchronized void withdraw(long amount) {
        if (amount > balance) {
            throw new IllegalArgumentException("insufficient funds");
        }
        balance = balance - amount;
        checkRep();
    }

    public synchronized long getBalance() {
        return balance;
    }
}
```

【为什么这样更好】 每个公开方法都被同一把锁（`this`）保护，「读—加—写」被整体变成一个原子区，因此不可能再出现两个线程各自读到同一个旧值的情形。类注释里的**线程安全论证**与 `checkRep()` 一起，把「这个类型为什么安全」变成可核查的文字，符合「锁的纪律要写下来」的要求。`getBalance()` 同样加锁，因为**读也可能看到部分修改的状态**；在 sp21 的 `SimpleBuffer` 例子中，连 `length()` 和 `toString()` 都被刻意加了锁，理由完全相同。

【代码对比解说】 有人会问：`balance = balance + amount` 只有一行，为什么需要锁？sp21 的回答是：**你无法从 Java 代码看出处理器会执行哪些原子操作**——`=`、`+=`、`++` 三个版本的编译结果甚至完全相同，却都含有同样的竞态。唯一可靠的办法是划定临界区。另一个常见误区是把 `long balance` 改成 `volatile`（**补充说明**）：`volatile` 只保证可见性与不重排序，**不保证**「读—改—写」的原子性，所以两个并发的 `deposit(1)` 依然可能丢更新；正确做法要么加锁，要么用 `AtomicLong.addAndGet`（后者是「使用已有的线程安全类型」策略的例子）。

【设计原则透视】 这是**表示不变量（RI）+ 抽象边界**与并发的结合：`balance >= 0` 这条不变量只有在「检查—扣减」被同一把锁保护时才能维持；一旦客户端能直接读到、写到 rep（`public` 字段），不变量就无从保证——sp21 明确说：*If `text` were public, then clients would be able to read and write it without first acquiring the lock, and SimpleBuffer would no longer be threadsafe.* 换言之，**封装是线程安全论证的前提条件**，这与 Reading 11 中「防止表示暴露」的论证是同一条原则。

场景 2：两账户转账——先锁自己的账户，还是按全局顺序锁?

❌ *错误代码*

```
/**
 * 错误：每个线程都先锁「转出账户」，再锁「转入账户」。
 * 两个方向相反的转账会互相等待，形成死锁。
 */
public class DeadlockingTransfer {

    public static void transfer(BankAccount from, BankAccount to, long amount) {
        synchronized (from) {                  // 线程 A 拿到账户 1；线程 B 拿到账户 2
            synchronized (to) {                // A 等账户 2；B 等账户 1 → 致命拥抱
                from.withdraw(amount);
                to.deposit(amount);
            }
        }
    }
}
```

【错误代码的问题】 1. **死锁（liveness 失败）**：线程 A 持有 1 等 2，线程 B 持有 2 等 1，等待图中出现环，两者永久卡住；账户被锁住，系统停止服务。 2. **时有时无，极难复现**：若 A 在 B 拿到第一把锁之前就完成了两次获取与释放，就一切正常——这是典型的 heisenbug 式缺陷（sp21 原文：*If the locks involved in a deadlock are also involved in a race condition … then the deadlock will be just as difficult to reproduce or debug.*）。 3. **可扩展性差**：任何第三处「先锁 B 再锁 A」的代码（比如一个审计方法、一个批量转账）都会重新引入环，靠人工审查很难维持。 4. **持有锁期间做危险工作**：如果 `withdraw` / `deposit` 内部还要做 I/O 或回调，持锁时间会被拉长，加剧争用与死锁概率。

✅ *正确代码*

```
/**
 * 正确：给锁定一个全局顺序（按账户号的自然序），所有代码都按这个顺序获取，
 * 从而在等待图中不可能出现环。
 */
public class OrderedTransfer {

    public static void transfer(BankAccount from, BankAccount to, long amount) {
        if (from == to) {                       // 自转账：一把锁就够，避免自锁比较的歧义
            from.withdraw(amount);
            to.deposit(amount);
            return;
        }
        BankAccount first  = from.accountId() < to.accountId() ? from : to;
        BankAccount second = (first == from) ? to : from;

        synchronized (first) {                  // 所有线程都从「小账户号」开始
            synchronized (second) {
                from.withdraw(amount);
                to.deposit(amount);
            }
        }
    }
}
```

【为什么这样更好】 有了全序之后，A 若先拿到账户 1 的锁，也必然先拿到账户 2 的锁；B 只有在 A 释放账户 1 之后才能开始，于是两个线程的获取顺序一致，等待图中不可能出现环。这正是 sp21 的锁顺序方案（原文用 `this.name.compareTo(that.name) < 0` 按人名排序），并且回答原文留下的问题：**真实社交网络里人名会重复**，所以更好的锁序键是**稳定且唯一的标识**（账户号、用户 ID；**补充说明**：Java 中常用 `System.identityHashCode(obj)` 作为兜底，但它有极小概率碰撞，严谨实现需要再加一层 tie-breaker，例如 `ConcurrentHashMap` 内部的做法）。

【代码对比解说】 `synchronized (first) { synchronized (second) { ... } }` 是标准的「嵌套锁」写法，注意它之所以可行，前提是方法内部调用的 `withdraw` / `deposit` 又去获取同一批锁时**不会卡住自己**——这依赖 Java 锁的**可重入性**。可重入性可以消解「同一线程重复获取同一把锁」的自锁，但**不能**消解两个线程之间的循环等待：`Wizard.friend` 的死锁就是两把不同的锁在两条线程间形成的环。因此「嵌套锁」的正确用法永远是配合**顺序**或**粗粒度**策略，而不能指望可重入性救场。

用等价但更明确的显式锁写法（**补充说明**）可以再加一层保险：

```
import java.util.concurrent.locks.ReentrantLock;

public final class LockedAccount {
    private final ReentrantLock lock = new ReentrantLock();
    private long balance;

    public boolean tryTransferFrom(long amount) {
        if (!lock.tryLock()) {          // 拿不到就先做别的事，绝不死等
            return false;
        }
        try {
            if (amount > balance) return false;
            balance -= amount;
            return true;
        } finally {
            lock.unlock();              // 必须放在 finally 里
        }
    }
}
```

【设计原则透视】 死锁是**存活性**问题，它不会破坏不变量（钱不会凭空出现或消失），但会让系统「永不推进」。因此分析死锁要用「画等待图找环」的方法，而不是检查后置条件。锁顺序策略体现了「把全局约束集中到一处」的设计思想，但代价是**不模块化**——代码必须知道系统里所有的锁；粗粒度锁用「牺牲并行度」换取「局部可推理」，而 `tryLock` 则通过打破「持有并等待/不可抢占」来直接消除环。三种手段对应的是同一组死锁必要条件的不同破法。

场景 3：只给修改器加锁 + 暴露 rep——还是把整个表示关进监视器?

❌ *错误代码*

```
/** 错误：rep 暴露，观察器不加锁。 */
public class SimpleBuffer implements EditBuffer {
    // 错误一：public 字段，客户端可以完全绕开锁读写
    public String text = "";

    public SimpleBuffer() {
        text = "";
    }

    // 错误二：只有 mutator 加锁，observer 不加
    public synchronized void insert(int position, String insertion) {
        text = text.substring(0, position) + insertion + text.substring(position);
    }

    public synchronized void delete(int position, int len) {
        text = text.substring(0, position) + text.substring(position + len);
    }

    /** 未加锁的观察器：可能读到修改到一半的表示。 */
    public int length() {
        return text.length();
    }

    /** 未加锁的观察器：返回以后 text 立刻可能被别的线程替换。 */
    public String toString() {
        return text;
    }
}
```

【错误代码的问题】 1. **封装失守导致论证失效**：`text` 是 `public` 的，任何客户端都能不加锁地读写它；此时无论类内部多小心，**锁的约定已经被破坏**，sp21 原文明确指出这会直接使类型不再是线程安全的。 2. **观察器可能看到「改到一半」的状态**：`insert` 的实现是「切两段 + 拼接 + 整体赋值」，虽因引用赋值而瞬间完成，但若 rep 变成多字段（例如 `GapBuffer` 的 `char[] a` + `gapStart` + `gapLength`），未加锁的 `length()` 完全可能读到 `gapStart` 与 `gapLength` 不匹配的中间态，违反 `0 <= gapLength <= a.length - gapStart`。 3. **返回内部别名的风险**：`toString()` 返回 `text` 本身（`String` 不可变所以这里侥幸安全），但如果 rep 是可变类型（数组、`List`），返回别名就等于把 rep 交给客户端，安全性与线程安全同时崩塌。 4. **无文档化的锁纪律**：维护者看不出「哪个锁保护哪些字段」，后续改动极易漏加锁。

✅ *正确代码*

```
/**
 * SimpleBuffer 是一个线程安全的 EditBuffer，使用简单的 rep。
 *
 * Rep invariant:
 *   true
 * Abstraction function:
 *   AF(text) = 字符序列 text[0], ..., text[text.length()-1]
 * Safety from rep exposure:
 *   text 是 private 且不可变
 * Thread safety argument:
 *   所有对 text 的访问都发生在 SimpleBuffer 的方法内，
 *   而这些方法全部由 SimpleBuffer 的锁（监视器模式）保护。
 */
public class SimpleBuffer implements EditBuffer {
    private String text;

    public SimpleBuffer() {
        text = "";                  // 构造方法不加 synchronized：对象尚未逸出
        checkRep();
    }

    private void checkRep() {
        assert text != null;
    }

    public synchronized void insert(int position, String insertion) {
        text = text.substring(0, position) + insertion + text.substring(position);
        checkRep();
    }

    public synchronized void delete(int position, int len) {
        text = text.substring(0, position) + text.substring(position + len);
        checkRep();
    }

    public synchronized int length() {
        return text.length();
    }

    public synchronized String toString() {
        return text;
    }
}
```

【为什么这样更好】 所有共享可变数据（此处即 `text`，也就是表示不变量所依赖的全部字段）都由**同一把锁**保护，因此两条锁的纪律同时满足：每个共享可变变量都有锁，且涉及不变量的所有变量都在同一把锁下、在释放前重建不变量。观察器也加锁，杜绝了「读到部分修改状态」的可能。类注释里的线程安全论证与 `checkRep()` 一起使这个类可维护——后来者改动时会看到论证并知道必须保持它。

【代码对比解说】 sp21 用一个很尖锐的练习说明「锁对象≠对象」：假设 `list` 是 `ArrayList<String>`，某线程进入 `synchronized (list) { ... }` 时，它**拥有 `list` 的锁**，但这**并不**阻止其他线程使用 `list`

的观察器或修改器——只有那些**自己也去获取同一把锁**的线程才会被挡住。所以两个加法缺一不可：加锁 + 所有访问者都遵守同一约定。`synchronized` 关键字写在方法签名上只是 `synchronized (this) { ... }` 的语法糖，用哪种写法不重要，重要的是**锁的对象对所有访问者一致**。

【设计原则透视】 这里体现了 **AF / RI / 表示暴露防护 / 线程安全论证** 四者的合流：RI 描述「表示始终必须满足什么」，线程安全论证描述「谁来保证它在并发下仍然成立」，而**防止表示暴露**是论证能够成立的前提。synchronized 方法把「读—改—写」变成原子区，等价于在 AF 的层面上保证「客户端观察到的永远是一个合法的抽象值」。

场景 4：跨多个方法的原子操作——客户端的发散调用，还是共享一个锁?

❌ *错误代码*

```
/**
 * 错误：findReplace 对 buf 做了三次调用，虽然每次调用各自原子，
 * 但整个方法不是原子的——别的线程可以在中间改动缓冲区。
 */
public final class TextOps {

    /**
     * 把 buf 中第一处 pattern 替换为 replacement。
     * @return 发生了替换则为 true
     */
    public static boolean findReplace(EditBuffer buf, String pattern, String replacement) {
        int i = buf.toString().indexOf(pattern);   // ① 观察
        if (i == -1) {
            return false;
        }
        buf.delete(i, pattern.length());           // ② 删除（此时别的线程可能已插入文本）
        buf.insert(i, replacement);                // ③ 插入（位置可能已经错了）
        return true;
    }
}
```

【错误代码的问题】 1. **检查—再行动（check-then-act）竞态**：`indexOf` 得到下标 `i` 之后，别的线程可能在 `i` 之前插入或删除文本，于是 ② 删掉的是**错误的区域**，③ 把替换文本插到了**错误的位置**——数据被静默破坏。 2. **三次调用之间失去互斥**：每个方法内部虽原子，但方法**之间**存在交错窗口（在 sp22 的语境里，这相当于在两次 `await` 之间丢失了控制权却没做检查）。 3. **`i == -1` 的返回值语义不可靠**：即使返回 `true`，也无法保证「替换谁替换成了什么」与调用者的预期一致。 4. **错误地依赖了「每个方法原子」这一弱保证**：这正是本讲反复强调的——原子性的单位是**临界区**，不是**方法**。

✅ *正确代码*

```
/**
 * 正确：客户端之间约定用 EditBuffer 对象本身互相同步，
 * 从而把三次调用扩大成同一个原子区。
 *
 * 与之配套，EditBuffer 的接口必须写明：
 *   Clients may synchronize with each other using the EditBuffer object itself.
 */
public final class TextOps {

    /**
     * 把 buf 中第一处 pattern 替换为 replacement。
     * @return 发生了替换则为 true
     */
    public static boolean findReplace(EditBuffer buf, String pattern, String replacement) {
        synchronized (buf) {                        // 与 buf 的所有其他客户端互斥
            int i = buf.toString().indexOf(pattern);
            if (i == -1) {
                return false;
            }
            buf.delete(i, pattern.length());
            buf.insert(i, replacement);
            return true;
        }
    }
}
```

【为什么这样更好】 这把监视器模式已经在每个方法周围建立的同步区**扩大**成一个更大的原子区，保证三次方法调用连续执行、不受其他线程干扰。它之所以成立，前提是 `EditBuffer` 的规格明确宣告「客户端可以用该对象本身互相同步」——这既是文档，也是**协议**：锁的约定必须所有参与方都遵守才有意义。

【代码对比解说】 一个诱人的「偷懒修法」是给方法加上 `static synchronized`：

```
// 看似修好了，其实两个目标都没达到
public static synchronized boolean findReplace(EditBuffer buf, String pattern, String repla
```

这样确实获取了一把锁，但因为是 `static` 方法，它获取的是**整个类的静态锁**，而不是实例对象的锁。后果有两重：其一，**性能灾难**——同一时刻只允许一个线程执行 `findReplace`，哪怕它们在编辑**完全不同的文档**（对多用户编辑器来说，相当于全系统只能有一个人做查找替换）；其二，**保护无效**——真正改动文档的其他代码并不会获取这把类锁，所以竞态依旧存在。这个例子是对「线程安全就是把 `synchronized` 撒满全程序」这一误解最有力的反驳。

另一个更进一步的修法是从接口层面消除问题：`EditBuffer` 依赖整数下标，而下标对别人的增删**极其脆弱**。更好的设计是引入 `Position`（能抵抗周围插入删除的游标位置）或 `Selection` 类型；若 `Position` 周围的文本被其他线程删光，它可以**主动告知**后续客户端（例如抛出异常），让客户端决定怎么办。这正是「为并发而设计数据类型」的含义。

【设计原则透视】 本场景把 **Reading 6 的规格说明**与锁的纪律绑在了一起：锁的约定（谁能拿哪把锁）是接口契约的一部分，不写进规格就无法被客户端遵守；同时它还展示了**抽象边界**的另一面——如果接口的形状（整数下标）本身就迫使客户端做非原子操作，那么再好的加锁也治不了根，得回到 Reading 10/12 的 ADT 设计层面改操作集合。`ConcurrentMap.putIfAbsent` / `replace` 正是「把常用复合操作做成原子操作」的标准范例。

场景 5：社交网络好友关系——细粒度锁互相调用，还是一把粗粒度锁?

❌ *错误代码*

```
import java.util.HashSet;
import java.util.Set;

/**
 * 错误：用监视器模式实现双向好友关系，friend() 会去调用对方的方法，
 * 于是同时持有两把锁——两条线程方向相反时必然死锁。
 */
public class Wizard {
    private final String name;
    private final Set<Wizard> friends;

    // Rep invariant:
    //   好友链是双向的：对每个 f in friends，f.friends 包含 this
    // Concurrency argument:
    //   监视器模式：对 rep 的所有访问由本对象的锁保护

    public Wizard(String name) {
        this.name = name;
        this.friends = new HashSet<Wizard>();
    }

    public synchronized boolean isFriendsWith(Wizard that) {
        return this.friends.contains(that);
    }

    public synchronized void friend(Wizard that) {
        if (friends.add(that)) {
            that.friend(this);          // 拿着自己的锁，去请求对方的锁 → 死锁风险
        }
    }

    public synchronized void defriend(Wizard that) {
        if (friends.remove(that)) {
            that.defriend(this);        // 同上
        }
    }
}
```

【错误代码的问题】 1. **经典致命拥抱**：线程 A 执行 `harry.friend(snape)` 拿住 Harry 的锁，线程 B 执行 `snape.friend(harry)` 拿住 Snape 的锁，随后 A 等 Snape、B 等 Harry，程序直接停住。sp21 原文形容：*The program simply stops.* 2. **问题的本质是「持有一些锁的同时等待另一些锁」**：即使两个方法各自都正确、都加锁，这个组合仍然可死锁。 3. **时对时错**：若 A 在 B 拿到第一把锁之前就完成了整个调用，程序看起来完全正常——又是一个难以复现的死锁。 4. **不可扩展**：每新增一个需要同时操作两个对象的操作（接受好友申请、批量导入好友），都要重新做一次死锁审查。

✅ *正确代码*

```
import java.util.HashSet;
import java.util.Set;

/**
 * 正确：粗粒度锁——所有 Wizard 属于同一个 Castle，
 * 统一用 Castle 对象的那把锁来同步，任何时刻最多持有一把锁。
 */
public class Wizard {
    private final Castle castle;
    private final String name;
    private final Set<Wizard> friends;

    // Rep invariant:
    //   好友链是双向的：对每个 f in friends，f.friends 包含 this
    // Concurrency argument:
    //   粗粒度锁：凡是访问任何 Wizard 的 rep 的方法，
    //   都必须在持有 castle 的锁的情况下进行；因此同一时刻
    //   只有一个线程能操作本社交网络中的任何关系，绝不会出现
    //   「持有 A 的锁等待 B 的锁」的情形。

    public Wizard(Castle castle, String name) {
        this.castle = castle;
        this.name = name;
        this.friends = new HashSet<Wizard>();
    }

    public boolean isFriendsWith(Wizard that) {
        synchronized (castle) {
            return this.friends.contains(that);
        }
    }

    public void friend(Wizard that) {
        synchronized (castle) {
            if (this.friends.add(that)) {
                that.friend(this);      // 仍然是嵌套调用，但只用一把锁 → 不会死锁
            }
        }
    }

    public void defriend(Wizard that) {
        synchronized (castle) {
            if (this.friends.remove(that)) {
                that.defriend(this);
            }
        }
    }
}
```

【为什么这样更好】 所有涉及好友关系的操作都只获取**同一把锁**，所以「持有 A 的锁等待 B 的锁」这种情形在结构上不可能出现，等待图里不可能有环。维护双向不变量的代码（`that.friend(this)`）依旧可以自然

书写，只是它不再需要第二把锁。代价是并行度：整个社交网络同一时刻只有一个线程能推进（相当于退化为顺序执行），这正是粗粒度锁的典型权衡。

【代码对比解说】 两条路线对应死锁必要条件的不同破法：**锁顺序**（排序后按序获取，破「循环等待」）保持细粒度、保留并行度，但要求代码知道所有锁，且常常「拿到第一把锁之前不知道还需要哪些锁」（sp21 提出的深度优先搜索难题）；**粗粒度锁**（`Castle` 一把锁）简单、模块化味道更好（锁只属于子系统），但牺牲并行。第三种是用 `tryLock` 超时/退避（**补充说明**），破「不可抢占」或「持有并等待」，代价是要处理「拿不到锁时怎么办」这一新问题。工程上，应用层代码通常选粗粒度或限定，操作系统内核才用细粒度 + 严格锁顺序。

【设计原则透视】 本场景展示了**不变量跨越多个对象**时的锁纪律：`f.friends ∋ this` 与 `this.friends ∋ f` 必须同时成立，因此「所有涉及该不变量的变量必须由同一把锁保护」这条规则直接指向粗粒度方案。它同时说明：**加锁的位置是一个设计决策，不是机械动作**——加了锁却选错锁（或选多把锁），会把正确性问题换成存活性问题，而后者更难发现。

场景 6：复合操作与线程安全集合——`synchronizedList` 就够了么？

❌ *错误代码*

```
import java.util.*;

/** 错误：以为用了线程安全集合就万事大吉。 */
public class SharedQueue {
    private final List<String> list = Collections.synchronizedList(new ArrayList<>());

    /** 检查–再行动：两个原子操作合起来并不原子。 */
    public String takeFirst() {
        if (!list.isEmpty()) {          // ① 此刻为空则返回 null
            return list.remove(0);      // ② 但两者之间别的线程可能已把它清空
        }
        return null;
    }

    /** 迭代也需要加锁：否则可能在遍历途中抛 ConcurrentModificationException。 */
    public String join() {
        StringBuilder sb = new StringBuilder();
        for (String s : list) {         // 底层迭代器不是线程安全的
            sb.append(s);
        }
        return sb.toString();
    }
}
```

【错误代码的问题】 1. **检查—再行动竞态**：`isEmpty()` 与 `remove(0)` 各自原子，但组合起来不是；另一个线程可能在两步之间把列表清空，导致 `remove(0)` 抛 `IndexOutOfBoundsException`，或者取走「不打算取走」的元素。 2. **迭代不是原子的**：`Collections.synchronizedList` 的文档明确要求，遍历时必须自行在列表上加锁，否则可能抛 `ConcurrentModificationException` 或读到不一致的内容。 3. **误以为「线程安全集

合 = 我的操作线程安全」：库只保证**单个方法调用**的原子性，不保证客户端自己拼出的复合操作。 4. **接口语义模糊**：`takeFirst()` 返回 `null` 既可能是「队列为空」也可能是「元素确实是 null」，调用者无法区分。

✅ *正确代码*

```
import java.util.*;
import java.util.concurrent.*;

/** 正确：复合操作必须整体加锁；或者干脆使用提供原子复合操作的并发类型。 */
public class SharedQueue {
    private final List<String> list = Collections.synchronizedList(new ArrayList<>());

    /** 检查与删除在同一个临界区内完成。 */
    public String takeFirst() {
        synchronized (list) {              // 与所有其他使用 list 的客户端互斥
            if (list.isEmpty()) {
                return null;
            }
            return list.remove(0);
        }
    }

    /** 迭代也必须持锁，防止遍历途中被修改。 */
    public String join() {
        synchronized (list) {
            StringBuilder sb = new StringBuilder();
            for (String s : list) {
                sb.append(s);
            }
            return sb.toString();
        }
    }
}

/** 另一种更彻底的做法：用 BlockingQueue 把「等非空 + 取出」做成一个原子操作。 */
class BlockingSharedQueue {
    private final BlockingQueue<String> queue = new LinkedBlockingQueue<>();

    /** 原子操作：要么取到元素，要么一直等到有元素为止。 */
    public String takeFirst() throws InterruptedException {
        return queue.take();
    }

    /** 原子操作：offer 与 take 都无需客户端额外加锁。 */
    public void put(String s) throws InterruptedException {
        queue.put(s);
    }
}
```

【为什么这样更好】 第一种写法把复合操作包进以 `list` 为锁的临界区，并遵循库文档给出的「迭代时需自行加锁」的约定——这既是正确性要求，也是 sp21 强调的「锁只是一种约定」。第二种写法从**接口设计**上根

治问题：`take()`（阻塞直到取到元素）本身就是原子操作，客户端根本不需要自己拼装「检查—再行动」，因而也不可能拼错。这正是 Reading 24 的主题，也是 `ConcurrentMap.putIfAbsent` / `replace` 这类补充 API 存在的理由。

【代码对比解说】 两种写法代表两种思路：**扩大临界区**（承认需要客户端协作，把协议写进文档）与**改进操作集合**（让每个操作本身语义完整、原子）。sp21 说得直白：*It's sometimes useful to make your datatype's lock available to clients, so that they can use it to implement higher-level atomic operations using your datatype.* 但更好的方向是减少这种需要——这也是为什么 `ConcurrentMap` 要在 `Map` 之上补几个原子方法。选择哪条路，取决于你能否修改接口：能改就改接口，不能改就得文档化锁协议。

【设计原则透视】 本场景把 **ADT 的操作选择**与线程安全直接联系起来：操作的语义决定了客户端是否必须做复合操作，而复合操作正是竞态的温床。`ConcurrentMap.putIfAbsent` 之于 `Map` 就是这种「为并发补操作」的范例；它也与 Reading 24 的消息传递设计呼应——那里的「LOOK before you TAKE」之所以会出错，正是因为协议强迫客户端用多条消息完成一件本可以是一条消息完成的原子操作。

---

sp22 原文的异步版本：用 promise 与 holds 实现互斥（TypeScript 对照）

sp22 的 Running Example 是一个图书馆：`checkout` 是异步的，若书不在馆就等待它被归还。下面的最终实现体现了本讲的两条核心思路：**（1）用循环反复检查条件**（因为 `await` 之后世界可能已经变了）；**（2）把「借出」这一组变更放在一个不含 `await` 的区间里一次性做完**：

```
// sp22 原版 TypeScript 写法（对照用）
public async checkout(books: Array<Book>, user: User): Promise<void> {
  const isInLibrary = (book: Book) => this.inLibrary.has(book);
  const notInLibrary = (book: Book) => ! isInLibrary(book);
  const waitForBook = (book: Book) => { // requires notInLibrary(book)
    const hold = new Deferred<void>();
    this.holdsForBook(book).push(hold);
    return hold.promise;
  };

  // 保守地反复等待：await 之后条件可能已被别人破坏
  while ( ! books.every(isInLibrary) ) {
    await Promise.all(books.filter(notInLibrary).map(waitForBook));
  }

  // 借出：这一段没有任何 await，因此是一个互斥区间
  assert(books.every(isInLibrary));
  for (const book of books) {
    assert(isInLibrary(book));
    this.inLibrary.delete(book);
    this.borrowedByUser(user).add(book);
  }

  this.checkRep();
}
```

对应的 Java 类比（**补充说明**）：Java 的互斥区间由 `synchronized` 划定，而「等待条件成立」由 `synchronized` 配合 `wait()` / `notifyAll()` 或 `Condition.await()` / `signalAll()`（`ReentrantLock`）完成。两者的结构惊人地相似：**「在循环里等待条件 + 在临界区内一次性完成变更」**——sp22 的 `while (!books.every(isInLibrary))` 就相当于 Java 中 `while (!condition) lock.wait();` 里那个必须存在的 `while`（防止虚假唤醒与「醒来后条件又被人破坏」）。

与其他设计原则的关联

本讲是 **Reading 21（并发）** 的直接延续：那一讲建立了共享内存与消息传递两个模型、线程与时间片、交错与竞态、heisenbug 以及四种线程安全策略，并演示了银行账户丢更新的例子；本讲把第四种策略（同步）展开成完整的实现技术。**Reading 22（承诺）** 提供了本讲的另一条线索——交错点与「没有 `await` 的代码段是天然互斥区间」，以及 `Deferred`（承诺者/消费者分离）这一工具，它正是图书馆预约（hold）的实现基础。

本讲的后继是 **Reading 24（消息传递）**：那里给出**不靠锁**的替代路线——让并发模块只通过线程安全的消息通道通信，把可变状态限定在各模块内部，从而绕开「共享可变数据」这个万恶之源；同时也会看到阻塞队列同样会引入死锁（队列满/空导致的循环等待），与锁的死锁是同一类问题。再往后 **Reading 25（套接字与网络）** 把消息传递搬到网络上，形成客户端/服务器架构。

向上游追溯：**Reading 6、Reading 7（规格说明与设计规格）** 说明了为什么「锁协议」必须写进规格（例如 `EditBuffer` 要声明客户端可用它自己互相同步）；**Reading 8（不可变性）** 给出了最省心的替代策略（共享不可变数据既无竞态也无死锁，sp21 提到布尔可满足性搜索天然适合并行化）；**Reading 10、Reading 11（ADT、AF 与 RI）** 提供了「用锁保护表示不变量」「防止表示暴露是线程安全论证的前提」这些论断的理论基础，`checkRep()` 的写法也来自那里；**Reading 12（接口、泛型与枚举）** 关系到「为并发设计操作集合」的接口形态；**Reading 3、Reading 4、Reading 13（测试、代码评审、调试）** 则解释了为什么竞态与死锁**不能**靠测试发现、必须在代码评审阶段用「找交错点/找环」的方式审查。

关键要点

- **并发程序的正确性不应依赖时序的偶然**：任何以「读—改—写」或「检查—再行动」形式出现的操作，都是竞态候选，必须用同一把锁把整段变成原子区。
- **互斥 = 临界区 = 同一把锁下的独占**：在 sp22 的协作式并发里是「不含 `await` 的代码段」，在 Java 里是 `synchronized` 块/方法；**每一个**访问共享可变数据的路径都必须进入临界区（包括观察器）。
- **锁只是约定，封装是前提**：只有所有客户端都获取同一把锁，锁才有效；一旦 rep 暴露（`public` 字段、返回内部别名），线程安全论证立即失效。
- **监视器模式 + 锁的纪律**：一个类的所有方法都由同一个锁（通常是 `this`）保护；涉及同一不变量的所有变量必须由同一把锁保护，并在释放锁前重建不变量；把线程安全论证写在代码里。
- **加锁换来的是存活性风险**：死锁源于「持有一些锁并等待另一些锁」形成的环；解法是**锁顺序**（破循环等待）、**粗粒度锁**（结构上只持一把）、或 **`tryLock` /超时**（破持有并等待/不可抢占）；`synchronized` 的可重入性只能救「同一线程重复取同一把锁」，救不了两个对象之间的环。

常见陷阱与注意事项

- **只给修改器加锁，观察器不加锁 → 读到「改到一半」的表示**：sp21 反复强调监视器模式里连 `length()`、`toString()` 都要加锁。后果是客户端可能观察到违反表示不变量的中间状态，且这种 bug 只在特定时序下出现。
- **给 `static` 方法加 `synchronized` 以「修复」实例数据的竞态 → 既慢又无效**：获取的是整个类的静态锁，导致不同实例的操作被迫串行（多用户编辑器里同一时刻只能有一个人做查找替换），而且真正改动数据的其他代码并不获取这把锁，竞态依旧。
- **把自己的锁暴露出去 / 暴露 rep（`public` 字段、返回可变内部对象）→ 锁协议被客户端绕过**：任何人不加锁就能读写，第 4 种策略立即失效。**补充提醒**：即便是 `Collections.synchronizedList` 返回的「线程安全」列表，迭代与复合操作仍需客户端自行加锁。
- **持有锁期间调用外部代码、做 I/O、或发送消息 → 死锁与性能双重风险**：持锁时间被不可控地拉长，并且外部代码可能反过来请求你的锁（或另一把锁）；`Wizard.friend` 之所以危险，正是因为它持着自己的锁去调用对方的方法。
- **以为「用了 `volatile` 或原子类就一定安全」→ 忽略复合操作仍需原子性**：`volatile` 只保证可见性与重排序约束，不保证「读—改—写」原子；`AtomicLong.incrementAndGet` 原子，但「先判断再自增」依旧需要 `compareAndSet` 循环或锁（**补充说明**）。
- **靠测试或调试来验证并发正确性 → 永远验证不了**：交错数量是天文数字，且缺陷是 heisenbug；正确做法是写出线程安全论证并在代码评审中审查锁的纪律，把「哪把锁保护哪些数据」当作接口契约的一部分来维护。

思考题（带答案）

问题 1：sp22 在单线程的 TypeScript 里说「不含 `await` 的代码段是互斥区间」，而 sp21 在多线程的 Java 里说「`synchronized` 块是临界区」。请解释这两种说法为什么是同一个概念的两个版本，并指出 Java 中**不能**照搬「看代码有没有特殊关键字」这一简单判断的原因。

答案：两者都在描述同一个性质——**在某个区间内，访问同一份共享可变数据的其他计算被排除在外**。在单线程协作式并发的 TS 中，控制权只在 `await`（以及 `return` / `throw`）处转移，所以「不含 `await` 的代码段」天然满足互斥，不需要额外机制；在多线程抢占式并发的 Java 中，线程可以在**任意**指令处被中断，所以互斥不会自动出现，必须由 `synchronized`（或 Lock）显式划定，而这个区间恰好就是「不含释放点」的区间——从「不让出控制权」的角度看，两者是同构的。不能照搬的原因有三：(1) Java 的交错点不是显式标记的，`while (!ready) {}` 这种代码在里面毫无 `await` 式的标记，却可能被抢占，所以「有没有关键字」不是判据，「有没有对共享数据的访问」才是；(2) `synchronized` 方法/块只对**获取同一把锁**的线程互斥，锁选错等于没有互斥，而 TS 的关键字 `await` 是语言级的、不存在「用错锁」的问题；(3) 重排序与可见性问题（sp21 的 `answer` / `ready` 例子）在 Java 中额外存在，`synchronized` 同时承担了内存屏障的职责，而在单线程 TS 中没有这个问题。结论：TS 里你推理的是「哪里让出了控制权」，Java 里你推理的是「哪里获取了哪把锁、谁还在不遵守约定」。

问题 2：下面这个类有任何并发缺陷吗？请指出并给出修复方案。

```
public class Counter {
    private int count = 0;
    private final Object lock = new Object();

    public void increment() {
        synchronized (lock) {
            count++;
        }
    }

    public int getCount() {
        return count;           // 注意这里没有加锁
    }

    public boolean isZero() {
        synchronized (lock) {
            return getCount() == 0;
        }
    }
}
```

答案：有。`getCount()` **没有获取** **`lock`**，因此它对 `count` 的读取不受互斥保护：其一，`isZero()` 虽然持有锁，但它调用的 `getCount()` 却绕过了锁——所以 `isZero()` 的临界区实际上并没有保护到那次读，它可能读到别的线程正在 `count++` 过程中的值（`int` 的读在 JVM 上通常不会撕裂，但**可见性与重排序**不保证，仍可能读到陈旧值，而且这个类一旦把 `count` 改成 `long` 或多个字段组成的不变量，问题立刻变成实质性错误）；其二，只要有一条访问路径不遵守锁的约定，整个论证就失效——正如 sp21 所说，锁只是一种约定，一个不守约定的客户端就能让系统不再线程安全。修复：让**所有**访问 `count` 的路径都获取同一把锁，例如把 `getCount()` 也改成 `synchronized (lock) { return count; }`（或改用 `this` 作为锁对象、按监视器模式统一写 `public synchronized int getCount()`）。修复之后，`isZero()` 与 `getCount()` 都以 `lock` 为锁，由于锁可重入，嵌套调用不会死锁。更好的做法是从接口设计上避免这种复合：让 `isZero()` 直接比较 `count == 0`（虽然结果相同，但语义更清楚），并把线程安全论证（「`count` 的所有访问都由 `lock` 保护」）写进类注释。

问题 3：图书馆场景中，`checkout` 早期版本在等待书被归还时先记录下「哪些书在馆」，等所有书到齐后再统一借出，结果出现了竞态：Frodo 在等 Two Towers 期间，Gandalf 把 Fellowship 借走了，而 Frodo 醒来后却认为自己借到了两本书。请说明这个 bug 的性质、sp22 给出的修法，以及修法为什么又引入了死锁。

答案：性质是**竞态条件（safety 被破坏）**：`checkout` 的行为对某些交错正确、对另一些交错错误——它的正确性取决于「Frodo 等待期间没有人借走 Fellowship」这个时序偶然，而这不是它能保证的。修复分两步：（1）**尽早变更**——书一到手就立刻在本地标记为已借出（`Frodo sees that Fellowship is still in the library and immediately marks it as checked out to himself`），不要等到循环结束才统一变更；（2）**保守地反复检查**——把等待写成循环 `while (!books.every(isInLibrary)) { await Promise.all(...); }`，因为 `await` 之后条件可能已经被别人破坏；等到条件成立后，再在一段**不含** **`await`** 的区间里一次性完成所有借出操作（这就是互斥）。这样 `Gandalf` 在 `Frodo` 持有 `Fellowship` 时只能等待。但「一拿到就变更」又引入了新问题：Frodo 按 `[fellowship, twoTowers, returnOfKing]` 顺序借、Gandalf 按 `[returnOfKing, twoTowers, fellowship]` 顺序借时，Frodo 持有 Fellowship 等 Two Towers，Gandalf

持有 Return of the King 等 Two Towers，而 Two Towers 归还后又可能被其中一方先取走并继续等待对方的书——形成**循环等待的死锁（liveness 被破坏）**：Frodo 等 Return of the King（Gandalf 持有），Gandalf 等 Two Towers（Frodo 持有），谁都动不了。这正是 Java 中「细粒度锁 + 无顺序」的图书馆版本；它的预防思路与锁顺序一致：让所有客户端按**同一种顺序**请求多本书（例如按书的唯一标识排序后依次 `checkout`），并且不要让「持有已借到的书」与「等待尚未借到的书」同时发生——sp22 在练习中提示的思路是改 `checkout` 的语义（例如先一次性把需要的书全部预留，或者让 `checkout` 在无法一次满足时回滚已占用的书再重试），从而在依赖图中不产生环。此外，按 Reading 24 的视角，还可以干脆改成消息传递：把「借书」做成一条原子请求消息，由图书馆模块独自串行处理，让客户端根本无法构造出这种交错。

Reading 24: 消息传递（Message-Passing）

> 说明：本讲 sp22 原版使用 TypeScript，本笔记按用户要求提供 Java 代码示例；类型/API 与 sp21（6.031 Java 版）原文保持一致。sp22 用 Python 线程的 `queue.Queue` 与 TypeScript worker 的 message port 讲消息传递，sp21 的孪生阅读（Queues and Message-Passing）用 Java 的 `BlockingQueue` 讲同一件事；本笔记以 sp22 的概念框架（共享内存 vs 消息传递、阻塞操作、生产者-消费者、消息类型、毒丸、竞态与死锁）为主结构，Java 代码取自 sp21 的 `DrinksFridge` / `FridgeResult` / `ManyThirstyPeople` 例子。凡属 Java 生态补充内容（`BlockingQueue` 的两种实现细节、`sealed` 接口、`Thread.interrupt` 的完整语义等）均标注为「补充说明」。

概述

本讲给出并发编程的**第二条路线**：不去精心地同步共享可变数据，而是让并发模块之间**只传递消息**、不共享内存——并发单元（线程、进程、不同的机器）之间通过一条**通信通道**（队列、管道、网络连接）交换**不可变消息**，可变状态被限定在每个模块内部。实现手段是**阻塞队列（blocking queue）**：`put` 在队列满时阻塞、`take` 在队列为空时阻塞，从而把「等待」这件事交给库，让代码写起来像普通的顺序代码。核心设计模式是**生产者-消费者（producer-consumer）**：生产者把请求放进队列，消费者取出并处理，把结果放回另一个队列。它与三大目标的关系是：**Safe from bugs**——消息传递把交互变成显式的、只共享不可变对象的交互，从根上避开了共享可变数据带来的竞态，也让每个模块更容易维持自己的线程安全不变量；**Easy to understand**——模块之间只有一条消息协议，模块内部是顺序代码，不需要推理「谁在什么时候改了哪个字段」；**Ready for change**——阻塞操作很方便，但 **阻塞就意味着可能死锁**（尤其当队列有容量上界时），所以协议设计必须像 Reading 23 的锁顺序一样被认真对待。

核心概念与设计原则详解

两种并发模型：共享内存与消息传递（Shared Memory vs Message Passing） – **定义与目的**：**共享内存模型**中，并发模块通过读写共享的可变对象来交互（同一进程内的多个线程是最典型的例子）。**消息传递模型**中，并发模块通过在通信通道上发送**不可变消息**来交互；这条通道既可以连接同一台机器上的两个线程，也可以连接网络两端的计算机。目的：给出一种让「并发交互」变得**显式**的构造方式，从而提升安全性。 – **直观解释（"它是什么？"）**：共享内存像**几个人共用一块白板**：谁都能擦谁的字，改动是隐式发生的，出了事很难定位是谁写的。消息传递像**传纸条**：你写完递过去，纸条上的内容不会再变；想知道别人的进展只能等他回

条。sp22 原文指出，共享内存的隐式交互极易导致**无意的交互**——程序的某些部分根本不知道自己身处并发环境，也没有遵守并发安全策略。 – **关键规则与最佳实践**： – 消息传递只共享**不可变**对象（消息本身），而共享内存要求共享可变对象——而共享可变对象即便在非并发编程里也是 bug 来源。 – 在消息传递中，**变更被限定在每个模块内部**：模块的状态是它自己的私事，其他模块只能通过消息请求它改变。 – 语言层面的对应：TypeScript 的 worker 各有独立全局环境，通常只靠消息传递通信；Java 的线程天生共享内存，所以要用**队列**把消息传递「搭出来」；而 Java 的**进程**之间天然是消息传递（标准输入输出流）。 – 消息传递并非万能：它**不能**消除竞态（见下文「消息传递中的竞态条件」），也可能死锁（见下文「消息传递中的死锁」）。

阻塞操作与阻塞队列（Blocking Operations and BlockingQueue） – **定义与目的**：**阻塞（blocking）** 的一般含义是「线程不做别的事，一直等到某个事件发生」；一个**阻塞方法**是「调用它可能阻塞，直到某个事件发生才返回」的方法。**阻塞队列**就是提供了阻塞操作的队列：`put(e)` 阻塞直到能把元素放到队尾（**若队列没有容量上界，`put` 永不阻塞**）；`get()` / `take()` 阻塞直到能从队首取出并返回元素（即等到队列非空）。目的：把「等待条件成立」这件容易写错的事（回想 Reading 23 中图书馆的忙等待与轮询）交给经过验证的库实现。 – **直观解释（"它是什么？"）**：阻塞队列就是**餐厅的传菜窗口**：厨师（生产者）把菜放上窗口，窗口满了就得等着（`put` 阻塞）；服务员（消费者）从窗口取菜，窗口空了就得等着（`take` 阻塞）。谁都不需要盯着对方看，窗口自己会「顶住」多余的一方。 – **关键规则与最佳实践**： – **务必使用 `put` / `take`，而不是 `add` / `remove`**（sp21 原文明确警告）：`add` 在队列满时抛异常，`remove` 在队列空时抛异常——它们**不会阻塞**，会让「等待」变成「崩溃」。 – Java 的两种实现（**补充说明**）：`ArrayBlockingQueue` 是固定容量、数组表示，队列满时 `put` 会阻塞；`LinkedBlockingQueue` 是可增长的链表表示，若不指定最大容量则永不装满，`put` 永不阻塞。 – 与 `readFile` / `readFileSync` 的类比：后者会**阻塞**运行它的线程，直到整个文件读进内存；在单线程 JS 进程里，阻塞函数会让**什么都干不了**，而在多线程 Python/Java 里，一个线程阻塞时其他线程仍可运行。 – 阻塞方法几乎总要处理**中断**：Java 中 `put` / `take` 会抛受检异常 `InterruptedException`，必须 try-catch 或声明抛出，并妥善决定「被中断时该做什么」。

生产者–消费者模式（Producer–Consumer Pattern） – **定义与目的**：生产者线程与消费者线程共享一个**线程安全**的队列：生产者把数据或请求放入队列，消费者取出并处理。可能有一个或多个生产者、一个或多个消费者同时操作同一个队列。目的：把「产出」与「消费」在时间上解耦——生产得快时数据在队列里排队，消费得快时消费者阻塞等待，双方各自按自己的节奏推进。 – **直观解释（"它是什么？"）**：这就是**餐厅厨房与传菜窗口**的分工：厨师不必等顾客点单才开火，服务员也不必等菜做好才去招呼客人；窗口（队列）吸收了两边的速度差。 – **关键规则与最佳实践**： – 这个队列**一定是共享且可变的**，所以必须确保它本身是并发安全的——通常直接使用库提供的 `BlockingQueue`（这是「使用已有的线程安全类型」策略）。 – 队列里流动的数据类型必须仔细选择：**选不可变类型**，这样生产者与消费者之间不存在通过「改同一个别名对象」而互相干扰的可能。 – 同时要像设计线程安全 ADT 的操作那样，**设计消息本身**：消息的语义要能防止竞态、让客户端能做它需要的原子操作（例如「借出并返回余量」是一条消息，而不是「先查再取」两条）。 – 用队列的长度作为「背压（backpressure）」信号时，要意识到有界队列会把「生产太快」转换成**阻塞**，而阻塞在特定协议下会变成死锁。

消息类型与带标签的联合（Message Types and Discriminated Unions） – **定义与目的**：消息通道通常能承载数组、映射、集合、记录等类型，但**不能承载用户自定义类的实例**（TypeScript 的 message port 不会把方法代码传过去）。因此常用**记录类型**表示消息，例如 `FridgeResult`；当通道上需要传多种消息时，用**带标签的联合（discriminated union）**把它们统一起来：每个变体带一个字面量类型的标签字段（如 `name: 'deposit' | 'withdrawal' | 'balance'`），其余字段随变体不同。目的：让「消息的合法形状」成为类型系统的一部分，从而在编译期排除大量协议错误。 – **直观解释（"它是什么？"）**：带标签的联合就像**邮政系统的信封分类**：信封上必须写「申请书 / 回执 / 停止通知」，邮局（类型检查器）据此判断内容是否配套；没有标签就只能靠猜，猜错就是运行时崩溃。 – **关键规则与最佳实践**： – 消息应当**不可变**：TypeScript 用 `readonly` 字段与不可变约定，Java 用 `private final` 字段 + 无修改器方法（并注意对可变载荷做防御性复制）。 – Java 中表达「联合类型」的标准做法是**接口 + 多个实现类**（这是 sp21 练习中被评为「最大程度利用静态检查」的写法）；**补充说明**：Java 17+ 可以用 `sealed interface` 让编译器检查 `switch` 的穷尽性，效果最接近 TypeScript 的判别联合。 – 不要用「魔法值」或 `null` 兼职表示特殊消息（见代码对比场景 3）。 – 消息的表示不变量（RI）与前置条件也要写出来——sp21/sp22 都用练习要求给 `FridgeResult` 写出 `checkRep()` 里的断言。

用消息传递实现线程安全的 ADT（A Threadsafe ADT via Message Passing） – **定义与目的**：消息传递版的抽象数据类型（如 `DrinksFridge`）把「状态」与「操作」都放进同一个模块内部：它持有一个私有的可变状态（`drinksInFridge`）、一条输入队列（接收请求）和一条输出队列（发送回复），并在启动时创建一个内部线程循环地从输入队列取请求、处理、把结果放回输出队列。目的：让**所有对该状态的访问都发生在同一个线程里**——于是根本不存在「两个线程同时改同一个字段」的可能，模块内部退化成顺序代码。 – **直观解释（"它是什么？"）**：这就像**只有一个收银员的窗口**：不管外面排了多少人，收银员总是一条一条地处理。顾客之间不直接打交道，所有互动都通过窗口（队列）发生。 – **关键规则与最佳实践**： – 状态是**模块私有**的：客户端永远拿不到 `drinksInFridge` 的引用，只能通过消息请求变更。 – 把「顺序」变成**协议**：请求进入 `in` 队列的顺序，就是它们被处理的顺序；回复进入 `out` 队列的顺序，就是它们完成的顺序。 – 抽象函数（AF）要写明通道的作用：例如 `AF(drinksInFridge, in, out) = 一个装有多瓶饮料的冰箱，它从 in 接收请求、向 out 发送回复`。 – 客户端只与服务端**交换消息**，因此不需要（也不能）理解服务端的内部结构；这正是 Reading 25 客户端/服务器架构的雏形。

消息传递版的线程安全论证（Thread Safety Arguments with Message Passing） – **定义与目的**：用消息传递实现并发时，线程安全论证可以依赖以下四件事：**（1）现有的线程安全数据类型**——那个同步队列一定是共享且可变的，必须确认它并发安全；**（2）消息的不可变性**——可能被多个线程同时访问的数据必须不可变；**（3）数据对单个生产者/消费者线程的限定**——生产者或消费者使用的局部变量对其他线程不可见，各线程只通过队列里的消息通信；**（4）通过队列「传递」可变数据的限定**——如果非要发送可变数据，必须像「烫手山芋」一样在放入队列的瞬间**抛弃所有引用**，使得任一时刻只有一个线程能访问它，这个论证必须被仔细地表述与实现。 – **直观解释（"它是什么？"）**：这是**接力棒**规则：接力棒（可变数据）在任一时刻只可能在一个人手里；交棒的那一刻，你必须真的松手，不能「递出去还捏着另一头」。 – **关键规则与最佳实践**： – 优先让消息**不可变**（第 2 条），这样根本不需要第 4 条那种微妙的论证。 – 与同步相比，消息传递让每个模块**更容易维持自己的线程安全不变量**：不必推理多个线程访问同一份共享数据（数据被转移到模块内部了）。 –

论证同样要**写进代码**（类注释），并配上 `checkRep()`。 – 注意区分：消息传递让「模块的内部状态」安全，但**不保证**「跨多条消息的复合操作」安全（见下文竞态条件）。

消息传递中的竞态条件（Race Conditions in Message Passing） – **定义与目的**：消息传递**不能**消除竞态。危险特别出现在**客户端必须发送多条消息才能完成一件事**的时候——这些消息（以及客户端对回复的处理）可能与其他客户端的消息交错。目的：提醒我们在设计**消息协议**时就要把原子性需求考虑进去。 – **直观解释（"它是什么？"）**：银行的经典剧本：「先查询余额，够就取款」是两条消息。两个客户同时查询、都看到还有 1 元、都发出取款请求——账户就被透支了。问题不在两条消息各自有错，而在**中间那段时间里世界变了**。sp22/sp21 共同的结论是：应当把操作设计成 **`withdraw-if-sufficient-funds`（余额足够才取款）** 这样一条原子消息，而不是让客户端拼装 `withdraw`。 – **关键规则与最佳实践**： – 设计协议时问：客户端完成一件事需要几条消息？如果需要多条，中间是否可能被别的客户端插队？ – 优先提供**语义完整、原子**的请求（对应 `ConcurrentMap.putIfAbsent` 这种「为并发补操作」的思路）。 – 冰箱的「LOOK before you TAKE」实验（先发 0 瓶的请求看余量、再决定要不要取）是典型的反例：三个礼貌的人可能都看到「还剩 2 瓶，够我拿 1 瓶还不至于空」，最后冰箱只剩 1 瓶——而不变量要求的是「不会有人取走最后一瓶」。 – 若客户端必须发多条消息，就要在协议层面引入**会话（session）**、**请求 id** 或**预留（reservation）**机制。

消息传递中的死锁（Deadlock in Message Passing） – **定义与目的**：阻塞让编程更简单，但也让死锁成为可能。通用判据：把系统画成**等待图**——节点是模块，若模块 A 正在阻塞等待模块 B 做某件事，就有边 A → B；**若某个时刻图中存在环，系统就死锁了**。最简单的环是双节点的 A → B 与 B → A，更大的系统可能有更长的环。 – **直观解释（"它是什么？"）**：`DrinksFridge` 的例子极为干净：请求队列与回复队列都设了容量上界（`maxsize` / `ArrayBlockingQueue(QUEUE_SIZE)`），客户端**一口气发 N 条请求，之后才开始读回复**。当 N > QUEUE_SIZE 时，未读的回复把回复队列填满，冰箱阻塞在「把回复放进 `out`」上、于是不再从 `in` 取请求；客户端继续往 `in` 里塞请求，直到把请求队列也填满而阻塞在自己的 `put` 上。于是：冰箱等客户端腾出回复队列的空间，客户端等冰箱腾出请求队列的空间——**致命拥抱**（当 N > 2×QUEUE_SIZE 时发生；N = QUEUE_SIZE 时恰好不会）。 – **关键规则与最佳实践**： – **死锁在有锁时更常见，但在消息传递中同样会发生**，只要通道有容量上界并被填满。死锁中的消息传递系统表现为「**就是卡住了**」。 – 消除死锁的第一条思路是**设计一个不可能出现环的系统**：如果 A 在等 B，就不能出现 B 已经在等（或将开始等）A 的情况。 – 第二条思路是**超时**：阻塞太久（100 毫秒？10 秒？取决于系统）就停止阻塞并抛异常——但随之而来的问题是「抛出异常之后该怎么办」，这需要应用层有明确的恢复策略。 – 与 Reading 23 的死锁对照记忆：那里环出现在**锁**之间（持有 A 的锁等 B 的锁），这里环出现在**队列**之间（占满 out 等读 out，占满 in 等取 in）。本质完全一致。

停止：毒丸与中断（Stopping: Poison Pill and Interrupt） – **定义与目的**：服务循环通常是 `while (true)`，需要一种**协议内的停止方式**。**毒丸（poison pill）** 是一条特殊消息，它告诉消费者结束工作。目的：让关闭过程**干净**——不丢失未完成的请求，不破坏共享状态（文件系统、数据库、通信通道）。 – **直观解释（"它是什么？"）**：毒丸就是「打烊通知」：排在它前面的菜照做，看到它才收摊。相比之下，强行 `terminate()` worker 或 `os._exit(0)` 等于**掀桌子**——正在做的工作掉在地上，共享状态可能被留在损坏的中间态（sp22 原文称后者 *generally a bad idea*）。 – **关键规则与最佳实践**： – 不要用魔法数字当毒丸

（*don't use magic numbers*），也不要用 `null`（*don't use null*）：应该把输入消息的类型改成**带标签的联合/ADT**，例如 `FridgeRequest = DrinkRequest | StopRequest`，然后发送 `StopRequest`。 – 收到停止消息后要**摘掉监听器**（TypeScript 中需要保存回调引用以便 `removeListener`；sp22 特别指出：一旦该线程没有更多代码要跑、也没有监听器挂着，它就自然终止）。 – Java 的另一条路线（**补充说明**）是 `Thread.interrupt()`：若目标线程正阻塞，被阻塞的方法会抛 `InterruptedException`；若它没在阻塞，则设置**中断标志**。使用这条路线的线程必须**既处理 `InterruptedException` 又检查中断标志**（`while (!Thread.interrupted())`）。 – 停止协议也要写进规格：客户端需要知道「怎么优雅地让服务停下来」，否则只会退化成强杀。

消息传递的代价与适用场景（Costs and Applicable Scenarios） – **定义与目的**：消息传递用「多一次拷贝/一次调度」换取「更少的共享状态」。它的代价包括：需要显式的协议设计、需要处理阻塞与中断、有界队列会引入死锁风险、跨进程/跨网络时还有序列化与延迟成本。它的优势在于：模块边界清晰、状态局部化、天然适配客户端/服务器与分布式场景。 – **直观解释（"它是什么？"）**：共享内存像**几个人在同一张桌子上拼图**（快，但容易撞手）；消息传递像**各自在自己的桌上拼，需要交换时喊一声递过去**（慢一点，但秩序井然）。 – **关键规则与最佳实践**： – 当模块状态复杂、且「谁在什么时候改了什么」很难论证时，消息传递往往比锁更容易维持正确性。 – 当需要极致性能、数据量巨大且共享天然（如图像缓冲区、大规模数值计算）时，共享内存 + 精心设计的锁/不可变性更划算。 – 消息传递天然适合**客户端/服务器架构**：服务端串行地处理请求，客户端并发地发起请求——这正是 Reading 25（套接字与网络） 的主题。 – 无论如何选择，都要**写下线程安全论证**：消息传递版的论证通常由「线程安全的消息队列 + 不可变消息 + 状态限定在单个模块内」三条组成。

代码示例与对比分析

场景 1：从队列取消息——轮询普通队列，还是阻塞在 `take()` 上？

❌ *错误代码*

```
import java.util.Queue;
import java.util.concurrent.ConcurrentLinkedQueue;

/** 错误：用普通（非阻塞）队列做消息传递，用 poll() 轮询。 */
public class TradeWorker implements Runnable {
    private final Queue<Trade> tradesQueue;

    public TradeWorker(Queue<Trade> tradesQueue) {
        this.tradesQueue = tradesQueue;
    }

    @Override
    public void run() {
        while (true) {
            Trade trade = tradesQueue.poll();      // 队列空时返回 null，不阻塞
            TradeProcessor.handleTrade(trade.numShares(), trade.stockName());
        }
    }
}

/** 交易消息；注意它必须是不可变的（下面还会再谈）。 */
interface Trade {
    int numShares();
    String stockName();
}

class TradeProcessor {
    static void handleTrade(int numShares, String stockName) {
        /* ... 处理一笔交易，需要一些时间 ... */
    }
}
```

【错误代码的问题】 1. **空队列时抛出 `NullPointerException`**：`Queue.poll()` 在队列为空时返回 `null`（而不是阻塞），紧接着 `trade.numShares()` 就会崩溃。这正是 sp21 练习「Mistakes were made」考察的要点。 2. **忙轮询（busy polling）**：即使不崩溃，这个循环也会在全空的情况下疯狂空转，把 CPU 打满——与 Reading 22 中禁止忙等待、Reading 23 中图书馆忙等待是同一类错误。 3. **消息顺序不确定**：多个 `TradeWorker` 从同一个队列取任务，谁先取到不确定，因此交易的**处理顺序与入队顺序不一致**；如果业务要求「同一账户的交易按序处理」，这就直接违反了规格。 4. **没有停止机制、也不响应中断**：`while (true)` 加上 `poll()` 的组合既无法优雅停止，也无法感知中断。

✅ *正确代码*

```
import java.util.concurrent.BlockingQueue;

/** 正确：阻塞队列 + 阻塞式 take()，并处理中断。 */
public class TradeWorker implements Runnable {
    private final BlockingQueue<Trade> tradesQueue;

    public TradeWorker(BlockingQueue<Trade> tradesQueue) {
        this.tradesQueue = tradesQueue;
    }

    @Override
    public void run() {
        // 收到中断请求（或毒丸）就干净地退出循环
        while (!Thread.interrupted()) {
            try {
                // 阻塞直到有消息到达；空队列时线程被挂起，不消耗 CPU
                Trade trade = tradesQueue.take();
                TradeProcessor.handleTrade(trade.numShares(), trade.stockName());
            } catch (InterruptedException ie) {
                Thread.currentThread().interrupt();   // 恢复中断标志
                break;                                // 停止工作
            }
        }
    }
}

/** 交易消息的接口：只提供观察器，不提供修改器。 */
interface Trade {
    int numShares();
    String stockName();
}

/**
 * 不可变的交易消息：所有字段 final，没有修改器。
 * Thread safety argument: 不可变，可被任意多个线程同时安全访问。
 */
final class ImmutableTrade implements Trade {
    private final int numShares;
    private final String stockName;

    ImmutableTrade(int numShares, String stockName) {
        this.numShares = numShares;
        this.stockName = stockName;
    }

    @Override public int numShares() { return numShares; }
    @Override public String stockName() { return stockName; }
}

class TradeProcessor {
    static void handleTrade(int numShares, String stockName) {
        /* ... 处理一笔交易，需要一些时间 ... */
```

```
    }
}
```

【为什么这样更好】 `take()` 在队列为空时**阻塞**而不是返回 `null`，因此既不会 NPE 也不会空转；线程被操作系统挂起，CPU 交给别人用，队列一有元素就被唤醒——这正是「阻塞让代码更好写」的含义。`while (!Thread.interrupted())` 配合 `catch (InterruptedException)` 让工作者能被优雅地关闭（**补充说明**：这是 sp21 给出的标准写法）。消息类型改成不可变也顺手消除了「消费者正在读、生产者同时改」的可能。

【代码对比解说】 这里有一个容易混淆的点：`poll()` 与 `take()` 的差别不是「快慢」，而是**语义**——`poll` 回答「现在有没有？」，`take` 回答「给我下一个（没有就等）」。用 `poll` 就意味着客户端要自己实现「等待」，而在消息传递中**唯一正确的等待方式就是阻塞**（否则就会退化成轮询）。另外注意 sp21 的原始练习中 `Queue<Trade>` 还可以是 `ConcurrentLinkedQueue` 这种线程安全的**非阻塞**队列：它保证了「多线程并发访问不会破坏内部结构」，但**不保证**「客户端想要的复合语义」——这正好与 Reading 23 中 `Collections.synchronizedList` 的教训完全一致：库只能保证单个操作的原子性，语义层面的原子性要靠接口设计。

【设计原则透视】 这体现了「**用已有的线程安全类型 + 正确的阻塞语义**」这一策略：队列本身是共享可变的，所以必须并发安全；而「等待」这件事被封装进库方法，客户端的代码因此不必再写任何同步逻辑。`Trade` 的不可变性则对应消息传递的立身之本——**只共享不可变对象**。二者合起来构成消息传递版线程安全论证的前两条。

场景 2：消息是可变对象，还是不可变值?

❌ *错误代码*

```
/**
 * 错误：可变的"消息"。生产者放进去之后还能继续改它，
 * 消费者拿到的可能是一个正在被改写、或已被改写过的对象。
 */
public class FridgeResult {
    private int drinksTakenOrAdded;
    private int drinksLeftInFridge;

    public FridgeResult(int drinksTakenOrAdded, int drinksLeftInFridge) {
        this.drinksTakenOrAdded = drinksTakenOrAdded;
        this.drinksLeftInFridge = drinksLeftInFridge;
    }

    // 修改器：任何人都能在消息发出后改写它
    public void setDrinksTakenOrAdded(int n) { this.drinksTakenOrAdded = n; }
    public void setDrinksLeftInFridge(int n) { this.drinksLeftInFridge = n; }

    public int drinksTakenOrAdded() { return drinksTakenOrAdded; }
    public int drinksLeftInFridge() { return drinksLeftInFridge; }

    @Override public String toString() {
        return (drinksTakenOrAdded >= 0 ? "you took " : "you put in ")
                + Math.abs(drinksTakenOrAdded) + " drinks, fridge has "
                + drinksLeftInFridge + " left";
    }
}
```

【错误代码的问题】 1. **消息可以在「传递途中」被改写**：生产者 `put` 之后如果还持有引用并调用 `setDrinksLeftInFridge`，消费者读到的就是被污染的数据——sp22 原文称之为 *the opportunity for (mis)communication by mutating an aliased message object*。 2. **失去消息传递最核心的安全属性**：消息传递之所以安全，前提是「模块之间只共享不可变对象」；一旦消息可变，就退化成了共享可变数据——也就是 Reading 21/23 中所有竞态的源头。 3. **不可复现的读值错误**：同一个消息对象被两个消费者同时观察时，可能一个看到旧值一个看到新值（缺乏 happens-before 保证时甚至可能永远看不到更新）。 4. **表示不变量无从谈起**：sp21/sp22 都要求给 `FridgeResult` 写出 `checkRep()` 的断言（例如「取走的瓶数不超过请求的瓶数」「剩余瓶数非负」）；可变对象在任意时刻都可能处于违反 RI 的中间状态。

✅ *正确代码*

```
/**
 * 一条线程安全的不可变消息，描述向 DrinksFridge 取用或放入饮料的结果。
 *
 * Rep invariant:
 *   drinksLeftInFridge >= 0
 *   drinksTakenOrAdded <= 0  ||  drinksLeftInFridge + drinksTakenOrAdded >= 0
 * Thread safety argument:
 *   不可变：所有字段都是 private final，且方法不返回可变内部状态，
 *   因此可以被任意多个线程同时安全访问（消息传递只共享这种对象）。
 */
public final class FridgeResult {
    private final int drinksTakenOrAdded;
    private final int drinksLeftInFridge;

    /**
     * 构造一条结果消息。
     * @param drinksTakenOrAdded 实际取走（正）或放入（负）的瓶数
     * @param drinksLeftInFridge 冰箱中剩余的瓶数，必须 >= 0
     */
    public FridgeResult(int drinksTakenOrAdded, int drinksLeftInFridge) {
        this.drinksTakenOrAdded = drinksTakenOrAdded;
        this.drinksLeftInFridge = drinksLeftInFridge;
        checkRep();
    }

    private void checkRep() {
        assert drinksLeftInFridge >= 0;
    }

    /** @return 实际取走（正）或放入（负）的瓶数 */
    public int drinksTakenOrAdded() { return drinksTakenOrAdded; }

    /** @return 冰箱中剩余的瓶数 */
    public int drinksLeftInFridge() { return drinksLeftInFridge; }

    @Override public String toString() {
        return (drinksTakenOrAdded >= 0 ? "you took " : "you put in ")
                + Math.abs(drinksTakenOrAdded) + " drinks, fridge has "
                + drinksLeftInFridge + " left";
    }
}
```

【为什么这样更好】 所有字段 `private final`、没有修改器、不暴露任何可变内部状态，因此这条消息**天生线程安全**：任意多个线程同时读它都不会出问题，也不需要任何锁。`checkRep()` 把表示不变量写下来并在构造时断言，使得「消息一旦创建就永远合法」。这正是 sp21 对 `FridgeResult` 的定义：*A threadsafe immutable message*。

【代码对比解说】 从线程安全论证的角度看，这两种写法的差距是数量级的：不可变版本只需要一句「它是不可变的」就完成了论证；可变版本则必须论证「谁在什么时候持有引用」「put 之后生产者是否还持有引用」「消费者是否可能在读到一半时被改写」——也就是 Reading 23 里那套复杂得多的锁与交错推理。**补充说**

明：若消息里真的必须携带可变载荷（例如一个 `List`），必须做**防御性复制**（构造时拷贝入参、观察器返回拷贝），并且最好在注释里说明「这是一次拷贝，不是别名」。另一个常见做法是让队列本身完成传递语义上的「所有权转移」（烫手山芋原则）：放进去的瞬间就抛弃所有引用。

【设计原则透视】 这里把 **Reading 8（不可变性）** 与 **Reading 11（AF / RI / 表示暴露防护）** 直接搬到了并发场景：不可变对象可以被任意共享而无需同步，从根上消灭了竞态；而「所有字段 final + private + 无修改器 + checkRep」正是构造不可变类型的标准配方。它还解释了消息传递为什么能提升安全性：**共享的东西从「可变对象」变成了「不可变消息」。**

场景 3：如何告诉服务端「停下来」——魔法数字、`null`，还是带标签的联合?

❌ *错误代码*

```
import java.util.concurrent.BlockingQueue;

/**
 * 错误：用魔法数字（或 null）当作停止信号。
 * 本协议中 n >= 0 表示取走 n 瓶，n < 0 表示放入 -n 瓶，
 * 因此 -1 是一条完全合法的正常请求："放入 1 瓶"。
 */
public class DrinksFridge {

    /** 魔法毒丸值：与合法请求重叠，含义靠约定，无法被编译器检查。 */
    private static final int STOP = -1;

    private int drinksInFridge;
    private final BlockingQueue<Integer> in;
    private final BlockingQueue<FridgeResult> out;

    public void start() {
        new Thread(() -> {
            while (true) {
                try {
                    int n = in.take();
                    if (n == STOP) {
                        break;                      // 但"放入 1 瓶"的请求永远无法送达了
                    }
                    FridgeResult result = handleDrinkRequest(n);
                    out.put(result);
                } catch (InterruptedException ie) {
                    ie.printStackTrace();           // 错误：吞掉中断，继续循环
                }
            }
        }).start();
    }

    private FridgeResult handleDrinkRequest(int n) {
        int change = Math.min(n, drinksInFridge);
        drinksInFridge -= change;
        return new FridgeResult(change, drinksInFridge);
    }
}
```

【错误代码的问题】 1. **毒丸与合法消息冲突**：`-1` 在本协议中本表示「放入 1 瓶饮料」，现在却被征用为停止信号——某位用户永远无法补货，而且这个 bug 完全不会被编译器或类型检查发现。 2. **`null` 行不通**：Java 的 `BlockingQueue` 不允许插入 `null`（会抛 `NullPointerException`），而且 sp22 原文明确说 *don't use null*。 3. **含义只存在于注释里**：客户端必须「知道」-1 是停止信号才能使用协议；接口没有表达力，任何新客户端都可能误用。 4. **中断被静默吞掉**：`catch (InterruptedException ie) { ie.printStackTrace(); }` 之后继续循环，既不恢复中断标志也不退出——线程无法被关闭（这与 Reading 22 中静默吞异常的坏味道同源）。

✅ *正确代码*

```
import java.util.concurrent.BlockingQueue;

/** 请求消息的联合类型：一条请求要么点饮料，要么要求停止。 */
public interface FridgeRequest { }

/** 取用（或放入）饮料的请求；不可变。 */
final class DrinkRequest implements FridgeRequest {
    private final int drinksRequested;

    DrinkRequest(int drinksRequested) { this.drinksRequested = drinksRequested; }

    /** @return 若 >= 0 取走至多 n 瓶；若 < 0 放入 -n 瓶 */
    int drinksRequested() { return drinksRequested; }
}

/** 停止请求：毒丸，用类型而非魔法值表达。 */
final class StopRequest implements FridgeRequest { }

/**
 * 正确：用类型区分消息语义，停止信号不再与合法请求冲突。
 *
 * Thread safety argument:
 *   1) in/out 是线程安全的 BlockingQueue；
 *   2) FridgeRequest 与 FridgeResult 都是不可变的；
 *   3) drinksInFridge 只被服务线程访问（限定在单个线程内）。
 */
public class DrinksFridge {

    private int drinksInFridge;
    private final BlockingQueue<FridgeRequest> in;
    private final BlockingQueue<FridgeResult> out;

    public DrinksFridge(BlockingQueue<FridgeRequest> requests,
                        BlockingQueue<FridgeResult> replies) {
        this.drinksInFridge = 0;
        this.in = requests;
        this.out = replies;
        checkRep();
    }

    private void checkRep() {
        assert drinksInFridge >= 0;
    }

    public void start() {
        new Thread(() -> {
            while (true) {
                try {
                    FridgeRequest req = in.take();
                    if (req instanceof StopRequest) {
                        break;                          // 优雅停止：处理完之前的请求才退出
                    }
                    int n = ((DrinkRequest) req).drinksRequested();
```

```
                    FridgeResult result = handleDrinkRequest(n);
                    out.put(result);
                } catch (InterruptedException ie) {
                    Thread.currentThread().interrupt();  // 恢复中断标志
                    break;                               // 停止工作
                }
            }
        }).start();
    }

    private FridgeResult handleDrinkRequest(int n) {
        int change = Math.min(n, drinksInFridge);
        drinksInFridge -= change;
        checkRep();
        return new FridgeResult(change, drinksInFridge);
    }
}
```

【为什么这样更好】 停止语义由**类型**表达：`StopRequest` 是一个独立类型，不可能与任何合法的 `DrinkRequest` 混淆——sp22 原文的判据正是「魔法数字坏、`null` 坏，应该改成联合类型」。中断处理也修正了：恢复中断标志并退出循环，线程可被关闭。同时类注释里写下了消息传递版的三条线程安全论证。

【代码对比解说】 sp21 的练习专门比较了三种实现 `FridgeRequest = DrinkRequest(n) + StopRequest` 的 Java 写法：(1) **接口 + 两个实现类** —— 正确，最大程度利用静态检查；(2) 两个互不相关的类 —— 错误，编译器无法阻止你把任意对象放进队列；(3) 用一个类加 `String requestType` 标签字段 —— 错误，标签是运行时的字符串常量，编译器无法检查穷尽性与字段匹配。**补充说明**：Java 17+ 的 `sealed interface FridgeRequest permits DrinkRequest, StopRequest` 配合 `switch` 模式匹配，可以让编译器检查「所有变体都被处理」，效果最接近 sp22 的判别联合。`instanceof` 的写法可以与 `sealed` 结合以获得穷尽性检查。

【设计原则透视】 这直接对应 **Reading 12（接口、泛型与枚举）** 的核心思想：**用类型表达约束，让编译器替你检查**。它把「协议」从注释里的约定提升为类型系统的一部分，属于「让错误写法写不出来」的安全策略；在 Reading 6/7 的语言里，这相当于把前置条件从文档搬进了签名。停止协议同样是服务端**规格说明**的一部分：客户端需要知道「如何优雅关闭」才可能正确使用这个模块。

场景 4：有界队列上的请求–回复——先发完再收，还是交错收发？

❌ *错误代码*

```
import java.util.concurrent.*;

/** 错误：客户端一口气发出 N 条请求，之后才开始读回复。 */
public class ManyThirstyPeople {
    private static final int QUEUE_SIZE = 100;
    private static final int N = 250;               // N > 2 * QUEUE_SIZE

    public static void main(String[] args) throws InterruptedException {
        BlockingQueue<FridgeRequest> requests = new ArrayBlockingQueue<>(QUEUE_SIZE);
        BlockingQueue<FridgeResult> replies = new ArrayBlockingQueue<>(QUEUE_SIZE);

        DrinksFridge fridge = new DrinksFridge(requests, replies);
        fridge.start();

        // 先给冰箱补足饮料
        requests.put(new DrinkRequest(-N));
        System.out.println(replies.take());

        // 发送 N 条请求—根本没有在读回复
        for (int x = 1; x <= N; ++x) {
            requests.put(new DrinkRequest(1));       // 第 201 次 put 会永久阻塞
            System.out.println("person #" + x + " is looking for a drink");
        }

        // 收集回复（永远到不了这里）
        for (int x = 1; x <= N; ++x) {
            System.out.println("person #" + x + ": " + replies.take());
        }

        System.out.println("done");
    }
}
```

【错误代码的问题】 1. **死锁**：`QUEUE_SIZE = 100`、`N = 250` 时，回复队列先被 100 条未读回复填满，冰箱阻塞在 `out.put`；客户端接着把请求队列也填满，阻塞在自己的 `requests.put`——冰箱等客户端读回复，客户端等冰箱取请求，形成环。sp21 原文的判据是：**当 N > 2×QUEUE_SIZE 时客户端也会阻塞**，此刻就是致命拥抱。 2. **表面上「能用」的错觉**：把 `N` 从 250 改成 100 时程序完全正常（`QUEUE_SIZE = 100, N = 100` 恰好不触发），于是这个 bug 会在负载上升或容量调整后突然出现。 3. **没有任何超时或失败出口**：程序既不打印错误也不退出，只能强杀——这正是 sp22 描述的「消息传递系统死锁时看起来就是卡住了」。 4. **无上界地占用内存或阻塞**：换用无上界队列虽然能绕过死锁，但会把内存风险换成内存风险（队列无限制增长）。

✅ *正确代码*

```
import java.util.concurrent.*;

/**
 * 正确：交错地发送请求与接收回复，保证两个队列永远不会同时被填满，
 * 因此等待图中不可能出现环。
 */
public class PipelineThirstyPeople {
    private static final int QUEUE_SIZE = 100;
    private static final int N = 250;

    public static void main(String[] args) throws InterruptedException {
        BlockingQueue<FridgeRequest> requests = new ArrayBlockingQueue<>(QUEUE_SIZE);
        BlockingQueue<FridgeResult> replies = new ArrayBlockingQueue<>(QUEUE_SIZE);

        DrinksFridge fridge = new DrinksFridge(requests, replies);
        fridge.start();

        requests.put(new DrinkRequest(-N));
        System.out.println(replies.take());

        // 一条请求一条回复地流水线推进：任一时刻在途消息至多 1 条
        for (int x = 1; x <= N; ++x) {
            requests.put(new DrinkRequest(1));
            System.out.println("person #" + x + " is looking for a drink");
            System.out.println("person #" + x + ": " + replies.take());
        }

        // 干净地关闭服务：发送毒丸，等它处理完前面的请求后自行退出
        requests.put(new StopRequest());
        System.out.println("done");
    }
}
```

【为什么这样更好】 交错收发把「未读回复的堆积量」限制在一个很小的常数（这里是 1），因此两个队列都不可能被填满，死锁的必要条件（循环等待）被结构性消除。这也说明：**死锁的根源往往不在代码的错误，而在协议的设计**——只要客户端与服务端之间的「在途消息数」有上界且小于两侧容量，就不可能出现环。关闭时用 `StopRequest` 毒丸而不是强杀，保证已入队的请求都被处理完。

【代码对比解说】 三种应对手法的对比：

手法	效果	代价
交错收发（流水线化）	结构性消除环	吞吐量受限于往返延迟
使用无界队列（`LinkedBlockingQueue`）	`put` 永不阻塞，故不会有这类死锁	内存无上界，生产过快会 OOM；掩盖背压问题
超时（`offer(e, timeout, unit)` / `poll(timeout, unit)`）	阻塞太久就抛异常，避免永久卡死	必须回答「超时之后怎么办」：重试？丢弃？回滚？

sp22 原文给出的两条最终建议正是「**设计无环系统**」与「**使用超时**」，并指出后者的真正难点在于异常之后的恢复策略。**补充说明**：`BlockingQueue` 提供了 `offer(e, timeout, unit)` 与 `poll(timeout, unit)` 这两个带超时的版本，比 `put` / `take` 更适合需要「不许永久阻塞」的系统。

【设计原则透视】 这是**存活性（liveness）**分析的直接应用：把系统画成等待图，节点是模块（客户端、冰箱），边是「在等对方做什么」；只要图里没有环，就不会死锁。它与 Reading 23 中「细粒度锁 + 无顺序 → 死锁」的问题结构完全同构，只不过环从锁转移到了队列容量上。同时它揭示了**协议设计也是抽象边界的一部分**：客户端必须遵守「不要一次塞满」这一隐含约定，而把这种约定写进规格（或干脆用接口设计让它不可能违反，例如让请求与回复成对返回）才是更稳妥的做法。

场景 5：多个客户端共用一个回复队列——回复会不会拿错?

❌ *错误代码*

```
import java.util.concurrent.*;

/**
 * 错误：所有客户端共用同一个 replies 队列，
 * 谁先 take 就拿到别人的回复。
 */
public class SharedReplyQueue {
    public static void main(String[] args) throws InterruptedException {
        BlockingQueue<FridgeRequest> requests = new LinkedBlockingQueue<>();
        BlockingQueue<FridgeResult> replies = new LinkedBlockingQueue<>();

        DrinksFridge fridge = new DrinksFridge(requests, replies);
        fridge.start();

        requests.put(new DrinkRequest(1));        // Alice 的请求
        requests.put(new DrinkRequest(2));        // Bob 的请求

        // Alice 与 Bob 谁先调用 take 谁先拿到，拿到的可能是对方的回复
        System.out.println("Alice sees: " + replies.take());
        System.out.println("Bob   sees: " + replies.take());
    }
}
```

【错误代码的问题】 1. **回复错配**：两个客户端把自己的请求放进同一个 `in` 队列，却从同一个 `out` 队列读回复；先来先取的顺序由调度决定，客户端无法判断拿到的是不是自己的回复。 2. **协议缺少「请求—回复」的对应关系**：消息里既没有请求标识，也没有专用回复通道，因此服务端也无从告知「这条回复是给谁的」。 3. **看起来正确、偶尔出错**：单客户端测试永远通过，只有在并发多客户端时才暴露，属于典型的 heisenbug 式设计缺陷。 4. **强制了「单客户端」的隐性限制**：客户端必须知道「这个服务同时只能有一个使用者」才能用对——这种限制如果不写进规格，就是隐藏的陷阱。

✅ *正确代码*

```
import java.util.concurrent.*;

/**
 * 正确：把「回复通道」放进请求消息里，
 * 每个客户端只读自己的私有队列，回复不再可能错配。
 */
final class DrinkRequestWithReply implements FridgeRequest {
    private final int drinksRequested;
    private final BlockingQueue<FridgeResult> replyTo;   // 客户端私有的回复通道

    DrinkRequestWithReply(int drinksRequested, BlockingQueue<FridgeResult> replyTo) {
        this.drinksRequested = drinksRequested;
        this.replyTo = replyTo;
    }

    int drinksRequested() { return drinksRequested; }
    BlockingQueue<FridgeResult> replyTo() { return replyTo; }
}

final class StopRequestWithReply implements FridgeRequest { }

class FridgeService {
    private int drinksInFridge;

    void start(BlockingQueue<FridgeRequest> in) {
        new Thread(() -> {
            while (true) {
                try {
                    FridgeRequest req = in.take();
                    if (req instanceof StopRequestWithReply) break;
                    DrinkRequestWithReply drink = (DrinkRequestWithReply) req;
                    int change = Math.min(drink.drinksRequested(), drinksInFridge);
                    drinksInFridge -= change;
                    // 回复送到该客户端自己的队列，绝不可能被别的客户端取走
                    drink.replyTo().put(new FridgeResult(change, drinksInFridge));
                } catch (InterruptedException ie) {
                    Thread.currentThread().interrupt();
                    break;
                }
            }
        }).start();
    }
}

/** 每个客户端持有自己的请求队列（或使用共享请求队列 + 私有回复队列）。 */
class Client {
    private final BlockingQueue<FridgeRequest> requests;
    private final BlockingQueue<FridgeResult> myReplies = new LinkedBlockingQueue<>();

    Client(BlockingQueue<FridgeRequest> requests) { this.requests = requests; }

    FridgeResult orderDrinks(int n) throws InterruptedException {
        requests.put(new DrinkRequestWithReply(n, myReplies));
```

```
            return myReplies.take();      // 只会拿到自己的回复
        }
    }
```

【为什么这样更好】 「谁该收到回复」这一信息被编码进消息本身（`replyTo` 字段），而不是依赖共享队列的取用顺序，于是回复错配在结构上不可能发生。这既保留了「服务端串行处理请求」的简单性，又允许多个客户端并发使用同一个服务。用**消息字段而不是全局共享状态**来承载会话信息，是消息传递设计的通用技巧。

【代码对比解说】 共享的 `replies` 队列其实是把「会话标识」这个本应属于消息的信息，偷偷塞进了「队列的取用顺序」里——而顺序在并发下是不确定的。修法有三类：(1) **每条请求自带回复通道**（如上，把 `replyTo` 放进消息）；(2) **请求带唯一 id，回复也带同一个 id**，客户端按 id 匹配（对网络场景最常见，因为连接本身可以充当通道）；(3) **为每个客户端建立独立会话**，服务端为每个会话开一条专线。三者共同的思想是：**把「对应关系」变成消息协议中显式的一部分**。这与 Reading 23 中「让数据类型为并发提供原子操作」是同一种设计自觉——不是让客户端去猜，而是让协议无法被误用。

【设计原则透视】 这一场景是**协议设计 = 规格设计**的典范：一个没有明确「请求与回复如何配对」的协议，其规格是不完整的。它也再次印证消息传递的基本纪律——**模块之间只通过消息交互**：一旦客户端之间通过「共享一个队列并依赖取用顺序」暗中耦合，就又回到了共享可变状态的老路（这个队列成了隐式的共享状态，只不过它存的是「谁该拿哪条回复」这一信息）。

与其他设计原则的关联

本讲与 **Reading 21（并发）** 直接衔接：那一讲提出了共享内存与消息传递两大模型，并演示了银行账户的竞态与消息传递版的竞态（`get-balance` 后 `withdraw` 的经典交错），本讲把消息传递这一模型完整展开。**Reading 22（承诺）** 提供了「异步 vs 阻塞」的对照：`readFile` 是异步的、不占线程，而 `readFileSync` 与本讲的 `take()` / `put()` 都是**阻塞**的——在单线程 JS 进程里阻塞函数会让整个程序停摆，在多线程环境里只冻结当前线程，这个差别决定了「阻塞」是帮助还是灾难。**Reading 23（互斥）** 是本讲的直接前驱与对照面：那里用锁同步共享可变数据，这里用通道同步消息；那里的死锁来自「持有 A 的锁等 B 的锁」，这里的死锁来自「`out` 满等读、`in` 满等取」，本质都是等待图中的环。

本讲的后继是 **Reading 25（套接字与网络）**：那里把消息传递搬到网络之上，形成客户端/服务器架构——服务端串行处理连接，客户端并发发起请求；有缓冲的网络通信通道与阻塞队列的工作方式完全相同，本讲的「消息协议」「请求–回复配对」「毒丸关闭」「在途消息数上界」都会以网络版本重新出现。

向上游追溯：**Reading 8（不可变性）** 是消息类型的理论基础（不可变消息天生线程安全，是消息传递安全性的支柱之一）；**Reading 10、Reading 11（ADT、AF 与 RI）** 提供了 `DrinksFridge` 的 AF/RI 写法与 `checkRep()` 的纪律，也解释了「把状态限定在模块内部」为什么让线程安全论证变简单；**Reading 12（接口、泛型与枚举）** 解释了如何用接口 + 实现类（乃至 `sealed`）表达带标签的联合消息；**Reading 6、Reading 7（规格说明）** 要求把阻塞语义、停止协议、请求–回复配对规则都写进规格；**Reading 13（调试）** 则提醒我们，消息传递的死锁与回复错配同样是难以复现的 heisenbug，只能靠论证与评审而非测试来排除。

关键要点

- **消息传递 = 只共享不可变消息 + 共享一条线程安全的通信通道**：状态被限定在各自模块内部，交互从「隐式地改共享数据」变成「显式地发消息」；这从根本上避开了共享可变数据带来的竞态。

- **用 `put` / `take`（阻塞语义），不要用 `add` / `remove`（抛异常语义）**：阻塞把「等待条件成立」交给库，是消息传递代码比手写同步简单得多的原因；但**阻塞就意味着可能死锁**。
- **消息类型要用类型系统表达**：不可变、`private final`、无修改器；多种消息用带标签的联合（Java 用接口 + 实现类，17+ 可用 `sealed`）；**绝不用魔法数字或 `null`** 表示特殊消息（包括毒丸）。
- **协议必须为并发而设计**：客户端若需要多条消息才能完成一件事，就会有竞态（应采取「余额足够才取款」式的原子请求）；请求与回复的配对关系必须显式地放进消息（回复通道或请求 id），不能依赖共享队列的取用顺序。
- **死锁的判据是等待图中有环**：预防手段是设计无环协议（例如让在途消息数有上界）或使用超时；关闭服务用毒丸或中断，而不是强杀。

常见陷阱与注意事项

- **用 `poll()` / `remove()` 处理消息队列 → `NullPointerException` 或异常崩溃**：`poll` 在空队列时返回 `null`、`remove` 抛 `NoSuchElementException`，都不阻塞。后果是消费者在处理空队列时崩溃或忙轮询烧 CPU。应改用 `take()`（必要时用 `poll(timeout, unit)` 带上限）。
- **消息可变（有 setter、暴露内部集合）→ 传递途中被改写，回到共享可变数据的老路**：生产者 `put` 之后仍持有引用并修改，消费者读到被污染的数据。应让消息 `private final` + 无修改器，对可变载荷做防御性复制，或遵循「放入队列即抛弃引用」的共识。
- **用魔法值或 `null` 当毒丸 → 与合法消息冲突，且编译器帮不上忙**：本讲的 `-1` 既是「放入 1 瓶」也是「停止」，导致某类正常请求永远无法送达；`BlockingQueue` 还不允许 `null`。应改成带有 `StopRequest` 变体的联合类型。
- **在有界队列上「先发完再收」→ 队列填满形成循环等待而死锁**：当 N > 2×QUEUE_SIZE 时客户端与冰箱互相等待；而且小规模测试（N ⩽ QUEUE_SIZE）完全正常，问题只在负载变化后爆发。应交错收发、使用无界队列（接受内存风险），或使用带超时的 `offer` / `poll` 并明确超时后的策略。
- **多个客户端共用一条回复队列 → 回复错配**：谁先 `take` 谁拿到的可能是别人的回复，且单客户端测试永远通过。应把回复通道或请求 id 放进消息，让配对关系显式化。
- **强行终止服务线程（`Thread.stop`、`Terminate`、`os._exit(0)`）→ 未完成的工作被丢弃，共享状态可能被留在损坏的中间态**：文件系统、数据库或通信通道可能被破坏。应使用毒丸消息或 `interrupt()` 优雅关闭，并在规格中写明关闭协议。

思考题（带答案）

问题 1：sp21 的 `DrinksFridge` 有 2 瓶饮料，两位顾客各请求 3 瓶。请问哪些结果是可能的？如果三位顾客改为执行「先看再拿（LOOK before you TAKE）」的算法（先请求 0 瓶查看余量，若显示还剩 1 瓶以上再请求 1 瓶），结果又会怎样？请说明这两个实验分别揭示了消息传递的什么性质。

答案：第一个实验中，冰箱的服务循环是**串行**处理请求的：`handleDrinkRequest` 用 `change = Math.min(n, drinksInFridge)` 把请求量砍到实际可用量，因此两位顾客各请求 3 瓶、冰箱只有 2 瓶时，可能的结局只有「一位顾客拿到 2 瓶、另一位拿到 0 瓶，冰箱剩 0 瓶」这一类——**不会**出现两人各拿 3 瓶（不可能超出库存），也**不会**出现「两人都拿 0 瓶且冰箱仍是 2 瓶」（第一位的请求必然把所有 2 瓶取走）。这个实验说明：**服务端对单条消息的处理是原子的**（这正是 Reading 23 中「把变更放进一个互斥区间」在消

息传递里的对应物），所以消息传递确实消灭了「同一个状态被两个线程同时修改」这类竞态；同时它也说明**不变量由服务端守护**（剩余瓶数不会为负）。第二个实验则暴露了消息传递**不能**消除的竞态：三位礼貌的顾客各自发送「查看余量」的消息，服务端按顺序回复「2」，三人都据此认为「拿走 1 瓶不会拿空冰箱」，于是都发出「取 1 瓶」的请求——最终有人拿到 1 瓶、有人拿到 0 瓶（甚至出现「只剩 1 瓶」这种顾客本意要避免的结局）。原因不在服务端，而在**协议**：完成「礼貌地拿一瓶」这件事需要**两条消息**，而这两条之间的空隙里世界会变。这正是 sp22/sp21 的共同结论：应当把操作设计成一条原子消息（类似 `withdraw-if-sufficient-funds`），或者引入预留/会话机制，而不是让客户端用多条消息拼装一个逻辑上不可分割的操作。

问题 2：下面的代码在 `N = 100`、`QUEUE_SIZE = 100` 时运行正常，把 `N` 改成 250 后却永久卡住。请画出等待图解释死锁成因，并给出至少两种修复方案及其代价。

```
BlockingQueue<DrinkRequest> requests = new ArrayBlockingQueue<>(QUEUE_SIZE);
BlockingQueue<FridgeResult> replies = new ArrayBlockingQueue<>(QUEUE_SIZE);
DrinksFridge fridge = new DrinksFridge(requests, replies);
fridge.start();
requests.put(new DrinkRequest(-N));
System.out.println(replies.take());
for (int x = 1; x <= N; ++x) { requests.put(new DrinkRequest(1)); }
for (int x = 1; x <= N; ++x) { System.out.println(replies.take()); }
```

答案：客户端先把 N 条请求全部 `put` 完，之后才开始 `take` 回复。当 N > QUEUE_SIZE 时，未读的回复先把 `replies` 填满（100 条），此时冰箱线程阻塞在 `out.put(...)` 上、因而**停止从 `requests` 取请求**；客户端则继续 `requests.put(...)`，直到请求队列也填满（再 100 条），于是客户端阻塞在自己的 `put` 上。等待图为：**客户端 → 冰箱**（客户端等冰箱腾出 `requests` 的空间，即取走请求）与 **冰箱 → 客户端**（冰箱等客户端读走 `replies` 里的回复以腾出空间），两条边构成环，故死锁。这也解释了阈值：只有当 N > 2×QUEUE_SIZE 时客户端的 `put` 才会阻塞（sp21 原文的结论）。修复方案与代价：(1) **交错收发**（每发一条请求就取一条回复，或维持一个小的在途窗口）——结构上消除环，代价是吞吐量受往返延迟制约；(2) **改用无上界队列** `LinkedBlockingQueue`——`put` 永不阻塞，这类死锁消失，代价是内存无上界，生产远快于消费时会 OOM，并把背压问题掩盖掉；(3) **使用带超时的 `offer` / `poll`**——阻塞过久就抛异常，避免永久卡死，代价是必须设计「超时之后怎么办」（重试、丢弃、回滚、降级）；(4) **重新设计协议**，使请求与回复天然配对（例如每条请求自带回复队列，或服务端保证「收到请求立即回复」），从而保证在途消息数有上界且远小于容量。选择哪一种，取决于你能接受的是延迟、内存还是恢复逻辑的复杂度；但无论哪种，**把「在途消息数有上界」这一约束写进协议与规格**才是根本。

问题 3：为什么消息传递被认为比共享内存更安全，但它仍然会产生竞态和死锁？请分别给出一个具体例子，并说明在设计消息协议时应当如何应对。

答案：更安全的原因在于**交互是显式的、共享的对象是不可变的**：共享内存中，并发模块通过隐式地读写同一块内存交互，程序里那些「不知道自己身处并发环境」的部分极易无意间破坏数据；而消息传递中，模块只能通过通道发送不可变消息来交互，谁在与谁交互、交互了什么，全部写在消息里，模块的私有状态不会被外人碰到。sp22 原文的两条理由正是：隐式交互容易导致无意的交互；且消息传递只共享不可变对象，而共享内存要求共享可变对象——后者即便在非并发编程中也是 bug 来源。但它仍然会竞态，因为**当完成一件事需要多条消息时，这些消息会与其他客户端的消息交错**：例如「先查余额、再取款」两条消息之间，别的客户端可

能已经取走了钱，导致透支；应对办法是把这类操作设计成**一条原子的请求消息**（如「余额足够才取款」），或在协议中加入预留/事务/请求 id 等机制。它仍然会死锁，因为**阻塞是有代价的**：当通道有容量上界并被填满时，会出现「冰箱等客户端读回复、客户端等冰箱取请求」的循环等待（本讲 `QUEUE_SIZE` / `N` 的例子），应对办法是设计**无环协议**（保证在途消息数有上界、或让请求与回复成对流动）、使用**超时**并明确超时后的恢复策略，以及用**毒丸/中断**做优雅关闭。总结成一句话：消息传递把「共享可变数据」这个 bug 源头换成了「协议设计」这个更容易被审查的对象，但它并不豁免你对交错与阻塞的推理——**并发的基本困难依然存在，只是被搬到了一个更显眼的地方**。

Reading 25: 网络编程（Networking）

> 说明：本讲 sp22 原版使用 TypeScript（用 Express 写 Web 服务器），本笔记按用户要求提供 Java 代码示例；类型/API 与 sp21（6.031 Java 版，Reading 24: Sockets & Networking）原文保持一致。凡属 Java 生态的额外补充（如 `HttpServer`、线程池、`Socket.setSoTimeout`），均明确标注为「补充说明」。

概述

本讲讨论**通过网络进行的客户端/服务器通信（client/server communication over the network）**：客户端主动连接服务器、发送请求、接收应答、断开连接；服务器可以同时服务许多客户端。核心设计原则有两条：其一，网络通信本质上就是并发的，因此客户端与服务器都必须**推理其并发行为**并做到线程安全；其二，客户端与服务器之间交换的字节序列必须被设计，正如我们为 ADT 设计操作一样——这个设计产物叫做**线路协议（wire protocol）**或 **Web API**。它与三大目标的关系是：Safe from bugs（协议与并发设计正确、异常与超时被处理、编码与 flush 不出错）、Easy to understand（文本协议可以人工阅读调试，套接字代码与业务代码分离）、Ready for change（协议带版本号、用接口隔离 MIDI/网络实现、把流抽象成可替换的参数）。

核心概念与设计原则详解

客户端/服务器设计模式（Client/Server Design Pattern） – **定义与目的**：一种用**消息传递（message passing）**进行通信的设计模式。其中有两类进程：**客户端（client）**主动发起通信，连接服务器、发送请求（request）、接收应答（reply）、最后断开；**服务器（server）**等待连接并应答。它解决的是"两个独立进程如何在不共享内存的前提下协作"的问题，直接关系到 Safe from bugs（跨进程边界必须有清晰的协议与错误处理）与 Ready for change（服务器可以换实现而不影响客户端）。 – **直观解释（"它是什么？"）**：像餐厅点菜。顾客（客户端）主动走进餐厅、看菜单点菜（请求）、等菜上桌（应答）、吃完走人（断开）。厨房（服务器）不主动找顾客，但可以同时服务很多桌。浏览器是 Web 服务器的客户端，Outlook 是邮件服务器的客户端。 – **关键规则与最佳实践**： – **发起方永远是客户端**：连接由客户端建立，服务器只负责 `accept`；这决定了谁负责处理"连不上"的错误。 – 客户端与服务器**不必在不同机器上**：服务器完全可以与客户端在同一台机器上（用 `localhost` 连接），这对测试极其重要。 – **一个服务器同时服务多个客户端，一个客户端也可以连多个服务器**，因此两端都需要处理并发。 – 客户端与服务器之间**不共享内存**，只能交换字节序列，所以必须约定消息格式（协议），而不能像同进程线程那样共享对象。

IP 地址、主机名与 DNS（IP Addresses, Hostnames, and DNS） – **定义与目的**：**网络接口（network interface）**由 **IP 地址**标识；IPv4 地址是 32 位数，写成四个 8 位部分，如 `18.9.22.69`。**主机名（hostname）**是可以被翻译成 IP 地址的名字，如 `web.mit.edu`；翻译工作由 **DNS（Domain Name System）**完成。它关系到 Easy to understand（人记名字，机器用数字）与 Ready for change（同一个主机名可以在不同时间映射到不同 IP，客户端代码无需修改）。 – **直观解释（"它是什么？"）**：IP 地址像门牌号，主机名像"某某大厦"这种好记的名字，DNS 就是电话簿/导航 App：你报名字，它给你门牌号。 – **关键规则与最佳实践**： – 用 `dig +short web.mit.edu`、`host`、`nslookup` 亲自验证名字到地址的翻译。 – `127.0.0.1` 是 **loopback / localhost** 地址，永远指本机；严格说首字节为 127 的地址都是 loopback，但 `127.0.0.1` 是标准写法。 – 同一个主机名可能映射到不同 IP，多个主机名也可能映射到同一个 IP，因此**不要在程序里硬编码 IP 地址**，要写主机名。 – 笔记本换网络环境（换 WiFi）时 IP 地址会变，这说明 IP 地址是"位置"而不是"身份"。

端口号（Port Numbers） – **定义与目的**：一台机器上可能同时跑多个服务器程序，因此需要把同一网络接口上的流量分派给不同进程。网络接口有多个**端口（port）**，由 16 位数标识；端口 0 被保留，因此端口号实际范围是 1–65535。它关系到 Safe from bugs（避免连错进程）与 Easy to understand（标准端口是常识）。 – **直观解释（"它是什么？"）**：IP 地址是大厦地址，端口号是大厦里的房间号。大厦（主机）只有一个地址，但里面有很多房间（服务）。 – **关键规则与最佳实践**： – 服务器进程 **bind（绑定）**到某个端口后即在该端口 **listening（监听）**；**同一时刻一个端口只能有一个监听者**，第二个进程去监听同一端口会失败（Java 中抛 `BindException`）。 – 客户端必须知道服务器监听哪一个端口号，这是连接的必要信息之一。 – 记住常用端口：22 = SSH，25 = 邮件（SMTP），80 = HTTP。`http://web.mit.edu` 实际上是在 `18.9.22.69` 的 80 端口上交谈。 – 非标准端口写在 URL 里：`http://128.2.39.10:9000` 表示该机器的 9000 端口。

套接字：监听套接字与连接套接字（Sockets: Listening vs Connected） – **定义与目的**：**套接字（socket）**表示客户端与服务器之间连接的一端。**监听套接字（listening socket）**由服务器用来等待远端客户端连接；**连接套接字（connected socket）**用来与连接另一端的进程收发消息。它给出了一条清晰的**抽象边界**：网络层的复杂性被封装在套接字之后，客户端与服务器只需读写字节流。 – **直观解释（"它是什么？"）**：像 USB 插口。监听套接字是空着的 USB 口，连接套接字是插着线的 USB 口；一条线有两个头，所以连接涉及两个套接字——客户端一个、服务器一个。 – **关键规则与最佳实践**： – 注意物理类比**不准确**的地方：真实 USB 口插上线就不再空着，但**监听套接字在接完一个客户端后依然存在**，仍然绑定同一端口，随时可以 `accept` 下一个客户端。这是"服务器能同时服务多个客户端"的机制根源。 – 在 Java 中：服务器用 `new ServerSocket(port)` 建监听套接字，用 `accept()` 拿到连接套接字；客户端用 `new Socket(hostname, port)` 直接建立连接套接字。 – 连接由"一对"套接字构成，**一方的输出流就是另一方的输入流**：客户端写 `socket.getOutputStream()`，数据流到服务器的 `socket.getInputStream()`。 – 一个端口只有一个监听者，但**一个监听者可以派生任意多个连接套接字**（每个连接一个），这就是并发服务器的基础。

缓冲区、分包与阵发性传输（Buffers and Bursty Transmission） – **定义与目的**：收发数据是**按块（chunks）**进行的，网络把大块切成**数据包（packets）**分别路由，接收端再把它们重新拼成字节流；数据到达后进入**缓冲区（buffer）**，即内存中保存待读数据的数组。它关系到 Safe from bugs：你必须假设数

据"可能已经到了、也可能还没到"，不能假设一次 `read` 就能拿到一整条消息。 – **直观解释（"它是什么？"）**：像快递。你寄一箱书，物流公司拆成多个包裹走不同路线，收件人陆续收到再拼回一箱。数据到达是**阵发（bursty）**的：要么已经在缓冲区里，要么你得等。 – **关键规则与最佳实践**： – 网络传输有**分片**：**不能**假设一次 `read()` 恰好读到一条完整消息，也不能假设多条消息不会粘在一次读取中——这正是"行式协议 + `readLine()`"存在的理由。 – 缓冲区是字节数组：**输出缓冲区满了 `write` 会阻塞，输入缓冲区空了 `read` 会阻塞**。 – 设计协议时用**明确的定界符或长度前缀**（例如每行以换行结束），让接收方能可靠地切分消息。 – 不要依赖"小消息不会被拆分"这种巧合，这在本地测试通过、在真实网络就会出 bug。

阻塞式 I/O（Blocking I/O） – **定义与目的**：输入/输出流表现出**阻塞行为**：当输入套接字的缓冲区为空，调用 `read` 会一直阻塞直到有数据；当输出套接字的缓冲区已满，调用 `write` 会一直阻塞直到有空位。它对程序员**非常方便**：可以像"读一定成功"那样写代码，由操作系统负责把该线程挂起与唤醒。 – **直观解释（"它是什么？"）**：像在食堂排队打饭。轮到之前你站着不动（线程被阻塞），但你不必自己安排"什么时候再来看看"——有人（操作系统）会在轮到你时叫你。 – **关键规则与最佳实践**： – 记住哪些调用可能阻塞：`Socket` 构造、`ServerSocket.accept()`、`BufferedReader.readLine()`、`PrintWriter.println()`（缓冲区满时）。 – **阻塞的是调用它的那个线程，不是整个进程**：其他线程照常运行，这正是"每连接一个线程"能工作的原因。 – 阻塞与 `BlockingQueue` 的消息传递范式同源：向套接字输出流写 ≈ `put()`，从输入流读 ≈ `take()`。 – 服务器绝不能在有其他客户端等待时被一个慢客户端长期阻塞，否则其它客户端会被饿死（starvation）。

字节流与字符流：编码问题（Byte Streams vs Character Streams, UTF-8） – **定义与目的**：套接字上的数据是**字节流**；Java 中 `InputStream` 表示数据流入你的程序，`OutputStream` 表示数据汇（sink）。但程序内部通常要处理 **Unicode 字符**（`String`），因此需要 `Reader` / `Writer` 做字节与字符的转换，`InputStreamReader` / `OutputStreamWriter` 就是把字节流适配成字符流的包装器。它关系到 Safe from bugs：**字符编码（character encoding）用错会静默产生乱码。** – **直观解释（"它是什么？"）**：字节流像传真机收到的点阵，字符流像有人把它翻译成一段文字；编码就是"点阵 ↔ 文字"的对照表，双方必须用同一张表。 – **关键规则与最佳实践**： – **网络通信一律显式使用 UTF-8**：`new InputStreamReader(in, StandardCharsets.UTF_8)`。 – 不要依赖默认编码：Java 可能从系统设置取默认编码（如 Windows 的 CP-1252），于是"文件兼容性"问题会污染网络代码。 – 编码 bug **难以发现**：UTF-8、CP-1252 都是 ASCII 的超集，纯英文文本一切正常；只有重音拉丁字母、非拉丁文字、emoji、弯引号 `“ ”` 才会变成乱码。 – 构造任何 `Reader` / `Writer` 时都显式写出编码，这是本讲所有示例的一致做法。

Java 套接字编程模型（ServerSocket / Socket / BufferedReader / PrintWriter） – **定义与目的**：这是把上面的概念落到 Java 的一套"最小可用 API 组合"。客户端：`new Socket(host, port)` → `getOutputStream()` / `getInputStream()` → 包装成 `PrintWriter` / `BufferedReader`；服务器：`new ServerSocket(port)` → `accept()` → 同样的流包装。它关系到 Easy to understand（模型只有四五个类）与 Ready for change（用接口类型 `BufferedReader` / `PrintWriter` 而不是具体实现，方便替换与测试）。 – **直观解释（"它是什么？"）**：套接字是"一根管道"，`BufferedReader` / `PrintWriter` 是装在管道两端的"行式收发器"：你按行说话，它负责缓冲与切行。 – **关键规则与最佳实践**： – `new Socket(host, port)` 在没

有服务器监听该端口时抛 `IOException`；`new ServerSocket(port)` 在端口已被占用时抛 `BindException`。 – **服务器套接字不提供字节流**，它只生产"新的客户端连接"，这就是 `accept()` 的语义。 – `readLine()` 在**对端关闭连接**时返回 `null`（不是抛异常）；用 `if (line == null) break;` 处理流结束。 – **写完必须 `flush()`**：`println` 只是写进 `PrintWriter` 的内部缓冲区，只有缓冲区满或关闭连接时才真正发送。`PrintWriter` 的 autoflush 只对 `println` 等少数操作生效，`print(msg + "\n")` 可能仍留在缓冲区里。 – 用 **try-with-resources** 保证 `close()` 一定被调用：`try (Socket socket = new Socket(host, port)) { ... }`，适用于 `InputStream` / `OutputStream` / `Reader` / `Writer` / `Socket` / `ServerSocket`。 – 「补充说明」：真实程序还应设置超时，避免永久阻塞——`socket.setSoTimeout(ms)`（读超时，抛 `SocketTimeoutException`）与 `new Socket()` 之后 `socket.connect(addr, timeoutMs)`（连接超时）。

并发服务器：每个连接一个线程（Concurrent Server, Thread-per-Connection） – **定义与目的**：单线程服务器一次只能服务一个客户端：服务器循环被钉死在某个客户端的 `readLine()` 上，直到该客户端断开才回去 `accept` 下一个。要同时服务多个客户端，就需要**为每个新客户端开一个新线程**处理 I/O，而主线程继续待在 `accept()` 上。它直接关系到 Safe from bugs（线程安全论证）与 Easy to understand（把"接受连接"和"处理一个客户端"拆成两段清晰的代码）。 – **直观解释（"它是什么？"）**：像银行大堂。大堂经理（主线程）只负责接引顾客到窗口；每个窗口的柜员（工作线程）各自服务一位顾客。经理不会陪某一位顾客办完全部业务。 – **关键规则与最佳实践**： – 主线程循环：`Socket socket = serverSocket.accept(); new Thread(() -> handleClient(socket)).start();` – 每个连接的数据**被限制（confinement）在它自己的线程里**，这是最省事的线程安全策略；只有真正共享的状态才需要同步。 – 为每个连接新建线程有资源上限：连接数很多时应改用线程池（「补充说明」：`Executors.newFixedThreadPool(n)` / `newCachedThreadPool()`）。 – 共享的可变状态（如全局计数器、共享缓存）必须用消息传递（线程安全队列）或 `synchronized` 保护，并写下**线程安全论证**。 – `handleClient` 里的异常必须捕获：线程中抛出的异常不会传播给主线程，会导致连接悄悄挂掉（并泄漏套接字）。

线路协议与 HTTP（Wire Protocol and HTTP） – **定义与目的**：**协议（protocol）**是两个通信方可以交换的一组消息；**线路协议（wire protocol）**特指以字节序列表示的消息集合，例如 "hello world" 与 "bye"（前提是双方已经约定字符如何编码成字节）。它取代了同进程消息传递中的"选择或设计一个 ADT"，成为跨网络的**通信契约**。 – **直观解释（"它是什么？"）**：像两个国家的外交辞令：说什么、按什么顺序说、说错了怎么办，都有约定；遵守约定，双方才听得懂。 – **关键规则与最佳实践**： – 许多互联网协议是**基于 ASCII 的文本协议**，可以用 `telnet` 直接手工对话（例如 `telnet www.eecs.mit.edu 80` 然后输入 `GET / HTTP/1.1` 与 `Host:` 头，最后以**空行**结束请求）。 – HTTP 是 Web 的语言：80 端口是它的标准端口。请求由**方法（method）**、请求 URI、版本号与头部构成；响应有**三位状态码**（`200 OK` 成功、`404 Not Found` 不存在、`400 Bad Request` 参数有问题、`500 Internal Server Error` 其它失败）与**应答体（reply body）**。 – HTTP 方法对应 ADT 操作分类：**GET 是 observer**（只读、可安全重复），**POST 是 mutator / producer / creator**（会改变或创建数据，浏览器会谨慎地弹窗确认重复提交）。PUT、DELETE 在 Web API 中较少用。 – 参数有三种传法：**路径分量**（`/points/42.3541,-71.1104`）、**查询参数**（`?area=MA&severity=Minor`）、**请求体**（body，只有 POST 之类才有）。GET 无 body，因此只能用前两种。 – 返回结果常用 **JSON**

（JavaScript Object Notation），它是交换结构化数据最常见的方式；**序列化（serialization）**指把内存中的数据结构转换成便于存储或传输的格式，不要自己发明格式。 – 协议要**版本化**（Ready for change）：HTTP 让客户端与服务器协商版本，新老实现可以共存。 – `telnet` / `curl` 都能与服务器说 HTTP：**能说 HTTP 的工具可以是浏览器、telnet、curl、你写的客户端**，关键在于协议而非实现。

用文法与规格说明描述协议（Grammar + Specs） – **定义与目的**：要精确说明"允许哪些消息"，应当写出**文法（grammar）**；但文法只相当于 ADT 的**方法签名**，还必须补充**前置条件（precondition）**与**后置条件（postcondition）**。它关系到 Safe from bugs（拒绝非法消息）与 Easy to understand（协议文档可读）。 – **直观解释（"它是什么？"）**：文法规定"句子的形状"，规格说明规定"这句话在什么情况下才能说、说了以后会发生什么"。 – **关键规则与最佳实践**： – 文法回答：哪些字节序列是合法消息（例如 `ON ::= "on " ID`、`ID ::= [1-9][0-9]*`，因此 `on 0` 非法）。 – 规格说明回答：字段取值范围的前置条件（是任意数字，还是服务器已知记录的 ID？）、消息的**发送时序**约束（某些消息只有在特定序列中才合法）、以及**后置条件**（服务器会改哪些数据、回什么消息）。 – 用现成工具解析协议（由文法自动生成的解析器、或正则表达式库）比手写字符串切分更不易出错。 – 限制单条消息的规模并校验字段，防止恶意客户端撑爆服务器缓冲区。

网络分层的抽象（Layers of Abstraction） – **定义与目的**：网络是**分层**的：物理链路 → IP（把包送达主机）→ TCP（提供可靠、有序的字节流）→ 应用协议（HTTP、SMTP）。**套接字正是 TCP 提供给应用层的抽象**：它把"重传、排序、拥塞控制"全部隐藏，只暴露"可靠字节流"这一简洁契约。 – **直观解释（"它是什么？"）**：像寄信的多级体系：你只管把信投进邮筒（写套接字），邮政系统负责分拣、转运、丢件重发，收件人收到的是完整有序的信。 – **关键规则与最佳实践**： – TCP 保证**字节流有序可靠**，但**不保证消息边界**——分帧（framing）必须由应用协议自己解决（这也是行式协议流行的原因）。 – 套接字之上可以再套抽象：`HttpURLConnection`、`ServerSocket` + `BufferedReader`、或整个 Web 框架；抽象层次越高，代码越少但控制力越弱。 – 抽象边界两侧的**契约**就是协议规格：一边的实现变了（换数据库、换语言），另一边不受影响，这就是"**平台无关性（platform independence）**"，与 ADT 的**表示独立性（representation independence）**是同一思想。 – 设计协议时不要泄露实现细节：HTTP 不规定网页如何存储、如何生成、客户端如何渲染。

把读写流抽象为 ADT：分离套接字代码与流代码（Separating Socket Code from Stream Code） – **定义与目的**：需要读写套接字的函数/模块，往往**只需要输入输出流，而不需要套接字本身**；把参数类型定为 `BufferedReader` / `PrintWriter`（而不是 `Socket`），就能用不来自套接字的流来测试它。它极大提升 Safe from bugs（可单元测试）与 Ready for change（换传输方式不改逻辑）。 – **直观解释（"它是什么？"）**：像家电的插头标准：电器（业务逻辑）只依赖"电"（流），不关心电来自火电还是水电（套接字还是内存数组）。 – **关键规则与最佳实践**： – 函数签名写 `void upperCaseLine(BufferedReader input, PrintWriter output) throws IOException`，而不是 `void upperCaseLine(Socket sock)`。 – 用 `ByteArrayInputStream` 提供**固定输入**、`ByteArrayOutputStream` 收集输出，二者就是**测试桩（test stub）**。 – 更完整的模块可以用 **mock object** 模拟真实客户端/服务器的整段交互序列，逐条断言消息。 – 把不涉及网络的 ADT（数据结构、算法）单独规格化、测试、实现，让它们本身与网络无关；它们若会被多

线程使用，就用消息传递/同步/不可变等策略保证线程安全。 – 并发与网络**难以测试和调试**：race condition 不可复现，网络延迟不可控，因此必须"为并发而设计"并给出正确性论证。

代码示例与对比分析

场景 1：为套接字流构造字符流时是否显式指定编码

❌ *错误代码*

```
// 错误：依赖平台默认字符编码
Socket socket = new Socket(hostname, port);

PrintWriter writeToServer = new PrintWriter(socket.getOutputStream());
BufferedReader readFromServer = new BufferedReader(
        new InputStreamReader(socket.getInputStream()));

writeToServer.println("café ☕ – naïve");
writeToServer.flush();
```

【错误代码的问题】 1. 在 Windows（默认 CP-1252）上运行的服务端与在 Linux（默认 UTF-8）上运行的客户端互相发送含重音字符、破折号或 emoji 的消息时，接收方会把它们解成乱码（mojibake）。 2. 纯英文测试全部通过，bug 只在非 ASCII 文本上出现，属于典型的"潜伏型"缺陷，极难定位。 3. 编码行为随运行环境变化，违反可移植性：同一份代码在不同机器上语义不同，违反"correct in the unknown future"。

✅ *正确代码*

```
// 正确：显式指定 UTF-8，网络通信的通用选择
Socket socket = new Socket(hostname, port);

PrintWriter writeToServer = new PrintWriter(
        new OutputStreamWriter(socket.getOutputStream(), StandardCharsets.UTF_8));
BufferedReader readFromServer = new BufferedReader(
        new InputStreamReader(socket.getInputStream(), StandardCharsets.UTF_8));

writeToServer.println("café ☕ – naïve");
writeToServer.flush();
```

【为什么这样更好】 两端都按 UTF-8 解释字节序列，编码成为**协议的一部分**而不是环境的偶然属性；任何符合 UTF-8 的客户端/服务器都能正确互通，字符编码 bug 被从根上消除。 **【代码对比解说】** 两种写法的**逻辑完全相同**，差异只在包装层的第三个参数。`InputStreamReader` / `OutputStreamWriter` 是"字节流 → 字符流"的**适配器（adapter）**：它必须知道用哪张对照表。不指定时它就问系统要一张表，而系统给出的答案可能与对端不同。注意 `OutputStreamWriter(OutputStream)` 与 `OutputStreamWriter(OutputStream, Charset)` 的差异是"静默依赖环境"与"显式声明契约"的差异，正是规格说明精神的体现：把隐含假设变成显式约定。 **【设计原则透视】** 这是**规格说明（Reading 6/7）**在 I/O 上的体现：编码是接口契约的一部分，不是实现细节。它也是**抽象边界**问题：`Reader` / `Writer` 的抽象把"字节 ↔ 字符"的转换集中到一处，前提是这

个抽象的配置必须与对端一致。与 Reading 11 的表示独立性类比：只要双方约定的"表示"（编码）一致，各自的内部实现如何都无所谓。

场景 2：发送消息后忘记 flush

❌ *错误代码*

```
// 错误：println 只写进了客户端侧缓冲区，消息可能根本没发出去
PrintWriter writeToServer = new PrintWriter(
        new OutputStreamWriter(socket.getOutputStream(), StandardCharsets.UTF_8));

String message = "hello";
writeToServer.println(message);   // 看似已发送
// 没有 flush，客户端随后阻塞在 readLine() 上等应答 —— 服务器根本没收到请求

// 同样错误的变体：构造时开启 autoflush，却用 print 而非 println
PrintWriter auto =
        new PrintWriter(new OutputStreamWriter(socket.getOutputStream(),
                StandardCharsets.UTF_8), true /* autoflush */);
auto.print(message + "\n");       // autoflush 对 print 不生效，仍留在缓冲区
```

【错误代码的问题】 1. **死锁式的互相等待**：客户端等应答、服务器等请求，程序永久挂起；在单机测试里往往"偶尔正常"，因为缓冲区填满会触发实际发送，从而掩盖了缺陷。 2. 交互式协议（一问一答）几乎必然卡死，用户以为程序崩溃。 3. 用 `print(message + "\n")` 替代 `println(message)` 会让 autoflush 悄然失效——这是一个"看起来等价"的陷阱，破坏 Easy to understand。

✅ *正确代码*

```
// 正确：写完后显式 flush
PrintWriter writeToServer = new PrintWriter(
        new OutputStreamWriter(socket.getOutputStream(), StandardCharsets.UTF_8));

String message = "hello";
writeToServer.println(message);
writeToServer.flush();            // 重要！否则这一行可能只是躺在缓冲区里

// 或者在构造时开启 autoflush，并且只用 println 这类会触发自动 flush 的操作
PrintWriter auto = new PrintWriter(
        new OutputStreamWriter(socket.getOutputStream(), StandardCharsets.UTF_8),
        true /* autoflush */);
auto.println(message);            // println 会触发自动 flush
```

【为什么这样更好】 `flush()` 把"逻辑上已发送"变成"物理上已发送"，消除了双方互等的死锁风险；显式 flush 让"何时真正上线"在代码中一目了然，读者无需推理缓冲区大小。 **【代码对比解说】** `PrintWriter` 是**带缓冲的装饰器（decorator）**：`println` 的规格只是"把行写入缓冲"，而不是"发到网络"。缓冲区存在的理由是性能（合并小写操作）；代价是**时序被推迟**。因此凡是"请求—应答"式的交互协议，都必须在一轮写完之后 flush。课程原文的提醒值得抄在笔记本上：*always remember to flush.* 同理，关闭流（`close()`）也会

触发 flush，因此 try-with-resources 能在退出时兜底，但**不要**依赖它来保证交互时序——应答可能在关闭之前根本不会到来。 **【设计原则透视】** 缓冲是**表示（rep）**的一部分，它引入了"逻辑状态 vs 物理状态"的差异；不 flush 的代码在**方法签名与规格层面看不出任何问题**，属于"规格没写清、实现有隐藏前置条件"的经典案例：真正的后置条件应是"消息已交给操作系统发出"。这也是 Easy to understand 的反面教材——代码的可见行为与读者预期的行为不一致。

场景 3：单线程服务器 vs 每连接一个线程的并发服务器

❌ *错误代码*

```
// 错误：整个服务器一次只能服务一个客户端
public static void main(String[] args) throws IOException {
    int port = 4589;
    try (ServerSocket serverSocket = new ServerSocket(port)) {
        while (true) {
            Socket socket = serverSocket.accept();   // 阻塞直到有新连接
            handleClient(socket);                    // 一直服务到该客户端断开
            // 在此之前，其他客户端只能排队等待，accept() 根本不会被再次调用
        }
    }
}

private static void handleClient(Socket socket) throws IOException {
    try (BufferedReader readFromClient = new BufferedReader(
                new InputStreamReader(socket.getInputStream(), StandardCharsets.UTF_8));
         PrintWriter writeToClient = new PrintWriter(
                new OutputStreamWriter(socket.getOutputStream(), StandardCharsets.UTF_8)))
        while (true) {
            String message = readFromClient.readLine();   // 慢客户端会把服务器钉在这里
            if (message == null) break;                    // 客户端关闭连接
            if (message.equals("quit")) break;              // 毒丸消息
            writeToClient.println("echo: " + message);
            writeToClient.flush();
        }
    }
}
```

【错误代码的问题】 1. **一个慢客户端阻塞所有人**：任何客户端的空闲连接都会让服务器无法接待新客户，可用性随客户端数量线性恶化甚至完全瘫痪（简单的拒绝服务）。 2. 交互式协议下用户体验崩坏：第二个客户端"连上了却没有任何反应"，看起来像网络故障。 3. 若把 `handleClient` 内部逻辑改成"等待某个外部事件"，服务器会永久失去服务能力，且没有明显报错，调试困难。

✅ *正确代码*

```
// 正确：主线程只负责接受连接，每个连接交给一个新线程处理
public static void main(String[] args) throws IOException {
    int port = 4589;
    try (ServerSocket serverSocket = new ServerSocket(port)) {
        while (true) {
            final Socket socket = serverSocket.accept();   // 阻塞直到有新连接
            // 新线程处理该客户端，主线程立刻回到 accept() 等下一个
            new Thread(new Runnable() {
                public void run() {
                    try {
                        handleClient(socket);
                    } catch (IOException ioe) {
                        ioe.printStackTrace();   // 线程内异常必须自己处理，不能上抛给主线程
                    }
                }
            }).start();
        }
    }
}

private static void handleClient(Socket socket) throws IOException {
    try (BufferedReader readFromClient = new BufferedReader(
                new InputStreamReader(socket.getInputStream(), StandardCharsets.UTF_8));
         PrintWriter writeToClient = new PrintWriter(
                new OutputStreamWriter(socket.getOutputStream(), StandardCharsets.UTF_8)))
        while (true) {
            String message = readFromClient.readLine();
            if (message == null) break;
            if (message.equals("quit")) break;
            writeToClient.println("echo: " + message);
            writeToClient.flush();
        }
    }   // 离开 try 时自动 close 流与套接字
}
```

【为什么这样更好】 阻塞式 I/O 下，"阻塞"只影响调用它的线程：一个线程被慢客户端阻塞时，主线程仍能接待新客户端，其他工作线程仍在服务自己的客户端。每个连接的读写状态**被限制（confinement）在该连接自己的线程里**，天然避免了共享可变状态，是最简单的线程安全策略。 **【代码对比解说】** 两种写法的差别只有"`accept()` 之后立刻开线程"这一步，但它把服务器的**并发度从 1 提升到 O(客户端数)**。代价是：线程是有限资源，连接数很大时应改用线程池；同时必须处理线程内异常（否则静默失败并泄漏 `Socket`）；若有跨连接的共享状态，就要引入消息传递或同步。此外，`quit` 毒丸与 `readLine()` 返回 `null` 代表两种不同的停止方式——后者意味着客户端直接关闭了自己这一端。 **【设计原则透视】** 这是 Reading 21（并发）、Reading 23（互斥）、Reading 24（消息传递） 的直接应用：并发模块必须给出**线程安全论证**；本方案用的是"线程限制 + 每连接独立状态"，共享状态为零，因此无需锁。`ServerSocket` 与 `Socket` 之间的分工也体现了抽象设计：把"接待"与"服务"两种职责分开，才能各自独立地阻塞而不互相妨碍。

场景 4：业务逻辑直接依赖 Socket vs 依赖读写流（可测试性）

❌ *错误代码*

```
// 错误：把网络细节写死在业务逻辑里，无法脱离真实网络测试
public static void upperCaseLine(Socket sock) throws IOException {
    BufferedReader in = new BufferedReader(
            new InputStreamReader(sock.getInputStream(), StandardCharsets.UTF_8));
    PrintWriter out = new PrintWriter(
            new OutputStreamWriter(sock.getOutputStream(), StandardCharsets.UTF_8), true);

    String line = in.readLine();
    if (line == null) return;
    out.println(line.toUpperCase());
    out.flush();
}
// 测试时必须先启动一个真实服务器、建立真实连接、控制时序 —— 慢、脆、且难覆盖边界情况
```

【错误代码的问题】 1. **不可单元测试**：要验证方法本身，必须搭起 `ServerSocket`、连上网络、处理端口占用与超时，测试变成集成测试，脆弱且慢。 2. 每次测试都用掉一个端口，测试之间互相干扰，并行运行会随机失败。 3. 逻辑与传输耦合，将来换成管道、文件或内存队列就要重写。

✅ *正确代码*

```
// 正确：只依赖字符流，因此可以用内存流作为测试桩
public static void upperCaseLine(BufferedReader input, PrintWriter output)
        throws IOException {
    String line = input.readLine();
    if (line == null) return;                  // 输入已结束
    output.println(line.toUpperCase());
    output.flush();
}

// 生产环境：接到套接字上
Socket sock = new Socket(hostname, port);
BufferedReader in = new BufferedReader(
        new InputStreamReader(sock.getInputStream(), StandardCharsets.UTF_8));
PrintWriter out = new PrintWriter(
        new OutputStreamWriter(sock.getOutputStream(), StandardCharsets.UTF_8),
        true /* autoflush */);
upperCaseLine(in, out);

// 单元测试：用内存字节流替换网络（测试桩）
String inString = "dog\ncat\n";
ByteArrayInputStream inBytes = new ByteArrayInputStream(inString.getBytes(StandardCharsets.
ByteArrayOutputStream outBytes = new ByteArrayOutputStream();

BufferedReader testIn = new BufferedReader(
        new InputStreamReader(inBytes, StandardCharsets.UTF_8));
PrintWriter testOut = new PrintWriter(
        new OutputStreamWriter(outBytes, StandardCharsets.UTF_8), true);

upperCaseLine(testIn, testOut);

assertEquals("cat", testIn.readLine(), "expected input line 2 remaining");
assertEquals("DOG\n", outBytes.toString(StandardCharsets.UTF_8), "expected upper case");
```

【为什么这样更好】 方法的**依赖被抽象成接口参数**（`BufferedReader` / `PrintWriter`），于是可以在测试里用内存流替换真实网络：`ByteArrayInputStream` 提供确定输入，`ByteArrayOutputStream` 捕获输出。测试快、可重复、无端口冲突，且能顺便断言"恰好消费了第一行"，覆盖了读位置的边界。 **【代码对比解说】** 关键改动是**把 `Socket` 从参数中拿掉**：`Socket` 只是"如何获得流"的一种方式，方法真正需要的是"一条能读行、能写行的字符流"。这种"依赖倒置"让同一个方法可以在三种环境里复用：真实套接字、内存流（单元测试）、mock 对象（模拟完整交互序列）。注意两处断言分别检查**输入被消费到什么位置**与**输出内容**，这是把"读写行为"也纳入规格的例子。 **【设计原则透视】** 这是 **Reading 12（用接口定义 ADT）** 与 **Reading 3（测试）** 的结合：面向接口编程让模块可替换；测试桩/mock 是"满足同一规格但行为可预测的替代实现"。它也体现 **Reading 7（设计规格）** 的思想：先把 `upperCaseLine` 的前置条件（`input`、`output` 处于打开状态）与后置条件（读一行、写其大写）写清，测试才能只针对这些条件构造用例。

场景 5：手工切割协议文本 vs 用文法/校验设计协议

❌ *错误代码*

```
// 错误：靠 split 和字符串拼接"差不多"地解析协议，既不校验也不限长
public static void handleMessage(String message, List<Light> lights) {
    String[] parts = message.split(" ");
    String command = parts[0];
    int id = Integer.parseInt(parts[1]);   // "help" 会数组越界；"on 0" 会被当成合法 ID

    if (command.equals("on")) {
        lights.get(id).turnOn();          // 未校验 id 范围，可能 IndexOutOfBounds
    } else if (command.equals("off")) {
        lights.get(id).turnOff();
    }
    // 其它命令被静默忽略，客户端永远得不到应答
}
```

【错误代码的问题】 1. 违反协议文法的输入（`on 0`、`on`、`OFF 1`、超长 ID）会导致 `ArrayIndexOutOfBoundsException` 或 `NumberFormatException`，**服务器被一个畸形消息打挂**（安全漏洞：拒绝服务）。 2. 非法输入被静默忽略，客户端无从知道出了什么问题——违反 Easy to understand，也让调试变成猜谜。 3. 没有对消息长度设上限，恶意客户端可以持续发送超长行耗尽服务器内存或缓冲区。

✅ *正确代码*

```
// 正确：按文法解析并严格校验；非法输入返回明确的错误应答
// 文法（来自课程原文的例子）：
//   MESSAGE ::= ( ON | OFF | HELP_REQ ) NEWLINE
//   ON      ::= "on " ID
//   OFF     ::= "off " ID
//   HELP_REQ::= "help"
//   ID      ::= [1-9][0-9]*
private static final Pattern ON_OR_OFF =
        Pattern.compile("(on|off) ([1-9][0-9]*)");
private static final Pattern HELP = Pattern.compile("help");

/** @param message 已由 readLine() 去掉行结束符的一行输入
 *  @return 给客户端的应答；非法输入返回 "error: bad request" */
public static String handleMessage(String message, int lightCount) {
    if (message.length() > 100) {                 // 限制规模，防止缓冲区耗尽
        return "error: message too long";
    }
    Matcher help = HELP.matcher(message);
    if (help.matches()) {
        return "turn lights on and off with: on <id> / off <id>";
    }
    Matcher m = ON_OR_OFF.matcher(message);
    if (!m.matches()) {
        return "error: bad request";              // 明确拒绝，而不是猜
    }
    int id = Integer.parseInt(m.group(2));
    if (id > lightCount) {                        // 前置条件：ID 必须是已知的灯
        return "error: unknown light " + id;
    }
    return m.group(1) + " " + id + " ok";
}
```

【为什么这样更好】 解析规则与**文法一一对应**（`(on|off) <id>`、`help`），任何不符合文法的输入都被显式拒绝并得到可读的错误应答；规模上限把"恶意或损坏的对端"变成可处理的错误而不是崩溃。代码与协议文档保持同构，改协议时改文法即可。 **【代码对比解说】** 前者的 `split(" ")[1]` 隐含假设"消息一定是两个词、第二个词一定是数字、数字一定是合法 ID"——这些假设都**不在协议规格里**，属于典型的隐藏前置条件。后者用一份正则把文法写死，再单独检查"ID 是否已知"这一真正的语义前置条件，层次清晰：**语法**（形状对不对）与**语义**（内容合不合法）分开处理。返回字符串而非抛异常，也让服务器可以继续服务同一个客户端——这正是线路协议该有的宽容度：坏消息不等于坏连接。 **【设计原则透视】** 呼应 **Reading 18（正则表达式与文法）** 与 **Reading 19（解析器）**：协议本身就是一种语言，先用文法定义它，再让解析器实现它，能显著减少 bug。同时体现 **Reading 6/7** 的规格思想——文法相当于方法签名，前置/后置条件才是完整的规格。安全视角上，这正是课程反复强调的："考虑损坏或恶意的客户端/服务器如何往协议里塞垃圾数据来破坏对端"，SMTP 允许谎报 `From:` 就是历史教训。

与其他设计原则的关联

- **Reading 24（消息传递 Message-Passing）**：套接字读写与 `BlockingQueue` 的 `put()` / `take()` 是同一种"阻塞式消息传递"范式；毒丸消息（`quit`）与"对端关闭连接导致 `readLine()` 返回 `null`"对应两种终止协议的方式。区别在于套接字传递的是**字节流**，因此必须额外设计协议（分帧、编码、序列化）。
- **Reading 21（并发）与 Reading 23（互斥）**：网络通信天然并发，服务器为每个连接开线程，因此必须写出**线程安全论证**。本讲推荐的策略是线程限制（每个连接的流只被自己的线程访问）与不可变/消息传递；一旦引入跨连接共享状态，就需要 Reading 23 的锁或 Reading 24 的线程安全队列。
- **Reading 6（规格说明）与 Reading 7（设计规格）**：线路协议、Web API 都需要前置条件与后置条件；文法只相当于方法签名。`upperCaseLine` 的"输入输出必须打开"就是前置条件。
- **Reading 10（抽象数据类型）与 Reading 11（抽象函数与表示不变量）**：协议是客户端与服务器之间的抽象边界，**平台无关性**与 ADT 的**表示独立性**是同一思想；把 `Socket` 换成流参数，正是"缩小依赖接口"的 ADT 设计手法。
- **Reading 12（接口、泛型、枚举、函数对象）**：用接口类型（`BufferedReader`、`PrintWriter`，乃至自定义的 `SequencePlayer` 式接口）编程，才能替换实现、注入测试桩。
- **Reading 3（测试）与 Reading 4（代码评审）**：`ByteArrayInputStream` / `ByteArrayOutputStream` 测试桩与 mock object 属于 Reading 3 的技术；把套接字代码与流代码分离正是代码评审中最常见的"可测试性"评点。
- **Reading 18（正则与文法）、Reading 19（解析器）**：协议用文法描述、由解析器实现；HTTP 请求行/头部的文法来自 RFC，正是"文法即规格"的现实案例。
- **Reading 26/27（小语言 I/II）**：协议是一种"小语言"，消息集合是它的句子；当消息结构复杂到需要树形表示时，就需要 Reading 26 的递归数据类型与 Reading 27 的访问者模式来处理消息。

关键要点

- **网络编程 = 并发编程 + 协议设计**：先把"谁先说话、说什么、出错怎么办"写成文法与规格，再写代码；不要一边写 `readLine` 一边想协议。
- **套接字之上永远自己分帧**：TCP 只保证有序可靠的字节流，不保证消息边界；行式协议 + `readLine()` 是最省心的分帧方式，但要显式处理 `null`（对端关闭）。
- **三件套不能忘**：显式 `StandardCharsets.UTF_8`、写完 `flush()`、用 try-with-resources 关闭；再加上「补充说明」中的超时设置，才算健壮。
- **服务器不要被任何单个客户端阻塞**：主线程只 `accept`，每连接一个线程（连接数大时用线程池），并保证线程内异常被捕获。
- **把 `Socket` 挡在业务逻辑之外**：让核心函数只依赖 `BufferedReader` / `PrintWriter`，用内存流做测试桩，让网络代码薄得只剩"连接、包装、关闭"。

常见陷阱与注意事项

1. **忘记 `flush()`** → 消息留在客户端缓冲区未发送，双方互等造成死锁；用 `print(msg + "\n")` 搭配 autoflush 更是"看起来等价"的陷阱。
2. **不指定字符编码** → 在非 ASCII 文本上出现乱码，且在本机英文测试中完全看不出来；Windows 上的 CP-1252 默认值是典型祸首。
3. **单线程服务器，或工作线程内异常未捕获** → 前者让一个慢/空闲客户端把所有其他客户端排队（表现为"连得上但没反应"）；后者让工作线程静默死亡，连接挂起、套接字泄漏，服务器表面正常但已有连接"黑洞"。
4. **把 `readLine()` 返回的 `null` 当作空字符串处理** → 对端已关闭连接时陷入死循环或写出错误应答；`null` 是流结束信号，必须 `break`。
5. **在业务逻辑里直接用 `Socket`** → 无法单元测试、端口冲突、逻辑与传输耦合；测试要么很慢要么很脆。
6. **协议解析不校验、不限长** → 畸形消息触发异常打挂服务器（可用性/安全漏洞），或超长消息耗尽内存；同时非法输入被静默忽略会让客户端无从调试。

思考题（带答案）

问题 1：为什么 `ServerSocket` 与 `Socket` 被设计成两个不同的类，而不是让 `ServerSocket.accept()` 返回它自己（像物理 USB 口那样"插上线就变成连接口"）？请从"一个端口只有一个监听者但可以有多个并发连接"的角度解释，并说明这对并发服务器的意义。

答案：物理插口的类比会误导人。真实 USB 口插上线后就不再空着，而网络监听套接字在接完一个客户端后**仍然存在、仍然绑定同一端口，并在下次 `accept()` 时交出下一个客户端**。因此必须区分两种角色：监听套接字负责"以端口为标识接待新连接"，连接套接字负责"与某一个具体对端收发字节"。如果 `accept()` 返回监听套接字自身，服务器就无法同时保留"端口的所有权"与"多个已建立连接"，也就无法在服务客户端 A 的同时继续接待客户端 B。正因为 Java 为每个新连接**新建**一个 `Socket` 对象，服务器才能把每个连接交给独立线程，形成"主线程 `accept` + 每连接一个工作线程"的并发结构（连接数大时改用线程池）。这也解释了为什么端口冲突的错误发生在 `new ServerSocket(port)`（`BindException`），而不是发生在 `accept()`。

问题 2：`EchoServer` 有两种结束与某个客户端对话的方式：客户端发送 `"quit"`，或客户端关闭自己这一端的连接。这两种方式在服务器端代码里分别表现为哪一行？为什么协议设计者需要同时支持二者？

答案：`"quit"` 是**应用层协议**里的毒丸消息（poison pill），由服务器循环中的 `if (message.equals("quit")) break;` 处理；客户端关闭连接则表现为 `readLine()` 返回 `null`，由 `if (message == null) break;` 处理。二者都需要支持，因为：毒丸消息是"礼貌的、有语义的告别"，客户端可以在结束前确保自己已收到所有应答；而连接关闭是**传输层事件**，客户端崩溃、网络断开、进程被杀死时都可能发生，服务器无法阻止，只能把它识别为流结束并释放资源。如果只处理毒丸，一旦客户端异常退出，服务器线程就会读到 `null` 却继续把它当字符串处理（例如 `null.equals("quit")` 抛 `NullPointerException`，或陷入死循环），造成线程与套接字泄漏。这与 Reading 24 中"`BlockingQueue` 的 `take()` 需要毒丸来终止消费者"的思想一致，只不过套接字额外提供了"对端消失"这一层信号。

问题 3：你要写一个"每行是一条 JSON 请求、每行是一条 JSON 应答"的网络服务。请说明为什么"每行一个 JSON 对象"这种设计同时改善了 Safe from bugs、Easy to understand 与 Ready for change，并指出它相

对"直接发送裸 JSON 流"的关键优势。若换成"发送长度前缀 + 二进制体"会带来什么取舍?

答案：分帧（framing）是这里的核心。裸 JSON 流的问题是接收方无法判断"当前这个对象到哪里结束"，必须自己做括号匹配式的增量解析，且消息间无法复用同一套简单读取逻辑。改为"每行一个 JSON"后：(1) Safe from bugs——用 `readLine()` 就能可靠地切出一条完整消息，解析器只需处理一整行，边界情况大幅减少；(2) Easy to understand——可以用 `telnet` 或 `nc` 手工敲一行 JSON 与服务对话，调试时能直接看懂报文；(3) Ready for change——行内 JSON 可以轻松增删字段而不破坏分帧，新老字段可共存，双方可以各自演进。JSON 本身是成熟的**序列化**格式，不要自己发明一套转义规则。代价与取舍：文本协议体积较大、解析有开销，且含换行的字符串必须转义（JSON 规范已处理）；若改用"长度前缀 + 二进制体"，则分帧更紧凑、可承载任意二进制数据、解析更快，但报文不再可读、不便手工调试，且必须严格约定字节序与长度字段宽度，对实现错误的容忍度更低。选择取决于"可读/可调试"与"紧凑/高效"哪个更重要——这正是 Reading 7 中设计规格时的权衡思路。

Reading 26: 小语言 I（Little Languages I）

> 说明：本讲 sp22 原版使用 TypeScript（`Music` 接口、`notes()` 工厂函数、`Pitch.make` 等），本笔记按用户要求提供 Java 代码示例；类型/API 与 sp21（6.031 Java 版，Reading 27: Little Languages I）原文保持一致，并沿用课程的音乐语言（`Music` / `Note` / `Rest` / `Concat` / `MusicLanguage` / `SequencePlayer` / `Pitch` / `Instrument`）。凡属为演示目的而补充的内容（例如给 Reading 19 的 `IntegerExpression` 增加 `Variable` 变体以展示 `eval(environment)`），均明确标注为「补充说明」。

概述

本讲的主线只有一句话：**当你需要解决一个问题时，不要只写一个解决这个问题的程序，而要构造一门能解决一整类相关问题的语言**（*when you need to solve a problem, instead of writing a program to solve just that one problem, build a language that can solve a range of related problems*）。为此需要两个关键思想：一是**把代码表示为数据（representing code as data）**，让表达式成为可以被存储、传递、操纵、延后求值的一等值；二是**用递归数据类型（recursive data type）表示这门语言的抽象语法树**，并按照**解释器模式（Interpreter pattern）**为它定义操作。它与三大目标的关系是：Safe from bugs（结构化表示替代大量重复的手写调用，减少人为失误；求值/解析逻辑可规格化、可测试）、Easy to understand（`notes("C D E F G A B C'")` 远比几十行 `addNote` 好读）、Ready for change（语言可以扩展出转调、变奏、混音等一整类新功能，而不必重写已有程序）。

核心概念与设计原则详解

把代码表示为数据（Representing Code as Data） – **定义与目的**：把"要做什么"编码成一个**数据结构**（`Formula`、`IntegerExpression`、`Music`），而不是写成一段立即执行的语句。它解决的是"我想在运行时操纵、保存、重复求值这段计算"的问题，直接关系到 Ready for change 与 Safe from bugs。 – **直观解释（"它是什么？"）**：Java 里写 `p && q`，这个表达式**一出现就被求值**，结果是个 `boolean`；而

`And(Variable("p"), Variable("q"))` 是一个**对象**，它"记住"了这个公式的形状，你可以把它放进集合、写进文件、传给别的线程，也可以在需要时求值一次、两次或一百次。 – **关键规则与最佳实践**： – 判断是否需要"把代码当数据"：**是否需要在求值之前操纵它，或者需要求值多次/延后求值**？是则建模为数据。 – 课程的 `Formula` 类型是标准例子：命题逻辑公式 `(p ∧ q)` 表示为 `And(Variable("p"), Variable("q"))`；用文法与解析器的术语说，公式构成一门**语言**，而 `Formula` 是它的**抽象语法树（abstract syntax tree, AST）**。 – 另一类"把代码当数据"的例子是**函数对象（functional object）/一等函数**：`class AndFunction implements BiFunction<Boolean,Boolean,Boolean>`，或 Java 的 lambda `(p, q) -> p && q`；它们同样是把计算变成可传递的值。 – 数据的**不可变性**至关重要：AST 一旦构造就不再改变，才能安全地被共享、被多次求值（参见 Reading 8）。

小语言与领域特定语言（Little Language / Domain-Specific Language） – **定义与目的**：为某个**狭窄领域**设计的语言称为**领域特定语言（domain-specific language, DSL）**，因为它的适用范围比 Java、Python 这类通用语言窄。本讲与下一讲要构造的**音乐语言**就是一门 DSL（little language）。它关系到 Easy to understand（领域内表达更简洁）与 Ready for change（语言可扩展）。 – **直观解释（"它是什么？"）**：通用语言像"什么都能做的手术刀组合"；DSL 像专门切寿司的刀——只干一件事，但干得特别好、特别顺手。 – **关键规则与最佳实践**： – **外部 DSL（external DSL）**：自带语法与语义，独立于任何通用语言。本课程已经见过的例子是**正则表达式**、**ParserLib 文法**、以及 Problem Set 3 的语言（Memely）。 – **内部 DSL（internal DSL）**：嵌入在通用语言里，借用宿主语言的语法与抽象机制，不另造语法（课程在 sp22 用 TypeScript、sp21 用 Java）。**一等函数与函数对象**让内部 DSL 特别强大，因为可以把计算模式抽成可复用的抽象。音乐语言是内部 DSL。 – 定义 ADT 本身就是"扩展语言"：新类型 = 新的名词（值），新操作 = 新的动词（操作），而这些名词动词又建立在既有的抽象之上。 – 语言的威力在于**解决一整类问题**：从"写 `p && q`"到"设计 `Formula` 类型"，从"写一个矩阵乘法函数"到"设计 `MatrixExpression` 类型"，差别就在于此。

递归数据类型与抽象语法树（Recursive Data Type / Abstract Syntax Tree） – **定义与目的**：用递归数据类型描述语言的语法结构。课程的整数表达式（Reading 19）定义为 `IntegerExpression = Number(n:int) + Plus(left:IntegerExpression, right:IntegerExpression)`；音乐语言定义为 `Music = Note(duration, pitch, instrument) + Rest(duration) + Concat(first, second)`。它关系到 Safe from bugs（结构由类型系统静态检查）与 Easy to understand（结构直接对应语法）。 – **直观解释（"它是什么？"）**：AST 是"句子的骨架图"：它保留了对语义重要的部分（怎么分组、有哪些数字/音符），而丢掉了无关的书写细节。 – **关键规则与最佳实践**： – **抽象语法树 vs 具体语法树（concrete syntax tree）**：`2+2`、`((2)+(2))`、`0002+0002` 产生三棵不同的具体语法树，却都对应同一个抽象值 `Plus(Number(2), Number(2))`。解析器的工作就是把前者变成后者。 – 递归数据类型的每个变体对应文法中的一条产生式，**变体名通常取产生式的名字**（`Because` / `Plus` / `Number` / `Note` / `Concat`）。 – 「补充说明」：不同教材习惯把变体命名为 `PlusExpression`、`VariableExpression` 之类（把类型名后缀带上）；6.031 的做法是**直接用产生式的名字**，例如 `Plus`、`Concat`、`Variable`，本笔记遵循课程命名。 – 选择表示时要考虑未来：课程原文特别说明，音乐语言选择**树形的** `Concat` 是"一个优雅的决定"，因为它让后续扩展（变奏、和声、重复）变得自然；真实的设过程可能需要多次迭代才能找到最合适的递归结构。

组合模式（Composite Pattern） – **定义与目的**：让**单个对象（primitive，基元）**与**对象组（composite，组合）**属于**同一个类型**，从而可以被同样对待。`Music` 正是组合模式：基元是 `Note` / `Rest`，组合是 `Concat`；`Formula` 的基元是 `Variable`，组合是 `Not` / `And` / `Or`。它关系到 Easy to understand（统一的递归操作）与 Ready for change（加新组合子不必改客户代码）。 – **直观解释（"它是什么？"）**：像文件夹与文件：文件是基元，文件夹是组合，但两者都是"文件系统条目"，都能被"删除""移动""计算大小"。组合模式自然产生**树**：基元在叶子，组合在内部结点。 – **关键规则与最佳实践**： – 组合变体（`Concat`、`Not`、`And`、`Or`）在实现操作时**递归**；基元变体（`Note`、`Rest`、`Variable`）实现**基例（base case）**。 – 组合模式在真实系统中的例子：HTML 的 **DOM**（`<img>` / `<input>` 是基元，`<div>` / `<span>` 是组合，都实现共同的 `Element` 接口）；sp21 原文用 **Swing 视图树**（`JLabel` / `JTextField` 是基元，`JPanel` / `JScrollPane` 是组合，共同实现 `JComponent`）。 – 组合模式让"对整棵树做一件事"变成三行递归代码；反之，若没有统一类型，就必须到处写 `instanceof`。 – 组合结构必须是**有向无环**的（实际上这里是无环树），否则递归操作会无限循环。

表示的选择与"空"的表示（Choosing the Rep; Emptiness） – **定义与目的**：选定操作的规格之后就要选表示。音乐语言的表示由三个变体构成：`Note(duration, pitch, instrument)`、`Rest(duration)`、`Concat(first, second)`。空的概念必须有一个**表示**：课程明确拒绝使用 `null`，而选择用**时值为 0 的 `Rest`** 表示"空音乐"。它关系到 Safe from bugs（消除 `NullPointerException`）与 Easy to understand（"没有音乐"与"一段静音"语义一致）。 – **直观解释（"它是什么？"）**：`rest(0)` 就像"长度为零的空白乐段"：它是一段合法的、可以参与拼接的音乐，只是听不见；而 `null` 是"这里什么都没有，谁碰到谁崩溃"。 – **关键规则与最佳实践**： – **永远给"空"一个合法的表示**，绝不用 `null` 或 `undefined` / `null` 作为哨兵值（与 Reading 9 的"避免 null、快速失败"一致）。 – 选择 `Rest(0)` 的额外好处：`duration()`、`play()`、`concat()` 的实现**无需任何特例分支**，组合模式的一致性得以保持。若引入 `Empty` 变体，虽然也合法，但每个操作都要多写一个分支。 – 避免**表示依赖（representation dependence）**：除了变体类本身，客户端应通过工厂函数 `note(...)`、`rest(...)`、`concat(...)` 构造音乐，而不是直接 `new Concat(new Note(...), ...)`。 – 「补充说明」：`Concat` 的音乐树不是平衡的——`notes("C D E")` 会生成左深树 `Concat(Concat(Rest(0), Note(C)), Note(D))` 之类；因此要留意递归深度与栈开销，必要时可以引入更聪明的构造策略。

为递归类型定义操作：解释器模式（Interpreter Pattern） – **定义与目的**：自课程引入递归数据类型以来，我们一直用**解释器模式**为这类类型实现函数：（1）在定义数据类型的接口中**把操作声明为实例方法**；（2）在**每个具体变体类中实现**它。它关系到 Easy to understand（每个变体的代码都在一起）与 Safe from bugs（静态检查保证每个变体都实现了操作）。 – **直观解释（"它是什么？"）**：像给每个员工一句本岗位的作业指导书：`Note` 知道怎么算自己的时值、怎么被播放；`Concat` 知道怎么把两段合起来。客户端只说"给我你的时值"，具体由对象自己回答。 – **关键规则与最佳实践**： – **动态分派（dynamic dispatch）是解释器模式的动力**：`m.duration()` 执行哪个方法体，由 `m` 指向的对象**实际类型（actual type / dynamic type）**决定，而不是由变量声明的类型决定。 – 声明类型与实际类型的区别要牢记：Java 中"声明类型（declared type）"来自声明，编译期可知；"实际类型"是对象构造时所用的类。实际类型必须是声明类型的**子类型**，这保证了"声明类型上能调用的方法，实际类型一定也有（规格相同或更强）"。 – 接口没有构造器，因此**实际类型永远是

类；当声明类型是接口时，二者必然不同。 – 组合变体递归实现，基元变体实现基例；递归调用写在变体自己的方法体里，因此**编译器能检查每个变体都实现了新操作**。 – 由此产生的代价（本讲先埋下伏笔，Reading 27 展开）：操作代码**分散在所有变体类中**；新增一个操作必须修改接口与所有变体类。

操作放在哪里：实例方法 vs 独立工厂函数（Instance Methods vs Static Functions） – **定义与目的**：音乐语言的操作被分成两组：`duration : Music → double` 与 `play : Music × SequencePlayer × double → void` 作为 **`Music` 接口的实例方法**；而 `notes : String × Instrument → Music`（以及 `note` / `rest` / `concat`）作为 **`MusicLanguage` 类的静态工厂方法**。它关系到 Ready for change（把"构造"与"观察/变更"分开）与 Easy to understand（客户知道去哪里找什么）。 – **直观解释（"它是什么？"）**：实例方法是"这段音乐自己会做的事"（多久、怎么放）；静态工厂是"造音乐的工具箱"（从文本造、从音符造、从两段拼）。 – **关键规则与最佳实践**： – `notes` **可以**放在 `Music` 里，课程选择放在单独的 `MusicLanguage`，为的是让所有"操作 Music 的函数"集中在一处，等这门语言长大后更好找。 – 工厂函数（`note` / `rest` / `concat`）的价值是**避免表示依赖**：客户端不必知道 `Note` / `Rest` / `Concat` 的存在，将来换表示也不会牵动客户端代码（与 Reading 10/11 的抽象函数思想一致）。 – `concat` 是音乐语言的**第一个 producer 操作**（返回新 `Music` 而不改变参数），它使语言具备组合能力：少量原语 + 组合子 = 表达能力。 – 判断方法该放哪里的经验：**需要访问私有表示的观察/变更操作放实例方法；纯粹构造值的工厂放静态工具类**（「补充说明」：也可以用 Java 8 的静态接口方法，但课程采用了独立类）。 – 变体的构造函数、`checkRep`、`toString`、`equals` / `hashCode` 仍需在各变体类中仔细实现（与 Reading 15 相等性一致）。

求值器与求值环境（Evaluator and the Environment） – **定义与目的**：对"把代码当数据"的类型，最典型的操作是**求值（evaluate）**。课程的规格是 `evaluate : Formula × Map<String,Boolean> → boolean`，前置条件是"公式里出现的所有变量都必须是 map 的键"，效果是"用 map 中的值替换变量后求值"。它关系到 Safe from bugs（前置条件明确、未绑定变量显式失败）与 Ready for change（同一个表达式可以用不同环境反复求值）。 – **直观解释（"它是什么？"）**：环境（environment）就是"变量名 → 值"的字典，相当于把公式里的字母填上具体真值/数值；同一个公式换一本字典，就能得到不同结果——这正是"表达式是数据"的直接红利。 – **关键规则与最佳实践**： – **环境必须作为参数传递**（或作为不可变字段由构造函数注入），**绝不能做成全局可变状态**：否则同一表达式在不同线程、不同时刻求值结果不同，且无法并发使用。 – 未绑定变量是**前置条件违反**，应当显式失败并给出可读信息（例如抛 `IllegalArgumentException("unbound variable: x")`）；注意 `Map<String,Integer>` 取值后自动拆箱会在 `null` 时抛 `NullPointerException`，信息量很差。 – 求值器通常是**纯函数**：给定表达式与环境，结果唯一确定，且不修改环境——这让它可以被任意缓存、并行与测试。 – 「补充说明」：把 `Variable` 变体加入 `IntegerExpression` 是为了演示 `eval(environment)` 这一常见写法（教材中常称 `eval`）；课程原文的整数表达式文法（Reading 19）只有 `Number` 与 `Plus`，变量与环境的例子出现在 `Formula` 的 `evaluate` 上。 – 求值可以**递归地进行**：`Plus.eval(env)` 先递归求左右子表达式，再相加——组合模式的又一处体现。

把解析器与解释器串起来（Parser → AST → Interpreter） – **定义与目的**：语言要能被"写下来"，就需要**文本记号（notation）**与**解析器（parser）**。音乐语言选用**简化版 abc 记号**（文本音乐格式）：`C D E F G A B C' B A G F E D C` 是一个八度上下行的 C 大调音阶（`C` 是中央 C，`C'` 是高一个八度的 C，每个音是四

分音符）；`C/2 D/2 _E/2 F/2 G/2 _A/2 _B/2 C'/2` 是升序 c 小调音阶、速度快一倍（`E`、`A`、`B` 为降号，每个音是八分音符）。它关系到 Easy to understand（文本可读、可 diff）与 Safe from bugs（解析器可测试）。 – **直观解释（"它是什么？"）**：解析器是"翻译官"：把人类写的字符序列翻译成 AST；解释器/播放器是"演奏者"：让 AST 变成声音或结果。链路是 **文本 → 具体语法树 / 词法切分 → 抽象语法树 → 求值/播放**。 – **关键规则与最佳实践**： – `notes(String abc, Instrument instrument)` 的实现策略：先把输入切成一个个符号（如 `A,,/2`、`.1/2`），**从空音乐 `rest(0)` 开始**，逐个解析符号并用 `concat` 累积。 – `parseSymbol(String, Instrument)` 只负责解析**类型（休止符或音符）与时值**，把音高（字母、升降号、八度）交给 `parsePitch`。 – `parsePitch(String)` 是递归的：**基例**是"单个字母（可带升降号）"；**递归例**是"末尾的 `'`/`,` 表示升降八度"，以及"开头的 `^`/`_` 表示升/降半音"。要能答出原文练习里的问题：`C`、`_C` 走基例，而 `C'`、`C/2` 不是基例处理的内容。 – 解析器的结构应当**跟随文法结构**（与 Reading 19 的 `makeAbstractSyntaxTree` 完全同构：`SUM` 逐个 `Plus` 累积、`NUMBER` 用 `parseInt` 造基元）。 – 解析失败要**快速失败**并给出可读错误（`throw new IllegalArgumentException("bad abc symbol: ...")`），不要返回半成品。 – 课程原文的判断值得记住：写音乐用简化 abc 记号，比"一页又一页的 `addNote`"**更易理解、更少 bug、更易修改**——这就是小语言的价值。

播放器设计：为什么 `play` 要带 `atBeat`（Scheduling Instead of Waiting） – **定义与目的**：`play : Music × SequencePlayer × double → void` 的规格是"在给定的**拍延迟**之后，用给定播放器播放这段音乐"。为什么不"现在就播"？因为要播放由 `Concat` 组合起来的音符序列，就必须有时间轴；而播放器的 `addNote` **本来就设计成可以调度未来时刻的音符**（它自己处理延迟）。它关系到 Safe from bugs（不依赖 `sleep` 的时序猜测）与 Easy to understand（每段音乐只关心"我从第几拍开始"）。 – **直观解释（"它是什么？"）**：像给乐队写总谱：每个乐手被告知"你从第 16 拍开始演奏你的声部"，而不是让指挥拿着秒表逐个喊"现在轮到你"，更不是让人原地睡觉等轮到自己。 – **关键规则与最佳实践**： – `Note.play(player, atBeat)` 调用 `player.addNote(instrument, pitch, atBeat, duration)`——把**音符、开始时刻、时长**交给调度器。 – `Rest.play(player, atBeat)` 什么都不做：静音只贡献时长。 – `Concat.play(player, atBeat)` 先播第一段（`first.play(player, atBeat)`），再让第二段**从第一段结束处开始**：`second.play(player, atBeat + first.duration())`。这就是为什么必须有 `atBeat`：只有 `Concat` 知道该给子音乐传什么起始时刻。 – **不要在 `play` 里 `sleep`**：那会阻塞调用线程（在 GUI 事件线程或网络服务器线程里是灾难），而且时序精度依赖操作系统调度；调度器已经替你完成延迟。 – 把"具体播放器"（`MidiSequencePlayer`）与"音乐"解耦：`Music` 只依赖 `SequencePlayer` **接口**，因此可以用假播放器做测试；课程用 `MusicPlayer` 这个工具类把两者接起来。

小语言的设计原则：简单、可组合、可扩展（Simple, Composable, Extensible） – **定义与目的**：一个好的小语言应当只有**少量原语**、由**组合子**拼出无限表达，并且能在不破坏已有代码的前提下扩展。它关系到三大目标的全部。 – **直观解释（"它是什么？"）**：像乐高：只有几种基本块（原语），但靠"拼接"这一个组合子（`Concat`）就能搭出任何东西；要加"变速"功能时，加一个新块而不是重做所有块。 – **关键规则与最佳实践**： – 选择**少量、正交**的原语：`Note`、`Rest`，加一个组合子 `Concat` 就够了。 – 组合子要能**任意嵌套**（`Concat` 接受任意 `Music`，包括 `Concat` 自身），这是递归类型带来的表达力。 – 让操作**可扩展**：把操作声明在接口上（解释器模式）；如果预期客户端要加很多**新操作**，则考虑 Reading 27 的**访问者模式（Visitor**

pattern)。 – 用文本记号 + 解析器替代手写构造代码，让"写音乐"这件事本身变得安全、可读、可改。 – 注意 `Music` 的操作是**尽量纯**的：`duration()` 是观察者；`play` 有副作用（向播放器调度音符）；工厂是生产者/创造者——分清这些类别（Reading 6/7）有助于写出好规格。

代码示例与对比分析

场景 1：手写一长串 `addNote` vs 用简化 abc 记号构造音乐

❌ *错误代码*

```
// 错误：把音乐"硬编码"成几十次 addNote，时刻全靠手算
SequencePlayer player = new MidiSequencePlayer();
Instrument instrument = PIANO;

// Row, row, row your boat 的前几个音（C C C D E ...）
player.addNote(instrument, new Pitch('C'), 0.0, 1.0);
player.addNote(instrument, new Pitch('C'), 1.0, 1.0);
player.addNote(instrument, new Pitch('C'), 2.0, 1.0);
player.addNote(instrument, new Pitch('D'), 3.0, 1.0);
player.addNote(instrument, new Pitch('E'), 4.0, 2.0);
player.addNote(instrument, new Pitch('E'), 6.0, 1.0);
player.addNote(instrument, new Pitch('D'), 7.0, 1.0);
player.addNote(instrument, new Pitch('E'), 8.0, 1.0);
player.addNote(instrument, new Pitch('F'), 9.0, 1.0);
player.addNote(instrument, new Pitch('G'), 10.0, 4.0);
// … 还有 20 多个音符，每个都要手算起始拍；改一个音的位置就要重算后面所有时刻
player.play();
```

【错误代码的问题】 1. **时序靠手工累加**：任何一个音符的时值改动，后面所有音符的 `atBeat` 都要重算，改一处错一片（典型的"重复代码 + 手工一致"缺陷）。 2. **不可读**：从上到下看不出旋律，代码与音乐之间没有直观对应，违反 Easy to understand；也无法与其他音乐师交流。 3. **不可复用**：想把这段旋律移调、或在前面加一段前奏，必须重写所有行；没有任何"结构化"的操作空间。 4. **不可测试**：没有中间产物（AST）可供断言，只能听声音判断对错。

✅ *正确代码*

```
// 正确：用简化 abc 记号写成"程序"，解析成 Music（AST），再让 Music 自己播放
Music tune = notes("C C C D E E D E F G", PIANO);   // 文本可读、可 diff
double beats = tune.duration();                       // 观察者：总时长
tune.play(new MidiSequencePlayer(), 0.0);             // 从第 0 拍开始调度

// MusicLanguage 的核心实现（节选）
public final class MusicLanguage {
    private MusicLanguage() { }   // 工具类，不可实例化

    /** @param abc 简化 abc 记号写成的音乐字符串
     *  @param instrument 演奏这段音乐的乐器
     *  @return 与 abc 对应的 Music */
    public static Music notes(String abc, Instrument instrument) {
        Music music = rest(0);                         // 空音乐：时值为 0 的休止符
        for (String symbol : abc.trim().split("\\s+")) {
            if (symbol.equals("|")) continue;          // 小节线只是排版分隔符
            music = concat(music, parseSymbol(symbol, instrument));
        }
        return music;
    }

    public static Music note(double duration, Pitch pitch, Instrument instrument) {
        return new Note(duration, pitch, instrument);
    }

    public static Music rest(double duration) {
        return new Rest(duration);
    }

    public static Music concat(Music first, Music second) {
        return new Concat(first, second);
    }
    // parseSymbol / parsePitch 见场景 4
}
```

【为什么这样更好】 音乐变成**数据**：`notes(...)` 把文本一次解析成 AST，之后可以随时 `duration()`、`play()`、或把它们 `concat` 起来。时值只在文本里写一次，时刻由 `Concat.play` 递归计算，不再手工累加；想移调或加前奏，只需重新组合 AST。 **【代码对比解说】** 左侧代码是"解决方案的程序"，右侧是"解决问题的语言"。这一转变带来三点结构变化：（1）出现一个**中间表示**（`Music` 树），使计算与数据分离；（2）出现**工厂函数**把构造集中起来，客户端不依赖具体变体类；（3）出现**解释器**（`duration` / `play`）把"对 AST 求值"的实现按变体分散，使扩展变体成为局部修改。注意 `MusicLanguage` 的构造器是私有的：它只是静态方法的容器（「补充说明」：也可以用 `final class` + 私有构造或 Java 接口静态方法表达）。 **【设计原则透视】** 这是 Reading 10/11 的 ADT 设计（表示 + 操作 + 抽象边界）在"语言"尺度上的应用，也是 Reading 19 中"文法 → 解析器 → AST"链路的延续：`notes` 相当于针对音乐领域的 `makeAbstractSyntaxTree`。语法（abc 记号）与语义（AST）分离，正是"抽象语法 vs 具体语法"的教科书式体现。

场景 2：用 `null` 表示"空音乐" vs 用 `rest(0)`

❌ *错误代码*

```
// 错误：用 null 表示"没有音乐"，于是每个操作都要防御性地判空
public static Music concat(Music first, Music second) {
    if (first == null && second == null) return null;        // 特例 1
    if (first == null) return second;                        // 特例 2
    if (second == null) return first;                        // 特例 3
    return new Concat(first, second);
}

public final class Concat implements Music {
    private final Music first, second;
    @Override public double duration() {
        double d1 = (first == null) ? 0 : first.duration();   // 判空散落各处
        double d2 = (second == null) ? 0 : second.duration();
        return d1 + d2;
    }
    @Override public void play(SequencePlayer player, double atBeat) {
        if (first != null) first.play(player, atBeat);
        if (second != null) second.play(player, atBeat + (first == null ? 0 : first.duratio
    }
}
```

【错误代码的问题】 1. **`NullPointerException` 潜伏在每一处遗漏的判空上**：任何新操作（`transpose`、`reverse`）都必须记得重新写一遍判空逻辑，漏一处就是运行时崩溃。 2. **不变量无法表达**：`Concat` 的表示不变量本应是"两段都是合法的 `Music`"，用 `null` 后这个 RI 变成"可能为 null"，`checkRep` 也就无从写起。 3. **表示依赖外泄**：客户端会开始写 `if (m != null)` 这种防御代码，`null` 成了公开表示的一部分。 4. **语义混淆**："没有音乐"与"一段静音"被强行区分，客户必须理解这两种"空"的差异。

✅ *正确代码*

```
// 正确：空音乐是一个合法的 Music —— 时值为 0 的 Rest
public static Music rest(double duration) {
    return new Rest(duration);
}

// 空音乐：rest(0)；拼接时无需任何特例
public static Music concat(Music first, Music second) {
    return new Concat(first, second);
}

public final class Rest implements Music {
    private final double duration;
    public Rest(double duration) {
        this.duration = duration;
        checkRep();
    }
    private void checkRep() {
        assert duration >= 0 : "rest duration must be non-negative";
    }
    @Override public double duration() { return duration; }
    @Override public void play(SequencePlayer player, double atBeat) {
        // 静音不发出任何声音，只贡献时值
    }
}

public final class Concat implements Music {
    private final Music first, second;
    public Concat(Music first, Music second) {
        if (first == null || second == null) {
            throw new NullPointerException("Concat requires non-null Music");
        }
        this.first = first;
        this.second = second;
    }
    @Override public double duration() { return first.duration() + second.duration(); }
    @Override public void play(SequencePlayer player, double atBeat) {
        first.play(player, atBeat);
        second.play(player, atBeat + first.duration());
    }
}
```

【为什么这样更好】 表示不变量恢复为"`first`、`second` 都是非 null 的 `Music`"，`checkRep` 可以真正检查它；`duration()` 与 `play()` 的实现不再有特例分支，客户代码也不需要判空。空音乐 `rest(0)` 在语义上也更干净：它是"零拍的静音"，与"有拍的静音"属于同一概念。 **【代码对比解说】** 关键差别是"**用类型系统消灭非法状态**"还是"用运行时判断补救非法状态"。前者让编译器与 RI 帮你守住边界；后者把正确性寄托在"每个代码路径都记得判空"上，而人一定会忘。注意正确版本在构造函数里对 `null` **快速失败（fail fast）**：错误在构造点暴露，而不是在执行 `play` 五层递归之后才爆炸——这使定位成本从"追调用栈"降到"看栈顶"。 **【设计原则透视】** 对应 **Reading 9（避免调试：用断言与 fail fast，避免 null）** 与 **Reading 11（表示不变量）**：RI 必须能在 `checkRep` 中表达并被检查；任何让 RI 无法写清的表示（如用 `null` 当哨兵）都

是坏表示。同时这也是**组合模式**成立的前提：只有当"基元"与"组合"共享同一套良构不变量时，统一递归操作才可能简洁。

场景 3：把求值环境做成全局可变状态 vs 把环境作为参数传递

❌ *错误代码*

```
// 错误：环境是全局可变的静态表，求值器依赖隐藏状态
public interface IntegerExpression {
    /** @return 本表达式的值（从全局环境查变量） */
    int eval();
}

public final class Variable implements IntegerExpression {
    private final String name;
    public Variable(String name) { this.name = name; }

    @Override public int eval() {
        Integer value = GlobalEnvironment.get(name);   // 依赖全局状态
        return value;                                   // 未绑定时自动拆箱抛 NPE，信息极少
    }
}

public final class GlobalEnvironment {
    private static final Map<String, Integer> BINDINGS = new HashMap<>();
    public static void bind(String name, int value) { BINDINGS.put(name, value); }
    public static Integer get(String name) { return BINDINGS.get(name); }
}

// 客户端用法：必须先"设置好世界"，再求值
GlobalEnvironment.bind("x", 3);
GlobalEnvironment.bind("y", 4);
int a = new Plus(new Variable("x"), new Variable("y")).eval();   // 7
GlobalEnvironment.bind("x", 100);
int b = new Plus(new Variable("x"), new Variable("y")).eval();   // 104 —— 同一个表达式，结果变
```

【错误代码的问题】 1. **同一表达式在不同时刻求值结果不同**：表达式不再是"数据 + 求值规则"，而是"数据 + 隐式世界状态"，违反可理解性与可测试性。 2. **线程不安全**：多线程并发 `bind` / `get` 一个 `HashMap` 会导致数据损坏（参见 Reading 21/23）；即使换成并发容器，语义上的"全局共享变量"仍然是设计缺陷。 3. **无法同时用两套环境**：想比较"x=3 时"与"x=100 时"的结果，只能串行地反复改全局状态，无法并行、无法重放。 4. **错误信息差**：未绑定变量表现为 `NullPointerException`（自动拆箱），调用者完全不知道是哪个变量没绑定。

✅ *正确代码*

```
// 正确：环境是显式的、不可被求值器修改的输入
public interface IntegerExpression {
    /** @param environment 变量名到值的映射；
     *         前置条件：包含本表达式出现的所有变量名（见 variables()）
     *  @return 本表达式在 environment 下的值 */
    int eval(Map<String, Integer> environment);

    /** @return 本表达式出现的所有变量名 */
    Set<String> variables();
}

public final class Variable implements IntegerExpression {
    private final String name;
    public Variable(String name) {
        if (name == null || name.isEmpty()) {
            throw new IllegalArgumentException("variable name must be non-empty");
        }
        this.name = name;
    }
    public String name() { return name; }

    @Override public int eval(Map<String, Integer> environment) {
        Integer value = environment.get(name);
        if (value == null) {
            throw new IllegalArgumentException("unbound variable: " + name);
        }
        return value;
    }
    @Override public Set<String> variables() { return Set.of(name); }
    @Override public String toString() { return name; }
}

public final class Constant implements IntegerExpression {
    private final int value;
    public Constant(int value) { this.value = value; }
    @Override public int eval(Map<String, Integer> environment) { return value; }
    @Override public Set<String> variables() { return Set.of(); }
    @Override public String toString() { return Integer.toString(value); }
}

public final class Plus implements IntegerExpression {
    private final IntegerExpression left, right;
    public Plus(IntegerExpression left, IntegerExpression right) {
        this.left = left; this.right = right;
    }
    public IntegerExpression left() { return left; }
    public IntegerExpression right() { return right; }

    @Override public int eval(Map<String, Integer> environment) {
        return left.eval(environment) + right.eval(environment);   // 递归、纯函数
    }
    @Override public Set<String> variables() {
        Set<String> result = new HashSet<>(left.variables());
```

```
        result.addAll(right.variables());
        return Set.copyOf(result);                                  // 返回不可变集合
    }
    @Override public String toString() { return "(" + left + " + " + right + ")"; }
}

// 客户端用法：环境由调用者拥有，表达式可被反复、并行地求值
IntegerExpression e = new Plus(new Variable("x"), new Variable("y"));
Map<String, Integer> env1 = Map.of("x", 3, "y", 4);
Map<String, Integer> env2 = Map.of("x", 100, "y", 4);
int r1 = e.eval(env1);   // 7
int r2 = e.eval(env2);   // 104 —— 同一个表达式对象，两次求值互不影响
```

【为什么这样更好】 求值器成为**纯函数**：给定（表达式，环境）唯一确定结果，不修改任何状态，因此可并发、可缓存、可重放、可单元测试。变量的缺失变成**明确的前置条件违反**，错误信息带上变量名。`variables()` 让前置条件"环境必须覆盖所有变量"可以被**程序化检查**（`environment.keySet().containsAll(e.variables())`），而不是只写在注释里。 **【代码对比解说】** 两版的核心差别是"**依赖是隐式的还是显式的**"。全局环境把依赖藏进静态状态，调用点看起来像一个参数都没有的无参方法 `eval()`，但真实依赖却无处不在（这叫隐藏耦合）；显式参数则把依赖写在签名里，读者一眼就能看出"求值需要环境"。此外，正确版本还展示了 ADT 设计的两条细节：(1) `Map` 参数应被当作只读输入，求值器绝不 `put`；(2) `variables()` 返回不可变集合，避免把内部表示暴露给客户端改动。 **【设计原则透视】** 这是 **Reading 8（可变性与不可变性）**、**Reading 21/23（并发与互斥）** 与 **Reading 6/7（规格）** 的交汇：可变静态状态是最糟糕的选择（全局可见 + 线程不安全）；把状态提升为参数，就把"并发问题"从根上取消了。`eval` 的规格把前置条件显式写出，正符合 Reading 7 中"前置条件越弱越好、但要写清"的原则；而 `variables()` 则是一种"可检查的前置条件"，让调用者能在运行前自查。

场景 4：手工切割音符字符串 vs 递归的 `parsePitch` + 结构化解析

❌ *错误代码*

```
// 错误：靠下标与特判"猜"记号结构，只支持一种八度与一种时值写法
private static Pitch parsePitch(String pitchText) {
    char letter = pitchText.charAt(0);          // "_E" 会被当成字母 '_'，直接出错
    if (pitchText.length() > 1 && pitchText.charAt(1) == '\'') {
        return new Pitch(letter).transpose(12); // 只能处理一个上八度，"C'" 就错
    }
    return new Pitch(letter);
}

private static Music parseSymbol(String symbol, Instrument instrument) {
    double duration = 1.0;
    if (symbol.length() > 2 && symbol.charAt(1) == '/') {
        duration = 1.0 / Double.parseDouble(symbol.substring(2));  // "2/4"、"C/2" 等等全乱
    }
    return new Note(duration, parsePitch(symbol.substring(0, 1)), instrument);
}
```

【错误代码的问题】 1. **混淆了"音高"与"时值"的解析**：`parseSymbol` 用位置下标假设"第 1 个字符是音高、后面是时值"，于是一旦记号变成 `_E/2` 或 `.1/2`，切分位置就错了。 2. **递归结构被写死成一层**：`C''`（两个八度）、`C,,`（低两个八度）都会解析错误，而这些在 abc 记号里完全合法。 3. **失败方式糟糕**：越界或 `NumberFormatException` 让调用者不知道是哪个符号有问题，违反 Easy to understand。 4. 与文法脱节：文法里 `pitch ::= accidental? letter octave*` 是递归的，代码却写成了非递归的字符扫描，二者结构不一致，将来改文法必然改错代码。

✅ *正确代码*

```
// 正确：解析器结构跟随文法，音高递归处理八度与升降号，时值单独处理
// 文法（本笔记采用的简化版）：
//   symbol     ::= rest | note
//   note       ::= pitch duration?
//   pitch      ::= accidental? letter octave*
//   accidental ::= '^' | '_' | '='
//   octave     ::= '\'' | ','
//   duration   ::= (digit+)? ( '/' digit+ )?

private static final Pattern SYMBOL_PATTERN =
        Pattern.compile("([.^_=A-Ga-g',]*)([0-9]*)(?:/([0-9]+))?");

/** @param symbol 单个 abc 符号，例如 "C"、"_E/2"、".1/2"、"C'"
 *  @return 对应的 Music（Rest 或 Note） */
private static Music parseSymbol(String symbol, Instrument instrument) {
    Matcher m = SYMBOL_PATTERN.matcher(symbol);
    if (!m.matches()) {
        throw new IllegalArgumentException("bad abc symbol: " + symbol);
    }
    String pitchOrRest = m.group(1);       // 音高部分或 "."
    String numerator   = m.group(2);       // 时值分子，可为空
    String denominator = m.group(3);       // 时值分母，可为 null

    double duration = 1.0;                 // 默认四分音符
    if (!numerator.isEmpty()) {
        duration *= Double.parseDouble(numerator);
    }
    if (denominator != null && !denominator.isEmpty()) {
        duration /= Double.parseDouble(denominator);
    }

    if (pitchOrRest.equals(".")) {
        return rest(duration);
    }
    return note(duration, parsePitch(pitchOrRest), instrument);
}

/** @param pitchText 形如 "C"、"_E"、"^F"、"C'"、"C,," 的音高文本（不含时值）
 *  @return 对应的 Pitch */
private static Pitch parsePitch(String pitchText) {
    if (pitchText.isEmpty()) {
        throw new IllegalArgumentException("empty pitch");
    }
    // 基例：只剩一个字母，没有升降号也没有八度记号
    if (pitchText.length() == 1 && Character.isLetter(pitchText.charAt(0))) {
        return new Pitch(pitchText.charAt(0));
    }
    // 递归例 1：末尾 ' 表示升高一个八度
    if (pitchText.endsWith("'")) {
        return parsePitch(pitchText.substring(0, pitchText.length() - 1)).transpose(12);
    }
    // 递归例 2：末尾 , 表示降低一个八度
    if (pitchText.endsWith(",")) {
```

```
            return parsePitch(pitchText.substring(0, pitchText.length() - 1)).transpose(-12);
        }
        // 递归例 3：开头的升降号
        switch (pitchText.charAt(0)) {
            case '^': return parsePitch(pitchText.substring(1)).transpose(1);    // 升半音
            case '_': return parsePitch(pitchText.substring(1)).transpose(-1);   // 降半音
            case '=': return parsePitch(pitchText.substring(1));                 // 还原号
            default:
                throw new IllegalArgumentException("bad pitch: " + pitchText);
        }
    }
```

【为什么这样更好】 代码结构与文法结构**同构**：`parseSymbol` 负责"类型 + 时值"，`parsePitch` 用递归同时处理任意多个八度记号与升降号，`C''`、`C,,`、`^F'` 都自然成立。每个失败点都抛出带原文的异常，非法输入立即暴露。时值与音高分离，使将来新增记号（如附点、连音）时改动范围可预测。 **【代码对比解说】** 左侧代码把"解析"降级成"按下标取字符"，必须为每种写法特判；右侧代码把"解析"还原为"按文法递归下降"，特判只出现在**文法真正分支的地方**（`.` 是休止符、末尾是 `'`/`,`、开头是升降号）。注意基例与递归例的划分：基例是"单个字母"，递归例每次**剥掉一个修饰符**再递归，因此必然终止——这与 Reading 14（递归）中"递归必须向基例前进"的规则完全一致。此外，`parsePitch` 依赖 `Pitch.transpose(int)` 这一 **producer 操作**（返回新的 `Pitch`，不修改原对象），体现了不可变类型在解析器中的好用之处。 **【设计原则透视】** 这里同时用到 **Reading 18（正则与文法）**、**Reading 19（解析器）** 与 **Reading 14（递归）**：文法既是协议的规格，也是解析器的实现蓝图；`parseSymbol`/`parsePitch` 的分层对应"具体语法 → 抽象语法"的转换，与 Reading 19 中 `makeAbstractSyntaxTree` 按 `EXPR`/`SUM`/`PRIMARY`/`NUMBER` 逐条规则处理是同一手法。解析器本身也应当被单独规格化：前置条件是"输入符合文法"，后置条件是"返回与输入对应的 AST"，不在前置条件内的输入必须快速失败。

与其他设计原则的关联

- **Reading 17（递归数据类型）**：本讲的全部数据表示（`Formula`、`Music`、`IntegerExpression`）都是递归数据类型；本讲在它之上加了"语言"的视角——递归类型即 AST，变体即语法产生式。
- **Reading 18（正则与文法）与 Reading 19（解析器）**：abc 记号、HTTP 请求行、问题集语言都需要文法；`notes`/`parseSymbol`/`parsePitch` 是 Music 领域的 `makeAbstractSyntaxTree`，把**具体语法树**转成**抽象语法树**。外部 DSL（正则、ParserLib 文法、PS3 语言）与本讲的内部 DSL 形成对照。
- **Reading 10（抽象数据类型）与 Reading 11（抽象函数与表示不变量）**：选择 `Music` 的表示、写 `checkRep`、用工厂函数避免表示依赖，都是 ADT 设计的基本功；`rest(0)` 表示空音乐是"让 RI 可表达"的范例。
- **Reading 12（接口、泛型、枚举、函数对象）**：`Music`/`SequencePlayer` 是接口，`Instrument` 是枚举，`MusicLanguage` 是静态工厂集合；函数对象（`BiFunction`、lambda）让我们能把"计算"也当作值传递，是内部 DSL 的支柱。
- **Reading 8（可变性与不可变性）**：AST 节点必须是不可变的，才能被安全地共享与多次求值；`Pitch.transpose` 返回新对象而非修改自身（producer 而非 mutator）。

- **Reading 6（规格说明）与 Reading 7（设计规格）**：`notes`、`duration`、`play`、`eval` 都需要前置/后置条件；`play` 的 `atBeat` 参数正是为了让规格写清"从哪里开始"，避免把时间耦合进实现。
- **Reading 9（避免调试）与 Reading 15（相等性）**：解析失败要 fail fast；变体类需要正确实现 `equals` / `hashCode` / `toString`，否则音乐无法比较与调试。
- **Reading 3（测试）与 Reading 4（代码评审）**：解析器与求值器都是纯函数，最容易测试；`Music` 只依赖 `SequencePlayer` 接口，可以用假播放器断言 `addNote` 的调用序列。
- **Reading 27（小语言 II）**：本讲用**解释器模式**实现操作；下一讲指出它的两个缺点（代码分散、加新操作要改所有变体），引入**访问者模式**作为替代，并讨论表达式问题（expression problem）。
- **Problem Set 3（Memely）**：该问题集要求实现一门生成图片 meme 的语言（`|` 水平拼接、`---` 垂直拼接），正是"递归数据类型 + 解析器 + 解释器"的完整练习，与本讲的音乐语言同构。

关键要点

- **先问"能不能做成一门语言"，并把代码当数据**：如果任务是一整类相关问题，就构造语言（递归数据类型 + 少量原语 + 组合子）而不是写一次性程序；表达式应当是可以存储、传递、延后求值、重复求值的**一等值**，这要求 AST 节点**不可变**。
- **递归类型 + 解释器模式**：操作声明在接口、实现在每个变体；组合变体递归、基元变体实现基例；动态分派决定执行哪段代码。
- **"空"必须有合法表示**：用 `rest(0)` 而不是 `null`；让表示不变量可写、可检查，让 `checkRep` 有用。
- **显式依赖、纯求值**：环境/播放器都应作为参数或构造注入，绝不做成全局可变状态；未绑定变量要显式失败。
- **解析器跟随文法，播放交给调度**：递归下降解析文本 → AST；`play(atBeat)` 把时间轴交给播放器调度，绝不在 `play` 里 `sleep`。

常见陷阱与注意事项

1. **把 AST 写成可变对象** → 表达式被共享后，某处修改会悄悄影响其他地方（别名 bug）；且无法安全地并发求值。应让所有字段 `final` 并做防御性复制。
2. **用 `null` 表示空音乐，或忘记 `checkRep`** → 前者让表示不变量无法表达、到处需要判空、`NullPointerException` 随机出现（应改用 `rest(0)` 或专门的空变体）；后者让非法状态（负时值、`null` 音高）流入系统，错误在很远的地方才暴露。
3. **把求值环境做成全局可变状态** → 同一表达式结果随外部状态变化，线程不安全，无法并行或重放；应作为参数或构造注入的不可变字段。
4. **在 `play` 里调用 `Thread.sleep` 或立即逐音播放** → 阻塞调用线程、时序不准、`Concat` 的两段会同时发声；应通过 `atBeat` 交给播放器调度。
5. **解析器用下标/特判代替递归** → `C''`、`C,,`、`_E/2`、`.1/2` 等合法记号解析错误，且改文法必然改错代码；应让代码结构与文法同构，并快速失败。

6. **客户端直接 `new Note(...)` / `new Concat(...)`** → 表示依赖：换表示就要改客户端。应统一走 `note` / `rest` / `concat` / `notes` 工厂。

思考题（带答案）

问题 1：为什么 `Music` 需要 `play(SequencePlayer player, double atBeat)` 这样的签名，而不写成 `play()`（现在就开始播）？请结合 `Concat` 的递归实现说明，并解释为什么这比"在 `play` 里 `Thread.sleep` 等一拍"更好。

答案：因为要在**时间轴**上组合音乐。`Concat(first, second)` 的语义是"先 `first` 后 `second`"，如果 `play` 都从"现在"开始，两段音乐会被安排在同一个时刻，听起来是同时发声，语义被破坏。加入 `atBeat` 之后，`Concat.play(player, atBeat)` 写成 `first.play(player, atBeat); second.play(player, atBeat + first.duration());` ——**每个组合变体负责把正确的起始时刻传给子音乐**，这就是"为什么必须有这个参数"的根本理由：只有 `Concat` 知道自己第一段的时长。底层的 `SequencePlayer.addNote(instrument, pitch, atBeat, duration)` 本来就能把音符调度到未来的某一拍，因此 `play` 完全不需要等待。相比之下，用 `Thread.sleep` 模拟"等一拍"有三个致命问题：(1) 它阻塞调用线程（若在 GUI 事件线程或网络服务器线程中调用，会冻结界面或拖垮服务器）；(2) 时序精度依赖操作系统的调度与计时器，抖动明显；(3) 它让"播放"变成一段**串行等待的过程**，无法整体调度、无法中途取消、也无法用假播放器在测试中断言调用序列。把时间交给调度器，也让 `Music` 只依赖 `SequencePlayer` 接口，从而保持可替换性与可测试性。

问题 2：本讲说"把代码表示为数据"是语言设计的关键。请以 `Formula`（或 `IntegerExpression`）为例，说明"数据"相比"直接写表达式"多出了哪些能力，并指出这些能力分别对应三大目标中的哪一个。

答案：直接写 `p && q` 时，表达式在**遇到它的那一刻**就被求值，结果只剩一个布尔值，表达式本身消失了。改成数据 `And(Variable("p"), Variable("q"))` 后，额外获得四种能力：(1) **存储与传递**——AST 可以放进字段、集合、文件、网络消息，也可以跨线程传递，这支持 Ready for change（同一份表达式能被不同模块复用）；(2) **延后求值**——可以先构造、稍后再 `eval(environment)`，这在"先收集条件、稍后判断"的场景里必不可少，也支持 Easy to understand（构造与求值是两件清晰的事）；(3) **多次求值与不同环境求值**——同一个表达式可用不同的 `Map` 求值任意次，这是"配置化/参数化"的基础，支持 Ready for change 与 Safe from bugs（无需重建表达式，减少出错机会）；(4) **分析与变换**——可以遍历它、统计变量（`variables()`）、化简、转成合取范式、打印成字符串，这就是"编译器/优化器"的能力，支持 Ready for change。相反，直接写表达式得到的只是**一个结果**，无法再做任何结构上的处理。需要强调的是，这些能力的前提是 AST 节点**不可变**：如果 AST 可以被修改，"同一份表达式"这个概念就不存在了，共享与并发也就无从谈起。

问题 3：为什么课程选择用 `Rest(duration: 0)` 而不是新增一个 `Empty` 变体来表示"空音乐"？请从"操作的实现复杂度""表示不变量"和"未来扩展"三个角度分析，并说明如果一定要用 `Empty` 变体，代价是什么。

答案：用 `rest(0)` 的好处有三点。第一，**操作实现无需特例**：`duration()` 就是返回 0，`play()` 什么都不做，`Concat` 照常递归—— `Rest` 已经把所有需要的行为定义好了；新增 `Empty` 则要在每个操作里多写一个分支，分支越多越容易漏（`Empty.duration()` 忘了改、`Empty.play()` 抛异常等等）。第二，**表示不变量保持统一**：`Rest` 的 RI 只是"时值非负"，`rest(0)` 天然满足；若引入 `Empty`，就会出现"有的音乐有时值字段、有的没有"的不一致，`Concat` 也必须额外规定"`Empty` 不能出现在某个位置"之类的人为约束。第

三，**将来扩展更自然**：一旦语言要支持"变速""重复""静音插入"等操作，`rest(0)` 与其他 `Rest` 一同处理，语义连续；`Empty` 则往往需要与 `Rest` 之间做转换，增加转换代码与出错机会。如果一定要用 `Empty`，代价是：每个操作多一个分支（解释器模式下分散在所有变体类里）、每个新操作都要记得处理它、客户端可能需要区分"空"与"零时长静音"两种语义上等价的状态，从而失去组合模式的简洁性。课程原文的立场很清楚："永远应该有一个表示'什么都没有'的值，而我们当然不会用 `null` 或 `undefined`"——`rest(0)` 就是那个既合法又省事的表示。

Reading 27: 小语言 II（Little Languages II）

> 说明：本讲 sp22 原版使用 TypeScript（`FormulaVisitor<R>` 接口、`callFunction` / `accept` 方法、`onVariable` / `onNot` 等命名），本笔记按用户要求提供 Java 代码示例；类型/API 与 sp21（6.031 Java 版，Reading 28: Little Languages II）原文保持一致（sp21 把访问者接口嵌套在 `Formula` 内，并用**重载**的 `on(...)` 方法表示各变体分支）。凡属为演示目的补充的内容（如 Java 8 `default` 方法、`makeVisitor` 工厂的等价写法）均明确标注为「补充说明」。

概述

本讲介绍**访问者模式（Visitor pattern）**：它是"把函数当作一等值"的又一个例子，也是**在递归数据类型上实现操作的另一种策略**。课程的动机很直接：为递归类型实现操作时，我们一直使用**解释器模式（Interpreter pattern）**——在接口上声明实例方法、在每个变体类中实现；但它有两个缺点：一是操作的代码分散在所有变体类里（难读、难改、难找 bug），二是新增一个操作必须修改接口与每一个变体类。访问者模式通过**双重分派（double dispatch）**让"函数"本身成为一个可以传递、储存、复用的对象，从而把代码**按操作分组**。它与三大目标的关系是：Safe from bugs（不再需要 `instanceof` 与强制转型，静态检查重新回到我们这边）、Easy to understand（一个操作的全部代码集中在一个类中）、Ready for change（新增操作无需触碰既有类型；代价是新增变体时要改动访问者接口，这就是**表达式问题**）。

核心概念与设计原则详解

回顾：解释器模式（Interpreter Pattern） – **定义与目的**：自课程引入递归数据类型以来，我们一直用这种方式实现函数：(1) 在定义数据类型的接口中**把操作声明为实例方法**；(2) 在**每个具体变体类中实现该操作**。它关系到 Easy to understand（每个变体的相关代码都在一起）与 Safe from bugs（编译器保证每个变体都实现了操作）。 – **直观解释（"它是什么？"）**：每个变体"自己会做这件事"。你问一段音乐"你多长？"，`Note`、`Rest`、`Concat` 各自按自己的方式回答。 – **关键规则与最佳实践**： – 用组合模式（Composite pattern）的术语说：**组合变体**（`Concat`、`Not`、`And`、`Or`）**递归**实现操作；**基元变体**（`Note`、`Rest`、`Variable`）实现**基例**。 – 例如 `Formula` 上的 `variables` 操作：`Variable` 返回 `Set.of(name)` 这一基例；`And` / `Or` 返回 `setUnion(left.variables(), right.variables())`；`Not` 返回 `formula.variables()`。 – 所有操作在接口上可见，客户端只需 `f.variables()`，无需知道 `f` 的实际类型。 – 这是 Reading 26 建立的默认做法；本讲要评估它的代价，并给出替代方案。

动态分派与静态检查（Dynamic Dispatch vs Static Checking） – **定义与目的**：理解访问者模式必须先分清两件事。**动态分派（dynamic dispatch）**：方法调用在**运行时**执行"对象**值**的**实际类型（actual type / dynamic type）**"中的那个实现。**静态检查（static checking）**：在程序**运行之前**，编译器检查方法是否存在、实参类型是否兼容——依据的是变量/表达式的**声明类型（declared type）**。它关系到 Safe from bugs（静态检查是最便宜的 bug 防线）与 Easy to understand（读者能预期调用落到哪里）。 – **直观解释（"它是什么？"）**：静态检查像"出门前的行李清单"（编译期核对）；动态分派像"到达后由现场的门卫决定谁接待你"（运行期按实际身份找对应的人）。 – **关键规则与最佳实践**： – 声明类型来自代码里的声明，编译期已知；实际类型是对象构造时使用的类，运行期才有。 – 当声明类型是接口时，**实际类型永远与声明类型不同**，因为接口没有构造器，只有类才能产生对象值。 – 实际类型必须是声明类型的**子类型**：这保证了"声明类型上能调用的方法，实际类型上一定存在，且规格相同或更强"。Java 用静态检查强制这一点（`Formula f = "not";` 无法编译，因为 `String` 不是 `Formula` 的子类型）。 – **动态分派正是解释器模式的动力**：`f.variables()` 具体跑哪段代码，只看 `f` 指向什么对象。 – 记住这一对概念是理解双重分派的关键：访问者模式**连续使用两次动态分派**，从而在不做任何类型测试的前提下选到正确的实现。

解释器模式的两个缺点（Downsides of the Interpreter Pattern） – **定义与目的**：课程明确列出两点代价，它们是访问者模式存在的理由。它关系到 Easy to understand（代码分散）与 Ready for change（加操作要动所有变体）。 – **直观解释（"它是什么？"）**：想象一张表格，**列是变体**（`Variable`、`And`、`Or`、`Not`），**行是操作**（`variables`、`evaluate`、`conjunctiveNormalForm`...）。解释器模式把代码**按列收集**：每个变体类里有该列的全部格子。 – **关键规则与最佳实践**： – 缺点一：**代码分散**。一个复杂操作的实现散布在所有变体类中；要通读、重构或修 bug，必须跑到各个类里分别改——更难理解，递归实现里的 bug 更容易引入、更难发现。 – 缺点二：**加操作是破坏性改动**。新增操作要改接口 + 每个实现类；如果变体很多、其中一些是别人写的或已发布，这个改动远比"把新操作的代码放在自己一处"困难。 – 结论不是"解释器模式不好"，而是"**它有适用场景**"：当变体经常增加而操作稳定时，解释器模式更合适。

模式匹配：我们真正想写的东西（Pattern Matching） – **定义与目的**：在 Reading 17 的练习里，我们写的是**按等式定义的函数**：`variables(Variable(x)) = setAdd(x, emptySet())`、`variables(Not(f)) = variables(f)`、`variables(And(f1,f2)) = setUnion(variables(f1), variables(f2))`.....大家自然想把它直译成 `switch`：按对象的"形状"分派。它关系到 Easy to understand（一处写完一个操作的所有情况）。 – **直观解释（"它是什么？"）**：像填空题：每种"公式的形状"对应一行答案，写在一起一目了然。许多语言（函数式语言、Scala、C# 的模式匹配、乃至新版 Java）原生支持这种写法。 – **关键规则与最佳实践**： – Java 的 `switch` **不能**按对象类型分派；若硬要用 `if (f instanceof ...)` 加强制转型来模拟，就得到课程原文所说的"**可怕的野兽（terrible beast）**"：静态检查被彻底抛弃，编译器不再检查我们的工作，因此**不安全**。 – 这种写法的典型形态：一串 `instanceof` 分支 + 逐分支 cast + 最后 `else throw new IllegalArgumentException("don't know what " + f + " is")`。 – 原文的两个练习揭示了它的两种失败模式：**去掉最后的 `else`** 会得到"可能的运行时错误"或"静默的错误答案"（取决于控制流）；**新增一个变体（如 `Xor`、`Literal`）** 时，代码既不会编译报错，也不会必然立刻失败——最坏情况是悄悄返回错误结果。这就是"不 safe from bugs"。 – 只有**子类型集合固定**时（如枚举、判别联合，课程在 Worker 消息传递中用判别联合做过）才可能安全地这样写；对开放的类层次结构不安全。

把函数表示为数据（Representing the Function as Data） – **定义与目的**：既然不能按形状 `switch`，就借用 Reading 26 的大思想：**把函数表示为数据**。我们为"作用于 `Formula` 的函数"专门定义一个类型，它的每个方法对应一个变体分支。它关系到 Ready for change（操作成为可传递的一等值）与 Easy to understand（每个操作集中成一处）。 – **直观解释（"它是什么？"）**：把"一个操作"从"散落在各处的代码"变成"一个对象"：这个对象带来一套方法，分别回答"如果遇到 `Variable` 怎么办""如果遇到 `And` 怎么办"。 – **关键规则与最佳实践**： – 先试 `Function<Formula, Set<String>>` 这类通用函数类型是**没用的**：lambda 体里只知道 `f` 是 `Formula`，不足以写出基例与递归例（这正是原文明说的"did that help? 没有"）。 – 因此要**自定义一个类型**，为每个变体提供一个方法：`R on(Variable)`、`R on(Not)`、`R on(And)`、`R on(Or)`（sp21 用重载 `on`；sp22 用 `onVariable` / `onNot` / `onAnd` / `onOr`；两者等价，重载版更简洁，分别命名版更利于 lambda 与文档）。 – 用泛型参数 `<R>` 表示"函数的结果类型"：`Visitor<Set<String>>` 是求变量集合的函数，`Visitor<Boolean>` 是求值的函数，`Visitor<Integer>` 是求深度/结点数的函数。 – 访问者对象本身就是一个**函数对象（functional object）**：可以储存、传递、返回、放进集合（与 Reading 20 的回调思想一致）。 – 还需要一个"入口"：客户端手里只有 `Formula`，怎么把访问者交给它？答案就是下一节的**双重分派**。

双重分派（Double Dispatch） – **定义与目的**：**双重分派 = 两次连续的方法调用**：第一次用动态分派到达具体变体（`f.accept(visitor)` 落到 `And.accept`）；变体随即发起第二次调用，同样用动态分派落在**表示该函数的对象**上（`visitor.on(this)` 落到 `VariablesInFormula.on(And)`）。它解决了"如何在不知道具体变体类型的情况下调用正确的分支"这一核心难题。 – **直观解释（"它是什么？"）**：像"双人确认"：你先找窗口（第一次分派，确定你是哪类业务），窗口再把你的材料转交给专门的处理人（第二次分派，确定谁来处理）。 – **关键规则与最佳实践**： – 第一次调用是**给客户端的入口**，把函数作为参数传进去：`public <R> R accept(Visitor<R> visitor);` – **每个具体变体负责把自己的实际类型"交给"访问者**：`Variable.accept` 写 `return visitor.on(this);`，`And.accept` 也写 `return visitor.on(this);`——由于 `this` 的静态类型在各类中不同，重载解析会选中正确的 `on` 重载；而 `visitor` 的实际类型由动态分派决定。 – 递归发生在**访问者一侧**：`onAnd` 里写 `and.left().accept(this)`，把"自己"（`this`）继续传下去。 – 这套机制让 `accept` 的实现**永远只有一行**，而所有"每种变体该怎么做"的知识都集中在访问者里。 – 「补充说明」：`accept` 的泛型方法 `<R> R accept(Visitor<R> visitor)` 在 Java 中需要**在接口与每个实现类上都声明类型参数**（`@Override public <R> R accept(Visitor<R> visitor)`），这是 Java 泛型的写法要求，容易漏写导致编译错误。

`Visitor<R>` 接口与 `accept` 方法（术语与最终形态） – **定义与目的**：把上面的机制正式命名为课程使用的形态：接口叫 `Visitor<R>`（原文中先叫 `FormulaFunction<R>`），入口方法叫 `accept`（原文中先叫 `callFunction`）。它关系到 Easy to understand（名字符合社区惯例）。 – **直观解释（"它是什么？"）**：**accept = 接受一次访问**：公式"接待"一位访问者，并让访问者按自己的类型处理自己。 – **关键规则与最佳实践**： – 命名演变：`FormulaFunction<R>` → `Formula.Visitor<R>`；`callFunction` → `accept`；`onVariable/onNot/onAnd/onOr` → sp21 用一个**重载**的 `on`。 – 使用方式极简：`Set<String> vars = f.accept(new VariablesInFormula());` – 访问者接口放在**数据类型内部**（`Formula.Visitor<R>`，sp21 做法）还是外部（`FormulaVisitor<R>`，sp22 做法）都可以；嵌套的好处是名字空间清晰、强调"这是

Formula 的配套接口"。 – 访问者接口是**数据类型契约的一部分**：`accept` 的规格要写清"`visitor` 非 null，返回值等于把该访问者应用于 `this`"。 – 由于每个变体的 `accept` 只调用 `visitor.on(this)`，**变体类本身仍然只有表示 + 少量操作**，不需要为每个新操作增加方法。

访问者即"类型上的 switch"，也是迭代器（Visitor as Switch and Iterator） – **定义与目的**：课程给出访问者的三重身份：(1) 它实现了**对类型的 switch**：`VariablesInFormula` 读起来非常接近我们最初想写的 `switch`，只是每个 case 变成了自己的 `on` 方法；(2) 它**表示一个递归类型上的函数**——可以创建实例、传递、按需应用；(3) 它**像迭代器（iterator）**一样沿着树的结点逐个走一遍并处理，就像遍历列表或集合。它关系到 Easy to understand 与 Ready for change。 – **直观解释（"它是什么？"）**：访问者像一位"上门的审计员"：数据树的每个结点都开门让审计员进来（`accept`），审计员在每类结点上执行自己的检查规则。 – **关键规则与最佳实践**： – 函数需要**额外参数**时（如 `evaluate : Formula × Map<String,Boolean> → boolean`），把参数**交给访问者的构造函数**并保存在 `final` 字段里，供整个遍历使用——这就是"带参数的访问者"。 – 遍历**顺序由访问者自己决定**：先序/后序、是否短路（`&&` / `||` 的短路求值）、是否需要剪枝，都写在 `on` 方法里；这是访问者比"固定迭代器"更强的地方。 – 「补充说明」：Java 8 可以用**访问者工厂 + 函数对象**让写法接近 `switch`：`makeVisitor(Function<Variable,R> onVariable, Function<Not,R> onNot, Function<And,R> onAnd, Function<Or,R> onOr)` 返回一个匿名 `Visitor` 实现，于是可以写 `formula.accept(makeVisitor(var -> map.get(var.name()), not -> ..., ...))`。这是 sp21 原文的做法，代价是"编译期不再强制你为每个变体提供分支"（改用匿名内部类逐方法实现时才会强制）。 – 更现代的语言特性（判别联合、模式匹配、sealed 类型）能直接表达这种 `switch`；Java 中访问者模式就是**用对象与双重分派实现类型安全的多分支**。

为什么用访问者：码表视角与表达式问题（Why Visitor? The Expression Problem） – **定义与目的**：把 `Formula` 的所有操作想象成一张**码表**：列是变体（`Variable`、`And`、`Or`、`Not`），行是操作（`variables`、`evaluate`、`conjunctiveNormalForm`、...），每格是"该操作在该变体上的实现"。**解释器模式按列组织代码，访问者模式按行组织代码**。它关系到 Ready for change：你更想为"加操作"还是"加变体"做好准备？ – **直观解释（"它是什么？"）**：同样的内容，一种按"章节（变体）"排版，另一种按"主题（操作）"排版；哪种更好，取决于你以后是经常加章节还是经常加主题。 – **关键规则与最佳实践**： – **解释器模式更容易加变体**：不必改动既有代码，只需在新变体类里实现所有操作的方法（但要实现全部操作，一次性工作量不小）。 – **访问者模式更容易加操作**：新建一个 `Visitor` 实现，把所有变体的处理写在一个类里，**既不改接口也不改变体**。 – 因此"如果'新增操作'是你最想为之做好准备的变化，就定义访问者接口并用访问者写函数"。 – 还有一个更硬的理由：当类型的设计者**希望客户端理解其内部结构并实现自己的操作**时，必须提供访问者接口。课程的经典例子是**解析树/语法树**：把具体语法树转换成抽象语法树（`makeAST`）时就出现了坏味道——一串 `switch (parseTree.name())`，最后又是 `default: throw new AssertionError("should never get here")`；而且只能谈论泛型的 `ParseTree` 结点，要调用具体变体的有用操作还得做不安全的转型。更复杂的解析库会用**访问者**实现 `makeAST`；**抽象语法树**更是普遍向客户端提供访问者接口，让客户端自己定义遍历操作。 – **代价（本讲的另一半真相）**：访问者把"加变体"变难了——新增一个变体就要**修改访问者接口**，于是**所有既有访问者实现都必须新增方法**，否则编译失败。这个"两条轴不能同时免费"的困境就是**表达式问题（expression problem）**。选择模式时，应当先问："未来更可能增加操作，还是增加变体？" – 「补充

说明」：Java 8 之后可以给 `Visitor` 接口的方法写 `default` 实现，从而让"新增变体"不再破坏既有访问者实现的编译。代价是**静态强制消失**：忘记实现新变体的分支不再报错，而是静默走默认行为（可能返回错误结果或抛 `UnsupportedOperationException`）——这正是"便利 vs 静态检查"的经典取舍。

访问者的抽象函数、表示不变量与不可变性（AF, RI, and Immutability of Visitors） – **定义与目的**：访问者是一个**对象**，因此也有表示（rep）、**抽象函数（abstraction function, AF）**与**表示不变量（representation invariant, RI）**。按 Reading 11 的框架把这两者写清楚，能决定你的访问者是"可复用的函数值"还是"藏了一堆状态的脆弱对象"。 – **直观解释（"它是什么？"）**：无状态访问者的 AF 是"这个对象 = 那个函数"（例如 `VariablesInFormula` 的实例 = 从公式取变量名集合的函数）。带参数的访问者（如 `EvaluateVisitor`）的 AF 还要说明"参数从哪来"：它的实例 = "用构造时给定的 map 求值"这一函数。 – **关键规则与最佳实践**： – **优先把访问者写成不可变的**：所有字段 `final`，额外参数由构造函数注入，遍历过程中的一切信息都通过**返回值**与**递归调用**传递。这样访问者可以被复用、缓存、并发共享（线程安全），也不需要 `reset`。 – 若访问者必须累积状态（如收集所有变量名到集合中），要把 RI 写清（累积器非 null、只在单次遍历中使用、遍历结束后不再改动）、在构造与每次变更后调用 `checkRep`，并且**绝不把内部可变集合直接返回**（那会造成表示暴露，Reading 11）。更好的做法是让 `on...` 返回结果、由外层 `accept` 的调用者组合，从根上避免可变状态。 – **不要复用"已用过一次"的有状态访问者**：同一个对象再次 `accept` 会把上一次的累积结果混进来，产生难以定位的错误答案。 – 访问者的方法规格要与数据类型的规格相容：`accept` 的返回值规格 = "把 visitor 应用于 this"，而 `Visitor.on` 的规格应当说明各变体分支的语义（例如"返回该变量的名字集合"）。 – **变体的不可变性没有改变**：访问者模式不要求变体可变；`Formula` 仍应是不可变类型（字段 `final`、`accept` 是观察者），双重分派只是"读取结构"，不修改结构。

用访问者实现求值、打印与变换（Evaluate, Print, Transform） – **定义与目的**：一个递归数据类型上最常出现的三类操作——**求值（evaluate）**、**打印/格式化（print/toString）**、**变换（transform，如代入、化简、转范式）**——都可以统一写成访问者。它关系到 Ready for change（三者都是"新操作"，正好是访问者擅长的方向）。 – **直观解释（"它是什么？"）**：求值访问者"把公式算成一个值"；打印访问者"把公式渲染成字符串"；变换访问者"把一个公式变成另一个公式"（结果类型 `R = Formula`）。 – **关键规则与最佳实践**： – **求值**：`EvaluateVisitor implements Formula.Visitor<Boolean>`，环境由构造函数注入；`on(Variable)` 查表，`on(Not)` 取反，`on(And)`/`on(Or)` 分别用 `&&`/`||`（可利用短路求值）。 – **打印**：`PrintVisitor implements Formula.Visitor<String>`；需要处理**运算符优先级与括号**时，可以把"父结点要求的优先级"作为**额外参数**——但在访问者模式里不方便在每个 `on` 上加参数，通常改用"构造函数注入初始状态 + 通过返回带优先级信息的结构"或"内部用带状态的小辅助类"来实现；简单场景直接在每个 `on` 里自行加括号即可（`And` 返回 `"(" + left + " ∧ " + right + ")"`）。 – **变换**：`SubstituteVisitor implements Formula.Visitor<Formula>`——把变量 `x` 替换为另一个公式，返回**新的** `Formula`（因为变体不可变，变换必须构造新树）。 – 由于访问者操作的都是**接口类型**（`Formula`、`Visitor<R>`），客户端可以自由添加自己的访问者，而实现者不必预知这些操作——这正是"为可扩展而设计"的实质。 – 注意**结果类型的表达力**：访问者 `<R>` 只能返回单一类型的值；若同一遍历需要同时算出多个结果（如"变量集合 + 深度"），要么定义一个小结果类作为 `R`，要么把这些信息放进一个（不可变的）记录里返回。

代码示例与对比分析

场景 1：`instanceof` + 强制转型的"模式匹配" vs 访问者与双重分派

❌ *错误代码*

```
// 错误：用 instanceof + cast 在外部实现操作，静态检查被抛弃
public static Set<String> variables(Formula f) {
    if (f instanceof Variable) {
        return Set.of(((Variable) f).name());
    } else if (f instanceof Not) {
        return variables(((Not) f).formula());
    } else if (f instanceof And) {
        And and = (And) f;
        return setUnion(variables(and.left()), variables(and.right()));
    } else if (f instanceof Or) {
        Or or = (Or) f;
        return setUnion(variables(or.left()), variables(or.right()));
    } else {
        throw new IllegalArgumentException("don't know what " + f + " is");
    }
}
```

【错误代码的问题】 1. **静态检查失效**：编译器不知道我们是否覆盖了所有变体；新增 `Xor`、`Literal` 之类的变体时，这段代码**不会编译报错**，而是可能在运行时抛异常（走到 `else`），或者更糟——静默返回错误答案。 2. **强制转型是危险操作**：`(Variable) f` 与 `instanceof` 检查必须严格对应，任何笔误（例如把 `(Or) f` 写成别的类型）都要到运行时才炸成 `ClassCastException`。 3. **每个新操作都要重抄一遍 if 链**：`depth`、`evaluate`、`toCNF` 各写一遍同样的分派骨架，重复且易漏。 4. **需要 getter 窥探表示**：外部函数必须通过 `name()` / `left()` / `formula()` 取字段，倾向于把表示细节暴露出去（表示依赖）。

✅ *正确代码*

```
// 正确：变体声明 accept，访问者接口把"每种变体怎么办"集中到一个类里
public interface Formula {
    /** 在一个 Formula 上调用一个访问者。
     *  @param <R> 结果类型
     *  @param visitor 要调用的访问者，前置条件：非 null
     *  @return 把 visitor 应用于 this 的结果 */
    public <R> R accept(Visitor<R> visitor);

    /** 代表"作用在不同种类 Formula 上的函数"。 */
    public interface Visitor<R> {
        R on(Variable var);
        R on(Not not);
        R on(And and);
        R on(Or or);
    }
}

public final class Variable implements Formula {
    private final String name;
    public Variable(String name) {
        if (name == null || name.isEmpty()) throw new IllegalArgumentException("name");
        this.name = name;
    }
    public String name() { return name; }

    // 第一次动态分派到达这里；这里再发起第二次动态分派
    @Override public <R> R accept(Visitor<R> visitor) {
        return visitor.on(this);
    }
}

public final class Not implements Formula {
    private final Formula formula;
    public Not(Formula formula) { this.formula = formula; }
    public Formula formula() { return formula; }

    @Override public <R> R accept(Visitor<R> visitor) { return visitor.on(this); }
}

public final class And implements Formula {
    private final Formula left, right;
    public And(Formula left, Formula right) { this.left = left; this.right = right; }
    public Formula left() { return left; }
    public Formula right() { return right; }

    @Override public <R> R accept(Visitor<R> visitor) { return visitor.on(this); }
}

public final class Or implements Formula {
    private final Formula left, right;
    public Or(Formula left, Formula right) { this.left = left; this.right = right; }
    public Formula left() { return left; }
    public Formula right() { return right; }
```

```
    @Override public <R> R accept(Visitor<R> visitor) { return visitor.on(this); }
}

/** 求公式中出现的所有变量名。 */
public final class VariablesInFormula implements Formula.Visitor<Set<String>> {
    @Override public Set<String> on(Variable var) {
        return Set.of(var.name());
    }
    @Override public Set<String> on(Not not) {
        return not.formula().accept(this);          // 递归：把同一个访问者继续传下去
    }
    @Override public Set<String> on(And and) {
        return setUnion(and.left().accept(this), and.right().accept(this));
    }
    @Override public Set<String> on(Or or) {
        return setUnion(or.left().accept(this), or.right().accept(this));
    }

    /** @return set1 与 set2 的并集（不可变） */
    private static <E> Set<E> setUnion(Set<E> set1, Set<E> set2) {
        Set<E> result = new HashSet<>(set1);
        result.addAll(set2);
        return Set.copyOf(result);
    }
}

// 客户端用法
Formula f = new And(new Or(new Variable("P"), new Variable("Q")),
                    new Not(new Variable("R")));
Set<String> names = f.accept(new VariablesInFormula());   // { "P", "Q", "R" }
```

【为什么这样更好】 编译器重新开始帮我们工作：`Visitor<R>` 接口声明了全部变体分支，任何实现类若漏掉一个方法都无法编译；新增变体时**所有访问者实现会立刻编译失败**（虽然这本身是代价，但至少不会静默出错）。操作代码集中在一处，`VariablesInFormula` 读起来就像最初想要的 `switch`。整个过程中**没有任何 cast、没有任何 instanceof**，因此也不会有 `ClassCastException`。 **【代码对比解说】** 关键机制是**双重分派**：`f.accept(v)` 用第一次动态分派（依据 `f` 的实际类型）落到 `And.accept`；`And.accept` 立刻用第二次动态分派（依据 `v` 的实际类型）调用 `v.on(this)`。两次分派合起来就把"哪种变体 × 哪个操作"唯一确定下来，等价于一次"类型 switch"，却完全由类型系统与重载解析完成。注意递归调用写成 `and.left().accept(this)`：访问者把自己继续传下去，遍历顺序完全由 `on` 方法决定。`accept` 在每个变体里只有一行，说明"分派"与"策略"被干净地分开了——这正是访问者模式的优雅之处。 **【设计原则透视】** 这是 **Reading 12（用接口定义 ADT）**、**Reading 17（递归数据类型）** 与 **Reading 26（把函数表示为数据）** 的组合：函数成为对象（`Visitor<R>` 的实例），类型安全由编译器保证。与 Reading 26 的 `instanceof` 反例对照，可以看到"**用类型系统表达分支 vs 用运行时测试表达分支**"的差别——前者是 Safe from bugs 的正道。

场景 2：新增一个操作——修改接口与全部变体 vs 新增一个访问者类

❌ *错误代码*

```
// 错误（在"操作会不断增加"的场景下）：把每个操作都做成实例方法
// 于是新增 depth() 操作必须触碰 5 个地方：接口 + 4 个变体类

public interface Formula {
    Set<String> variables();
    boolean evaluate(Map<String, Boolean> map);
    Formula substitute(String name, Formula replacement);
    int depth();                        // 新增：破坏性改动 1
    public <R> R accept(Visitor<R> visitor);
}

public final class Variable implements Formula {
    private final String name;
    @Override public Set<String> variables() { return Set.of(name); }
    @Override public boolean evaluate(Map<String, Boolean> map) { /* ... */ return false; }
    @Override public Formula substitute(String n, Formula r) {
        return n.equals(name) ? r : this;
    }
    @Override public int depth() { return 1; }                    // 新增：破坏性改动 2
    @Override public <R> R accept(Visitor<R> v) { return v.on(this); }
}

public final class Not implements Formula {
    private final Formula formula;
    @Override public Set<String> variables() { return formula.variables(); }
    @Override public boolean evaluate(Map<String, Boolean> map) { return !formula.evaluate(
    @Override public Formula substitute(String n, Formula r) {
        return new Not(formula.substitute(n, r));
    }
    @Override public int depth() { return 1 + formula.depth(); }  // 新增：破坏性改动 3
    @Override public <R> R accept(Visitor<R> v) { return v.on(this); }
}
// And、Or 同理（破坏性改动 4、5）……
```

【错误代码的问题】 1. **一次新操作 = 修改接口 + 所有实现类**：5 个文件（接口 + 4 个变体）必须同时改、同时重新编译、同时重新测试；变体越多，代价越大。 2. **改动落在别人的代码里**：若某些变体由其他程序员维护甚至已经发布，这种"为了加一个操作而改遍所有类"的变更非常难推动。 3. **操作本身被撕碎**：想通读 `depth` 的实现，必须跳 4 个类；想重构它，要做 4 处一致的改动，任一处不一致就是 bug。 4. **接口变得臃肿**：所有操作都被塞进数据类型的接口，客户被迫依赖一个巨大的接口，而不是只依赖自己需要的操作。

✅ *正确代码*

```
// 正确：为"新增操作"提供访问者接口；新增操作只是新增一个类
public final class DepthInFormula implements Formula.Visitor<Integer> {
    @Override public Integer on(Variable var) { return 1; }
    @Override public Integer on(Not not) { return 1 + not.formula().accept(this); }
    @Override public Integer on(And and) {
        return 1 + Math.max(and.left().accept(this), and.right().accept(this));
    }
    @Override public Integer on(Or or) {
        return 1 + Math.max(or.left().accept(this), or.right().accept(this));
    }
}

// 客户端（甚至可以是我们之外的人）自由定义自己的操作，实现者不需要预知它
Formula f = new Or(new Variable("p"), new Not(new Variable("q")));
int d = f.accept(new DepthInFormula());        // 3

// 同一个类型上再加一个操作：又是一个新类，零改动既有代码
public final class CountVisits implements Formula.Visitor<Integer> {
    private final Map<String, Integer> counts = new HashMap<>();   // 见场景 3：有状态访问者的耳
    @Override public Integer on(Variable var) { return 1; }
    @Override public Integer on(Not not) { return 1 + not.formula().accept(this); }
    @Override public Integer on(And and) {
        return 1 + and.left().accept(this) + and.right().accept(this);
    }
    @Override public Integer on(Or or) {
        return 1 + or.left().accept(this) + or.right().accept(this);
    }
}
```

【为什么这样更好】 新增操作**不触碰** `Formula` 接口与任何变体类：既有代码零修改、零重新测试，变更被限制在一个新文件里。这使类型可以"向外开放"：客户（下一个人、另一个团队）能定义自己的遍历，而实现者无需为每个未来操作预留方法。原文明说：如果"新增操作"是你最希望准备好的变化，那就定义访问者接口、把函数写成访问者。 **【代码对比解说】** 两种写法的差别就是"码表按列排版还是按行排版"：解释器模式把一列（一个变体的所有操作）放在一起，于是加行（操作）要动所有列；访问者模式把一行（一个操作的所有变体）放在一起，于是加行只是一处新增，但加列（变体）要动所有行。注意 `DepthInFormula` 与 `CountVisits` 都是**独立文件**：它们的存在不需要修改数据类型的源代码，也不需要数据类型作者知道它们的存在。 **【设计原则透视】** 这就是 **表达式问题（expression problem）** 的操作侧优势，对应 Ready for change；同时它也是"**面向接口编程 + 依赖倒置**"（Reading 12）的极端形态：数据类型的作者提供 `accept` 这个扩展点，把"新操作"的实现责任交给客户端。课程强调的另一用途是**解析树/AST**：具体语法树转抽象语法树的代码天然是"按结点类型分派"的，用访问者替代一串 `switch (parseTree.name())`（以及 `default: throw new AssertionError(...)`）能重新拿回静态检查。

场景 3：可变的"累积型"访问者 vs 不可变的函数式访问者（AF / RI / 线程安全）

❌ *错误代码*

```
// 错误：访问者把结果累积在可变字段里，且暴露内部集合、缺少 RI 与 reset
public final class CollectingVisitor implements Formula.Visitor<Void> {
    private Set<String> names;        // 可变状态：既可能为 null，也会在多次遍历间泄漏
    private int visits;

    @Override public Void on(Variable var) {
        if (names == null) names = new HashSet<>();   // 惰性初始化：忘记重置就累积旧结果
        names.add(var.name());
        visits++;
        return null;
    }
    @Override public Void on(Not not) {
        visits++;
        not.formula().accept(this);
        return null;
    }
    @Override public Void on(And and) {
        visits++;
        and.left().accept(this);
        and.right().accept(this);
        return null;
    }
    @Override public Void on(Or or) {
        visits++;
        or.left().accept(this);
        or.right().accept(this);
        return null;
    }

    public Set<String> names() { return names; }   // 表示暴露：调用者可直接改内部集合
    public int visits() { return visits; }
}

// 复用同一个访问者对象两次：
CollectingVisitor cv = new CollectingVisitor();
f1.accept(cv);
Set<String> r1 = cv.names();      // { "P", "Q" }
f2.accept(cv);
Set<String> r2 = cv.names();      // { "P", "Q", "R" } —— 被上一次的结果污染了！
```

【错误代码的问题】 1. **状态跨遍历泄漏**：同一个访问者对象被复用（或客户端"顺手"多访问一次）时，结果会累积，正确答案变成错误答案，且没有异常提示。 2. **`names` 可能为 `null`**：若公式里没有变量（例如全是常量的公式），`names()` 返回 `null`，调用者必须判空；RI 无法写清。 3. **表示暴露（rep exposure）**：`names()` 直接把内部 `HashSet` 交出去，调用者一改，访问者的"结果"就被篡改；AF 与 RI 都被破坏。 4. **返回 `Void` 依赖副作用**：遍历语义藏在可变字段里，阅读 `on` 方法看不出"这个访问者到底有没有遍历左子树"；同时该对象不可并发复用（线程不安全），难以缓存。 5. **没有 `checkRep`**：连"累积器在被读取时非 null"这样的基本不变量都没有检查。

✅ *正确代码*

```
// 正确：不可变的函数式访问者；额外参数由构造函数注入，信息通过返回值传递
public final class VariablesInFormula implements Formula.Visitor<Set<String>> {
    // 无字段、无状态：所有结果都由返回值与递归调用传递

    @Override public Set<String> on(Variable var) {
        return Set.of(var.name());
    }
    @Override public Set<String> on(Not not) {
        return not.formula().accept(this);
    }
    @Override public Set<String> on(And and) {
        return setUnion(and.left().accept(this), and.right().accept(this));
    }
    @Override public Set<String> on(Or or) {
        return setUnion(or.left().accept(this), or.right().accept(this));
    }
    private static <E> Set<E> setUnion(Set<E> a, Set<E> b) {
        Set<E> result = new HashSet<>(a);
        result.addAll(b);
        return Set.copyOf(result);          // 返回不可变副本，杜绝表示暴露
    }
}

/** 带参数的访问者：环境由构造函数注入，字段 final。
 *  AF: 该对象表示函数  formula ↦ 在 map 下求 formula 的值
 *  RI: map != null，且（调用前由 evaluate 检查）包含 formula 中出现的所有变量 */
public final class EvaluateVisitor implements Formula.Visitor<Boolean> {
    private final Map<String, Boolean> map;

    public EvaluateVisitor(Map<String, Boolean> map) {
        if (map == null) throw new NullPointerException("map");
        this.map = map;
        checkRep();
    }
    private void checkRep() {
        assert map != null : "map must be non-null";
    }

    @Override public Boolean on(Variable var) {
        Boolean value = map.get(var.name());
        if (value == null) {
            throw new IllegalArgumentException("unbound variable: " + var.name());
        }
        return value;
    }
    @Override public Boolean on(Not not) { return !not.formula().accept(this); }
    @Override public Boolean on(And and) {
        return and.left().accept(this) && and.right().accept(this);   // 短路求值
    }
    @Override public Boolean on(Or or) {
        return or.left().accept(this) || or.right().accept(this);
    }
}
```

```
/** 把公式中出现的所有变量都用 map 中的真值代入求值。
 *  @param formula 待求值的公式
 *  @param map 前置条件：必须为 formula 中出现的每个变量提供取值
 *  @return formula 在 map 下的取值 */
public static boolean evaluate(Formula formula, Map<String, Boolean> map) {
    return formula.accept(new EvaluateVisitor(map));
}

// 每次调用都新建一个访问者：无共享、无泄漏、可并发
boolean r1 = evaluate(new And(new Variable("a"), new Not(new Variable("b"))),
                      Map.of("a", true, "b", false));   // true
```

【为什么这样更好】 访问者成为**纯函数值**：给定访问者与公式，结果唯一确定，不修改任何状态；同一个访问者实例可以被反复使用、可以并发共享、可以被缓存（因为不可变）。`Map` 是构造时注入的 `final` 字段，RI 可写、可查（`checkRep`），未绑定变量显式失败并带上变量名。返回不可变集合，彻底避免表示暴露。

【代码对比解说】 两版的差别是"**把结果放在哪里**"：左版放在访问者的可变字段里（需要 reset、可能为 null、会泄漏、不可并发），右版放在**返回值**里（由递归调用组合起来）。这与 Reading 26 中"环境作为参数而不是全局状态"是同一个道理——**让依赖和数据沿调用栈流动，而不是留在对象的字段里**。注意右版也保留了"有状态访问者"的正当用法：**构造函数注入的只读参数**不算可变状态，它是"这个函数值的一部分"。当确实需要累积（例如统计词频）时，应把累积器限制在单次遍历内、在方法返回前转成不可变结果，并在注释中写清 AF/RI 与"不得复用"的前置条件。 **【设计原则透视】** 直接对应 **Reading 11（AF 与 RI）**、**Reading 8（不可变性）** 与 **Reading 21/23（并发）**：不可变对象天生线程安全，不需要锁，也不需要"谁的访问者"这类约定；而可变的共享访问者则是典型的并发 bug 温床（某个线程 reset 了另一个线程正在使用的累积器）。`evaluate` 的规格还把"map 必须覆盖所有变量"写成显式前置条件，符合 Reading 7 对规格的要求。

场景 4：选错模式——变体频繁增加却使用访问者 vs 按"变化轴"选择模式

❌ *错误代码*

```
// 反例：表达式类型的变体经常增加，却把操作都做成了访问者
public interface IntegerExpression {
    public interface Visitor<R> {
        R on(Constant c);
        R on(Plus p);
        R on(Variable v);
    }
    public <R> R accept(Visitor<R> visitor);
}

// 现在要支持乘法：必须修改 Visitor 接口（破坏性改动）
public interface Visitor<R> {
    R on(Constant c);
    R on(Plus p);
    R on(Variable v);
    R on(Times t);   // 所有既有访问者实现（包括客户端写的）都必须新增这个方法，否则编译失败
}

// 结果：客户端代码成批编译失败，被迫实现一个它们不关心、甚至无法正确实现的分支
```

【错误代码的问题】 1. **破坏性变更**：新增变体让**所有**既有访问者实现编译失败；若这些实现由第三方维护（例如客户定义的十几个操作），升级代价极高。 2. **被迫实现的"假分支"**：客户端往往对新变体一无所知，只能写 `throw new UnsupportedOperationException("Times not supported")`，把类型安全换成了运行时错误。 3. **无法逐步演进**：想分两次发布（先加变体，后更新访问者）都做不到，必须一次性同步全部实现。 4. **模式与需求错位**：类型的设计者选错了"为哪条变化轴做准备"。

✅ *正确代码*

```
// 正确：变体频繁增加、操作相对稳定时，用解释器模式（操作作为实例方法）
public interface IntegerExpression {
    /** @param environment 变量到值的映射；前置条件：包含本表达式出现的所有变量
     *  @return 本表达式的值 */
    int eval(Map<String, Integer> environment);
    /** @return 本表达式出现的所有变量名 */
    Set<String> variables();
}
```

```
// 新增变体：只加一个类，接口与所有既有类都不动
public final class Times implements IntegerExpression {
    private final IntegerExpression left, right;
    public Times(IntegerExpression left, IntegerExpression right) {
        this.left = left;
        this.right = right;
    }
    public IntegerExpression left() { return left; }
    public IntegerExpression right() { return right; }

    @Override public int eval(Map<String, Integer> environment) {
        return left.eval(environment) * right.eval(environment);
    }
    @Override public Set<String> variables() {
        Set<String> result = new HashSet<>(left.variables());
        result.addAll(right.variables());
        return Set.copyOf(result);
    }
    @Override public String toString() { return "(" + left + " * " + right + ")"; }
}
```

```
//  （可选）混合方案：把稳定的核心操作留在接口上，同时用 accept 提供"客户端自定义操作"的扩展点
public interface IntegerExpression {
    int eval(Map<String, Integer> environment);
    Set<String> variables();
    public <R> R accept(Visitor<R> visitor);       // 扩展点：把"新操作"交给客户端

    public interface Visitor<R> {
        R on(Constant c);
        R on(Plus p);
        R on(Variable v);
        R on(Times t);
    }
}
```

【为什么这样更好】 选择标准是"**哪条轴更可能变**"：变体常增 → 解释器模式（加变体只加一个类，零破坏）；操作常增 → 访问者模式（加操作只加一个类，零破坏）。当两者都要时，可以**混合**：把少数稳定、核心的操作放在接口上（简单直接），同时提供一个 `accept` 扩展点让客户端实现自己的遍历。这样既保留了日常使用的简洁，又把"未来未知的操作"开放出去。 **【代码对比解说】** 左版把"扩展点"押在了操作维度上，于是变体维度变得脆弱；右版（解释器模式）把扩展点押在变体维度上。注意二者并非互斥：真正成熟的数据类型往往**同时**提供两者（例如课程中 AST 的做法——核心操作在类型上，同时向客户端提供访问者接口）。选择前应回到"码表"：列出你预期会新增的行与列，数一数哪一维更多，再决定把代码组织成行还是列。 **【设计原则透视】** 这就是 **表达式问题** 的完整表述：在不修改既有代码的前提下，无法同时让"新增变体"与"新增操作"都变得容易。它也体现 **Reading 7（设计规格）** 与 **Reading 4（代码评审）** 的精神：设计决策要基于"未来会怎么变"的证据，而不是跟风使用某个模式。关于"新增变体"的代价，还有一个必要的「补充说明」：Java 8 起可以给 `Visitor` 的方法写 `default` 实现，从而让新增变体**不再导致编译失败**——代价是失去静态强制，忘记实现的分支会静默走默认行为（可能返回错误结果），属于"便利换掉安全检查"的典型权衡。

与其他设计原则的关联

- **Reading 26（小语言 I）**：本讲是它的直接续篇。Reading 26 确立了"把代码表示为数据"与解释器模式；本讲指出解释器模式的两个缺点，并给出访问者这一替代实现策略。两讲共同回答"如何为一门小语言实现操作"。
- **Reading 17（递归数据类型）**：`Formula`、`Music`、`IntegerExpression` 都是递归数据类型；访问者正是"在这个类型上实现的函数"，`variables` 的四个等式就是访问者的四个 `on` 方法。
- **Reading 12（接口、泛型、枚举、函数对象）**：`Visitor<R>` 用泛型参数化结果类型；访问者实例是**函数对象**，可以像 Reading 20（回调）那样被传递、储存、返回；`accept` 是接口上的扩展点。
- **Reading 11（抽象函数与表示不变量）**：访问者对象也有 AF/RI——无状态访问者的 AF 是"这个对象 = 那个函数"，带参数访问者的 AF 说明参数含义与来源；RI 要求参数非 null、状态只在单次遍历中使用。
- **Reading 8（可变性与不可变性）**：递归类型的变体必须不可变，`accept` 是观察者；访问者本身**优先写成不可变**，才能被复用与并发共享。
- **Reading 21（并发）与 Reading 23（互斥）**：不可变访问者天然线程安全；可变累积型访问者在多线程下需要保证"每个线程用自己的实例"，否则会出现结果混合或数据竞争。
- **Reading 19（解析器）**：把具体语法树转成抽象语法树的 `makeAST` 是典型的"按结点类型分派"的坏味道（`switch (parseTree.name())` + `default: throw new AssertionError`）；复杂解析库用访问者实现它，AST 也常向客户端提供访问者接口。
- **Reading 6（规格说明）与 Reading 7（设计规格）**：`accept` 与 `on` 都需要规格；`evaluate` 的前置条件"map 覆盖所有变量"是典型例子。
- **Reading 3（测试）与 Reading 15（相等性）**：访问者是纯函数值时最容易测试（给定公式与访问者，断言结果）；变换型访问者返回新树，需要 `equals` 才能写出简洁的断言。
- **Reading 4（代码评审）**：`instanceof` + cast 的分派链、`default: throw new AssertionError(...)` 都是评审中一眼可见的坏味道；访问者是"如何把这类坏味道改造掉"的示范。

关键要点

- **双重分派是访问者的心脏**：`obj.accept(visitor)` 用第一次动态分派选中变体，变体再用 `visitor.on(this)` 用第二次动态分派选中操作分支；两次分派合起来替代了"类型 switch"，且全程不需要 cast。
- **按变化轴选模式**：变体常增 → 解释器模式（加变体只加一个类）；操作常增 → 访问者模式（加操作只加一个类）；两者都重要时可以混合使用。
- **`instanceof` + cast 的"模式匹配"不安全**：静态检查失效，新增变体会静默出错或运行时崩溃；这是必须消除的坏味道。
- **访问者优先写成不可变函数值**：额外参数由构造函数注入、字段 `final`、结果通过返回值传递；写清 AF/RI，绝不返回内部可变集合（表示暴露）。

- **访问者是"类型上的 switch + 迭代器 + 一等函数"**：它让客户端可以在不改动数据类型的前提下定义自己的操作，这也是 AST/解析树提供访问者接口的原因。

常见陷阱与注意事项

1. **在 `accept` 里做别的事（或忘记写 `visitor.on(this)`）** → 双重分派断链，访问者永远收不到正确分支；`accept` 应当只有一行。
2. **新增变体时忘记更新所有访问者** → 若接口方法无 `default`，编译失败（还算安全）；一旦为图方便加了 `default` 或落进 `else` 分支，就会静默返回错误结果（例如变量集合漏掉新变体里的变量）。
3. **用 `instanceof` + 强制转型替代访问者** → 静态检查失效、`ClassCastException` 风险、每个新操作重复一遍分派链，新增变体时既可能崩也可能给出错误答案。
4. **把访问者写成有状态且被复用的对象** → 结果跨遍历泄漏、`null` 累积器、并发下互相污染；应改为无状态或"只在单次遍历内累积、返回不可变结果"。
5. **在访问者里直接返回内部可变集合/列表** → 表示暴露，调用者一改就破坏访问者的 AF/RI；应返回 `Set.copyOf(...)`、`List.copyOf(...)` 等不可变副本。
6. **用 `<R>` 硬塞多个结果** → 需要同时算出多个量时，若强行用副作用或 `Object` 装结果，会牺牲类型安全；应定义一个小结果类作为 `R`，或拆成两个访问者。

思考题（带答案）

问题 1：请逐步描述 `f.accept(new VariablesInFormula())` 的执行过程，其中 `f` 是 `And(Or(Variable("P"), Variable("Q")), Not(Variable("R")))`。说明每一步是**动态分派**还是**静态检查**在起作用，并解释为什么这个设计能在不使用任何 `instanceof`/cast 的情况下完成"按类型分派"。

答案：执行过程是：(1) `f` 的实际类型是 `And`，`f.accept(...)` 通过**动态分派**进入 `And.accept`，其方法体是 `return visitor.on(this);`；(2) `visitor` 的实际类型是 `VariablesInFormula`，因此 `visitor.on(this)` 再次通过**动态分派**进入 `VariablesInFormula.on(And)`——这是第二次分派，即双重分派；注意 `this` 的静态类型是 `And`，所以 Java 的**重载解析**（编译期、静态）选中 `on(And)` 这个重载；(3) `on(And)` 计算 `and.left().accept(this) || ...` 的并集，先对左子式 `Or(...)` 递归：动态分派进入 `Or.accept`，再动态分派到 `on(Or)`；(4) `on(Or)` 分别对 `Variable("P")` 与 `Variable("Q")` 递归，二者都落到 `on(Variable)`，返回 `{"P"}`、`{"Q"}`，并集成 `{"P","Q"}`；(5) 回到 `on(And)`，对右子式 `Not(Variable("R"))` 递归：`Not.accept` → `on(Not)` → `not.formula().accept(this)` → `on(Variable)` 返回 `{"R"}`，`on(Not)` 原样返回 `{"R"}`；(6) 最后并集成 `{"P","Q","R"}`。整个过程**没有一次 `instanceof` 或强制转型**，因为"选哪个分支"完全由两次动态分派 + 一次重载解析完成：第一次分派用对象的实际类型确定所在变体，第二次分派用访问者的实际类型确定操作实现。静态检查在这一过程中负责保证：`Visitor<R>` 声明了每个变体的方法（漏实现无法编译）、`accept` 的签名与泛型一致、`on` 的参数类型与 `this` 兼容。因此，"类型分派"的职责被交给了类型系统本身，而不是运行时的类型测试。

问题 2：你的团队要为一个"报表表达式"类型实现操作：变体有 6 种（数字、变量、加、乘、求和、条件），预计**每季度会新增 1–2 个变体**（业务方不断提出新算子），而操作基本固定（求值、类型检查、打印）。团队

中有一位同事主张用访问者模式，因为"访问者是 6.031 推荐的做法"。请给出你的判断与理由，并说明如果要同时给外部客户提供"自定义操作"的能力，你会怎么设计。

答案：应当选择**解释器模式**（把操作声明为接口上的实例方法，在每个变体类中实现），因为团队的变化轴是"**变体频繁增加**"而操作稳定。用访问者模式的话，每次新增算子都要修改 `Visitor` 接口，于是所有既有访问者实现——包括客户端写的——都会编译失败，被迫新增一个自己可能无法正确实现的分支（通常只能写 `UnsupportedOperationException`），这是典型的破坏性变更；而解释器模式下新增变体只是新增一个类，既有代码零改动。这与"访问者更容易加操作、解释器更容易加变体"的结论完全一致（表达式问题）。如果同时要给外部客户提供自定义操作的能力，可以采用混合设计：把稳定的核心操作（求值、类型检查、打印）留在接口上作为实例方法，同时提供 `accept(Visitor<R>)` 作为**扩展点**，让客户实现自己的遍历；并且要考虑两类风险的代价：(1) 新增变体时会让既有访问者编译失败——可以通过「补充说明」中的 Java 8 `default` 方法缓解，但要接受"忘记实现会静默走默认行为"的代价，或者在 `default` 实现里统一抛 `UnsupportedOperationException` 并配以测试；(2) `accept` 必须作为规格的一部分被文档化并测试（每个变体都应有一行 `return visitor.on(this);` 的测试或等价断言）。最后，把决策理由（哪条轴在变、代价是什么）写进设计说明，这样下一个维护者不会推翻它。

问题 3：为什么课程的 `EvaluateVisitor` 要把 `Map<String,Boolean>` 通过**构造函数**传进去并存在字段里，而不是给每个 `on` 方法都加一个 `map` 参数？请从接口设计、递归便利性、AF/RI 与不可变性的角度分析，并说明这种做法在并发场景下的含义。

答案：(1) **接口设计**：`Visitor<R>` 的每个 `on` 方法签名由数据类型决定（参数是变体对象，返回 `R`）；如果给 `on` 加参数，访问者接口就必须为每个操作量身定制，无法用一个统一的 `Visitor<R>` 表示所有单参数函数，也会让 `accept` 的签名随之改变（`accept` 需要把额外参数透传下去，于是数据类型被迫知道操作的参数类型，破坏抽象边界）。(2) **递归便利性**：递归调用写的是 `and.left().accept(this)`——只传 `this` 一个参数；若需要额外参数，就必须写成 `and.left().accept(this, map)`，把所有参数在整棵树上反复传递，代码冗长且容易传错。把参数存进字段后，递归调用保持极简。(3) **AF/RI**：这样做的代价是访问者"有状态"，因此必须写清 AF（"该对象表示函数 `formula ↦ 在 map 下求 formula 的值`"）与 RI（"`map` 非 null，且在调用前包含公式中出现的所有变量"），并在构造函数与 `checkRep` 中检查。(4) **不可性与并发**：关键是这些字段是 **`final` 且只读**的——访问者只是在"出生"时固定了参数，此后从不修改，因此它仍然是**不可变对象**，可以被安全地共享、复用、并发使用（多个线程各自用同一个 `EvaluateVisitor` 求值互不干扰）；相反，如果字段可变且遍历中会被写入（如累积器），那就变成有状态可变对象，必须保证每个线程用独立实例，否则会出现结果污染与数据竞争。因此"构造函数注入参数"是"让函数携带上下文"的正确做法，而"遍历中修改字段"才是需要警惕的可变状态。

Reading 28: 软件工程中的伦理（Ethical Software Engineering）

> 说明：本讲 sp22 原版使用 TypeScript，本笔记按用户要求提供 Java 代码示例；类型/API 与 sp21（6.031 Java 版）原文保持一致。本讲在 sp21 中的编号是 Reading 30，两版正文几乎逐段对应，只有少量案例引文与资源清单不同，差异处本笔记会单独标注。

概述

到本讲为止，6.031 一直在用三个技术目标衡量软件：**Safe from bugs（SFB，免于缺陷）**、**Easy to understand（ETU，易于理解）**、**Ready for change（RFC，易于修改）**。本讲引入第四个属性——**伦理（ethicality）**，它和前三个一样是"好软件"的组成部分，一样可以在迭代式设计过程中被检验、被修复，而不是事后贴上去的标签。课程的原话是：软件构造从来不是发生在真空里的——你和别人一起工作、维护别人设计的软件、软件被其他人使用、而这些人的使用又影响到更多的人。因此，写得正确、清晰、可修改，是你对同事、对所在组织、对用户、乃至对**并不使用你的软件却受其影响的人**（expanding circles of influence，扩展的影响圈）应尽的职业义务。本讲不规定任何一套道德信条，也不告诉你发生冲突时哪个属性更重要；它给你的是工具：ACM 道德准则的条款、四种**道德透镜（moral lenses）**，以及一个可以放进代码评审流程里的提问方法。这些工具直接服务于三大目标：伦理缺陷往往就是未被发现的 SFB（例如把私密数据写进日志）、未被讨论的 ETU 问题（例如不可读的代码让别人误解了规格）、以及未被识别的 RFC 债务（例如一个把紧急处置责任硬编码给"操作员"的设计，后来无论如何都改不动）。

本讲的学习目标只有两条，但要求很高：能够解释设计、构建、维护软件时需要考虑的伦理原则；能够用**四种不同的道德透镜**审视一个具体系统的伦理后果。

核心概念与设计原则详解

伦理作为软件的一个质量属性（Ethicality as a Quality Property） – **定义与目的**：伦理性和性能、安全、可用性一样，是软件的一个属性；课程明确指出，这些属性"cannot be bolted on later"（不能在事后用螺栓拧上去），必须作为迭代式设计过程的一部分被"烘焙"进系统。它解决的是"软件是否对人造成伤害"这一质量问题，而 SFB/ETU/RFC 只覆盖"软件是否按意图工作"。 – **直观解释（"它是什么？"）**：把"伦理测试"想象成戴上一顶不同的帽子。平时你戴上"测试帽"，会拼命想办法弄坏自己的代码；现在戴上"伦理帽"，你要拼命设想**别人会怎样滥用、误用、伤害**你构建的东西。作者的原话是：不要指望自己或别人"自然而然就做对了"。 – **关键规则与最佳实践**： – 把伦理后果写进设计评审的议程，而不是当作事后公关问题。 – 对你交付的每一个功能问三遍：谁会受益、谁会受损、受损者有没有发言权。 – 记录"为什么做了这个权衡"——权衡的**过程**本身就是一种可审查的产物。 – 承认伦理缺陷的修复成本与正确性缺陷一样，越晚修越贵。 – 记住伦理属性没有"规格说明"可以对照，所以必须靠人来提问，而不能靠测试套件自动发现。

扩展的义务圈（Expanding Circles of Influence） – **定义与目的**：课程把软件工程的责任从内向外分成四层：(1) 与你共事的同事，(2) 你所在的组织，(3) 你的软件的用户，(4) 即使不使用软件也被它影响的人，以及这些人所生活的世界。它解决的是"我对谁负责"这个边界问题——边界划得太小，就会出现"我只负责让代码编译通过"这种自我免责。 – **直观解释（"它是什么？"）**：社交网络是最直观的例子。一个**奇迹般地从未向任何社交网站透露过一条信息**的人，仍然会深深受到亲友、社群、社会使用社交网络的影响。软件的影响不是"系统 × 个人"的二元函数，而是"系统 × 个人 × 社群 × 社会"的复杂交互。 – **关键规则与最佳实践**： – 判断影响时，把**非用户（non-users）**也列进利益相关者清单。 – 警惕**涌现属性（emergent properties）**：系统被真实的人在一个复杂世界里使用时，会产生设计者没有预料到的整体行为。 – 采纳 ACM 准则 3.7：要"recognize and take special care of systems that become integrated into the infrastructure of society"（识别并特别关照那些已经融入社会基础设施的系统）。 – 产品采用率越高，伦理责任越大——这是

随成功一起增长的负债。 – 当系统成为基础设施时，用"社会会怎样依赖它"而不是"用户会不会喜欢它"来驱动设计。

抄袭、版权与许可证（Plagiarism, Copyright, and Licenses） – **定义与目的**：这是最小尺度的伦理问题：任何时候使用别人写的代码，都必须**署名（attribute）**并拥有**许可证（license）**。它解决的是"知识产权的边界在哪里"。 – **直观解释（"它是什么？"）**：软件工程有强烈的共享文化——自由软件运动（free software movement）认为"用户不能自由运行、修改、再分发"的软件是不道德的；开源运动（open source movement）主张共享源码在实践上更有优势；Stack Overflow 上那些协作积累的解释与示例就是这种文化的产物。但是，共享的前提是**作者用许可证授予你这些权利**；没有许可证就复制源码，既违法（侵犯版权）也不道德。若作者放弃版权、把代码放进**公有领域（public domain）**，则是例外：任何使用都不需要许可证。 – **关键规则与最佳实践**： – 使用外部代码前先确认许可证，并保留版权声明。 – 作业与项目里引用的外部材料必须明确署名（6.031 的 Collaboration and Sharing 政策也同样要求）。 – 注意"免费"不等于"自由"：free software 指的是**自由**（可运行、可修改、可再分发），不一定是 $0。 – 不要把课程作业的解答公开到 GitHub——starter code 的版权属于课程组，你的解答是衍生作品，公开分发需要事先许可（这是 6.031 合作政策的明确条款）。 – 自己写的东西也要留痕：提交信息、注释里的出处链接，都是伦理习惯的一部分。

ACM 道德准则（ACM Code of Ethics and Professional Conduct） – **定义与目的**：ACM 是计算机科学界最主要的国际专业组织（它颁发图灵奖）。它的准则为本讲提供了一套可引用的条款编号体系，让"我觉得这样不太好"变成"这违反了第 1.2 条"。它解决的是"如何把直觉变成可讨论、可追责的规范"。 – **直观解释（"它是什么？"）**：准则分三大节：第 1 节 **General Ethical Principles（一般伦理原则）**，第 2 节 **Professional Responsibilities（职业责任）**，第 3 节 **Professional Leadership Responsibilities（职业领导责任）**。第 1 节里课程特别点名的条款是：1.1 对人类福祉的义务、1.2 防止伤害、1.3 诚实、1.4 公平与包容、1.5 尊重著作权与版权、1.6 尊重隐私、1.7 保密（尤其对雇主）。 – **关键规则与最佳实践**： – 1.2（避免伤害）在课程中被明确地连接到第 3 节的 3.7（关照已成为社会基础设施的系统）。 – 2.1 "strive to achieve high quality in both the processes and products"——质量和过程都是义务。 – 2.2 "maintain high standards of professional competence, conduct, and ethical practice"；准则 2.2 的原文还强调：职业胜任力"starts with technical knowledge and with awareness of the social context in which their work may be deployed"，并且需要沟通、反思性分析、识别与应对伦理挑战的技能。 – 准则本身不是法律，它是一份**专业共同体对成员的期望**；它给你语言，不给你答案。 – 引用条款时要具体到编号，这样在评审会上才能把讨论从"价值观之争"拉回"我们是否履行了这项义务"。

伦理结构，而不只是伦理个体（Ethical Structures, not just Ethical Individuals） – **定义与目的**：这是本讲最重要的一条社会学洞见：当一个大组织造出有害系统（造成伤害、歧视、加剧不平等、侵犯隐私）时，人们倾向于寻找一个"邪恶大反派"。但更常见的原因是：**组织没有设计出任何结构，来保证众多参与者各自看似合理的努力汇聚成一个合乎伦理的结果**。它解决的是"为什么好人也造出坏系统"。 – **直观解释（"它是什么？"）**：课程的原话很辛辣：如果真有一个大反派，那大概是因为你在看电影——电影才靠个体英雄与个体恶人推动叙事。更糟的是，我们生活在一个许多既有结构本身就不合伦理的世界里（它们歧视、延续不平等、剥夺自主），所以一个**完全不关心伦理的组织，会在自己的产品里复制周围社会的伦理失败**。 – **关键规则与最佳**

实践： – 面试时问："你们怎么做代码评审？"——这一个问题同时探测了 2.1（高质量过程）、2.2（技术知识共享）、2.4（专业反馈）。 – 继续问："你们如何评审新功能可能被滥用的方式？"、"设计新产品时你们会考虑并沟通哪些不同的人群或群体？"、"有没有哪一次你们**因为伦理原因**改变了计划中的功能？是怎么决定的？" – 如果对方答不上来，就要像听到"我们不需要代码评审，我们相信每个工程师都能写出好代码"一样警惕。 – 案例：Facebook 面向第三方开发者的 API 允许应用采集并滥用**使用该应用的用户的所有好友**的个人信息。没有任何一个工程师独自设计、构建、部署了这个 API。真正的问题是：当有工程师提出伦理质疑时，Facebook 这个组织是**被设计成放大并审查这些问题，还是压制它们**？ – 反面案例之后是正面案例：面对反疫苗错误信息，Pinterest 认为不能依赖算法来"提升真话、压低谎言"，于是停止返回数百个健康相关关键词的搜索结果，改为展示人工挑选的公共卫生来源。要问的是：**有多少员工参与了这个决策与实现**——数量本身就是"结构"的证据。

四种道德透镜（Four Moral Lenses） – **定义与目的**：这是本讲提供的可操作程序（procedure）。课程把它归纳为四个不同的视角，用来审视一个项目的正面与负面影响：**Outcomes（结果）**——项目的成本与收益；**Process（过程）**——这些成本与收益是**如何**达成的；**Structure（结构）**——好坏结果与过程的**模式（patterns）**；**Character（品格）**——把项目当作一个人来看待。它解决的是"我说不清楚哪里不对"的问题。 – **直观解释（"它是什么？"）**：它像一副四色滤光镜：同一张照片，换一片滤镜就看出一类问题。只看 Outcomes 会漏掉"用欺骗手段达成好结果"的问题；只看 Process 会漏掉"程序完全合规但结果灾难"的问题；Structure 让你从单个用户跳到"某一类人是否被系统性地伤害"；Character 让你问"如果这个系统是一个人，它是个什么样的人"。 – **关键规则与最佳实践**： – **四个透镜都要用**，不要只挑自己最顺手的那一个。 – 后果（Outcomes）透镜下要同时列成本与收益，并且问"成本落在谁身上、收益归谁"。 – 过程（Process）透镜关心手段：是否欺骗、是否未经同意、是否剥夺选择权。 – 结构（Structure）透镜关心模式：是否某一群体**系统性地**承受更差的错误率、更少的救济渠道。 – 品格（Character）透镜关心"这个项目/公司作为一个行为者，表现出什么品质"——慷慨、诚实、体贴，还是操纵、轻慢。 – 本讲的练习覆盖了典型场景：卖用户数据（结果与过程）、人脸识别对深色皮肤的准确率显著更差（结构上的不平等）、触屏对看不见屏幕的人不可用、车机触屏让驾驶员分心、餐厅平板在屏幕熄灭时用户看不懂于是"常亮 + 闪亮视频广告"（品格）、同事写了不可读的代码与不完整的规格导致别人误解（品格）、代码完全能跑但一次都没测试就提交到团队仓库（过程）。

透镜背后的伦理学传统：后果主义、义务论、美德伦理、社会契约（补充说明：课程原文只给出 Outcomes / Process / Structure / Character 四个透镜名称，以下是把它们与伦理学主要传统的对应关系整理出来，属于解说性补充，不是 6.031 原文的表述。） – **定义与目的**：四种透镜并非凭空而来，它们分别呼应伦理学中的四大传统，理解对应关系能让你在讨论中更快地定位分歧的性质。 – **直观解释（"它是什么？"）**： – **后果主义（consequentialism）↔ Outcomes 透镜**：一个行为的对错完全由它产生的后果（成本与收益）决定。它的力量在于关注真实的人受到的伤害；它的弱点在于难以比较不同人的收益，也容易为"多数人的好处"牺牲少数人。 – **义务论（deontology）↔ Process 透镜**：某些行为本身（欺骗、违约、把人当作纯粹手段）就是错的，不论后果多好。它解释了为什么"未经用户知情就收集数据"即使"服务因此更好"也仍然是错的。 – **美德伦理（virtue ethics）↔ Character 透镜**：不问"这个行为对不对"，而问"一个有德性的人/组织会怎么做"。它把项目当成行为者，考察诚实、正直、关怀、公正这些品质。 – **社会契约论与正义论（social contract /**

justice）↔ Structure 透镜：关注规则与制度如何系统性地分配利益与负担，关注某一类人是否被结构性地排除或损害。它解释了个体工程师都"没做错"而组织仍产出歧视性系统的机制。 – **关键规则与最佳实践**： – 当讨论卡住时，先判断分歧属于哪种传统：是后果估算不同，还是手段本身不可接受，还是结构性不公。 – 用 Outcomes 说服商人，用 Process 说服法务，用 Structure 说服公平性评审，用 Character 说服团队自己。 – 不要假装四个透镜总能给出一致结论：课程明确说"this reading does not prescribe a moral code"。 – 结构透镜最容易被忽略，也最难修——因为它要求改变流程，而不是改一行代码。 – 社会契约视角在本讲的具体落点是 ACM 的 3.7：融入社会基础设施的系统需要"special care"。

伦理测试（Ethical Testing）与设计评审中的伦理提问（Ethics in Design Review） – **定义与目的**：把伦理检查流程化：像写测试一样，尽可能悲观地设想你的作品会被如何滥用（misused）和恶用（abused）。它解决的是"伦理审查只在出事后才发生"的问题。 – **直观解释（"它是什么？"）**：课程把两种帽子并列：测试帽让你努力攻破自己的代码；伦理帽让你努力攻破自己的**意图**。和测试一样，伦理缺陷的修复很棘手——因为它牵涉的远不止系统和源码（还牵涉产品策略、激励、组织结构）。 – **关键规则与最佳实践**： – 在设计评审里固定留出"滥用场景"环节，并指定一个人扮演攻击者。 – 把"我们考虑过但决定不做"的伦理结论写进设计文档，作为 RFC 与 ETU 的一部分。 – 用 Reading 4（代码评审）的流程承载伦理评审：读规格、读代码、问"这段代码在什么输入下会造成伤害"。 – 关联 Reading 6（规格说明）与 Reading 7（设计规格）：**模糊的规格本身就是伦理风险**，因为调用者会按最方便自己的方式理解它。 – 追问"这个功能如果被一个恶意客户使用会怎样"，而不是只问"我们的用户会不会喜欢"。

举报与责任（Whistleblowing and Responsibility） （补充说明：6.031 原文没有出现 whistleblowing 这个词，但本讲的 ACM 准则第 1.2 条（避免伤害）、第 2.5 条（对系统影响的全面评估）、第 2.2 条（维持高标准的职业操守）以及"伦理结构"的讨论，自然会导出这个话题；以下内容是把它接回课程框架的延伸，不是原文表述。） – **定义与目的**：举报是指组织内部成员在内部渠道失效后，向外部披露严重危害公众利益的行为。它解决的是"当结构本身压制伦理问题时，个体还剩什么手段"。 – **直观解释（"它是什么？"）**：课程的问题"当有工程师提出伦理质疑时，Facebook 是被设计成放大这些问题，还是压制它们？"就是举报问题的上游：**好的结构让举报变得没必要**。 – **关键规则与最佳实践**： – 把举报当作**最后一招**：优先使用内部渠道、书面记录、把问题升级给有权限的人。 – 你的第一道防线是**留下书面记录**：把质疑写进评审意见、设计文档、issue 里，这样问题不再依赖某个人的记忆。 – 用具体条款说话（1.1、1.2、3.7），把"我不舒服"变成"我们违反了已承诺的义务"。 – 承认代价真实存在，同时承认 ACM 准则把公众安全置于雇主忠诚之上（1.1 与 1.2 位于 1.7 保密义务之前）。 – 最好的职业策略仍然是选择结构：面试时问"你们如何评审滥用风险"，就是在选择不必举报的环境。

伦理与三大目标的张力（Tension with SFB / ETU / RFC） – **定义与目的**：伦理要求常常和三大目标、和进度、和商业指标冲突，本讲要求你**识别**这种冲突而不是假装它不存在。 – **直观解释（"它是什么？"）**：典型冲突有四类： – **为进度牺牲安全性**：赶 deadline 时跳过测试、直接 `commit`，这正是本讲练习里 Charlie 的做法（代码能跑，但一次都没测就提交）——它伤害的不是抽象的"质量"，而是队友的时间与项目的正确性。 – **为可观测性牺牲隐私**：为了 SFB 和可调试性，把用户邮箱、IP 原样写进日志；日志的便利是收益，隐私损失是落在**非自愿的第三方**身上的成本。 – **为易理解性牺牲隐私/安全**：把配置写得"一眼看懂"，于是把密钥硬编

码进源码（这属于 ETU 的诱惑，却是安全与伦理的失败）。 – **为可修改性牺牲公平**：为了以后好改，把处置策略做成可配置开关，并**默认关闭**最保守的安全行为——Uber 自动驾驶案例中的设计正是"系统设计排除了紧急制动的激活"。 – **关键规则与最佳实践**： – 冲突要**显式记录**，不要靠"大家都懂"来消化。 – 让最保守（最不容易造成不可逆伤害）的行为成为**默认值**；把"关闭它"变成需要论证的决定。 – 把伦理取舍写进规格说明（Reading 6/7）：写清前置条件、后置条件，以及"在什么情况下系统必须拒绝服务"。 – 记住课程结论：正确、清晰、可修改只是**起点**；性能、安全、可用性、伦理都不能事后加装。

代码示例与对比分析

下面五组对比都取自本讲的真实议题：日志中的隐私、数据收集的范围、安全关键系统的默认值、算法偏见，以及"不可读的代码 + 不完整的规格"这一被本讲明确点名的伦理问题。每组都请用四种透镜各看一遍：**Outcomes** 是收益与成本怎么分配，**Process** 是这些收益与成本如何达成，**Structure** 是它形成了什么样的系统性模式，**Character** 是这个项目作为一个"人"表现出的品格。

场景 1：为了好排查问题，把用户邮箱和 IP 原样写进日志

❌ *错误代码*

```
import java.util.logging.Logger;

/** 用户活动的记录器。 */
public class ActivityLogger {

    private static final Logger LOG =
            Logger.getLogger(ActivityLogger.class.getName());

    /**
     * 记录一次用户活动。
     *
     * @param email  用户邮箱（身份标识）
     * @param ip     用户的 IP 地址（网络位置）
     * @param action 活动名称
     */
    public static void logActivity(String email, String ip, String action) {
        // 为了方便排查问题，把能拿到的东西全部写进日志
        LOG.info("user=" + email + " ip=" + ip + " action=" + action);
    }

    public static void main(String[] args) {
        logActivity("alice@example.com", "18.26.4.9", "view-profile");
    }
}
```

【错误代码的问题】 1. **隐私伤害是永久且不可撤回的**：日志一旦聚合、备份、导出到第三方分析平台，就再也收不回来；删除生产库里的记录并不会删除日志里的副本。 2. **日志系统通常不设访问控制**：日志往往对运维、实习生、外部监控服务都可见，实际访问范围远超"需要知道"的边界（对应 ACM 1.6 与 1.7）。 3. **它静默地扩大了系统的影响圈**：用户以为自己在和一个界面交互，实际上把自己的社交关系与网络位置交给了日志管

道下游的所有人。 4. **可测试性伪装成理由**：它看起来是"为 SFB 服务"，实际上用一个未经验证的便利假设，换取了不可逆的隐私成本。

✅ *正确代码*

```
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.util.logging.Logger;

/** 用户活动的记录器：只记录与运维目的直接相关、且无法反推身份的信息。 */
public class ActivityLogger {

    private static final Logger LOG =
            Logger.getLogger(ActivityLogger.class.getName());

    /** 部署时注入的随机盐，不要写死在源码里。 */
    private static final String SALT = System.getenv("ACTIVITY_LOG_SALT");

    /**
     * 记录一次用户活动。日志中只出现假名化的用户标识，不出现邮箱与 IP。
     *
     * @param userId 用户的内部 ID（不是邮箱、不是手机号）
     * @param action 活动名称，取值来自固定的枚举集合
     */
    public static void logActivity(String userId, String action) {
        LOG.info("user=" + pseudonymize(userId) + " action=" + action);
    }

    /**
     * 不可逆的假名化：同一用户在同一盐下可被关联，但无法从日志反推身份。
     *
     * @param userId 用户内部 ID
     * @return 十六进制摘要字符串
     */
    static String pseudonymize(String userId) {
        try {
            MessageDigest sha256 = MessageDigest.getInstance("SHA-256");
            byte[] digest = sha256.digest(
                    (SALT + userId).getBytes(StandardCharsets.UTF_8));
            StringBuilder hex = new StringBuilder(digest.length * 2);
            for (byte b : digest) {
                hex.append(Character.forDigit((b >> 4) & 0xF, 16));
                hex.append(Character.forDigit(b & 0xF, 16));
            }
            return hex.toString();
        } catch (NoSuchAlgorithmException e) {
            throw new IllegalStateException("SHA-256 应当始终可用", e);
        }
    }

    public static void main(String[] args) {
        logActivity("u-10237", "view-profile");
    }
}
```

【为什么这样更好】 日志仍然能完成它的本职工作（按用户维度关联行为、定位故障），但**不再携带可直接识别个人的信息**：邮箱与 IP 根本不进入这条链路，用户标识被加盐摘要成假名。同时，动作名称被限定为受控词表，避免有人为了"方便"把自由文本里的隐私内容一起塞进来。整个设计的默认值是"能少记就少记"，而不是"能多记就多记"。

【代码对比解说】 两段代码的功能差异极小，伦理差异极大——这正是本讲强调的重点：**伦理问题常常藏在参数类型和日志格式里，而不在算法里**。左版的 `email` 与 `ip` 是"看起来更有用"的字段，因为它们让调试变简单；右版用一个内部 ID 和一个摘要函数把同一件调试工作做完，代价是需要一次额外的用户 ID 传递。注意 `pseudonymize` 有意做成包级可见（`static`，无修饰符）以便测试——**可测试性与隐私并不冲突**，只要你在设计时就把它们放在一起考虑。还要注意右版并没有声称"绝对匿名"：加盐摘要是假名化（pseudonymization），不是匿名化（anonymization），因为掌握盐的人仍然可以枚举比对。诚实地说明这个边界，本身就是 Character 透镜下的品质。

【设计原则透视】 左版是典型的**表示暴露**：把内部数据（身份）原样穿过边界，写到一个不受 RI 保护的第三方存储里。右版给日志内容建立了一条**抽象边界**：外部只看到假名，真实身份的映射关系只存在于受控的地方。用 Reading 11（抽象函数与表示不变量）的语言说，日志格式有一个明确的 AF——"日志里的 user 字段表示某个用户，但只有配合外部映射才能确定是谁"；这条 AF 必须在注释里写清楚，否则未来的维护者会误以为它是邮箱。用 Reading 6（规格说明）的语言说，`logActivity` 的前置条件被收窄了（不接受邮箱），这比在后置条件里承诺"我们不会泄漏"要强得多——**限制输入比承诺行为更可靠**。此外，加盐值从环境变量读取而非硬编码，是 Reading 28 与安全实践的交汇点：源码会进版本库、会被 diff、会被贴到 issue 里（这也呼应 Reading 29 中"不要把密钥提交进仓库"）。

场景 2：默认收集一切，"以后也许用得上"

❌ *错误代码*

```
import java.util.HashMap;
import java.util.Map;

/** 用户资料：什么都能存，因为"以后也许用得上"。 */
public class UserProfile {

    private final Map<String, String> fields = new HashMap<>();

    /** 把上游传过来的所有字段照单全收。 */
    public void ingest(Map<String, String> everything) {
        fields.putAll(everything);
    }

    /** 分析用的导出：把原始数据原样交给第三方。 */
    public Map<String, String> exportForAnalytics() {
        return fields;                       // 内部表示直接交给了调用者
    }

    public static void main(String[] args) {
        UserProfile p = new UserProfile();
        Map<String, String> signup = new HashMap<>();
        signup.put("email", "alice@example.com");
        signup.put("phone", "+1-617-555-0100");
        signup.put("birthdate", "2001-03-14");
        signup.put("homeAddress", "77 Massachusetts Ave");
        signup.put("browsingHistory", "...");
        p.ingest(signup);
        System.out.println(p.exportForAnalytics().size());   // 5
    }
}
```

【错误代码的问题】 1. **过度收集（over-collection）**：注册流程并不需要生日、住址、浏览历史，但代码把它们全部收下——因为"以后也许用得上"。收集本身即风险，因为数据一旦存在就可能被泄漏、被传唤、被转卖。 2. **没有同意边界**：代码里没有任何地方表达"用户同意了什么"，所以没有任何机制能在未来阻止一个字段被加入导出。 3. **返回可变内部引用**：`exportForAnalytics` 直接把 `fields` 交出去，调用者可以清空或篡改它——这是 Reading 11 意义上的表示暴露，也是 Reading 24（队列/线程安全）意义上的并发隐患。 4. **成本落在用户身上，收益归组织**：这正是 Outcomes 透镜要你列清楚的那张表。

✅ *正确代码*

```
import java.util.Collections;
import java.util.HashMap;
import java.util.Map;
import java.util.Set;

/** 用户资料：只保存用户明确同意、且本次功能确实需要的字段。 */
public class UserProfile {

    /** 注册流程真正需要的字段，并且用户已经勾选同意。 */
    private static final Set<String> CONSENTED_FIELDS =
            Set.of("email", "displayName");

    private final Map<String, String> consented = new HashMap<>();

    /**
     * 保存一个字段——仅当它在同意范围内。
     *
     * @param field 字段名
     * @param value 字段值
     * @throws IllegalArgumentException 如果该字段不在用户同意范围内
     */
    public void put(String field, String value) {
        if (!CONSENTED_FIELDS.contains(field)) {
            throw new IllegalArgumentException("未获得同意，不得收集字段: " + field);
        }
        consented.put(field, value);
    }

    /** 分析用的导出：只读视图，且只包含最小必要字段。 */
    public Map<String, String> exportForAnalytics() {
        return Collections.unmodifiableMap(new HashMap<>(consented));
    }

    public static void main(String[] args) {
        UserProfile p = new UserProfile();
        p.put("email", "alice@example.com");
        p.put("displayName", "alice");
        try {
            p.put("browsingHistory", "...");   // 立刻失败，而不是悄悄收集
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

【为什么这样更好】 三条防线被同时建立：(1) **白名单**——只有 `CONSENTED_FIELDS` 里的字段能进入对象，任何越界尝试立刻抛出异常，而不是安静地存下来；(2) **只读导出**——`Collections.unmodifiableMap(new HashMap<>(consented))` 既做了防御性拷贝又做了不可变包装，第三方拿到的东西无法反向污染内部状态；(3) **显式失败优于静默接受**——`IllegalArgumentException` 让"收集了不该收集的数据"变成一个会在开发阶段就暴露的缺陷，而不是一个三年后才被记者发现的丑闻。

【代码对比解说】 左版的 API 形状是 `ingest(Map<String,String> everything)`：它把决定权交给调用者，自己只做搬运。右版的 API 形状是 `put(String field, String value)` 加一条白名单：**它把伦理判断编码进了接口**。这就是本讲"伦理结构"在代码层面的样子——不是靠每个工程师每次记得多想一步，而是靠接口本身让违规变得不可能（或者至少变得刺眼）。右版还有一个容易被忽略的收益：`CONSENTED_FIELDS` 是一个集中、可审查、可被法务与产品共同确认的清单，它在代码评审中一眼可见；左版则没有任何地方可以让人问"我们到底收集了什么"。

【设计原则透视】 这是 Reading 11（AF/RI）最直接的伦理应用：右版有一条明确的**表示不变量**——"`consented` 的键集合必须是 `CONSENTED_FIELDS` 的子集"，并且 `put` 在入口处维护它；左版的 RI 实际上是空集，因为什么都能进。同时它体现 Reading 8（不可变性）与 Reading 24（队列，防御性拷贝）的原则：跨越抽象边界时要么传不可变对象，要么传副本。**用伦理语言重述技术规则**：RI 保护的是"表示的正确性"，白名单保护的是"收集行为的正当性"，两者的实现手法完全相同。最后，`Set.of` 创建的是不可变集合（Java 9+；若要兼容更早版本可用 `Collections.unmodifiableSet(new HashSet<>(Arrays.asList(...)))`）——让同意清单本身也无法被运行时改写。

场景 3：安全关键系统的默认值——把紧急处置责任交给"操作员"

❌ *错误代码*

```
/** 自动驾驶的紧急制动子系统。 */
public class EmergencyBraking {

    /** 碰撞时间阈值（秒）。 */
    private static final double BRAKE_THRESHOLD_SECONDS = 1.2;

    /**
     * 遇到障碍时的处理。
     *
     * @param distanceMeters 与障碍物的距离（米）
     * @param speedMps       当前车速（米/秒）
     */
    public void onObstacle(double distanceMeters, double speedMps) {
        double timeToCollision = distanceMeters / Math.max(speedMps, 1e-9);
        if (timeToCollision <= BRAKE_THRESHOLD_SECONDS) {
            // 为了减少"车辆行为异常"的可能，系统不自行紧急制动，
            // 而是提示操作员介入
            System.out.println("ALERT: operator must brake");
        }
    }
}
```

【错误代码的问题】 1. **把最危险的默认值当成最安全的默认值**：代码承认它**已经识别出碰撞即将发生**（`timeToCollision` 已经算出来了），却选择不采取唯一能减轻伤害的动作。这正是 sp21 原文引用的 NTSB 初步报告措辞："emergency braking maneuvers are not enabled while the vehicle is under computer control, to reduce the potential for erratic vehicle behavior"。 2. **隐含假设从未被验证**：它假设操作员在场、专注、且反应时间足够。而在现实事故中，唯一被自动驾驶汽车撞死的行人，恰恰是在软件最终判断需要

紧急制动时死去的——因为系统**没有被编程为自行制动**。sp22 版本引用了 NTSB 最终报告更严厉的说法："The system design precluded activation of emergency braking for collision mitigation, relying instead on the operator's intervention to avoid a collision or mitigate an impact." 3. **权衡没有被当作规格决定记录下来**：这个选择（安全 vs 平顺）本应由产品、法务、安全团队共同签署，却在代码里表现为一条注释。 4. **不可逆伤害 vs 可逆不适**：把"乘客觉得刹车突兀"和"行人死亡"放在同一个天平上，却没有说明为什么前者更重。

✅ *正确代码*

```
/**
 * 自动驾驶的紧急制动子系统。
 *
 * 设计决策（见 design-review-2024-03.md）：当碰撞时间低于阈值时，
 * 系统自行制动，不依赖操作员是否在场。安全关键系统的默认行为是
 * 失效安全（fail-safe）：不确定时选择伤害更小的动作。
 */
public class EmergencyBraking {

    private static final double BRAKE_THRESHOLD_SECONDS = 1.2;

    /** 系统自行介入的次数，供事后审计与安全报告使用。 */
    private int interventionCount;

    /**
     * 遇到障碍时的处理。
     *
     * @param distanceMeters 与障碍物的距离（米）
     * @param speedMps       当前车速（米/秒）
     */
    public void onObstacle(double distanceMeters, double speedMps) {
        double timeToCollision = distanceMeters / Math.max(speedMps, 1e-9);
        if (timeToCollision <= BRAKE_THRESHOLD_SECONDS) {
            applyBrakes();
            interventionCount += 1;
            System.out.println("EMERGENCY BRAKE ttc=" + timeToCollision);
        } else {
            notifyOperator(distanceMeters, speedMps);
        }
    }

    /** 驱动制动器。 */
    private void applyBrakes() {
        // 与车辆总线的接口略
    }

    /** 在还有充足余量时提醒操作员。 */
    private void notifyOperator(double distanceMeters, double speedMps) {
        System.out.println("ADVISORY distance=" + distanceMeters);
    }

    /** 供安全审计使用：系统在多少次险情中自行介入。 */
    public int getInterventionCount() {
        return interventionCount;
    }

    public static void main(String[] args) {
        EmergencyBraking braking = new EmergencyBraking();
        braking.onObstacle(5.0, 20.0);    // 提醒
        braking.onObstacle(10.0, 20.0);   // 紧急制动
        System.out.println(braking.getInterventionCount());   // 1
```

```
    }
}
```

【为什么这样更好】 第一，**动作与识别对齐**：既然系统已经算出碰撞时间不足，它就必须做出与这一认知相称的动作，否则"识别"本身只是把责任转嫁给一个可能根本不在场的人。第二，**默认值站在伤害更小的一边**：不确定时选择制动，而不是选择"什么都不做"。第三，**可审计**：`interventionCount` 让"系统干预了多少次"成为可报告、可回归测试的事实——它把一次伦理判断变成了一个可持续检验的指标。第四，**权衡被写进文档并在注释里指向它**，这样未来的维护者能看见"这是一个被讨论过的决定"，而不是一条不知来由的注释。

【代码对比解说】 两段代码的差别只有三行：`applyBrakes()` 被调用而不是被跳过、计数器加一、文档注释指向一份设计评审记录。但它们的伦理性质完全不同。左版是一个**不可逆伤害的默认值**，并且它把一个未经验证的假设（"操作员会介入"）编码成了系统行为；右版把同样的信息（碰撞时间）用于驱动最保守的动作。注意右版并没有删除 `notifyOperator`，而是把它移到"还有余量"的分支里——伦理设计通常不是禁止某个功能，而是**调整它在决策树中的位置**。另外，右版保留了完整的阈值常量并让它是 `static final`，方便在被质疑时快速定位与复现计算。

【设计原则透视】 这是**规格说明**层面的伦理（Reading 6/7）：左版的规格隐含地写着"本方法只负责报警"，这个后置条件在正常工况下无懈可击，在生死工况下是灾难；右版的规格把"在 TTC ≤ 阈值时必须使车辆减速"写成了硬性后置条件。它同时体现了 Reading 8（不可变性/状态最小化）与 Reading 9（避免调试）的反面：左版没有任何可观测状态，事故后无法回答"系统当时判断了什么"；`interventionCount` 给了调查者一个观测点。最后，它与"伦理结构"直接呼应：一条 `if` 分支的默认方向，是组织把风险推给谁的具体体现——**代码里的默认值就是组织价值观的落地形式**。

场景 4：人脸解锁的阈值——只看整体平均错误率

❌ *错误代码*

```
import java.util.ArrayList;
import java.util.List;

/** 人脸解锁的阈值选择：只看整体平均错误率。 */
public class FaceUnlock {

    /**
     * 从冒名者得分中挑一个阈值。
     *
     * @param impostorScores 所有测试者的冒名者得分
     * @return 使整体错误率最低的阈值
     */
    public static double chooseThreshold(List<Double> impostorScores) {
        double sum = 0.0;
        for (double s : impostorScores) {
            sum += s;
        }
        double mean = sum / impostorScores.size();
        return mean + 0.05;      // 在平均得分之上留一点余量
    }

    public static void main(String[] args) {
        List<Double> scores = new ArrayList<>();
        for (int i = 0; i < 1000; i++) {
            scores.add(0.10 + (i % 50) * 0.002);
        }
        System.out.println(chooseThreshold(scores));
    }
}
```

【错误代码的问题】 1. **平均值掩盖系统性差异**：如果识别对不同肤色的用户误差差异很大，正负误差在求平均时会互相抵消，得到"整体表现良好"的假象，而某一群体始终被拒或被误认。 2. **无法回答公平性问题**：代码里根本不存在"群体"这个概念，所以任何公平性审计都无从下手——你连数据都没有。 3. **把伦理问题伪装成技术指标问题**：`chooseThreshold` 的签名暗示这是一个纯数值优化，于是没人会在评审会上问"这对谁更差"。 4. **标签化风险**：误拒让用户被锁在自己的设备外，误认则让攻击者进入——两种错误在不同的群体上代价不同，而平均指标对二者一视同仁。

✅ *正确代码*

```java
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/** 人脸解锁的阈值选择：验收标准是"最差子群体"，不是整体平均。 */
public final class FaceUnlock {

    /** 合格线：任何一个子群体的错误率都不得超过它。 */
    public static final double MAX_GROUP_ERROR_RATE = 0.01;

    /** 评估函数：给定阈值，给出各子群体的错误率。 */
    public interface Evaluator {
        Map<String, Double> at(double threshold);
    }

    /**
     * 返回最差子群体的错误率。
     *
     * @param groupErrorRates 子群体名称到错误率的映射
     * @return 最大的错误率（没有子群体时返回 0.0）
     */
    public static double worstGroupErrorRate(Map<String, Double> groupErrorRates) {
        double worst = 0.0;
        for (double rate : groupErrorRates.values()) {
            worst = Math.max(worst, rate);
        }
        return worst;
    }

    /**
     * 选择一个满足所有子群体要求的阈值。
     *
     * @param candidates 候选阈值，按偏好顺序排列
     * @param evaluator  评估函数
     * @return 第一个通过"最差子群体"验收的阈值
     * @throws IllegalArgumentException 如果没有任何候选阈值合格
     */
    public static double chooseThreshold(List<Double> candidates,
                                         Evaluator evaluator) {
        for (double threshold : candidates) {
            Map<String, Double> rates = evaluator.at(threshold);
            if (worstGroupErrorRate(rates) <= MAX_GROUP_ERROR_RATE) {
                return threshold;
            }
        }
        throw new IllegalArgumentException("没有阈值能通过最差子群体验收");
    }

    public static void main(String[] args) {
        Evaluator evaluator = threshold -> {
            Map<String, Double> rates = new HashMap<>();
            rates.put("groupA", 0.002 / threshold);
            rates.put("groupB", 0.003 / threshold);
```

```
            rates.put("groupC", 0.008 / threshold);
            return rates;
        };
        System.out.println(chooseThreshold(
                List.of(0.20, 0.30, 0.40, 0.60, 0.80), evaluator));   // 0.8
    }
}
```

【为什么这样更好】 验收标准从"平均错误率"换成了**最差子群体错误率（worst-group error rate）**：一个阈值只有在**每一个**子群体上都达标才算合格。这条规则的作用是结构性的——它把"某些人体验更差"从统计噪声变成了会阻塞发布的硬性条件。为了让这条规则可执行，代码必须显式地把"子群体"作为一等概念（`Map<String, Double>` 的键），并要求评估函数按群体报告，而不是吐出一个总数。当没有任何候选阈值能通过时，系统**明确失败**（抛异常），而不是悄悄退回到一个会伤害某一群体的默认值。

【代码对比解说】 左版的签名 `chooseThreshold(List<Double>)` 是"给我数字，我给你数字"，它在接口层面就抹掉了公平性；右版把 `Evaluator` 作为参数注入，是因为**评估本身需要按群体切分数据**，这件事不能藏在被调函数里假装不存在。注意右版仍然是一个"贪心地按偏好顺序取第一个合格阈值"的简单策略——伦理改进通常不是更复杂的算法，而是**换一个验收标准**。还要注意 `MAX_GROUP_ERROR_RATE` 是一个具名常量并附有注释，这让它在代码评审里可被质疑、可被修改、可被测试引用；把它藏进某个魔法数字里，就等于把公平性决定藏了起来。

【设计原则透视】 这是**规格说明（Reading 6/7）**中"前置条件/后置条件应当可检验"的伦理版本：`worstGroupErrorRate <= MAX_GROUP_ERROR_RATE` 是一句能被自动化测试直接检查的后置条件，而"识别要公平"不能被检查。它也对应 **Structure 透镜** 的核心：公平问题的本质是"某一类人系统性地承受更差的模式"，所以必须用按群体分组的做法把它暴露出来。从 Reading 11（AF/RI）的角度看，`Evaluator` 是一个清晰的抽象边界：它只承诺"给定阈值返回各群体错误率"，把数据来源、样本量、分组定义都留给实现，这样将来加入新的敏感属性（性别、年龄、口音）不需要修改 `chooseThreshold`。补充说明：真实系统还会同时报告误拒率与误认率，并记录样本量，因为一个只有 3 个样本的子群体上的"0% 错误率"毫无意义。

场景 5：不可读的代码与不完整的规格——本讲点名的"Alice 与 Bob"问题

❌ *错误代码*

```
import java.util.ArrayList;
import java.util.List;

public class Datalist {
    private List<String> l = new ArrayList<>();

    // x 是什么？返回的是什么？会不会改到传入的列表？
    // n 的取值边界在哪里？调用者完全不知道。
    List<String> f(List<String> x, int n) {
        List<String> r = new ArrayList<>();
        for (int i = 0; i < x.size(); i++) {
            if (x.get(i).length() >= n) {
                r.add(x.get(i));
            }
        }
        return r;
    }
}
```

【错误代码的问题】 1. **规格缺失导致调用者按自己的猜测行事**：本讲的练习正是这样描述的——Alice 写了一个方法，代码不可读、规格不完整，队友 Bob 即将写调用它的代码，于是误解了它的行为（本讲练习给出的透镜是 Character：Alice 的行为表现出的不是"技术能力不足"，而是对同事不够负责的品格）。 2. **边界行为未定义**：`n` 为负数时会发生什么？返回的列表调用者能不能改？参数列表会不会被修改？这些都不是"细节"，而是别人据此写代码的全部依据。 3. **命名抹掉了语义**：`Datalist`、`f`、`l`、`x`、`n`、`r` 全是单字母或无语义的词，代码无法自我解释，读者只能靠反推。 4. **未使用的可变字段**：`private List<String> l` 从未被使用，读者会怀疑它是否隐含某种状态语义——这让 ETU 进一步恶化。

✅ *正确代码*

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/** 只做筛选、不修改输入的字符串列表工具。 */
public class StringFilter {

    /**
     * 返回 strings 中长度至少为 minLength 的元素，保持原有顺序。
     *
     * @param strings   待筛选的字符串；本方法不修改它，调用者之后仍可修改它
     * @param minLength 保留的最小长度，必须 >= 0
     * @return 新的不可变列表，与 strings 不共享任何可变状态
     * @throws IllegalArgumentException 如果 minLength < 0
     */
    public static List<String> atLeast(List<String> strings, int minLength) {
        if (minLength < 0) {
            throw new IllegalArgumentException("minLength: " + minLength);
        }
        List<String> result = new ArrayList<>();
        for (String s : strings) {
            if (s.length() >= minLength) {
                result.add(s);
            }
        }
        return Collections.unmodifiableList(result);
    }
}
```

【为什么这样更好】 第一，**规格完整**：Javadoc 写清了前置条件（`minLength >= 0`）、后置条件（返回新列表、保持顺序、不修改输入）、返回值是否可变（不可变）、以及违反前置条件时的行为（抛异常）。Bob 不需要读实现就能正确调用它。第二，**命名承载语义**：`StringFilter.atLeast` 让调用点 `StringFilter.atLeast(names, 3)` 读起来接近自然语言。第三，**防御性且不可变**：参数被当作只读来用，返回值被包装成不可变，消除了"谁改了谁的列表"这类经典困惑（Reading 8、Reading 24）。第四，**边界被显式处理**：负数长度立刻失败，而不是产生一个"看起来正常"的空列表。

【代码对比解说】 两版的功能在正常输入下完全一致，区别在于**别人能否安全地依赖它**。本讲把它归到 Character 透镜，因为这里的失败不是"结果变坏了"（Outcomes），也不是"流程被跳过了"（Process），而是**一个人对同事的态度**：写不可读的代码、留不完整的规格，等于把自己的方便建立在别人的时间上。这也解释了为什么本讲会把"代码质量"放进伦理讨论：团队里的代码是共享物，低质量的共享物是一种（轻度的）不尊重。值得强调的是，右版的改进**没有增加任何复杂度**：它只是把原本存在于 Alice 头脑中的信息写了下来。

【设计原则透视】 这是 Reading 6（规格说明）与 Reading 7（设计规格）的直接应用：**规格是调用者与实现者之间的合同**，规格含糊就等于合同漏洞，而漏洞的成本总是由调用者承担。它也对应 Reading 11（AF/RI）：`Collections.unmodifiableList` 明确划定了抽象边界，返回对象不允许被外部改写，因此实现者可以放心地在将来改变内部实现。用 Reading 4（代码评审）的话说，这类代码在评审里应该被直接拦下——评审者最该问的问题不是"这段代码能不能跑"，而是"别人读了能不能用对"。补充一句：本讲另一个练习（"Commit it"）与此互补——Charlie 写了一个完全能工作的方法，**一次都没测试**就提交到团队仓库；那里用

的是 **Process** 透镜，因为代码的产出本身没问题，错的是他跳过了团队约定的过程，把不确定性留给了队友。

与其他设计原则的关联

- **与 Reading 4（代码评审 Code Review）**：本讲反复建议把"代码评审"作为探测组织伦理结构的探针（面试问题"你们怎么做代码评审"），因为评审是唯一一个"有人会认真读你的规格、并问这段代码会被怎样滥用"的固定环节。伦理评审不是另开一个会，而是给现有评审加一段议程。
- **与 Reading 5（版本控制 Version Control）/ Reading 29（团队版本控制 Team Version Control）**：本讲练习中 Charlie 的做法（代码能跑、从不测试、直接提交）在 Reading 29 的团队准则里被逐条反驳：Review what you commit、Run the tests、Don't use `commit -a`。伦理要求"不要给队友制造清理垃圾的时间"，而版本控制纪律是它的工程化落地。同时，Reading 29 里"不要把编译产物和密钥提交进仓库"也直接服务于本讲的隐私与保密义务（ACM 1.6、1.7）。
- **与 Reading 6/7（规格说明与设计规格 Specifications / Designing Specifications）**：伦理取舍必须写成规格——前置条件、后置条件、异常行为。模糊的规格是伦理风险，因为它把解释权交给了最方便的一方。Scenario 5 就是这一点的教科书案例。
- **与 Reading 8/9（不可变性与避免调试 Immutability / Avoiding Debugging）**：不可变对象与防御性拷贝减少"谁改了谁的数据"这类事故，同时减少为了排查事故而过度记录用户数据的需求——**更少的状态意味着更少的监控冲动**，间接减轻隐私压力。
- **与 Reading 11（抽象函数与表示不变量 AF/RI）**：本讲的三组对比都可以用 AF/RI 的语言重写：日志的假名化是给日志内容定义一条诚实、受限的 AF；同意白名单是一条表示不变量；`unmodifiableList` 是抽象边界的守卫。**伦理约束在代码里就是 RI**。
- **与 Reading 13（调试 Debugging）**：事故复盘（如 NTSB 报告）依赖系统留下的可观测痕迹。Scenario 3 加入 `interventionCount` 正是为了让"系统当时判断了什么"可被事后检验——这与 Reading 9/13 的"避免调试"原则一脉相承，只是服务对象从工程师扩展到了公众调查者。
- **与 Reading 21/23（并发与互斥 Concurrency / Mutual Exclusion）**：`exportForAnalytics` 返回可变内部引用在单线程下是表示暴露，在并发下是数据竞争；跨线程共享的数据还会引出"日志聚合服务能否看到个人数据"这类新的隐私边界。补充说明：Java 中可用 `Collections.unmodifiableMap` 或 `Map.copyOf`（Java 10+）实现不可变快照。
- **与 Reading 22（Promise）/ Reading 25（Sockets and Networking）**：一旦数据离开进程边界（网络请求、第三方分析 SDK），隐私与同意义务的复杂度会陡增；本讲的"扩展的影响圈"在分布式场景下会扩大好几层。
- **与 Reading 29（团队版本控制）互为前后**：Reading 29 讨论团队如何协作产出可维护的代码，本讲讨论这次协作对**组织之外的人**意味着什么。先读 29 再读 28，是"我们怎么一起工作"到"我们的工作对谁负责"的自然递进。

关键要点

- **伦理性是与 SFB/ETU/RFC 并列的软件属性，而且不能事后加装**：请在每一轮迭代里就把它作为验收项，而不是上线前的合规检查。

- **用四种透镜逐个扫描，别只用最顺手的那一个**：Outcomes（成本与收益归谁）、Process（手段是否正当）、Structure（是否形成系统性伤害模式）、Character（这个项目像什么样的人）。
- **把伦理判断编码进接口、默认值和 RI**：白名单、失效安全的默认值、不可变返回、明确的异常——代码结构比个人自觉更可靠，这也是"伦理结构而非伦理个体"的工程含义。
- **规格即责任**：写出完整的前置条件、后置条件和异常行为，是对同事、对用户、对未来的自己的最低义务；本讲的 Alice/Bob 与 Charlie 两个练习分别对应"写不清楚"和"没验证就交付"。
- **在做任何取舍时留下书面记录**：把"我们考虑过、为什么这样决定"写进设计文档，让权衡成为可审查的产物——它是未来唯一能证明"我们当时认真想过"的东西。

常见陷阱与注意事项

1. **把伦理当成"公关/合规问题"、以为它能像普通 bug 一样在最后集中修复** → 后果：伦理检查发生在功能冻结之后，任何修改都变成"来不及了"，所有问题都被降级为文档免责声明；而且这类缺陷往往牵涉产品策略、激励与组织结构（本讲原话："it involves so much more than just the system and its source code"），越晚发现修复代价越高，甚至根本不可能由工程师单独修复。正确做法是在设计评审阶段就用四个透镜提问。
2. **只做后果计算，忽略过程与结构** → 后果：用"总体收益更大"为欺骗性数据收集、或为某一群体系统性更高的错误率辩护；讨论永远无法收敛，因为双方在用不同透镜说话。
3. **相信"只要雇好人、走对路就没事"** → 后果：组织复制周围社会的伦理失败。本讲的反问很到位：如果面试官说"我们不需要代码评审，我们相信每个工程师"，你会警惕；那么对"我们没有评审滥用风险的流程"也应该同样警惕。
4. **为了让代码"能跑、好调试"而顺手记录过多用户数据** → 后果：把不可逆的隐私成本当作可回滚的工程决定（Scenario 1）；日志一旦扩散就收不回来。
5. **在安全关键系统中把最保守的行为设为默认关闭** → 后果：直接对应 Uber 自动驾驶事故——系统看到了危险，却被设计成不采取行动；"减少车辆行为异常"这种可逆的不适被拿来与不可逆的人身伤害比较，而没有人为此签署决定（Scenario 3）。
6. **用整体指标代替分群体指标** → 后果：平均值把系统性差异抹平，公平性问题在通过验收的报表里彻底消失（Scenario 4）。补充说明：真实评估必须同时报告各群体的样本量，否则小样本上的"完美指标"会误导决策。

思考题（带答案）

问题 1：你的软件在用户不知情的情况下记录用户活动并把数据卖给第三方。请分别用 Outcomes、Process、Structure、Character 四个透镜说明这件事为什么是坏的；如果只能用一句话向产品经理说明，你会选哪个透镜，为什么？

答案：**Outcomes 透镜**——用户承担了隐私暴露的成本（可能的骚扰、歧视、要挟），而收益全部归公司；成本与收益的分配不对称。**Process 透镜**——问题不只是"数据被卖了"，而是"在用户不知情、未同意的情况下被卖了"：手段本身（隐瞒）就是错的，即使服务因此变得更好也不能洗白。**Structure 透镜**——这种做法的真正危害在于它形成的模式：默认收集、默认共享、默认不告知，久而久之整个行业把"用户的数据属于数据管道"当作常态，个人无论怎么小心都无法退出。**Character 透镜**——把项目当成一个人看：它偷偷拿走你的

东西去换钱，这是不诚实、不尊重人的品格，与"这个产品是否好用"无关。（补充说明：课程原文此处为练习题，没有公布答案；以上是依据四个透镜定义给出的推理。就本题而言，把重点放在 Outcomes 或 Process 上都是站得住的，选择取决于你要说服谁。如果只能说一句话给产品经理，通常选 Outcomes 最有效——因为"成本落在用户身上、收益归我们"是商业语言；而对工程与法务团队，Process（未经同意的收集本身就不可接受）更能形成硬约束。真正的教学点是：**四个透镜都说一遍，才能判断这个决定在每个维度上是否都站得住**。）

问题 2：本讲用 Alice（不可读的代码与不完整的规格，导致同事误解）和 Charlie（代码能跑但一次都没测试就提交到团队仓库）说明了两种不同的失职。请指出各自对应的道德透镜，并解释为什么它们不是同一个透镜。

答案：Alice 对应 **Character**——她的失败不在于产出了坏结果（Outcomes），她的方法也并没有违反某条流程（Process）；问题在于她作为一个协作者所表现出的品格：把自己的方便建立在同事的时间与困惑之上，对自己的共享物缺乏责任感。Charlie 对应 **Process**——他的代码"perfectly working"（本讲原文措辞），所以从结果看暂时没有伤害任何人；他错在**跳过了团队约定的过程**：不测试就提交，等于把自己的不确定性转嫁给队友，让"随时可以拉取的仓库"变成一个不可信的起点。两者之所以不是同一个透镜，是因为它们指向不同的修复手段：Character 问题靠文化、评审规范与共同标准来改（"我们这里不允许别人看不懂的代码"），Process 问题靠流程与自动化来改（提交前必须跑测试、自动化构建通过才能推送，对应 Reading 29 的"Run the tests"与"Automate"）。这也正好说明为什么四个透镜缺一不可：如果只用 Outcomes，Charlie 在 bug 暴露之前都不会被认为做错了任何事。

问题 3：一个公司要上线"人脸解锁"功能，整体识别准确率 99.2%，但在深色皮肤用户上的误拒率是其他用户的 6 倍。技术负责人说："我们有 99.2% 的准确率，而且我们已经发布了隐私政策。"请用本讲的概念指出这句话的两个问题，并给出至少两条可执行的设计改动。

答案：**两个问题**。第一，"整体准确率"这个指标本身就是选择过的（Outcomes/Structure 透镜）：把 6 倍的差异平均掉，等于用多数群体的良好表现掩盖少数群体承受的系统性伤害；本讲的练习明确指出"准确率对深色皮肤用户显著更差"是要用**结构**透镜去看的问题——它是模式性的，不是一个随机误差。第二，"我们发布了隐私政策"是**过程合格**的辩护，但过程上的合规声明不能替代对具体伤害的评估（本讲关于"伦理结构"的讨论正是要拆掉这种"我们已经走了流程所以没问题"的自我安慰；"有没有隐私政策"与"这个系统是否公正地工作"是两个不同的问题）。**可执行的设计改动**：(1) 把验收标准从"整体准确率"改成"最差子群体错误率不超过阈值"，并在没有阈值合格时**阻止发布**（对应 Scenario 4 的 `MAX_GROUP_ERROR_RATE` 与显式抛出异常）；(2) 为误拒率高的群体提供不依赖生物特征的降级路径（PIN、备用设备），并在设计文档中记录该降级路径作为后置条件；(3) 把分群体评估纳入持续集成，让每次模型更新都必须重新通过最差群体验收，而不是一次性的人工评估。补充说明：还要记录每个子群体的样本量，并让评估所用的分组定义公开可审查——否则你无法证明"达标"不是靠样本稀薄换来的。

Reading 29: 团队版本控制（Team Version Control）

> 说明：本讲 sp22 原版使用 TypeScript，本笔记按用户要求提供 Java 代码示例；类型/API 与 sp21（6.031 Java 版）原文保持一致。本讲以 Git 命令行为主，因此代码对比大量使用 `bash 与 text` 代码块；凡涉及"合并后代码会变成什么样"的地方，仍然给出 Java 源码，这样才能看清"自动合并成功但语义坏掉"的过程。

概述

前面几周你做问题集时，虽然一直在用 Git，但很少需要和**同时间往同一个仓库 push/pull 的其他人**协调；进入小组项目之后，这件事变成日常。本讲的目标只有两条：复习 Git 基础与提交图（commit graph），以及练习多用户的 Git 场景。它给出的核心结论是：团队版本控制的成败不取决于你记住多少命令，而取决于一套协作纪律——**沟通、写规格、写测试、跑测试、自动化、提交前审查、开工前先拉取、收工前同步**。本讲与三大目标的关系非常直接：**Safe from bugs** 靠"提交前跑测试 + 自动化构建"来保证（"it worked on my machine"从此不再成立）；**Easy to understand** 靠可读的提交图与有意义的提交信息来保证（历史是写给未来的人看的沟通记录）；**Ready for change** 靠小的原子提交、频繁同步与可回滚的历史来保证（任何一次改动都能被定位、被理解、被撤销）。课程还明确给出了一条反直觉的建议：在 6.031 这种 1–2 周、3 人规模的项目里，**不要**使用分支或变基（branching / rebasing），把精力放在清晰沟通与频繁同步上；分支的价值在项目更大、周期更长、人更多时才显现。

核心概念与设计原则详解

提交图（Commit Graph / DAG） – **定义与目的**：Git 仓库中记录的历史是一个**有向无环图（directed acyclic graph, DAG）**。每个节点是一个提交（commit，也叫 version / revision），即项目所有文件在那一刻的完整快照；每个提交有唯一的十六进制 ID。它解决的是"我们如何知道自己从哪儿来、两条改动线在哪里分开又在哪里合上"。 – **直观解释（"它是什么？"）**：任何分支（例如默认的 `main`）的历史都从某个初始提交开始，然后可能**分叉**（多个开发者并行改动，或同一个人在两台机器上工作而中间没有 commit–push–pull），再**合回来**。课程要求你能指着 `ex05-hello-git` 的输出，说清 `main` 的历史在哪里分开、在哪里合上：`text * b0b54b3 (HEAD, origin/main, origin/HEAD, main) Greeting in Java * 3e62e60 Merge |\ | * 6400936 Greeting in Scheme * | 82e049e Greeting in Ruby |/ * 1255f4e Change the greeting * 41c4b8f Initial commit` 这里 `3e62e60` 是**合并提交（merge commit）**，它有两个父提交；`|\` 与 `|/` 之间就是分叉与回合。 – **关键规则与最佳实践**： – 图形输出来自 `git log --graph --oneline --decorate --all`；课程使用的 `git lol` 就是它的别名（可用 `git config --global alias.lol "log --graph --oneline --decorate --all"` 配置）。 – 括号里的 `HEAD`、`main`、`origin/main` 是**引用（refs）**：`HEAD` 是你当前所在的位置，`main` 是本地分支，`origin/main` 是上次与远程通信时远程分支的位置。 – 不需要背下 `Pro Git` 2.3 节列出的所有选项，只要知道"什么是可能的"，需要时再去查。 – 能把"分叉—合并"讲清楚，是理解后面所有冲突问题的前提。

合并、合并冲突与"先拉取再开工"（Merging, Merge Conflicts, Pull Before You Start Working） – **定义与目的**：合并（merge）把两条历史线的改动结合起来。Git 能自动合并**不同位置**的改动；当两边改了**同一处**

时，它无法替你决定，于是报告**合并冲突（merge conflict）**。它解决的是"并行工作如何安全地汇合"。 – **直观解释（"它是什么？"）**：本讲用三组练习把这件事拆开（下面思考题会给出答案）： – **不同方法的改动**：Alice 改 `greet(..)`，Bob 改 `greeting()`，Git 自动合并，结果正确。 – **同一行被两边改动**：Alice 把 `return "Hello";` 改成 `return "Ciao";`，Bob 把它改成 `return "Hello, ";`——同一行、两个不同结果，Git **无法**自动合并。 – **接口约定悄悄变了**：Alice 把 `greet(..)` 从"打印"改成"返回字符串"，Bob 新写了一个调用它的 `Main.java`。Git 自动合并**成功**，编译也**通过**，但运行起来什么都不打印——**合并成功不等于语义正确**。 – **关键规则与最佳实践**： – **开工前先 pull**：否则你的起点是旧版本，你必然要在以后合并，而且很可能要花时间解冲突。 – 合并后必须**重新运行测试**——Git 保证的是文本层面不重叠，不是语义层面不打脸。 – 接口变更（签名、返回值、语义）要**事先沟通**，这是本讲第一条团队准则"Communicate"的具体含义。 – 冲突不是灾难，而是一次"需要人来做的决定"；解冲突时必须理解**双方各自的意图**，而不是随便删掉一边。 – 永远不要为了图快用 `push --force` 覆盖别人的提交——那是在用别人的工作换自己的方便。

八条团队准则（Team Guidelines for Version Control） – **定义与目的**：每个团队都会形成自己的版本控制标准，规模与周期是主要影响因素。课程给出了适用于 6.031 这种小规模团队项目的八条准则，它们解决的是"如何用纪律替代运气"。 – **直观解释（"它是什么？"）**：逐条是：**Communicate（沟通）**——告诉队友你**要**做什么、**正在**做什么、**做完**了什么，这是避免别人花时间清理坏代码的最佳方式；**Write specs（写规格）**——这是 6.031 在意的东西，也是沟通的一部分；**Write tests（写测试）**——不要等代码堆成山才开始测，也不要一个人写测试另一个人写实现（除非那份实现是准备丢掉的原型），先写测试以确保大家对规格达成一致，每个人都要为自己代码的正确性负责；**Run the tests（跑测试）**——测试不跑就没用，开工前跑一遍，提交前再跑一遍；**Automate（自动化）**——6.031 用 Didit 在你 push 到 github.mit.edu 时自动跑测试，这也消除了"在我机器上是好的"这种说法：要么自动化构建通过，要么就得修；**Review what you commit（审查你要提交的东西）**——用 `git diff --staged` 或图形工具看一眼，跑测试，**不要用** `git commit -a`；**Pull before you start working**；**Sync up（同步）**——一天或一次工作结束时，确认所有人都 push/pull 完毕、处于同一个提交、并且对项目状态满意。 – **关键规则与最佳实践**： – 提交的粒度应当是"对项目的一次改动"，而不是"改过的每个文件各来一次"，也不是"包括还不能编译的中间状态"。 – 不要提交不能编译的代码、不要提交调试输出、不要提交实际上用不到的东西——"don't break the build"。 – 不要对你编辑过的每个文件做整体重排版（reformat），那会让 diff 变成一团噪声，掩盖真正的改动。 – 团队项目要求每个成员在每次课堂 check-in 时，能展示自己的**工作副本已 commit、已 push、且与远程仓库同步**。 – 迭代 #0 时，每位成员把工作提交到 `iter0/<用户名>/` 目录下（因为大家可能产生同名文件），迭代 #1 之后才统一使用 `src/` 与 `test/`——这就是用**目录结构**规避冲突的典型手法。

提交粒度与提交信息（Commit Granularity and Commit Messages）（补充说明：课程原文强调"每次提交要有描述你改了什么的有用信息"，并在团队项目规范里明确要求 useful commit message；以下关于提交粒度与信息格式的九条细则属于工程实践补充，用于把这条要求变得可执行。） – **定义与目的**：提交是历史的最小单位，也是未来阅读这份历史的人的**唯一路标**。它解决的是"三个月后（或三小时后）我能不能弄明白这里为什么变成了这样"。 – **直观解释（"它是什么？"）**：好的提交回答三个问题：**改了什么、为什么改、影响谁**。它是一个自包含的、可单独审查（review）、可单独回滚（revert）的改动单元。 – **关键规则与最佳实践**： – **原子性**：一次提交只做一件事；"修 bug + 顺手重命名 + 调整格式"应当分成三次提交。 – **可编译**：每

个提交都不应破坏构建，因为任何人都可能从任意提交开始工作。 – **主题行简洁**：一句话说明做了什么，通常控制在一行以内；需要的话用 `模块: 动作` 的形式。 – **正文讲清楚"为什么"**：代码本身说明了"是什么"，提交信息的价值在于动机与取舍。 – **不要用** `update`、`fix`、`stuff`、`asdf` 这类信息——它们等于告诉后来的人"这段历史你不用读了"。 – **多人共同完成的一次提交**，在信息里写上合作者（课程的项目规范明确要求这样做，助教也会通过 Git 日志查看个人贡献）。

灾难恢复（Disaster Recovery） – **定义与目的**：本讲把"恢复"分成两层：**预防**与**补救**。预防是定期 add/commit/push，这样工作安全地存在远程仓库里，随时可以用 `git clone` 拉到一个全新目录；补救则针对已经发生的问题。它解决的是"我搞坏了仓库怎么办"。 – **直观解释（"它是什么？"）**：三个关键命令各司其职： – `git revert <revision>`：撤销**一整个提交**。它**不是**把仓库倒回旧版本，而是在 HEAD 处新建一个提交，抵消旧提交的效果；旧提交仍然留在历史里。 – `git show <revision>`：查看某个提交的 diff（删除是红色、新增是绿色）；`git show <revision>:<path>` 直接输出某个文件在那个版本的内容，找到可用版本后**复制粘贴**是最简单的恢复策略。 – `git checkout <revision> -- <path>`：把工作目录里的**某个文件**替换成旧版本。`--`（后面有空格）和路径都不能少。它不产生提交，但会把改动放入暂存区。 – **关键规则与最佳实践**： – **永远不要**运行不带 `-- <path>` 的 `git checkout <revision>`：那会让 `HEAD` 指向旧提交，你就不再处于 `main` 分支上——这就是"detached HEAD（分离头指针）"，课程的原话是"nobody wants that"。 – 如果某个命令警告你 HEAD 已分离，**先停下来求助**，不要继续操作，以免丢工作；已经发生的分离状态几乎总能恢复，但必须小心处理。 – 只想撤销一个提交里**某一处**有问题的改动时，用 `git show` 找到旧版本内容，而不是 revert 整个提交。 – 用图形工具或 VS Code 的 TIMELINE 视图来浏览历史、定位要恢复的数据，但**执行命令仍推荐命令行**。 – 恢复之后要**跑测试并提交**，否则你的"恢复"只存在于本地工作目录里。

分支、拉取请求与代码评审（Feature Branches, Pull Requests, Code Review）（补充说明：6.031 **明确不推荐**在课程规模的项目中使用分支与变基，本小节介绍的是业界在更大团队中的标准做法，用于理解为什么课程做出这个取舍。） – **定义与目的**：功能分支（feature branch）把一个功能的开发隔离在独立的历史线上，拉取请求（pull request, PR）是"请求把你的分支合并进主干"的正式提案，代码评审（code review）是合并前必须通过的审查。它解决的是"如何在不阻塞他人的前提下，让改动在进入主干之前被多人看过"。 – **直观解释（"它是什么？"）**：主干（`main`）始终是**可发布**的状态；每个人从主干拉出分支、在自己的分支上做小步提交、推送、开 PR，CI 跑测试、同事读代码，通过后再合并回主干。整个过程把 Reading 4（代码评审）从"事后传阅"变成了"合并的前置条件"。 – **关键规则与最佳实践**： – 分支要**短命**：活几天、而不是几周，否则合并冲突会指数级增长。 – PR 要**小**：几百行以内的 PR 才能被认真评审。 – 保护主干：要求 CI 通过 + 至少一人 approve 才允许合并。 – 合并前先把自己分支上的主干最新版本合进来（或 rebase），在**自己的分支上**解冲突，而不是把冲突留给主干。 – 课程对照：6.031 的 1–2 周、3 人项目里，"所有人都在 `main` 上频繁提交 + 高频同步 + Didit 自动跑测试"比引入分支更简单可靠；课程原话是分支"extremely important when the size of the project, the length of the time, or the number of people is much larger"（sp22 新增的这句解释比 sp21 更明确地给出了判据）。

合并 vs 变基（merge vs rebase）（补充说明） – **定义与目的**：两者都把两条历史线结合起来，但**历史形状**不同。它解决的是"我希望历史看起来是什么样"。 – **直观解释（"它是什么？"）**： – `git merge` 保留两条线的真实形状，产生一个合并提交，历史是"忠实的"但会有分叉。 – `git rebase` 把你的提交逐个"搬到"目标分支的最新提交之上，历史变成一条直线，读起来干净，但**等于重写了提交 ID**。 – **关键规则与最佳实践**： – 只对**尚未推送、别人看不到的本地提交**做 rebase。 – **绝不**对已经推送并被别人基于其工作的分支做 rebase（或 force push）——别人的历史会因此断裂。 – 团队达成统一约定：要么"合并优先"，要么"变基优先"，不要各自为政。 – 变基过程中每个提交都可能产生冲突，需要逐个解决；这也是它在小项目里不划算的原因。 – 记住课程的立场：在 6.031 的项目里用不到它；理解概念是为了在更大团队里能接得上。

不该进仓库的东西：`.gitignore`、编译产物与密钥（补充说明：课程原文要求"不要提交没用的东西""审查你要提交的内容"，但没有展开 `.gitignore` 的写法；以下内容是这条要求的具体化。） – **定义与目的**：版本库应当只包含**人写的源码与配置**。编译产物可以从源码重建，编辑器设置因人而异，密钥则根本不应该存在于任何被共享的地方。`.gitignore` 让 Git 忽略这些路径。 – **直观解释（"它是什么？"）**：判断标准很简单——**"这能不能从仓库里的其他东西重新生成？"** 能，就不该提交；**"这是不是秘密？"** 是，就**绝对不能**提交（提交过的密钥即使后来删除，它仍然留在历史里，必须视为已泄漏并立即轮换）。 – **关键规则与最佳实践**： – 常见的忽略项：`*.class`、`build/`、`target/`、`out/`、`node_modules/`、`.vscode/`、`.idea/`、`*.log`、`.env`、`*.pem`。 – `.gitignore` 只对**未跟踪**文件生效；已经提交过的文件必须先 `git rm --cached <file>`，再依赖 `.gitignore`。 – 密钥、令牌、口令一律通过环境变量或密钥管理服务注入，绝不硬编码、绝不提交。 – 提交前用 `git status` 与 `git diff --staged` 双重确认；`.gitignore` 不是万能的，它只挡住你想到过的东西。 – 一旦密钥进过远程仓库，视为已经泄漏：立刻轮换密钥，并检查历史中是否还有残留。

代码示例与对比分析

Git 是一个命令行工具，所以下面多数对比以**仓库状态与命令序列**为单位（`bash / text`）。但"合并"这件事的后果最终会体现在**Java 源码**上，因此凡是涉及合并语义的场景，都给出 Java 代码，让你看清"Git 说合并成功"与"代码还能正确工作"之间的距离。

场景 1：提交前不审查，用 `git commit -a` 把所有改动一并塞进历史

✗ *错误代码*

```
#改了三个文件（其中一个是调试用的笔记），不想多想，直接一把梭
$ git status
On branch main
Changes not staged for commit:
        modified:   Hello.java
        modified:   HelloTest.java
        modified:   notes-todo.txt

$ git commit -a -m "fix"
[main 9f31c02] fix
 3 files changed, 412 insertions(+), 8 deletions(-)
```

```
// 被顺手提交进仓库的调试输出：History 里永远留着它
public class Hello {
    public static String greeting() {
        System.out.println("DEBUG greeting() called");   // TODO 删掉
        return "Hello";
    }
}
```

【错误代码的问题】 1. **塞进了与提交目的无关的文件**：`notes-todo.txt` 让你的队友在 review 时看到一堆噪声，也增加了仓库体积。 2. **调试输出进入历史**：`println` 会污染所有使用者的标准输出（在 6.031 的测试里甚至会让自动化测试的输出无法阅读），而删除它需要额外一次提交——历史里永远留着"那段错误存在过"的记录。 3. **`-a` 绕过了暂存区**：你失去了"提交前看一眼将要进入历史的确切内容"这道唯一的人工闸门，正是这道闸门让 6.031 反复强调"Review what you commit"。 4. **提交粒度被破坏**：一次提交里混着修复、笔记与调试代码，将来要 revert 时无法只撤销坏的那部分。

✅ *正确代码*

```
#只暂存本次真正想要的改动，并逐个 hunk 确认
$ git status
On branch main
Changes not staged for commit:
        modified:   Hello.java
        modified:   HelloTest.java
Untracked files:
        notes-todo.txt          # 本地笔记：不提交，写进 .gitignore 或干脆留在本地

$ git add -p Hello.java HelloTest.java
$ git diff --staged          # 提交前审查：即将进入历史的确切内容
$ ./gradlew test             # 测试通过才提交（6.031 项目里由 Didit 在 push 后自动跑）
$ git commit -m "hello: greeting() 支持自定义问候语"
[main 4c81de0] hello: greeting() 支持自定义问候语
 2 files changed, 37 insertions(+), 6 deletions(-)
$ git push
```

```
// 提交进仓库的是干净的实现：没有调试输出，没有 TODO 残留
public class Hello {
    /**
     * @param language 问候语所使用的语言，例如 "en" 或 "it"
     * @return 对应的问候语
     */
    public static String greeting(String language) {
        return "it".equals(language) ? "Ciao" : "Hello";
    }
}
```

【为什么这样更好】 三步闸门各挡住一类事故：`git add -p` 让你**逐块选择**改动（把调试输出留在工作区），`git diff --staged` 让你看清**将要被记录的内容**（而不是你以为的内容），跑测试让你确认**这份内容不会破坏构建**。提交信息从 `fix` 变成 `hello: greeting() 支持自定义问候语`，队友在 `git log` 里一眼就知道这

次提交动了什么、动了哪个模块。最后 `git push` 让工作进入远程仓库——这正是"预防灾难"的核心动作：**工作只要在远程，就几乎不会丢**。

【代码对比解说】 这里的对比不是两个版本的 Java 代码，而是**两种仓库状态**：左版的历史里多了一个 `fix` 提交，它同时包含调试代码、无关笔记与真正的修复；右版的历史里只有一次干净、可审查、可单独回滚的改动。请注意 `git commit -a` 并不是"危险命令"，它只是把"人类审查"这一步删掉了——在本讲的价值体系里，**被删掉的这一步才是关键**。另外注意 `git add -p` 与 `git diff --staged` 是**成对使用**的：前者决定进入暂存区的内容，后者让你复核这个决定；只做前者不做后者，仍然可能把调试输出提交上去。

【设计原则透视】 这直接对应 Reading 3（测试）的"测试不跑就没用"：本讲把"跑测试"从个人习惯升级为**提交的前置条件**，并由 Didit 这类自动化工具兜底。它也对应 Reading 9（避免调试）中"不要留下调试输出"的纪律——`System.out.println` 的残留既是 ETU 问题（噪声掩盖真正输出），也是 SFB 问题（在某次测试中它会成为误报的来源）。从**抽象边界**的角度看，`git diff --staged` 就是"提交"这一操作的规格：它让你在副作用发生前确认输入是否符合预期，与 Reading 6 中"实现前先写清后置条件"是同一套思维。

场景 2：提交粒度与提交信息——二十次 `update` 与一次"什么都有"的巨型提交

❌ *错误代码*

```
$ git log --oneline
f10a2b1 update
9c4e77a update
77bb3de asdf
31a90cc stuff
0d5e114 fix
c2b8f70 WIP
a40031e update
...            # 还有十几个一模一样的
```

```
另一种同样糟糕的形态：一次提交做完了整个迭代
$ git show --stat 3e62e60
 Hello.java        | 210 ++++++++++++++++++--------
 HelloTest.java    |  96 +++++++++------
 Main.java         |  44 ++++++
 build.gradle      |   3 +-
 team-contract.pdf | Bin 0 -> 184320 bytes
 notes-todo.txt    |  18 +++
 6 files changed, 371 insertions(+), 118 deletions(-)
```

【错误代码的问题】 1. **历史失去检索价值**：`update`、`fix`、`stuff`、`asdf` 让你的队友（以及三个月后的你）无法用 `git log` 找到"上次改 `greeting()` 是哪次提交"。 2. **无法精确回滚**：想撤销的只是 `Hello.java` 里那处坏改动，但 `git revert 3e62e60` 会把整轮的 371 行新增、118 行删除**全部**撤销，包括你想保留的部分。 3. **无法有效评审**：一个包含六个文件、数百行改动的提交无法被认真 review（对应 Reading 4 的评审实践），队友只能"扫一眼然后 approve"。 4. **把可编译性与不可编译性混在一起**：巨型提交往往包含"中途还不编译"的状态，破坏"每个提交都不破坏构建"的承诺。

✅ *正确代码*

```
$ git log --oneline
b0b54b3 hello: greeting(String) 支持语言参数
4c81de0 hello: 把 greet() 从打印改为返回值
3e62e60 main: 新增 Main.java 并打印问候语
1255f4e hello: 修改问候语文本
41c4b8f Initial commit
```

```
$ git show -s --format=%B 4c81de0
hello: 把 greet() 从打印改为返回值

Main 需要把问候语写进日志文件而不是标准输出，所以 greet() 不能再
自己 println。这次改动只调整返回类型与调用点，不改问候语内容。

- Hello.java: greet(String) 返回 String，不再打印
- HelloTest.java: 断言返回值而不是捕获标准输出
由 Alice 与 Bob 结对完成（同一键盘）。
```

【为什么这样更好】 每次提交都以一句 `模块：做了什么` 开头，于是 `git log --oneline` 本身就是一份可读的项目年表；需要细节时，`git show -s --format=%B` 或 `git show` 给出**为什么**这么改（`Main` 需要写日志文件，所以不能再用 `println`），以及**影响面**（改了两个文件、调用点同步更新）。原子化的提交让 `git revert` 变成一把手术刀而不是一把锤子。最后一行记录合作者，既满足课程项目"多人共同完成的提交要注明合作者"的要求，也让助教能从 Git 日志中看到每个人的实际贡献。

【代码对比解说】 提交信息的质量不是审美问题，而是**信息检索问题**：`git log --oneline` 是你寻找"这个 bug 是从哪次改动引入的"的第一入口，如果入口全是 `update`，你只能退回到逐行 `git blame`。同理，提交粒度直接决定 `git revert`、`git bisect`、代码评审这三件事是否可用。注意右版的正文里写的是**动机**（为什么改）而不仅仅是**动作**（改了什么）——动作在 diff 里已经写着，动机只能由人写下来。补充说明：业界常见的约定是主题行不超过约 50 个字符、正文每行约 72 个字符，并用 `模块: 动作` 或 `type(scope): subject` 之类的前缀；这些约定属于补充实践，核心仍是"一次一件事 + 说清为什么"。

【设计原则透视】 这与 Reading 6/7（规格说明）同构：提交信息就是这次改动的**规格**——它说明意图（前置条件）、说明效果（后置条件），并让审查者可以判断实现是否符合意图。它也与 Reading 11（AF/RI）呼应：一次提交应当有清晰的**抽象边界**，即"这次改动引入了什么新的不变式"，把不相关的改动混进来，等于同时修改多条不变式而无法分别验证。最后，它与 Reading 29 自身的第一条团队准则"Communicate"直接相连：**提交信息是异步沟通的主要载体**，你不可能每次都当面告诉队友你改了什么。

场景 3：不先 pull 就开工，于是"自动合并成功但语义坏了"

❌ *错误代码*

```
// 合并结果：Hello.java（Alice 的改动）
public class Hello {
    /** 现在返回问候语，而不是打印它。 */
    public static String greet(String name) {
        return greeting() + ", " + name;
    }

    public static String greeting() {
        return "Hello";
    }
}
```

```
// 合并结果：Main.java（Bob 新加的文件，基于旧版 hello 写的调用）
public class Main {
    public static void main(String[] args) {
        Hello.greet("Eve");    // 返回值被丢弃：编译通过，运行却什么都不输出
    }
}
```

【错误代码的问题】 1. **Git 无法替你发现语义冲突**：Alice 改了 `Hello.java` 的方法体，Bob 新增了 `Main.java`，两处改动不重叠，所以 Git **自动合并成功**；但 `greet` 的语义已经从"打印"变成"返回"，Bob 的调用点因此失效。 2. **不先 pull 就是基于旧版本开发**：Bob 写 `Main.java` 时的起点里，`greet` 还是返回 `void` 的，他的代码在那个版本上是正确的——错误来自"起点过时"，这正是本讲把"Pull before you start working"单列一条的原因。 3. **没有测试覆盖调用点**：如果有一个断言 `greet` 行为的测试（或一个端到端测试检查输出），这次语义破坏会在 CI 上立刻暴露。 4. **"编译通过"给人虚假的安全感**：合并后构建是绿的，于是没有人怀疑它——这是最危险的一类失败。

✅ *正确代码*

```
$ git pull                      #开工前先拉取，确认自己的起点是最新的
Already up to date.

$ git log --oneline -5          #看到接口变更的提交，先与队友确认调用点是否都已同步
4c81de0 hello: 把 greet() 从打印改为返回值
```

```
// 与新的规约一致的调用点：使用返回值，而不是假设它会打印
public class Main {
    public static void main(String[] args) {
        System.out.println(Hello.greet("Eve"));    // 按新规格显式输出
    }
}
```

```
import static org.junit.Assert.assertEquals;

import java.io.ByteArrayOutputStream;
import java.io.PrintStream;

import org.junit.Test;

/** 覆盖 greet(..) 的返回值语义，让接口变更无处可藏。 */
public class HelloTest {

    @Test
    public void testGreetReturnsGreeting() {
        assertEquals("Hello, Eve", Hello.greet("Eve"));
    }

    @Test
    public void testGreetDoesNotPrint() {
        PrintStream original = System.out;
        ByteArrayOutputStream captured = new ByteArrayOutputStream();
        System.setOut(new PrintStream(captured));
        try {
            Hello.greet("Eve");                  // 新规格：只返回，不打印
        } finally {
            System.setOut(original);
        }
        assertEquals("", captured.toString());
    }
}
```

【为什么这样更好】 三个动作把这类事故挡住：**先 pull** 保证起点是最新的；**看 log 并与队友沟通**让接口变更成为共同知晓的事实（本讲第一条准则）；**测试覆盖返回值语义**让"语义坏了"变成一次红的构建，而不是一个上线后才发现的问题。注意右版的 `Main` 只是显式地把返回值打印出来——它并没有试图恢复旧行为，而是**接受新规格**，这正是"合并后要重新理解代码含义"的体现。

【代码对比解说】 本讲的第三组练习问的正是这里：合并结果会怎样？答案是"**我们能自动合并，但合并后的代码是坏的（无错误、结果错误）**"——在 Java 里 `Hello.greet("Eve");` 作为表达式语句完全合法（返回值被丢弃），所以既没有静态错误也没有运行时异常，只是**什么都不会输出**。这组对比的教学价值在于告诉你：Git 的冲突检测是**基于文本位置**的，而软件的正确性依赖**语义契约**。任何"接口/语义变更"都必须由人来传播，工具帮不上忙。补充一点 TypeScript 对照（sp22 原版）：`greet("Eve");` 在 TS 中同样合法，结果也一样是静默无输出。

【设计原则透视】 这是 Reading 6（规格说明）最直接的团队版本：`greet` 的**后置条件**从"打印一行问候"变成"返回问候字符串"，规格变了，所有调用点都必须重新验证。它也是 Reading 3（测试）的用例：好的测试把规格钉住，使得"别人静默地改变了语义"这件事无法悄悄发生。从抽象边界的角度看，`Hello.greet` 是一个模块边界，跨边界的约定必须显式且共享；本讲那句"pull before you start working"的技术含义其实是：**你的局部心智模型必须与仓库的当前状态同步**，否则你写出的代码是针对一个已经不存在的世界的。

场景 4：把冲突标记直接提交上去，或者靠删掉一边来"解决"冲突

❌ *错误代码*

```
$ git merge bob
Auto-merging Hello.java
CONFLICT (content): Merge conflict in Hello.java
Automatic merge failed; fix conflicts and then commit the result.

$ git add Hello.java
$ git commit -m "merge"      # 冲突标记还在文件里！
```

```
// 被提交上去的 Hello.java：Git 的冲突标记原样留在源码里
public class Hello {
    public static void greet(String name) {
        System.out.println(greeting() + name);
    }

    public static String greeting() {
<<<<<<< HEAD
        return "Ciao";
=======
        return "Hello, ";
>>>>>>> bob
    }
}
```

【错误代码的问题】 1. **代码根本不能编译**：`<<<<<<<`、`=======`、`>>>>>>>` 都不是 Java 语法，构建立刻失败——直接违反团队准则"don't break the build"。 2. **把决策责任推给别人**：冲突标记的含义是"这里需要一个人来决定"，把它提交上去等于告诉队友"你来替我决定"。 3. **没有运行测试就提交**：如果提交前跑过测试，这次合并根本不可能通过。 4. **丢掉了对方的工作**：另一种同样糟糕的"解决"方式是随手删掉一边——无论删掉哪一边，都有人白做了。

✅ *正确代码*

```
$ git merge bob
Auto-merging Hello.java
CONFLICT (content): Merge conflict in Hello.java
Automatic merge failed; fix conflicts and then commit the result.

$ git diff                      #1) 看清楚两边各自想做什么（冲突区域的上下文）
$ git log --oneline --left-right HEAD...bob

#2) 与队友确认：Bob 想把逗号移到 greeting() 里，Alice 想把问候语改成 Ciao；两者的意图可以同时满足，于是
$ git add Hello.java
$ ./gradlew test                #3) 合并后必须重新跑测试
$ git commit -m "hello: 合并 Ciao 问候语与逗号位置调整

Bob 把逗号移进 greeting()，Alice 把问候语改为 Ciao；两者互不冲突，
合并后 greeting() 返回 \"Ciao, \"，greet() 只拼接姓名。"
```

```
// 手工合并的结果：同时保留双方的意图，并且可以编译、可以被测试覆盖
public class Hello {
    public static void greet(String name) {
        System.out.println(greeting() + name);
    }

    /** @return 含尾随逗号与空格的问候语，例如 "Ciao, " */
    public static String greeting() {
        return "Ciao, ";
    }
}
```

【为什么这样更好】 冲突解决被拆成三个**必须显式完成**的步骤：**读懂两边意图**（用 `git diff` 看冲突区域，用 `git log --left-right` 看两边各自带来了什么提交）、**做决定并写下来**（在提交信息里记录这个决定，让未来的人知道为什么是 `"Ciao, "` 而不是别的写法）、**重新验证**（跑测试并确认构建是绿的）。合并提交的信息本身成了一份微型设计文档。

【代码对比解说】 左版的失败不是"命令用错了"，而是**把一个必须由人完成的判断（"这里应该是什么"）交给了提交按钮**。右版则把它当成一次真正的设计活动：Alice 改问候语文本、Bob 改逗号位置，这两个意图**并不矛盾**，正确结果是把两者都保留下来（`"Ciao, "`），而不是二选一。顺便回答本讲的第二组练习：如果两边都改了 `greeting()` 的 `return` 那一行（一个改成 `"Ciao"`，一个改成 `"Hello, "`），Git **无法**自动合并，必须由人决定。请注意这里有一个容易忽略的原则：**冲突标记从来不是噪声，它是一份待办事项**。

【设计原则透视】 这与 Reading 6（规格说明）再次同构：冲突区域的两个版本代表**两份互不兼容的规格**，解决冲突就是重新确定唯一的规格，并把它写进代码与提交信息。它也体现 Reading 11（AF/RI）的思维：合并后的 `greeting()` 有一个新的表示契约（"含尾随逗号与空格"），必须写进 Javadoc，否则下一个调用者还会踩同样的坑。最后，它印证本讲的结论：**工具只能发现文本重叠，语义一致性必须由人和测试来保证**——这也解释了为什么 6.031 宁可不推荐分支，也要强调"频繁同步 + 频繁跑测试"。

场景 5：灾难恢复——手工复制、detached HEAD 与正确的 revert

❌ *错误代码*

```
#想撤销上午那次坏提交，于是……
$ git checkout 82e049e          # 忘了 -- <path>!
Note: switching to '82e049e'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches ...
HEAD is now at 82e049e Greeting in Ruby

#继续在这里改代码、commit……
$ git commit -am "继续修"
[detached HEAD 7b1c9aa] 继续修
```

```
另一种"手工恢复"：把旧版本内容从聊天记录/邮件里复制粘回文件，
既没有提交信息说明发生了什么，也没有测试证明恢复是正确的。
```

【错误代码的问题】 1. **detached HEAD 意味着你的提交不属于任何分支**：`7b1c9aa` 不在 `main` 的历史上，一旦切换分支，它就很难被找回。 2. **恢复过程不可追溯**：手工复制粘贴不留下任何记录，未来没人知道"这里为什么被改回去了"。 3. **可能连带撤销不该撤销的东西**：本讲明确说过，一个提交往往同时改进了三个函数，其中只有一个改动是错的。 4. **忘记重新测试**：恢复后的代码没有经过验证就继续开发，等于把不确定性叠加在不确定性上。

✅ *正确代码*

```
#情况 A：想撤销一整个提交 → 用 revert，它在 HEAD 处新建一个反向提交
$ git lol
* b0b54b3 (HEAD, origin/main, origin/HEAD, main) Greeting in Java
...
$ git revert 82e049e
[main abcd123] Revert "Greeting in Ruby"
 1 file changed, 1 deletion(-)
 delete mode 100644 hello.rb
$ git lol
* abcd123 (HEAD, main) Revert "Greeting in Ruby"
* b0b54b3 (origin/main, origin/HEAD) Greeting in Java
...
$ ls
Hello.java     hello.scm      hello.txt        # hello.rb 已消失，历史仍然完整
```

```
#情况 B：只想恢复某一个文件 → 用 show 找到好版本，用 checkout -- <path> 取回
$ git show 41c4b8f:hello.txt
Hello, version control!

$ git checkout 41c4b8f -- hello.txt      # 注意 -- 与路径都不能省
$ git diff --staged                      # 确认暂存的正是这次恢复
$ ./gradlew test                         # 恢复后重新验证
$ git commit -m "hello.txt: 恢复 41c4b8f 版本的问候语（撤销 again 的改动）"
```

【为什么这样更好】 `git revert` 把"撤销"变成历史里一个**可读、可审查、可再撤销**的事件：它不重写历史，不改变别人的起点，而且在 `git lol` 的图上能直接看到发生了什么。`git checkout <rev> -- <path>` 则把恢复精确到单个文件，配合 `git diff --staged` 让恢复本身也经过一次人工审查。两者都保留了完整的因果链——这正是 Reading 13（调试）所说的"让问题可被追溯"。

【代码对比解说】 左版的核心风险是**用一个不可追溯的动作换取一时的方便**；右版则把恢复也纳入版本控制纪律：恢复是一次提交、恢复要跑测试、恢复要写清楚依据（引用了哪个 SHA）。这里最容易踩的坑就是 `git checkout <revision>` 少了 `-- <path>`：命令本身不会报错，它只是"帮你"切到了一个旧的提交上，然后你所有的后续提交都会悬空。课程的原话非常直接：如果你看到"HEAD is detached"的警告，**先停下来求助**；已经发生的分离几乎总能恢复，但必须小心处理。

【设计原则透视】 这与 Reading 5（版本控制）里"对象图不可变"的思想一致：Git 的历史只能被**追加**，不能被悄悄改写——`revert` 之所以优于"手工改回去"，正是因为前者保持了历史的**可审计性**（auditability），这与 Reading 28（伦理）中的伦理结构、以及 Reading 6 中"让行为可被检验"的原则相通。它也提醒你：**恢复是一种开发活动，不是一种文件操作**——它需要规格（我要回到哪个状态）、需要测试（回到的状态是否正确）、需要记录（为什么这样恢复）。

场景 6：把编译产物与密钥提交进仓库

❌ *错误代码*

```
$ git add .
$ git commit -m "commit everything"
$ git status --short
A  build/Hello.class
A  build/HelloTest.class
A  .idea/workspace.xml
A  notes-todo.txt
A  secrets.properties
```

```
// 硬编码在源码里的 API 密钥：一旦提交，它就永久留在 Git 历史里
public class AnalyticsClient {

    /** 分析服务的 API 密钥。 */
    private static final String API_KEY = "sk-live-3f9a1c7b2e5d4806";

    /**
     * 上报一个事件。
     *
     * @param event 事件名
     */
    public void send(String event) {
        // HTTP 请求略：Authorization: Bearer <API_KEY>
        System.out.println("sending " + event + " with key " + API_KEY);
    }
}
```

【错误代码的问题】 1. **`git add .` 把一切都收进来**：`.class` 文件可以从源码重建，提交它们会导致每次重新编译都产生无意义的 diff；`.idea/` 是个人编辑器设置，会与队友的设置反复冲突。 2. **密钥进入远程仓库等于已经泄漏**：即使下一个提交把它删掉，`git log -p` 仍然能翻出明文；克隆过仓库的每个人都有一份副本。 3. **调试语句把密钥打印到日志**：`System.out.println(... + API_KEY)` 让密钥出现在构建日志与终端回滚缓冲区里，泄漏面进一步扩大。 4. **清理成本极高**：从历史中彻底移除一个文件需要重写历史（filter-repo 之类工具），在共享仓库里几乎不可能安全地做——所以唯一现实的对策是**轮换密钥**。

✅ *正确代码*

```
$ cat .gitignore
#编译产物（以 # 开头的行是注释）
*.class
build/
target/
out/

#编辑器与 IDE 设置
.idea/
.vscode/
*.swp

#依赖与日志、本机笔记
node_modules/
*.log
notes-todo.txt

#绝不提交的密钥与本机配置
.env
*.pem
secrets.properties
```

```
$ git status --short          # 只看到真正属于源码的改动
 M Hello.java
 M HelloTest.java

$ git add Hello.java HelloTest.java
$ git diff --staged
$ git commit -m "hello: 密钥改由环境变量注入，移除硬编码的 API_KEY"
$ git push
```

```
// 密钥从环境变量注入：源码里不出现任何秘密
public class AnalyticsClient {

    private final String apiKey;

    /**
     * @param apiKey 从环境变量或密钥管理服务取得的 API 密钥
     * @throws IllegalStateException 如果密钥缺失
     */
    public AnalyticsClient(String apiKey) {
        if (apiKey == null || apiKey.isEmpty()) {
            throw new IllegalStateException(
                    "缺少 API 密钥：请设置环境变量 ANALYTICS_API_KEY");
        }
        this.apiKey = apiKey;
    }

    /**
     * 从环境变量构造客户端。
     *
     * @return 使用 ANALYTICS_API_KEY 的客户端
     */
    public static AnalyticsClient fromEnvironment() {
        return new AnalyticsClient(System.getenv("ANALYTICS_API_KEY"));
    }

    /**
     * 上报一个事件。
     *
     * @param event 事件名
     */
    public void send(String event) {
        // HTTP 请求略：Authorization: Bearer <apiKey>
        System.out.println("sending " + event);   // 日志里绝不出现密钥
    }
}
```

【为什么这样更好】 三条防线同时生效：`.gitignore` 让生成物与本机配置从一开始就进不了暂存区（`git status` 因此变得**可读**——这本身就是 ETU 的收益）；密钥通过构造函数注入、由环境变量提供，于是源码可以公开、可以分享、可以在任何机器上克隆而不会泄漏秘密；构造函数对缺失密钥**立即失败**，把"部署时忘了配环境变量"变成一个启动即崩的明确错误，而不是一个运行时静默的认证失败。日志里也不再打印密钥。

【代码对比解说】 左版的提交名单本身就是一份"我们没想清楚什么该进仓库"的自白；右版把 `.gitignore` 当成一份**显式的政策文档**——它写下了团队关于"什么属于版本库"的共识。Java 侧的关键差异是**依赖注入**：密钥不再是编译期常量，而是构造对象时传入的运行时值，这既解决了泄漏问题，也让 `AnalyticsClient` 变得**可测试**（测试可以注入一个假密钥）。注意 `.gitignore` 的两条重要限制：它只对未跟踪文件生效，已被提交的文件必须先 `git rm --cached`；它也只挡住你**想到过**的路径，所以 `git add .` 这种习惯仍然危险——用 `git add <file>` 明确指定要提交的内容，是最省心的做法。

【设计原则透视】 这与 Reading 11（AF/RI）的表示不变量同构：`apiKey` 的 RI 是"非空且来自受信任的注入点"，构造函数在入口处强制它成立（这正是"在对象创建时建立不变式"的经典写法）。它也与 Reading 8（不可变性）一致：`private final String apiKey` 一旦设定就不再变化，避免"某个方法偷偷换了密钥"这类事故。放到 Reading 28（伦理）的框架里看，硬编码密钥违反了 ACM 准则 1.6（尊重隐私）与 1.7（保密）的精神：它把系统与用户的秘密暴露给**每一个能读到仓库的人**，而这些人里有相当一部分你并不认识——这正是"扩展的影响圈"在版本控制层面的一次具体呈现。

与其他设计原则的关联

- **与 Reading 5（版本控制 Version Control）**：本讲是 Reading 5 的多用户版本。Reading 5 讲过对象图、分支、`clone/add/commit/push/log/merge` 与 `main` 这一默认分支名（旧教程里的 `master` 请直接替换为 `main`），本讲则把"多人同时推拉同一个仓库"的协调问题补齐。建议把两讲一起读：前者解释"Git 是什么数据结构"，后者解释"团队怎么用它"。
- **与 Reading 4（代码评审 Code Review）**：本讲的要求"Review what you commit"（`git diff --staged`）是**自我评审**，而 Reading 4 的代码评审是**同行评审**；在更大的团队里，这两件事由功能分支与拉取请求串成一条流水线（补充说明：6.031 的小项目不要求分支）。提交的原子性直接决定评审质量——一个 400 行的提交无法被认真评审。
- **与 Reading 3（测试 Testing）**：本讲把"写测试、跑测试、自动化"作为团队准则的前三条，并把 Didit 的自动构建当作"it worked on my machine"的解药。测试在这里获得了一个新角色：它是**合并后语义检查**的唯一手段（见场景 3）。
- **与 Reading 6/7（规格说明与设计规格 Specifications / Designing Specifications）**：提交信息是一次改动的规格；接口变更（方法签名与后置条件）必须被传播到所有调用点；冲突解决的本质是重新确定唯一规格。本讲与 Reading 6 是"团队层面"与"代码层面"的同一件事。
- **与 Reading 9（避免调试 Avoiding Debugging）**：`commit -a` 最经典的危害就是把 `println` 调试输出带进仓库（sp21 原文的原话是"a great way to fill your repo with println s"）。避免调试输出、避免依赖打印来观察行为，正是本讲"不要提交调试输出"的技术前提。
- **与 Reading 11（抽象函数与表示不变量 AF/RI）**：合并后的每个模块都必须重新确认自己的 RI 是否仍然成立（场景 4 中 `greeting()` 的"含尾随逗号与空格"就是一条新的 RI）；密钥注入的构造检查是"在创建时建立不变式"的教科书用法。
- **与 Reading 13（调试 Debugging）**：`git log`、`git show`、`git blame`、`git bisect` 是调试时间维度上 bug 的主要工具；本讲强调的历史可读性（有意义的提交信息、原子提交）直接决定这些工具是否好用。`git revert` 也是一次"可追溯的修复"。

- **与 Reading 28（软件工程中的伦理 Ethical Software Engineering）**：本讲练习中"Charlie 写了一个能工作的方法，一次都没测试就提交到团队仓库"用的正是 **Process 透镜**——他违反的是团队约定的过程；而把密钥或用户数据提交进仓库，则同时违反 ACM 准则的 1.6（隐私）与 1.7（保密）。团队版本控制纪律是伦理要求最日常的落地形式。

关键要点

- **开工前 `git pull`，收工前 `git push`，并且确认所有人处于同一个提交**：起点过时是所有"自动合并成功但代码坏了"事故的根源。
- **提交前必须过三道闸门：`git diff --staged` 看一眼、跑一遍测试、写一句有用的提交信息；并且一次提交只做一件事、保证它能编译**：不要用 `git commit -a` 跳过审查，原子提交才是 `git revert`、代码评审与 `git log` 检索能力的前提。
- **合并成功 ≠ 语义正确**：Git 只能发现文本重叠；接口变更靠沟通传播，正确性靠合并后重新跑测试来确认。
- **恢复用工具、不要用手工**：撤销整个提交用 `git revert`，恢复单个文件用 `git checkout <rev> -- <path>`（`--` 与路径都不能省）；看到 detached HEAD 警告就停下来求助。
- **`.gitignore` 从第一天就写，密钥绝不进仓库**：判断标准是"能不能从仓库重建"（不能重建才提交）与"是不是秘密"（是秘密就永远不提交）。

常见陷阱与注意事项

1. **不先 pull 就开始改代码** → 后果：你编辑的是旧版本，之后必然要合并，而且很可能要花时间解冲突；更糟的是像场景 3 那样"合并成功但语义坏了"，构建是绿的而行为是错的。
2. **用 `git commit -a` 或 `git add .` 一把梭，或者把生成物写进 `.gitignore` 却忘记 `git rm --cached`** → 后果：调试 `println`、本地笔记、`.class` 文件、IDE 设置乃至密钥被一起提交，历史里永久留下噪声与秘密；而被跟踪过的文件不会因为 `.gitignore` 自动消失，它继续产生无意义的 diff，`.gitignore` 形同虚设。
3. **把冲突标记直接提交，或者随手删掉一边** → 后果：代码无法编译、违反"don't break the build"，或者无声地删掉了队友的工作。
4. **提交信息写成 `update`、`fix`、`stuff`** → 后果：历史失去检索价值，`git log` / `git blame` / `git revert` 全部失效；三个月后连你自己都不知道那次改动的动机。
5. **在引用与历史上冒险：`git checkout <revision>` 忘了 `-- <path>`，或者对已推送的分支做 rebase / `push --force`** → 后果：前者进入 detached HEAD，之后的提交不属于任何分支、极易丢工作；后者改写同事的基点，让他们的提交悬空并制造难以理解的冲突（补充说明：6.031 本身不推荐使用 rebase）。看到 detached HEAD 警告必须停下来求助。
6. **以为"团队里只要有人跑过测试就行"** → 后果：本讲明确要求每个人为自己的代码正确性负责，并且要跑测试后再提交；依赖别人兜底，等于把不确定性留给整个团队。

思考题（带答案）

问题 1：Alice 与 Bob 从同一版本出发，分别改动了同一份 `Hello` 程序。请分别回答下面两种情形，并解释 Git 为什么给出了不同的结果。**(a)** Alice 修改了 `greet(..)`（在问候语后加上 `"!"`），Bob 修改了 `greeting()`（把返回值从 `"Hello"` 改为 `"Ciao"`）：Git 合并后 `Hello.greet("Eve")` 的结果是什么？**(b)** Alice 把 `greeting()` 改成 `return "Ciao";`，而 Bob 把逗号移进 `greeting()`（`greet(..)` 改成 `System.out.println(greeting() + name);`、`greeting()` 改成 `return "Hello, ";`）：Git 能自动合并吗？

答案（a）：结果是 **`Ciao, Eve!`**。两人改的是文件中**不同的位置**：Alice 动的是 `greet(..)` 方法体里的字符串拼接，Bob 动的是 `greeting()` 的 `return` 语句。Git 的合并是**基于共同祖先的三方合并（three-way merge）**：把"共同祖先→Alice"与"共同祖先→Bob"两组差异分别计算出来，由于两组差异涉及的行不重叠，它就能同时应用，因此**自动合并成功**，而且结果在语义上也是正确的。合并后的关键代码是：

```
public static void greet(String name) {
    System.out.println(greeting() + ", " + name + "!");   // 来自 Alice
}
public static String greeting() {
    return "Ciao";                                        // 来自 Bob
}
```

教学要点是：**自动合并成功且结果正确**，这是最常见的情形，也正是"Git 帮你省掉了大量协调工作"的地方。但 (b) 会告诉你，自动合并成功并不保证正确。

答案（b）：**不能自动合并，会产生合并冲突**。原因在于 `greeting()` 方法体里那一行 `return "Hello";` **被两个人用不同方式修改了**：Alice 改成 `"Ciao"`，Bob 改成 `"Hello, "`。三方合并的规则是"只有一方改动的行取改动后的版本，双方都改了同一行的就交给人工"，因此 Git 会报出 `CONFLICT (content): Merge conflict in Hello.java`，并把两个版本用 `<<<<<<<` / `=======` / `>>>>>>>` 标记在文件里：

```
    public static String greeting() {
<<<<<<< HEAD
        return "Ciao";
=======
        return "Hello, ";
>>>>>>> bob
    }
```

需要人来判断"正确的意图是什么"：如果两个人分别想改问候语语言与标点位置，这两件事并不矛盾，正确结果是 `return "Ciao, ";`（并把它写进 Javadoc，作为新的表示契约）。这个例子还说明一件事：**冲突是文本层面的，解决冲突是设计层面的**——解完必须跑测试。(a) 与 (b) 的对照正是本讲的核心：Git 只比较行的重叠情况，它既不知道也不关心代码的语义。

问题 2：Alice 把 `greet(..)` 从"打印问候语"改成"返回问候字符串"，Bob 新增了一个 `Main.java`，内容是 `Hello.greet("Eve");`。Git 会怎样？运行 `Main` 会打印什么？

答案：**Git 会自动合并成功，因为两处改动不重叠**（Alice 改 `Hello.java` 的方法体与签名，Bob 新增 `Main.java`）。但在 Java 里，`Hello.greet("Eve");` 作为**表达式语句**是合法的——返回值被丢弃，不会产

生静态错误，也不会抛出运行时异常。因此运行 `Main` 的结果是**什么都不打印**：没有错误，但结果与 Bob 的意图不符。这正是本讲练习的选项里"we can automatically merge, but the resulting code is broken (no error, wrong answer)"所描述的情形。修复方式是让调用点与新规格一致（`System.out.println(Hello.greet("Eve"));`），并且为 `greet(..)` 的返回值语义补上测试，让这类语义破坏在下一次 CI 运行时立刻暴露。**这题的核心教训**：Git 的冲突检测只覆盖文本位置，不覆盖语义契约；所以"Pull before you start working"、接口变更要沟通、合并后必须重新跑测试，这三条团队准则缺一不可。（补充说明：sp22 的 TypeScript 版本结论完全相同。）

问题 3：你是三人小组的一员。队友 Charlie 刚刚把一个功能提交到共享仓库，提交历史是这样的：`git log --oneline` 显示 `a1b2c3d update`，而这次提交同时包含三个文件、四百多行改动，其中还带着两个 `System.out.println("DEBUG ...")`。请指出至少三个具体问题，并给出你会怎样与 Charlie 沟通（用本讲的准则说话）。

答案：**具体问题**：(1) 提交信息 `update` 无法被检索，将来没人能从 `git log` 找到这次改动的动机，`git revert a1b2c3d` 也会把四百多行全部撤销，无法只撤销坏的那部分；(2) 一次提交包含三个文件、四百多行，无法被认真 review（违反 Reading 4 的评审实践），而且很可能包含"中途不编译"的状态，违反"don't break the build"；(3) 两个 `DEBUG` 输出会污染标准输出，可能让自动化测试的输出无法阅读，甚至让某些测试产生误判（对应 Reading 9 的"避免调试输出"）；(4) 从 Reading 28 的 **Process 透镜**看，问题不在于代码能不能跑，而在于他跳过了团队约定的过程，把自己的不确定性转嫁给了队友。**沟通方式**（用本讲的准则，而不是用"你写得真烂"）：先引用共同约定——"我们约好提交前跑一遍测试、用 `git diff --staged` 看一眼、并且每个提交都要能描述自己改了什么"；然后提出可操作的补救：如果这次提交还没被别人基于它工作，可以本地整理后重新提交（补充说明：重写已推送的历史会影响他人，必须先与全组确认；更安全的方式是追加一个清理提交，把 `println` 删掉）；最后把它变成流程改进而不只是个人批评——在提交信息里写清动机、以后用 `git add -p` 逐块暂存、push 后让 Didit 自动跑测试，让"高质量提交"成为**结构**而不是靠每个人自觉（这正是 Reading 28 的"伦理结构而非伦理个体"在版本控制里的翻版）。

第三部分：软件构造核心原则速查表

> 本速查表按类别汇总 MIT 6.031 全部关键概念、规则与代码模板，可作为写代码时的检查清单，也可作为考前复习的"一页纸"。 术语以 sp22 原版为准，代码为 Java。

3.1 总纲：三大目标（The Big Three）

缩写	目标	判据	主要手段
SFB	Safe from bugs 免于 bug	今天正确，未来也正确	静态类型、规格说明、不变量、不可变性、测试、断言
ETU	Easy to understand 易于理解	与未来的程序员清晰沟通	好命名、注释、抽象、封装、避免魔法数字、DRY

缩写	目标	判据	主要手段
RFC	Ready for change 易于修改	能适应变化而不重写	表示独立性、接口、解耦、规格说明、模块化

黄金法则：任何一个设计决策都应当能回答"它如何改善 SFB / ETU / RFC 中的至少一项，代价是什么"。

3.2 静态检查与类型（Reading 1–2）

概念	要点
静态检查	编译期检查；错误在运行前被抓住
动态检查	运行期检查；错误在发生时被抓住
无检查	错误可能无声地产生错误结果
类型安全	类型系统保证变量永远持有该类型的合法值

优先级：静态检查 > 动态检查 > 无检查。因为越早发现错误，定位与修复成本越低。

Java 关键规则

```
// 优先使用 final：让"不可重新赋值"成为编译期保证
final int n = 5;              // 不可重新赋值
final List<String> names = new ArrayList<>();  // 引用不可变，但内容仍可变！
```

> 陷阱：`final` 只保证**引用不可重新赋值**，不保证**对象不可变**。这是 ETU 上最容易误解的一点。

3.3 规格说明（Reading 6–7）★核心

规格说明 = 契约：规定方法"应该做什么"，不规定"如何做"。

```
/**
 * Find the first occurrence of a value in a list.
 *
 * @param lst  list to search (must not be null)
 * @param val  value to search for
 * @return     the smallest index i such that lst.get(i).equals(val),
 *             or -1 if val does not occur in lst
 * @throws NullPointerException if lst is null
 */
public static int find(List<Integer> lst, int val) { ... }
```

组成	含义	谁负责
前置条件 precondition	调用者必须满足的条件	调用者的义务；违反时实现可以做任何事

组成	含义	谁负责
后置条件 postcondition	实现必须保证的结果	实现者的义务；调用者可以依赖
副作用 side-effect	方法对输入之外状态的修改	必须显式写明（尤其是 mutating methods）
异常 exception	前置条件被违反时的信号	只应表示"调用者的错误"，不应表示"实现的失败"

规格强弱规则

规则	说明
减少前置条件 = **更强**	能接受更多输入
增加后置条件 = **更强**	保证更多输出
前置条件更弱 + 后置条件更强 = **更强**	更强 = 更容易被调用者依赖
不可比较	一个前置更弱但后置也更弱时，两者 **incomparable**

写好规格的八条准则

1. **声明式优于操作式**：说"返回最小的索引"，而不是"从 0 开始循环比较"。
2. **允许实现自由度**（underdetermined / nondeterministic）：除非调用者真的依赖唯一答案，否则不要规定过死。
3. **前置条件不要过强**：能接受的输入越多越好，但代价是实现要处理更多情况——找平衡点。
4. **不要用 `null`**：把 `null` 从接口中彻底排除；需要"可能没有"时用 `Optional` 或明确的返回约定（如 `-1`）。
5. **写明空输入与边界**（emptiness / boundary）。
6. **说明可变性**：方法是否修改输入对象、是否返回内部对象的别名。
7. **异常用于前置条件违反**，不要用于控制流。
8. **规格说明不能被实现"顺手加强"**——实现必须满足规格，但不能依赖超出规格的行为。

代码模板：前置条件检查

```
public static int find(List<Integer> lst, int val) {
    // fail fast：尽早、显式地检查前置条件
    if (lst == null) throw new NullPointerException("lst must not be null");
    ...
}
```

3.4 测试（Reading 3）★核心

测试优先编程（test-first programming）：先写测试，再写实现。理由：强迫你在写代码前想清楚规格与边界。

概念	要点
验证 verification	确信产品**此刻**正确
确认 validation	确信产品**满足用户真实需求**
系统性测试	按输入空间划分选取用例，而不是随机或穷举
输入空间划分 partitioning	把输入分成若干 **subdomain**，每个子域至少取一个代表
边界值 boundary value	子域的边界（0、空、最大、最小、临界点）是最易出 bug 之处
黑盒测试	只看规格，不看实现
白盒测试（glass box）	参考实现选取用例；用于补充黑盒测试
覆盖率 coverage	语句覆盖是最弱的标准；分支/路径覆盖更强
单元测试 vs 集成测试	单个模块 vs 模块组合；集成测试前可用 **stub** 替代未完成模块
回归测试	每次修改后自动重跑全部测试
迭代开发	小步实现、小步测试

测试用例设计模板

```
// 1. 先写测试策略注释，说明如何划分输入空间
//    Testing strategy:
//      partition on lst: empty list, singleton list, list with duplicates, list without ta
//      partition on val: present in lst, absent from lst
@Test public void testFindEmptyList() {
    assertEquals(-1, Find.find(List.of(), 3));
}
@Test public void testFindAbsent() {
    assertEquals(-1, Find.find(List.of(1, 2, 3), 9));
}
```

黄金法则：测试必须能**暴露 bug**——一个永远通过的测试没有价值。写测试时问自己："我要怎么写才能让这个测试失败？"

3.5 不可变性（Reading 8）★核心

可变性的危险来自别名（aliasing）：当两个引用指向同一个可变对象，任何一方都能在另一方不知情时改变其状态。

不可变性的三大好处

好处	机制
SFB	不可变对象的状态不可能被意外改变，天然免于一类 bug，也天然线程安全

好处	机制
ETU	读代码时不必追踪"谁还持有这个对象的引用"
RFC	不可变对象可以安全共享、安全缓存、安全作为 Map 的键

不可变类的设计规则（四条）

1. **所有字段声明为 `private final`**。
2. **不提供任何 mutator 方法**（包括 `setX`）。
3. **不暴露任何可变内部对象的引用**：getter 必须返回**防御性拷贝**。
4. **不把可变外部对象的引用存进字段**：构造函数必须对可变参数做**防御性拷贝**。

代码模板：防御性拷贝

```
public final class Period {
    private final Date start;
    private final Date end;

    public Period(Date start, Date end) {
        if (start.after(end)) throw new IllegalArgumentException("start after end");
        // 入向防御性拷贝
        this.start = new Date(start.getTime());
        this.end   = new Date(end.getTime());
    }
    public Date start() {
        // 出向防御性拷贝
        return new Date(start.getTime());
    }
}
```

`Collections.unmodifiableList` 的局限

```
List<String> inner = new ArrayList<>(List.of("a"));
List<String> view = Collections.unmodifiableList(inner);  // 只是只读视图
inner.add("b");        // 视图跟着变！因为它是 view，不是 copy
// view.add("c");      // 会抛 UnsupportedOperationException
```

> `unmodifiableList` 提供的是**不可修改的视图**（底层仍可变），不是**不可变对象**。要真正不可变，必须做拷贝并切断对底层集合的引用：`List.copyOf(inner)`。

3.6 抽象数据类型 ADT（Reading 10–12）★核心

ADT 的定义：由**一组操作**刻画的数据类型，其内部表示对使用者隐藏。

概念	含义
抽象 abstraction	忽略底层细节，只保留高层概念
模块化 modularity	系统由可独立理解、替换的单元组成
封装 encapsulation	通过访问控制（`private`）保护内部状态
信息隐藏 information hiding	使用者不需要、也不能知道内部表示
表示独立性 representation independence	内部表示的改变不影响使用者

操作分类（必须掌握）

类别	定义	示例（`List<E>`）
Creator	创建新对象（构造函数或工厂方法）	`new ArrayList<>()`、`List.of()`
Producer	由旧对象产生新对象（不修改旧的）	`List.of(...)`、`concat`、`subList`
Observer	观察对象状态，返回其他类型的值	`size()`、`get(i)`、`isEmpty()`
Mutator	修改对象状态	`add()`、`remove()`、`clear()`

设计 ADT 的准则

1. **操作应当少而正交**：每个操作做一件事，组合覆盖所有需要的行为。
2. **不要暴露内部表示**（representation exposure）。
3. **优先选择不可变 ADT**；可变 ADT 只在必要时使用。
4. **用 `private` 字段 + `public` 方法**实现封装。
5. **在规格说明中写明所有操作的前置/后置条件**，让使用者不需要看实现。

Java 中的 ADT 实现方式

方式	适用场景
类 + `private` 字段	通用；具体类型
接口 + 实现类	需要多种实现或多态时（`List` / `ArrayList`）
泛型 `class Bag<E>`	需要容纳任意元素类型
`enum`	值域小而有限的类型（`enum Suit { HEARTS, SPADES }`）
抽象类	多个实现共享部分代码时

```
// 接口与实现分离
public interface Shape { double area(); }
public final class Circle implements Shape {
    private final double r;
    public Circle(double r) { this.r = r; }
    @Override public double area() { return Math.PI * r * r; }
}
// 使用者只依赖 Shape 接口，可以自由更换实现（RFC）
```

3.7 抽象函数与表示不变量（Reading 11）★★最核心

两个必须写在代码里的注释

```
public class Tweet {
    private final String author;
    private final String text;
    private final List<String> hashtags;

    // Abstraction function:
    //   AF(t) = a tweet posted by t.author, with text t.text,
    //           and hashtags t.hashtags
    // Representation invariant:
    //   author != null && text != null && hashtags != null
    //   && no element of hashtags is null
    //   && no element of hashtags is the empty string
    // Safety from rep exposure:
    //   All fields are private and final.
    //   String is immutable, so returning it directly is safe.
    //   hashtags is copied in the constructor and its getter returns
    //   an unmodifiable copy, so no internal alias escapes.
}
```

概念	定义	要点
抽象函数 AF	从内部表示 R 到抽象值 A 的映射：`AF: R → A`	回答"内部状态代表什么"；**可以是多对一**（非单射）
表示不变量 RI	内部表示必须始终满足的条件：`RI: R → boolean`	回答"哪些内部状态是合法的"
表示暴露	内部表示逃逸到 ADT 之外	破坏封装，使 RI 无法被保证；用防御性拷贝防止
有益的可变性	对使用者不可见、且保持 AF 不变的内部修改	如缓存（memoization）；对观察者而言对象仍是"不可变的"

AF 与 RI 的三条铁律

1. **所有构造函数必须建立 RI，所有 mutator 必须保持 RI。**
2. **所有 observer 和 producer 可以假设 RI 成立**（只要前置条件满足）。

3. **AF 和 RI 必须写在代码注释里**，否则它们只存在于实现者脑中，ETU 直接受损。

`checkRep()` 模板

```
private void checkRep() {
    assert author != null;
    assert text != null;
    assert hashtags != null;
    for (String h : hashtags) {
        assert h != null && !h.isEmpty();
    }
}
// 在每个构造函数的末尾、每个 mutator 的末尾调用 checkRep()
```

> `assert` 默认在 JVM 中关闭，需要在运行参数中显式启用 `-ea`。这正是 6.031 提倡的"fail fast"。

AF/RI 的其他用途 – **定义 `equals()`**：两个对象相等 ⟺ 它们的抽象值相等（而非内部表示相同）。 – **定义 `toString()`**：应当输出**抽象值**，而不是内部字段的机械拼接。 – **证明方法正确**：写代码前先问"这个操作会破坏 RI 吗？"

配方（Recipes for programming） 1. 写规格说明（前置/后置条件）。 2. 选择内部表示，写下 AF 和 RI。 3. 实现构造函数、观察者、生产者、修改者。 4. 在每个方法结尾调用 `checkRep()`。 5. 写测试。

3.8 相等性（Reading 15）★核心

等价关系三性质：自反（reflexive）、对称（symmetric）、传递（transitive）。任何 `equals()` 实现都必须满足这三条，否则 `Set` / `Map` 的行为会出错。

概念	含义	适用
引用相等	`a == b`：指向同一个对象	始终可用
值相等 / 观察相等（observational equality）	两个对象的所有观察结果都相同	不可变类型
行为相等（behavioral equality）	两个对象在未来的所有操作下表现一致	可变类型（几乎不可能实现，一般不用）

规则 – **不可变类型**：应当实现 `equals()`，用**抽象值**定义相等。 – **可变类型**：应当使用**引用相等**（不覆写 `equals()`），因为观察相等会随时间失效。

`equals()` 实现模板

```
@Override
public boolean equals(Object obj) {
    if (this == obj) return true;                    // 快速路径 + 自反
    if (!(obj instanceof Duration)) return false;    // 类型检查（null 会返回 false）
    Duration other = (Duration) obj;
    return this.minutes == other.minutes;            // 比较所有决定抽象值的字段
}

@Override
public int hashCode() {
    // 契约：equals 相等 ⇒ hashCode 必须相等
    return Objects.hash(minutes);
}
```

`hashCode()` 契约 1. `a.equals(b)` ⟹ `a.hashCode() == b.hashCode()`（**必须**） 2. `a.hashCode() == b.hashCode()` 不要求 `a.equals(b)`（哈希碰撞是允许的） 3. 同一次程序运行中，只要对象状态未变，`hashCode()` 必须稳定不变。

最重要的陷阱：**`hashCode()` 绝不能基于可变字段**。如果对象被放进 `HashSet` 后字段被修改，`hashCode` 变化，对象就"丢失"在错误的哈希桶里，再也查不到了。

3.9 递归与递归数据类型（Reading 14、16、17）

递归实现的结构

```
// 1. 基础情形 base case
// 2. 递归步骤 recursive step：把问题分解为更小的同类问题
/** @return the subsequences of s, each as a String, in an unspecified order */
public static List<String> subsequences(String s) {
    List<String> result = new ArrayList<>();
    if (s.isEmpty()) {                 // base case
        result.add("");
        return result;
    }
    char first = s.charAt(0);
    String rest = s.substring(1);
    for (String sub : subsequences(rest)) {   // recursive step
        result.add(sub);
        result.add(first + sub);
    }
    return result;
}
```

递归 vs 迭代：递归在**递归定义的数据结构**（链表、树、表达式）上更自然、更易证明正确；迭代在需要极致性能、或深度可能很大（栈溢出）时更好用。

递归数据类型 `ImList<E>`（不可变列表）

```
public interface ImList<E> {
    static <E> ImList<E> empty() { ... }        // creator
    ImList<E> cons(E e);                        // producer
    E first();                                  // observer（前置条件：非空）
    ImList<E> rest();                           // producer（前置条件：非空）
    boolean isEmpty();                          // observer
    int size();                                 // observer
}
```

> 关键点：`cons` / `rest` 是 **producer**（返回新列表，不修改原列表），因此 `ImList` 是**不可变**的。递归数据类型的 AF 天然是"递归的"：`AF(empty) = 空序列`；`AF(cons(e, l)) = [e] + AF(l)`；`RI(cons(e, l)) = e != null && RI(l)`。

函数式三件套（Reading 16）

操作	签名	含义
`map`	`(A → B) × List<A> → List<B>`	对每个元素应用函数，长度不变
`filter`	`(A → boolean) × List<A> → List<A>`	保留满足谓词的元素
`reduce`	`(B × A → B) × B × List<A> → B`	从左到右累积，把列表折叠成单个值

```
List<String> names = people.stream()
        .filter(p -> p.age() >= 18)      // 过滤
        .map(Person::name)               // 变换
        .collect(Collectors.toList());   // 收集
int total = numbers.stream().reduce(0, Integer::sum);  // 归约
```

3.10 文法、解析与小语言（Reading 18–19、26–27）

文法（grammar）四要素

要素	含义	示例
终结符 terminal	不能再展开的符号	`"("`, `"+"`, `"3"`
非终结符 nonterminal	可继续展开的符号	`<expr>`, `<term>`, `<number>`
产生式 production	展开规则	`<expr> ::= <term> "+" <expr>`
根非终结符 root	起始符号	`<expr>`

正则表达式运算符：连接、重复 `*` `+` `?`、选择 `|`、字符类 `[...]`、分组 `(...)`。

> 正则表达式**无法**描述递归嵌套结构（如配对的括号），需要文法。

解析器（parser）流水线

```
字符序列 --(文法/解析器生成器)--> 解析树 parse tree --(遍历)--> 抽象语法树 AST --> 解释器求值
```

Java 中用 `Pattern` / `Matcher`

```
Pattern p = Pattern.compile("(\\d+)-(\\d+)");
Matcher m = p.matcher("2024-05");
if (m.matches()) {
    int year = Integer.parseInt(m.group(1));
}
```

小语言（little language / DSL）的价值：为一个特定领域设计专门的、小而精确的语言，比用通用语言硬编码更 **ETU**（表达意图更清晰）也更 **RFC**（新增能力只需扩文法）。

访问者模式（visitor pattern，Reading 27）

```
interface Visitor<R> { R visitPlus(Plus e); R visitInt(Int e); }
interface Expr { <R> R accept(Visitor<R> v); }   // 双重分派 double dispatch
```

优点	代价
新增**操作**只需加一个 Visitor 类，不必改数据类型	新增**数据变体**必须修改所有 Visitor（expression problem）

3.11 并发（Reading 21–24）★核心

两大模型

模型	机制	优点	风险
共享内存 shared memory	多个线程读写同一块内存	直接、高效	**竞态条件**、死锁
消息传递 message passing	并发单元之间通过队列传消息，不共享内存	天然避免竞态	需要设计协议；可能死锁、可能阻塞

基本概念：进程 process（独立地址空间）、线程 thread（共享地址空间）、时间分片 time slicing、交错 interleaving、竞态条件 race condition。

竞态条件的本质

```
// ❌ 错误：counter++ 不是原子操作
public void increment() { counter++; }   // 实际是 read-modify-write 三步
```

两个线程可能都读到 `counter = 5`，都写回 `6`，结果丢失一次自增。

并发安全的四层策略（按优先级）

1. **不要共享可变状态**：优先使用不可变对象（R8、R11）。不可变对象天然线程安全。

2. **限制可变状态的作用域**：把可变状态封装在单个线程内。
3. **用同步机制保护共享可变状态**：
4. **互斥锁** `mutex` / **临界区** `critical section`（R23）；
5. **消息传递 + 阻塞队列**（R24）。
6. **用不可变快照通信**：线程之间传递不可变消息，而不是共享可变对象。

Java 同步写法（Reading 23）

```
public class BankAccount {
    private int balance;   // guarded by this

    public synchronized void deposit(int amount) {
        balance += amount;
        checkRep();
    }
    public synchronized int getBalance() { return balance; }
    private void checkRep() { assert balance >= 0; }
}
```

> 规则：**每个被锁保护的字段都必须在注释中声明它由哪把锁保护**（"guarded by"）。

死锁（deadlock）的成因与预防 死锁发生的四个必要条件：互斥、持有并等待、不可抢占、循环等待。 **最实用的预防手段是打破"循环等待"——全局统一的锁获取顺序。**

```
// ❌ 错误：两个线程以不同顺序获取同一对锁 → 死锁
// 线程 A：lock(x); lock(y);
// 线程 B：lock(y); lock(x);

// ✅ 正确：用唯一的全局顺序（例如按对象身份哈希）决定加锁顺序
private static void lockInOrder(Object a, Object b) { ... }  // 先锁"小"的
```

消息传递写法（Reading 24）

```
// 阻塞队列作为同步机制：生产者-消费者模式
private final BlockingQueue<Message> queue = new LinkedBlockingQueue<>();
queue.put(msg);        // 队列满时阻塞
Message m = queue.take();  // 队列空时阻塞
```

> 消息传递天然避免了共享内存的竞态，是"用通信代替共享"（Do not communicate by sharing memory; share memory by communicating）的体现。

并发黄金法则 1. 能不用并发就不用；能用不可变就不用锁。 2. 每个共享可变字段都必须有明确的保护策略。 3. 不要依赖 `sleep()` 来"修"竞态——那是掩盖而非修复。 4. 并发 bug 无法靠测试可靠发现（调度具有偶然性）：应靠**设计**与**推理**。

3.12 网络（Reading 25）

概念	要点
客户端/服务器模式	服务器监听端口、等待连接；客户端主动发起连接
地址与端口	IP 定位主机，端口（0–65535）定位主机上的服务
TCP	可靠的、双向的**字节流**；不保留消息边界
Socket	通信端点；`ServerSocket` 负责 `accept()`，`Socket` 负责读写
协议	客户端与服务器交换的字节序列的格式约定（如 HTTP）
阻塞式 I/O	`read()` 在无数据时阻塞；`accept()` 在无连接时阻塞

```
// 服务器骨架
try (ServerSocket server = new ServerSocket(PORT)) {
    while (true) {
        Socket sock = server.accept();          // 阻塞直到有连接
        new Thread(() -> handle(sock)).start(); // 每个连接一个线程
    }
}
```

> **关键设计原则**：把"网络读写"抽象成 ADT 的接口（如 `MessageQueue`），业务逻辑就不必关心字节流细节——这正是 R10 表示独立性在网络层的应用，也让程序在测试时可以用假实现替换真实网络（stub）。

3.13 代码质量与调试（Reading 4、9、13）

代码审查清单

类别	检查点
Bug	潜在 bug、off-by-one、边界条件、异常的吞掉
重复	DRY：重复代码意味着修改时会漏改
一致性	代码与规格说明是否一致
防御性	是否 fail fast、是否检查前置条件
作用域	全局变量、过大的变量作用域、一变量多用
常量	魔法数字应命名为具名常量
命名	名字要表达意图；避免 `data`、`temp`、`flag`
排版	一致的缩进、空白帮助阅读、不要一行塞太多

类别	检查点
注释	解释"为什么"，不复述"是什么"
设计	是否误用/未用 ADT、规格说明、不变量等概念

避免调试的四道防线（Reading 9，按优先级）

防线	手段
1. 让 bug **不可能**发生	静态类型、不可变对象、`final`、把非法状态设计成不可表示
2. 让 bug **显而易见**（localize）	模块化、封装、缩小变量作用域、fail fast
3. 让 bug **尽早失败**	`assert`、`checkRep()`、前置条件检查
4. **彻底测试**	系统性测试、回归测试

系统化调试：科学方法（Reading 13） 1. **重现 bug**（找到最小可重现输入，可用 delta debugging / slicing 缩小范围） 2. **观察数据**：实际值是什么？ 3. **提出假设**：哪个环节出错了？为什么？ 4. **做实验验证假设**（用 `assert`、打印、断点） 5. **重复**，直到定位 6. **修复**，并**添加回归测试**

> 反模式：靠猜、随机改代码、"试试加上这个看看行不行"。调试的目标是**理解**，不是让程序暂时不报错。

3.14 版本控制与团队协作（Reading 5、29）

概念	要点
仓库 / 工作副本	repository（对象图）vs working copy（你的文件系统）
提交 commit	一次快照；含作者、时间、信息、指向父提交的指针
HEAD	当前所在提交
暂存区 staging area	`git add` 后、`git commit` 前的中间区域
分支 / 合并	提交图上的分支引用；`merge` 产生合并提交
合并冲突	同一处被两边修改；需手工解决后 `git add` + `git commit`

团队工作流（推荐）

```
git switch -c feature/parser       # 1. 从主干切出功能分支
# ... 小步提交，提交信息写清"为什么" ...
git push -u origin feature/parser  # 2. 推送到远端
# 3. 发起 Pull Request，请同伴代码审查（Reading 4）
# 4. 审查通过后合并回主干，删除分支
git switch master && git pull && git merge feature/parser
```

反模式清单 – 直接在 `master` 上开发并提交半成品。 – 一次提交改动上千行（无法审查）。 – 提交信息写 "fix"、"update"、"asdf"。 – 提交编译产物、IDE 配置、**密钥**（应写入 `.gitignore`）。 – 用 `git push --force` 覆盖他人提交。

3.15 软件工程伦理（Reading 28）

四种道德透镜

透镜	核心问题
后果主义 consequentialism	这个系统的结果对最大多数人是好是坏?
义务论 deontology	是否违反了不可逾越的责任与规则（如不欺骗、不伤害）?
美德伦理 virtue ethics	一个正直、诚实的工程师会怎么做?
社会契约 social contract	是否违背了公众对专业人士的信任?

实践检查点 – 隐私：是否收集了超出必要范围的数据? 是否默认安全? – 偏见：训练数据/规则是否对某些群体不公? – 透明度：用户是否知道系统在做什么? – 安全关键：失效模式是否会伤害人? 是否有降级方案? – 举报（whistleblowing）：内部渠道失败时，工程师的责任边界在哪?

> MIT 6.031 把伦理放在最后一讲，意在说明：**SFB / ETU / RFC 是技术标准，而"对谁负责"是工程标准。二者缺一不可。**

3.16 设计模式速查（全课程出现的模式）

模式	用途	出现在
ADT / 封装	分离使用与实现	R10–R12
迭代器 Iterator	顺序访问而不暴露内部表示	R8、R12
工厂方法 Factory	把创建逻辑与使用解耦（creator）	R10
观察者 / 监听器 Listener	事件源通知多个订阅者	R20
回调 Callback	把控制流的"下一步"作为参数传入	R16、R20
不可变对象 Immutable	免于别名 bug、天然线程安全	R8、R21
组合模式 Composite	用统一接口处理"单个"与"组合"	R26
解释器 Interpreter	递归数据类型 + 求值函数 = 小语言	R26
访问者 Visitor	不改数据类型就增加新操作	R27
监视器 Monitor	用锁把 ADT 变成线程安全的	R23

模式	用途	出现在
生产者–消费者	用阻塞队列解耦生产与消费速度	R24
客户端/服务器	通过网络通信的两个角色	R25

3.17 一页纸复习：20 条黄金法则

1. **先写规格，再写实现。** 说不清"做什么"就不该开始写"怎么做"。
2. **前置条件是调用者的义务，后置条件是实现者的义务。** 不要搞反。
3. **规格要声明式，不要操作式。**
4. **允许实现自由度**：除非调用者真的依赖，否则不要规定唯一答案。
5. **把 `null` 赶出你的接口。**
6. **优先不可变；可变对象必须小心别名。**
7. **可变对象进出 ADT 都要防御性拷贝。**
8. **`final` 只防重新赋值，不防内部可变。**
9. **AF 和 RI 必须写成代码注释**，否则它们不存在。
10. **所有构造函数建立 RI，所有 mutator 保持 RI，所有 observer 假设 RI 成立。**
11. **`checkRep()` 放在每个构造函数和 mutator 的末尾。**
12. **`equals` 相等 ⟹ `hashCode` 相等**；`hashCode` 绝不能用可变字段。
13. **可变类型用引用相等，不可变类型用值相等。**
14. **测试先写，测试要能失败，边界值优先。**
15. **fail fast**：错误越早暴露越便宜。
16. **DRY**：重复的代码是未来 bug 的温床。
17. **避免共享可变状态；做不到就用锁或消息传递保护它。**
18. **统一锁顺序以避免死锁。**
19. **并发正确性靠设计保证，不能靠测试碰运气。**
20. **代码是写给人读的**——包括未来的你。